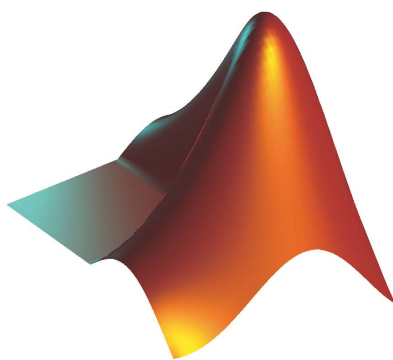


我的MATLAB报告

王保山 编著



2021 年 6 月 16 日

内容简介

本书是在编者大学时期的MATLAB数学报告的基础上整理润色而成的。全书共分为五个部分，第一部分(第一至第七章)主要介绍MATLAB的基础知识、程序设计、可视化绘图、矩阵代数、多项式与方程、符号计算以及数据读入和导出等MATLAB入门的基础知识，第二部分(第八至第十二章)主要介绍数值代数、数值逼近、数值积分以及微分方程数值解法等数值计算方法，第三部分(第十三和十四章)主要介绍统计学数据分析和关于气候数据的分析报告，第四部分(第十五至第二十一章)主要介绍古典密码、图与网络、动态规划、网站排名、遗传算法、分形以及混沌等数学实验，第五部分(第二十二至第二十四章)主要介绍图形界面的设计和一些有趣的程序。

本书可作为一般院校数学类专业的实验参考书，也可以作为学习MATLAB的工具书。

作者简介

王保山，男，山东菏泽人。2011年9月考入中国海洋大学数学科学学院数学与应用数学专业，2015年7月本科毕业并获得理学学士学位，同年9月考入中国海洋大学数学科学学院计算数学专业，2018年6月硕士毕业并获得理学硕士学位，现为中国海洋大学计算数学专业在读博士研究生，主要从事计算流体力学高精度数值方法的研究。自2015年9月至2017年1月，担任数学科学学院《数学实验基础》和《数学实验》等课程助教。五年来，面向计算数学专业的研究生新生讲授编程技术等科研基础。曾获得数学建模竞赛国家二等奖两次、一等奖一次，其中2016年的竞赛论文被评为赛事优秀论文，曾获得2017年度硕士研究生国家奖学金，山东省2019年度优秀硕士学位论文奖。

通讯地址：山东省青岛市崂山区松岭路238号，中国海洋大学数学科学学院

联系邮箱：wbs@stu.ouc.edu.cn

前言

写书是为了什么？

每个人都有着不同的目的，古人或是为了传承精神、技艺而著书立说，或是无所事事，就创几部作品来，譬如周文王等一些人物。有人说，写书是老人们的事，他们在无聊之余写本书来打发时间而已。

而我为什么要写这本书？

我写书的目的，不是想澄清史实，而是想讲述自己与MATLAB的故事。

2011年夏天，我顺利考入中国海洋大学数学系。第一学期，我开始接触MATLAB。当时是陈丕炜老师给我们上MATLAB数学实验课。在陈老师的教导下，我基本上掌握了MATLAB的基础知识，并做了一些有趣的实验。之后，我又学习了施心慧老师的数学实验课和曹圣山老师的数值分析课，旁听了数值代数等课程，同时自学了罗华飞老师的《MATLAB GUI设计学习手记》第一版和第二版以及图书馆里与MATLAB相关的书籍。后来只要有时间，我便去机房蹭课，经过两年的上机实践，我积累了一些MATLAB的编程经验。学习的同时，我乐意做一名MATLAB的实践者和传播者。

当时专业课不怎么样、六级还没过的我打算写书，显然不是闲的无聊，更不是脑子进水，而是我想写本书看看哪些地方我能讲清楚，哪些地方还不能讲明白。能讲清楚的地方说明自己已经掌握，不明白的地方还需花费功夫进修提升。写书是检验自己学习成果的好方法。因为写书要严谨，每个地方都要考虑周到，自然不能像我们的期末考试那样注入水分。写书还要对读者负责，不可把热心的读者引入歧途。在读者阅读之余能够收获知识，自然是极为高兴的事。

写这本书还有另外的目的。如果一个人在大学时代没有什么想法，那么，这个人这辈子就完了。大学四年的美好时光转瞬即逝，而我们又浑浑噩噩，碌碌无为。等到我们离开时再去后悔，那该是多么的无奈啊。我是处于边缘的小人物，不曾在众人面前有过言语，与同学们交往不深，做事情总是三分钟热度，三分钟之后便懈怠下来，以致一事无成。大学三年级的我决定写下这本书，虽然这本书是由我的实验报告整理而成，但增添删减也花费了我好几年的时间（编写至今，我已成为计算数学专业的三年级博士研究生），希望自己坚持写下的这本书能对大家有所帮助。

其实，一开始写这本书并不是为大家提供指导，而是为了给自己留一个笔记，方便日后学习领会，并继续探索程序设计这一领域。我喜欢编程，刚好你也喜欢，就像小朋友相互切磋吃哪块饼干更好吃一样，我和你们一样是一个学习者。我没有把编程看作一个谋生的技能，而是一个兴趣爱好。这本书之所以被写下，是因为其中有我认为值得分享和记录的技巧点，这些技巧点有软件的操作技巧，也有对于程序设计本身的思考。我想，它们能点醒我，大概也能点醒某一部分人。如果写得不好，那是我

能力的问题，但并不妨碍对程序设计本身的热爱和深究。我是比较喜欢动手的那类人。我喜欢编程，喜欢用程序分析数据，书中的内容既是基于自己的兴趣，也有从实际问题引出的，这些问题是我整个大学时期的MATLAB数学实验报告中所做过的。书中的例子简单且十分有趣，希望学完这本书能点燃你对书中内容的兴趣和热情。

读者会发现本书与其他教科书有很大不同，这是作者有意为之。市面上已经有很多深入探讨MATLAB软件及其应用的图书，但是它们大多偏重与MATLAB的应用，而关于模型设计和算法实现的图书则很少，因此，将MATLAB与模型设计和算法实现结合起来是本书的主要目标。事实上，程序是我们研究的各种算法的具体体现，算法是比较抽象的，而程序则可以一句一句读懂。因此，程序在本书中占据了很大的篇幅，为了理解方便，程序中均在难点处添加了注释。市面上已经有很多书籍对本书中的问题进行了探讨并给出了解决方案，在这一方面本书可以说受到了它们的影响。然而尽管本书提供了很多程序来展示如何使用函数去解决某些特定的问题，但它并非一本函数参考手册，想要深入了解所有的MATLAB内部函数，可以通过相应的帮助机制获得详细的描述文档。

本书对于编程的强调并不意味着基础理论不重要。相反，它们非常重要。学习本书的同时最好参考一下其他基础理论上论述更严密的材料。

书中的程序均是在MATLAB R2015a平台上编写执行的，将它们应用到其他版本时，请注意不同版本之间的差异。

借用《小别离》中的话（有改动）。我其实习惯用代码来表达我自己的情感。不少朋友对本书的持续关注，让我明白，其实我在写书的时候，不仅仅是在电脑前敲击着键盘，抒发着自己的情感，而是让我的生活变得更加丰富多彩。非常感谢数学院的老师们，特别是我的导师Wai Sun Don教授，正是他们教给了我数学知识和MATLAB的编程技术，把我带进了MATLAB的美妙世界。还有我的同学们和朋友们，他们给了我很多鼓励帮助，在这里表示衷心地感谢。

由于我接触MATLAB的时间不长，学识有限，书中的问题一定不少，敬请大家批评指正。

编者

2021年6月16日

写于S606室

编程的艺术

其实早就想写写这个话题，当今学校里讲编程的课，或多或少偏重编程平台的语法和功能的实现，也是某些自觉苦逼的程序猿们每天在干的事情。

我认为一个好的程序应该包括功能性和结构性两个部分，并且结构性更重要，所谓的结构性就像是树干，它描述了程序从哪儿来到哪儿去，而怎么去则由功能性部分实现，即

程序设计 = 结构性设计 + 功能性实现

举个例子，将大象装进冰箱，打开冰箱、装进大象、关上冰箱是结构性的部分，至于如何打开冰箱、如何装大象、如何关上冰箱则是功能性的部分。整个程序可以写为

```
function 将大象装进冰箱

    分配内存;
    打开冰箱;
    装进大象;
    关闭冰箱;

    %=====
    function 分配内存

        大象一只;
        冰箱一个;
        人员若干;
        工具若干;

    end
    %=====
    function 打开冰箱

        打开冰箱的若干步骤;

    end
    %=====
    function 装进大象

        装进大象的若干步骤;

    end
    %=====
    function 关闭冰箱
```

关闭冰箱的若干步骤；

```
end
%=====
end
```

交通导航越来越普遍，而我也越来越离不开导航，越来越不认识路。可那又怎样，只要知道目的地，沿着正确的导航走，是可以到达目的地的。程序也一样，无论是写程序、读程序，请先找好路再走，先自上而下把结构性部分写好读懂，搭好框架，然后再一点点自下而上实现程序的功能。

设计程序的时候，多想想我在哪儿，我要干什么，我需要什么东西，我要怎么干，我要干成什么样。我也不清楚自己写了多少程序，一写起来就过了很多年，最后在导师的指导下，选择了这种编程风格。头一开始，写成模块（参数输入一个模块，内存分配一个模块，Index设置一个模块，网格剖分一个模块，初始条件一个模块，边界条件一个模块，输出结果一个模块，空间离散一个模块，时间离散一个模块）花费了我很多时间，但也避免了更多的改bug的时间。

实现论文中的算法，包含两部分，一部分是论文算法的理解，一部分是程序实现。我建议把论文中的公式按先后顺序，列一列，先弄清楚先后顺序，比如某个公式需要哪些变量，这些变量又需要通过另一个公式计算。弄清楚思路再敲代码，磨刀不误砍柴工嘛。

思路不清晰的话，写写删删的真的很磨人，效率不高而且错误频出。没有想好程序思路，就如同没有清晰的人生规划，总会在某个阶段深夜难眠，迷茫不前，大好青春白白地浪费在犹犹豫豫之中。

别人是一按下回车就能知道程序的结果，我们是一按下回车就知道程序一定不对。我觉得是程序思路不清晰和编程技巧不足的缘故。结构性的程序思路很好解决，只要先后逻辑通畅，反而功能性部分涉及编程人员的技术和经验，这部分的提升没有好办法，请选择多多练习，积累编程经验。

多读论文（精读重要论文，追踪科研进展）、多写程序（真的要自己写，验证自己的科研想法）、多整理论文（锻炼写作能力，以便与同仁分享科研成果），这三个是评价我们是否是计算数学一线科学家的标准，我觉得这三样缺一个都不能称为一线科研人员。

而且程序的编写特别重要。咱们做计算数学的，会用到矩阵和差分，写这些程序要特别注意矩阵的尺寸。程序经常是这样的错误。敲这部分的代码，需要想想，我们是在那个点上计算差分的。

想写程序不用计划何时开始，如果思路想通了，那就写吧，浮生冷暖都已在键盘之上了。

编者

2021年6月16日

写于S606室

导师的经验分享

写完上一篇之后，导师回复并补充了很多干货。我根据自己的见解翻译整理，也发布在这里，其中斜体英文是导师的原话。

Programming is very much similar to composing sounds. It could be a master piece of musical orchestra with structures and patterns of various musical instruments along with their known limitations or a pure annoying loud random noises most people found hideous.

编程与合音非常相似。它可能是一个管弦乐大师的作品，既包含了各种乐器的结构和模式，也包含他们已知的局限性。它或者是一个纯粹恼人的大音量的随机噪音，令大多数人都觉得很可怕。

的确是这样的，去年楼管大爷退休的时候，我得到一个笛子。作为吹笛子的手，我吹不响，即使吹响了也巨难听。想吹好笛子，就需要了解笛子的构造，控制口型、吹气角度，注意按孔和换气，同样也要对曲子本身有足够的了解。于编程而言，新手需要着重了解编程语言的规则，如变量或数组的加减乘除、条件语句、循环语句、控制语句、输入输出规则、自定义函数的规则等，熟悉掌握规则对于编程是十分必要的。

It requires a lot of self discipline and being a perfectionist.

这需要大量的自律并成为一个完美主义者。

学习编程的阶段有些逆人性，你越着急，bug越频出。我推荐用对齐的方式编写程序，用“对齐强迫症，细节显文明”形容太合适了，哈哈哈。我还有个习惯，就是，任何数值上的运算、逻辑上的判断都放到代码里。编程时，很多人们总是迷失在纷杂的数学公式中，而丢掉了最初的目的——我们是用计算机来运算的，而不是替计算机运算的。计算机的运算和判断要比我们准确快速得多。家里有驴，何用自己拉磨，对吧。

Generalization is very important if one wants the piece of code be useful in different scenarios. Specialization is important when one wants the code to do useful things.

如果你想让代码片段在不同的场景中使用，通用化是非常重要的。当你想让代码做有用的事情时，专门化是很重要的。

通用化和专门化是程序的两个定位，需要编程人员仔细思量，在适合通用化的地方做到通用，在需要专门化的地方做到专业。其实科学研究也是这样，一部分科研人员争论自己研究的内容足够通用，可应用的领域很多；一部分科研人员争论自己的研究足够专业，针对特定的研究领域给出了强有力的解决方案。

通用化和专门化，就和为人处世一样，在大是大非面前坚持原则，在其他情况下选择圆滑。细想起来，我们是在写程序么，我们这是在生活呀，朋友们！

Think of programming as building a thing with Lego pieces. Stacking different compatible pieces of Lego blocks (or generic subroutines or modules) together to make the whole (program) to do things you want it to do.

把编程想象成用乐高积木搭建东西将不同兼容的乐高块(或通用子程序和模块)堆叠在一起，使整个程序做你想让它做的事情。

好啦，你已经知道了编程的终极秘籍——搭积木！嗨起来吧！

等等，导师似乎还有话要说！

One more piece of advise: debugging a program is a lot worse than and takes much longer than one expected. 75%+ of time is spending on debugging a poorly written code than actual doing useful works with the program. A well designed program can reduce significantly the amounts of time one spent debugging. Each subroutine or module should be simplified and well debugged before use. A well designed data structures and function abstraction are the backbone of a well designed program.

还有一点建议：调试一个程序比我们预期的要糟糕得多，花费的时间也要长得多。75%以上的时间花在调试写得很差的代码上，而不是实际用程序做有用的工作。一个设计良好的程序可以大大减少花在调试上的时间。每个子程序或模块在使用之前都应该简化并调试好。一个设计良好的数据结构和功能抽象化是一个设计良好程序的支柱。

编者

2021 年 6 月 16 日

写于S606室

目 录

第一部分	入门基础篇	1	4.4	实验报告	76
第一章	MATLAB基础	3	第五章	多项式与方程	79
1.1	MATLAB简介	3	5.1	多项式	80
1.2	变量	4	5.2	零点与极值	81
1.3	数组	6	5.3	迭代法求方程的根	84
1.4	数组运算	7	5.4	实验报告	86
1.5	数据类型	10	第六章	符号计算	91
1.6	实验报告	12	6.1	符号对象	92
第二章	程序设计	17	6.2	符号运算	94
2.1	Editor与M文件	17	6.3	便捷函数绘图	99
2.2	控制语句与举例	20	6.4	实验报告	102
2.3	编程规范	23	第七章	I/O操作	109
2.4	实验报告	28	7.1	读写mat文件	110
第三章	可视化绘图	37	7.2	读写txt文件	111
3.1	绘制曲线	38	7.3	读写Excel文件	116
3.2	绘制曲面	42	7.4	读写图像文件	119
3.3	特殊图形的绘制	44	7.5	GIF动画制作	122
3.4	图形标注	49	7.6	实验报告	125
3.5	图形编辑器	56	第二部分	数值计算篇	127
3.6	实验报告	57	第八章	数值代数	129
第四章	矩阵代数	65	8.1	消去法	130
4.1	矩阵代数的指令	66	8.2	分解法	134
4.2	线性方程组求解	68	8.3	迭代法	137
4.3	实用模型	71	8.4	幂法与反幂法	143

8.5	实验报告	146	14.2	统计图示	255
第九章	数值逼近	157	14.3	一元线性回归分析	260
9.1	插值	158	14.4	多元线性回归分析	265
9.2	拟合	161	14.5	气候分类	266
9.3	回归	162	14.6	气候排名	271
9.4	实验报告	163	14.7	结论与展望	274
9.5	补充实验	170	第四部分	数学实验篇	277
第十章	数值积分	177	第十五章	古典密码	279
10.1	求积公式	178	15.1	加法密码	279
10.2	逐次分半法	179	15.2	维吉尼亚密码	282
10.3	Romberg求积法	182	15.3	Hill ₂ 密码	284
10.4	实验报告	183	第十六章	图与网络	287
第十一章	常微分方程数值解法	185	16.1	图论初步	287
11.1	Euler法及其改进	186	16.2	最小生成树算法	291
11.2	解常微分方程的指令	189	16.3	最短路径算法	293
11.3	导弹追击问题	192	16.4	图论算法汇总	296
11.4	实验报告	197	第十七章	动态规划	299
第十二章	偏微分方程数值解法	207	17.1	动态规划的概念与逆序算法	299
12.1	导数近似与误差阶验证	208	17.2	实验报告	301
12.2	差分方程求解与收敛阶验证	213	第十八章	网站排名	307
12.3	一维扩散方程的求解	215	18.1	问题背景与基础知识	307
12.4	二维热传导方程的求解	218	18.2	PageRank算法及其改进	310
第三部分	统计分析篇	223	18.3	问题与修正	315
第十三章	统计分析	225	18.4	实验报告	319
13.1	基础知识	225	第十九章	遗传算法	325
13.2	参数估计与假设检验	228	19.1	预备知识	325
13.3	统计描述与分析报告	231	19.2	算法程序	332
13.4	方差分析	238	19.3	实验报告	336
13.5	回归分析	243	第二十章	分形实验	339
第十四章	气候数据的分析报告	253	20.1	分形初步	339
14.1	数据与说明	254	20.2	实验报告	345

第二十一章混沌实验	353	22.5	UI对象	386
21.1 混沌初步	353	22.6	2048游戏设计	394
21.2 实验报告	357	第二十三章GUI设计		421
第五部分 GUI设计篇	365	23.1 GUIDE		421
第二十二章句柄图形对象	367	23.2 预定义对话框		423
22.1 Root对象	367	23.3 QQ登录框的设计		433
22.2 Figure对象	369	第二十四章有趣的程序		439
22.3 Axes对象	371	参考文献		447
22.4 核心对象	375	常用函数表		449

我的 MATLAB 报告

王保山

第一部分

入门基础篇

我的 MATLAB 报告

王保山

第一章 MATLAB基础

1.1 MATLAB简介

MATLAB是“matrix”和“laboratory”两个单词的组合，意为“矩阵实验室”。它是由美国MathWorks公司发布的，主要面对科学计算、可视化以及交互式程序设计的高科技计算环境。它与Mathematica、Maple并称为三大数学软件，主要应用于工程计算、控制设计、信号处理与通讯、图像处理、信号检测、金融建模设计与分析等领域。它，以矩阵为基本数据单位，指令表达式与数学、工程中常用的形式十分相似，在很大程度上摆脱了传统非交互式程序设计语言(如C、FORTRAN)的编辑模式，代表了当今国际科学计算软件的先进水平。

在启动MATLAB后，我们就进入了MATLAB的桌面(Desktop)。

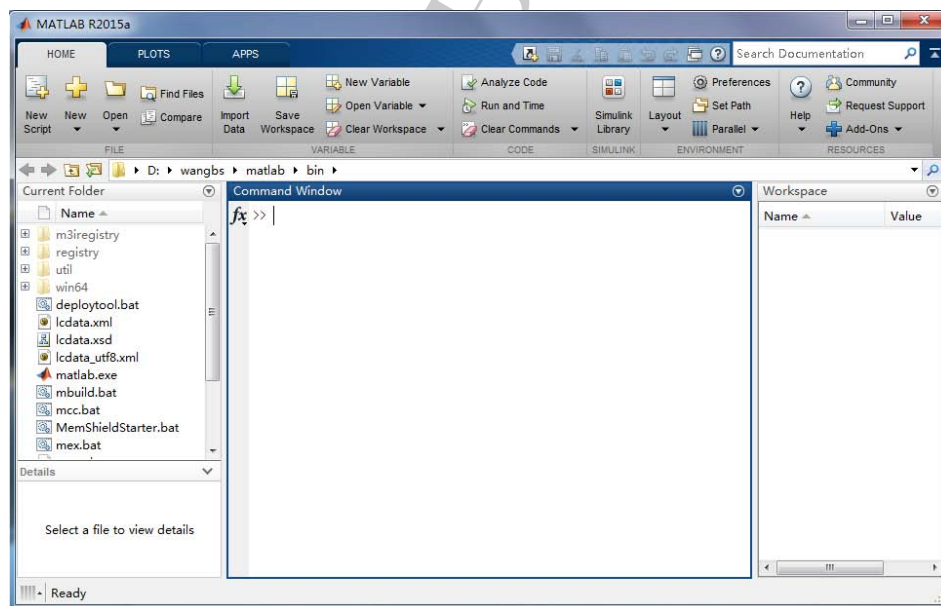


图 1.1: MATLAB桌面

如图1.1所示，MATLAB桌面从上到下，从左到右，依次是标题栏、菜单栏、工具栏、当前目录(Current Folder)、指令窗口(Command Window)、工作空间(Workspace)。此外常用的窗口还有程

序编辑器(Editor)、指令历史(Command History)。他们的具体功能详见表1.1。

表 1.1: MATLAB的主要窗口及其功能

主要窗口	窗口功能说明
当前目录	窗口列出了当前目录下的程序文件(.m文件)和数据文件(.mat)等, 可进行打开(Open)、运行(Run)、重命名(Rename)、删除>Delete)、剪切(Cut)、复制(Copy)、粘贴(Paste)、显示帮助(View Help)、创建压缩文件(Create Zip File)操作
程序编辑器	编写程序的编辑器, 详见第二章
指令窗口	输入运算指令, 按回车(Enter)键即可执行; “>>” 是指示输入的指令, 本身不是命令字符; 可用上键(↑)、下键(↓)调用上一、下一指令
工作空间	窗口列出工作空间的所有变量的变量名(Name)、值(Value)、尺寸(Size)、字节数(Bytes)和类型(Class), 可进行打开(Open Selection)、保存(Save As)、清除>Delete)、修改>Edit Value)等操作
指令历史	窗口列出指令窗口执行过的指令, 可进行复制(Copy)、执行(Evaluate Selection)、删除>Delete Selection)等操作

MATLAB的其他常用窗口, 例如, 图形窗口(Figure)、GUIDE编辑窗口等, 在后面章节中详细展开, 这里不作介绍。工具栏和菜单栏常用的功能有新建(New)、打开、剪切、复制、粘贴、全选>Select All)、删除、查找(Find)、清除(Clear) 等, 类同于Office软件, 至于具体的功能, 读者可在编程实践中积累。

1.2 变量

变量, 顾名思义, 就是变化的量。在MATLAB中, 变量名总是以字母开头, 由字母、数字或下画线组成, 区分大小写, 默认的有效字符长度为63个。不像C、C++、Java等语言, MATLAB的变量不必事先声明, 当新变量名出现时, MATLAB会自动声明并分配内存。

变量赋值的格式为

```
变量名 = 值;
```

例 1.1 变量的赋值

```
>> a = 1; A = 2; b = a + 1, c = A * a, c + b
b =
    2
c =
    2
```

```
ans =
4
```

注 1.1 若表达式的结果没有赋予任何变量，系统会自动使用`ans`(`answer`的缩写)来存放答案。句中的“*”号和“+”号分别为乘法和加法运算，“;”号用于禁止输出表达式结果，而“,”号则允许输出表达式结果。

为了计算方便，MATLAB在启动时就对一些变量进行了赋值，这些变量被称为预定义变量，如表 1.2 所示，`i`, `pi`, `Inf` 等，都是预定义变量，可以直接使用。

表 1.2: 预定义变量

变量名	变量说明	预定义变量的值
<code>i, j</code>	虚数单位	$i = \sqrt{-1}$
<code>pi</code>	圆周率	$\pi = 3.1415926\cdots$
<code>eps</code>	浮点数识别精度	$\text{eps} = 2^{-52} \approx 2.2204 \times 10^{-16}$
<code>realmin</code>	最小正实数	$\text{realmin} = 2^{2-2^{10}} \approx 2.2251 \times 10^{-308}$
<code>realmax</code>	最大正实数	$\text{realmax} = 2^{2^{10}} \approx 1.7977 \times 10^{308}$
<code>Inf</code>	无穷大	
<code>NaN</code>	无意义的数	

例 1.2 预定义变量的使用

```
>> a = 1 + i; b = 2 - 3 * i; a + b, a * b
ans =
3.0000 - 2.0000i
ans =
5.0000 - 1.0000i
```

注 1.2 预定义变量不会出现在工作空间中，若被用户重新赋值，将暂时失去原来的值和功能，除非他们被清除(`clear`)或重启MATLAB后才能恢复。

变量的定义总是以字母开头，由字母、数字或下划线组成。命名变量时应避免与系统的预定义变量(表 1.2)、内部函数(`max`, `mean`, `length`等)、保留字(`for`, `if`, `end`等)的名称发生冲突。

例 1.3 变量冲突

```
>> i = 1; a = i + 1
a =
    2
```

此时，虚数单位i失效，只有clear后才能恢复。与变量信息有关的函数见表 1.3。

表 1.3: 与变量信息有关的函数

函数名	功能说明	备注
format	设置数据显示格式	short(默认格式，显示到小数点后4位)，long(长格式，显示到小数点后16位)，rational(有理格式，显示近似分数)
who	显示变量名	查看工作空间中的所有变量名
whos	显示变量信息	显示工作空间中的所有变量信息
clear	清除变量	清空命令窗口用clc
save	保存变量	
load	导入变量	

1.3 数组

MATLAB的基本数据单元就是数组，而且数组不必指定维数。一维数组被称为向量，二维数组被称为矩阵。矩阵在MATLAB中比较常用，向量可看成矩阵的特例，高维数组可以理解成每一页相当于一个矩阵。在输入时需用“[]”封装数组，按行输入，同行元素可用空格或逗号分隔，不同行元素用分号或回车分隔。注意数组元素的编址一律从1开始，而不是从0开始。

例 1.4 数组

```
>> a = [1 2 2; 2 3 4; 4 5 6], b = a(3,2) % 取a的第3行第2列元素
a =
     1     2     2
     2     3     4
     4     5     6
b =
     5
```

注 1.3 在MATLAB中，“%”用来表明该行“%”后面的内容为注释内容，注释内容将不会被程序执行。

对于构造等差数列(一维数组)，MATLAB提供了便捷有效的冒号运算和linspace函数。

例 1.5 等差数列的生成

```

>> a = 0:2:9    % 初值:增量:终值
a =
    0     2     4     6     8
>> a = 0:6      % 增量为1时可省略
a =
    0     1     2     3     4     5     6
>> a = 10:-1:3  % 增量为负的情况
a =
   10     9     8     7     6     5     4     3
>> a = linspace(1, 6, 3) % 将[1, 6]区间等分为2份, 取3个端点值
a =
    1.0000    3.5000    6.0000

```

1.4 数组运算

数组基本信息的操作函数如表 1.4 所示, 利用这些函数, 我们能够更方便地掌握数组的信息。

表 1.4: 与数组操作有关的函数

函数名	功能说明	函数名	功能说明
length	查询向量长度	size	查询数组尺寸
min	按列求数组最小值	max	按列求数组最大值
sum	向量或矩阵(按列)求和	prod	按列求积
reshape	数组重排	find	取“真”的编址
all	检验是否全为“真”	any	检验是否存在“真”

例 1.6 数组操作

```

>> a = 1:9; b = [1 3 2; 4 1 5];
>> c = length(a), d = size(b), e = size(b,1), f = max(b)
c =
    9
d =
    2     3
e =
    2
f =
    4     3     5

```

数组运算可通过运算符实现, 这些运算符可分为算术、关系和逻辑三大类。

(1) 算术运算符

算术运算符可分为两类：矩阵运算和数组运算。矩阵运算按照线性代数的规则进行运算，而数组运算则是数组对应元素之间的运算，如表 1.5 所示。

表 1.5: 算术运算符

运算符	运算方式	功能说明
+, -	矩阵运算, 数组运算	加减法
*, /, \, ^, '	矩阵运算	乘, 除, 左除, 乘方, 转置
.*, ./, .\, .^	数组运算	点乘, 点除, 点左除, 点乘方
:	矩阵运算, 数组运算	索引, 用于增量操作

例 1.7 矩阵乘法与数组乘法

```
>> a = [1 2 3; 4 5 6; 7 8 9]; b = [1 3 2; 4 6 5; 7 9 8];
>> c = a * b, d = a .* b
c =
    30    42    36
    66    96    81
   102   150   126
d =
     1     6     6
    16    30    30
    49    72    72
```

(2) 关系运算符

关系运算用于比较数组对应元素的大小关系，其运算符如表 1.6 所示。

表 1.6: 关系运算符

运算符	功能说明	运算符	功能说明
<	小于	>	大于
<=	小于等于	>=	大于等于
==	等于	~=	不等于

表 1.7: 逻辑运算符

运算符	功能说明	运算符	功能说明
&, and	逻辑与	, or	逻辑或
~	逻辑非	xor	逻辑异或
&&	捷径与		捷径或

(3) 逻辑运算符

MATLAB的逻辑运算有两大类：元素运算和捷径运算，如表 1.7 所示。

捷径运算首先判断第一个运算对象，如果可以知道结果，则直接返回，不再判断之后的运算对象。这样既提高了运算效率，又避免了一些不必要的错误。

例 1.8 捷径运算


```
>> a = 1; b = 0; x = b && (a / b > 1)
x =
    0
```

b为0时，捷径运算符就不会计算(a / b > 1)的值，也就避免了被0除的错误。

(4) 特殊运算符

还有一些特殊的运算符在MATLAB中经常使用，如表 1.8 所示。

表 1.8: 特殊运算符

运算符	功能说明	运算符	功能说明
[]	封装向量或矩阵	{ }	封装单元数组
()	封装函数参数或数组下标	...	续行符
;	命令结束符，分隔数组的行	.	域访问符
:	创建数组或数组下标；用于循环	,	分隔符，分隔数组的行元素
'	矩阵转置；封装字符串	@	访问函数句柄
%	注释符	*	通配符

(5) 运算符的优先级

MATLAB在执行表达式时将按照运算符的优先级进行，即先执行优先级高的运算符，然后从左向右执行同等级的运算符。运算符按优先级由高到低排列如表 1.9 所示。

表 1.9: 运算符的优先级

序号	运算符	序号	运算符
1	() (优先级最高)	7	<, <=, >, >=, ==, ~=
2	./, .^, ', ^	8	&
3	+, -, ~ (单目运算)	9	
4	.*, ./, .\, *, /, \	10	&&
5	+, - (双目运算)	11	(优先级最低)
6	:		

利用上述运算符，我们可以进行一些较为复杂的计算。为了方便地进行运算，MATLAB为我们提供了一些常用的数学函数，如表 1.10 所示，这些函数都可以对数组(按每个元素)进行运算。

表 1.10: 常用的数学函数

函数	功能说明	函数	功能说明	函数	功能说明
sin	正弦	exp	指数	fix	向0取整
cos	余弦	log	自然对数	floor	向 $-\infty$ 取整
tan	正切	log10	常用对数	ceil	向 $+\infty$ 取整
cot	余切	sqrt	开方	real	复数实部
asin	反正弦	abs	绝对值、模	imag	复数虚部
acos	反余弦	mod	模除求余	angle	复数辅角
atan	反正切	rem	除法余数	conj	复数共轭

注 1.4 上述三角函数的参数要求是弧度制，若要计算角度制的三角函数值，可在三角函数名后加“d”，如计算90°时的正弦值可用 $\text{sin}(90)$ ，其他函数与之类似。

例 1.9 指数函数与取整函数

```
>> a = [1 2; 3 4]; b = exp(a), c = floor(b)
b =
    2.7183    7.3891
   20.0855   54.5982
c =
     2     7
    20    54
```

1.5 数据类型

MATLAB中常用的数据类型主要有：数值(double)、逻辑(logical)、字符(char)、元胞(cell)和结构(structure)。它们可以进一步组成字符数组、元胞数组和结构数组。虽然MATLAB中也有单精度型数值和整数型数值，但不常用，这里主要介绍双精度(double)型数值。表 1.11 中的函数可以实现不同数据类型之间的相互转化。

表 1.11: 数据类型的转化

函数	功能说明	函数	功能说明
double	按ASCII码转化为数值	char	按ASCII码转化为字符
str2num	字符串转化为数值数组	str2double	字符串转化为数字
num2str	数值转化为字符串	num2cell	数值转化为元胞
cell2struct	元胞转化为结构	struct2cell	结构转化为元胞

(1) 字符串

MATLAB中字符串用单引号对来标识，组成字符数组。double、char函数可将字符串和数值按ASCII码换算为数值和字符串。数字字符串与数值之间的转换可用num2str、str2num函数实现。

例 1.10 字符与数值的转化

```
>> a = 'MATLAB', b = double(a)
a =
MATLAB
b =
    77    65    84    76    65    66
>> a = 'I Love MATLAB'
a =
I Love MATLAB
>> a = 1234; b = num2str(a), c = ['3011' b]
b =
1234
c =
30111234
```

(2) 元胞与结构

将不同类型、不同尺寸的数组，加上大括号“{ }”封装后，便形成了一个元胞。几个元胞组合在一起可构成元胞数组。

例 1.11 元胞数组

```
>> a = {'I love'; 76}; b = {'matlab'; 78};
>> c = [a, b], d = size(c)
c =
    'I love'    'matlab'
    [    76]    [    78]
d =
     2     2
```

一个结构可通过“域”来定义，它比元胞更丰富、更灵活。几个结构可以组成结构数组，只要其域名一致。

赋值格式为

```
数组名.域名 = 值;
```

例 1.12 结构数组

```

>> hand1.a = 'I love';
>> hand1.b = [1 2 3; 4 5 6];
>> hand2.a = 'matlab';
>> hand2.b = 1;
>> hand = [hand1;hand2]
hand =
2x1 struct array with fields:
    a
    b
>> hand(1)
ans =
    a: 'I love'
    b: [2x3 double]
>> hand(2).b
ans =
    1

```

1.6 实验报告

1. 执行下列指令，观察结果并理解其意义。

```

[1 2; 3 4] + 10 - 2i      log10([1 10 100])
prod([1 2; 3 4])          [a, b] = find([1 2; 3 4] >= [4 3; 2 1])
all([1 2 3; 4 5 6] > 1)  any([1 2 3; 4 5 6] > 1)

```

解 在命令行运行上述指令，可以发现其意义，详见下面的注释。

```

>> [1 2; 3 4] + 10 - 2i      % 矩阵与复数运算
ans =
    11.0000 - 2.0000i    12.0000 - 2.0000i
    13.0000 - 2.0000i    14.0000 - 2.0000i
>> log10([1 10 100])        % 取常用对数
ans =
     0     1     3
>> prod([1 2; 3 4])          % 求积
ans =
     3     8
>> [a, b] = find([1 2; 3 4] >= [4 3; 2 1]) % 寻找A >= B的位置
a =
     2
     2
b =
     1
     2
>> all([1 2 3; 4 5 6] > 1)  % 判断是否全 > 1
ans =
     0     1     1
>> any([1 2 3; 4 5 6] > 1)  % 判断是否有 > 1的元素
ans =
     1     1     1

```

2. 对矩阵 $C = \begin{pmatrix} 1 & 2 & 3 & 0 & 5 \\ -1 & 8 & -3 & 7 & 2 \\ 5 & 7 & -6 & 2 & 1 \end{pmatrix}$, 执行下列命令, 观察结果并理解其意义。

- (1) $C([1, 2], [3, 5])$ (2) $C(:, 3)$ (3) $C([1, 3], \text{end})$ (4) $C(7)$ (5) $C(7:\text{end})$
 (6) $C(:, \text{end} - 1)$ (7) $C(2, :)$ (8) $C([2, 2], [3, 3])$ (9) $C + 10$ (10) $2 * C + 1$

解 意义详见注释。

```
>> C = [1, 2, 3, 0, 5; -1, 8, -3, 7, 2; 5, 7, -6, 2, 1];
>> C([1, 2], [3, 5]) % 取第1,2行, 第3,5列的元素组成矩阵
ans =
     3     5
    -3     2
>> C(:, 3) % 取第3列所有元素组成矩阵
ans =
     3
    -3
    -6
>> C([1, 3], end) % 取第1、3行最后一个元素组成矩阵
ans =
     5
     1
>> C(7) % 取矩阵(按列排序)的第7个元素
ans =
     3
>> C(7:end) % 取矩阵(按列排序)的第7到最后的元素
ans =
     3    -3    -6     0     7     2     5     2     1
>> C(:, end - 1) % 取倒数第2列所有元素
ans =
     0
     7
     2
>> C(2, :) % 取第2行所有元素
ans =
    -1     8    -3     7     2
>> C([2, 2], [3, 3]) % 取第2行, 第3列元素组成2×2矩阵
ans =
    -3    -3
    -3    -3
>> C + 10 % 矩阵C每个元素+10组成新矩阵
ans =
    11    12    13    10    15
     9    18     7    17    12
    15    17     4    12    11
>> 2 * C + 1 % 矩阵C每个元素×2再+1组成新矩阵
ans =
     3     5     7     1     11
    -1    17    -5    15     5
    11    15   -11     5     3
```

3. 对矩阵 $a = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$, 进行如下运算, 观察结果并理解其意义。

- (1) $a([2 \ 1], :) = a([1 \ 2], :)$ (2) $a([1 \ 3], :) = a([2 \ 2], :)$
 (3) $a = a([2 \ 3], :)$ (4) $b = [1 \ 2 \ 3]; a(:, 4) = b'$

解 意义详见注释。

```
>> a = [1 2 3; 4 5 6; 7 8 9];
>> a([2 1], :) = a([1 2], :) % 将矩阵a第1行与第2行互换
a =
     4     5     6
     1     2     3
     7     8     9
>> a = [1 2 3; 4 5 6; 7 8 9];
>> a([1 3], :) = a([2 2], :) % 将a的第2行元素赋值给第1,3行
a =
     4     5     6
     4     5     6
     4     5     6
>> a = [1 2 3; 4 5 6; 7 8 9];
>> a = a([2 3], :) % 取a的第2,3行元素组成新矩阵a
a =
     4     5     6
     7     8     9
>> a = [1 2 3; 4 5 6; 7 8 9];
>> b = [1 2 3]; a(:, 4) = b' % 矩阵拼接, 将b转置, 加到a的第4列
a =
     1     2     3     1
     4     5     6     2
     7     8     9     3
```

4. 设 $a = \begin{pmatrix} 2 & 2 \\ -1 & 5 \end{pmatrix}$, $b = \begin{pmatrix} 1 & -1 \\ 0 & 2 \end{pmatrix}$, $c = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$, $d = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, 写出下列命令的运算结果, 若有不合法运算, 请说明理由。

- (1) $r = a + b$ (2) $r = a * b$ (3) $r = a .* b$ (4) $r = a * c$
 (5) $r = a .* c$ (6) $r = a \setminus b$ (7) $r = a .\setminus b$ (8) $r = a.^b$

解 意义详见注释。

```
>> a = [2 -2; -1 5]; b = [1 -1; 0 2]; c = [1; -2]; d = [1 0; 0 1];
>> r = a + b % 矩阵相加
r =
     3    -3
    -1     7
>> r = a * d % 矩阵相乘
r =
     2    -2
    -1     5
>> r = a .* d % 矩阵点乘, 对应元素作乘
r =
```

```

      2      0
      0      5
>> r = a * c      % 矩阵相乘
r =
      6
     -11
>> r = a .* c      % 不合法运算，点乘运算表示两个矩阵对应元素作乘，要求点乘运算双方size相等、元素相同，而a是
                        % 2×2的矩阵，c为2×1的矩阵，元素个数不相等，无法运算，系统报错。
Error using .*
Matrix dimensions must agree.
>> r = a \ b      % 左除运算
r =
      0.6250      -0.1250
      0.1250      0.3750
>> r = a ./ b      % 左除点运算
r =
      0.5000      0.5000
           0      0.4000
>> r = a .^ b      % 点乘方运算
r =
      2.0000      -0.5000
      1.0000     25.0000

```

5. 已知函数 $f(x) = x^4 - 2x$ 在 $(-2, 2)$ 内有两个根。取步长0.05，通过计算求得的最小值点和两个根的近似值(提示：求近似值等价于求函数绝对值的最小值点)。

解 方法一

```

>> x = -2:0.05:2;      % 在[-2, 2]内取点
>> y = x.^ 4 - 2 * x;   % 得到x对应的函数值y
>> miny = min(y);       % 取y的最小值
>> minx = x(y == miny)  % 取最小值点，即y = miny对应的x值
minx =
      0.8000
>> a = abs(y);          % 求y的绝对值
>> x1 = x(a == min(a))  % 根x1
x1 =
           0
>> xx = x(a ~= min(a)); % 去除x1，取a不等于x1的点
>> yy = a(a ~= min(a)); % 去除x1对应的函数值
>> x2 = xx(yy == min(yy)) % 根x2
x2 =
      1.2500

```

方法二

```

>> x = -2:0.05:2;      % 在[-2, 2]内取点
>> y = x.^ 4 - 2 * x;   % 得到x对应的函数值y
>> miny = min(y);       % 取y的最小值
>> minx = x(y == miny)  % 取最小值点
minx =

```

```
0.8000
>> y = abs(y);           % 求y的绝对值
>> miny1 = min(y);       % 取y的最小值
>> minx1 = x(y == miny1) % 近似根1
minx1 =
    0
>> y(y == miny1) = [];   % 删除近似根1的函数绝对值
>> x(x == minx1) = [];   % 删除近似根1
>> miny2 = min(y);       % 取y的次小值
>> minx2 = x(y == miny2) % 近似根2
minx2 =
    1.2500
```

方法三

```
>> x = -2:0.05:2;       % 在[-2, 2]内取点
>> y = x.^4 - 2 * x;    % 得到x对应的函数值y
>> miny = min(y);       % 取y的最小值
>> minx = x(y == miny)  % 取最小值点
minx =
    0.8000
>> y = abs(y);           % 求y的绝对值
>> miny1 = min(y);       % 取y的最小值
>> minx1 = x(y == miny1) % 近似根1
minx1 =
    0
>> y(y == miny1) = 1000; % 赋常值
>> miny2 = min(y);       % 取y的次小值
>> minx2 = x(y == miny2) % 近似根2
minx2 =
    1.2500
```

故最小值点为 $x = 0.8$ ，两个根分别为0和1.25。

第二章 程序设计

2.1 Editor与M文件

在指令窗口执行的程序可读性差，对于复杂的程序，如果仍在指令窗口内输入，则调试、修改、保存是相当不方便的。这就需要编辑M文件来执行。M文件是包含MATLAB语言代码的一类文件，扩展名为“.m”，它是普通的文本文件，用户可以使用记事本、word等来编辑，也可以使用MATLAB中程序编辑器(Editor)来编写。

点击窗口工具栏New按钮(黄色“+”号)或在命令行输入“edit”命令，可以进入Editor窗口。

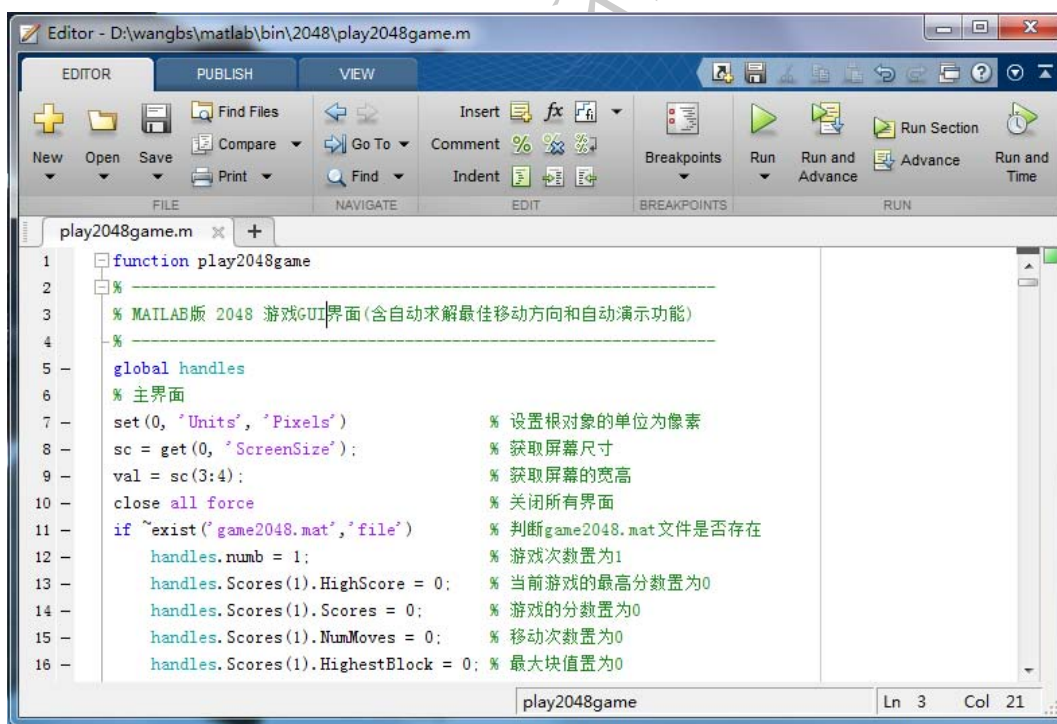


图 2.1: Editor界面

Editor界面如图 2.1 所示，由上到下依次是标题栏、菜单栏、工具栏、文件栏、编辑区(左边一栏

为行号，滑块的右边为提示信息，中间为编辑区)、状态栏(包括当前文件名和光标位置)。其中工具栏是MATLAB编程中最常用的，其简单介绍见表 2.1。

表 2.1: 工具栏按钮及其功能

按钮	功能	按钮	功能	按钮	功能
New	新建文件	Open	打开文件	Save	保存文件
Find Files	查找文件	Compare	文件对比	Print	打印文件
Insert	插入	Comment	添减注释	Indent	代码排版
back/Forward	光标移动	Go To	到... 位置	Find	查找或替换
Breakpoints	断点操作	Run	运行	Run and Advance	运行并向前
Run Section	运行选中	Advance	下一元胞	Run and Time	运行并计时

例 2.1 第一章实验报告里的第5题也可以写成M文件的形式。

```
x = -2:0.05:2;          % 在[-2, 2]内取点
y = x.^ 4 - 2 * x;      % 得到x对应的函数值y
miny = min(y);          % 取y的最小值
minx = x(y == miny)     % 取最小值点
a = abs(y);              % 求y的绝对值
x1 = x(a == min(a))     % 取根x1
xx = x(a ~= min(a));    % 去除x1
yy = a(a ~= min(a));    % 去除x1对应的函数值
x2 = xx(yy == min(yy)) % 取根x2
```

注 2.1 在保存文件时需要注意:

- (1) 文件名一律以字母开头，仅由字母、数字、下划线组成，不能含有其他符号;
- (2) M文件一般用小写字母，MATLAB并不区分文件名的大小写;
- (3) M文件名应防止与变量名冲突，文件名一般用四个以上的字符命名;
- (4) MATLAB只执行已保存的M文件，所以修改程序后应及时保存;
- (5) M文件一般保存在当前所在目录，以便执行。

M文件可分为两类: M脚本文件和M函数文件。

如例2.1的文件，我们将多条MATLAB语句写在Editor中，并保存e2.1.m文件在当前目录下，这样就得到了一个名为ex2.1.m的脚本文件。

例 2.2 在指令窗口输入文件名(或点击“Run”按钮)，便可得到如下结果

```
>> ex2_1
minx =
    0.8000
x1 =
    0
x2 =
    1.2500
```

显而易见，M脚本文件只是一串指令的集合，没有参数传递的功能，当我们需要修改程序中的某些变量值时，就需要修改脚本文件，这不仅不便而且容易出错。相比而言，M函数文件可以实现参数的传递，在实际应用中更广泛。M函数文件总是以关键词“function”开头，格式为：

```
function 输出变量 = 函数名(输入变量)
% 注释：函数功能
%   Input   输入变量含义与格式
%   Output  输出变量含义与格式
%   调用方法
%   调用举例
%
% 编写者
% 编写时间及版本

可执行语句组
```

例 2.3 在Editor中编写函数文件以实现从正整数 x 中提取第 n 位数字。

```
function y = qushuzi(x, n)
% 从数x中提取第n个数字
% Input -x 数字 -n 取第n位数字，个位为第1位
% Output -y 数字
% Example
% x = 142857; n = 3;
% y = qushuzi(x, n)
%
% Copyright Bao-Shan Wang
% $ Matlab R2015a $ $ Date: 2015/10/03 11:19:54 $
y = fix((x - fix(x / 10^n) * 10^n) / 10^(n - 1));
```

注 2.2 输出变量必须在函数中赋值。

保存为qushuzi.m，然后在指令窗口调用。

```
>> x = 142857; n = 3;
>> y = qushuzi(x, n)
y =
    8
```

为了方便用户更灵活的编写程序，M函数文件中默认了nargin、nargout和varargin这几个变量，nargin表示该函数的输入变量的个数，nargout表示函数的输出变量的个数，而varargin表示可变输入输出变量的个数。

至于M文件的保存，除了要注意之前所说的五点外，还应注意，MATLAB在调用M函数文件时是以磁盘文件名调用的，而不是函数名，因此为增强函数的可读性，两者命名最好一致。M函数文件可以被其他的函数文件、M脚本文件、M语句调用，除此之外，M函数文件还可以作为参数被调用。

例 2.4 feval函数可用于执行函数为参数的语句。

```
>> feval(@qushuzi, 142857, 3)      % qushuzi函数详见例 2.3
ans =
     8
>> feval(@sort, [1 4 2 7 9 6 0 5]) % sort函数可以实现数组排序
ans =
     0     1     2     4     5     6     7     9
```

一些比较简单的函数表达式可以使用inline函数或匿名函数方式，而不必写成外部M函数。

例 2.5 使用inline函数或匿名函数。

```
>> feval(inline('sort(x)'), [1 4 2 7 9 6 0 5])
ans =
     0     1     2     4     5     6     7     9
```

注 2.3 inline函数可能在之后的版本中移除。

对于一些结构比较复杂的程序，命令行上的操作不太灵活，且一旦出错，修改起来十分麻烦，因此建议大家使用M脚本文件或M函数文件。以后的程序若没有“>>”标志则认为该程序是M文件，若有“function”标志则可认为是M函数文件。

2.2 控制语句与举例

以上我们见到的函数和命令都是顺序的，自上而下执行的，而对于复杂的程序来说，仅有顺序结构显然是不行的，这时就需要用到复杂的程序结构，例如循环、分支等，详见表 2.2，需要注意的是循环结构和分支结构都有特定的关键词，并且以end结尾。

例 2.6 求最小整数n使得

$$\sum_{m=0}^n \ln(1+m) > 100.$$

表 2.2: MATLAB常用的结构语句

结构语句		语法	说明
循环语句	for 循环	for 变量= 数组 执行语句 end	循环变量依次取数组中的值，直到遍历数组。数组格式为：初值:增量:终值
	while 循环	while 条件 执行语句 end	当满足条件时，执行语句直到条件不满足。使用while循环时要注意避免出现死循环
分支语句	if else 分支	if 条件1 语句1 elseif 条件2 语句2 else 语句n end	若满足条件1，则执行语句1，并结束；若满足条件2，则执行语句2，并结束...，若不满足上述条件，则执行语句n，并结束。常用的结构有：“if 条件, 语句; end”结构和 “if 条件, 语句1; else 语句2; end” 结构
	switch case 分支	switch 分支变量 case 值1 语句1 case 值2 语句2 otherwise 语句n end	若分支变量取值为值1，则执行语句1，并结束；若取值为值2，则执行语句2，并结束...，若都不满足，则执行语句n，并结束。其中otherwise项可以省去不写
中断语句	pause	pause/pause(n)	暂停执行直到敲击键盘或暂停n秒后再继续执行
	break	break	中断执行，用于循环语句，表示跳出循环
	return	return	中断执行，回到主调函数或命令窗口
	error	error(str)	中断程序，输出错误信息字符串str
其他	continue	continue	循环中跳过continue之后的语句进行下一步循环
	input	input	暂停程序，待键盘输入后再执行

注 2.4 使用while循环时应充分小心，若循环条件设置不当，循环将一直运行，无法跳出，导致整个程序进入死循环。此时，可使用快捷键“Ctrl+C”强行中断。

解 方法一 使用while循环

```
s = 0;
n = -1;
while s <= 100
    n = n + 1;
    s = s + log(1 + n);
end
n
```

方法二 使用for循环和if语句

```

s = 0;
n = -1;
for i = 0:50
    n = n + 1;
    s = s + log(1 + i);
    if s > 100
        break;
    end
end
n

```

运行结果为

```

n =
    37

```

例 2.7 用循环语句编写求 $n!$ 的函数 $jiecheng(n)$ ，用该函数求 $\binom{20}{15}$ 的值。

解 编写阶乘函数

```

function y = jiecheng(n)
% 求n的阶乘
% Input -n 非负整数
% Output -y n的阶乘
y = 1;
if n >= 2
    for i = 1:n
        y = y * i;
    end
end
end

```

用阶乘函数求 $\binom{20}{15}$ 的值

```

>> n = 20; m = 15;
>> cn = jiecheng(n);
>> cm = jiecheng(m);
>> ch = jiecheng(n - m);
>> c = cn / cm / ch
c =
    15504

```

例 2.8 编写求矩阵的最大值及编址(行标和列标)的函数，并以

$$A = \begin{pmatrix} 1 & 3 & 4 & 5 \\ 2 & 9 & 3 & 10 \\ 4 & 6 & 1 & 78 \end{pmatrix}$$

为例求解验证。

解 编写函数

```
function [y, m, n] = mymax(A)
% 求矩阵A的最大值及其编址(m,n)
% Input -A 矩阵
% Output -y A的最大值 -[m,n] 最大值的编址
[s, t] = size(A);
y = A(1, 1);
m = 1;
n = 1;
for i = 1:s
    for j = 1:t
        if A(i, j) > y
            y = A(i, j);
            m = i;
            n = j;
        end
    end
end
```

求A的最大值及编址

```
>> a = [1 3 4 5; 2 9 3 10; 4 6 1 78];
>> [y, m, n] = mymax(a)
y =
    78
m =
     3
n =
     4
```

2.3 编程规范

2.3.1 程序优良的评价

评价程序的好坏优劣，除了要考虑程序实现的功能外，还要考虑程序代码的可读性和高效性。

(1) 速度快，效率高。好的程序应该是速度快、效率高的，但在实际应用中速度和质量往往不能兼得。例如一个预报天气的程序可以准确地预报一周以后的天气变化，但程序计算要在花费一个月的时间，显然这样的程序并不好。当速度与质量不能兼得时，用户可根据程序的侧重点来调整速度与质量的比重。本节将在后面着重介绍提高速度的方法。

(2) 简洁又明了。好的程序应该是可读性高的。代码应当简洁明了，方便用户读取、使用和理解。

2.3.2 编程帮助

编程过程中用户经常会遇到不熟悉的函数，为此MATLAB提供了以下三种形式的帮助：

(1) 指令窗口的文本形式。这是最可信、最原始、最常用的帮助形式。直接在指令窗口中由help、lookfor等命令获得所有包含在M文件中的帮助注释。

例 2.9 `sum`函数的帮助注释。

```
>> help sum
sum Sum of elements.
    S = sum(X) is the sum of the elements of the vector X. If X is a matrix,
    S is a row vector with the sum over each column. For N-D arrays,
    sum(X) operates along the first non-singleton dimension.
    S = sum(X,DIM) sums along the dimension DIM.
    ..... % 限于篇幅, 省略了部分内容
Examples:
If X = [0 1 2; 3 4 5]
then sum(X, 1) is [3 5 7] and sum(X, 2) is [3; 12]
    ..... % 限于篇幅, 省略了部分内容
```

(2) 帮助导航的HTML形式。这是系统性叙述MATLAB规则和用法的帮助形式，也是最重要的帮助形式，可直接在指令窗口中由doc、docsearch等命令获得帮助。

例 2.10 `sum`的HTML形式。

```
>> doc sum % 结果如图 2.2
```

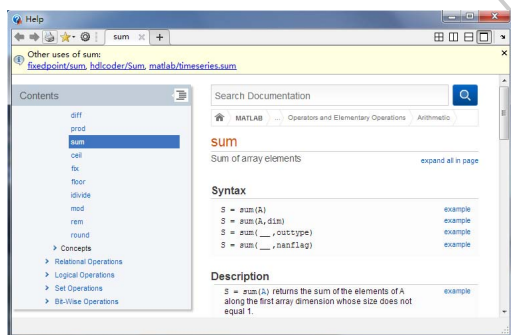


图 2.2: `sum`的HTML形式



图 2.3: Web形式

(3) Web形式。Web形式的帮助可以通过浏览器访问MathWorks官网¹、中文帮助网页²或者访问MATLAB相关的中文论坛³获得。中文帮助如图 2.3 所示。这种形式的帮助主要包括PDF文件、视频演示文件、前人的经验交流、活动、培训等，但Web形式更新快，请以实际为准。

¹ <http://cn.mathworks.com/>

² <http://cn.mathworks.com/help/matlab/index.html>

³ <http://www.ilovematlab.cn>

2.3.3 文件命名

若M文件是函数，保存的文件名最好与函数名一致。文件名的命名最好是由大小写英文、数字、下划线等组成。MATLAB的搜索是按优先级从高到低的顺序(如表 2.3)搜索的。简单的单词命名容易与内部函数同名，导致内部函数无法使用。

表 2.3: 函数调用的优先级

顺序	优先级	顺序	优先级
1	变量	5	重载方法
2	子函数	6	当前目录中的同名函数
3	私有函数	7	搜索路径中其他目录中的函数
4	类构造函数		

注 2.5 (1) 搜索工作空间是否存在同名变量，若有则停止搜索；

(2) 其他目录中的函数优先级最低。若函数不在搜索路径中，则无法被调用；

(3) 若查询同名函数中究竟哪个函数被调用了，可用`which`函数查询。

2.3.4 提高执行速度

(1) 若想了解程序的运行时间，可使用`tic/toc`来计时：`tic`开始计时，`toc`停止计时；

(2) 编程时应预分配内存，减少多次访问内存的时间，以加快程序的运行速度；

例 2.11 未分配内存与预分配内存的比较。

```
tic;
for i = 1:1000
    for j = 1:1000
        a(i, j) = i + j;
    end
end
toc;
tic;
a = zeros(1000, 1000); % 预分配内存，将a赋值为1000×1000的零矩阵
for i = 1:1000
    for j = 1:1000
        a(i, j) = i + j;
    end
end
toc;
```

输出结果如下：

```
Elapsed time is 0.099116 seconds.  
Elapsed time is 0.017638 seconds.
```

(3) 数据输出会花费一定的时间，减少不必要的输出可加快程序的执行速度，屏蔽不必要的输出可在指令后加“;”；

(4) 函数调用会花费一定的时间，因此将被调用的函数置为子函数或匿名函数可节省搜索时间；

(5) MATLAB 优于处理向量化数据，编程时应尽量采用向量化代码，尽量避免使用循环(尤其内嵌分支语句时)，因为 MATLAB 是实时为变量分配内存的，每次循环都将访问内存而增加运行时间；

(6) 运行次数多的分支结构应放在前面；

(7) 多层循环嵌套时应将循环次数少的放在外层，运行次数多的放在内层；

例 2.12 互换矩阵的两行元素。

```
a = 1:1000;  
a = reshape(a, 2, []);          % 重排数组a为2×500矩阵  
tic;  
a([2 1], :) = a([1 2], :);      % 第一种方法  
toc;  
a = 1:1000;  
a = reshape(a, 2, []);  
tic;  
for j = 1:500  
    for i = 1:2  
        b(i, j) = a(3 - i, j); % 第二种方法  
    end  
end  
a = b;  
toc;  
a = 1:1000;  
a = reshape(a, 2, []);  
tic;  
for i = 1:2  
    for j = 1:500  
        b(i, j) = a(3 - i, j); % 第三种方法  
    end  
end  
a = b;  
toc;  
a = 1:1000;  
a = reshape(a, 2, []);  
tic;  
for i = 1:2  
    b(i, :) = a(3 - i, :);      % 第四种方法  
end  
toc;
```

输出结果如下：

```
Elapsed time is 0.000018 seconds.
Elapsed time is 0.000119 seconds.
Elapsed time is 0.000077 seconds.
Elapsed time is 0.000010 seconds.
```

- (8) 在读取数据文件时，应尽量“多拿少取”，读取次数尽量少，争取一次读取全部所需数据；
- (9) 查看程序的执行速度可以使用Profiler分析代码效率，点击“Run and Time”按钮可以生成分析报告，图 2.4 展示的是例 2.7 的分析报告。

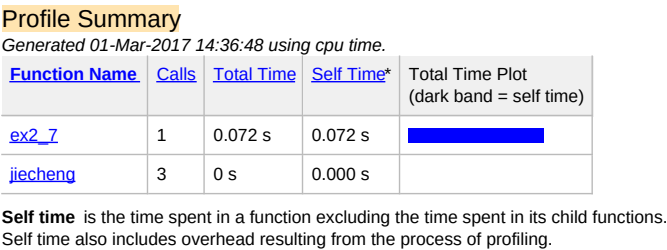


图 2.4: 例 2.7 的Profiler分析报告

2.3.5 代码排版

为保证M文件的可读性，代码排版的要求如下：

- (1) 每行代码控制在80列之内，代码用续行符“...”分行，在Editor中有条灰色的竖线，分行时尽量选择在竖线附近；
- (2) 分行显示的3条原则：
 - 1) 在一个逗号或空格后分行；
 - 2) 在一个操作符之后分行；
 - 3) 分行时对齐表达式；
- (3) 代码的缩排一般为3个空格或1个“Tab”键，建议采用MATLAB默认的缩排格式，选中一段代码后按“Ctrl+I”键，MATLAB将自动缩排选中代码；
- (4) 一行代码应该只包含一条可执行语句，(当然，短的if、for、while语句可写在一行)，一行写多条执行语句，不仅影响代码的美观，还会减慢代码的运行速度和可读性；
- (5) 合理使用空格：
 - 1) 在=, ==, ~=, >, >=, <, <=, +, -, *, /, %, &, |, &&, ||的前后加空格；
 - 2) ~, ^符号前后不需要空格，冒号表达式为了直观不需要添加空格；
 - 3) 在逗号，分号的后面添加空格，前面则不需要加空格；

4) 关键字后面加空格，而函数名后面则不需要加空格；

5) 块内部的逻辑组语句前后用空行隔开；块与块之间用多个空行隔开；

(6) 调试时，对于暂时用不到的代码，可以设置成注释而不参与运算，选中代码，按下“Ctrl+R”键，将选中的代码设置成注释，按下“Ctrl+T”键可取消注释，也可以用“%{.....%}”注释整段代码，还可以在代码前后加“if 0”和“end”语句。

例 2.13 注释整段代码。

```
%% 方法一
if n >= 2
    for i = 1:n
        y = y * i;
    end
end
%% 方法二
%{
if n >= 2
    for i = 1:n
        y = y * i;
    end
end
%}
%% 方法三
if 0
    if n >= 2
        for i = 1:n
            y = y * i;
        end
    end
end
end
```

2.4 实验报告

1. 设 x 为一个长度为 $n(n > 1)$ 的数组，编写程序求其均值和标准差。

$$\bar{x} = \sum_{i=1}^n x_i, \quad s = \sqrt{\frac{1}{n-1} \left(\sum_{i=1}^n x_i^2 - n\bar{x}^2 \right)}.$$

解 方法一 编写函数

```
function [y, z] = m_s(x)
% 求数组x的均值y和标准差z
n = length(x);
y = 0;
w = 0;
for i = 1:n
    y = y + x(i);
```

```
end      % 该循环等价于 y = sum(x);
y = y / n;
for j = 1:n
    w = w + x(j)^2;
end      % 该循环等价于 w = sum(x.^2);
z = sqrt((w - n * y^2) / (n - 1));
```

方法二 调用内部函数

```
function [y, z] = m_s1(x)
y = mean(x);      % mean函数用来求均值
z = std(x);        % std函数用来求方差
```

应用上述函数求解

```
>> n = 100;
>> x = rand(n, 1);      % 生成n个[0, 1]上的均匀分布随机数
>> [y, z] = m_s(x)
y =
    0.4681
z =
    0.2869
>> [y, z] = m_s1(x)
y =
    0.4681
z =
    0.2869
```

2. 编写MATLAB文件，要求从键盘输入一个正整数 n ，判断它能否被3整除，若能，则输出“ n 能被3整除”，否则输出“ n 不能被3整除，余数为...”，并输入7验证(提示：输出字符串结果可用disp(str)函数)。

解 编写M脚本文件如下：

```
n = input('请输入一个正整数: ');
r = mod(n, 3);      % 模除取余数
if r ~= 0
    disp([num2str(n), ' 不能被3 整除, 余数为', num2str(r)]);
else
    disp([num2str(n), ' 能被3 整除']);
end
```

验证程序

请输入一个正整数: 7
7 不能被3 整除, 余数为1

3. 用循环语句生成Fibonacci数列, $F_1 = F_2 = 1, F_k = F_{k-1} + F_{k-2}, k = 3, 4, \dots$, 并验证 $\frac{F_k}{F_{k-1}} \rightarrow \frac{1+\sqrt{5}}{2}$ (提示: 计算误差为 10^{-8})。

解 编写M脚本文件ch2_3.m如下:

```
f(1) = 1;
f(2) = 1;
n = 3;
format long % 长精度显示
x = 1 / 2 * (1 + sqrt(5))
df = 1;
while abs(x - df) > 1e-8
    f(n) = f(n - 1) + f(n - 2);
    df = f(n) / f(n - 1);
    n = n + 1;
end
df
format short % 恢复为默认的短格式
```

执行结果

```
>> ch2_3
x =
    1.618033988749895
df =
    1.618033998521803
```

4. 编写程序将数组 $x = [1, 4, 2, 8, 5, 7]$ 按从小到大的顺序排列。

解 使用最简单的方法, 利用如下循环进行比较并排序:

```
function x = mysort(x)
% 数组x按大小顺序排序组成新的数组x
n = length(x); % 求数组长度
for i = 1:n - 1 % 取前n-1个位置循环
    for j = i + 1:n % 比较其后的元素
        if x(i) > x(j) % 若大于其后的元素
            x([i, j]) = x([j, i]); % 交换位置
        end
    end
end
end
```

调用mysort函数进行排序

```
>> x = [1 4 2 8 5 7];
>> y = mysort(x)
y =
     1     2     4     5     7     8
```

5. 编写程序判断数组 $A = [1, 4, 2, 8, 5, 7]$ 和 $B = [1, 8, 4, 2, 7, 5]$ 是否相等。

解 数组相等是指数组尺寸一致，对应元素均相等。编写函数如下：

```
function f = besame(A, B)
% 判断A矩阵与B矩阵是否相等，相等f=1，否则为0
f = 1; % 先假设A = B
[s1, t1] = size(A); % 求矩阵A的尺寸
[s2, t2] = size(B); % 求矩阵B的尺寸
if s1 ~= s2 || t1 ~= t2 % 判断尺寸是否一致
    f = 0;
    return;
end
for i = 1:s1 % 判断对应元素是否相等
    for j = 1:t1
        if A(i, j) ~= B(i, j)
            f = 0;
        end
    end
end
end
```

调用besame函数判断 A 与 B 是否相等

```
>> A = [1 4 2 8 5 7];
>> B = [1 8 4 2 7 5];
>> f = besame(A, B)
f =
     0
```

6. 求一个六位数，它的2、3、4、5、6倍均是该六位数的数字组合。

解 编程前作如下分析：

- (1) 数 n 和 $6n$ 是六位数，则整数 $n \in [100000, 166666]$ ；
- (2) 六个数字的组合可以组成一个向量，元素可按位取出；
- (3) 可以先排序再比较向量是否相等。

按照上述思路编写程序ch2.6.m如下：

```
x = zeros(1, 6); y = zeros(1, 6); % 预分配内存
for n = 100000:166666 % 对n进行循环
    for s = 1:6 % 取n的六个数字
```

```

        x(s) = qushuzi(n, s);
    end
    x = mysort(x);           % 排序
    f = 1;
    for t = 2:6              % 对[2:6]*n进行取数字排序
        for s = 1:6
            y(s) = qushuzi(n * t, s);
        end
        y = mysort(y);       % 也可用sort函数
        if besame(x, y) == 0  % 也可用isequal函数
            f = 0;
        end
    end
    if f == 1
        disp(['这个数是', num2str(n)]); % 输出结果
    end
end
end

```

执行程序求解

```

>> ch2_6
这个数是 142857

```

注 2.6 142857, 又名走马灯数, 被誉为世界上最神奇的数字, 它不仅是 $\frac{1}{7}(=0.142857142857\cdots)$ 的循环节, 而且还有更多神奇的地方等待你去发掘!

7. 考虑梯子长度问题:

一幢楼房的后面是一个很大的花园。在花园中紧靠着楼房有一个温室, 温室伸入花园宽 $2m$, 高 $3m$, 温室的正上方是楼房的窗台。清洁工打扫窗台周围, 他得用梯子越过温室, 一头放在花园中, 一头靠在楼房的墙上, 其截面图如图 2.5 所示。因为温室OACD不能承受梯子BE的压力, 所以梯子太短是不行的。现清洁工只有一架 $7m$ 长的梯子, 你认为它能达到要求么? 能满足要求的梯子的最小长度是多少? 试通过向量运算和枚举算法(通过循环遍历所有可能的情况, 并从中取最小梯长)求解(提示: 长度精确到 $0.01m$, 角度精确到 1°)。

解 当梯子BE与温室OACD恰好在C点接触时, 不会对温室产生损害, 调整角度后梯子长度可达到最小。设 $\angle OBE = \alpha$, $|AC| = h = 3$, $|CD| = w = 2$, 则有

$$|BE| = |BC| + |CE| = \frac{|AC|}{\sin \alpha} + \frac{|CD|}{\cos \alpha} = \frac{h}{\sin \alpha} + \frac{w}{\cos \alpha}$$

编写函数如下:

```

function y = ladder_length(w, h, alpha)
% 计算梯子长度
% Input -w 温室宽度 -h 温室的高 -alpha 梯子角度
% Output -y 梯子长度
y = w ./ cos(alpha) + h ./ sin(alpha);

```

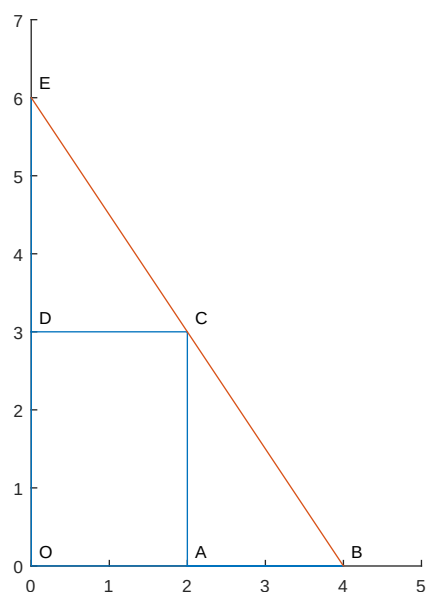



图 2.5: 梯子与温室的示意图

编写向量法和枚举法程序并求解：

```
%% 向量法
w = 2; h = 3;
alpha = linspace(0, pi/2, 91);
L = ladder_length(w, h, alpha);
[minL, b] = min(L);
minL
ang = b - 1    % 注意角度与b的关系
%% 枚举法
w = 2; h = 3;
minL = inf; ang = 0;
for i = 0:90
    L = ladder_length(w, h, i * pi/180);
    if minL > L, minL = L; ang = i; end
end
minL, ang
```

计算结果

```
minL =
    7.0235
ang =
    49
```

故梯子最小长度为7.03m(考虑实际情况), 7m的梯子是不达到要求的。

8. 考虑一个有趣的问题:

(1) 已知2015年11月11日是星期三, 试计算: 公元1年1月1日是星期几?

(2) 编写万年历函数 $w = \text{wnl}(\text{year}, \text{month}, \text{day})$ 实现计算year年month月day日为星期w的功能。
(提示: 星期日用7表示)

解 编程前作如下分析:

平年有365天, 为52周余1天, 由于周期性, 可以考虑平年为1天, 闰年为2天。现编写判断闰年的程序:

```
function lpflag = isleapyear(year)
% 判断是否为闰年
% Input -year 年份, 数值
% Output -lpflag lpflag = 1, 闰年; lpflag = 0, 平年
lpflag = (~mod(year, 4) && mod(year, 100)) || ~mod(year, 400);
```

知道当年是否为闰年后, 我们就可以计算year年month月day日是该年的第几天, 编写程序如下:

```
function ydays = yeardays(year, month, day)
% 计算该日期为当年的第几天
% Input -year 年份 -month 月份 -day 天数
% Output -ydays 日期在当年的第几天
d = 28 + isleapyear(year);
lmdays = [31, d, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31];
if month < 1 || month > 12, error('Error in month!'); end
if lmdays(month) < day, error('Error in day!'); end
ydays = sum(lmdays(1:month-1)) + day;
```

(1) 计算1年1月1日为星期几

```
year = 2015; month = 11; day = 11; % 日期
ydays = yeardays(year, month, day); % 计算该日期为当年第几天
years = abs(year - 1); % 间隔多少年
% 计算期间的闰年数
lpyear = fix(years / 4) - fix(years / 100) + fix(years / 400);
days = years + lpyear + ydays - 1; % 去除52周的天数与ydays的和, 即与1年1月1日的间隔
w = 3 - mod(days, 7) % 因2015-11-11为周三
if w <= 0, w = w + 7; end
```

计算结果

```
w =
1
```

故1年1月1日为星期一。

(2) 参照(1)的程序, 将“1年1月1日为星期一”作为标准, 编写wnl函数如下:

```
function w = wnl(year, month, day)
% 计算该日期为星期几
% Input -year 年份 -month 月份 -day 天数
% Output -w 星期几
ydays = yeardays(year, month, day);
years = year - 1;
lpyear = fix(years / 4) - fix(years / 100) + fix(years / 400);
days = years + lpyear + ydays;
w = mod(days - 1, 7) + 1;
```

注 2.7 也可使用Zeller公式(只适用于1582年10月15日之后的日期), 编写程序如下:

```
function w = wnl(year, month, day)
datedata = year * 1000 + month * 100 + day;
if datedata <= 15821015, error('Error in date!'); end
if month <= 2, year = year - 1; month = month + 12; end
a = day + 1 + 2 * month + 3 * (month + 1) / 5 + year + ...
    year / 4 - year / 100 + year / 400;
w = fix(mod(a - 1, 7)) + 1;
```

注 2.8 这里只对日期是否在1582年10月15日之后进行判断, 并没有判断是否为错误日期(如, 1月32日), 读者可参考yeardays函数自行编写判断语句。

我的 MATLAB 报告

王保山

第三章 可视化绘图

MATLAB除了快速便捷的科学计算外，还有可视化绘图和仿真的功能。

人们很难从一大堆原始的数据中发现它们的含义，而数据图形却能使视觉感官直接感受到数据的许多内在本质，发现数据的内在联系。MATLAB可以表达出数据的二维、三维、甚至四维的图形，通过图形的线型、立面、色彩、光线、视角等属性的控制，可把数据的内在特征表现得淋漓尽致。

3.1 绘制曲线

MATLAB中绘制曲线的函数主要有plot, fplot, plot3等, 具体功能如表 3.1所示。

表 3.1: 绘制曲线的函数和功能

函数	功能
plot(x)	绘制向量x对应于其索引值的二维曲线
plot(x, y)	绘制以数据(x(i),y(i))为节点的折线
plot(x, y, s)	绘制(x,y)的曲线, s为曲线标记, 如表 3.2
plot(x1, y1, x2, y2, ...)	绘制多条折线(x1,y1),(x2,y2),...
plot(x1, y1, s1, x2, y2, s2, ...)	绘制多条折线, 标记分别为s1,s2,...
fplot(fun, [a, b])	绘制函数fun在区间[a,b]上的图
fplot(fun, [a, b], s)	绘制函数fun在区间[a,b]上的图, 标记为s
plot3(x, y, z)	绘制(x,y,z)为节点的三维空间折线
plot3(x, y, z, s)	绘制(x,y,z)为节点的三维空间折线, 标记为s

表 3.2: 曲线的设定

颜色	标记	线性
b 蓝色(默认)	无标记(默认)	- 实线(默认)
g 绿色	.	: 虚线
r 红色	o	-. 点划线
c 青色	x	- 划线
m 洋红色	+	十字
y 黄色	*	星
k 黑色	s	方块
	d	菱形
	v	下三角形
	^	上三角形
	<	左三角形
	>	右三角形
	p	五角形
	h	六角形

例 3.1 $plot(x)$

```
x = sin(0:0.01:2*pi);  
plot(x) % 结果如图 3.1
```

例 3.2 $plot(x, y)$

```
x = 0:0.01:2*pi;  
y = sin(x);  
plot(x, y) % 结果如图 3.2
```

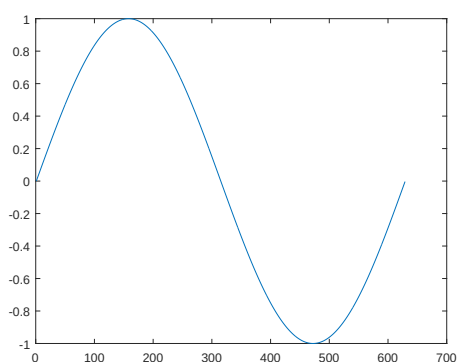


图 3.1: $plot(x)$ 绘制的正弦图像

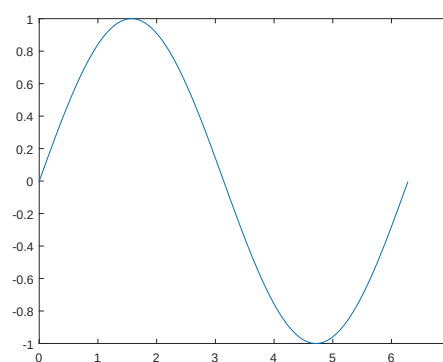


图 3.2: $plot(x, y)$ 绘制的正弦图像

例 3.3 $plot(x1, y1, x2, y2)$

```
x1 = 0:0.01:2*pi;  
y1 = sin(x1);  
y2 = cos(x1);  
plot(x1, y1, 'r-', x1, y2, 'b--') % 结果如图 3.3
```

注 3.1 预绘制多条曲线时，若不指定曲线的颜色，*MATLAB*将按表 3.2中的颜色顺序自动分配给每条曲线。

例 3.4 $fplot(fun, [a, b])$

```
y = 'sin(x)';  
fplot(y, [0, 2 * pi]) % 结果如图 3.4
```

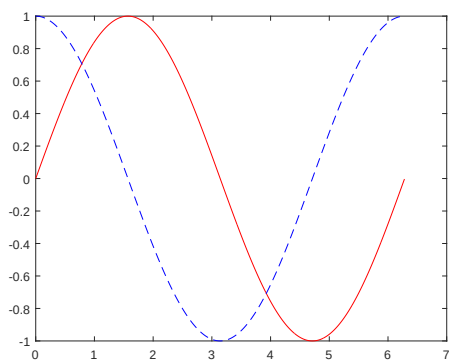


图 3.3: plot(x1, y1, x2, y2)绘制的正弦图像

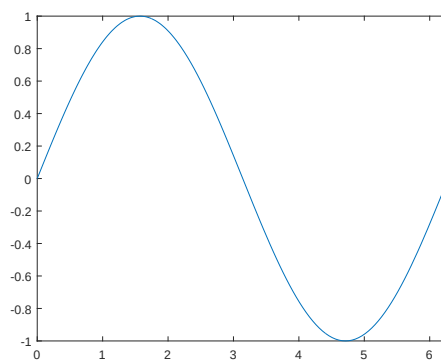


图 3.4: fplot(fun, [a, b])绘制的正弦图像

例 3.5 plot(x, y)的另一种用法

```
x = 0:0.01:2*pi;
y1 = sin(x);
y2 = cos(x);
y = [y1; y2];
plot(x, y) % 结果如图 3.5
text(3 + 0.1, sin(3), '$f(x) = \sin(x)$', 'interp', 'latex') % 标注图形, 详见本章第四节
text(5 + 0.1, cos(5), '$f(x) = \cos(x)$', 'interp', 'latex')
```

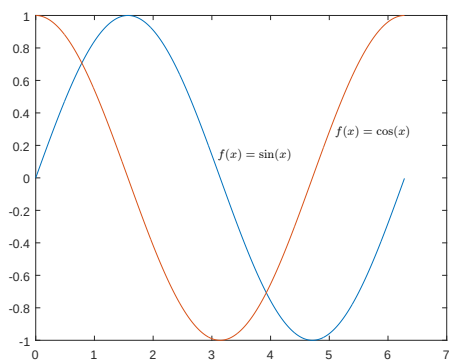


图 3.5: plot(x,y)的另一种用法

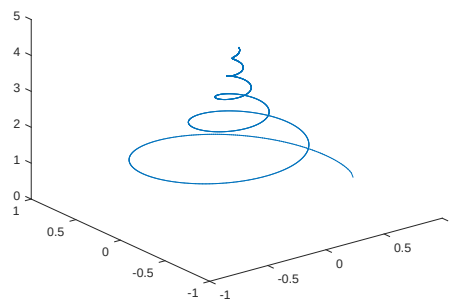


图 3.6: 螺旋线图像

例 3.6 绘制螺旋线, $0 \leq t \leq 20$ 时, 参变量函数为

$$\begin{cases} x = e^{-0.2t} \cos \frac{\pi}{2} t, \\ y = e^{-0.2t} \sin \frac{\pi}{2} t, \\ z = \sqrt{t}. \end{cases}$$


```

t = 0:0.01:20;
x = exp(-0.2 * t) .* cos(1/2 * pi * t);
y = exp(-0.2 * t) .* sin(1/2 * pi * t);
z = sqrt(t);
plot3(x, y, z)      % 结果如图 3.6

```

例 3.7 绘制 $[-3, 3]$ 上的分段函数, 要求 $y > 0$ 时标记为红色实线, $y < 0$ 时标记为黑色虚线, $y = 0$ 时标记为蓝圈。

$$y = \begin{cases} 1.1, & x > 1.1, \\ 1.1, & |x| \leq 1.1, \\ -1.1, & x < -1.1. \end{cases}$$

```

x = -3:0.01:3;
y = x;
y(x > 1.1) = 1.1;
y(x < -1.1) = -1.1;
plot(x(y < 0), y(y < 0), 'k:', ...
      x(y == 0), y(y == 0), 'bo', ...
      x(y > 0), y(y > 0), 'r-') % 结果如图 3.7

```

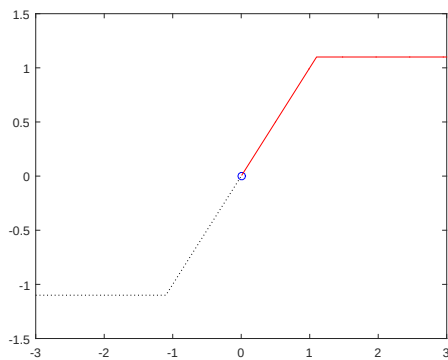


图 3.7: $[-3, 3]$ 上的分段函数

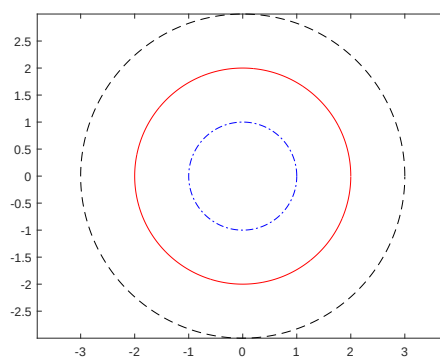


图 3.8: 同心圆图像

例 3.8 绘制圆心为原点的同心圆, 半径 $r = 1, 2, 3$ 。

```

t = 0:0.01:2*pi;
x = cos(t);
y = sin(t);
plot(x, y, 'b-.', 2 * x, 2 * y, 'r-', 3 * x, 3 * y, 'k--')
axis equal      % 将两坐标轴设置为等距模式, 结果如图 3.8

```

3.2 绘制曲面

MATLAB中绘制曲面图与绘制曲线图的用法类似，只是多了一步网格化过程(可使用meshgrid函数实现)，常用的绘制曲面的函数如表 3.3 所示。

表 3.3: 与绘制曲面有关的函数

函数	功能
[x,y] = meshgrid(xa, ya)	以xa和ya为基础生成X-Y平面上的网格数据
mesh(x, y, z)	绘制数据(x,y,z)的网面图，x,y,z应是同阶矩阵
mesh(x, y, z, c)	绘制数据(x,y,z)的网面图，颜色标记为c（c与z同阶）
mesh(z)	绘制数据z对应于其编址的网面
surf(x, y, z)	绘制曲面图，用法与mesh类似
contour(x, y, z)	绘制等高线图，用法与mesh类似
contour3(x, y, z)	绘制三维等高线图，用法与mesh类似

例 3.9 meshgrid的用法

```
>> xa = 1:3;
>> ya = 5:9;
>> [x,y] = meshgrid(xa, ya)
x =
     1     2     3
     1     2     3
     1     2     3
     1     2     3
     1     2     3
y =
     5     5     5
     6     6     6
     7     7     7
     8     8     8
     9     9     9
>> z = x + y;
>> mesh(x,y,z) % 结果如图 3.9
```

例 3.10 绘制马鞍面 $z = x^2 - y^2$, $x, y \in [-1, 1]$

```
xa = -1:0.05:1;
ya = xa;
[x, y] = meshgrid(xa, ya);
z = x.^2 - y.^2;
mesh(x, y, z) % 结果如图 3.10
```

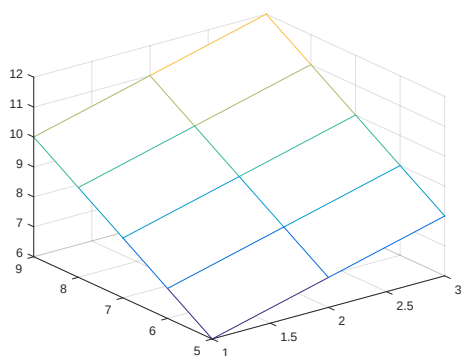


图 3.9: mesh绘制的曲面

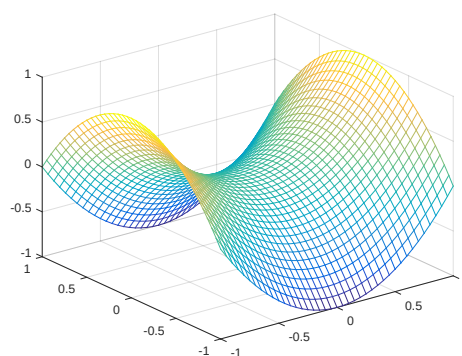


图 3.10: 马鞍面图像

例 3.11 绘制单位上半球面

解 方法一 利用参数形式

$$\begin{cases} x = \sin \phi \cos \theta, \\ y = \sin \phi \sin \theta, \\ z = \cos \phi, \end{cases} \quad 0 \leq \theta \leq 360^\circ, 0 \leq \phi \leq 90^\circ.$$

```
t = linspace(0, 360, 20); f = linspace(0, 90, 20);
[t, f] = meshgrid(t, f);
x = sind(f) .* cosd(t); y = sind(f) .* sind(t); z = cosd(f);
surf(x, y, z) % 结果如图 3.11
```

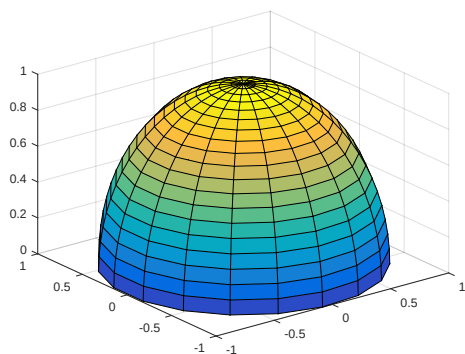


图 3.11: 参数形式的半球面

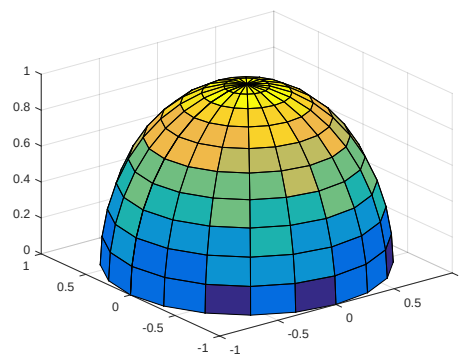


图 3.12: sphere生成的半球面

方法二 利用sphere函数, 生成球面数据

```
[x, y, z] = sphere; z = abs(z); surf(x, y, z) % 结果如图 3.12
```

例 3.12 绘制peaks曲面及其三维等高线

```
>> peaks % 结果如图 3.13
z = 3*(1-x).^2.*exp(-(x.^2) - (y+1).^2) ...
    - 10*(x/5 - x.^3 - y.^5).*exp(-x.^2-y.^2) ...
    - 1/3*exp(-(x+1).^2 - y.^2)
>> contour3(peaks); % 结果如图 3.14
```

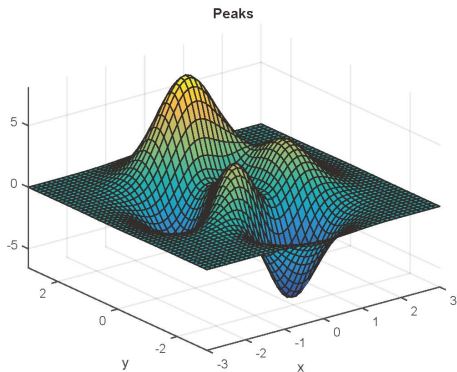


图 3.13: peaks曲面

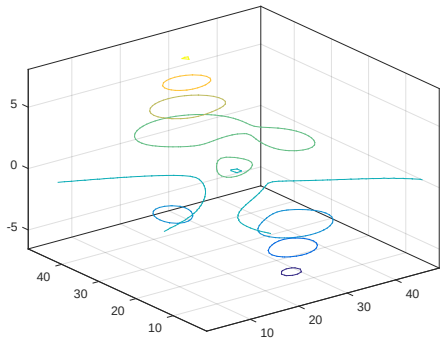


图 3.14: peaks的三维等高线

3.3 特殊图形的绘制

尽管我们常用的图形是曲线、曲面，但在数学统计、工程技术、地质勘探、物理研究中我们还会遇到表 3.4 中的特殊图像，掌握这些特殊函数对我们是大有帮助的。

表 3.4: 绘制特殊图像的函数

函数	图像	函数	图像	函数	图像
bar	垂直条形图	barh	水平条形图	stairs	阶梯图
pie	二维饼图	pie3	立体饼图	quiver	箭头图
contour	二维等高线图	contour3	三维等高线图	compass	罗盘图
comet	二维彗星图	comet3	三维彗星图	cylinder	柱状图
scatter	二维散点图	scatter3	三维散点图	ribbon	带状图
trimesh	三角形网格图	trisurf	三角形表面图	waterfall	瀑布图
stem	二维火柴杆图	stem3	三维火柴杆图	slice	切片图

注 3.2 其他的统计图详见第十四章。

例 3.13 条形图和饼图的绘制

```
subplot(3, 1, 1)
bar(rand(10, 5), 'stacked')
colormap(cool) % 颜色映射
subplot(3, 1, 2)
bar(0:.25:1, rand(5), 1)
subplot(3, 1, 3)
bar(rand(2, 3), .75, 'grouped') % 结果如图 3.15
```

注 3.3 *MATLAB*中小数可以简写，如0.75可以简写为.75。

```
pie([2, 4, 3, 5], {'North', 'South', 'East', 'West'}) % 如图 3.16
```

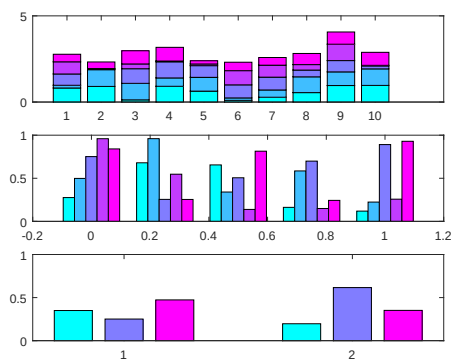


图 3.15: 条形图

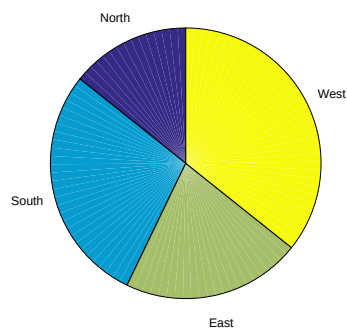


图 3.16: 饼图

例 3.14 彗星图和散点图的绘制

```
t = -pi:pi/200:pi;
comet(t, tan(sin(t)) - sin(tan(t))) % 结果如图 3.17
```

```
load seamount % 载入seamount.mat
scatter(x, y, 5, z) % 结果如图 3.18
```

注 3.4 读入`.mat`文件(*MATLAB*中的一种文件类型)可使用`load`函数，详见第七章。

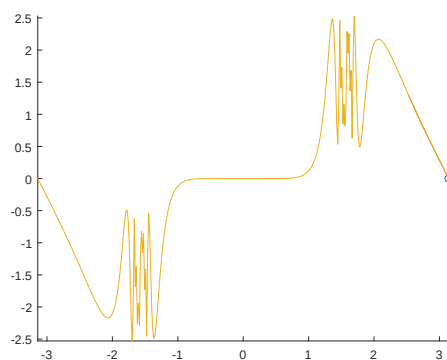


图 3.17: 彗星图

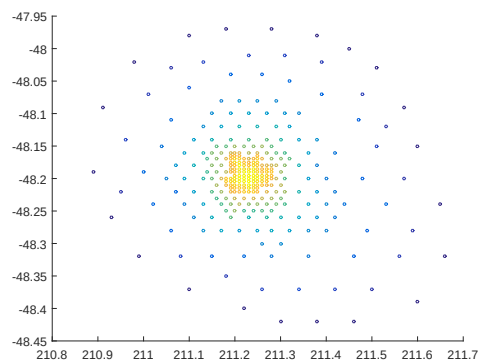


图 3.18: 散点图

例 3.15 网格图和阶梯图的绘制

```
[x, y] = meshgrid(1:15, 1:15);
tri = delaunay(x, y);
z = peaks(15);
trimesh(tri, x, y, z) % 结果如图 3.19
```

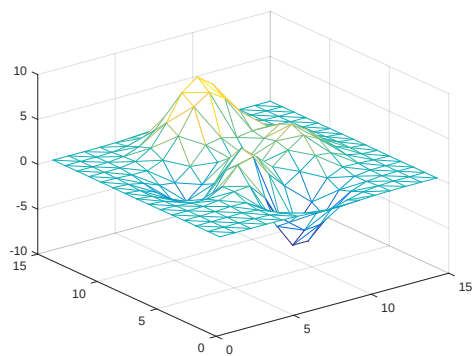


图 3.19: 网格图

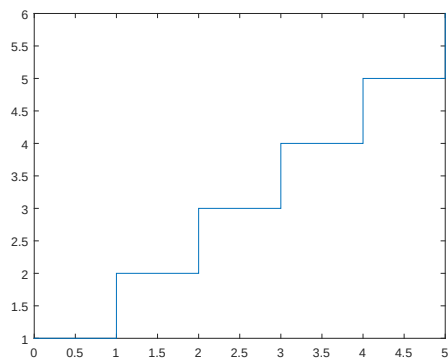


图 3.20: 阶梯图

```
x = 0:5;
y = x + 1;
stairs(x, y); % 结果如图 3.20
```

例 3.16 箭头图和罗盘图的绘制

```

[x, y] = meshgrid(-2:.2:2, -1:.15:1);
z = x .* exp(-x.^2 - y.^2);
[px,py] = gradient(z, .2, .15);
contour(x, y, z)
hold on
quiver(x, y, px, py) % 常用来绘制向量场
hold off
axis image % 使坐标框紧紧的环绕着图, 结果如图 3.21

```

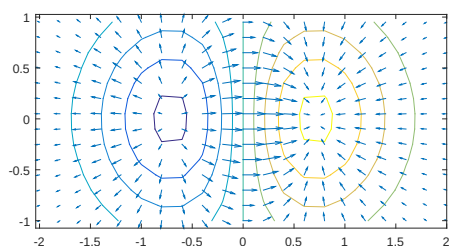


图 3.21: 箭头图

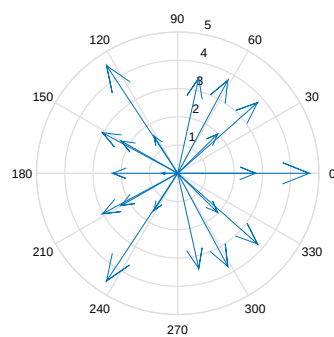


图 3.22: 罗盘图

```

z = eig(randn(20, 20)); % eig求特征值, randn生成标准正态分布数
compass(z) % 结果如图 3.22

```

例 3.17 柱状图和带状图的绘制

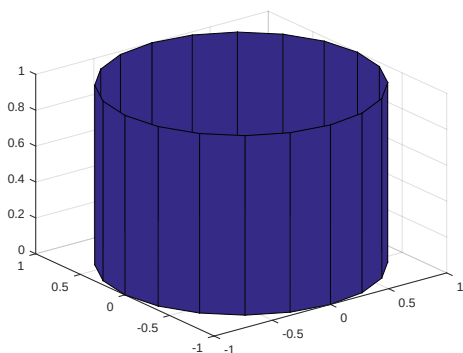


图 3.23: 柱状图

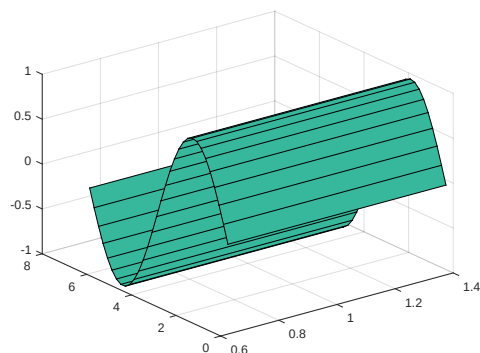


图 3.24: 带状图

```
cylinder % 结果如图 3.23
```

```
x = 0:0.2:2*pi;  
y = sin(x);  
ribbon(x,y) % 结果如图 3.24
```

例 3.18 瀑布图和切片图的绘制

```
[x, y, z] = peaks(30);  
waterfall(x, y, z) % 结果如图 3.25
```

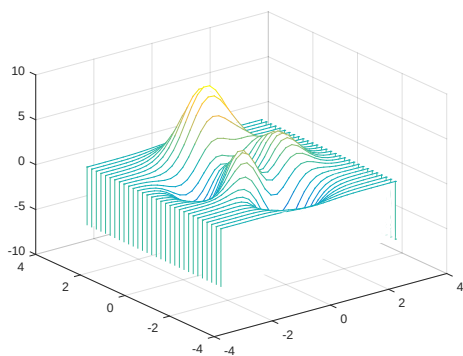


图 3.25: 瀑布图

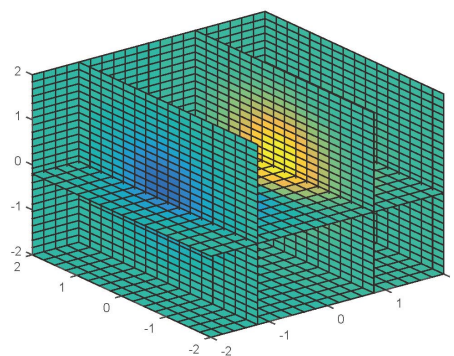


图 3.26: 切片图

```
[x, y, z] = meshgrid(-2:.2:2, -2:.25:2, -2:.16:2);  
v = x .* exp(-x.^2 - y.^2 - z.^2);  
slice(x, y, z, v, [-1.2 .8 2], 2, [-2 -.2]) % 结果如图 3.26
```

例 3.19 火柴杆图的绘制

```
x = 0:0.2:8;  
y = sin(x);  
stem(x, y) % 结果如图 3.27
```

```
x = linspace(0, 1, 10);  
y = x ./ 2;  
z = sin(x) + cos(y);  
stem3(x, y, z, 'fill')  
view(-25, 30) % 结果如图 3.28
```

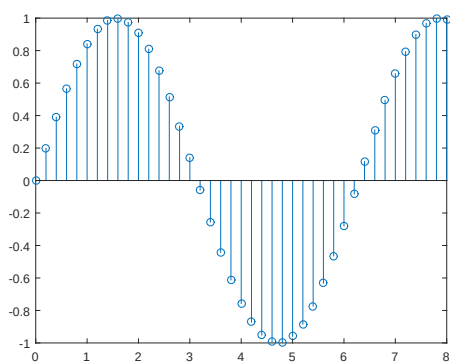



图 3.27: 火柴杆图

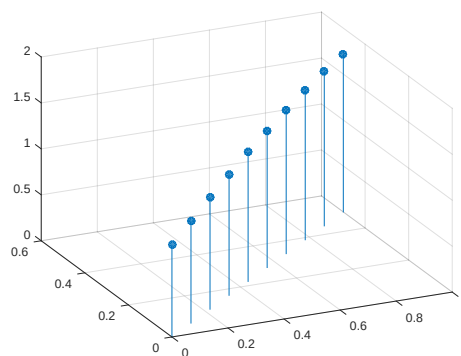


图 3.28: 三维火柴杆图

3.4 图形标注

3.4.1 常用的图形标注

一个图像除了必要的曲线或曲面外，还很有必要为其添加各种标注，以丰富图像的说服力。在MATLAB中可以很容易地添加标注，具体的命令如表 3.5 所示。

例 3.20 图像标注

```
t = linspace(0, 2*pi, 100);
r = 2 * (1 + sin(t - pi));
x = r .* cos(t); y = r .* sin(t);
plot(x, y, 'r'); % 画心形线
fill(x, y, 'r'); % 以红色填充(x,y)围成的区域
axis([-3.1, 3.6, -4.1, 0.6]); % 定制坐标轴
legend('爱你'); % 添加图例
a = 3.1 * ones(1,7); b = linspace(-0.5, -3.9, 7);
txt = {'你'; '的'; '月'; '亮'; '我'; '的'; '心'};
text(a, b, txt); % 结果如图 3.29
```

例 3.21 在一幅图中绘制正 n 边形($n = 3, 4, 5, 6$)。

解 方法一 使用subplot函数

```
txt = {'三角形', '四边形', '五边形', '六边形'};
for i = 1:4
    t = (0.5:2/(2+i):2.5) * pi;
    x = cos(t); y = sin(t);
    subplot(2, 2, i); plot(x, y); title(txt{i}); % 绘制第i个子图并添加标题
```

表 3.5: 常用的标注命令

命令	说明
title(str)	添加名为str的图片标题
xlabel, ylabel, zlabel	标注坐标轴x, y, z, 用法类似于title
text(x, y, str)	在(x, y)处添加文本str
text(x, y, z, str)	在(x, y, z)处添加文本str
gtext(str)	在鼠标选中图形中的位置, 添加标注
gtext(str1, str2, ...)	依次在鼠标选中的位置添加标注
legend(str1, str2, ...)	添加图例str1, str2, ...
hold on/off	保持/释放现有图形
grid on/off	显示/不显示网格
box on/off	显示/不显示坐标框
axis on/off	显示/不显示坐标轴
figure/figure(i)	新建窗口/将第i个窗口设为当前窗口
subplot(n, m, k)	将图形窗口分成 $n \times m$ 个子图, 并指向第k个子图
close/close all	关闭窗口/关闭所有窗口
axis([a, b, c, d])	定制坐标轴范围: $x \in [a, b]$, $y \in [c, d]$
axis([a, b, c, d, e, f])	定制坐标轴范围, 其中 $z \in [e, f]$
axis equal/axis image	坐标轴等距显示/以图片格式显示
view(n)	$n = 2$ 时为二维视图, $n = 3$ 时为三维视图
view([x, y])/view([x, y, z])	以(x, y)/(x, y, z)为视角, 二维视图/三维视图

```
ylim([-1.2, 1.2]); axis equal;
end
```

% 坐标轴等距显示, 结果如图 3.30

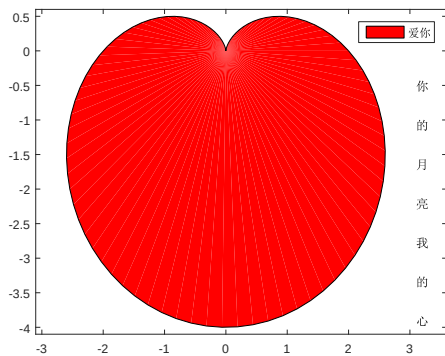
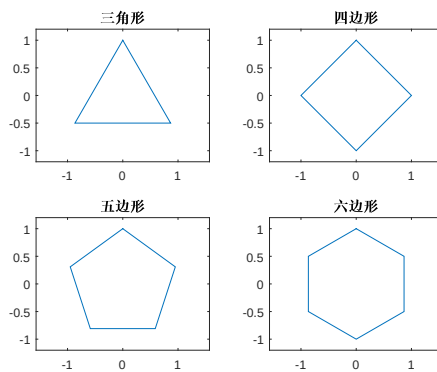


图 3.29: 心形线的标注

图 3.30: subplot子图上的正 n 边形

方法二 使用hold on

```
txt = {'三角形', '四边形', '五边形', '六边形'};
mk = {'b-^', 'm-s', 'r-p', 'k-h'};
for i = 1:4
    hold on
    t = (0.5:2/(2+i):2.5) * pi;
    x = cos(t); y = sin(t);
    plot(x, y, char(mk(i)));
end
ylim([-1.2, 1.2]); axis equal
hold off
legend(txt, 'location', 'best') % 结果如图 3.31
```

例 3.22 绘制 $[-3.5, 3.5]$ 上的标准正态分布概率密度曲线并标明公式，最大值处标记红“+”号。

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

```
x = -3.5:0.01:3.5;
y = exp(-x.^2 / 2) / sqrt(2 * pi);
ym = max(y);
xm = x(ym == y);
plot(x, y, 'b-', xm, ym, 'r+')
ylim([0, 0.45])
% 下面的代码使用了LaTeX公式，详见下一部分一标注LaTeX公式
text(2, 0.3, ['$$f(x)=\frac{1}{\sqrt{2\pi}}e^{-\frac{x^2}{2}}$$'], ...
    'interpreter', 'latex'); % 结果如图 3.32
```

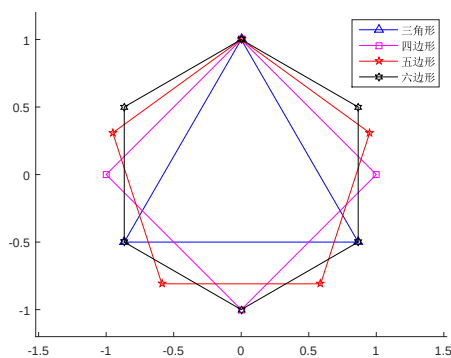


图 3.31: 同一图上的正 n 边形

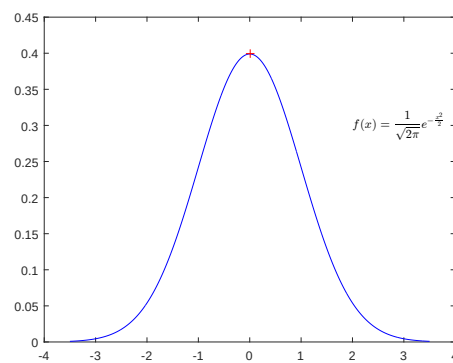


图 3.32: 标准正态分布概率密度曲线

3.4.2 标注LaTeX公式

MATLAB图形窗口中可使用LaTeX格式进行标注，其格式为：

```
text(x, y, '$$公式字符串 $$', 'interpreter', 'latex')
```

常用的LaTeX字符见表 3.6。

表 3.6: LaTeX字符

符号	字符	符号	字符	符号	字符	符号	字符
α	<code>\alpha</code>	β	<code>\beta</code>	γ	<code>\gamma</code>	δ	<code>\delta</code>
ϵ	<code>\epsilon</code>	ε	<code>\varepsilon</code>	ζ	<code>\zeta</code>	η	<code>\eta</code>
θ	<code>\theta</code>	ϑ	<code>\vartheta</code>	ι	<code>\iota</code>	κ	<code>\kappa</code>
λ	<code>\lambda</code>	μ	<code>\mu</code>	ν	<code>\nu</code>	ξ	<code>\xi</code>
π	<code>\pi</code>	ϖ	<code>\varpi</code>	ρ	<code>\rho</code>	ϱ	<code>\varrho</code>
σ	<code>\sigma</code>	ς	<code>\varsigma</code>	τ	<code>\tau</code>	υ	<code>\upsilon</code>
ϕ	<code>\phi</code>	φ	<code>\varphi</code>	χ	<code>\chi</code>	ψ	<code>\psi</code>
ω	<code>\omega</code>	Γ	<code>\Gamma</code>	Δ	<code>\Delta</code>	Θ	<code>\Theta</code>
Λ	<code>\Lambda</code>	Ξ	<code>\Xi</code>	Π	<code>\Pi</code>	Σ	<code>\Sigma</code>
Υ	<code>\Upsilon</code>	Φ	<code>\Phi</code>	Ψ	<code>\Psi</code>	Ω	<code>\Omega</code>
$\aleph$	<code>\aleph</code>	$\hbar$	<code>\hbar</code>	$\imath$	<code>\imath</code>	$\jmath$	<code>\jmath</code>
ℓ	<code>\ell</code>	$\wp$	<code>\wp</code>	$\Re$	<code>\Re</code>	$\Im$	<code>\Im</code>
$\prime$	<code>\prime</code>	$\emptyset$	<code>\emptyset</code>	$\angle$	<code>\angle</code>	∞	<code>\infty</code>
∂	<code>\partial</code>	∇	<code>\nabla</code>	$\triangle$	<code>\triangle</code>	$\forall$	<code>\forall</code>
$\exists$	<code>\exists</code>	$\neg$	<code>\neg</code>	$\surd$	<code>\surd</code>	$\top$	<code>\top</code>
$\bot$	<code>\bot</code>	$\backslash$	<code>\backslash</code>	$\langle$	<code>\langle</code>	$\rangle$	<code>\rangle</code>
$\lceil$	<code>\lceil</code>	$\rceil$	<code>\rceil</code>	$\lfloor$	<code>\lfloor</code>	$\rfloor$	<code>\rfloor</code>
$\leq$	<code>\leq</code>	$\geq$	<code>\geq</code>	$\ll$	<code>\ll</code>	$\gg$	<code>\gg</code>
$\subset$	<code>\subset</code>	$\supset$	<code>\supset</code>	$\subseteq$	<code>\subseteq</code>	$\supseteq$	<code>\supseteq</code>
$\in$	<code>\in</code>	$\ni$	<code>\ni</code>	$\vdash$	<code>\vdash</code>	$\dashv$	<code>\dashv</code>
$\equiv$	<code>\equiv</code>	$\approx$	<code>\approx</code>	$\neq$	<code>\neq</code>	$\sim$	<code>\sim</code>
$ $	<code>\mid</code>	$\parallel$	<code>\parallel</code>	$\cong$	<code>\cong</code>	$\simeq$	<code>\simeq</code>
$\pm$	<code>\pm</code>	$\mp$	<code>\mp</code>	$\times$	<code>\times</code>	$\div$	<code>\div</code>
$\uparrow$	<code>\uparrow</code>	$\downarrow$	<code>\downarrow</code>	$\Uparrow$	<code>\Uparrow</code>	$\Downarrow$	<code>\Downarrow</code>
$\leftarrow$	<code>\leftarrow</code>	$\rightarrow$	<code>\rightarrow</code>	$\Leftarrow$	<code>\Leftarrow</code>	$\Rightarrow$	<code>\Rightarrow</code>
$\pounds$	<code>\pounds</code>	$\cdot$	<code>\cdot</code>	$\diamond$	<code>\diamond</code>	$\leftrightarrow$	<code>\leftrightarrow</code>
$\oslash$	<code>\oslash</code>	$\copyright$	<code>\copyright</code>	$\S$	<code>\S</code>	$\Leftrightarrow$	<code>\Leftrightarrow</code>
$\clubsuit$	<code>\clubsuit</code>	$\heartsuit$	<code>\heartsuit</code>	$\spadesuit$	<code>\spadesuit</code>	$\diamondsuit$	<code>\diamondsuit</code>

注 3.5 标注数学公式求和与连乘时需在公式前加`\displaystyle`表示输出标准公式。*MATLAB*中函数`texlabel`可将字符表达式转换为`TeX`格式字符串，标注`LaTeX`公式时需要将参数`interpreter`的值设置

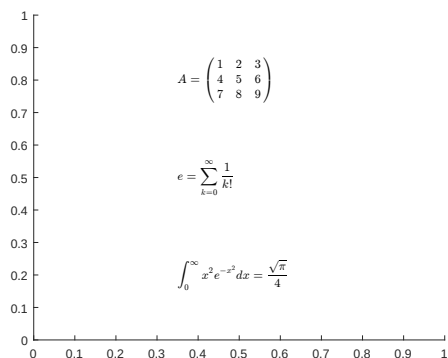
为 $LaTeX$ ，并且公式符合 $LaTeX$ 格式才能标注，否则 $MATLAB$ 会以字符串形式标注并输出警告，常用的 $LaTeX$ 数学公式详见表 3.7。

表 3.7: 常用数学公式

符号	字符	符号	字符	符号	字符
x_k	<code>x_{k}</code>	x^n	<code>x^{n}</code>	$\underline{abc}$	<code>\underline{abc}</code>
$\sqrt{x}$	<code>\sqrt{x}</code>	$\sqrt[n]{x}$	<code>\sqrt[n]{x}</code>	$\overline{abc}$	<code>\overline{abc}</code>
$\hat{x}$	<code>\hat{x}</code>	$\bar{x}$	<code>\bar{x}</code>	$\overleftarrow{abc}$	<code>\overleftarrow{abc}</code>
$\vec{x}$	<code>\vec{x}</code>	$\tilde{x}$	<code>\tilde{x}</code>	$\overrightarrow{abc}$	<code>\overrightarrow{abc}</code>
$\dot{x}$	<code>\dot{x}</code>	$\ddot{x}$	<code>\ddot{x}</code>	$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	<code>\matrix{1&2\cr3&4}</code>
$\widetilde{abc}$	<code>\widetilde{abc}</code>	$\oint_{\Omega}$	<code>\oint_{\Omega}</code>	$\sum_{i=1}^n x_i$	<code>\sum_{i=1}^n x_i</code>
$\widehat{abc}$	<code>\widehat{abc}</code>	$\frac{a}{b}$	<code>\frac{a}{b}</code>	$\prod_{i=1}^n x_i$	<code>\prod_{i=1}^n x_i</code>

例 3.23 标注积分方程、矩阵和 e 的定义式

```
text(0.35, 0.2, ['$\int_0^{\infty} x^2 e^{-x^2} dx = \frac{\sqrt{\pi}}{4}$'], ...
    'interpreter', 'latex'); % 积分方程
text(0.35, 0.8, ['$A = \left( \begin{matrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{matrix} \right)$'], ...
    'interpreter', 'latex'); % 矩阵
text(0.35, 0.5, ['$e = \sum_{k=0}^{\infty} \frac{1}{k!}$'], ...
    'interpreter', 'latex'); % e的定义式，结果如图 3.33
```

图 3.33: 标注积分方程、矩阵和 e 的定义式

$$y = \frac{1}{1 + \frac{1}{1+z}}$$

$$\text{cov}(x, y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

图 3.34: 标注多重分数和协方差公式

例 3.24 标注多重分数和协方差公式

```
axis off
str = ['$y=\frac{1}{1+\frac{1}{1+x}}$'];
text(0.35, 0.85, str, 'interpreter', 'latex'); % 多重分数
str = ['$\mathrm{cov}(x,y)=\frac{\sqrt{\sum_{i=1}^n(x_i-\overline{x})(y_i-\overline{y})}}{\sqrt{\sum_{i=1}^n(x_i-\overline{x})^2}}$'];
text(0.2, 0.35, str, 'interpreter', 'latex'); % 协方差公式, 如图 3.34
```

3.4.3 Legend和Annotation

除了字符注释外，MATLAB还提供了图例和注释类型。其中，图例可用legend函数(详见表 3.8)创建，注释类型可由annotation函数(详见表 3.10)设定。

表 3.8: legend图例

命令	说明
legend(str1, str2, ...)	添加图例str1, str2, ...
legend('off')	移除当前图形中的图例
legend('toggle')	创建或移除当前图形中的图例
legend('hide')/legend('show')	隐藏/显示当前图形中的图例
legend('boxoff')/legend('boxon')	移除/显示当前图形中图例的方框
legend(..., 'location', LOC)	设置图例位置

表 3.9: LOC有效字符串

字符串	说明	字符串	说明
North	坐标轴内顶部	NorthEast	坐标轴内顶部右方
South	坐标轴内底部	NorthWest	坐标轴内顶部左方
East	坐标轴内右部	SouthEast	坐标轴内底部右方
West	坐标轴内左部	SouthWest	坐标轴内底部左方
NorthOutside	坐标轴外顶部	NorthEastOutside	坐标轴外顶部右方
SouthOutside	坐标轴外底部	NorthWestOutside	坐标轴外顶部左方
EastOutside	坐标轴外右部	SouthEastOutside	坐标轴外底部右方
WestOutside	坐标轴外左部	SouthWestOutside	坐标轴外底部左方
Best	坐标轴内最佳位置	BestOutside	坐标轴外最佳位置

注 3.6 LOC的格式为[l, d, w, h]或表 3.9 中的字符串，其中l表示距界面左端的距离，d表示距界面底端的距离，w表示图例框的宽度，h表示图例框的高度。实际上，MATLAB中很多位置都是依照这种“左底宽高”的方式设定的，希望读者融会贯通。

表 3.10: annotation注释

命令	说明
annotation(type)	添加类型为type的注释, type = {line(线条); arrow (箭头); doublearrow (双箭头); textarrow(文本箭头); textbox(文本框); ellipse (椭圆); rectangle(矩形)}
annotation(type, LOC)	注释位置为LOC, type = {textbox; ellipse; rectangle}
annotation(type, x, y)	添加从(x(1),y(1))到(x(2),y(2))的注释, type = {line; arrow; doublearrow; textarrow}
annotation(type, x, y, 'string', str)	添加文本为str的注释, type = {textarrow; textbox}

例 3.25 在标准坐标轴下绘制椭圆, $(x, y) = (4 \cos \theta, 3 \sin \theta)$, $0 \leq \theta \leq 2\pi$ 。

```
hold on
t = 0:0.01:2*pi;
x = 4 * cos(t); y = 3 * sin(t);
axis equal off; % 等刻度显示, 并隐藏坐标轴
axis([-4.5, 4.5, -4.5, 4.5]); % 定制坐标轴
annotation('arrow', [0.05,0.95], [0.515,0.515]); % 绘制x轴
annotation('arrow', [0.515,0.515], [0.05,0.95]); % 绘制y轴
text( 5, -0.5, 'x轴'); % 标注x轴
text(-1.2, 4.5, 'y轴'); % 标注y轴
str = '$$\frac{x^2}{16}+\frac{y^2}{9}=1$'; % 椭圆公式
text(0, -0.5, '$$\it{0}$$', 'interpreter', 'latex'); % 原点
plot(x, y);
h = legend(str, 'location', 'Best'); % 添加图例, Best表示自动在最佳位置进行标注
set(h, 'interpreter', 'latex') % set设置
hold off % 结果如图 3.35
```

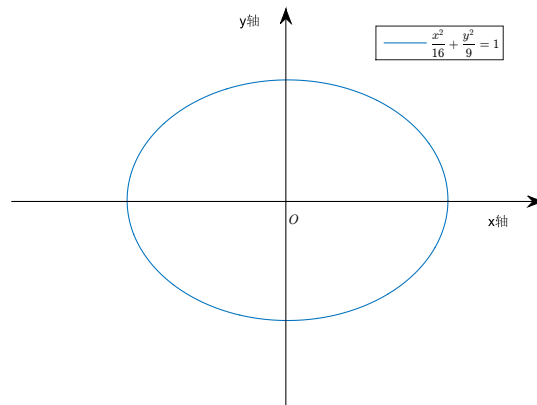


图 3.35: 标准坐标系下绘制椭圆

3.5 图形编辑器

MATLAB图形窗口、坐标轴、图形颜色等往往是系统自动选定的，当然也可以用相关的命令来设置，详见第二十二章。MATLAB图形编辑器给我们提供了更为简单的操作方法。空白的图形窗口(Figure)如图 3.36 所示。

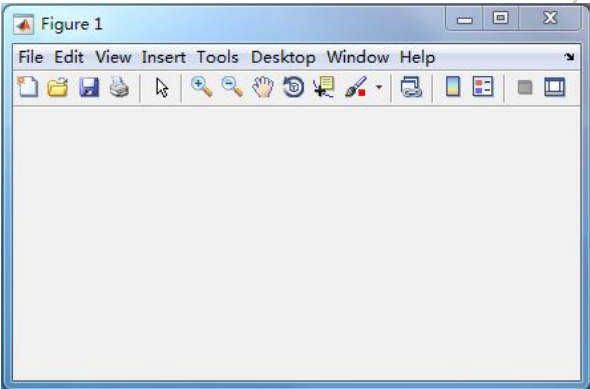


图 3.36: 空白的Figure界面

X Label
Y Label
Z Label
Title
Legend
Colorbar
Line
Arrow
Text Arrow
Double Arrow
TextBox
Rectangle
Ellipse
Axes
Light

图 3.37: Insert菜单

Edit Plot
Zoom In
Zoom Out
Pan
Rotate 3D
Data Cursor
Brush
Link
Reset View
Options
Pin to Axes
Snap To Layout Grid
View Layout Grid
Smart Align and Distribute
Align Distribute Tool ...
Align
Distribute
Brushing
Basic Fitting
Data Statistics

图 3.38: Tools菜单栏

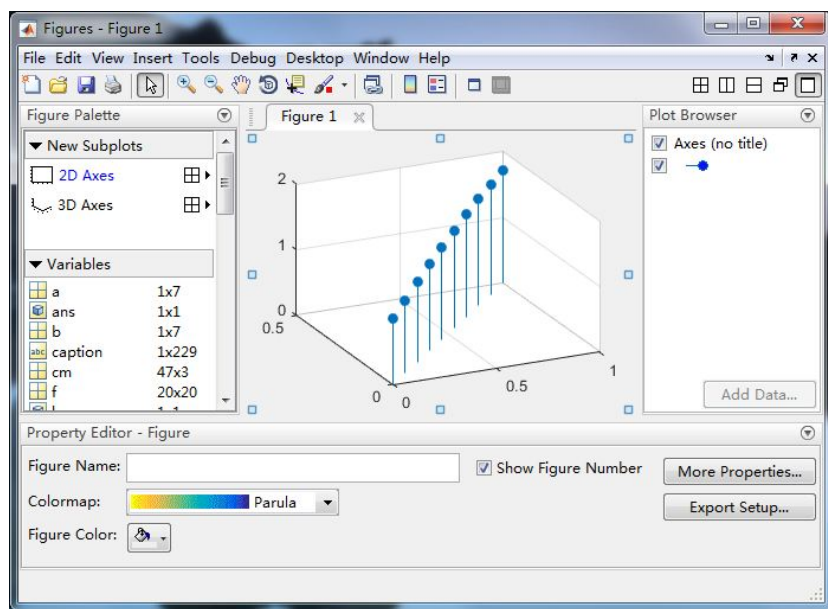


图 3.39: 图形编辑器界面

窗口上有菜单和工具条，其中包括通用的文件操作命令、编辑命令，对图形的坐标轴、线型等特性进行设置的专用工具，还可以为图形添加标注。

1. 点击菜单栏Insert，如图 3.37 所示，可设置坐标轴标注(Label)，标题(Title)，图例(Legend)，颜色条(Colorbar)，注释类型，坐标轴(Axes)，灯光(Light)；

2. 点击菜单栏Tools，如图 3.38 所示，可设置编辑窗口(Edit Plot)，放大(Zoom In)，缩小(Zoom Out)，拖动(Pan)，3D旋转(Rotate 3D)，数据点(Data Cursor)，选中曲线上的点(Brush，可右击选中Remove以删除选中的点)，边界等；

3. 工具栏按钮从左往右可实现新建图形窗口，打开.fig文件，保存图形，输出图形，编辑，放大，缩小，拖动，3D旋转，显示数据点，选中曲线上的点，Link Plot，添加颜色条，添加图例等功能，最右端的按钮可图形编辑器界面，如图 3.39 所示。在图形编辑器里，我们可以根据需要来设置各子窗口的布局，点击菜单栏View可以设置子窗口是否显示。

3.6 实验报告

1. 在同一直角坐标系中作出三条曲线：

$$y = x \sin \frac{1}{x}, \quad y = x, \quad y = -x, \quad x \in [-\pi, \pi].$$

```
x = -pi:0.01:pi;
a = x .* sin(1 ./ x);
```

```

b = x;
c = -x;
plot(x, [a; b; c]);
xlabel('x轴');
ylabel('y轴');
title('三条曲线对比');
text(x(end-10), b(end), '$y=x$', 'interpreter', 'latex');
text(x(end-10), c(end), '$y=-x$', 'interpreter', 'latex');
text(x(end-30), a(end), '$y=x\sin\frac{1}{x}$', ...
    'interpreter', 'latex'); % 在曲线末尾处添加标记, 结果如图 3.40

```

2. 画出参数曲线:

$$\begin{cases} x = a(\cos t)^3, \\ y = a(\sin t)^3. \end{cases}$$

```

t = -pi:0.01:pi;
a = 1;
x = a .* (cos(t)) .^ 3;
y = a .* (sin(t)) .^ 3;
plot(x, y) % 结果如图 3.41

```

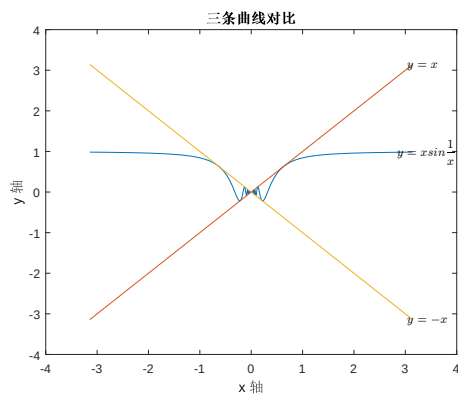


图 3.40: 三条曲线对比

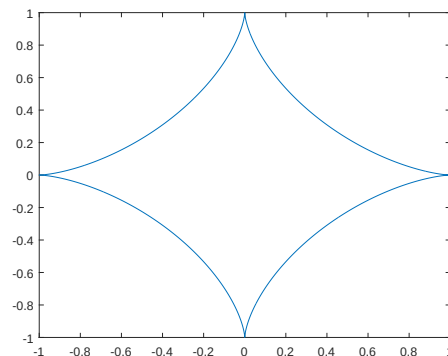


图 3.41: 星形线

3. 用subplot函数在6个子窗口中分别画6条曲线:

$$f_i(x) = \cos(i * x), \quad i = 1, 2, \dots, 6, \quad -3\pi \leq x \leq 3\pi.$$

```

x = -3*pi:0.01:3*pi;
for i = 1:6
    y = cos(i * x);
    subplot(3, 2, i)
    plot(x, y);
    title(['i=', num2str(i)])
end % 结果如图 3.42

```

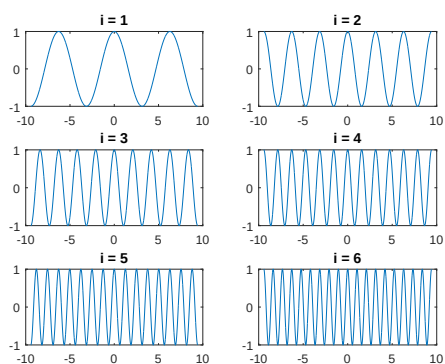


图 3.42: 三角函数子图

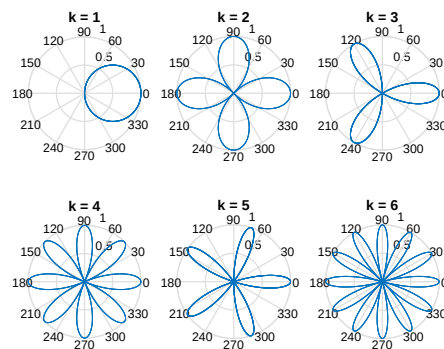


图 3.43: 极坐标子图

4. 用subplot命令在6个子窗口中分别画6条极坐标曲线:

$$\rho = \cos(k * x), \quad k = 1, 2, \dots, 6.$$

```
x = -3*pi:0.01:3*pi;
for k = 1:6
    y = cos(k * x);
    subplot(2, 3, k); polar(x, y);
    title(['k_□', num2str(k)])
end
% 结果如图 3.43
```

5. 作出 $t \in [0, 2\pi]$ 上的3-D曲线:

$$\begin{cases} x = \sin t, \\ y = \cos t, \\ z = \cos 2t. \end{cases}$$

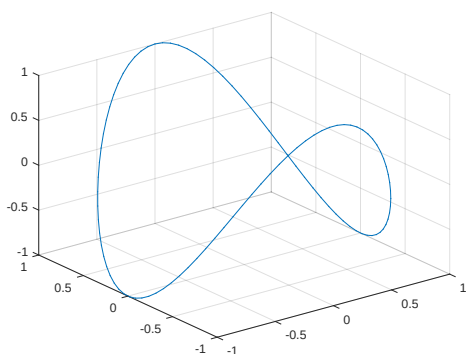


图 3.44: 三维曲线

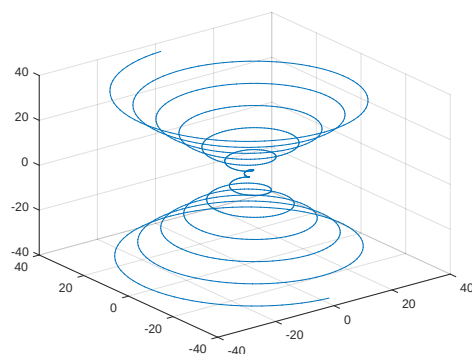


图 3.45: 三维螺旋线

```
t = linspace(0, 2*pi, 100);
x = sin(t);
y = cos(t);
z = cos(2 .* t);
plot3(x, y, z);
grid on % 结果如图 3.44
```

6. 作出 $t \in [-12\pi, 12\pi]$ 上的三维螺旋线:

$$\begin{cases} x = t \sin t, \\ y = t \cos t, \\ z = t. \end{cases}$$

```
t = linspace(-12*pi, 12*pi, 1000);
x = t .* sin(t);
y = t .* cos(t);
z = t;
plot3(x, y, z);
grid on % 结果如图 3.45
```

7. 作出下列曲面: (提示: 可用 mesh, surf, contour3, surfc)

(1) 抛物面 $z = \frac{x^2}{3} + \frac{y^2}{2}$,

(2) 旋转曲面 $z = \frac{\sin \sqrt{x^2 + y^2}}{\sqrt{x^2 + y^2}}$,

(3) 圆锥面 $z^2 = x^2 + y^2$,

(4) 旋转曲面 $\begin{cases} x = (3 + 0.3 \cos u) \cos v, \\ y = (3 + 0.3 \cos u) \sin v, \\ z = 0.3 \sin v. \end{cases}$

```
%% (1)
x = -1:0.1:1; y = x;
[x, y] = meshgrid(x, y);
z = x.^ 2 / 3 + y.^ 2 / 2;
surfc(x, y, z) % 结果如图 3.46
%% (2)
x = -1:0.1:1; y = x;
[x, y] = meshgrid(x, y);
r = sqrt(x.^ 2 + y.^ 2); z = sin(r) ./ r;
surf(x, y, z) % 结果如图 3.47
%% (3)
x = -10:0.1:10; y = -10:0.1:10;
[x, y] = meshgrid(x, y);
z = sqrt(x.^ 2 + y.^ 2);
mesh(x, y, z);
hold on
z = -z;
contour3(x, y, z); % 结果如图 3.48
%% (4)
ua = -10:0.1:10; va = -10:0.1:10;
[u, v] = meshgrid(ua, va);
```

```

x = (3 + 0.3 * cos(u)) .* cos(v);
y = (3 + 0.3 * cos(u)) .* sin(v);
z = 0.3 .* sin(v);
mesh(x, y, z) % 结果如图 3.49

```

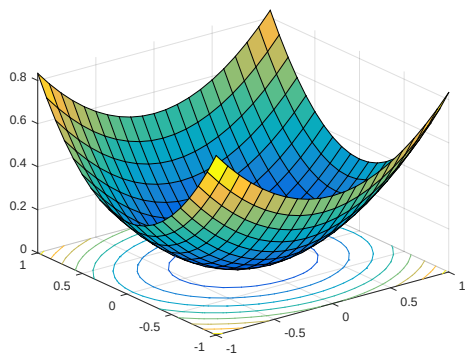


图 3.46: 抛物面

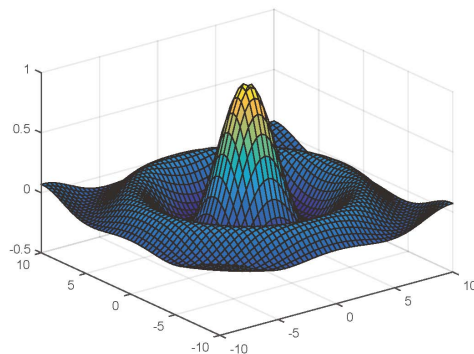


图 3.47: 旋转曲面

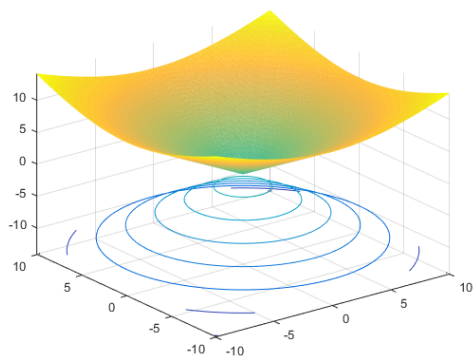


图 3.48: 圆锥面

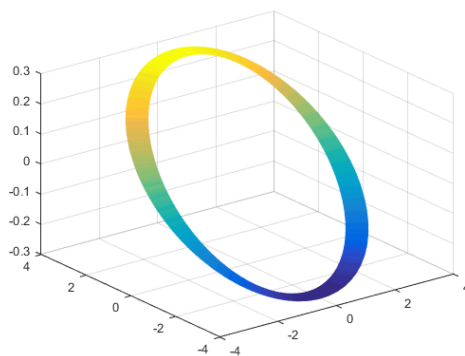


图 3.49: 旋转曲面

8. 考虑一个简单的实际问题

(1) 某型号大炮可射出速度为 $v = 100\text{m/s}$ 的炮弹，发射角度 α 可调，试讨论 α 多大时，射程最大？请画出此时的轨迹图 ($g = 9.8\text{m/s}^2$ ，角度精确到 1°)。

(2) 为提高射程，将大炮至于高为 $h = 500\text{m}$ 的山上，试讨论 α 多大时，射程最大？请画出此时的轨迹图 (角度精确到 1°)。

解 对速度 v 进行分解，可得

$$\begin{cases} v_x = v \cos \alpha, \\ v_y = v \sin \alpha. \end{cases}$$

于是，以时间 t 为参数的运动轨迹方程可统一写为

$$\begin{cases} x = v_x t, \\ y = v_y t - \frac{1}{2} g t^2 + h. \end{cases}$$

炮弹落地($y = 0$)的时间为

$$T = \frac{v_y + \sqrt{v_y^2 + 2gh}}{g}.$$

此时，射程为

$$S = v_x T.$$

编写函数如下：

```
function s = myrange(v, alpha, h)
% 计算炮弹射程s
% Input -v 速度 -alpha 发射角，角度制 -h 发射处的高度
% Output -s 射程
g = 9.8;
vx = v * cosd(alpha);
vy = v * sind(alpha);
T = (vy + sqrt(vy.^2 + 2 * g * h)) / g;
s = vx .* T;
```

注 3.7 *sind*、*cosd*函数的输入参数是角度制的，它们与*sin*、*cos*函数功能一致，其他相应的三角函数也有这种形式。

```
function trackplot(v, alpha, s, h)
% 绘制轨迹图
% Input -v 速度 -alpha 发射角，角度制 -s 射程 -h 发射处的高度
g = 9.8;
x = 1:s;
vx = v * cosd(alpha);
vy = v * sind(alpha);
t = x / vx;
y = vy * t - 1/2 * g * t.^2 + h;
plot(x, y)
xlim([0, s]) % 控制x轴范围
```

(1)

```
>> v = 100;
>> alpha = 1:90;
>> h = 0;
>> s = myrange(v, alpha, h);
>> [S, a] = max(s) % 注意alpha的值，因此a便是最佳发射角
S =
    1.0204e+03
```

```
a =  
    45  
>> trackplot(v, a, S, h)    % 结果如图 3.50
```

故发射角 $\alpha = 45^\circ$ ，射程 $S = 1020.4m$ 。

(2)

```
>> v = 100;  
>> alpha = 1:90;  
>> h = 500;  
>> s = myrange(v, alpha, h);  
>> [S, a] = max(s)    % 注意alpha的值，因此a便是最佳发射角  
S =  
    1.4358e+03  
a =  
    35  
>> trackplot(v, a, S, h)    % 结果如图 3.51
```

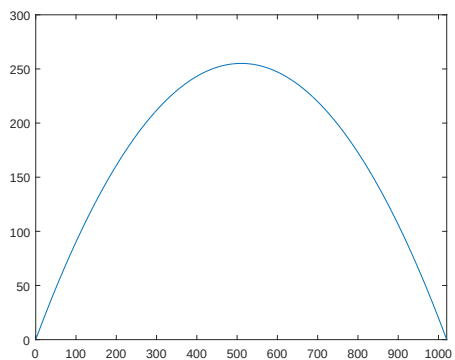


图 3.50: 在山下发射时炮弹轨迹

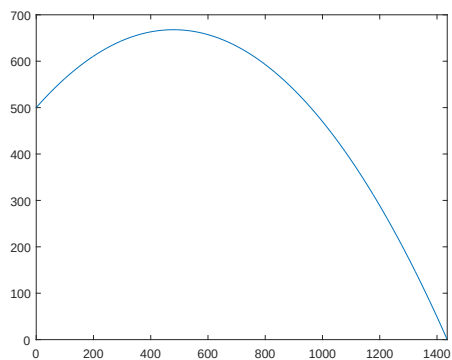


图 3.51: 在山上发射时炮弹轨迹

故发射角 $\alpha = 35^\circ$ ，射程 $S = 1435.8m$ 。

注 3.8 该题目较为简单，可用其他思路解决。编者认为思路分析和程序实现都很重要，思路分析得越清楚，程序实现就越简单且不易出错。尤其对复杂算法来说，思路分析特别重要，希望读者思路分析清楚后再编写程序。

我的 MATLAB 报告

王保山

第四章 矩阵代数

本章介绍MATLAB有关线性代数运算的指令，比如矩阵除法、解线性方程组、矩阵分解等。

表 4.1: 矩阵代数的主要指令

函数	功能	函数	功能
zeros	生成全零矩阵	reshape	矩阵重排
ones	生成全一矩阵	rank	矩阵的秩
eye	生成单位矩阵	det	方阵的行列式
rand	生成[0, 1]上均匀分布的矩阵	inv	矩阵的逆
trace	方阵的迹	eig	特征值与特征向量
diag	对角阵	rref	矩阵的行最简形式
tril	取矩阵下三角	orth	正交规范化
triu	取矩阵上三角	null	求基础解系
flipud	矩阵上下翻转	norm	矩阵或向量范数
fliplr	矩阵左右翻转	jordan	约当标准行分解

4.1 矩阵代数的指令

矩阵运算符在第一章已经讲过，这里不再重复，注意左除运算与右除运算、矩阵运算与数组运算之间的区别。

例 4.1 矩阵乘除法与数组乘除法

```
>> a = [1 2; 3 4];
>> b = [5 6; 7 8];
>> c = a * b
c =
    19    22
    43    50
>> d = a .* b
d =
     5    12
    21    32
>> e = a \ b
e =
    -3    -4
     4     5
>> f = a / b
f =
    3.0000   -2.0000
    2.0000   -1.0000
```

表 4.2: 特殊矩阵的生成

函数	功能	函数	功能
zeros(m, n)	生成 $m \times n$ 全零矩阵	zeros(n)	生成 $n \times n$ 全零矩阵
ones(m, n)	生成 $m \times n$ 全一矩阵	ones(n)	生成 $n \times n$ 全一矩阵
rand(m, n)	生成 $m \times n$ 的均匀分布矩阵	eye(n)	生成单位矩阵

例 4.2 特殊矩阵的生成

```
>> a = zeros(3, 4)
a =
     0     0     0     0
     0     0     0     0
     0     0     0     0
>> b = ones(2, 3)
b =
     1     1     1
     1     1     1
>> c = eye(3)
c =
```

```

1      0      0
0      1      0
0      0      1
>> d = rand(2, 3)
d =
0.8147    0.1270    0.6324
0.9058    0.9134    0.0975

```

表 4.3: 矩阵处理与分析

函数	功能	函数	功能
trace(A)	返回方阵A的迹	B = reshape(A, m, n)	重排矩阵A为 $m \times n$ 的矩阵
diag(A)	取A的对角线向量	B = reshape(A, m, [])	重排矩阵A为m行矩阵
diag(x)	以向量x为对角的矩阵	[V, D] = eig(A)	A的特征值D与特征向量V
tril(A)	取矩阵A的下三角部分	R = rref(A)	矩阵的行最简形式
triu(A)	取矩阵A的上三角部分	Q = orth(A)	A的列向量的正交规范基
flipud(A)	上下翻转矩阵A	Z = null(A)	返回 $Ax=b$ 的基础解系
fliplr(A)	左右翻转矩阵A	n = norm(x)	返回向量x的2范数
rank(A)	返回矩阵A的秩	n = norm(A)	返回矩阵A的2范数
det(A)	返回方阵A的行列式	B = inv(A)	返回矩阵A的逆
rot90(A)	矩阵A旋转90度	[V, J] = jordan(A)	Jordan标准型分解

例 4.3 矩阵处理与分析

```

>> a = 1:9;
>> a = reshape(a, 3, [])
a =
1      4      7
2      5      8
3      6      9
>> b = trace(a), c = diag(a)
b =
15
c =
1
5
9
>> d = tril(a), e = triu(a), f = flipud(a)
d =
1      0      0
2      5      0
3      6      9
e =
1      4      7
0      5      8

```

```

      0      0      9
f =
      3      6      9
      2      5      8
      1      4      7
>> g = fliplr(a), h = rank(a), i = det(a)
g =
      7      4      1
      8      5      2
      9      6      3
h =
      2
i =
      0
>> k = rref(a), l = orth(a), m = null(a)
k =
      1      0     -1
      0      1      2
      0      0      0
l =
    -0.4797    0.7767
    -0.5724    0.0757
    -0.6651   -0.6253
m =
    -0.4082
     0.8165
    -0.4082
>> [V, D] = eig(a), [V, J] = jordan(a)
V =
    -0.4645   -0.8829    0.4082
    -0.5708   -0.2395   -0.8165
    -0.6770    0.4039    0.4082
D =
    16.1168         0         0
         0   -1.1168         0
         0         0   -0.0000
V =
     1.0000   -2.1861    0.6861
    -2.0000   -0.5931    0.8431
     1.0000    1.0000    1.0000
J =
         0         0         0
         0   -1.1168         0
         0         0    16.1168

```

4.2 线性方程组求解

4.2.1 矩阵除法

矩阵除法是解线性方程组的快速算法，MATLAB会根据A的特点自动选定合适的算法求解。

(1) 当A是方阵时，线性方程组的解为 $A \setminus b$ 或 $\text{inv}(A)*b$;

- (2) 当A不是方阵, $Ax = b$ 存在唯一解时, 解为 $A \setminus b$;
- (3) 当A不是方阵, $Ax = b$ 为不定方程(存在无穷多解)时, 其中的一个特解为 $A \setminus b$;
- (4) 当A不是方阵, $Ax = b$ 为超定方程(无解)时, 其中的一个最小二乘近似解为 $A \setminus b$, 使 $Ax - b$ 的范数达到最小。

例 4.4 求解下列方程组

$$(1) \begin{cases} 3x + 2y = 5 \\ 2x + 3y = 5 \end{cases}$$

$$(2) \begin{cases} 3x + 2y + z = 5 \\ 2x + 3y + z = 5 \end{cases}$$

$$(3) \begin{cases} 3x + 2y = 5 \\ 2x + 3y = 5 \\ 3x - 4y = 5 \end{cases}$$

```
>> a = [3 2; 2 3]; b = [5; 5];
>> x = a \ b % 求唯一解
x =
    1.0000
    1.0000
>> a = [3 2 1; 2 3 1]; b = [5; 5];
>> x = a \ b % 求一特解
x =
     1
     1
     0
>> a = [3 2; 2 3; 3 -4]; b = [5; 5; 5];
>> x = a \ b % 求最小二乘近似解
x =
    1.8182
    0.1724
```

4.2.2 求线性方程组通解

在方程组有无穷多解时, 我们可以用以下三种方法求通解:

- (1) 用rref化为行最简形式后求解;
- (2) 用矩阵除法求出一个特解, 再用null求得其齐次方程组的基础解系;
- (3) 用符号数学工具箱中的solve求解(详见第六章)。

例 4.5 求线性方程组的通解

$$\begin{cases} 2w + x + y + z = 5 \\ w + 2x + y + 3z = 7 \\ 3w + 4x - y + 3z = 10 \end{cases}$$

解 方法一

```
>> a = [2 1 1 1; 1 2 1 3; 3 4 -1 3]; b = [5; 7; 10];
>> R = rref([a, b])
R =
    1.0000         0         0   -0.6000    0.5000
         0    1.0000         0    1.4000    2.5000
         0         0    1.0000    0.8000    1.5000
```

此时原方程等价于

$$\begin{cases} w = 0.6z + 0.5 \\ x = -1.4z + 2.5 \\ y = -0.8z + 1.5 \end{cases}$$

令 $z = k$, 可得通解为 $k(0.6, -1.4, -0.8, 1)^T + (0.5, 2.5, 1.5, 0)^T$ 。

方法二

```
>> a = [2 1 1 1; 1 2 1 3; 3 4 -1 3]; b = [5; 7; 10];
>> x0 = a \ b           % 求一特解
x0 =
    0.5000
    2.5000
    1.5000
         0
>> x = null(a)          % 求齐通解
x =
    0.3015
   -0.7035
   -0.4020
    0.5025
```

得通解为 $k(0.3015, -0.7035, -0.4020, 0.5025)^T + (0.5, 2.5, 1.5, 0)^T$ 。

4.2.3 方阵的相似对角化

MATLAB中eig函数求得的每个特征向量都是单位向量, 并且属于同一特征值的线性无关的特征向量已正交化, 在此基础上容易进行相似对角化。

例 4.6 用相似变化矩阵 p 将 A 相似对角化

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 9 & 8 & 5 \\ 8 & 6 & 3 \end{pmatrix}$$

解 方法一 用eig函数

```
>> A = [2 1 1; 9 8 5; 8 6 3];
>> [p, t] = eig(A)
p =
    -0.1257    -0.5702    -0.2209
    -0.8038     0.8100    -0.2999
    -0.5814    -0.1369     0.9280
t =
    13.0237         0         0
         0     0.8195         0
         0         0    -0.8432
```

方法二 用jordan函数分解

```
>> A = [2 1 1; 9 8 5; 8 6 3];
>> [v, j] = jordan(A)
v =
    0.2161 - 0.0000i    -0.2381 + 0.0000i    4.1648 + 0.0000i
    1.3825 + 0.0000i    -0.3231 + 0.0000i   -5.9165 + 0.0000i
    1.0000 + 0.0000i    1.0000 + 0.0000i    1.0000 + 0.0000i
j =
    13.0237         0         0
         0    -0.8432         0
         0         0     0.8195
```

以上两种方法尽管形式略有不同，但结果一致，求得的特征值均为(13.0237, 0.8195, -0.8432)，相应的特征向量为 $(0.2161, 1.3825, 1)^T$, $(4.1648, -5.9165, 1)^T$, $(-0.2381, -0.3231, 1)^T$ 。

4.3 实用模型

4.3.1 投入产出分析

在一个国家或区域的经济系统中，各部门(或企业)既有投入又有产出。生产的产品供给系统各部门和系统外的需求，同时也消耗系统内各部门提供的产品。投入产出分析，就是研究经济系统各个部门间表现为投入与产出的相互依存关系的经济数量方法。美国哈佛大学教授W. Leontief于20世纪30年代建立了国民经济的投入产出的数学模型(Leontief模型)，具体如下：

设某经济系统内有 n 个经济部门，定义符号 x_i 为部门 i 的总产出， c_{ij} 为部门 j 单位产品对部门 i 产品的消耗， d_i 为外部对部门 i 的需求， f_j 为部门 j 新创造的价值，则各部门总产出应满足分配平衡方程组和消耗平衡方程组分别为

$$x_i = \sum_{j=1}^n c_{ij}x_j + d_i, \quad i = 1, \dots, n,$$

$$f_i = x_i - \sum_{j=1}^n c_{ij}x_j, \quad j = 1, \dots, n.$$

令 $C = (c_{ij}), X = (x_1, \cdots, x_n)^T, D = (d_1, \cdots, d_n)^T, F = (f_1, \cdots, f_n)^T$, 于是, 分配平衡方程组可写为

$$X = CX + D,$$

也可写为

$$AX = D,$$

其中, $A = E - C$, E 为单位矩阵。经济学上, C 称为直接消耗矩阵, A 称为Leontief矩阵。令

$$B = C \begin{pmatrix} x_1 & & & \\ & x_2 & & \\ & & \ddots & \\ & & & x_n \end{pmatrix}, Y = [1, \cdots, 1]B,$$

则新创造价值向量 $F = X - Y^T$, B 表示各部门间的投入产出关系, 称为投入产出矩阵, Y 表示各部门的总投入, 称为总投入向量。分析过程可用表 4.4 表示。

表 4.4: 投入产出分析表

	消耗部门	外界需求	总产出
生产部门	B_{ij}	D_i	X_i
新创造的价值	F_j		
总投入	X_j		

注 4.1 $i = 1, \cdots, n, j = 1, \cdots, n$, 且有总投入等于总产出, $\sum_{i=1}^n X_i = \sum_{j=1}^n X_j$ 。注意区分 X_i 与 X_j 的含义。

例 4.7 在某经济年度内, 各经济部门的投入产出表如表 4.5 所示(单位: 亿元)。假设某经济年度工业、农业及第三产业的最后需求均为17亿元, 预测该经济年度工业、农业及第三产业的产出(提示: 对于一个特定的经济系统而言, 直接消耗矩阵和Leontief矩阵可视作不变)。

表 4.5: 各经济部门的投入产出分析表

		消耗部门			最后需求	总产值
		工业	农业	第三产业		
生产部门	工业	6	2	1	16	25
	农业	2.25	1	0.2	1.55	5
	第三产业	3	0.2	1.8	15	20

解 由表 4.5 可知, 投入产出矩阵 B 、需求向量 D 、总产出向量 X 分别为

$$B = \begin{pmatrix} 6 & 2 & 1 \\ 2.25 & 1 & 0.2 \\ 3 & 0.2 & 1.8 \end{pmatrix}, D = \begin{pmatrix} 16 \\ 1.55 \\ 15 \end{pmatrix}, X = \begin{pmatrix} 25 \\ 5 \\ 20 \end{pmatrix}.$$

由此可计算直接消耗矩阵 C 和Leontief矩阵 A , 再由“最后需求均为17亿元”这一条件求解总产出, 编写程序如下:

```
>> B = [6 2 1; 2.25 1 0.2; 3 0.2 1.8]; % 投入产出矩阵
>> n = size(B,1); % 部门数
>> D = [16 1.55 15]'; % 需求向量
>> X = [25 5 20]'; % 总产出
>> C = B./diag(X) % 直接消耗矩阵
C =
    0.2400    0.4000    0.0500
    0.0900    0.2000    0.0100
    0.1200    0.0400    0.0900
>> A = eye(n) - C % Leontief矩阵
A =
    0.7600   -0.4000   -0.0500
   -0.0900    0.8000   -0.0100
   -0.1200   -0.0400    0.9100
>> nD = 17*ones(n,1); % 新的需求向量
>> X = A\nD % 新的总产出
X =
    37.5696
    25.7862
    24.7690
```

故经济年度工业、农业及第三产业的产出分别为37.5696亿元、25.7862亿元和24.7690亿元。

4.3.2 状态转移模型

所谓状态是指客观事物可能出现或存在的状态; 状态转移是指客观事物由一种状态转移到另一种状态。一个系统的某些因素, 在转移过程中, 第 n 次状态只受第 $n-1$ 次状态的影响, 即下一状态只与当前所处状态有关, 而与过去状态无关。状态转移矩阵, 是俄国数学家马尔科夫提出的, 常应用于排队理论、信号处理、人口流动、基因遗传等众多领域。

例 4.8 对城乡人口流动作年度调查, 发现有一个稳定的朝向城镇流动的趋势, 每年农村居民的5%移居城镇而城镇居民的1%迁出, 现在总人口的20%位于城镇。假如城乡总人口保持不变, 并且人口流动的这种趋势将继续下去, 那么

- (1) 一年以后住在城镇人口所占比例是多少? 两年以后呢? 十年以后呢?
- (2) 很多年以后呢?
- (3) 如果现在总人口70%位于城镇, 很多年以后城镇人口所占比例是多少?
- (4) 计算状态转移矩阵的最大特征值及对应的特征向量, 与问题(2)、(3)有何联系?

表 4.6: 状态转移表

下一状态 \ 当前状态	农村居民	城镇居民
农村居民	95%	1%
城镇居民	5%	99%

解 题干中说, 每年农村居民的5%移居城镇, 城镇居民的1%将迁入农村, 转移情况如表 4.6 所示。

于是, 状态转移矩阵可写为

$$A = \begin{pmatrix} 0.95 & 0.01 \\ 0.05 & 0.99 \end{pmatrix}.$$

由于下一状态只与当前所处状态有关, 而与过去状态无关, 于是有递推公式

$$X^{(n)} = AX^{(n-1)}$$

根据初始状态 $X^{(0)} = (0.8, 0.2)^T$, 我们可以计算出 n 年后的状态, 编写程序如下:

```
function X = stm(A, X0, n)
% 计算n时刻的状态X
% Input -A 状态转移矩阵 -X0 初始状态 -n 时刻
% Output -X n时刻的状态
X = X0;
% 循环求解, 也可以用X = A^n * X0 进行求解
for i = 1:n
    X = A * X;
end
```

利用该函数求解问题(1)

```
>> A = [0.95, 0.01; 0.05, 0.99];
>> X0 = [0.8, 0.2]';
>> n = 1;
>> X = stm(A, X0, n)
X =
    0.7620
    0.2380
>> n = 2;
>> X = stm(A, X0, n)
X =
    0.7263
    0.2737
>> n = 10;
>> X = stm(A, X0, n)
```

```
X =
    0.5078
    0.4922
```

求解问题(2)，很多年以后，这里先取 $n = 1000$ ，研究 $n = 1001$ 的情况。

```
>> A = [0.95, 0.01; 0.05, 0.99];
>> X0 = [0.8, 0.2]';
>> n = 1000;
>> X = stm(A, X0, n)
X =
    0.1667
    0.8333
>> X = A * X
X =
    0.1667
    0.8333
```

$n = 1000$ 时，状态转移前后城乡比重没有发生变化，即系统达到平衡状态，因此很多年以后城镇人口比重为83.33%，且保持稳定。

求解问题(3)，修改初始状态，与问题(2)的求解思路相同。

```
>> A = [0.95, 0.01; 0.05, 0.99];
>> X0 = [0.3, 0.7]';
>> n = 1000;
>> X = stm(A, X0, n)
X =
    0.1667
    0.8333
>> X = A * X
X =
    0.1667
    0.8333
```

结果发现，系统仍将达到平衡，城镇人口比重仍为83.33%。

求解问题(4)

```
>> [V, D] = eig(A)
V =
   -0.7071   -0.1961
    0.7071   -0.9806
D =
    0.9400         0
         0    1.0000
>> v = V(:,2) / V(2,2) * X(2) % X为问题(3)求出的结果
```

```
v =
    0.1667
    0.8333
```

转移矩阵的最大特征值为 $\lambda = 1$ ，对应的特征向量为 $v = (0.1667, 0.8333)^T$ 。

实际上，很多年以后，城乡人口将达到平衡，应用极限理论可知 $X = AX$ ，于是，问题就转化为求状态转移矩阵 A 的 $\lambda = 1$ 对应的特征向量问题。

注 4.2 另一种方法求解问题(2)(3)，利用 $X = AX$ 和 $X_1 + X_2 = 1$ ，编程如下：

```
>> D = A - eye(2);
>> B = [D(1,:); 1,1];
>> Y = [0; 1];
>> X = B \ Y
X =
    0.1667
    0.8333
```

4.4 实验报告

1. 至少用两种方法生成矩阵 A

$$A = \begin{pmatrix} 2015 & & & \\ & 2015 & & \\ & & \ddots & \\ & & & 2015 \end{pmatrix}_{10 \times 10}.$$

解 生成矩阵 A 的方法很多，这里只列十种。

```
%% 方法一
n = 10;
a = eye(n);
A = 2015 * a
%% 方法二
a = ones(n,1);
A = 2015 * diag(a)
%% 方法三
a = 2015 * ones(n,1);
A = diag(a)
%% 方法四
A = zeros(n);
A(1:n+1:end) = 2015
%% 方法五
a = zeros(n);
```

```

a = a + 2015;
a = tril(a);
A = triu(a)
%% 方法六
a = zeros(n) + 2015;
a = tril(a);
A = triu(a)
%% 方法七
A = zeros(n);
for i = 1:10
    A(i,i) = 2015;
end
%% 方法八
A = zeros(n);
A(eye(n) == 1) = 2015
%% 方法九
A = zeros(n);
A(ones(n) == eye(n)) = 2015
%% 方法十
A = zeros(n);
A(A ~= eye(n)) = 2015

```

注 4.3 尽管生成方法很多，但效率却不相同，请对上述十种方法的效率进行排序。

2. 求下列矩阵的行列式、逆、特征值和特征向量。

$$(1) \begin{pmatrix} 4 & 1 & -1 \\ 3 & 2 & -6 \\ 1 & 3 & 3 \end{pmatrix} \quad (2) \begin{pmatrix} 5 & 6 & & \\ 1 & 5 & 6 & \\ & 1 & 5 & 6 \\ & & 1 & 5 & 6 \\ & & & 1 & 5 \end{pmatrix}$$

解 第一问

```

>> a = [4 1 -1; 3 2 -6; 1 3 3];
>> det(a)
ans =
    74
>> inv(a)
ans =
    0.3243    -0.0811   -0.0541
   -0.2027     0.1757    0.2838
    0.0946   -0.1486    0.0676
>> [v, j] = eig(a)
v =
    0.8431 + 0.0000i    0.0906 - 0.2375i    0.0906 + 0.2375i
   -0.2601 + 0.0000i    0.7518 + 0.0000i    0.7518 + 0.0000i
    0.4707 + 0.0000i   -0.0717 - 0.6042i   -0.0717 + 0.6042i
j =
    3.1332 + 0.0000i    0.0000 + 0.0000i    0.0000 + 0.0000i
    0.0000 + 0.0000i    2.9334 + 3.8747i    0.0000 + 0.0000i
    0.0000 + 0.0000i    0.0000 + 0.0000i    2.9334 - 3.8747i

```

第二问 注意带状矩阵的生成方法

```
>> n = 5;
>> a = zeros(n);
>> a( 1:n+1:end) = 5;
>> a( 2:n+1:end) = 1;
>> a(n+1:n+1:end) = 6;
>> a
a =
     5     6     0     0     0
     1     5     6     0     0
     0     1     5     6     0
     0     0     1     5     6
     0     0     0     1     5

>> det(a)
ans =
    665

>> inv(a)
ans =
    0.3173   -0.5865    1.0286   -1.6241    1.9489
   -0.0977    0.4887   -0.8571    1.3534   -1.6241
    0.0286   -0.1429    0.5429   -0.8571    1.0286
   -0.0075    0.0376   -0.1429    0.4887   -0.5865
    0.0015   -0.0075    0.0286   -0.0977    0.3173

>> [v, j] = eig(a)
v =
    0.7843   -0.7843   -0.9860   -0.9237   -0.9237
    0.5546    0.5546    0.0000    0.3771   -0.3771
    0.2614   -0.2614    0.1643   -0.0000    0.0000
    0.0924    0.0924    0.0000   -0.0628    0.0628
    0.0218   -0.0218   -0.0274    0.0257    0.0257

j =
    9.2426         0         0         0         0
         0    0.7574         0         0         0
         0         0    5.0000         0         0
         0         0         0    2.5505         0
         0         0         0         0    7.4495
```

第五章 多项式与方程

本章主要介绍有关非线性方程求解、函数极值与曲线拟合的指令，如表 5.1。

表 5.1: 函数与方程的相关指令

函数	功能	函数	功能
polyval	多项式求值	fzero	求一元函数的实根
conv	多项式乘积	fsovl	方程或方程组求解
deconv	多项式除法	fminbnd	一元函数极值
roots	多项式求根	fminsearch	多元函数极值
polyfit	多项式拟合	lsqnonlin	最小二乘法
feval	函数求值	lsqcurvefit	曲线拟合
inline	Inline函数	optimset	优化计算参数设置

在数据建模时我们经常会遇到最优化的问题，比如利润最高、费用最少等，还有预测问题和方程求解问题等，这些问题的解决都离不开本章的内容。除此之外，我们将介绍一些方程求根的经典迭代方法。

5.1 多项式

MATLAB中一个多项式可用降幂排列的系数行向量表示，如 $x^3 + 2x - 20$ 可表示为 $[1, 0, 2, -20]$ ，注意不要漏掉二次项系数0。与多项式有关的函数见表 5.2。

表 5.2: 与多项式有关的函数

函数	功能
$y = \text{polval}(p, x)$	求多项式 p 在 x 处的值 y ， x 可为一点或向量
$p = \text{conv}(p1, p2)$	求多项式 $p1, p2$ 的乘积多项式 p
$[p, r] = \text{deconv}(p1, p2)$	求多项式 $p1$ 除以 $p2$ 的商 p 和余项 r
$x = \text{roots}(p)$	求多项式 p 的所有复数根 x
$p = \text{polyfit}(x, y, k)$	用 k 次多项式 p 拟合数据 (x, y)

例 5.1 计算多项式 $p_1(x) = x^3 + 2x^2 + 10x - 20$ 与 $p_2(x) = x^2 + 4x + 1$ 的乘积、商、余和所有的复根。

```
>> p1 = [1 2 10 -20];
>> p2 = [2 4 1];
>> p = conv(p1, p2)
p =
    2    8   29    2  -70  -20
>> [p, r] = deconv(p1, p2)
p =
    0.5000    0
r =
    0    0   9.5000 -20.0000
>> x1 = roots(p1)
x1 =

-1.6844 + 3.4313i
-1.6844 - 3.4313i
 1.3688 + 0.0000i
>> x2 = roots(p2)
x2 =

-1.7071
-0.2929
```

例 5.2 编写程序求向量表达的 n 次多项式 p 的导函数的系数向量 dp (提示 $p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$ 的导数为 $p'(x) = a_n n x^{n-1} + a_{n-1} (n-1) x^{n-2} + \dots + a_1$)。

```
function dp = derp(p)
n = length(p) - 1;
dn = n:-1:1;
```



```
dep = p(1:end-1);
dp = dep .* dn;
```

验证: $p(x) = x^3 + 2x^2 + 10x - 20$ 的导函数为 $p'(x) = 3x^2 + 4x + 10$

```
>> p = [1, 2, 10, -20];
>> dp = derp(p)
dp =
     3     4    10
```

注 5.1 MATLAB 的内部函数 *polyder* 可用于求解多项式的导数。

例 5.3 用二次多项式拟合数据: $x = [1, 2, 4, 8, 16]$; $y = [-1, 2, 10, 60, 252]$ 。

```
x = [ 1, 2, 4, 8, 16];
y = [-1, 2, 10, 60, 252];
p = polyfit(x, y, 2)
xx = linspace(0, 20, 100);
yy = polyval(p, xx);
plot(x, y, 'o', xx, yy, '-')
legend('拟合曲线', '数据点', 'location', 'best')
```

拟合结果为 $p(x) = 1.038x^2 - 0.7514x - 1.5294$, 结果如图 5.1。

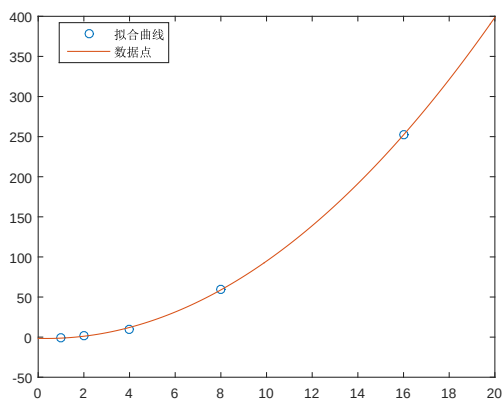


图 5.1: 二次多项式拟合曲线

5.2 零点与极值

MATLAB 中求函数零点和极值相关的函数见表 5.3。

表 5.3: 函数零点与极值的相关函数

函数	功能
<code>x = fzero(fun, x0)</code>	求函数fun在x0附近的零点x， x0可为一点或区间[a, b]
<code>[x, f, h] = fsolve(fun, x0)</code>	求函数fun在x0附近的一个零点x及函数值f, h为可靠度
<code>min(y), max(x)</code>	求y的最小值和最大值
<code>[x, f] = fminbnd(fun, a, b)</code>	求一元函数fun在区间[a, b]内的极小值点x和极小值f
<code>[x, f] = fminsearch(fun, x0)</code>	求多元函数fun在x0附近的极小值点x和极小值f

例 5.4 求多项式函数 $f(x) = x^3 + 2x^2 + 10x - 20$ 在区间 $[1, 2]$ 内的零点。

解 方法一 用fzero函数

```
>> fun = inline('x^3+2*x^2+10*x-20');
>> x = fzero(fun, [1, 2])
x =
    1.3688
```

方法二 用fsolve函数

```
>> [x, f, h] = fsolve(fun, 1.5)
x =
    1.3688
f =
    2.7721e-10
h =
     1
```

注 5.2 在函数零点范围未知的情况下，通常是先画出函数的图像确定零点的大致范围，再求解，如例 5.5。

例 5.5 用fzero求出函数 $f(x) = x^3 - 2x^2 - 10x - 2$ 的所有零点。

```
>> fun = inline('x^3-2*x^2-10*x-2');
>> fplot(fun, [-2.5, 5])
>> grid on % 结果如图 5.2
>> x1 = fzero(fun, [-2.5, -2])
x1 =
   -2.1750
>> x2 = fzero(fun, [-1, 0])
x2 =
   -0.2097
```

```
>> x3 = fzero(fun, [ 4 , 5])
x3 =
    4.3847
```

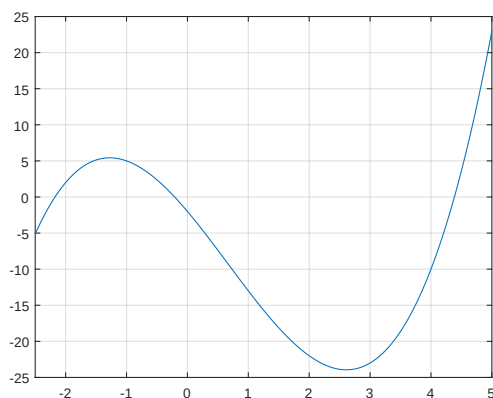


图 5.2: $f(x) = x^3 - 2x^2 - 10x - 2$ 的函数图像

故函数零点 $x_1 = -2.1750$, $x_2 = -0.2097$, $x_3 = 4.3847$ 。

例 5.6 求函数 $f(x) = x^3 - 2x^2 - 10x - 2$ 在区间 $[-2.5, 5]$ 上的最小值点。

```
>> x = -2.5:0.01:5;
>> y = x.^3 - 2 * x.^2 - 10 * x - 2;
>> [m, k] = min(y); % 求最小值m及其编址k
>> x_min = x(k) % 接近最小值点的点
x_min =
    2.6100
>> fun = inline('x^3-2*x^2-10*x-2');
>> [x, f] = fminbnd(fun, 2.5, 3)
x =
    2.6103
f =
   -23.9446
```

故最小值点 $x = 2.6103$, 最小值 $y = -23.9446$ 。

例 5.7 求函数 $f(x) = x^3 - 2x^2 - 10x - 2$ 在区间 $[-2.5, 5]$ 上的最大值点。

```
>> x = -2.5:0.01:1;
>> y = x.^3 - 2 * x.^2 - 10 * x - 2;
>> [m, k] = max(y);
```

```
>> x_max = x(k)
x_max =
    -1.2800
>> fun = inline('-x^3+2*x^2+10*x+2');
>> [x, f] = fminbnd(fun, -1.5, -1) % 求最大值转化为最小值
x =
    -1.2770
f =
    -5.4261
```

故最大值点 $x = -1.2770$ ，最大值 $y = -5.4261$ 。

例 5.8 求二元函数 $f(x, y) = 5 + x^2 + y^2$ 在 $(1, 0)$ 附近的极小值点。

```
>> fun = inline('5+x(1)^2+x(2)^2'); % x, y写为同一向量
>> [x, f] = fminsearch(fun, [1, 0])
x =
    1.0e-04 *
    0.2056    0.2469
f =
    5.0000
```

注 5.3 注意最大值的求法和多元函数的表示方法。

5.3 迭代法求方程的根

解非线性方程 $f(x) = 0$ 的主要算法是迭代法。迭代法的主要思想是从解的初始近似值 x_0 开始，按照 $x_{n+1} = g(x_n)$ 的格式，得到近似序列 $\{x_k\}$ ，逐步逼近精确解 x 。

迭代法成功的关键因素有：

- (1) $f(x) = 0$ 与 $x = g(x)$ 应该有相同的解；
- (2) 选取适当的初值 x_0 ；
- (3) 构造的迭代序列要收敛， $x = g(x)$ 应满足 $|g'(x)| < 1$ 。

经典的迭代法主要有二分法、割线法和Newton迭代法。

5.3.1 二分法

设 $f(x)$ 是区间 $[a, b]$ 上的连续函数，且在 (a, b) 内有一单根，二分法的计算过程如下：

- (1) 计算端点值 $f(a), f(b)$ ；
- (2) 计算区间中点值 $f(\frac{a+b}{2})$ ；

(3) 判断: 若 $f(\frac{a+b}{2}) = 0$, 则计算停止; 若 $f(\frac{a+b}{2})$ 与 $f(a)$ 异号, 则根存在于区间 $(a, \frac{a+b}{2})$ 中, 以 $\frac{a+b}{2}$ 取代 b ; 否则, 根存在于区间 $(\frac{a+b}{2}, b)$ 中, 以 $\frac{a+b}{2}$ 取代 a ;

(4) 重复(2), (3)步, 直到区间范围缩小到精度范围内, 将所得的区间中点作为根的近似解。

二分法的优点是简单且易于实现(具体的程序详见本章的报告部分), 但它只能求实根且不能求偶重根。

5.3.2 割线法

割线法(或弦位法)是选取曲线 $y = f(x)$ 上的两点 $P_0(x_0, y_0), P_1(x_1, y_1)$, 过这两点作一条直线, 其方程为

$$y = f(x_1) + \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x - x_1),$$

当 $y_0 \neq y_1$ 时, 直线与 x 轴交点为

$$x_2 = x_1 - \frac{f(x_1)}{f(x_1) - f(x_0)}(x_1 - x_0),$$

依次进行下去, 即可得迭代格式

$$x_{k+1} = x_k - \frac{f(x_k)}{f(x_k) - f(x_{k-1})}(x_k - x_{k-1}).$$

弦位法的计算过程(具体的程序详见本章的实验报告部分)如下:

- (1) 选定初值 x_0, x_1 , 计算 $f(x_0), f(x_1)$;
- (2) 按迭代公式计算 $x_2, f(x_2)$;
- (3) 若 $f(x_2) = 0$, 则计算停止, 否则用 $(x_2, f(x_2)), (x_1, f(x_1))$ 分别替代 $(x_1, f(x_1)), (x_0, f(x_0))$;
- (4) 重复(2), (3)步, 直到区间范围缩小到精度范围内。

5.3.3 Newton迭代法

Newton迭代法(或切线法)的迭代格式为

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

迭代法的几何意义十分明显, 在选取 x_0 后, 过 $(x_0, f(x_0))$ 作 $f(x)$ 的切线, 该切线方程与 x 轴的交点即为

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)},$$

依次进行下去, 即可得上述的迭代格式。

例 5.9 Newton迭代法的程序

```
function x = newton(fun, dfun, x0, e)
% Newton迭代法
% Input -fun 方程fun(x) = 0 -dfun fun的导函数
%       -x0 迭代初值       -e 计算精度
% Output -x 近似根
if nargin < 4, e = 1e-4; end
x = x0;
x0 = x + 2 * e; % 使while成立, 进入循环
while abs(x0 - x) > e
    x0 = x;
    x = x0 - feval(fun, x0) / feval(dfun, x0);
end
```

例 5.10 用Newton迭代法求解方程 $f(x) = x^3 + 2x^2 + 10x - 20$ 的根。

```
>> fun = inline('x^3+2*x^2+10*x-20');
>> dfun = inline('3*x^2+4*x+10');
>> x0 = 1;
>> x = newton(fun, dfun, x0)
x =
    1.3688
```

5.4 实验报告

1. 求多项式 $x^2 + x + 1$ 的所有根和极值点。

```
>> p = [1 1 1];
>> roots(p)
ans =
    -0.5000 + 0.8660i
    -0.5000 - 0.8660i
>> fun = inline('x^2+x+1');
>> [x, f] = fminbnd(fun, -1, 0)
x =
    -0.5000
f =
    0.7500
```

2. 考虑函数

$$f(x, y) = \frac{y^3}{9} + 3x^2y + 9x^2 + y^2 + xy + 9$$

(1) 绘制 $f(x, y)$ 在 $-2 < x < 1, -7 < y < 1$ 上的图, 观察极值点的位置;

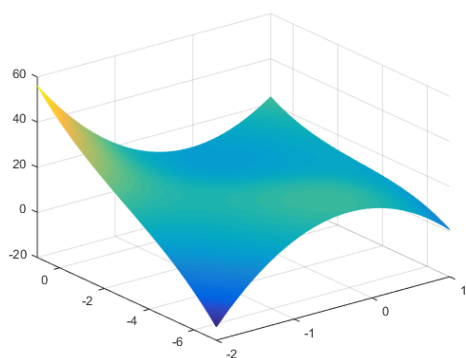
(2) 求极值点和极值。

解 (1) 绘制函数图像

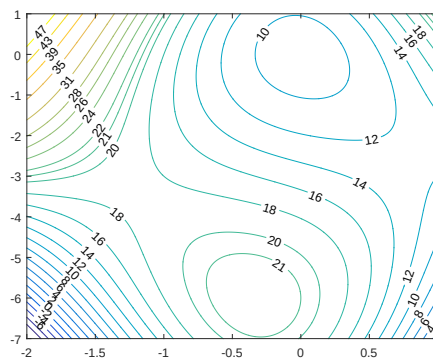
```

x = -2:0.01:1;
y = -7:0.01:1;
[x, y] = meshgrid(x, y);
z = 1/9 * y.^3 + 3 * x.^2 .* y + 9 * x.^2 + y .* y + x .* y + 9;
figure
mesh(x, y, z)
axis([-2, 1, -7, 1, -20, 60]) % 结果图 5.3(a)
figure
[C, h] = contour(x, y, z, [-10:2:20, 21, 22:2:28, 31:4:50]);
clabel(C, h, 'LabelSpacing', 1000) % 添加数值, 结果如图 5.3(b)

```



(a)



(b)

图 5.3: 二元函数图像

注 5.4 由于三维视图不便观察, 我们可以通过二维视图的颜色变化、等值线的变化、梯度场走势或等高线趋势判断极值点位置。观察图 5.3, 发现极值点出现在(0,0) 和(0,-6)附近。

(2) 求极值点和极值

```

>> str = ['1/9*(x(2))^3+3*(x(1))^2*x(2)+...
          '9*(x(1))^2+(x(2))^2+x(1)*x(2)+9'];
>> fun = inline(str);
>> [x, f] = fminsearch(fun, [0, 0])
x =
    0    0
f =
    9
>> str = ['-1*(', str, ')'];
>> fun = inline(str);
>> [x, f] = fminsearch(fun, [0, -6])
x =
-0.3333   -6.0000
f =
-22.0000

```

故极小值点为(0,0), 极小值为9, 极大值点为 $(-\frac{1}{3}, -6)$, 最大值为22。

3. 按要求编写求解方程 $x^3 + 2x^2 + 10x - 20 = 0$ 在区间[1,2]内的根。

- (1) 编写二分法程序求方程的根;
- (2) 编写弦位法程序求方程的根;
- (3) 编写基于向量表示的多项式p的Newton法程序求根。

解 (1) 二分法程序

```
function x = dichotomy(fun, a, b, eps)
% 二分法求方程的根
% Input -fun 方程对应的函数 -[a, b] 根所在的区间 -eps 精度
% Output -x 近似根
if nargin < 4, eps = 1e-4; end
s = sign(feval(fun, a));
t = sign(feval(fun, b));
if s * t >= 0, error('出错了'); end
x = (a + b)/2;
while abs(feval(fun, x)) > eps || abs(a - b) > eps
    if sign(feval(fun, x)) == s
        a = x;
    else
        b = x;
    end
    x = (a + b) / 2;
end
```

应用二分法求根

```
>> fun = inline('x^3+2*x^2+10*x-20');
>> x = dichotomy(fun, 1, 2)
x =
    1.3688
```

(2) 弦位法程序

```
function x = secant(fun, x0, x1, eps)
% 弦位法求方程的根
% Input -fun 方程对应的函数 -[x0, x1] 迭代初值 -eps 精度
% Output -x 近似根
if nargin < 4, eps = 1e-4; end
y = x0; x = x1;
while abs(x - y) > eps
    z = x - feval(fun, x) * (x - y) / (feval(fun, x) - feval(fun, y));
    y = x; x = z;
end
```

应用弦位法求根


```
>> fun = inline('x^3+2*x^2+10*x-20');  
>> x = secant(fun, 1, 2)  
x =  
    1.3688
```

(3) Newton法程序

```
function x = newtonp(p, x0, eps)  
% 基于多项式的Newton迭代法  
% Input -fun 方程fun(x) = 0 -dfun fun的导函数  
%       -x0   迭代初值 -eps 计算精度  
% Output -x 近似根  
if nargin < 4, eps = 1e-4; end  
dp = derp(p); % derp函数见例6.2  
x = x0;  
x0 = x + 2 * eps; % 使while成立, 进入循环  
while abs(x0 - x) > eps  
    x0 = x;  
    x = x0 - polyval(p, x0) / polyval(dp, x0);  
end
```

应用Newton法求根

```
>> p = [1 2 10 -20];  
>> x0 = 1;  
>> x = newtonp(p, x0)  
x =  
    1.3688
```

我的 MATLAB 报告

王保山

第六章 符号计算

利用符号数学工具箱，我们可以进行解析数学运算和任意精度的数值运算，包括矩阵、微分、积分等。由于本章范围太广，所涉内容不必逐一掌握，本章可当作工具参考，也可比其他章节综合掌握。符号计算的常用指令如表 6.1 所示。

表 6.1: 符号计算的相关指令

函数	功能	函数	功能
sym	数值或字符串转化为符号	diff	函数求导
syms	定义符号变量	taylor	泰勒展开
subs	变量替换	taylortool	泰勒展开计算器
digits	定义数值精度	jacobian	Jacobi矩阵
vpa	任意精度计算	int	积分
double	符号对象转化为数值	solve	解方程
char	符号对象转化为字符串	dsolve	解微分方程
factor	因式分解	ezplot	简捷函数曲线
expand	展开式	ezpolar	极坐标作图
collect	合并同类项	ezplot3	空间曲线作图
finverse	求函数的逆	ezmesh	空间网格作图
compose	求复合函数	ezsurf	空间曲面作图
simplify	化简	ezcontour	等高线作图
numden	分式通分	ezcontourf	画填色等位线
limit	符号极限	maple	执行Maple指令
symsum	级数求和	mhhelp	Maple帮助
funtool	函数计算器	mtaylor	泰勒展开的Maple指令
pretty	按书写格式输出	extrema	约束极值的Maple指令

6.1 符号对象

符号运算使用一种特殊的数据类型，称为符号对象，它的声明指令如表 6.2 所示，它用字符串形式表达但又不同于字符串，如例 6.1 所示。

表 6.2: 符号对象的声明

指令	功能
s = sym(str)	将数值或字符串str转化为符号对象s
s = sym(num, 'd')	将数值表达式num转化为符号表达式s
syms var1 var2 ...	定义var1, var2, ...为符号变量
subs(s, old, new)	将符号表达式s中的符号变量old替换为new

例 6.1 符号变量

```
>> str = 'Ilovetmatlab';
>> s = sym(str)
s =
Ilovetmatlab
>> whos s           % 查询变量信息
      Name      Size      Bytes  Class  Attributes
      s         1x1         112   sym
>> num = 2015;
>> s = sym(num, 'd')
s =
2015.0
>> syms a b c
>> c = a + b
c =
a + b
>> syms r t
>> c = subs(c, a, t)
c =
b + t
>> c = subs(c, b, r)
c =
r + t
>> c = subs(c, [r t], [1 2])
c =
3
```

注 6.1 符号对象占用的字节数大于数值或字符，因此运算速度会很慢，如例 6.2 所示。

例 6.2 用数值方法和符号方法分别计算s的值并比较执行时间。

$$s = \sum_{i=0}^{10^4} i$$

```

%% 数值方法
tic
s = 0;
for i = 1:10000
    s = s + i;
end
s
toc
%% 符号方法
tic
syms s t
s = t;
for i = 1:10000
    s = s + i;
end
s = subs(s, t, 0)
toc

```

执行结果为

```

s =
    50005000
Elapsed time is 0.000492 seconds.
s =
    50005000
Elapsed time is 11.073460 seconds.

```

符号数值计算默认精度为32位十进制，是数值计算(16位)的两倍，通过表 6.3 中的命令可以设置计算精度。

表 6.3: 符号计算的精度设置

指令	功能
<code>digits(n)</code>	将数值计算精度设为n
<code>x = vpa(s)</code>	求s的数值结果
<code>x = vpa(s, n)</code>	用n位计算精度求s的数值结果
<code>double(s)</code>	将符号对象转化为双精度数值
<code>char(s)</code>	将符号对象转化为字符串

例 6.3 计算 π 的值

```

>> syms s
>> s = pi;
>> x = vpa(s)

```

```
x =
3.1415926535897932384626433832795
>> x = vpa(s,16)
x =
3.141592653589793
>> x = double(s)
x =
3.1416
```

注 6.2 注意理解例 6.3 中 x 的数据类型。下面给出 π 的小数点后前 200 位。

```
>> x = vpa(s,201) % 为便于观察，四位1组，每行10组展示
x =
3.1415 9265 3589 7932 3846 2643 3832 7950 2884 1971
6939 9375 1058 2097 4944 5923 0781 6406 2862 0899
8628 0348 2534 2117 0679 8214 8086 5132 8230 6647
0938 4460 9550 5822 3172 5359 4081 2848 1117 4502
8410 2701 9385 2110 5559 6446 2294 8954 9303 8196
```

6.2 符号运算

MATLAB 中大部分矩阵和数组运算符及指令都可以应用到符号矩阵中。

例 6.4 求符号矩阵的逆及逆的上三角部分

```
>> str = '[a b; c d]';
>> A = sym(str)
A =
[ a, b]
[ c, d]
>> B = inv(A)
B =
[ d/(a*d - b*c), -b/(a*d - b*c)]
[ -c/(a*d - b*c), a/(a*d - b*c)]
>> pretty(B) % 按书写格式输出
/      d      b      \
| -----, ----- |
| a d - b c  a d - b c |
|              |
|      c      a      |
| -----, ----- |
\  a d - b c  a d - b c /
>> C = triu(B) % 取上三角部分
C =
[ d/(a*d - b*c), -b/(a*d - b*c)]
[ 0, a/(a*d - b*c)]
>> pretty(C) % 按书写格式输出
```

```

/      d      b      \
| -----, - ----- |
| a d - b c   a d - b c |
|               |
|               a       |
|      0,      ----- |
\               a d - b c /

```

注 6.3 符号表达式的计算结果一般很长而且复杂，建议用`pretty`命令输出书写格式。

大部分MATLAB数学函数和逻辑关系运算也可用与符号对象中，如表 6.4 中的函数。

表 6.4: 用于符号计算的数学函数

指令	功能
<code>factor(expr)</code>	对 <code>expr</code> 作因式分解
<code>expand(expr)</code>	将 <code>expr</code> 展开
<code>simplify(expr)</code>	将 <code>expr</code> 化简
<code>g = finverse(f, v)</code>	求函数 <code>f(v)</code> 的反函数 <code>g(v)</code>
<code>fg = compose(f, g)</code>	求函数 <code>f(v)</code> 和函数 <code>g(v)</code> 的复合函数 <code>f(g(v))</code>
<code>[n, d] = numden(expr)</code>	分式通分， <code>n</code> 为分子， <code>d</code> 为分母
<code>funtool</code>	函数分析图形界面

例 6.5 函数 $f(x) = x^3 + 7x^2 + 8x - 9$ 与 $g(x) = x^2 + 4x + 1$ 的符号运算。

```

>> syms x
>> f = x^3 - 7*x^2 + 8*x - 9;
>> g = x^2 + 4 * x + 1;
>> h = f + g
h =
x^3 - 6*x^2 + 12*x - 8
>> k = simplify(h)
k =
(x - 2)^3
>> m = factor(h)
m =
[ x - 2, x - 2, x - 2]
>> n = expand(h)
n =
x^3 - 6*x^2 + 12*x - 8
>> p = collect(k, x)
p =
x^3 - 6*x^2 + 12*x - 8

```

例 6.6 考虑函数 $f(x) = \ln x$ 的反函数 $h(y)$ 及其与 $g(x) = x^2 + 2x + 1$ 的复合函数 $h(g(x))$ 和 $g(h(x))$ 。

```

>> syms x
>> f = log(x);
>> g = x ^ 2 + 2 * x + 1;
>> h = finverse(f, x)
h =
exp(x)
>> p1 = compose(h, g)
p1 =
exp(x^2 + 2*x + 1)
>> p2 = compose(g, h)
p2 =
exp(2*x) + 2*exp(x) + 1

```

故 $h(x) = e^x$, $h(g(x)) = e^{x^2+2x+1}$, $g(h(x)) = e^{2x} + 2e^x + 1$ 。

符号微积分的指令见表 6.5。

表 6.5: 符号微积分的主要指令

指令	功能
limit(s, x, a)	当x趋于a时, 符号表达式s的极限
limit(s, x, a, 'right')	当x趋于a时, 符号表达式s的右极限
limit(s, x, a, 'left')	当x趋于a时, 符号表达式s的左极限
symsum(s, n, a, b)	符号表达式s表示的通项当自变量n由a到b的和
diff(s, x)	符号表达式s对x的导数
diff(s, x, n)	符号表达式s对x的n阶导数
taylor(s, n, a, x)	符号表达式s在a点对x的展开到n-1次的泰勒展开式
taylortool	taylor分析图形界面
jacobian(f, x)	向量函数f的Jacobi矩阵
int(s)	符号表达式s的不定积分
int(s, v)	符号表达式s关于向量v的不定积分
int(s, a, b)	符号表达式s在[a, b]上的定积分
int(s, v, a, b)	符号表达式s在[a, b]上关于向量v的定积分
s = solve(str, x)	求关于变量x的由str表达的方程的解s
s = dsolve(str, x0, x)	求变量x的由初值为x0的str表达的微分方程的解s

注 6.4 求方程组的解可使用 $s = \text{solve}(\text{str1}, \text{str2}, \dots, x_1, x_2, \dots)$, 求微分方程组的解可使用 $s = \text{dsolve}(\text{str1}, \text{str2}, \dots, x_1, x_2, \dots)$ 。当求不出解析解时, MATLAB会自动求原点附近的一个近似解。微分方程中y导数的表达方式 Dy , 二阶导数为 D^2y , 以此类推。

例 6.7 验证极限 $\lim_{x \rightarrow 0} \frac{\sin x}{x}$ 存在并求值。


```

syms x
y = sin(x)/x;
x1 = limit(y, x, 0, 'right');
x2 = limit(y, x, 0, 'left');
x0 = NaN;
if x1 == x2
    x0 = limit(y, x, 0);
end
x0

```

执行结果为

```

x0 =
1

```

从结果看, $\lim_{x \rightarrow 0} \frac{\sin x}{x}$ 存在且值为1。

例 6.8 用 *symsum* 生成函数 $f(x) = x^9 + x^8 + \cdots + x + 1$ 并在区间 $[-1.2, 1]$ 上绘图。

```

>> syms x n
>> y = x^n;
>> s = symsum(y, n, 0, 9)
s =
x^9 + x^8 + x^7 + x^6 + x^5 + x^4 + x^3 + x^2 + x + 1
>> str = char(s);
>> fun = inline(str);
>> fplot(fun, [-1.2, 1])    % 结果如图 6.1

```

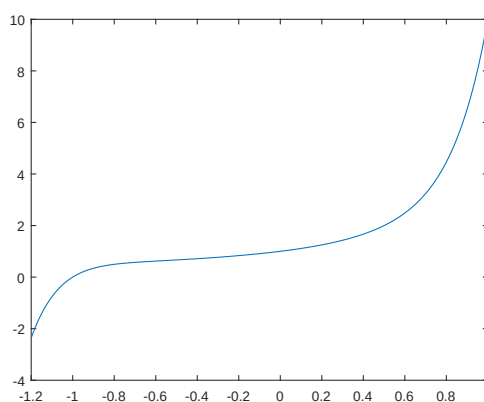


图 6.1: $f(x) = x^9 + x^8 + \cdots + x + 1$ 的函数图像

例 6.9 求函数 $f(x) = \frac{x^2 + 1}{x + 1}$ 对 x 的导函数。

```
>> syms x
>> z = (x^2 + 1) / (x + 1);
>> s = diff(z, x)
s =
(2*x)/(x + 1) - (x^2 + 1)/(x + 1)^2
>> pretty(s)
      2
  2 x   x  + 1
  ---- - ----
  x + 1   2
          (x + 1)
```

故导函数为 $f'(x) = \frac{2x}{x+1} - \frac{x^2+1}{(x+1)^2}$ 。

例 6.10 求 $f(x) = e^x$ 的泰勒展开式，展开到 8-1 阶。

```
>> syms x
>> y = exp(x);
>> s = taylor(y, x, 'order', 8)
s =
x^7/5040 + x^6/720 + x^5/120 + x^4/24 + x^3/6 + x^2/2 + x+1
>> [c,t] = coeffs(s,x) % 提取系数c和x的项
c =
[ 1/5040, 1/720, 1/120, 1/24, 1/6, 1/2, 1, 1]
t =
[ x^7, x^6, x^5, x^4, x^3, x^2, x, 1]
```

因此展开式为 $f(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \frac{x^5}{120} + \frac{x^6}{720} + \frac{x^7}{5040}$ 。

例 6.11 求 $f(x) = \frac{1}{1+x^2}$ 的不定积分以及区间 $[0, 1]$ 上的定积分。

```
>> syms x
>> y = 1/(1 + x^2);
>> s = int(y)
s =
atan(x)
>> s = int(y, 0, 1)
s =
pi/4
```

由此可见利用积分可以实现对 π 值的计算，详见本章实验报告部分。

例 6.12 求解方程 $ax^2 + bx + c = 0$ 。

```
>> syms x a b c
>> str = 'a*x^2+_b*x+_c';
>> s = solve(str, x)      % 自变量x可省略, s = solve(str)
s =
-(b + (b^2 - 4*a*c)^(1/2))/(2*a)
-(b - (b^2 - 4*a*c)^(1/2))/(2*a)
>> pretty(s)
/      2      \
|  b + sqrt(b  - 4 a c) |
| - ----- |
|      2 a      |
|      |      |
|      2      |
|  b - sqrt(b  - 4 a c) |
| - ----- |
|      2 a      |
\      /
```

例 6.13 求解常微分方程边值问题

$$\begin{cases} xy'' - 3y' = x^2, \\ y(1) = y(5) = 0. \end{cases}$$

```
>> syms x y
>> str = 'x*D2y-_3*Dy=_x^2';
>> x1 = 'y(1)=0';
>> x2 = 'y(5)=0';
>> s = dsolve(str, x1, x2, 'x')
s =
(31*x^4)/468 - x^3/3 + 125/468
```

故解析解为 $y = \frac{31x^4}{468} - \frac{x^3}{3} + \frac{125}{468}$ 。

6.3 便捷函数绘图

MATLAB的符号工具箱提供了一种更为便捷的绘图方式，主要的绘图指令如表 6.6 所示。

表 6.6: 便捷绘图指令

指令	功能
ezplot(fun, [a, b])	作函数fun在区间[a, b]上的图, 区间默认为[-2π, 2π]
ezpolar(fun, [a, b])	作极坐标图, 区间默认为[0, 2π]
ezplot3(x, y, z, [a, b])	作曲线(x(t),y(t),z(t))的图, 区间默认为[0, 2π]
ezmesh(z, [a, b, c, d])	作z(x, y)在区间[a, b; c, d]上的图
ezmesh(x, y, z, [a, b, c, d])	作(x(x, t), y(s, t), z(s, t))的图
ezmesh(..., n)	用n × n个结点绘图, n默认为60

注 6.5 表 6.6 中的函数可以是显函数、隐函数或参变量函数,可由字符串、函数句柄、*Inline*函数或匿名函数表示。其他的绘制曲面的便捷函数用法与 *ezmesh* 函数类似,不再赘述。

例 6.14 绘制区间 $[-2\pi, 2\pi]$ 上 $\sin x$ 的函数图像。

```
ezplot('sin(x)', [-2, 2]*pi) % 结果如图 6.2
```

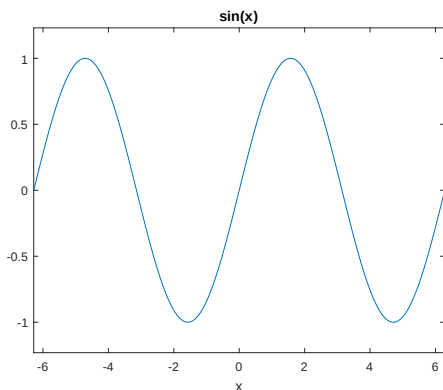


图 6.2: ezplot 绘制的正弦图像

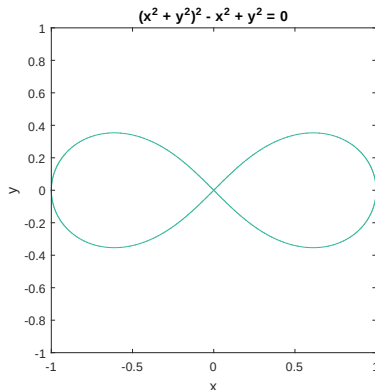


图 6.3: ezplot 绘制的隐函数图像

例 6.15 绘制隐函数 $F(x, y) = (x^2 + y^2)^2 - x^2 + y^2 = 0, x \in [-1, 1]$ 的图像。

```
fun = inline('(x^2+_y^2)^2-_x^2+_y^2');
ezplot(fun, [-1, 1])
axis equal % 结果如图 6.3
```

例 6.16 ezplot 绘制星形线

$$\begin{cases} x = \cos^3 t, \\ y = \sin^3 t. \end{cases}$$

```
x = 'cos(t)^3';
y = 'sin(t)^3';
ezplot(x, y)
axis square % 结果如图 6.4
```

例 6.17 绘制极坐标曲线 $r(\theta) = \frac{\sin \theta}{\theta}, \theta \in [-6\pi, 6\pi]$ 的图像。

```

r = 'sin(t)/t';
ezpolar(r, [-6, 6]*pi)
axis square      % 结果如图 6.5

```

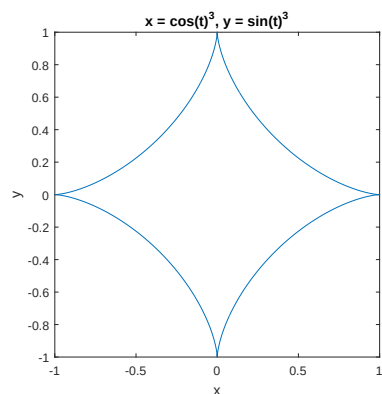


图 6.4: ezplot绘制的星形线

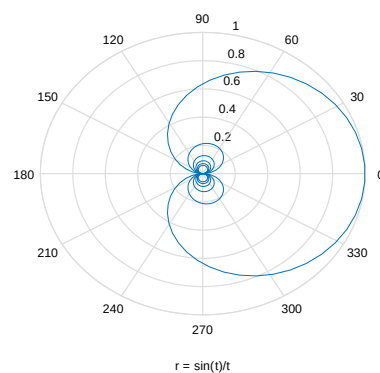


图 6.5: ezpolar绘制的极坐标图

例 6.18 绘制曲面 $z(x, y) = xe^{-(x^2+y^2)}$, $x, y \in [-2, 2]$ 的图像及其等高线。

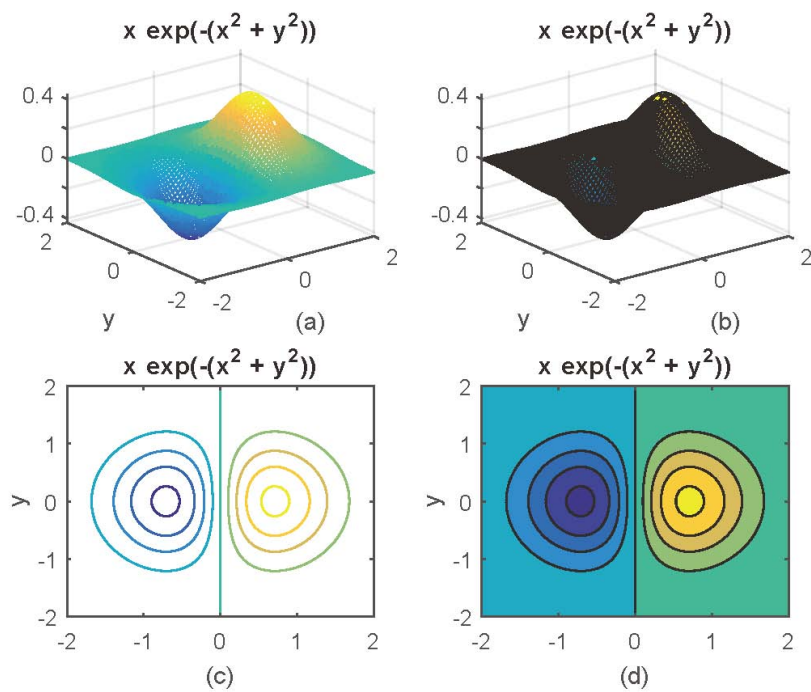


图 6.6: 便捷绘制曲面及等高线

```

fun = 'x*exp(-(x^2+y^2))';
r = [-2, 2, -2, 2];
subplot(221)
ezmesh(fun, r)      % 如图 6.6(a), 曲面网格图
xlabel('a')
subplot(222)
ezsurf(fun, r)      % 如图 6.6(b), 曲面图
xlabel('b')
subplot(223)
ezcontour(fun, r)   % 如图 6.6(c), 等高线图
xlabel('c')
subplot(224)
ezcontourf(fun, r) % 如图 6.6(d), 彩色等高线图
xlabel('d')

```

6.4 实验报告

1. 本章已用符号运算来求 π 的值, 试用数值方法(至少两种)求 π 的近似值。

解 (1) 利用莱布尼兹级数

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} (-1)^k \frac{1}{2k+1}.$$

```

s = 0;
n = 1000;
for i = 0:n
    s = s + (-1)^i / (2 * i + 1);
end
format long      % 长格式显示
p = 4 * s

```

结果如下

```

p =
    3.142591654339544

```

(2) 利用威廉姆斯无穷乘积式

$$\frac{\pi}{2} = \prod_{k=1}^{\infty} \frac{4k^2}{4k^2 - 1}.$$

```

s = 1;
n = 2000;
for i = 2:2:n
    s = s * i^2 / (i^2 - 1);
end
p = 2 * s

```

结果如下

```
p =
    3.140807746030388
```

(3) 利用韦达关系式

$$\frac{2}{\pi} = \frac{\sqrt{2}}{2} \times \frac{\sqrt{2+\sqrt{2}}}{2} \times \frac{\sqrt{2+\sqrt{2+\sqrt{2}}}}{2} \times \dots$$

```
s = 1;
t = 0;
n = 1000;
for i = 1:n
    t = sqrt(t + 2);
    s = s * t / 2;
end
p = 2 / s
```

结果如下

```
p =
    3.141592653589794
```

(4) 利用级数

$$\frac{\pi}{6} = \sqrt{\frac{1}{3}} \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)3^k}.$$

```
s = 0;
n = 1000;
for i = 0:n
    s = s + (-1) ^ i / (3 ^ i * (2 * i + 1));
end
p = 6 * s / sqrt(3)
```

结果如下

```
p =
    3.141592653589794
```

(5) 利用拉马努金级数

$$\frac{1}{\pi} = \frac{\sqrt{8}}{9801} \sum_{k=0}^{\infty} \frac{(4k)!(1103 + 26390k)}{(k!)^4(369)^{4k}}.$$

```

s = 0;
n = 2;          % 注意运算次数
for i = 0:n
    t1 = factorial(4 * i) * (1103 + 26390 * i);
    t2 = factorial(i) ^ 4 * 369 ^ (4 * i);
    s = s + t1 / t2;
end
p = 9801 / sqrt(8) / s

```

结果如下

```

p =
    3.141592628644980

```

(6) 利用刘徽割圆术

$$\begin{cases} a_{n+1} = \sqrt{2 - \sqrt{4 - a_n^2}}, \\ \pi = 3 \times 2^{n+1} a_n, \end{cases}$$

其中 a_n 是正 6×2^n 边形的边长。

```

a = 1;
n = 12;
for i = 1:n
    a = sqrt(2 - sqrt(4 - a^2));
end
p = 3 * 2 ^ n * a

```

结果如下

```

p =
    3.141592645321216

```

(7) 利用积分

$$\pi = 4 \int_0^1 \frac{1}{1+x^2} dx.$$

```

n = 1000;
x = linspace(0, 1, n);
y = 1 ./ (1 + x.^ 2);
dx = 1 / n;
s = 4 * sum(y) * dx

```

结果如下


```
s =  
3.141450894102710
```

(8) 利用蒙特卡罗模拟法

$$\pi = 4 \times \frac{m}{n},$$

其中 m 为落在四分之一圆(如图 6.7 的阴影部分)内的点数, n 为总点数。

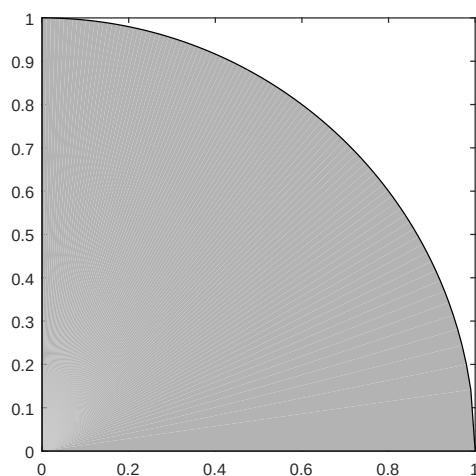


图 6.7: 蒙特卡罗模拟法示意图

```
n = 1000;  
x = linspace(0, 1, n);  
y = 1 ./ (1 + x.^ 2);  
dx = 1 / n;  
s = 4 * sum(y) * dx
```

结果如下

```
p =  
3.141560000000000
```

注 6.6 此时的蒙特卡罗模拟法精度低, 可用增大模拟点数, 或多次模拟求平均值的方法提高精度。

```

n = 100000;
s = zeros(1, 100);
for k = 1:100
    x = rand(1, n);
    y = rand(1, n);
    m = sum((x.^2 + y.^2) <= 1);
    p(k) = 4 * m / n;
end
p = mean(p)

```

结果如下

```

p =
    3.141563200000000

```

2. 绘制函数 $f(x) = \frac{1}{x^2 + 2x - 3}$, $x \in [-8, 6]$ 的图像。

解 方法一 利用便捷函数绘图

```

ezplot('1/(x^2+2*x-3)', [-8, 6]) % 如图 6.8(a)

```

方法二 利用plot函数绘图

```

n = 30;
x1 = linspace(-8, -3, n);
x2 = linspace(-3, 1, n);
x3 = linspace(1, 6, n);
x = [x1, x2, x3];
x(x == -3 | x == 1) = []; % 删除奇异点
y = 1 ./ (x.^2 + 2 * x - 3);
plot(x(x < -3), y(x < -3), 'b', x(x > -3 & x < 1), ...
     y(x > -3 & x < 1), 'b', x(x > 1), y(x > 1), 'b')
axis([-8, 6, -0.9, 0.6]) % 结果如图 6.8(b)

```

方法三 利用fplot函数绘图

```

fun = inline('1/(x.^2+2*x-3)', 'x');
fplot(fun, [-8, -3.1]) % 直接绘制[-8, 6]上的图会报错
hold on
fplot(fun, [-2.9, 0.9])
fplot(fun, [1.1, 6])
axis([-8, 6, -0.9, 0.6])
hold off % 结果如图 6.8(c)

```

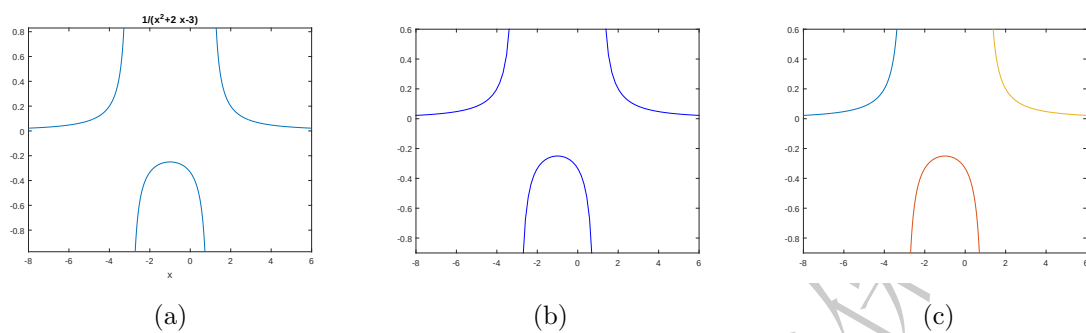


图 6.8: 罔函数的图像

注 6.7 罔函数图只是本书对上述图形的称呼，并非一般的数学名词。三种方法的对比突出了便捷绘图函数的强大威力，希望得到大家的重视。

我的 MATLAB 报告

王保山

第七章 I/O操作

I/O操作指的是用MATLAB读入/写入文件。众所周知，众多科学实验会得到大量的实验数据，而这些数据往往会保存到文件(如.txt文件，.xls文件，.mat文件等)中。如果人工输入到MATLAB中，不仅费时费力，而且容易出错，因而掌握I/O操作就十分必要，主要的I/O指令如表 7.1。

表 7.1: 主要的I/O指令

函数	功能
dlmwrite	允许用户向文件中以ASCII码的形式写入数据。行与行，列与列之间可由用户来指定分隔符
dlmread	读取数据。列数是最大的列数，不足的补零对齐
xlsread	读取Excel文件
xlswrite	写数据到Excel文件中
save	保存数据到指定文件
load	读取指定文件数据
importdata	从磁盘文件载入数据
textread	可以读取纯数字类型的文件，也可以读取包含文本信息和数值信息的文本文件
textscan	与textread类似，比textread功能更强大

这一章我们主要学习.mat文件、.txt文件和.xls文件数据的读写，下面逐一介绍。

7.1 读写mat文件

mat文件是MATLAB保存数据的一种标准的二进制文件，扩展名为“.mat”。它支持MATLAB中所有的数据类型，可通过save、load读写，如表 7.2 所示。

表 7.2: 保存mat文件的指令

指令	功能
save	保存工作变量中的全部数据到matlab.mat中
save filename	保存工作变量中的全部数据到filename中
save filename var	保存变量var到filename中
save filename var1 var2 ...	保存变量var1, var2...到filename中
save filename s*	保存s开头的所有变量到filename中

注 7.1 “*”为通配符，保存中间含有s的变量可用“*s*”，最后字符为s的变量可用“*s”；load用来读入mat文件，是save的逆过程，用法与save类似，不再赘述。

例 7.1 保存变量数据

```
x = 1:9;
x1 = 6:8;
yx = 1:4;
y = 7:9;
save mymat1.mat
save mymat2.mat x x1 y
save mymat3.mat x*      % 结果如图 7.1
```

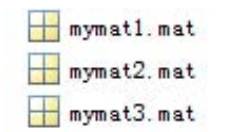


图 7.1: 保存mat文件



图 7.2: mymat1.mat

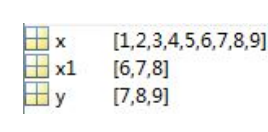


图 7.3: mymat2.mat

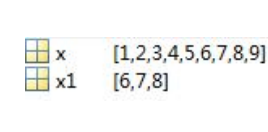


图 7.4: mymat3.mat

```
>> clear
>> load mymat1.mat      % 结果如图 7.2
>> clear
>> load mymat2.mat      % 结果如图 7.3
>> clear
>> load mymat3.mat      % 结果如图 7.4
```

例 7.2 读取myshafa.mat中的数据并绘图

```
load myshafa.mat
mesh(x/3, y, 30*u);
surf(x/3, y, 30*u);
axis off equal
shading interp % 伪彩色图
view(50, 30) % 设置视角，绘图结果如图 7.5
```

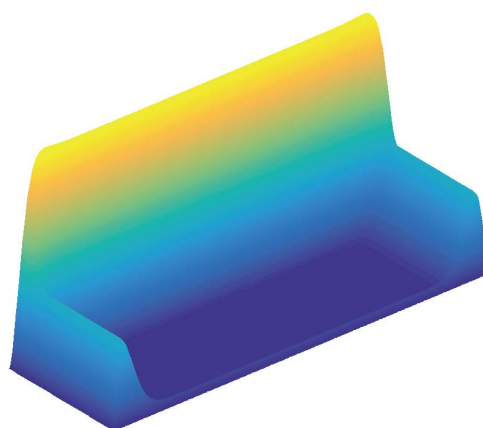


图 7.5: 沙发图

7.2 读写txt文件

读写txt文件的指令如表 7.3 所示。

表 7.3: 读写txt文件的指令

指令	功能
save	保存数据到指定文件，文件要注明txt后缀
load	读取指定文件数据
textread	读取文本文件
textscan	与textread类似，功能更强大

save与load在读取txt文件时与读取mat文件类似，我们重点来了解textread函数和textscan函数的用法，如表 7.4。

表 7.4: textread和textscan的用法

指令	参数说明
[A, B, C, ...] = textread(fname)	fname是文件名，当前目录下的文件不用输全路径；fmt是读取格式，如表 7.5；N为读取次数，默认为1；p为通用参数，v为参数值，如表 7.6
[...] = textread(fname, fmt)	
[...] = textread(fname, fmt, N)	
[...] = textread(fname, fmt, p, v)	
[...] = textread(fname, fmt, N, p, v)	
C = textscan(fid, fmt)	fid为fopen命令返回的文件标识符，fid = fopen(fname)，用textscan读取文件后，需用fclose(fid)使fid无效，用clear fid清除fid
C = textscan(fid, fmt, N)	
C = textscan(fid, fmt, p, v)	
C = textscan(fid, fmt, N, p, v)	

表 7.5: 有效的fmt格式

格式	说明	格式	说明
%n	单精度和整数	%d	十进制整数
%u	带符号的十进制整数	%f	浮点数
%o	带符号八进制整数	%c	字符序列，由域宽指定字符个数
%s	不含空白的字符串，跳过所有的空白符(空格，\n，\t等)		
%[...]	转换字符序列，如'%d%n'，读取数据直到第一个不符合的数据		
%[^...]	转换字符序列，与%[...]相反		

表 7.6: 有效参数与参数值

参数	说明	默认值
bufsize	字符串的最大长度，单位为字节	4095
commentstyle	可忽略的文本	none
delimiter	定界字符	none
emptyvalue	定界文件中空单元的值	NaN
endofline	行尾字符	由文件决定
expchars	指数字符	'eEdD'
headerlines	文字指针跳过的行数	0
whitespace	空线间隔	'\b\t'

注 7.2 此外还有 *MultipleDelimsAsOne*，*ReturnOnError*和*TreatAsEmpty*参数，默认值分别为 0，1，none。

注 7.3 *textscan*函数适于读取大文件的方式比*textread*函数更灵活，因而在之后的版本中*textread*函

数将被取代。

例 7.3 *layer01.txt*存储的是某一地区的地理海拔高度数据，试读取并绘出该地区的二维地图，颜色用来标明海拔高度。

```
[x,y,z] = textread('layer01.txt','%f%f%f');
X = reshape(x, 451, []);
Y = reshape(y, 451, []);
Z = reshape(z, 451, []);
mesh(X, Y, Z) % 三维绘图
axis equal % 等宽显示
axis off % 隐藏坐标轴
shading interp % 伪彩色图
colorbar % 添加颜色条
view(2) % 二维视图
```

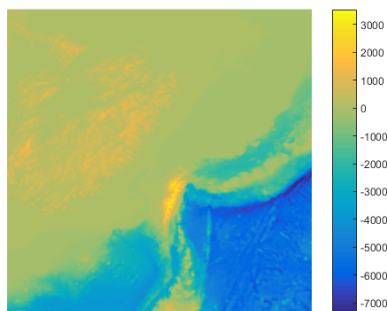


图 7.6: 地形图

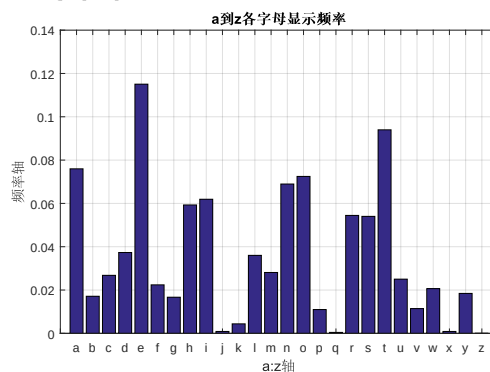


图 7.7: 字母频率统计图

例 7.4 大家对26个英文字母再熟悉不过了，那么怎样统计一段英文中每个字母的频率呢？试编写程序统计一段英文中各字母出现的频率。英文原文保存在*speech.txt*中。

```
A = textread('speech.txt','%c');
a = zeros(1, 26);
b = a;
m = length(A);
for n = 1:26
    a(n) = sum(A == (n + 96));
    b(n) = a(n) / m;
end
name = {' ', char(97:122)'};
bar(b);
axis([0, 27, 0, 0.14])
% 将当前坐标轴x轴的刻度标签设为name
set(gca, 'XTick', 0:27, 'XTickLabel', name);
grid on
title('a到z各字母显示频率');
```

```
xlabel('a:z轴');
ylabel('频率轴')           % 结果如图 7.7
```

例 7.5 有些专业数据是数据与文字说明混合在一起的, 比如, 全球海洋盐度的ARGO数据(如下所示, 20个类似的文件放置在PSU文件夹下, 且均为.dat文件, 该文件可由记事本打开, 它的读写与.txt类似), 这类数据, 我们往往需要读取其中的时间(JULIAN DAY)、经纬度坐标(LATITUDE和LONGITUDE)以及修正盐度(数据首行中的Corrected Salinity (PSU))。

全球海洋盐度的ARGO数据实例

```
*SoftWare Version 2.0  2015/08/30

*HEADER
  PLATFORM NUMBER      :1900726
  .....% 限于篇幅, 省略了部分内容
  DATA MODE           :A(R=Real Time; D=Delayed Mode; A=Real Time Adjusted)
  JULIAN DAY            :23889.5962 (days since 1950-01-01 00:00:00 UTC)
  DATE                 :2015-05-29
*LOCATION
  LATITUDE              :-25.917
  LONGITUDE             :-112.353
*FILE
  COLUMN 1              :Pressure (dbar)                                F7.1
  COLUMN 2              :Corrected Pressure (dbar)                     F7.1
  COLUMN 3              :Quality on Pressure                           I3
  COLUMN 4              :Temperature (degree_Celsius)                  F9.3
  COLUMN 5              :Corrected Temperature (degree_Celsius)        F9.3
  COLUMN 6              :Quality on Temperature                         I3
  COLUMN 7              :Salinity (PSU)                                F9.3
  COLUMN 8              :Corrected Salinity (PSU)                       F9.3
  COLUMN 9              :Quality on Salinity                            I3
=====
  4.5    4.5    1    23.067    23.067    1    36.233    36.218    1
  9.5    9.5    1    23.068    23.068    1    36.233    36.218    1
  .....% 限于篇幅, 省略了部分内容
```

编写读取程序如下

```
fidx = 1; ffmt = {'dat', 'csv'};           % 文件类型和文件扩展名
fidtxt = fopen(['psu.', char(ffmt(fidx))], 'w'); % 打开文件标识符
form = '%8d\t%.2d\t%.8g\t%.8g\t%.8g\n';      % 每行的输出形式
path = 'PSU/';                             % 数据文件夹
file = dir([path, '*.dat']);                 % 获取path目录下文件信息
n = length(file);                           % path的dat文件个数
for i = 1:n                                 % 逐个读取文件
    if (mod(i,100))==0, disp(i); end          % 每100输出一次读取的文件数
    str = char(file(i).name);                 % 当前读取的文件名
    fid = fopen([path, str]);                 % 打开当前文件标识符
    tline = fgetl(fid);                       % 读取当前行文字
```

```

while ischar(tline)                                % 逐行读取文件
    tline = fgetl(fid);                             % 读取当前行文字
    try
        ind = strfind(tline, ':');                  % 获取tline中的位置, 若不包含, 则返回空
        if ~isempty(strfind(tline, 'JULIAN_DAY'))    % 寻找时间行
            ind = strfind(tline, '(');
            nn = str2double(tline(ind+1:ind-1)) + ...
                datenum('1950-01-01 00:00:00');      % 记录时间数
            tt = datestr(nn, 30);                    % 日期字符
            td = str2double(tt(1:8));                 % 计算日期
            hr = str2double(tt(10:11));               % 计算小时
            continue;                                % 节省时间, 直接开始下一循环
        end
        if ~isempty(strfind(tline, 'LATITUDE'))      % 寻找纬度行
            la = str2double(tline(ind+1:end));        % 计算纬度
            continue;                                % 直接开始下一循环
        end
        if ~isempty(strfind(tline, 'LONGITUDE'))    % 寻找经度行
            lo = str2double(tline(ind+1:end));        % 计算经度
            continue;                                % 直接开始下一循环
        end
        if ~isempty(strfind(tline, 'Corrected_Salinity'))
            indc = strfind(tline, 'COLUMN');          % 读取'COLUMN'位置
            column = str2num(tline(indc+7:ind-1));    % 读取PSU列数
            continue;                                % 直接开始下一循环
        end
        if ~isempty(strfind(tline, '====='))
            tline = fgetl(fid);                        % 读取tline
            dd = str2num(tline); psu = dd(column);    % 读取PSU
            fprintf(fidtxt, form, [td, hr, la, lo, psu]); % 写入目标文件
            break;                                    % 结束当前文件的读取
        end
    catch
        char(file(i).name)                            % 若当前文件读取有误
        % 输出文件名
    end
end
fclose(fid);                                          % 关闭文件标识符
end
fclose(fidtxt);                                      % 读取结束, 关闭文件占用

```

结果如下

20150505	02	-36.601	105.208	35.283
20150509	09	-25.994	-111.965	36.123
20150529	14	-25.917	-112.353	36.218
20150502	13	-31.171	-113.446	35.314
20150504	04	-32.263	-171.075	35.756
20150508	16	-57.077	-117.794	33.872
20150502	01	-46.206	-159.324	34.584
20150504	10	-26.055	90.25	35.664
20150502	02	-44.85	112.822	34.792
20150502	02	-56.65	-114.87	34.016
20150504	20	-10.438	42.412	34.88
20150501	00	33.693	21.571	38.176
20150503	13	-47.716	61.321	33.755

20150503	20	36.338	19.655	38.433
20150504	08	37.072	18.381	38.746
20150504	14	33.626	16.32	37.85
20150502	23	32.965	27.573	38.533
20150503	08	34.051	27.673	38.637
20150503	17	35.92	19.438	38.842
20150504	03	34.341	29.162	39.134

注 7.4 与例 7.3和例 7.4不同的是，该程序使用了`fgetl`函数和`strfind`函数，每次读取数据文件的一行并判断该行中是否包含所需数据。为减少不必要的时间，程序中还使用了`continue`命令。

7.3 读写Excel文件

读取Excel文件的常用函数如表 7.7 所示。

表 7.7: 读取Excel文件的常用函数

指令	功能
<code>num = xlsread(fname)</code>	将名为 <code>fname</code> 的文件的数据读入 <code>num</code> 中
<code>num = xlsread(fname, -1)</code>	允许用户选择要读取的数据范围，读取数据
<code>num = xlsread(fname, sheet)</code>	<code>sheet</code> 为工作表名称的字符串
<code>num = xlsread(fname, range)</code>	<code>range</code> 表示数据范围,格式如'A1:A12', 'B1:B12'
<code>num = xlsread(fname, sheet, range)</code>	读取工作表 <code>sheet</code> 中 <code>range</code> 范围内的数据
<code>[num, txt] = xlsread(...)</code>	文字与数字混杂时读入数据 <code>num</code> 和文字 <code>txt</code>
<code>[num, txt, raw] = xlsread(...)</code>	<code>raw</code> 为未处理的单元数据

注 7.5 如果工作表中既包含数字数据又包含文字信息，则此时会把数据读入到`num`中，自动忽略首行文本行，尾行文本行，首列文本列，尾列文本列。对于数字和文本混杂的行或列，`MATLAB`会在相应的位置用`NaN`来代替。

例 7.6 读取`information.xls`中的数据

解 方法一

```
>> num = xlsread('information.xls')
num =
    1.0e+03 *
    1.8300    1.8400    1.8500    1.8600    1.8700    1.8800
    0.0129    0.0171    0.0232    0.0314    0.0386    0.0502
    1.8900    1.9000    1.9100    1.9200    1.9300    1.9400
```

```

0.0629    0.0760    0.0920    0.1065    0.1232    0.1317
1.9500    1.9600    1.9700    1.9800    1.9900    2.0000
0.1507    0.1793    0.2040    0.2265    0.2514    0.2750

```

注 7.6 MATLAB中 $a \times 10^b$ 可表示为 aeb ，如 1.5×10^{-6} 可以表示为 $1.5e-06$ 。

方法二

```

>> [num,txt] = xlsread('information.xls')
num =
    1.0e+03 *
    1.8300    1.8400    1.8500    1.8600    1.8700    1.8800
    0.0129    0.0171    0.0232    0.0314    0.0386    0.0502
    1.8900    1.9000    1.9100    1.9200    1.9300    1.9400
    0.0629    0.0760    0.0920    0.1065    0.1232    0.1317
    1.9500    1.9600    1.9700    1.9800    1.9900    2.0000
    0.1507    0.1793    0.2040    0.2265    0.2514    0.2750
txt =
    '年',
    '人口 (百万)',
    '年',
    '人口 (百万)',
    '年',
    '人口 (百万)',

```

方法三



图 7.8: 选中待读取的数据

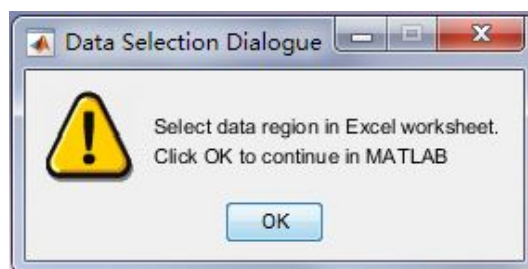


图 7.9: 数据选择对话框

```

>> num = xlsread('information.xls', -1)
num =
    1.0e+03 *
    1.8300    1.8400    1.8500    1.8600    1.8700    1.8800
    0.0129    0.0171    0.0232    0.0314    0.0386    0.0502
    1.8900    1.9000    1.9100    1.9200    1.9300    1.9400

```

```

0.0629    0.0760    0.0920    0.1065    0.1232    0.1317
1.9500    1.9600    1.9700    1.9800    1.9900    2.0000
0.1507    0.1793    0.2040    0.2265    0.2514    0.2750

```

方法四

```

>> num = xlsread('information.xls', '人口预测')
num =
    1.0e+03 *
    1.8300    1.8400    1.8500    1.8600    1.8700    1.8800
    0.0129    0.0171    0.0232    0.0314    0.0386    0.0502
    1.8900    1.9000    1.9100    1.9200    1.9300    1.9400
    0.0629    0.0760    0.0920    0.1065    0.1232    0.1317
    1.9500    1.9600    1.9700    1.9800    1.9900    2.0000
    0.1507    0.1793    0.2040    0.2265    0.2514    0.2750

```

方法五

```

>> num = xlsread('information.xls', '人口预测', 'B1:G6')
num =
    1.0e+03 *
    1.8300    1.8400    1.8500    1.8600    1.8700    1.8800
    0.0129    0.0171    0.0232    0.0314    0.0386    0.0502
    1.8900    1.9000    1.9100    1.9200    1.9300    1.9400
    0.0629    0.0760    0.0920    0.1065    0.1232    0.1317
    1.9500    1.9600    1.9700    1.9800    1.9900    2.0000
    0.1507    0.1793    0.2040    0.2265    0.2514    0.2750

```

方法六

```

>> [num,txt,row] = xlsread('information.xls', '人口预测')
num =
    1.0e+03 *
    1.8300    1.8400    1.8500    1.8600    1.8700    1.8800
    0.0129    0.0171    0.0232    0.0314    0.0386    0.0502
    1.8900    1.9000    1.9100    1.9200    1.9300    1.9400
    0.0629    0.0760    0.0920    0.1065    0.1232    0.1317
    1.9500    1.9600    1.9700    1.9800    1.9900    2.0000
    0.1507    0.1793    0.2040    0.2265    0.2514    0.2750
txt =
    '年'
    '人口 (百万)'
    '年'
    '人口 (百万)'
    '年'
    '人口 (百万)'
row =
    Columns 1 through 4
    '年'      [    1830]      [    1840]      [    1850]

```

```
'人口（百万）' [ 12.9000] [ 17.1000] [23.2000]
'年'           [ 1890] [ 1900] [ 1910]
'人口（百万）' [ 62.9000] [ 76] [ 92]
'年'           [ 1950] [ 1960] [ 1970]
'人口（百万）' [150.7000] [179.3000] [ 204]
Columns 5 through 7
[ 1860] [ 1870] [ 1880]
[ 31.4000] [ 38.6000] [ 50.2000]
[ 1920] [ 1930] [ 1940]
[106.5000] [123.2000] [131.7000]
[ 1980] [ 1990] [ 2000]
[226.5000] [251.4000] [ 275]
```

使用xlswrite函数可以将数据写到Excel文件中。主要用法为

```
xlswrite(fname, M, sheet, range)
```

注 7.7 *M*为要写入的数据。若名为*fname*的文件不存在，*MATLAB*会自动新建文件并写入数据。

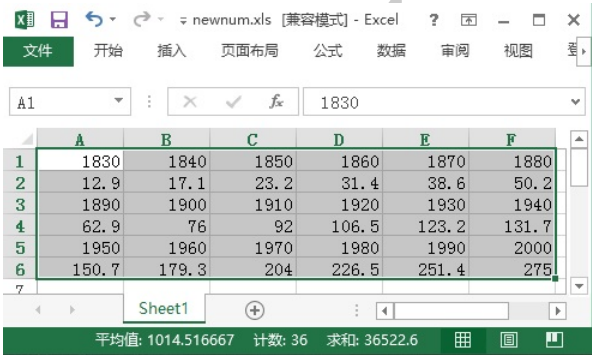


图 7.10: newnum.xls的写入

例 7.7 将例 7.6 中的*num*写入*newnum.xls*中

```
>> xlswrite('newnum.xls', num) % 结果如图 7.10
```

7.4 读写图像文件

读写图像文件如表 7.8所示。

表 7.8: 读写图像文件的常用函数

指令	功能
imread	读取图像文件(详见表 7.9)
imwrite	写数据到图像文件中(详见表 7.10)

表 7.9: imread函数的常用格式

指令	功能
A = imread(fname, fmt)	读取扩展名为fmt的fname图像文件
[X, map] = imread(fname, fmt)	把文件fname读取到X的map向量中
[...] = imread(fname)	从filename中识别格式, 读取文件
[...] = imread(url, ...)	用url读取网络图片

表 7.10: imwrite函数的常用格式

指令	功能
imwrite(A, fname, fmt)	将A写入扩展名为fmt的fname文件中
imwrite(X, map, fname, fmt)	将[X, map]写入fname文件中
imwrite(..., fname, fmt)	写入扩展名为fmt的fname文件中
imwrite(..., p, v, ...)	设置参数p的值为v
imshow(im)	在坐标轴上显示图片, 保持原有比例
image(im)	显示图片, 调整比例适应坐标轴

注 7.8 参数p较多, 读者可以使用`help imwrite`查看, 这里不再详细介绍。

例 7.8 读取`eg.bmp`, 原图如图 7.11(a), 注意`image`与`imshow`之间的区别。

解 方法一

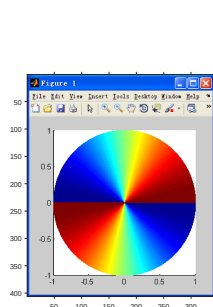
```
A = imread('eg.bmp');
imshow(A);    % 结果如图 7.11(a)
```

方法二

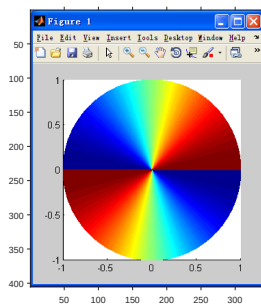
```
A = imread('eg.bmp', 'bmp');
imshow(A);    % 结果如图 7.11(b)
```

方法三

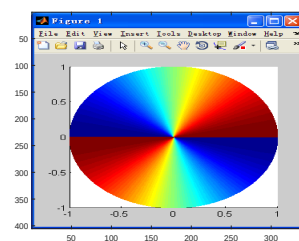

```
A = imread('eg.bmp', 'bmp');
image(A);    % 结果如图 7.11(c)
```



(a) 原图



(b) imshow



(c) image

图 7.11: 图像的读取与展示

例 7.9 读取网络图片

```
A = imread(['http://img.bimg.126.net/photo/'...
'idDQNqm7mDnd0LiHaCZyJg==/3174474787343523137.jpg'], 'jpg');
imshow(A)    % 结果如图 7.12
```

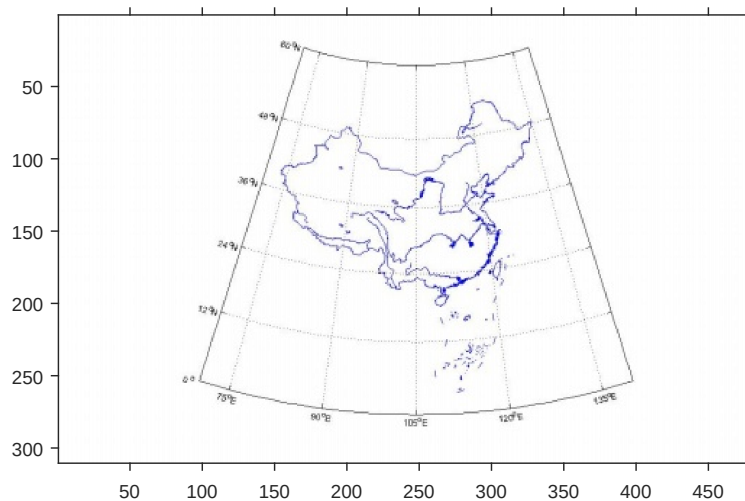


图 7.12: 网络上的中国地图

7.5 GIF动画制作

首先了解一下从Figure界面保存图片的代码：

```
for k = 1:n
    %
    % 在这里输入绘图指令
    %
    w = getframe;           % 获取以坐标轴为界的图
    imind = frame2im(w);    % 将单帧图像转化为RGB图
    [imind, cm] = rgb2ind(imind, 256); % RGB图转为索引图
    % 图片保存
    if k ~= 1
        imwrite(imind, cm, filename, 'gif',...
            'WriteMode', 'append', 'DelayTime', 0.01);
    else
        imwrite(imind, cm, filename, 'gif', ...
            'Loopcount', inf, 'DelayTime', 0.01);
    end
end
end
```

注 7.9 这里用到的*imwrite*参数的意义和有效参数值如表 7.11。

表 7.11: *imwrite*函数的常用格式

参数	说明	有效值
WriteMode	写入模式	{overwrite}(覆盖)、append(扩展)
DelayTime	间隔时间	0 到655中的任意数
LoopCount	循环次数	{Inf}或0到65535的任意整数
BackgroundColor	背景颜色	颜色字符串或颜色数组

注 7.10 “{ }”内的值为默认值。

例 7.10 万花尺由母尺和子尺两部分组成。常见的母尺是内环形齿轮，子尺是带多孔的外环形齿轮。作画时，将子尺内置于母尺内环之中，轮牙镶嵌，笔头插在子尺的小孔中，用笔带动子尺顺着母尺的内沿齿轮反复作圆周运动。在作画过程中，两者内外齿要始终靠合。完成后纸上便会留下一个不可思议的美丽花朵。子尺上小孔的极小位移会引起图案类型的极大变化。子尺形状的变化有很多，除了圆形，还有椭圆形，弧边的三角形、十字形、梅花形、方形、多边形等等。母尺的变化不大，但有些母尺的外沿也带齿轮，把子尺置于母尺外沿作环绕运动，这时画出的是花形圈。简化万花尺，将母尺和子尺视为大圆和小圆，编写万花尺函数，并保存动画以记录笔尖的移动轨迹。

```

function flowers(a, b, r, mode, N, gifname)
% 万花尺程序
% Input -a 大圆半径-b 小圆半径-r 画笔距小圆圆心的距离
%       -mode: 非0表示小圆沿大圆内侧转动
%              0表示小圆沿大圆外侧转动
%       -gifname gif文件名
% 有趣的参数和图形
%       a = 1; b = 1;      mode = 0; r = -b; % 1花尺, 心形线
%       a = 4; b = 1.2;    mode = 1; r = b;  % 10花尺
%       a = 4; b = 2;      mode = 1; r = 3;  % 2花尺
%       a = 4; b = 1;      mode = 1; r = b;  % 4花尺
%       a = 4; b = 1.5;    mode = 1; r = b;  % 8花尺
%       a = 4; b = 0.75;   mode = 1; r = b;  % 16花尺
%       a = 4; b = 8/3;    mode = 1; r = b;  % 3花尺
%       a = 4; b = 8/9;    mode = 1; r = b;  % 9花尺
%       a = 4; b = 4/3;    mode = 1; r = b/2; % 3花尺
%       a = 4; b = 1;      mode = 1; r = b/2; % 4花尺
%       a = 4; b = 2;      mode = 1; r = b;  % 2花尺
if nargin < 3, r = b; end % 输入参数个数<3时
if nargin < 4 || mode, mode = -1; else mode = 1; end
if nargin < 5, N = 10; end
if nargin < 6, gifname = 'flowers.gif'; end

[k, m] = cylnumber(a, b, N);
t = 0:pi/200:2*k*pi;
tt = 0:pi/200:2*pi;
x1 = a + mode * b;
xx = a * cos(tt);
yy = a * sin(tt);
xm = a + abs(r) + abs(mode * b);

plot(xx, yy);
axis equal off
title([num2str(m), '花尺'])

% 设置颜色、标记、标记尺寸、擦除模式
h = line('Color', 'k', 'Marker', '*', 'MarkerSize', 1);
h2 = line('Color', 'k', 'Marker', '.', 'MarkerSize', 8);
h1 = line('Color', 'r', 'Marker', '.', 'MarkerSize', 20);

n = length(t);
if mode == -1, nk = n:-1:1; else nk = 1:n; end

for k = 1:n

    x = x1 * cos(t(k)) + r * cos(x1 / b * t(nk(k)));
    y = x1 * sin(t(k)) + r * sin(x1 / b * t(nk(k)));
    x2 = x1 * cos(t);
    y2 = x1 * sin(t);
    x3 = x2(k) + b * cos(tt);
    y3 = y2(k) + b * sin(tt);

    hold on
    set(h, 'xdata', x3, 'ydata', y3); % 画图、画线设置
    plot(x, y, 'r.');
```

```

try
    f = getframe;
    imind = frame2im(f);
    [imind, cm] = rgb2ind(imind, 256);

    if k ~= 1
        imwrite(imind, cm, gifname, 'gif', ...
            'WriteMode', 'append', 'DelayTime', 0.05)
    else
        imwrite(imind, cm, gifname, 'gif', ...
            'Loopcount', inf, 'DelayTime', 0.05)
    end
end

pause(0.01)

end

function [k, m] = cylnumber(a, b, N)
k = 1;
m = a / b;
while rem(m, 1) && k < N
    k = k + 1;
    m = k * a / b;
end
end

```

调用flowers函数，结果如图 7.13 所示。



图 7.13: 万花尺动画截图

例 7.11 读取一系列图片 (*b1.jpg, b2.jpg, ..., b7.jpg*), 生成 GIF 动画。

```

for k = 1:7
    filename = ['b', num2str(k), '.jpg'];
    m = imread(filename);
    [imind, cm] = rgb2ind(m, 256);
    if k ~= 1
        imwrite(imind, cm, 'out.gif', 'gif', ...
            'WriteMode', 'append', 'DelayTime', 1)
    else
        imwrite(imind, cm, 'out.gif', 'gif', ...
            'Loopcount', inf, 'DelayTime', 1)
    end
end

```

```
end
end % 结果如图 7.14
```

注 7.11 注意批量读取图片的方法。

赵钱孙李 奚范彭郎
周吴郑王 鲁韦昌马
冯陈褚卫 苗凤花方
蒋沈韩杨 俞任袁柳
朱秦尤许 酆鲍史唐
何吕施张 费廉岑薛
孔曹严华 雷贺倪汤
金魏陶姜 滕殷罗毕
戚谢邹喻 郝邬安常
柏水窦章 乐于时傅
云苏潘葛 皮卞齐康

(a)

伍余元卜 席季麻强
顾孟平黄 贾路娄危
和穆萧尹 江童颜郭
姚邵湛汪 梅盛林刁
祁毛禹狄 鍾徐丘骆
米贝明臧 高夏蔡田
计伏成戴 樊胡凌霍
谈宋茅庞 虞万支柯
熊纪舒屈 咎管卢莫
项祝董梁 经房裘缪
杜阮蓝闵 干解应宗

(b)

丁宣贲邓 乌焦巴弓
郁单杭洪 牧隗山谷
包诸左石 车侯宓蓬
崔吉钮龚 全郗班仰
程嵇邢滑 秋仲伊宫
裴陆荣翁 宁仇栾暴
荀羊於惠 甘斜厉戎
甄巍家封 祖武符刘
芮羿储靳 景詹束龙
汲邴糜松 叶幸司韶
井段富巫 郤黎蓟薄

(c)

图 7.14: 动画截图

7.6 实验报告

2013年“高教社杯”全国大学生数学建模竞赛赛题B题第一问：对于给定的来自同一页印刷文字的文件的碎纸机破碎纸片(仅纵切)，建立碎纸片拼接复原模型和算法，并针对附件1、附件2给出的中、英文各一页文件的碎片数据进行拼接复原。如果复原过程需要人工干预，请写出干预方式及干预的时间节点。

解 编写程序如下：

```
str = '附件1/'; % 英文图片在附件2
%% 获取str目录下的所有文件名
str1 = ls(str);
str1 = str1(3:end,:);
%% 计算文件数目，初始化索引
len = length(str1);
ind = 1:len;
%% 读取图片信息
for i = ind
    a = double(imread([str,str1(i,:)]));
    data(i).im = a;
    datae1(:,i) = a(:,1);
```

```

datae2(:,i) = a(:,size(a,2));
end
%% 查找原来第一个图片条的索引
indexn = 1:len;
ind(1) = indexn(all(datae1 == 255));
%% 计算各图片条的边界数据的距离, 这里用方差(详见第十三章)表示
for i = 1:len
    for j = 1:len
        co(i,j) = var((datae1(:,i) - datae2(:,j)));
    end
end
[~, b] = min(co);
%% 获取索引值
for i = 1:len-1
    ind(i+1) = b(ind(i));
end
%% 图像拼接
im = [];
for i = 1:len
    im = [im, data(ind(i)).im];
end
imwrite(im, 'out.jpg') % 保存图片, 如图 7.15
imshow(im) % 显示图片

```

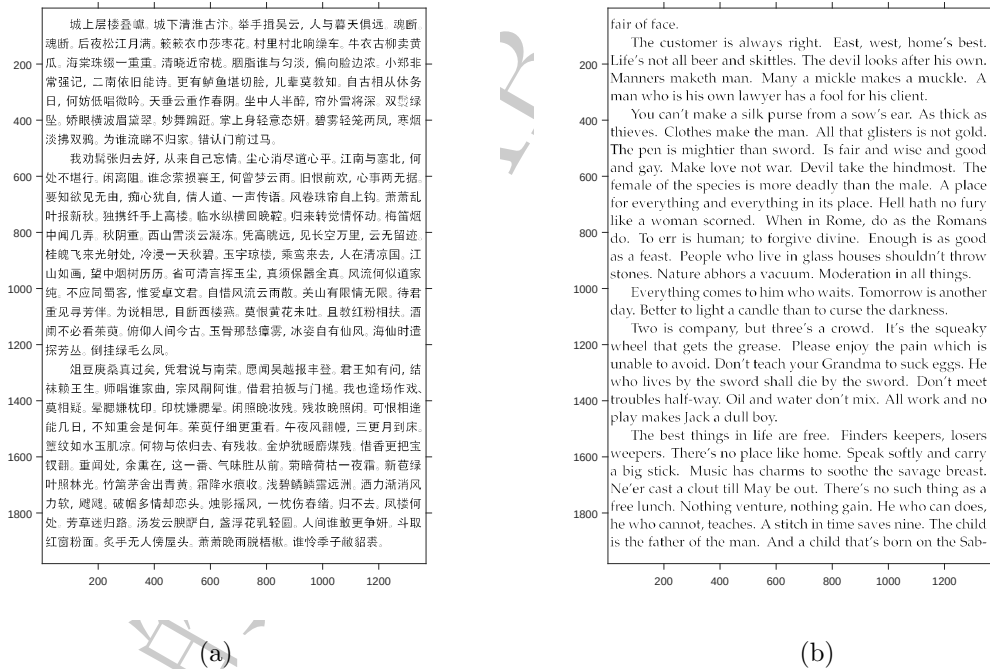


图 7.15: 碎纸片复原

注 7.12 由于该程序用到统计方面的知识, 请事先了解相关统计学概念(详见第十三章)。另外, 注意图片数据的保存和拼接, 批量处理时经常使用这种方法。

第二部分

数值计算篇

我的 MATLAB 报告

王保山

第八章 数值代数

本章主要介绍求解线性方程组的数值算法和求解矩阵特征值、特征向量的数值算法。这一章只给出算法和程序，对于算法推导，有兴趣的读者可以参考徐萃薇、孙绳武编著的《计算方法引论》(第三版)第六章、第八章、第九章的内容。为方便阅读，现给出算法对应表。

表 8.1: 本章内容与《计算方法引论》内容的对应表

算法	《计算方法引论》中对应的页数
Gauss消去法	105 ~ 114
主元素消去法	114 ~ 118
LU分解	118 ~ 124
Cholesky分解	124 ~ 127
Jacobi迭代法	160 ~ 163
Gauss-Seidel迭代法	163 ~ 164
松弛法	164 ~ 166
幂法	185 ~ 190
反幂法	191 ~ 192

8.1 消去法

8.1.1 Gauss消去法

Gauss消去法是一个古老的方法，其基本做法就是把方程组通过消元转化成与之等价的三角方程组，然后逐个回代求解 $x_n, x_{n-1}, \dots, x_1$ ，如下：

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \dots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n \end{cases} \Rightarrow \begin{cases} u_{11}x_1 + u_{12}x_2 + \dots + u_{1n}x_n = g_1 \\ u_{22}x_2 + \dots + u_{2n}x_n = g_2 \\ \dots \\ u_{nn}x_n = g_n \end{cases}$$

我们先从最简单的上三角方程组开始求解。

算法 8.1 求解上三角方程组的算法($BACKSUB(A, b, n)$)。

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + u_{1n}x_n = b_1 \\ a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \dots \\ a_{nn}x_n = b_n \end{cases}$$

我们可以从 x_n 出发，求解 x_{n-1} ，再利用 x_n 和 x_{n-1} 求解 x_{n-2} ，依次进行下去，便可以求出所有的未知量，这就是回代过程，算法如下：

Algorithm 1: $BACKSUB(A, b, n)$

Input: A, b, n

Output: x

```

1  $x_n = b_n / a_{nn}$ 
2 for  $k = n - 1, n - 2, \dots, 1$  do
3    $x_k = (b_k - a_{k,k+1}x_{k+1} - \dots - a_{kn}x_n) / a_{kk}$ 
4 end
```

算法 1 在MATLAB中可编写为

```

function x = backsub(A, b)
% Input -A n×n矩阵 -b n×m矩阵
% Output -x n×m矩阵, 方程组Ax = b的解
[n, m] = size(b);
x = zeros(n, m);
x(n,:) = b(n,:) / A(n,n);
for k = n-1:-1:1
    x(k,:) = (b(k,:) - A(k,(k+1):n) * x((k+1):n,:)) / A(k,k);
end
```

注 8.1 该程序考虑了右端项为 m 列的情况，并对每一列进行循环求解。

例 8.1 求解上三角方程组

$$\begin{cases} 2x_1 + 2x_2 + 3x_3 = 3 \\ 3x_2 + x_3 = -5 \\ 6x_3 = 6 \end{cases}$$

```
>> A = [2 2 3; 0 3 1; 0 0 6]; b = [3; -5; 6];
>> x = backsub(A, b)
x =
     2
    -2
     1
```

算法 8.2 Gauss消去法 ($GAUSS(A, b, n)$).

Algorithm 2: $GAUSS(A, b, n)$

Input: A, b, n

Output: x

```
1 for  $k = 1, 2, \dots, n-1$  do
2   if  $a_{kk} = 0$  stop
3   for  $i = k+1, k+2, \dots, n$  do
4      $a_{ik} = a_{ik}/a_{kk}$ 
5     for  $j = k+1, k+2, \dots, n$  do
6        $a_{ij} = a_{ij} - a_{ik}a_{kj}$ 
7     end
8      $b_i = a_i - a_{ik}b_k$ 
9   end
10 end
11  $x = BACKSUB(A, b, n)$ 
```

相应的MATLAB程序为

```
function x = gauss(A, b)
% Input   -A n×n矩阵 -b n×m矩阵
% Output  -x n×m矩阵, 方程组Ax = b的解
n = size(A, 1);
for k = 1:n-1
    if A(k,k) % 逻辑判断, 非零为真
        for i = k+1:n
            A(i,k) = A(i,k) / A(k,k);
```

```

        for j = k+1:n
            A(i,j) = A(i,j) - A(i,k) * A(k,j);
        end
        b(i,:) = b(i,:) - A(i,k) * b(k,:);
    end
else
    error('Matrix is singular to working precision.')
end
end
x = backsub(A, b);

```

例 8.2 用 *Gauss* 消去法求解方程组 $Ax = b$, 其中

$$A = \begin{pmatrix} 2 & 2 & 3 \\ 4 & 7 & 7 \\ -2 & 4 & 5 \end{pmatrix}, b = \begin{pmatrix} 3 \\ 1 \\ -7 \end{pmatrix}$$

```

>> A = [2 2 3; 4 7 7; -2 4 5];
>> b = [3; 1; -7];
>> x = gauss(A, b)
x =
     2
    -2
     1

```

8.1.2 列主元消去法

列主元 *Gauss* 消去法是按列选取绝对值最大的元素, 通过换行使之成为主元素(对角线元素), 这种方法对舍入误差增长的控制较为有利。

算法 8.3 列主元 *Gauss* 消去法 ($CMEGAUSS(A, b, n)$)。

对应的 MATLAB 程序为

```

function x = cmegauss(A, b)
% 列主元Gauss消去法
% Input   -a n×n矩阵 -b n×m矩阵
% Output  -x n×m矩阵, 方程组Ax = b的解
%% 初步处理
n = size(A, 1); % 获得方程组个数n
A = [A, b];      % A增广矩阵
%% 列主元Gauss消去法
for k = 1:n-1
    %% 选列主元
    [m, m] = max(abs(A(k:end,k))); % 选主元, 记行标m
    A([k, k+m-1], k:end) = A([k+m-1, k], k:end); % 互换两行

```

Algorithm 3: *CMEGAUSS*(A, b, n)**Input:** A, b, n **Output:** x

```

1 for  $k = 1, 2, \dots, n-1$  do
2   按列选主元素  $\alpha := \max_{k \leq i \leq n} |a_{ik}|$ , 并保存主元素所在的行标  $i_k$ 
3   若  $\alpha = 0$ , 则矩阵  $A$  奇异, 停止计算; 否则, 顺序执行
4   若  $i_k = k$ , 则转向第5步; 否则换行
      
$$\begin{cases} a_{i_k, j} \leftrightarrow a_{kj}, & j = 1, 2, \dots, n \\ b_{i_k} \leftrightarrow b_k \end{cases}$$

5   计算乘子  $m_{ik} = a_{ik}/a_{kk} \Rightarrow a_{ik}, i = k+1, k+2, \dots, n$ 
6   消元
      
$$\begin{cases} a_{ij} := a_{ij} - m_{ik}a_{kj}, & i, j = k+1, k+2, \dots, n \\ b_i := b_i - m_{ik}b_k, & i = k+1, k+2, \dots, n \end{cases}$$

7 end
8  $x = \text{BACKSUB}(A, b, n)$ 

```

```

%% Gauss消去法
if A(k,k)
    A(k+1:n,k) = A(k+1:n,k) / A(k,k); % 消元系数
    A(k+1:n,k+1:end) = A(k+1:n,k+1:end) - ...
        A(k+1:n,k) * A(k,k+1:end); % 消元
else
    error(['Warning: Matrix is singular to working precision because of A(', num2str(k), ...
        ', ', num2str(k), ') = 0']);
end
end
x = backsub(A(:,1:n), A(:,n+1:end)); % 回代函数

```

例 8.3 用列主元 Gauss 消去法再求方程组 $Ax = b$ 的解, 其中

$$A = \begin{pmatrix} 2 & 2 & 3 \\ 4 & 7 & 7 \\ -2 & 4 & 5 \end{pmatrix}, b = \begin{pmatrix} 3 \\ 1 \\ -7 \end{pmatrix}$$

```

>> A = [2 2 3; 4 7 7; -2 4 5];
>> b = [3; 1; -7];
>> x = cmegauss(A, b)
x =
    2.0000

```

-2.0000
1.0000

还有一种主元素消去法,叫做全主元消去法,它不再按列找按模最大者作为主元,而是在整个系数矩阵中寻找模最大的元素,因此与列主元消去法相比,全主元消去法又多了换列过程,读者可参考实验报告第6题的程序类似地理解。

8.2 分解法

8.2.1 LU分解法

当方程组 $Ax = b$ 的系数矩阵 A 可以实现LU分解时,即

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} = \begin{pmatrix} 1 & & & \\ l_{21} & 1 & & \\ \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & \cdots & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ & u_{22} & \cdots & u_{2n} \\ & & \ddots & \vdots \\ & & & u_{nn} \end{pmatrix} = LU,$$

这样方程组的求解就变得很简单;当多个方程组的系数矩阵相同时,即 $Ax = b$, $Ay = c$, $Az = d$, $\dots$, 先对 A 进行一次LU分解可大大降低运算量。

算法 8.4 LU分解法可用多种方法来实现,现给出基于Gauss消去法的LU分解($LUFACT(A, n)$)。

Algorithm 4: $LUFACT(A, n)$

Input: A, n

Output: L, U

```

1 for  $k = 1, 2, \dots, n-1$  do
2   for  $j = k, k+1, \dots, n$  do
3      $u_{kj} = a_{kj} - l_{k1}u_{1j} - l_{k2}u_{2j} - \cdots - l_{k,k-1}u_{k-1,j}$ 
4   end
5   for  $i = k+1, k+2, \dots, n$  do
6      $l_{ik} = (a_{ik} - l_{i1}u_{1k} - l_{i2}u_{2k} - \cdots - l_{i,k-1}u_{k-1,k})/u_{kk}$ 
7   end
8 end
```

对应的MATLAB程序为

```

function [L, U] = lufact(A)
% LU分解
% Input -A n×n矩阵
% Output -L 单位下三角矩阵 -U 上三角矩阵
n = size(A, 1);
for k = 1:n-1
    if A(k,k)
        A(k+1:n,k) = A(k+1:n,k) / A(k,k);
        A(k+1:n,k+1:end) = A(k+1:n,k+1:end) - ...
            A(k+1:n,k) * A(k,k+1:end);
    else
        error(['Warning: Matrix is singular to working precision because of A(', num2str(k), ...
            ', ', num2str(k), ') = 0']);
    end
end
U = triu(A);           % 抽取上三角U
L = tril(A);           % 抽取下三角
L(1:n+1:end) = 1;      % 或者用L(eye(n) == 1) = 1;

```

将系数矩阵分解后，原方程组就转化为两个三角形方程组 $Ly = b$, $Ux = y$ 参考消去法的回代过程，原方程的求解程序可以很容易地写出：

```

function x = lubacksub(L, U, b)
% LU分解求方程组
% Input -L 单位下三角矩阵 -U 上三角矩阵 -b 右端项
% Output -x LUx = b的解
n = size(U, 1);
% 解单位下三角形方程组
for k = 2:n
    b(k,:) = b(k,:) - L(k,1:k-1) * b(1:k-1,:);
end
x = backsub(U, b); % 解上三角形方程组

```

例 8.4 用LU分解法再求方程组 $Ax = b$ 的解，其中

$$A = \begin{pmatrix} 2 & 2 & 3 \\ 4 & 7 & 7 \\ -2 & 4 & 5 \end{pmatrix}, b = \begin{pmatrix} 3 \\ 1 \\ -7 \end{pmatrix}$$

```

>> A = [2 2 3; 4 7 7; -2 4 5];
>> b = [3; 1; -7];
>> [L, U] = lufact(A)
L =
     1     0     0
     2     1     0
    -1     2     1
U =
     2     2     3
     0     3     1

```

```

      0      0      6
>> x = lubacksub(L, U, b)
x =
     2
    -2
     1

```

8.2.2 Cholesky分解法

Cholesky分解法，又称平方根法，它将对称正定矩阵 A 分解为 $A = LL^T$ ，其中 L 是一个实的非奇异的下三角矩阵。当限定 L 的对角线元素为正时，分解是唯一的。实现Cholesky分解的方法有很多，按列计算的直接分解算法($CHOLESKY(A, n)$)如下：

算法 8.5 Cholesky分解

Algorithm 5: $CHOLESKY(A, n)$

Input: A, n

Output: L

```

1 for  $k = 1, 2, \dots, n$  do
2    $l_{kk} = (a_{kk} - \sum_{i=1}^{k-1} l_{ki}^2)^{1/2}$ 
3    $l_{jk} = (a_{jk} - \sum_{i=1}^{k-1} l_{ji}l_{ki})/l_{kk}, j = k+1, k+2, \dots, n$ 
4 end

```

用Cholesky分解法求解方程组 $Ax = b$ 的程序如下：

```

function x = cholesky(A, b)
% Cholesky分解法
% Input  -A n×n对称正定矩阵 -b n维列向量
% Output -x n维列向量, Ax = b的解
n = length(b);
L = eye(n);
% 求矩阵L
for k = 1:n
    L(k,k) = sqrt(A(k,k) - L(k,1:k-1) * L(k,1:k-1)');
    L(k+1:n,k) = (A(k+1:n,k) - L(k+1:n,1:k-1) * ...
        L(k,1:k-1)') / L(k,k);
end
% 求解下三角方程组Ly = b
y = zeros(n,1);
y(1) = b(1) / L(1);
for k = 2:n
    y(k) = (b(k) - L(k,1:k-1) * y(1:k-1)) / L(k,k);
end
x = y;

```



```
end
x = backsub(L', y); % 求解上三角方程组L'x = y
```

例 8.5 用 *Cholesky* 分解法求方程组 $Ax = b$ 的解, 其中

$$A = \begin{pmatrix} 5 & -4 & 1 \\ -4 & 6 & -4 \\ 1 & -4 & 6 \end{pmatrix}, b = \begin{pmatrix} 2 \\ -1 \\ -1 \end{pmatrix}$$

```
>> A = [5 -4 1; -4 6 -4; 1 -4 6];
>> b = [2; -1; -1];
>> x = chollesky(A, b)
x =
    0.3333
   -0.1667
   -0.3333
>> format rat % 显示为有理格式
>> x
x =
    1/3
   -1/6
   -1/3
>> format short % 恢复为短格式
```

8.3 迭代法

迭代法是利用迭代序列逐步逼近正确解的近似方法。在系数矩阵稀疏时, 迭代法比直接法更为有效, 但迭代法需要考虑迭代序列的构造、是否收敛、收敛速率以及误差估计等众多问题。迭代法主要有Jacobi迭代、Gauss-Seidel迭代、松弛法, 其基本思想是利用方程组构造向量序列 $x^{(n)}$, 使它收敛到某个极限向量 x^* , 于是 x^* 就是方程组的解。

8.3.1 Jacobi迭代

设方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \cdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

的系数矩阵 A 非奇异, 且 $a_{ii} \neq 0$, 令

$$\begin{cases} b_{ij} = -a_{ij}/a_{ii}, & i \neq j, \\ g_i = b_i/a_{ii}, & i = 1, 2, \cdots, n, \end{cases}$$

方程组可以改写为

$$\begin{cases} x_1 = & +c_{12}x_2 + c_{13}x_3 + \cdots + c_{1,n-1}x_{n-1} + c_{1n}x_n + g_1 \\ x_2 = c_{21}x_1 + & +c_{23}x_3 + \cdots + c_{2,n-1}x_{n-1} + c_{2n}x_n + g_2 \\ x_3 = c_{31}x_1 + c_{32}x_2 + & + \cdots + c_{3,n-1}x_{n-1} + c_{3n}x_n + g_3 \\ & \cdots \\ x_n = c_{n1}x_1 + c_{n2}x_2 + c_{n3}x_3 + \cdots + c_{n,n-1}x_{n-1} + & + g_n \end{cases}$$

写成迭代的矩阵格式为 $x^{(k+1)} = Cx^{(k)} + g$, 其中

$$C = \begin{pmatrix} 0 & c_{12} & c_{13} & \cdots & c_{1n} \\ c_{21} & 0 & c_{23} & \cdots & c_{2n} \\ c_{31} & c_{32} & 0 & \ddots & c_{3n} \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ c_{n1} & c_{n2} & c_{n3} & \cdots & 0 \end{pmatrix}, g = \begin{pmatrix} g_1 \\ g_2 \\ g_3 \\ \vdots \\ g_n \end{pmatrix}$$

算法 8.6 Jacobi迭代算法 ($JACOBI(A, b, n, x^{(0)}, \varepsilon, N)$).

Algorithm 6: $JACOBI(A, b, n, x^{(0)}, \varepsilon, N)$

Input: A, b, n , 初始迭代向量 $x^{(0)}$, 容许误差 ε , 最大迭代步数 N

Output: x

- 1 $k = 1$
 - 2 **for** $i = 1, 2, \dots, n$ **do**
 - 3 $x_{ii} = (b_i - \sum_{j=1, j \neq i}^n a_{ij}x_j^{(0)})/a_{ii}$
 - 4 **end**
 - 5 若 $\|x - x^{(0)}\| < \varepsilon$, 则输出 x , 停机, 否则转下一步
 - 6 若 $k < N$, 则 $k = k + 1, x^{(0)} = x$, 转第2步, 否则输出计算失败, 停机
-

MATLAB程序如下:

```
function x = jacobi(A, b, x0, epsilon, N)
% Jacobi迭代法
% Input -A n×n矩阵 -b n维列向量 -x0 初始迭代向量 -epsilon 容许误差 -N 最大迭代次数
% Output -x n维列向量, Ax = b的解
n = length(b); x = zeros(n, 1);
for k = 1:N
    for j = 1:n
        x(j) = (b(j,:) - A(j,[1:j-1,j+1:n]) * x0([1:j-1,j+1:n])) / A(j,j);
    end
    err = abs(norm(x - x0));
    relerr = err / (norm(x) + eps);
    if (err < epsilon) || (relerr < epsilon), break; end
    x0 = x;
end
```

例 8.6 用 *Jacobi* 迭代法求方程组 $Ax = b$ 的解，并与精确解比较，其中

$$A = \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & 2 & -1 \\ & & -1 & 2 \end{pmatrix}, b = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

```
>> A = [2 -1 0 0; -1 2 -1 0; 0 -1 2 -1; 0 0 -1 2];
>> b = [1; 0; 1; 0];
>> x0 = [1; 1; 1; 1];
>> epsilon = 1e-8;
>> N = 50;
>> x = jacobi(A, b, x0, epsilon, N)
x =
    1.2000
    1.4000
    1.6000
    0.8000
>> x1 = A \ b           % 精确解
x1 =
    1.2000
    1.4000
    1.6000
    0.8000
>> err = norm(x - x1, inf) % 无穷范数衡量误差
err =
    1.3088e-05
```

注 8.2 在比较向量是否相等时可使用无穷范数，若 $\|u - v\|_{\infty} \leq \delta$ ，只要保证 δ 可以任意地接近于 0 即可说明 $u \approx v$ 。

8.3.2 Gauss-Seidel 迭代

Gauss-Seidel 迭代的过程为

$$\begin{cases} x_1^{(k+1)} = \frac{1}{a_{11}}(-c_{12}x_2^{(k)} - c_{13}x_3^{(k)} - \cdots - c_{1,n-1}x_{n-1}^{(k)} - c_{1n}x_n^{(k)} + g_1) \\ x_2^{(k+1)} = \frac{1}{a_{22}}(-c_{21}x_1^{(k+1)} - c_{23}x_3^{(k)} - \cdots - c_{2,n-1}x_{n-1}^{(k)} - c_{2n}x_n^{(k)} + g_2) \\ x_3^{(k+1)} = \frac{1}{a_{33}}(-c_{31}x_1^{(k+1)} - c_{32}x_2^{(k+1)} - \cdots - c_{3,n-1}x_{n-1}^{(k)} - c_{3n}x_n^{(k)} + g_3) \\ \vdots \\ x_n^{(k+1)} = \frac{1}{a_{nn}}(-c_{n1}x_1^{(k+1)} - c_{n2}x_2^{(k+1)} - c_{n3}x_3^{(k+1)} - \cdots - c_{n,n-1}x_{n-1}^{(k+1)} + g_n) \end{cases}$$

记

$$L = \begin{pmatrix} 0 & & & & \\ c_{21} & 0 & & & \\ c_{31} & c_{32} & 0 & & \\ \vdots & \vdots & \ddots & \ddots & \\ c_{n1} & c_{n2} & \cdots & c_{n,n-1} & 0 \end{pmatrix}, U = \begin{pmatrix} 0 & c_{12} & c_{13} & \cdots & c_{1n} \\ & 0 & c_{23} & \cdots & c_{2n} \\ & & \ddots & \ddots & \vdots \\ & & & 0 & c_{n-1,n} \\ & & & & 0 \end{pmatrix}, g = \begin{pmatrix} g_1 \\ g_2 \\ g_3 \\ \vdots \\ g_n \end{pmatrix}$$

则迭代过程可由矩阵表示为 $x^{(k+1)} = Lx^{(k+1)} + Ux^{(k)} + g$ 。若 $(I - L)^{-1}$ 存在, 还可以表示为 $x^{(k+1)} = (I - L)^{-1}Ux^{(k)} + (I - L)^{-1}g$, 迭代矩阵为 $(I - L)^{-1}U$ 。

算法 8.7 Gauss-Seidel迭代算法($GAUSS_SEIDEL(A, b, n, x^{(0)}, \varepsilon, N)$)。

Algorithm 7: $GAUSS_SEIDEL(A, b, n, x^{(0)}, \varepsilon, N)$

Input: A, b, n , 初始迭代向量 $x^{(0)}$, 容许误差 ε , 最大迭代步数 N

Output: x

1 $k = 1$

2 计算

$$\begin{aligned} x_1 &= (b_1 - \sum_{j=2}^n a_{1j}x_j^{(0)})/a_{11} \\ x_i &= (b_i - \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^n a_{ij}x_j^{(0)})/a_{ii}, \quad i = 2, 3, \dots, n-1 \\ x_n &= (b_n - \sum_{j=1}^{n-1} a_{nj}x_j^{(0)})/a_{nn} \end{aligned}$$

3 若 $\|x - x^{(0)}\| < \varepsilon$, 则输出 x , 停机, 否则转下一步

4 若 $k < N$, 则 $k = k + 1, x^{(0)} = x$, 转第2步, 否则输出计算失败, 停机

MATLAB程序如下:

```
function x = gauss_seidel(A, b, x0, epsilon, N)
% Gauss-Seidel迭代法
% Input -A n×n矩阵 -b n维列向量 -x0 初始迭代向量
%        -epsilon 容许误差 -N 最大迭代次数
% Output -x n维列向量, Ax = b的解
n = length(b);
x = zeros(n, 1);
for k = 1:N
    for j = 1:n
        if j ~= 1 && j ~= n
            x(j) = (b(j) - A(j,1:j-1) * x(1:j-1) - ...
                    A(j,j+1:n) * x0(j+1:n)) / A(j,j);
        elseif j == 1
            x(1) = (b(1) - A(1,2:n) * x0(2:n)) / A(1,1);
        else
            x(n) = (b(n) - A(n,1:n-1) * x(1:n-1)) / A(n,n);
        end
    end
    err = norm(x-x0);
    relerr = err / (norm(x) + eps);
    if (err < epsilon) || (relerr < epsilon), break; end
    x0 = x;
end
```

例 8.7 用Gauss-Seidel迭代法求方程组 $Ax = b$ 的解, 其中

$$A = \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & 2 & -1 \\ & & -1 & 2 \end{pmatrix}, b = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

```
>> A = [2 -1 0 0; -1 2 -1 0; 0 -1 2 -1; 0 0 -1 2];
>> b = [1; 0; 1; 0];
>> x0 = [1; 1; 1; 1];
>> epsilon = 1e-8;
>> N = 50;
>> x = gauss_seidel(A, b, x0, epsilon, N)
x =
    1.2000
    1.4000
    1.6000
    0.8000
>> c = A * x      % 带入验证
c =
    1.0000
   -0.0000
    1.0000
     0
```

8.3.3 松弛法

松弛法可以看成Gauss-Seidel迭代的加速，即将松弛因子 ω 加入到Gauss-Seidel迭代格式中，使之成为

$$x^{(k+1)} = (I - \omega L)^{(-1)}((1 - \omega)L + \omega U)x^{(k)} + \omega(I - \omega L)^{(-1)}g.$$

此时，迭代矩阵为 $(I - \omega L)^{(-1)}((1 - \omega)L + \omega U)$ 。

注 8.3 当松弛因子 $\omega = 1$ 时，松弛法就是Gauss-Seidel迭代法。

算法 8.8 松弛法($RELAXATION(A, b, n, \omega, x^{(0)}, \varepsilon, N)$)。

MATLAB程序如下：

```
function x = relaxation(A, b, omega, x0, epsilon, N)
% 松弛法
% Input   -A n×n矩阵 -b n维向量 -omega 松弛因子 -x0 初始迭代向量
%         -epsilon 容许误差 -N 最大迭代次数
% Output  -x n维向量, Ax = b的解
n = length(b);
x = zeros(n, 1);
for k = 1:N
    for j = 1:n
        if j ~= 1 && j ~= n
            x(j) = (1 - omega) * x(j) + omega * ...
```

Algorithm 8: *RELAXATION*($A, b, n, \omega, x^{(0)}, \varepsilon, N$)**Input:** A, b, n , 松弛因子 ω , 初始迭代向量 $x^{(0)}$, 容许误差 ε , 最大迭代步数 N **Output:** x 1 $k = 1$

2 计算

$$x_1 = (b_1 - \sum_{j=2}^n a_{1j}x_j^{(0)})/a_{11}$$

$$x_i = (b_i - \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^n a_{ij}x_j^{(0)})/a_{ii}, \quad i = 2, 3, \dots, n-1$$

$$x_n = (b_n - \sum_{j=1}^{n-1} a_{nj}x_j^{(0)})/a_{nn}$$

3 若 $\|x - x^{(0)}\| < \varepsilon$, 则输出 x , 停机, 否则转下一步4 若 $k < N$, 则 $k = k + 1, x^{(0)} = x$, 转第2步, 否则输出计算失败, 停机

```

        (b(j) - A(j,1:j-1) * x(1:j-1) - ...
        A(j,j+1:n) * x0(j+1:n)) / A(j,j);
elseif j == 1
    x(1) = (1 - omega) * x(1) + omega * ...
    (b(1) - A(1,2:n) * x0(2:n)) / A(1);
else
    x(n) = (1 - omega) * x(n) + omega * ...
    (b(n) - A(n,1:n-1) * x(1:n-1)) / A(n,n);
end
end
err = norm(x - x0);
relerr = err/(norm(x) + eps);
if (err < epsilon) || (relerr < epsilon), break; end
x0 = x;
end

```

例 8.8 用松弛法求方程组 $Ax = b$ 的解, 其中

$$A = \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & 2 & -1 \\ & & -1 & 2 \end{pmatrix}, b = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

```

>> A = [2 -1 0 0; -1 2 -1 0; 0 -1 2 -1; 0 0 -1 2];
>> b = [1; 0; 1; 0];
>> omega = 1.5;
>> x0 = [1; 1; 1; 1];
>> epsilon = 1e-8;
>> N = 50;

```

```
>> x = relaxation(A, b, omega, x0, epsilon, N)
x =
    1.2000
    1.4000
    1.6000
    0.8000
```

迭代序列的收敛是有条件的，下面给出保证收敛得条件。

设 A 为 $n \times n$ 的矩阵， $\lambda_i (i = 1, 2, \dots, n)$ 是 A 的特征值，则称 $\rho(A) = \max_{1 \leq i \leq n} |\lambda_i|$ 为 A 的谱半径，即 A 的按模最大特征值。

根据迭代收敛定理，对任何初始向量 $x^{(0)}$ 和常向量 f ，由迭代格式

$$x^{(k+1)} = Mx^{(k)} + f, \quad k = 0, 1, 2, \dots$$

产生的向量序列 $\{x^{(k)}\}$ 收敛且与初值无关的重要条件是

$$\rho(M) < 1,$$

也可利用谱半径与范数的关系， $\rho(M) < \|M\|$ ，粗略地判断迭代是否收敛。

8.4 幂法与反幂法

8.4.1 幂法

幂法是通过矩阵特征向量来求出特征值的一种迭代法，它主要是用来求按模最大特征值和相应的特征向量的。基本思想是要求矩阵 A 的特征值和相应的特征向量，先选取一个初始向量 $x^{(0)}$ ，构造Krylov序列：

$$x^{(0)}, x^{(1)} = Ax^{(0)}, x^{(2)} = Ax^{(1)}, \dots, x^{(k+1)} = Ax^{(k)}, \dots$$

当 k 增大时，序列的收敛情况与按模最大的特征值的分布有关，分析序列的极限，便可求出按模最大特征值与相应的特征向量。

幂法算法简单，且易于在计算机上实现，其具体算法如下：

算法 8.9 幂法($POWER_METHOD(A, x, \varepsilon, N)$)。

MATLAB程序如下：

```
function [lamda, V] = power_method(A, x0, epsilon, N)
% 幂法求矩阵的特征值与特征向量
% Input  -A n×n矩阵  -x0 初始向量 -epsilon 容许误差 -N 最大迭代次数
% Output -lamda 特征值  -V 特征向量
```

Algorithm 9: *POWERMETHOD*(A, x, ε, N)**Input:** A , 初始迭代向量 x , 容许误差 ε , 最大迭代步数 N **Output:** λ, x 1 $k = 1, \mu = 0$ 2 求 $\lambda = x_r$, 使得 $|x_r| = \max_{1 \leq i \leq n} |x_i|$

3 计算

$$y = x/\lambda$$

$$x = Ay$$

4 若 $\|\lambda - \mu\| < \varepsilon$, 则输出 λ, x , 停机, 否则转下一步5 若 $k < N$, 则 $k = k + 1, \mu = \lambda$, 转第2步, 否则输出计算失败, 停机

```

lamda = 0;
k = 0;
state = 1;
while k <= N && state == 1
    Y = A * x0;
    m = max(abs(Y));
    Y = 1/m * Y;
    err = max(abs(lamda - m), norm(x0 - Y));
    if (err <= epsilon), state = 0; end
    k = k + 1;
    x0 = Y;
    lamda = m;
end
V = x0;

```

例 8.9 用幂法求矩阵 A 的按模最大特征值和特征向量

$$A = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 6 \end{pmatrix}.$$

```

>> A = [4 2 2; 2 5 1; 2 1 6];
>> x0 = ones(3, 1);
>> epsilon = 1e-4;
>> N = 1000;
>> [lamda, V] = power_method(A, x0, epsilon, N)
lamda =
    8.3877
V =
    0.8078
    0.7721
    1.0000
>> [v, D] = eig(A) % 验证
v =

```



```

    0.8280    0.1555    0.5387
   -0.4697    0.7172    0.5149
   -0.3062   -0.6793    0.6669
D =
    2.1259         0         0
         0    4.4865         0
         0         0    8.3876
>> [lam, ind] = max(diag(D));
>> U = v(:,ind) / v(1,ind) * V(1) % 将U放大, 使U(1) = V(1)
U =
    0.8078
    0.7721
    1.0000

```

8.4.2 反幂法

反幂法是用来求 A 的按模最小特征值和特征向量的。设 A 没有零特征值, 则 A^{-1} 存在。若 A 的特征值为 $\lambda_i (i = 1, 2, \dots, n)$, ν_i 是对应于 λ_i 的特征向量, 则 $1/\lambda_i$ 就是 A^{-1} 的特征值, 对应的特征向量仍为 ν_i 。于是, 求 A 的按模最小特征值就可以化为求 A^{-1} 的按模最大特征值, 因此可以把幂法应用到 A^{-1} 上, 这就是反幂法的基本思想。

算法 8.10 带原点移位的反幂法($INVERSE_POWER(A, \lambda^*, x, \varepsilon, N)$)。

Algorithm 10: $INVERSE_POWER(A, \lambda^*, x, \varepsilon, N)$

Input: A , 特征值近似值 λ^* , 初始迭代向量 x , 容许误差 ε , 最大迭代步数 N

Output: λ, x

- 1 $k = 1, \mu = 0$
 - 2 作三角分解 $A - \lambda^* I = LU$
 - 3 求 $\alpha = x_r$, 使得 $|x_r| = \max_{1 \leq i \leq n} |x_i|$
 - 4 计算 $y = x/\alpha$, $Lz = y$, $Ux = z$
 - 5 若 $\|1/\alpha - 1/\mu\| < \varepsilon$, 则输出 $\lambda = \lambda^* + 1/\alpha, x$, 停机, 否则转下一步
 - 6 若 $k < N$, 则 $k = k + 1, \mu = \alpha$, 转第3步, 否则输出计算失败, 停机
-

MATLAB程序如下:

```

function [lamda, V] = inverse_power(A, x0, lam, epsilon, N)
% 带原点移位的反幂法
% Input -A n×n矩阵 -x0 初始向量 -lam 特征值的近似值
%       -epsilon 容许误差 -N 最大迭代次数
% Output -lamda 特征值 -V 特征向量
A = A - lam * eye(size(A, 1));
[L, U] = lufact(A); % LU分解
lamda = 0;

```

```

k = 0;
state = 1;
while k <= N && state
    c = max(abs(x0));
    Y = 1 / c * x0;
    x0 = lubacksub(L, U, Y);    % 更新X
    err = abs(1 / lamda - 1 / c);
    lamda = c;
    if err < epsilon, state = 0; end
    k = k + 1;
end
lamda = lam + 1 / c;
V = x0;

```

例 8.10 求给定矩阵 A 的按模最小特征值和特征向量

$$A = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 6 \end{pmatrix}.$$

```

>> A = [4 2 2; 2 5 1; 2 1 6];
>> x0 = ones(3, 1);
>> epsilon = 1e-8;
>> N = 1000;
>> [lamda, V] = inverse_power(A, x0, 0, epsilon, N)
lamda =
    2.1259
V =
    0.4704
   -0.2668
   -0.1740

```

8.5 实验报告

1. 利用高斯消去法求矩阵的逆并验证。

$$A = \begin{pmatrix} 2 & 2 & 3 \\ 4 & 7 & 7 \\ -2 & 4 & 5 \end{pmatrix}$$

解 构造线性方程组 $Ax = b$ ，其中 $b = \text{eye}(3)$ 。

```
>> A = [2 2 3; 4 7 7; -2 4 5];
>> b = eye(3);
>> x = gauss(A, b)
x =
    0.1944    0.0556   -0.1944
   -0.9444    0.4444   -0.0556
    0.8333   -0.3333    0.1667
>> y = inv(A) % 验证
y =
    0.1944    0.0556   -0.1944
   -0.9444    0.4444   -0.0556
    0.8333   -0.3333    0.1667
```

2. 编写求解三对角方程组 $Ax = f$, 其中

$$A = \begin{pmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & a_{n-1} & b_{n-1} & c_{n-1} \\ & & & a_n & b_n \end{pmatrix}, x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix}, f = \begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ \vdots \\ f_n \end{pmatrix}$$

的追赶法(Tomas算法, 如算法 8.11)程序, 并求解

$$A = \begin{pmatrix} 2 & 1 & & & \\ 1 & 2 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & 2 & 1 \\ & & & 1 & 2 \end{pmatrix}, f = \begin{pmatrix} 6 \\ 11 \\ 16 \\ 23 \\ 25 \\ 19 \end{pmatrix}.$$

相关定理: 若三对角方阵 A 满足

- (1) $|b_1| > |c_1| > 0$,
- (2) $|b_i| \geq |a_i| + |c_i|, a_i c_i \neq 0, i = 2, \dots, n-1$,
- (3) $|b_n| > |a_n| > 0$,

则 A 有如下形式的唯一的 LU 分解:

$$A = \begin{pmatrix} 1 & & & & \\ l_2 & 1 & & & \\ & l_3 & \ddots & & \\ & & \ddots & 1 & \\ & & & l_n & 1 \end{pmatrix} \begin{pmatrix} u_1 & c_1 & & & \\ & u_2 & c_2 & & \\ & & u_3 & \ddots & \\ & & & \ddots & c_{n-1} \\ & & & & u_n \end{pmatrix} = LU,$$

这样就可以得出 Tomas 算法。

算法 8.11 *Tomas*算法 ($TOMAS(a, b, c, f, n)$)。**Algorithm 11:** $TOMAS(a, b, c, f, n)$ **Input:** a, b, c, f, n **Output:** x

```

1  $u_1 = b_1, y_1 = f_1$ 
2 for  $i = 2, 3, \dots, n$  do
3    $l_i = a_i / u_{i-1}$ 
4    $u_i = b_i - c_{i-1} l_i$ 
5    $y_i = f_i - l_i y_{i-1}$ 
6 end
7  $x_n = y_n / u_n$ 
8 for  $i = n-1, n-2, \dots, 1$  do
9    $x_i = (y_i - c_i x_{i+1}) / u_i$ 
10 end

```

解 编写Tomas算法的程序如下:

```

function x = thomas(a, b, c, f)
% Thoms算法, 追赶法求三对角方程组, 形式为
%   +-      +-      +-      +-      +-
%   / b(1) c(1)      /   / x(1) /   / f(1) /
%   / a(2) b(2) c(2)      * / x(2) / = / f(2) /
%   /           ....      /   / ... /   / ... /
%   /           a(n) b(n) /   / x(n) /   / f(n) /
%   +-      +-      +-      +-      +-
% Input  -[a, b, c]  对角线向量      -f  右端项数值
% Output -x        求解结果x
n = length(f); x = zeros(n, 1);
U = ones(n, 1);
U(1) = b(1);
for i = 2:n
    L = a(i-1) / U(i-1);
    U(i) = b(i-1) - L * c(i-1);
    f(i) = f(i) - L * f(i-1);
end
x(n) = f(n) / U(n);
for i = n-1:-1:1
    x(i) = (f(i) - c(i) * x(i+1)) / U(i);
end

```

当 $a_i = a$, $b_i = b$, $c_i = c$ 时, Thoms算法更为简单, 编程如下:

```

function f = thomas_algorithm(a, b, c, f)
% Thoms算法, 追赶法求三对角方程组
% Input  -[a, b, c]  对角线数值      -f  右端项数值
% Output -f        求解结果x

```

```

n = length(f);
U = ones(n, 1);
U(1) = b;
for i = 2:n
    L = a / U(i - 1);
    U(i) = b - L * c;
    f(i) = f(i) - L * y(i - 1);
end
f(n) = f(n) / U(n);
for i = n-1:-1:1
    f(i) = (f(i) - c * f(i + 1)) / U(i);
end

```

这种算法经常在微分方程数值求解中使用。下面求解方程组

```

>> a = 1; b = 2; c = 1;
>> f = [6; 11; 16; 23; 25; 19];
>> x = thomas_algorithm(a, b, c, f)
x =
    1.0000
    4.0000
    2.0000
    8.0000
    5.0000
    7.0000

```

3. 设系数矩阵

$$A = \begin{pmatrix} a & 1 & 3 \\ 1 & a & 2 \\ -3 & 2 & a \end{pmatrix}$$

问： a 取何值时，Jacobi迭代收敛？ a 取何值时，Gauss-Seidel迭代收敛？ $a = 3$ 时，试比较两种迭代的谱半径。

解 $a \rightarrow \infty$ 时，Jacobi迭代矩阵

$$M = \begin{pmatrix} 0 & -1/a & -3/a \\ -1/a & 0 & -2/a \\ 3/a & -2/a & 0 \end{pmatrix}$$

可以证明 $a \rightarrow \infty$ 时，Jacobi迭代(简称J迭代)是收敛的，同理，Gauss-Seidel迭代(简称G-S迭代)也收敛，故只需考虑 a 为有限的情况，现考虑 $-3 \leq a \leq 5$ 的情况。

```

function convflag = jac_gau_seid
% 判断迭代法是否收敛
% Output -index 第一列为a的取值，第二列为J迭代的收敛情况，
%               第三列为G-S迭代的收敛情况，取值为1表示收敛
a = -3:0.01:5;

```

```

b = [1; 1; 1];
x0 = [1; 1; 1];
epsilon = 1e-4;
N = 1000;
n = length(a);
convflag = zeros(n, 3);
convflag(:,1) = a';
for i = 1:n
    A = [a(i) 1 3; 1 a(i) 2; -3 2 a(i)];
    x = jacobi(A, b, x0, epsilon, N);
    x = floor(x * 100) / 100;
    y = gauss_seidel(A, b, x0, epsilon, N);
    y = floor(y*100) / 100;
    c = A \ b;
    c = floor(c * 100) / 100;
    convflag(i,2) = isequal(x, c);
    convflag(i,3) = isequal(y, c);
end

```

输出结果见表 8.2 和表 8.3。

表 8.2: 两种迭代的收敛情况

取点	J迭代是否收敛	G-S迭代是否收敛	取点	J迭代是否收敛	G-S迭代是否收敛
-3.00	1	1	2.00	0	0
...	...	...	2.01	1	0
-2.02	1	1	...	...	...
-2.01	0	1	2.87	1	0
...	...	...	2.88	1	1
-1.83	0	1	...	...	...
-1.82	0	0	4.99	1	1
...	...	...	5.00	1	1

表 8.3: 两种迭代的收敛区域

a值	$(-\infty, 2.02]$	$[-2.01, -1.83]$	$[-1.82, 2]$	$[2.01, 2.87]$	$[2.88, +\infty)$
J迭代	收敛	不收敛	不收敛	收敛	收敛
G-S迭代	收敛	收敛	不收敛	不收敛	收敛

接着求 $a = 3$ 时的谱半径

```

>> A = [3 1 3; 1 3 2; -3 2 3];
>> JM = eye(3) - 1 / 3 * A;
>> JEig = max(abs(eig(JM))) % 计算J迭代矩阵的谱半径

```

```

JEig =
    0.6667
>> L = tril(JM);
>> U = triu(JM);
>> GSM = (eye(3) - L) \ U;
>> GSEig = max(abs(eig(GSM))) % 计算G-S迭代矩阵的谱半径
GSEig =
    0.9107

```

故 $a = 3$ 时，J迭代矩阵和G-S迭代矩阵的谱半径分别为 $2/3, 0.9107$ 。

4. 用谱半径的方法求解第3题。

```

function convflag = pr_jac_gau_seid
a = -3:0.01:5;
n = length(a);
convflag = zeros(n, 3);
convflag(:,1) = a';
for i = 1:n
    A = [a(i) 1 3; 1 a(i) 2; -3 2 a(i)];
    if a(i)
        JM = eye(3)-1/a(i)*A;
    else
        continue;
    end
    JEig = max(abs(eig(JM)));
    L = tril(JM);
    U = triu(JM);
    GSM = (eye(3) - L) \ U;
    GSEig = max(abs(eig(GSM)));
    convflag(i,2) = JEig < 1;
    convflag(i,3) = GSEig < 1;
end

```

a 的范围的确定有多种方法，这里只给出两种方法，有兴趣的读者可以考虑另外可行的办法。

(1) 绘图法

根据程序中的convflag数据，将其第1列作为横坐标，将其第2列和第3列作为纵坐标分别绘出J迭代和G-S迭代的判断曲线(凹下去的部分所对应的 a 值为不收敛值)。代码和结果如下：

```

convflag = pr_jac_gau_seid;
x = convflag(:,1);
y = convflag(:,2);
z = convflag(:,3);
subplot(121)
plot(x, y)
title('J迭代')
axis([-3, 5, -1, 2])
grid on
subplot(122)
plot(x, z)

```

```

title('G-S迭代')
axis([-3, 5, -1, 2])
grid on           % 结果如图 8.1

```

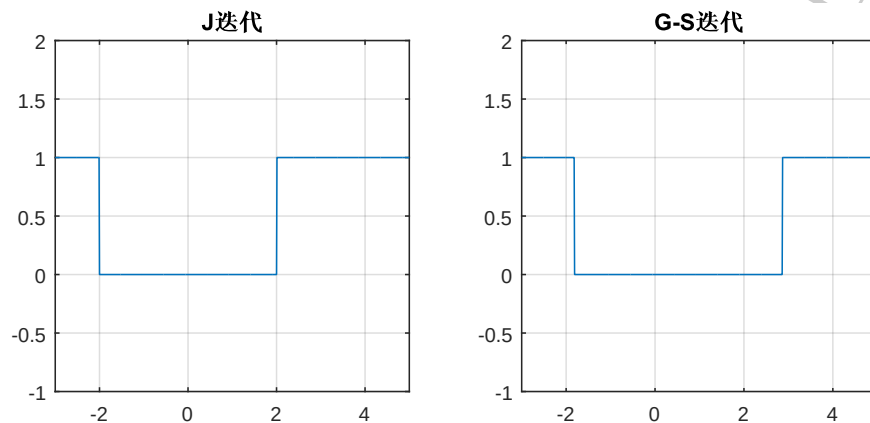


图 8.1: 两种迭代的收敛情况

通过观察图像，我们可以粗略地确定两种迭代收敛时 a 的范围。

(2) 分析法

根据 $a \rightarrow \infty$ 时对迭代矩阵的分析可知，两种迭代不收敛时 a 的区间只有“一个”。因此，只要找出不收敛时的区间即可。相应的代码和结果如下：

```

convflag = pr_jac_gau_seid;
x = convflag(:,1);
y = convflag(:,2);
z = convflag(:,3);
n = x(y == 0);
Jbs1 = n([1,end])';
disp('J迭代不收敛时a的范围是: ');
disp(['[', num2str(Jbs1(1)), ', ', num2str(Jbs1(2)), ']' ]);
m = x(z == 0);
GSbs1 = m([1,end])';
disp('G-S迭代不收敛时a的范围是: ');

```

输出结果

```

J迭代不收敛时a的范围是:
[-2,2]
G-S迭代不收敛时a的范围是:
[-1.81,2.86]

```


5. 考虑对称正定线性方程组 $Ax = b$ ，其中向量 b 是随机生成的，其元素服从区间 $[0, 1]$ 上的均匀随机数，矩阵 $A = LL^T$ ，这里 L 是随机生成的一个下三角阵，其元素是服从区间 $[1, 2]$ 上均匀分布的随机数。对 $n = 10, 20, \dots, 500$ 分别应用 Gauss 消去法、列主元 Gauss 消去法和 Cholesky 分解法求解该方程组。请画出他们所使用的 CPU 时间约系数矩阵阶数 n 的关系。横轴为阶数 n ，纵轴为时间，比较 3 种方法的速度。

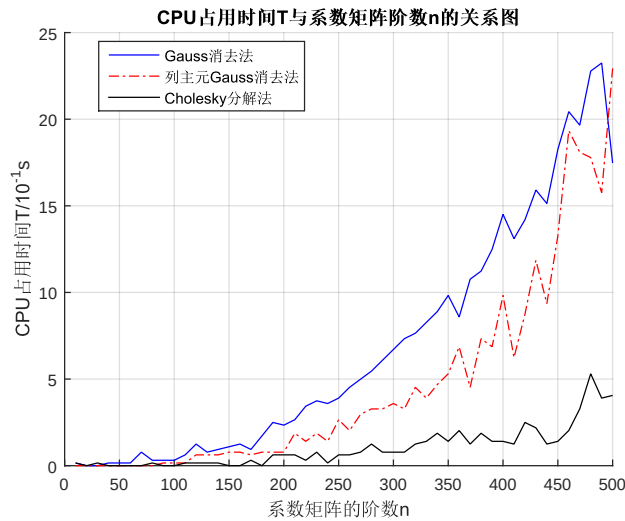


图 8.2: 消去法与分解法的 CPU 占用时间比较

解 编写函数

```
k = 50;
t = zeros(3, 50); % 每一行数据表示一种方法的运行时间
warning off all
for i = 1:k
    n = 10 * i;
    b = rand(n, 1); % 随机产生 [0,1] 上均匀分布的 n×1 矩阵 b
    c = rand(n, n) + 1; % 随机产生 [1,2] 上分布的 n×n 矩阵 c
    L = tril(c); % 抽取矩阵 c 的下三角部分
    A = L * L'; % 生成系数矩阵 A
    t0 = cputime; % 开始计时
    gauss(A, b); % Gauss 消去法求解
    t(1,i) = cputime - t0; % 计算 Gauss 消去法的 CPU 占用时间
    t0 = cputime; % 重新计时
    cmegauss(A, b); % 列主元 Gauss 消去法求解
    t(2,i) = cputime - t0; % 列主元 Gauss 消去法的 CPU 占用时间
    t0 = cputime; % 重新计时
    cholesky(A, b); % cholesky 分解法求解
    t(3,i) = cputime - t0; % 计算 cholesky 分解法的 CPU 占用时间
end
% 画占用 CPU 时间与系数矩阵 A 的阶数 n 的关系图
hold on
grid on
xn = 1:k;
```

```

plot(10*xn, 10*t(1,:), 'b');
plot(10*xn, 10*t(2,:), 'r-');
plot(10*xn, 10*t(3,:), 'k');
legend('Gauss消去法','列主元Gauss消去法',...
       'Cholesky分解法','Location','Best');
title('CPU占用时间T与系数矩阵阶数n的关系图');
xlabel('系数矩阵的阶数n');
ylabel('CPU占用时间T/10^-1s'); % 结果如图 8.2

```

注 8.4 程序运行占用CPU的时间可由`cputime`命令返回, 也可用`tic/toc`命令获得。

6. Gauss消去法、列主元Gauss消去法、全主元Gauss消去法以及LU分解、列主元LU分解、全主元LU分解, 这些方法都可以通过Gauss消元过程实现, 并且可以将上述6个算法写成一个通用程序。请查找相关资料, 试编写通用函数。

```

function varargout = gaussian(A, b, model)
% 基于Gauss消去法的几种方法
% Input -A 方程组系数矩阵 -b 右端项, 默认为[], 或者行数与A的行数一致
%        -model 使用方法, 默认为1; 若输入参数个数为1时, model = 3
%        0 顺序消去法          3 LU分解
%        1 列主元消去法        4 列主元LU分解
%        2 全主元消去法        5 全主元LU分解
% Using By
% x = gaussian(A, b)
% x = gaussian(A, b, model) % mode = 0:5
% [x, a] = gaussian(A, b, model) % mode = 0:2
% [~, L, U] = gaussian(A, [], model) % mode = 3:5
% [x, L, U] = gaussian(A, b, model) % mode = 3:5
% [x, L, U, a] = gaussian(A, b, model) % mode = 3:5
% [x, L, U, a] = gaussian(A)

% Copyright Baoshan Wang.
%% 前期准备
if nargin == 2, model = 1; end
if nargin == 1, b = []; model = 3; end
[n, m] = size(A); % 获得方程组个数n
%% 判断输入变量是否满足
if n ~= m || isempty(A), error('系数矩阵不是方阵'); end
if ~isempty(b) && size(b,1) ~= n, error('右端项有误'); end
am = model == 2 || model == 5; % 判断是否为全主元方法
if am, ind = 1:n; end % 若为全主元方法, 则生成解的顺序向量
%% Gauss消元
for k = 1:n-1
    switch model
        case {1,4}
            %% 选列主元
            [~, m1] = max(abs(A(k:n,k))); % 选列主元
            A([k,k+m1-1],:) = A([k+m1-1,k],:); % 互换两行
            b([k,k+m1-1],:) = b([k+m1-1,k],:); % 互换两行
        case {2,5}
            %% 选主元
            [mm, m] = max(abs(A(k:n,k:n))); % 选主元
            [~, m2] = max(mm); % 选主元

```

```

        m1 = m(m2); % 行列
        A([k,k+m1-1],:) = A([k+m1-1,k],:); % 互换两行
        b([k,k+m1-1],:) = b([k+m1-1,k],:); % 互换两行
        A(:,[k,k+m2-1]) = A(:,[k+m2-1,k]); % 互换两列
        ind([k,k+m2-1]) = ind([k+m2-1,k]); % 互换次序
    end
    %% 第k步消元过程
    if A(k,k)
        A(k+1:n,k) = A(k+1:n,k) / A(k,k); % 消元系数
        A(k+1:n,k+1:n) = A(k+1:n,k+1:n) - ...
            A(k+1:n,k) * A(k,k+1:n); % 消元
    else
        error(['Matrix is singular to working precision', ...
            'because of a(', num2str(k), ', ', num2str(k), ...
            ') = 0']);
    end
end
end
%% 回代过程
L = tril(A); % 取下三角
L(1:n+1:end) = 1; % 或者用L(eye(n) == 1) = 1;
U = triu(A); % 取上三角U
if ~isempty(b)
    b = lubacksub(L, U, b); % 回代
    if am, b(ind,:) = b; end % 调整顺序
end
%% 输出结果
varargout{1} = b; % 输出方程的解x
if ismember(model, 3:5) % LU分解, 输出L、U和A
    varargout{2} = L; % 输出单位下三角矩阵L
    varargout{3} = U; % 取上三角U
    varargout{4} = A; % 消元后的系数矩阵A
    if nargin > 5, error('Too many output arguments.');
end
else
    varargout{2} = A; % 消元后的系数矩阵A
    if nargin > 2, error('Too many output arguments.');
end
end

```

我的 MATLAB 报告

王保山

第九章 数值逼近

在解决实际问题 and 数学计算中，插值与拟合是应用十分广泛的方法。在数学分析中，我们用 $y = f(x)$ 来描述一条平面曲线，但在实际问题中，函数 $f(x)$ 往往是通过实验观测得到的一组数据来给出的，如何通过这组数据来寻找出函数 $f(x)$ 的一个近似表达式呢？可以利用插值，也可以利用拟合。

表 9.1: 插值与拟合的相关指令

指令	说明
<code>p = polyfit(x, y, k)</code>	用 k 次多项式拟合向量数据 (x, y) ，返回多项式的降幂系数。当 $k > n-2$ 时，该命令实现多项式插值
<code>yi = interp1(x, y, xi)</code>	根据数据 (x, y) 给出在 xi 的分段线性插值结果 yi
<code>yi = interp1(x, y, xi, 'spline')</code>	三次样条插值
<code>yi = interp1(x, y, xi, 'pchip')</code>	分段三次插值
<code>pp = spline(x, y)</code>	返回 <code>pp</code> 形式的样条插值的分段多项式
<code>yi = spline(x, y, xi)</code>	等价于 <code>yi = interp1(x, y, xi, 'spline')</code>
<code>[breaks, coefs] = unmkpp(pp)</code>	将 <code>pp</code> 形式展开， <code>breaks</code> 为结点， <code>coefs</code> 为各段多项式系数
<code>yi = ppval(pp, xi)</code>	求 <code>pp</code> 形式在 xi 的函数值 yi
<code>C = leastsq(fun, c)</code>	使用迭代法搜索最有参数 C ，自变量为 c 的函数 <code>fun</code> 是误差向量 $y - f(c, x)$ (x, y 为数据)
<code>C = curvefit(fun, c, x, y)</code>	使用迭代法搜索最有参数 C ， <code>fun</code> 指 $f(c, x)$
<code>[b, bint, r, rint, stats] = regress(y, x, alpha)</code>	多元线性回归，可用 <code>rcoplot(r, rint)</code> 绘制残差图

9.1 插值

9.1.1 Lagrange插值

n 次Lagrange插值多项式 $\varphi_n(x)$ 可以表示为 $\varphi_n(x) = a_0 + a_1x + \cdots + a_nx^n$, 结合插值条件 $\varphi_n(x_i) = f(x_i) = y_i$, 可得系数 a_i 应满足方程组

$$\begin{cases} a_0 + a_1x_0 + a_2x_0^2 + \cdots + a_nx_0^n = y_0 \\ a_0 + a_1x_1 + a_2x_1^2 + \cdots + a_nx_1^n = y_1 \\ \cdots \\ a_0 + a_1x_n + a_2x_n^2 + \cdots + a_nx_n^n = y_n \end{cases}$$

该方程组的系数行列式为范德蒙行列式

$$V(x_0, x_1, x_2, \cdots, x_n) = \begin{vmatrix} 1 & x_0 & \cdots & x_0^n \\ 1 & x_1 & \cdots & x_1^n \\ \vdots & \vdots & & \vdots \\ 1 & x_n & \cdots & x_n^n \end{vmatrix} = \prod_{0 \leq j < i \leq n} (x_i - x_j)$$

引入基函数

$$l_i(x) = \frac{(x - x_0) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)}, \quad i = 0, 1, \cdots, n.$$

不难发现, 该基函数满足性质

$$l_i(x_j) = \begin{cases} 1 & x_i = x_j \\ 0 & x_i \neq x_j \end{cases}.$$

于是 $\varphi_n(x)$ 可由他们的线性组合表示, 即

$$\varphi_n(x) = \sum_{j=0}^n y_j l_j(x).$$

注 9.1 基函数的引入详见徐萃薇、孙绳武编著的《计算方法引论》第三版第二章, 23~25。

例 9.1 编写一般情况($n+1$ 个插值节点, 构造 n 次Lagrange插值多项式)下的Lagrange插值方法的通用程序。

解 数值方法

```

function yi = lagrange(x, y, xi)
% lagrange插值 yi = lagrange(x, y, xi)
% Input -(x, y) 数值点 -xi 被插值点
% Output -yi xi处的插值
n = length(x);
m = length(xi);
yi = zeros(size(xi));
for i = 1:m
    z = xi(i);
    s = 0.0;
    for k = 1:n
        p = 1.0;
        for j = 1:n
            if j ~= k
                p = p * (z - x(j)) / (x(k) - x(j));
            end
        end
        s = p * y(k) + s;
    end
    yi(i) = s;
end

```

符号方法

```

function yi = lagrange_sym(x, y, xi)
% lagrange插值 yi = lagrange_sym(x, y, xi)
% Input -(x, y) 数值点 -xi 被插值点
% Output -yi xi处的插值
syms t;
if length(x) == length(y)
    n = length(x);
else
    error('维数不相等');
end
f = 0.0;
for i = 1:n
    l = y(i);
    for j = 1:n
        if j ~= i
            l = l * (t - x0(j)) / (x(i) - x(j));
        end
    end
    f = f + l;
end
yi = subs(f, 't', xi);

```

9.1.2 其他插值

其他插值，如分段线性插值、样条插值，在MATLAB里都有现成的函数，如表 9.2。

表 9.2: 插值的相关指令

插值类别	指令	说明
多项式插值	<code>p = polyfit(x, y, k)</code>	(x,y)是数据点, (xi,yi)是插值点, 参数字符串决定插值类型
分段线性插值	<code>yi = interp1(x, y, xi)</code>	
三次样条插值	<code>yi = interp1(x, y, xi, 'spline')</code>	
分段三次插值	<code>yi = interp1(x, y, xi, 'pchip')</code>	

例 9.2 多项式插值、分段线性插值、三次样条插值和分段三次插值

```
x = 0:10;
y = sin(x);
plot(x, y, 'ko')
hold on
xi = 0:0.5:10;
% 多项式插值
p = polyfit(x, y, 11);
yi1 = polyval(p, xi);
plot(xi, yi1, '-r')
% 分段线性插值
yi2 = interp1(x, y, xi);
plot(xi, yi2, '-s')
% 三次样条插值
yi3 = interp1(x, y, xi, 'spline');
plot(xi, yi3, '-p')
% 分段三次插值
yi4 = interp1(x, y, xi, 'pchip');
plot(xi, yi4, '-h')
legend('数据点', '多项式插值', '分段线性插值', '三次样条插值', '分段三次插值', 'location', 'best')
hold off
```

结果如图 9.1

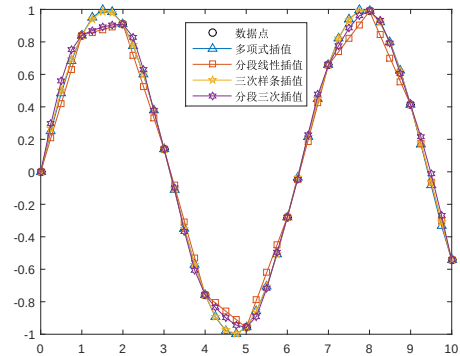


图 9.1: 几种插值

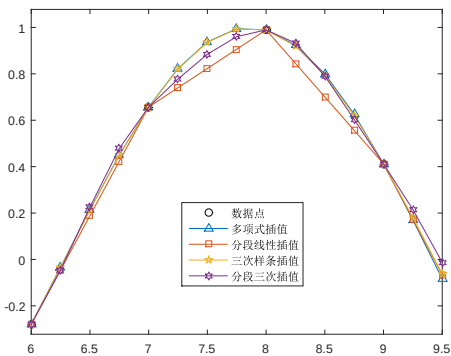


图 9.2: 局部放大图

9.2 拟合

拟合的MATLAB指令如表 9.3。

表 9.3: 拟合的相关指令

拟合	指令	说明
多项式拟合	$p = \text{polyfit}(x, y, k)$	$k > \text{length}(x) - 1$
样条拟合	$pp = \text{csaps}(x, y, p)$	pp 形式的光滑拟合，其中 $0 < p < 1$, p 越大，拟合结果与数据越接近
最小二乘拟合	$c = \text{lsqnonlin}(\text{fun}, c0)$	用迭代法搜索最优参数 c , $\text{fun} = y - f(c, x)$, $c0$ 是 c 的近似值
	$c = \text{lsqcurvefit}(\text{fun}, c0, x, y)$	(x, y) 可由外部输入, $\text{fun} = f(c, x)$

例 9.3 多项式拟合与样条拟合

```
x = 0:0.5:2*pi;
y = sin(x);
plot(x, y, 'ko')
hold on
p = polyfit(x, y, 3);
xi = 0:0.25:2*pi;
yi1 = polyval(p, xi);
plot(xi, yi1, '--')
pp = csaps(x, y, 0.9);
yi2 = ppval(pp, xi); % 求pp形式在xi处的值
plot(xi, yi2, '-')
legend('数据点', '多项式拟合', '样条拟合', 'location', 'best')
axis([0, 2 * pi, -1.1, 1.1]) % 结果如图 9.3
```

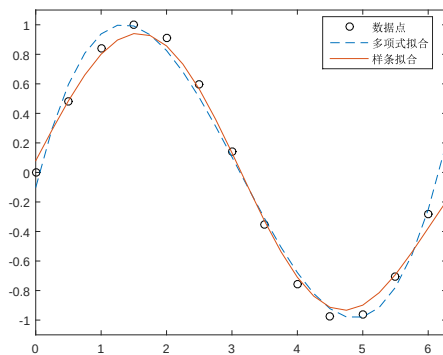


图 9.3: 多项式拟合与样条拟合

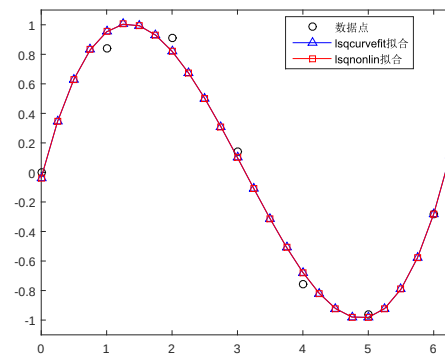


图 9.4: 最小二乘拟合

例 9.4 最小二乘拟合

```
x = 0:2*pi;
y = sin(x);
plot(x, y, 'ko')
hold on
fun = inline('c(1)*x.^3+c(2)*x.^2+c(3)*x+c(4)', 'c', 'x');
c = lsqcurvefit(fun, [1, 1, 1, 1], x, y);
xi = 0:0.25:2*pi;
yi1 = polyval(c, xi);
plot(xi, yi1, 'b-')
c = lsqnonlin(@(c)lsfun(c, x, y), [0, 0, 0, 0]);
yi2 = polyval(c, xi);
plot(xi, yi2, 'r-s')
legend('数据点', 'lsqcurvefit拟合', 'lsqnonlin拟合', 'location', 'best')
axis([0, 2 * pi, -1.1, 1.1]) % 结果如图 9.4
```

辅助函数lsfun如下

```
function e = fun(c)
x = 0:0.1:2*pi;
y = cos(x);
e = y - (c(1) * x.^3 + c(2) * x.^2 + c(3) * x + c(4));
```

9.3 回归

理论上，用最小二乘法总能找到拟合的最优曲线，但该曲线是否有效，还需要用回归分析的方法进行检验。

表 9.4: 回归的相关指令

指令	说明
[b, bint, r, rint, stats] = regress(y, x, alpha)	多元线性回归。y 是因变量观测数据($n \times 1$ 向量); x 是自变量观测数据($n \times p$ 矩阵); alpha为置信度，默认为0.05; b是参数估计值; bint是b的置信区间; r为残差向量 $y - xb$; rint是r 的置信区间; stats为 1×4 的检验统计量，stats(1)是回归方程的置信度，stats(2)是F统计量值，stats(3)是相应的p值，p 值越小说明回归方程系数不为零stats(4)是方差的误差估计。
[beta, R, J] = nlinfit(x, y, 'model', beta0)	x是因素数据矩阵，每一列是一个变量; y是响应数据矩阵; model 参数是参数为beta的函数形式 $y = f(\text{beta}, x)$; beta0是参数的迭代初值; beta返回参数估计; R返回残差; J返回用于估计预报误差的Jacobi矩阵

例 9.5 最小二乘拟合的检验

```

>> c = lsqnonlin(@fun, [0, 0, 0, 0])
c =
    0.0039    0.1939   -1.3494    1.4469
>> x = 0:1:2*pi;
>> x = x';
>> y = cos(x);
>> x0 = ones(size(x));
>> x = [x0, x, x.^2, x.^3];
>> [b, bint, r, rint, stats] = regress(y, x);
>> b, stats
b =
    1.1311
   -0.9908
    0.0884
    0.0126
stats =
    0.9184   11.2574    0.0386    0.1033
>> rcoplot(r, rint)
>> set(gca, 'color', [1, 1, 1]) % 绘制残差图 9.5

```



图 9.5: 残差图

9.4 实验报告

1. 在下面的函数中选择两个, 在 n 个结点上(n 不要太大, 如5~11)分别用Lagrange、分段线性、三次样条插值三种插值方法, 计算 m 个插值点的函数值(m 要适中, 如50~100)。通过数值和图形输出, 将插值结果与精确值进行比较。

$$(1) y = \sin x, \quad 0 \leq x \leq 2\pi,$$

$$(2) y = \sqrt{1-x^2}, \quad -1 \leq x \leq 1,$$

$$(3) y = \cos^{10} x, \quad -2 \leq x \leq 2,$$

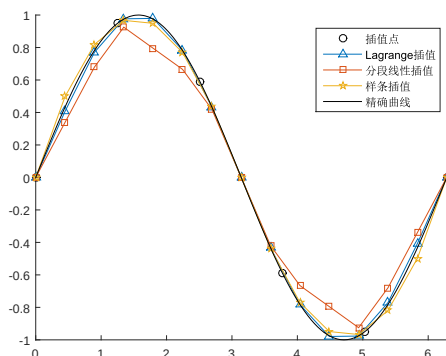
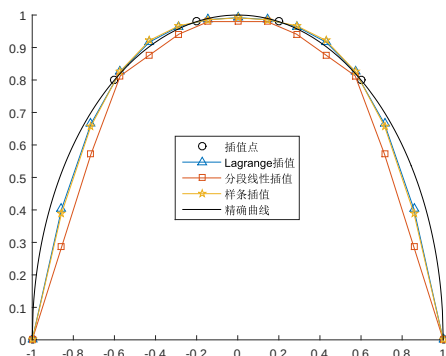
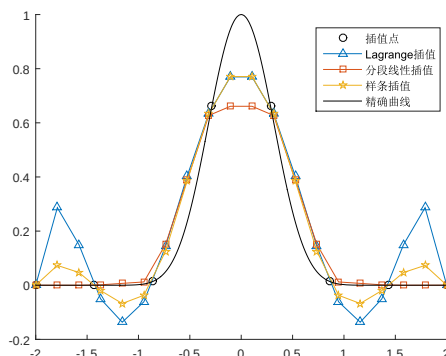
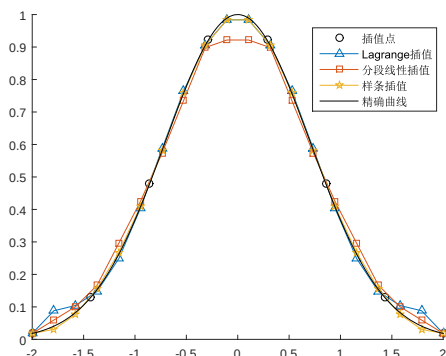
$$(4) y = e^{-x^2}, \quad -2 \leq x \leq 2,$$

解 编写插值结果的绘图函数如下:

```

function interplot(itpfun, a, b, n, m)
% 绘制几种插值的图像
% Input -itpfun 精确函数 -[a, b] 区间 -[n, m] 数据点数和插值节点数
hold on
x = linspace(a, b, n);
y = feval(itpfun, x);
plot(x, y, 'ko')
xa = linspace(a, b, m);
ya = lagrange(x, y, xa);
plot(xa, ya, '-^')
ya = interp1(x, y, xa);
plot(xa, ya, '-s')
ya = interp1(x, y, xa, 'spline');
plot(xa, ya, '-p')
fplot(itpfun, [a, b], 'k-')
legend('插值点', 'Lagrange插值', '分段线性插值', '样条插值', '精确曲线', 'location', 'best')
hold off

```

图 9.6: $\sin x$ 的几种插值对比图 9.7: $\sqrt{1-x^2}$ 的几种插值对比图 9.8: $\cos^{10} x$ 的几种插值对比图 9.9: e^{-x^2} 的几种插值对比

调用上述函数进行绘图

```
>> interplot(@(x)sin(x), 0, 2*pi, 6, 15) % 结果如图 9.6
>> interplot(@(x)sqrt(1-x.^2), -1, 1, 6, 15) % 结果如图 9.7
>> interplot(@(x)cos(x).^10, -2, 2, 8, 20) % 结果如图 9.8
>> interplot(@(x)exp(-x.^2), -2, 2, 8, 20) % 结果如图 9.9
```

2. 以 $f(x) = \frac{1}{1+25x^2}$ 为例，图示Runge现象。

```
a = -1:0.2:1;
b = 1 ./ (1 + 25 * a.^ 2);
x = -1:0.01:1;
y = lagrange(a, b, x); % lagrange插值
c = -1:0.01:1;
d = 1 ./ (1 + 25 * c.^ 2);
hold on
plot(x, y, '- ', c, d, 'k-')
hold off
legend('lagrange插值图像', '原函数图像', 'location', 'North')
title('Runge现象') % 结果如图 9.10
```

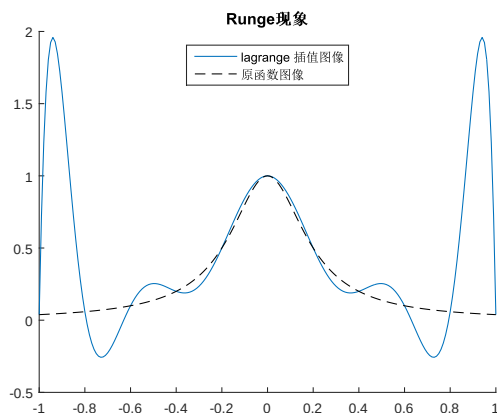


图 9.10: Runge现象

从图 9.10 上看，Lagrange插值在原点附近能够很好地逼近原函数，在两端逼近的效果却很差。当 $n \rightarrow \infty$ 时， n 次插值多项式只能在区间 $|x| < 0.72 \cdots$ 内收敛，在这个区间外则是发散的，这一现象称为Runge现象。

3. 研究数值微分方法的稳定性问题。

(1) 考虑两点公式，其截断误差 $R_1 = f'(x_0) - \frac{f(x_1) - f(x_0)}{h} = -\frac{h}{2}f''(\xi)$, $\xi \in (x_0, x_1)$ ，很明显，当 $h \rightarrow 0$ 时， $R_1 \rightarrow 0$ 。但记公式的实际计算结果为 M_1^* ，则 $M_1^* = \frac{f(x_1) - f(x_0)}{h}$ 。研究当 $h \rightarrow 0$ 时，

$f'(x_0) - M_1^*$ 的变化情况。或三点公式, 其截断误差 $R_2 = f''(x_0) - \frac{f(x_2) - 2f(x_1) + f(x_0)}{h^2} = O(h^2)$, 很明显, 当 $h \rightarrow 0$ 时, $R_2 \rightarrow 0$ 。但记公式的实际计算结果为 M_2^* , 则 $M_2^* = \frac{f(x_2) - 2f(x_1) + f(x_0)}{h^2}$ 。研究当 $h \rightarrow 0$ 时, $f''(x_1) - M_2^*$ 的变化情况。

(2) 分析数值微分的稳定性的实例。设 $f(x) = e^x$, $x_0 = 0, x_1 = 0.1, x_2 = 0.2$, 用数值微分(三点公式)求 $f''(0)$ 并比较真正的误差和估计的误差限。

解 (1) 以 $f(x) = e^x$ 为例。

```
n = 10000;
err = zeros(1,n);
h = err;
for i = 1:n
    h(i) = 0.001 * i;
    x = [1 - h(i), 1, 1 + h(i)]; y = exp(x);
    y = round(y * 1e4) / 1e4; % 4位有效数字计算
    ff1 = (y(3) - y(1)) / (2 * h(i)); ff2 = exp(1);
    err(i) = abs(ff1 - ff2);
end
figure(1),
axes1 = axes('Position', [0.1, 0.1, 0.8, 0.8]); % 第1个坐标轴位置
axes(axes1);
h1 = plot(h, err, 'b-.'); % 画总体图
title('误差与h的关系图')
axes2 = axes('Position', [0.17, 0.45, 0.4, 0.35]); % 第2个坐标轴位置
axes(axes2);
h2 = plot(h, err, 'r');
title('局部放大图')
axis([-0.01, 0.25, -0.005, 0.04]); % 局部显示
hm = [h1; h2];
h1 = {'总体图', '局部图'};
legend(hm, h1, 'Location', [0.65, 0.2, 0.2, 0.1]); % 添加标注
an = annotation('arrow', [0.1, 0.17], [0.1, 0.45], 'Color', 'k'); % 画箭头
```

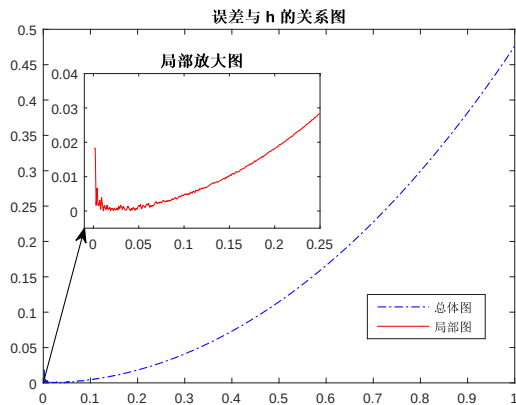


图 9.11: 误差与 h 的关系图

(2) 步长 $h = 0.1$ 。

```
>> h = 0.1;
>> x = (0:2) * 0.1;
>> y = exp(x);
>> ff1 = (y(1) - 2 * y(2) + y(3)) / (h ^ 2)
ff1 =
    1.1061
>> ff2 = 1;
>> err = abs(ff1 - ff2)
err =
    0.1061
```

4. (选作)考虑 n 次Lagrange插值多项式的构造: 取 $1, x, x^2, \dots, x^n$ 为次数不超过 n 次的代数多项式这个线性空间的一组基函数, 构造 n 次Lagrange插值多项式 $\varphi_n(x)$, 假设 $\varphi_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$, 只需由方程组

$$\begin{pmatrix} 1 & x_0 & \cdots & x_0^n \\ 1 & x_1 & \cdots & x_1^n \\ \vdots & \vdots & & \vdots \\ 1 & x_n & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix}$$

求得系数 $a_0, a_1, \dots, a_n$ 。这是范德蒙(Vandermonde)方程组, 也是一个著名的病态方程组。试设计程序, 展示并观察该方程组的病态现象。

```
function a = vande(x, y)
% 生成Vandermonde矩阵A, 并求解 Aa = y
% Input -x n维向量 -y n×m的矩阵
% Output -a Lagrange插值多项式的系数
s = length(x);
A = zeros(s);
for j = 1:s
    A(:,j) = x .^ (j - 1); % 生成Vandermonde矩阵
end
a = A \ y;
```

选取四组节点, 运行程序

```
>> x1 = [0.1, 1, 1000];
>> y1 = [2; 3; 4];
>> a1 = vande(x1, y1)
a1 =
    1.8888
    1.1123
   -0.0011
>> x2 = [0.1, 1, 1040];
>> a2 = vande(x2, y1)
a2 =
    1.8888
```

```

1.1123
-0.0011
>> x3 = [1, 1.1, 1.0999];
>> y3 = [1; 1.1; 1.0999];
>> a3 = vande(x3, y3)
a3 =
    0
    1
    0
>> y4 = [1; 1.1; 1.099];
>> a4 = vande(x3, y4)
a4 =
  99.0991
 -188.1892
  90.0901

```

虽然第一组与第二组差别很大，但它们的结果是几乎一样的；虽然第三组与第四组差别不大，但它们的结果差别却很大，故出现病态。

5. (选作)观察对于非光滑函数进行多项式插值问题：在 $[0, 1]$ 上取 $f(x) = |\sin k\pi x|$ ，选取不同的 k 和 n ，用等距节点做 n 次Lagrange插值多项式 $\varphi_n(x)$ ，观察 $|f(x) - \varphi_n(x)|$ 对不同 k 的影响。

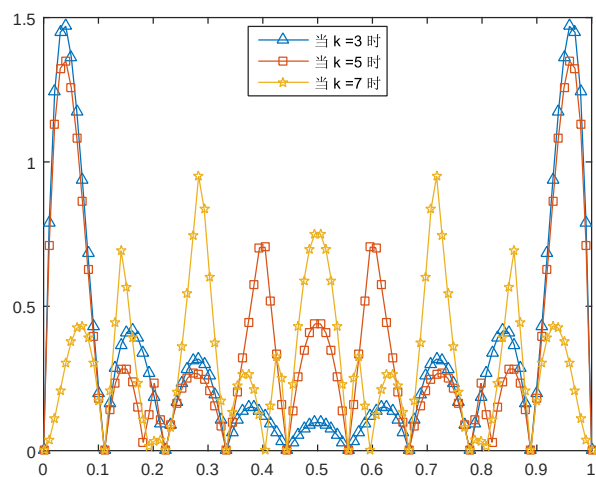


图 9.12: 误差与 k 的关系图

解 在 $[0, 1]$ 中取 $n = 10$ 个点进行插值。

```

mk = {'-^', '-s', '-p'};
x1 = linspace(0, 1, 10);
x2 = linspace(0, 1, 100);
for i = 3:2:7
    y1 = abs(sin(i * pi * x1));

```



```

y2 = lagrange(x1, y1, x2);
y3 = abs(sin(i * pi * x2));
err = abs(y3 - y2);
plot(x2, err, char(mk((i-1)/2)));
hold on
end
legend(strcat('当 k = ', num2str((3:2:7)')), '小时', ...
'location', 'Best') % strcat用于合并字符串, 详见第十五章

```

6. 发现并观察数据拟合法正规方程组的病态问题。

解

```

function b = zhgbt(x_1, x_2)
% 数据拟合法正规方程组的病态问题
% Input -x_1 x2(100,1)数据 -x_2 x2(100,2)数据
% Output -b 多项式拟合的系数
% Example
% b = zhgbt(101,102)
x2 = 100 * ones(99,2); % 构造数据x2, 100×2矩阵
x2(100,1) = x_1;
x2(100,2) = x_2;
x1 = ones(100, 1); % 全1的100维向量
y = 100 * ones(100, 1); % y数据, 100维向量
x = [x1, x2]; % x为100×3矩阵
A = x' * x; % A为的系数矩阵
y = x' * y; % y为3维向量
b = A \ y;

```

举例求解

```

>> b = zhgbt(100.1,101)
b =
    100
     0
     0
>> b = zhgbt(100.1,100.99)
b =
  114.5117
   -0.1614
    0.0163

```

虽然100.1、101、100.99非常接近, 但拟合出的系数相差很大, 故出现病态。

另一种判断病态的方法

```

function idx = btbz(x_1, x_2)
% 用范数判断病态 idx = btbz(x_1, x_2)
% Input -x_1 x2(100,1)数据 -x_2 x2(100,2)数据

```

```
% Output -idx 病态程度
% Example
% idx = btbz(101, 103)
x2 = 100*ones(99,2); % 构造数据x2, 100×2矩阵
x2(100,1) = x_1;
x2(100,2) = x_2;
x1 = ones(100, 1); % 全1的100维向量
x = [x1, x2]; % x为100×3矩阵
A = x' * x; % A为3×3的系数矩阵
lambda = abs(eig(A));
m1 = min(lambda);
m2 = max(lambda);
idx = m2 / m1;
```

求解

```
>> idx = btbz(101, 103)
idx =
    1.1400e+20
```

9.5 补充实验

注 9.2 实验的数据均在Excel工作簿——“实验报告数据.xls”中的某个工作表中，工作表名称在各题中给出。

- 1. 某天的气温变化如表 9.5 (工作表：气温变化)，试用最小二乘法找出这一天的气温变化规律。

表 9.5: 气温变化表

时刻 $t(h)$	0	1	2	3	4	5	6	7	8	9	10	11	12
温度 $T(^{\circ}C)$	15	14	14	14	14	15	16	18	20	22	23	25	28
时刻 $t(h)$	13	14	15	16	17	18	19	20	21	22	23	24	
温度 $T(^{\circ}C)$	31	32	31	29	27	25	24	22	20	18	17	16	

解 绘制点图，判断拟合函数的形式

```
data = xlsread('实验报告数据.xls', '气温变化');
x = [data(1,:), data(3,1:end-1)];
y = [data(2,:), data(4,1:end-1)];
plot(x, y, 'o')
ylim([10, 35]) % 限制y轴范围，结果如图 9.13
```

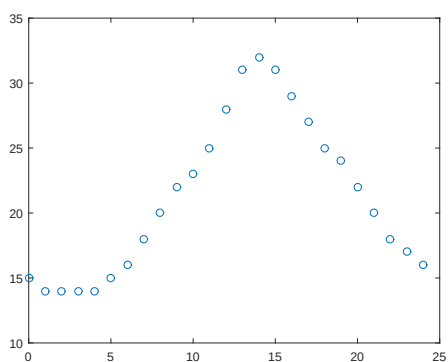


图 9.13: 温度变化的散点图

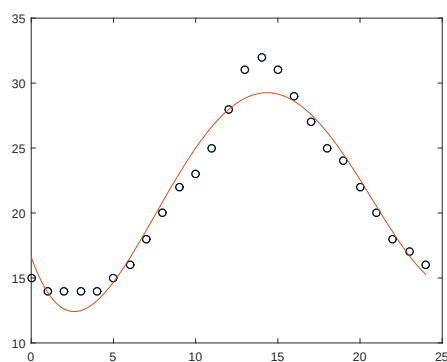


图 9.14: 温度变化的拟合曲线

```
>> fun = inline('c(1)*x.^4+c(2)*x.^3+c(3)*x.^2+c(4)*x+c(5)', 'c', 'x');
>> c = lsqcurvefit(fun, [0, 0, 0, 0, 0], x, y)
c =
    0.0009
   -0.0521
    0.8658
   -3.5257
   16.6041
>> xi = 0:0.1:24;
>> yi = polyval(c, xi);
>> plot(x, y, 'o', xi, yi) % 结果如图 9.14
```

故最终的拟合结果为 $0.0009x^4 - 0.0521x^3 + 0.8658x^2 - 3.5257x + 16.6041$ 。

2. 用电压 $V = 10V$ 的电池给电容器充电, 电容器上 t 时刻的电压为 $u(t) = V - (V - V_0)e^{-t/\tau}$, 其中 V_0 是电容器的初始电压, τ 是充电常数。试由表 9.6 (工作表: 电容器) 中的一组 t, V 数据确定 V_0 和 τ 。

表 9.6: 电压变化表

$t(h)$	0.5	1	2	3	4	5	7	9
$V(V)$	6.4	6.5	7.3	8.2	8.7	9	9.4	9.6

解 绘制点图

```
data = xlsread('实验报告数据.xls', '电容器');
x = data(1,:);
y = data(2,:);
plot(x, y, 'o')
xlim([0, 10]) % 控制x轴范围, 结果如图 9.15
```

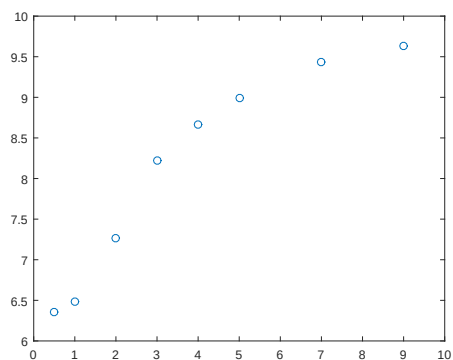


图 9.15: 电压变化的散点图

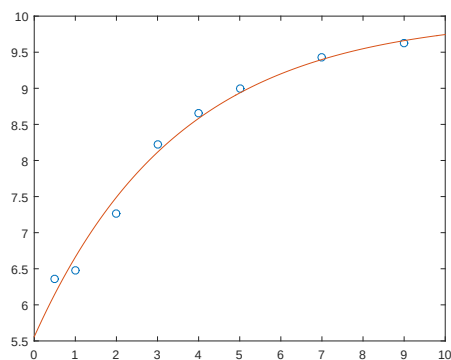


图 9.16: 电压变化的拟合曲线

最小二乘法拟合

```
>> fun = inline('10-(10-c(1))*exp(-x/c(2))', 'c', 'x');
>> c = lsqcurvefit(fun, [6, 3], x, y)
c =
    5.5577
    3.5002
>> xi = 0:0.1:10;
>> yi = 10 - (10 - c(1)) * exp(-xi / c(2));
>> plot(x, y, 'o', xi, yi) % 结果如图 9.16
```

故 $V_0 \approx 5.56$, $\tau \approx 3.50$ 。

3. 给定数据见表 9.7 (工作表: 三次样条)。

表 9.7: 样条插值数据

x	0.25	0.30	0.39	0.45	0.53
y	0.5000	0.5477	0.6245	0.6708	0.7280

分别就下列边界条件求三次样条函数 $S(x)$ 并作图。

(1) $S'(0.25) = 1$, $S'(0.53) = 0.6868$;

(2) $S''(0.25) = S''(0.53) = 0$ 。

解 (1)

```
data = xlsread('实验报告数据.xls', '三次样条');
x = data(1,:);
y = data(2,:);
s = csape(x, y, 'complete', [1, 0.6868]); % 一阶导数边界条件
fnplt(s, 'b');
hold on;
plot(x, y, 'o');
axis([0.23, 0.55, 0.48, 0.75]) % 结果如图 9.17
```

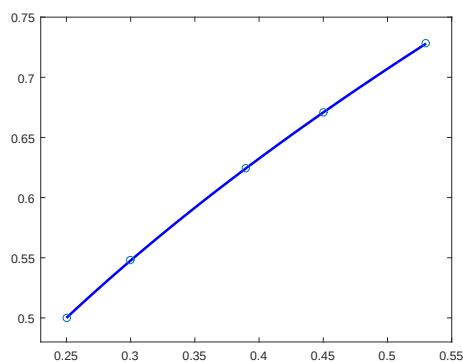


图 9.17: 三次样条插值一阶导数边界

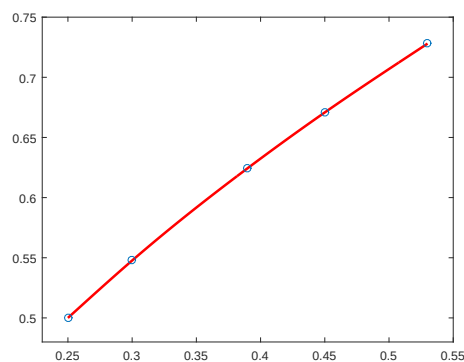


图 9.18: 三次样条插值二阶导数边界

```
data = xlsread('实验报告数据.xls', '三次样条');
x = data(1,:);
y = data(2,:);
s = csape(x, y, 'variational');
fnplt(s, 'r');
hold on;
plot(x, y, 'o');
axis([0.23, 0.55, 0.48, 0.75]) % 结果如图 9.18
```

4. 某商品的需求量与消费者的平均收入及商品价格的统计数据见表 9.8 (工作表: 需求量收入价格)。

表 9.8: 需求量与收入、商品价格

需求量	100	75	80	70	50	65	90	100	110	60
收入	1000	600	1200	500	300	400	1300	1100	1300	300
价格	5	7	6	6	8	9	5	4	3	9

(1) 建立回归模型并进行检验;

(2) 预测平均收入为1000, 价格为6时的商品需求量。

解 (1)

```
data = xlsread('实验报告数据.xls', '需求量收入价格');
x1 = data(2,:);
x2 = data(3,:);
y = data(1,:);
subplot(221)
plot(x1, y, '+')
title('收入与需求量的点图')
xlabel('收入')
ylabel('需求量')
subplot(222)
plot(x2, y, '+')
title('价格与需求量的点图')
xlabel('价格')
ylabel('需求量')
x = [ones(10,1), x1', x2'];
y = y';
[b, bint, r, rint, stats] = regress(y, x, 0.05);
b, bint, stats
subplot(2, 2, [3, 4])          % 合并两个子图
rcoplot(r, rint)
set(gca, 'color', [1, 1, 1])   % 结果如图 9.19
```

输出结果

```
b =
    111.6918
     0.0143
    -7.1882
bint =
    56.0503    167.3334
    -0.0120     0.0406
   -13.2306    -1.1458
stats =
    0.8944    29.6533    0.0004    52.0311
```

由以上结果可见, 相关系数 $R^2 = 0.8944$, $p = 0.0004 < 0.05$, 观察图 9.19 中的残差分布, 各个点的置信区间均包含零点。因此, 此回归模型是成立的, 需求量 y 与收入 x_1 、价格 x_2 之间的关系为 $y = 111.6918 + 0.0143x_1 - 7.1882x_2$ 。

(2)

```
>> xx = [1, 1000, 6];
>> yy = xx * b
yy =
    82.8594
```

故预测平均收入为1000、价格为6时的商品需求量为82.8594。

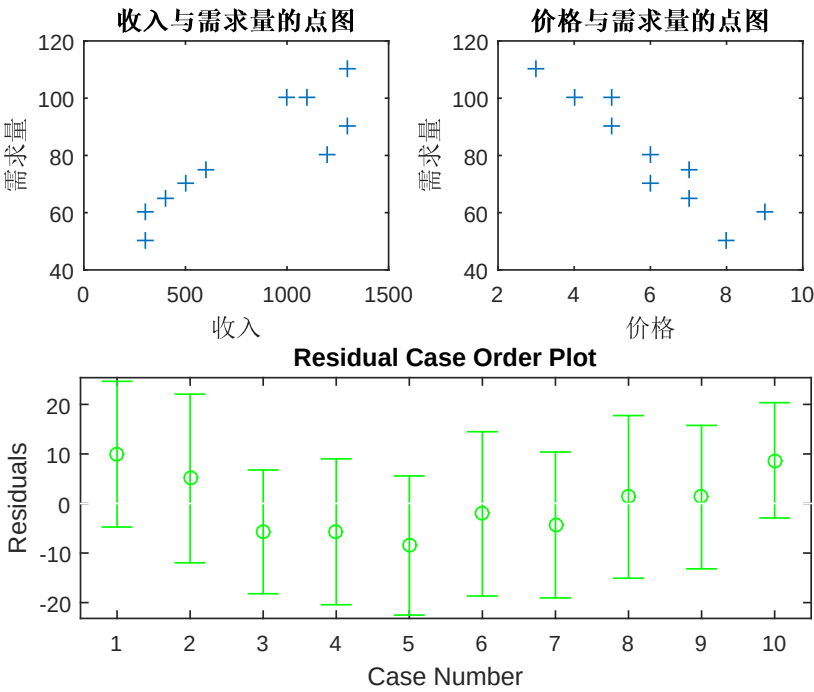


图 9.19: 需求量与收入、商品价格的散点图与回归残差图

表 9.9: 山区海拔

480	135	137	139	140	141	96	94	88	80	69	57	43	29	21	15
440	137	139	141	143	144	114	111	105	95	82	69	54	38	30	21
400	138	141	143	145	147	132	128	120	108	94	78	62	46	37	35
360	142	143	145	148	151	155	151	143	130	120	98	85	75	55	50
320	143	145	146	150	155	160	155	160	160	160	155	150	150	155	155
280	95	119	137	150	120	110	155	160	155	138	107	90	105	115	120
240	91	109	127	150	120	110	135	145	120	115	101	88	100	105	110
200	88	106	123	139	150	150	140	90	110	106	95	87	90	93	95
160	83	98	118	132	145	142	140	130	70	90	85	84	38	78	75
120	74	88	108	113	125	128	123	104	90	50	70	78	75	65	55
80	65	76	88	97	102	105	102	83	80	70	30	50	55	48	35
40	51	62	73	80	85	87	85	78	72	65	50	20	30	35	32
0	37	47	55	60	67	69	67	62	58	45	40	30	10	15	25
Y/X	0	40	80	120	160	200	240	280	320	360	400	440	480	520	560

5. (选做)表 9.9 (工作表: 山区海拔)是一山区海拔高度每400m的网格数据(单位: 10m)。为了作修建道路的成本预算, 要求给出每100m的网格数据(注意山峰、山谷和溪流)。

解

```
data = xlsread('实验报告数据.xls', '山区海拔');  
x = 0:40:560;  
y = 0:40:480;  
[x, y] = meshgrid(x, y);  
z = data(1:end-1, 2:end);  
xi = 0:10:560;  
yi = 0:10:480;  
[xi, yi] = meshgrid(xi, yi);  
zi = interp2(x, y, z, xi, yi, 'spline'); % 二维三次样条插值  
mesh(xi, yi, zi)  
axis([0, 560, 0, 480, 0, 200])  
view(25, 65) % 结果如图 9.20
```

```
>> h = contour(xi, yi, zi); % 等高线  
>> clabel(h) % 标注等高线  
>> axis equal % 结果如图 9.21
```

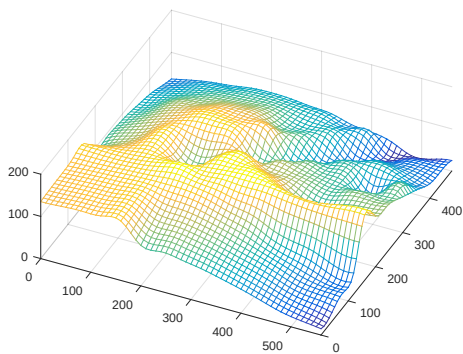


图 9.20: 山区地形图

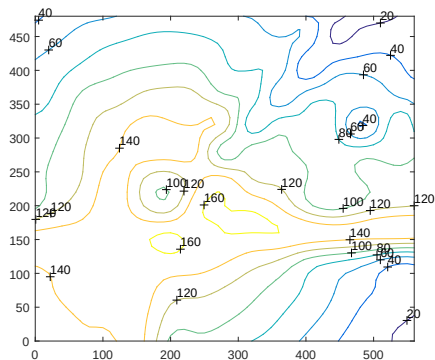


图 9.21: 山区等高线图

第十章 数值积分

对区间 $[a, b]$ 上定义的Riemann可积函数 $f(x)$ 来构造定积分

$$\int_a^b f(x)dx$$

的近似计算公式是数值积分研究的重点。在实际问题中，被积函数的原函数往往无法用初等函数表示，或者被积函数是用离散的点表示的，这些情况都需要用到数值积分。

表 10.1: 数值微积分的主要函数

微分函数	功能	积分函数	功能
diff	数值差分	trapz	梯形积分法
gradient	数值导数与梯度	quad	Simpson 积分法
polyder	多项式导数	quadl	高精度积分
quiver	方向导数图	dblquad	二重积分
		triplequad	三重积分

10.1 求积公式

10.1.1 梯形公式

过 $(a, f(a))$, $(b, f(b))$ 两点作直线

$$P_1(x) = \frac{x-a}{b-a}f(b) + \frac{x-b}{a-b}f(a)$$

用 $P_1(x)$ 代替 $f(x)$, 可得

$$\int_a^b f(x)dx \approx \int_a^b P_1(x)dx = \frac{b-a}{2}(f(a) + f(b))$$

若将区间 $[a, b]$ n 等分, 在每个小区间上应用梯形公式就可得到复化梯形公式

$$\int_a^b f(x)dx = \sum_{k=0}^n \int_{x_k}^{x_{k+1}} f(x)dx \approx \sum_{k=0}^n \frac{x_{k+1} - x_k}{2} (f(x_{k+1}) + f(x_k))$$

若记 $h = \frac{b-a}{n}$, 则有 $T_n = \frac{h}{2}(f(a) + f(b) + 2 \sum_{k=1}^{n-1} f(a + kh))$ 。

例 10.1 编写复化梯形求积公式的程序, 并用之求积分 $\int_0^\pi \sin x dx$ 。

解 复化梯形求积的程序

```
function t = trape(fun, a, b, n)
% 复化梯形求积公式
% Input -fun 被积函数 -[a,b] 积分区间 -n n等分
% Output -t 积分值
x = linspace(a, b, n+1);
h = (b - a)/n;
s = 0;
for i = 2:n
    s = s + 2 * feval(fun, x(i));
end
t = h / 2 * (feval(fun, a) + feval(fun, b) + s);
```

计算积分

```
>> fun = inline('sin(x)');
>> a = 0; b = pi;
>> n = 100;
>> t = trape(fun, a, b, n)
t =
    1.9998
```

10.1.2 抛物线公式

将区间 $[a, b]$ 2等分, 过 $(a, f(a))$, $(b, f(b))$, $(\frac{a+b}{2}, f(\frac{a+b}{2}))$ 三点, 作抛物线 $P_2(x)$, 用 $P_2(x)$ 代替 $f(x)$, 可得

$$\int_a^b f(x)dx \approx \frac{b-a}{6}(f(a) + 4f(\frac{a+b}{2}) + f(b))$$

若将区间 $[a, b]$ n 等分, 在每个小区间上应用抛物线公式就可得到复化抛物线公式

$$\begin{aligned} \int_a^b f(x)dx &= \sum_{k=0}^n \int_{x_k}^{x_{k+1}} f(x)dx \\ &\approx \sum_{k=0}^n \frac{h}{3}(f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})) \\ &= \frac{h}{3}(f(a) + f(b) + 4 \sum_{k=1}^n f(x_{2k-1}) + 2 \sum_{k=1}^{n-1} f(x_{2k})) \\ &= S_n \end{aligned}$$

例 10.2 编写复化抛物线求积公式的程序, 并用之求积分 $\int_{-1}^1 \frac{1}{\sqrt{5-4x}}dx$ 。

```
function s = para(fun, a, b, n)
% 复化抛物线求积公式
% Input -fun 被积函数 -[a,b] 积分区间 -n n等分
% Output -s 积分值
x = linspace(a, b, n+1);
h = (b - a) / n;
s = 0;
for i = 1:n
    s = s + feval(fun, x(i)) + 4 * feval(fun, (x(i) + x(i+1)) / 2) + ...
        feval(fun, x(i+1));
end
s = h / 6 * s;
```

计算积分

```
>> fun = inline('1/sqrt(5-4*x)');
>> a = -1; b = 1;
>> n = 100;
>> s = para(fun, a, b, n)
s =
    1.0000
```

10.2 逐次分半法

逐次分半法是在求积过程中依据精度要求自动确定等分份数 n 的一种方法。通常的做法是在前一次划分区间的基础上, 在把每个小区间二等分, 逐次分半, 并以前后两次计算结果的差来估计误差。

算法 10.1 梯形公式的逐次分半法。

Algorithm 12: $TRAPE(a, b, \varepsilon)$

Input: 区间范围 a, b , 精度 ε

Output: I

- 1 置 $n = 1$, $h = \frac{b-a}{2}$, $T_0 = h(f(a) + f(b))$
 - 2 置 $F = 0$, 对 $i = 1, 2, \dots, n$ 求 $F = F + f(a + (2i - 1)h)$
 - 3 $T = \frac{T_0}{2} + hF$
 - 4 若 $|T - T_0| < 3\varepsilon$, 输出 $I = T$, 停止计算; 否则, 转步骤 5
 - 5 置 $n = 2n$, $h = \frac{h}{2}$, $T_0 = T$, 转步骤 2
-

对应的MATLAB程序为

```
function t = trape2(fun, a, b, eps)
% 梯形求积公式的逐次分半法
% Input -fun 被积函数 -[a,b] 积分区间 -eps 精度
% Output -t 积分值
if nargin < 4, eps = 1e-6; end
n = 1;
h = (b - a) / 2;
t0 = h * (feval(fun, a) + feval(fun, b));
t = t0 + 100 * eps;
while abs(t - t0) > 3 * eps
    t0 = t;
    f = 0;
    for i = 1:n
        f = f + feval(fun, a + (2 * i - 1) * h);
    end
    t = t0 / 2 + h * f;
    n = 2 * n;
    h = h / 2;
end
```

例 10.3 用梯形求积公式的逐次分半法求积分 $\int_1^2 \frac{1}{x} dx$ 。

```
>> fun = inline('1./x');
>> a = 1;
>> b = 2;
>> t = trape2(fun, a, b)
t =
    0.6931
```

算法 10.2 抛物线公式的逐次分半法。

对应的MATLAB程序为

Algorithm 13: $PARA(a, b, \varepsilon)$ **Input:** 区间范围 a, b , 精度 ε **Output:** I

- 1 置 $n = 2$, $h = \frac{b-a}{4}$, $F_1 = h(f(a) + f(b))$, $F_2 = f(\frac{a+b}{2})$, $S_0 = \frac{b-a}{6}(F_1 + 4F_2)$
- 2 置 $F_3 = 0$, 对 $i = 1, 2, \dots, n$ 求 $F_3 = F_3 + f(a + (2i-1)h)$
- 3 $S = \frac{h}{3}(F_1 + 2F_2 + 4F_3)$
- 4 若 $|S - S_0| < 15\varepsilon$, 输出 $I = S$, 停止计算; 否则, 转步骤5
- 5 置 $n = 2n$, $h = \frac{h}{2}$, $F_2 = F_2 + F_3$, $S_0 = S$, 转步骤2

```

function s = para2(fun, a, b, eps)
% 抛物线的求积公式的逐次分半法
% Input -fun 被积函数 -[a,b] 积分区间 -eps 精度
% Output -t 积分值
if nargin < 4, eps = 1e-6; end
n = 2;
h = (b - a) / 4;
f1 = feval(fun, a) + feval(fun, b);
f2 = feval(fun, (a + b) / 2);
s0 = (b - a) / 6 * (f1 + 4 * f2);
s = s0 + 100 * eps;
while abs(s - s0) > 15 * eps
    s0 = s;
    f3 = 0;
    for i = 1:n
        f3 = f3 + feval(fun, a + (2 * i - 1) * h);
    end
    s = h / 3 * (f1 + 2 * f2 + 4 * f3);
    n = 2 * n;
    h = h / 2;
    f2 = f2 + f3;
end

```

例 10.4 用抛物线公式的逐次分半法求积分 $\int_0^{\frac{\pi}{2}} \cos^2 \frac{x}{2} dx$ 。

```

>> fun = inline('cos(x/2)^2');
>> a = 0;
>> b = pi / 2;
>> s = para2(fun, a, b)
s =
    1.2854

```

两个逐次分半法的程序while循环条件的控制也可以用 ε 来控制, 提高了计算精度, 但由于使用了循环的嵌套, 运算速度要慢很多。下一节我们讲学习具有加速收敛技巧的Romberg求积法。

10.3 Romberg求积法

Romberg求积法需要用到Richardson外推法,有兴趣的读者可以去研读徐萃薇、孙绳武编著的《计算方法引论》第三版第五章第五节的内容。这里只给出Romberg求积算法和程序。

算法 10.3 Romberg求积法。

Algorithm 14: *ROMBERG*(a, b, ε)

Input: 区间范围 a, b , 精度 ε

Output: I

1 置 $n = 1, k = 1, h = \frac{b-a}{2}, T_0^{(0)} = h(f(a) + f(b))$

2 置 $F = 0$, 对 $i = 1, 2, \dots, n$ 求 $F = F + f(a + (2i-1)h)$

3 $T_0^{(k)} = \frac{T_0^{(k-1)}}{2} + hF$

4 对 $m = 1, 2, \dots, n$ 计算

$$T_m^{(k-m)} = \frac{4^m T_{m-1}^{k-m+1} - T_{m-1}^{k-m}}{4^m - 1}$$

5 若 $|T_m^{(0)} - T_{m-1}^{(0)}| < \varepsilon$, 输出 $I = T_m^{(0)}$, 停止计算; 否则, 转步骤 6

6 置 $n = 2n, h = \frac{h}{2}, k = k + 1$, 转步骤 2

对应的MATLAB程序为

```
function s = Romberg(fun, a, b, eps)
% Romberg求积法
% Input -fun 待求积函数 -[a,b] 积分区间 -eps 允许的误差值
% Output -s Romberg积分
if nargin == 3, eps = 1.0e-6; end
if nargin < 3, error('Error in input!'); end
t0 = 100; t1 = -100;
n = 2;
h = (b - a) / 2;
t(1,1) = h * (feval(fun, a) + feval(fun, b));
while abs(t1 - t0) >= eps
    f = 0;
    for i = 1:(2^(n-1))
        f = f + h / 2 * (feval(fun, a + h * (i-1)) + feval(fun, h * i + a));
    end
    t(n,1) = f;
    for j = 2:n
        for i = 1:(n-j+1)
            t(i,j) = (4^(j-1) * t(i+1,j-1) - t(i,j-1)) / (4^(j-1) - 1);
        end
    end
    t0 = t(1,n); t1 = t(1,n-1);
    n = n + 1; h = h / 2;
end
```

```
end
s = t1;
```

例 10.5 用Romberg求积法求积分 $\int_0^1 e^{2x} dx$ 。

```
>> fun = inline('exp(2*x)');
>> a = 0; b = 1;
>> s = Romberg(fun, a, b)
s =
    3.1945
```

10.4 实验报告

1. 已知椭圆的周长可以表示成 $s = a \int_0^{2\pi} \sqrt{1 + \rho^2 \cos^2 \theta} d\theta$ ($0 < \rho < 1$), 取 $a = 1$, 针对 ρ 从 0.1 到 0.9 (取步长 $d\rho = 0.1$), 用Romberg积分方法求出各自的周长 s 。

解 编写程序如下

```
function s = call_romberg
% Romberg 求积法求椭圆积分
% Output -s 9×2 矩阵, 第1列为p的数据, 第2列是参数为p的椭圆积分
p = 0.1:0.1:0.9;
s = zeros(9, 2);
s(:,1) = p';
a = 0; b = 2 * pi; % 积分上下限
p2 = p.^2; % p的平方
for i = 1:9
    str = ['(1+', num2str(p2(i)), '*cos(x)^2)^(1/2)'];
    f = inline(str);
    s(i,2) = Romberg(f, a, b);
end
```

求解结果

```
>> s = call_romberg
s =
    0.1000    6.3145
    0.2000    6.4076
    0.3000    6.5598
    0.4000    6.7672
    0.5000    7.0248
    0.6000    7.3274
    0.7000    7.6696
    0.8000    8.0464
    0.9000    8.4532
```

2. 用Romberg积分法和二分法确定积分上限 k , $k \in (0, 1)$, 其中

$$\int_0^k e^{2x} dx = \frac{3}{2}.$$

解 编写待求根函数如下:

```
function d = intfun(k)
ifun = inline('exp(2* $x$ )');
r = Romberg(ifun, 0, k);
d = r - 3/2;
```

用二分法求根

```
>> k = dichotomy(@intfun, 0, 1) % 二分法程序详见第五章
k =
    0.6931
```

经计算得, $k = 0.6931$ 。实际上, 通过解析的方法, 我们也可以得出 k 的值:

$$\int_0^k e^{2x} dx = \frac{1}{2} e^{2x} \Big|_0^k = \frac{1}{2} (e^{2k} - 1) = \frac{3}{2},$$

于是可得 $k = \ln 2 \approx 0.6931$ 。

第十一章 常微分方程数值解法

常微分方程(ODE)在研究实际问题中有着十分广泛的应用。有些常微分方程可以利用分离变量法、积分因子法来求出准确解,但很多常微分方程问题都没有准确解,只能用数值解法求其近似值。解常微分方程的指令见表 11.1。

表 11.1: 解常微分方程的指令

函数	功能	函数	功能
ode45	四、五阶Runge-Kutta法	ode15s	梯形积分法
ode23	二、三阶Runge-Kutta法	ode23s	刚性方程组二阶Rosenbrock法
ode113	多步Adams法	ode23tb	刚性方程组低精度算法
ode23t	适度刚性问题梯形算法	bvpinit	边值问题预估解
odeset	解ode选项设置	bvp4c	边值问题解法
dsolve	求微分方程的精确解	deval	微分方程解的求值

11.1 Euler法及其改进

一阶常微分方程初值问题

$$\begin{cases} \frac{dy}{dx} = f(x, y) \\ y(x_0) = y_0 \end{cases}$$

可用Euler法求近似解。Euler法的思路很简单：在节点处用差商代替导数

$$y'(y_k) = \frac{y(t_{k+1}) - y(t_k)}{h}$$

从而导出递推公式

$$y_{k+1} = y_k + hf(t_k, y_k), \quad k = 0, 1, 2, \dots,$$

因而Euler法的计算公式可以归结为

$$\begin{cases} y_0 = x_0 \\ y_{k+1} = y_k + hf(t_k, y_k), \quad k = 0, 1, 2, \dots, n-1. \end{cases}$$

例 11.1 编写Euler法程序求解方程并图示与精确解的比较。

$$\begin{cases} y' = y - \frac{2x}{y}, \quad x \in [0, 1], \quad h = 0.1, \\ y(0) = 1. \end{cases}$$

解 编写Euler法程序

```
function [x, y] = euler(odefun, tspan, y0, h)
% Euler法求解常微分方程
% Input -odefun 常微分方程 -tspan 求解区间 -y0 初值 -h 步长
% Output -x 自变量 -y 函数值
x = tspan(1):h:tspan(2); y = zeros(n,1);
n = length(x);
y(1) = y0;
for i = 1:n-1
    y(i+1) = y(i) + h * feval(odefun, x(i), y(i));
end
x = x(:); y = y(:);
```

求解方程并图示结果

```
>> odefun = inline('y - 2*x/y', 'x', 'y');
>> tspan = [0,1];
>> y0 = 1;
>> h = 0.1;
>> [x, y] = euler(odefun, tspan, y0, h)
x =
    0
 0.1000
```

```

0.2000
0.3000
0.4000
0.5000
0.6000
0.7000
0.8000
0.9000
1.0000
y =
1.0000
1.1000
1.1918
1.2774
1.3582
1.4351
1.5090
1.5803
1.6498
1.7178
1.7848
>> plot(x, y, 'k-o')
>> hold on
>> fun = 'Dy=y-2*x/y';
>> s = dsolve(fun, 'y(0)=1', 'x')
s =
(2*x + 1)^(1/2)
>> ezplot(s, [0, 1])
>> legend('Euler法', '精确解', 'location', 'best') % 结果如图 11.1

```

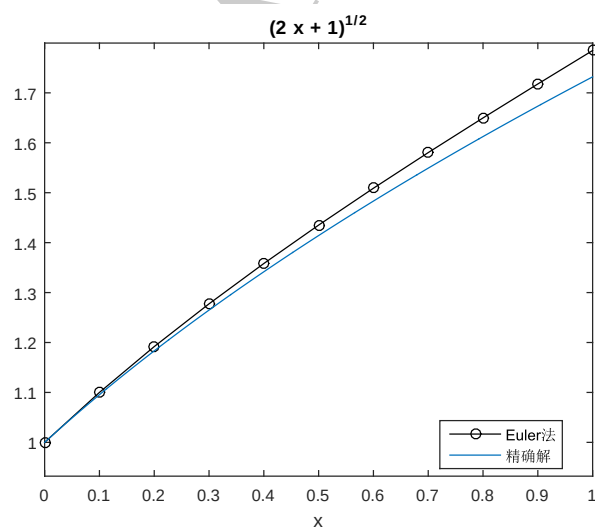


图 11.1: Euler法的近似解与精确解的比较

对本题而言，由递推公式和图 11.1 可知，Euler法的计算误差随着递推而增加，因而它的计算精度并不高，在实际计算中并不推崇。这里再介绍改进的Euler方法。

改进的Euler法的主要思想是先用Euler法求解出初值再利用下面的迭代公式求解。

$$\begin{cases} y_{n+1}^{(0)} = y_n + hf(x_n, y_n) \\ y_{n+1}^{(k+1)} = y_n + \frac{h}{2}(f(x_n, y_n) + f(x_n, y_n^{(k)})), \quad k = 0, 1, \dots, n-1 \end{cases}$$

程序如下：

```
function [x, y] = eulerm(odefun, tspan, y0, h, eps)
% 改进Euler法解常微分方程
% Input -odefun 常微分方程 -tspan 求解区间[a,b] -y0 初值 -h 步长 -eps 计算精度
% Output -x 自变量 -y 函数值
if nargin == 4, eps = 1e-6; end % 输入参数个数为4时, 精度默认为1e-6
x = tspan(1):h:tspan(2); % 按步长取点
n = length(x);
y = zeros(n,1);
y(1) = y0; % 初值
for i = 1:length(x)-1
    yNew = y(i);
    yOld = y(i) + h * feval(odefun, x(i), y(i)); % 迭代初值
    fi = feval(odefun, x(i+1), y(i));
    k = 1; % 迭代步数
    flag = 1; % 将循环条件flag设为1以进入循环迭代
    while flag && k < 500
        yNew = y(i) + h/2 * (fi + feval(odefun, x(i+1), yOld));
        flag = abs(yNew - yOld) > eps;
        yOld = yNew; k = k + 1; % 更改迭代
    end % 迭代收敛或迭代次数超过500次时终止循环
    y(i+1) = yNew;
end
x = x(:);
```

例 11.2 用Euler法和改进Euler法求解微分方程 $u' = -5u$, $u(0) = 1$, $0 \leq t \leq 1$, 并比较误差。

解 绘图比较

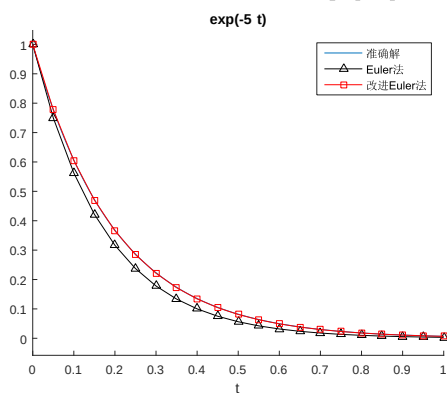


图 11.2: 改进Euler法与精确解比较

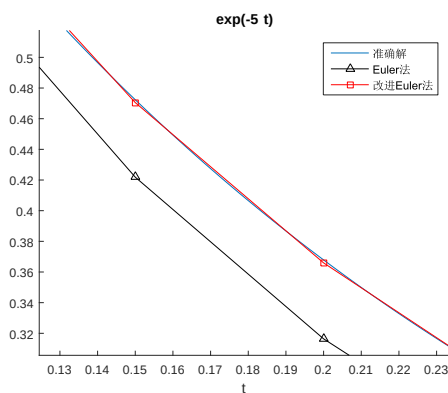


图 11.3: (图 11.2 的)局部放大图

```
odefun = inline('-u^5*u', 't', 'u');
tspan = [0, 1];
u0 = 1; h = 0.01;
[x, y] = euler (odefun, tspan, u0, h);
hold on
s = dsolve('Du=-5*u', 'u(0)=1', 't');
ezplot(s, [0, 1])
plot(x, y, 'k-')
[x, y] = eulerm(odefun, tspan, u0, h);
plot(x, y, 'r-s');
legend('准确解', 'Euler法', '改进Euler法') % 结果如图 11.2
```

尽管改进的Euler法能够减小误差，但迭代法求方程的根的使用却加大了计算量，而且还需要判断迭代法是否收敛。相比而言，目前比较常用方法是Runge-Kutta法和Adams法。

11.2 解常微分方程的指令

11.2.1 初值问题解法

常微分方程初值问题解法的MATLAB指令(如ode45、ode23等)具有相同的格式，以最常用的ode45为例，调用格式如表 11.2。

表 11.2: 解常微分方程初值问题的指令

调用格式	功能说明
$[t, y] = \text{ode45}(\text{odefun}, \text{tspan}, y_0)$	odefun用以表示 $f(t, y)$ 的函数句柄或inline导入的函数，t是标量，y是标量或向量；tspan是变量范围，二维向量， $[t_0, t_n]$ 表示自变量初值 t_0 和终值 t_n ，高维向量用 $[t_0, t_1, t_2, \dots, t_n]$ ； y_0 表示初值向量；输出向量t为自变量数据，y为相应的函数值，若无参数输出，则输出函数图像
$[t, y] = \text{ode45}(\text{odefun}, \text{tspan}, y_0, \text{options}, p_1, p_2, \dots)$	options为计算参数(如计算精度)设置，默认为空矩阵[]； $p_1, p_2, \dots$ 为附加参数，odefun可由 $f(t, y, p_1, p_2, \dots)$ 表示

例 11.3 用四、五阶Runge-Kutta法求解方程并图示结果。

$$\begin{cases} y' = y - \frac{2x}{y}, & x \in (0, 4), \\ y(0) = 1. \end{cases}$$

```

odefun = inline('y_2 - x_1 / y', 'x', 'y');
tspan = [0, 4];
y0 = 1;
[x,y] = ode45(odefun, tspan, y0);
plot(x, y, '-o') % 结果如图 11.4

```

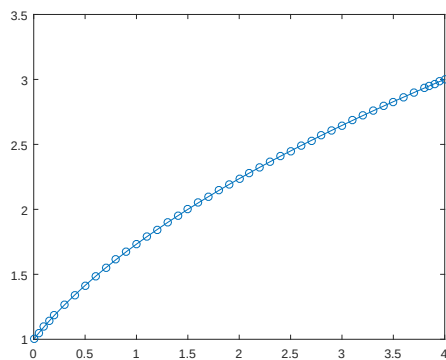


图 11.4: Runge-Kutta法求解方程

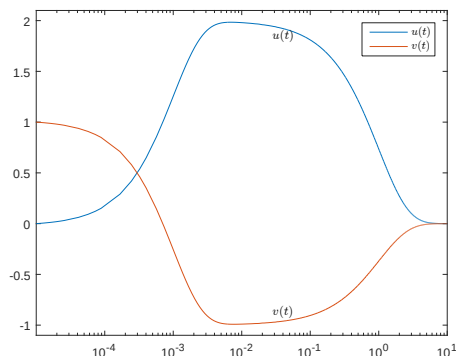


图 11.5: Runge-Kutta法求解方程组

例 11.4 用四、五阶 *Runge-kutta* 法计算方程组

$$\begin{cases} u' = 998u + 1998v, \\ v' = -999u - 1999v, \end{cases} \quad \begin{cases} u(0) = 1, \\ v(0) = 0, \end{cases} \quad t \in [10^{-5}, 10].$$

解 先构造方程组

```

function y = odefun(t, x)
y(1) = 998 * x(1) + 1998 * x(2);
y(2) = -999 * x(1) - 1999 * x(2);
y = y(:);

```

求解方程组

```

tspan = [1e-5, 10];
y0 = [0; 1];
[x, y] = ode45(@odefun, tspan, y0);
semilogx(x, y) % 单对数坐标轴x轴绘图
str1 = '$$u(t)$$'; str2 = '$$v(t)$$';
h = legend(str1, str2);
set(h, 'interp', 'latex')
text(x(80), y(80,1) - 0.1, str1, 'interp', 'latex')
text(x(80), y(80,2) + 0.1, str2, 'interp', 'latex')
axis([1e-5, 10, -1.1, 2.1]) % 结果如图 11.5

```

11.2.2 边值问题解法

一阶常微分方程边值问题

$$\begin{cases} y'(t) = f(t, y(t)), & x \in (a, b) \\ y(a) = y(b) = 0 \end{cases}$$

的MATLAB解法如表 11.3。

表 11.3: 解常微分方程边值问题的指令

调用格式	功能说明
sinit = bvpinit(tinit, yinit)	由在粗略结点tinit的预估解yinit生成粗略解网络sinit
sol = bvp4c(odefun, bcfun, sinit)	odefun是微分方程的函数，bcfun是边值条件函数，sol是一个解的结构，sol.x是求解结点，sol.y是数值解
sx = deval(sol, ti)	计算sol在ti的值

例 11.5 求解边值问题

$$\begin{cases} y'' + y = 0, & t \in (0, 4), \\ y(0) = 0, y(4) = 2. \end{cases}$$

解 令 $x(1) = y, x(2) = y'$ ，改写方程为

$$\begin{cases} x_1' = -x_2 \\ x_2' = -x_1 \\ x_1(0) = 0 \\ x_1(4) - 2 = 0 \end{cases}$$

```
>> sinit = bvpinit(0:4, [0; 0]);
>> odefun = inline('[x(2); -x(1)]', 't', 'x');
>> bcfun = inline('[x(1); x(1)-2]', 'xa', 'xb');
>> sol = bvp4c(odefun, bcfun, sinit)
sol =
    solver: 'bvp4c'
         x: [1x17 double]
         y: [2x17 double]
        yp: [2x17 double]
       stats: [1x1 struct]
>> ti = linspace(0, 4, 101);
>> y = deval(sol, ti);
>> plot(ti, y(1,:), sol.x, sol.y(1,:), 'o', sinit.x, sinit.y(1,:), '+')
>> legend('解曲线', '解的结点', '粗略解', 'location', 'best') % 结果如图 11.6
```

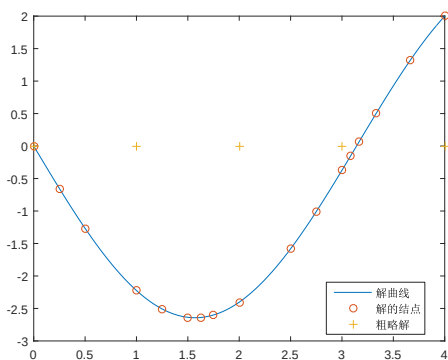


图 11.6: 常微分方程边值问题求解

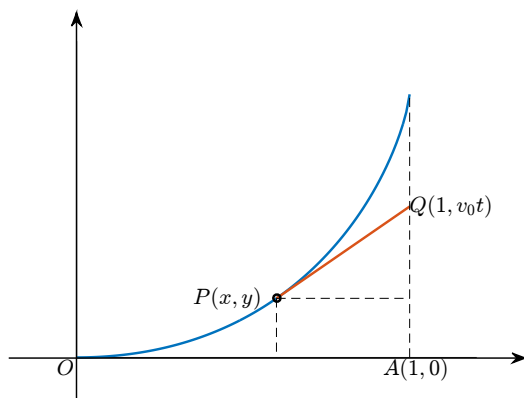


图 11.7: 导弹追击示意图

11.3 导弹追击问题

11.3.1 提出问题

设位于坐标原点的甲舰向位于 x 轴上点 $A(1, 0)$ 处的乙舰发射导弹，导弹头始终对准乙舰。如果乙舰以最大的速度 v_0 (常数)沿平行于 y 轴的直线行驶，导弹的速度是 $5v_0$ ，求导弹运行的曲线方程，乙舰行驶多远时，导弹将它击中？

11.3.2 模型建立与求解

解析法

如图 11.7 所示，由于导弹头始终对准乙舰，故此时直线 $\overline{PQ}$ 就是导弹的轨迹曲线弧 $\widehat{OP}$ 在点 P 处的切线。于是可得 $y' = \frac{v_0 t - y}{1 - x}$ ，即

$$v_0 t = (1 - x)y' + y.$$

弧 $\widehat{OP}$ 的长度为 $|\overline{AQ}|$ 的5倍，即

$$\int_0^x \sqrt{1 + (y')^2} dx = 5v_0 t.$$

联立两式，消去 t 可得模型

$$(1 - x)y'' = \frac{1}{5}v_0 t,$$

其中，初始条件为 $y(0) = y'(0) = 0$ 。

求解程序如下：


```
>> s = dsolve('(1-x)*D2y-1/5*sqrt(1+(Dy)^2)', 'y(0)=0', 'Dy(0)=0', 'x');
>> s = simplify(s);
>> pretty(s)
/          5 \
| - #2 - #1 - -- |
|          24 |
|          |
|          5 |
| #2 + #1 + -- |
\          24 /
where
          4/5          6/5
      5 (-1) (x - 1)
#1 == -----
          12
          1/5          4/5
      5 (-1) (x - 1)
#2 == -----
          8
```

故导弹运行的曲线为 $y = -\frac{5}{8}(1-x)^{\frac{4}{5}} \pm \frac{5}{12}(1-x)^{\frac{6}{5}} \pm \frac{5}{24}$ 。

当 $x = 1$ 时, $y = \frac{5}{24}$, 即当乙舰航行到点 $(1, \frac{5}{24})$ 处被击中。击中时间 $t = \frac{y}{v_0} = \frac{5}{24v_0}$ 。

数值解法

令 $y_1 = y'$, $y_2 = y_1'$, 将方程化为一阶微分方程组

$$\begin{cases} y_1' = y_2 \\ y_2' = \frac{1}{5}\sqrt{1+y_1^2} \end{cases}$$

先编写微分方程组

```
function dy = eq1(x, y)
% 微分方程组
dy = zeros(2, 1);
dy(1) = y(2);
dy(2) = 1/5 * sqrt(1 + y(1)^2) / (1 - x);
```

再编写代码求解

```
x0 = 0; xf = 0.9999;
[x, y] = ode15s(@eq1, [x0, xf], [0, 0]);
hold on
plot(x, y(:,1), 'b')
y = 0:0.01:0.2;
```

```

plot(1, y, 'ko')
axis([0, 1.1, 0, 0.25]);
title('导弹追击问题的数值解法');
legend('导弹', '乙舰', 'Location', 'Best');
hold off % 结果如图 11.8

```

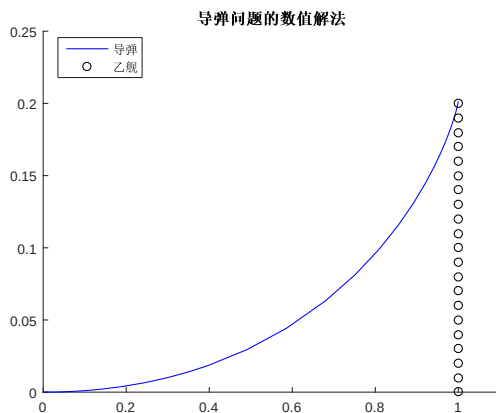


图 11.8: 导弹追击问题的数值解法

参数方程求解法

设时刻 t 乙舰的坐标为 $(X(t), Y(t))$, 导弹的坐标为 $x(t), y(t)$, 导弹的速度恒为 w , 则有 $(\frac{dx}{dt})^2 + (\frac{dy}{dt})^2 = w^2$ 。由于弹头始终对准乙舰, 故导弹的速度平行于乙舰与导弹头位置的差向量, 即

$$\begin{pmatrix} \frac{dx}{dt} \\ \frac{dy}{dt} \end{pmatrix} = \lambda \begin{pmatrix} X - x \\ Y - y \end{pmatrix}, \quad \lambda > 0$$

消去 λ 得

$$\begin{cases} \frac{dx}{dt} = \frac{w}{\sqrt{(X-x)^2 + (Y-y)^2}}(X-x) \\ \frac{dy}{dt} = \frac{w}{\sqrt{(X-x)^2 + (Y-y)^2}}(Y-y) \end{cases}$$

因乙舰以速度 v_0 沿直线 $x=1$ 运动, 设 $v_0=1$, 则 $w=5$, $X=1$, $Y=t$, 于是方程就转化为

$$\begin{cases} \frac{dx}{dt} = \frac{5}{\sqrt{(1-x)^2 + (t-y)^2}}(1-x) \\ \frac{dy}{dt} = \frac{5}{\sqrt{(1-x)^2 + (t-y)^2}}(t-y) \\ x(0) = y(0) = 0 \end{cases}$$

先编写微分方程组

```
function dy = eq2(t, y)
dy = zeros(2, 1);
dy(1) = 5 * (1 - y(1)) / sqrt((1 - y(1))^2 + (t - y(2))^2);
dy(2) = 5 * (t - y(2)) / sqrt((1 - y(1))^2 + (t - y(2))^2);
```

```
tf = 2;
[t, y] = ode45(@eq2, [0, tf], [0, 0]);
hold on
plot(y(:,1), y(:,2), 'b-')
Y = 0:0.01:tf;
plot(1, Y, 'ko')
plot(y(:,1), y(:,2), 'b-')
title('参数解法');
axis([0, 1.1, 0, 0.25]);
legend('导弹', '乙舰', 'Location', 'Best')
hold off % 结果如图 11.9
```

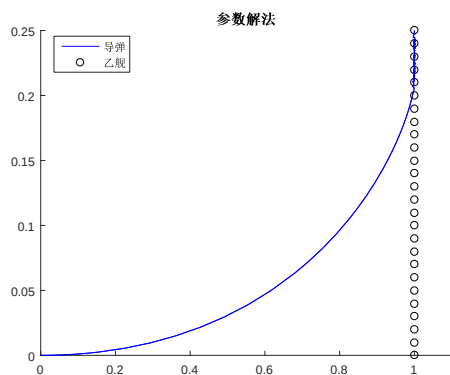


图 11.9: 导弹追击问题的参数解法

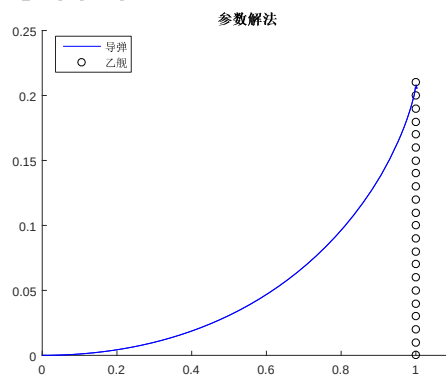


图 11.10: 导弹击中乙舰的轨迹

导弹大致在(1,0.2)处击中乙舰，与前面的结论一致。可按二分法逐步修改 tf ，取 $tf = 1, 0.5, 0.25, \dots$ ，直到 $tf = 0.21$ 时，得到图 11.10。故在 $t \approx 0.21$ 时乙舰被击中。

11.3.3 结论

通过以上分析我们知道，击中时间 $t = \frac{y}{v_0} = \frac{5}{24v_0}$ 。若 $v_0 = 1$ ，则乙舰在 $t \approx 0.21$ 时被击中。

11.3.4 动画模拟

题目要求

一只猫咪凭着敏锐的视觉发现其正东方向 c m处有一只老鼠，该老鼠正沿着墙根以 b m/s的速度向

正北方向奔跑, 猫咪立即以最大速度 a m/s前往追捕。在猫咪追捕老鼠的过程中, 猫咪的速度方向始终保持指向老鼠。试建立它的微分方程模型, 然后求解此模型, 并用动画模拟猫追老鼠的全过程。

建立模型

以猫的初始位置为原点, 以猫和老鼠的初值位置的连线为 x 轴建立平面直角坐标系。设任意 t 时刻猫所在位置的坐标为 (x, y) , 此时老鼠所在位置的坐标为 (c, bt) 。由于猫前进的速度方向始终保持指向老鼠, 设 t 时刻猫的前进方向与 x 轴的夹角为 θ 。于是有

$$\begin{cases} \frac{dx}{dt} = a \cos \theta = \frac{a(c-x)}{\sqrt{(c-x)^2 + (bt-y)^2}} \\ \frac{dy}{dt} = a \sin \theta = \frac{a(bt-y)}{\sqrt{(c-x)^2 + (bt-y)^2}} \end{cases}$$

动画模拟

```
function cat_mouse
% 猫追耗子的动画演示。墙垂直于猫和耗子连线。猫与耗子相距15（单位不详），
% 耗子以速度20，沿墙根往前跑，猫以速度40 正对着耗子追赶。
mov = VideoWriter('cat_mouse.avi'); % 视频名称
mov.FrameRate = 30; open(mov);
ts = 0:0.005:0.5; % ts取点
x0 = [0, 0]; % 设x0初值
[t, x] = ode45(@cat_m, ts, x0); % ode45解方程
n = length(x);
figure(1) % 新建窗口
plot(0, 0, 'r.') % 作点
hold on
h1 = line('Color', [0, 0, 1], 'Marker', 'r.', 'MarkerSize', 40); % 设置颜色等属性，下同
h2 = line(15, 0, 'MarkerFaceColor', [0, 1, 0], 'Marker', 'h', 'MarkerSize', 15);
plot(15, 0, 'r.') % 作点
line([15, 15], [0, 11], 'LineWidth', 2) % 画线，并设置线宽
axis equal; axis([0, 16, 0, 11])
title('猫追耗子动画演示') % 以下为动画部分
for i = 1:n-1
    plot(x([i, i+1], 1), x([i, i+1], 2)) % 画点部分
    set(h1, 'xdata', x(i+1, 1), 'ydata', x(i+1, 2)); % 数据存储
    set(h2, 'xdata', 15, 'ydata', 20 * t(i+1)); % 数据存储
    drawnow; % 刷新屏幕
    pause(0.005) % 控制时间间隔
    f = getframe(gcf); % 获取当前图片
    writeVideo(mov, f); % 添加到mov
end
hold off
close(mov); % 关闭mov

% 子函数，微分方程组
function dx = cat_m(t, x)
a = 20; b = 40; c = 15;
s = sqrt((c - x(1))^2 + (a * t - x(2))^2);
dx = [b * (c - x(1)) / s; b * (a * t - x(2)) / s];
```

模拟结果

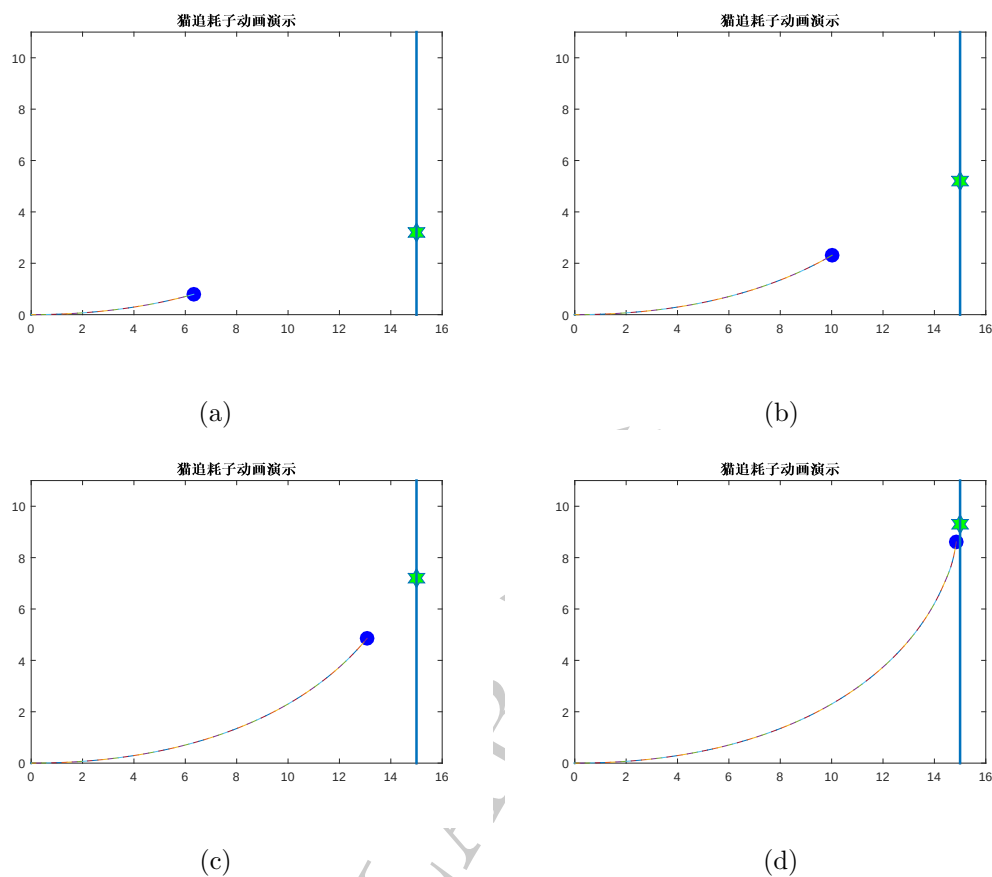


图 11.11: 猫追耗子的动画演示

11.4 实验报告

1. 分别用Euler法和ode45解下列常微分方程并与解析解比较。

(1) $y' = x + y$, $y(0) = 1$, $1 < x < 3$

(2) $y'' - 0.01(y')^2 + 2y = \sin(t)$, $y(0) = 0$, $y'(0) = 1$, $0 < t < 5$

解 (1)

```
odefun = inline('x_+y', 'x', 'y');
[x1, y1] = euler(odefun, [0, 3], 1, 0.1);
[x2, y2] = ode45(odefun, [0, 3], 1);
s = dsolve('Dy_=_x_+y', 'y(0)=1', 'x');
str = char(s); % 转化为字符串表达式
```

```

fun = inline(str);
hold on
plot(x1, y1, 'o', x2, y2, '+');
fplot(fun, [0, 3]);
legend('Euler法', 'ode45法', '解析解', 'location', 'best');
hold off          % 结果如图 11.12

```

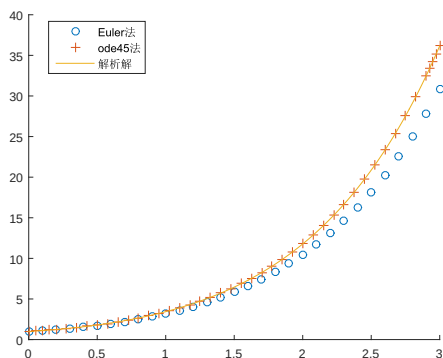


图 11.12: Euler法和ode45解常微分方程的比较

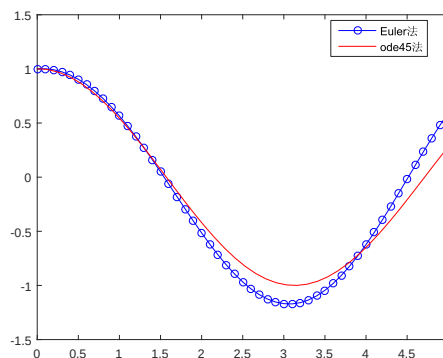


图 11.13: Euler法和ode45解常微分方程组的比较

先求解析解

```

>> s = dsolve('D2y-(0.01*y*Dy)^2+2*y=sin(t)', 'y(0)=0', 'Dy(0)=1', 't')
s =
[ empty sym ]

```

编写解常微分方程组的Euler法程序

```

function [x, y] = eulers(odefun, tspan, y0, h)
% Euler法求解常微分方程组
% Input -odefun 常微分方程 -tspan 求解区间 -y0 初值 -h 步长
% Output -x 自变量 -y 函数值
x = tspan(1):h:tspan(2);
n = length(x);
y = ones(length(y0), n);
y(:,1) = y0;
for i = 1:n-1
    y(:,i+1) = y(:,i) + h * feval(odefun, x(i), y(:,i));
end
x = x(:);
y = y';

```

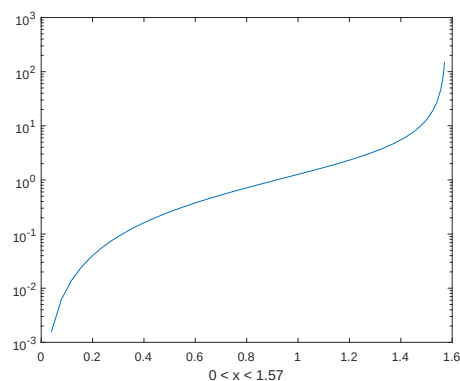
求解

```

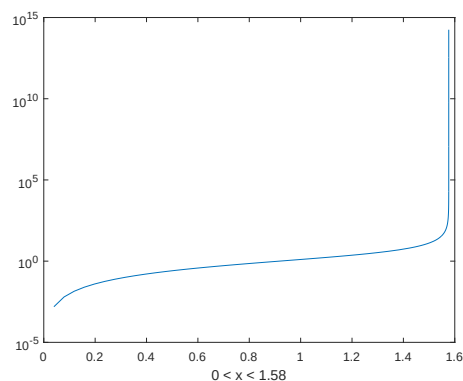
odefun = @(t, y)[y(2); 0.01 * y(2) ^ 2 - 2 * y(1) + sin(t)];
[x1, y1] = eulers(odefun, [0, 5], [0; 1], 0.1);
[x2, y2] = ode45(odefun, [0, 5], [0; 1]);
plot(x1, y1(:,2), 'bo-', x2, y2(:,2), 'r-');
legend('Euler法', 'ode45法', 'location', 'best'); % 结果如图 11.13

```

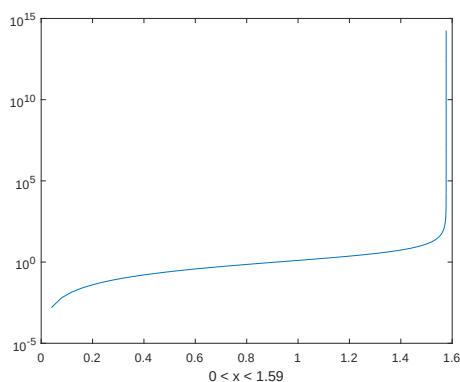
2. 求一通过原点的曲线，它在 (x, y) 处的切线斜率等于 $2x + y^2$, $0 < x < 1.57$ 。若 x 上限增为1.58, 1.60会发生什么？



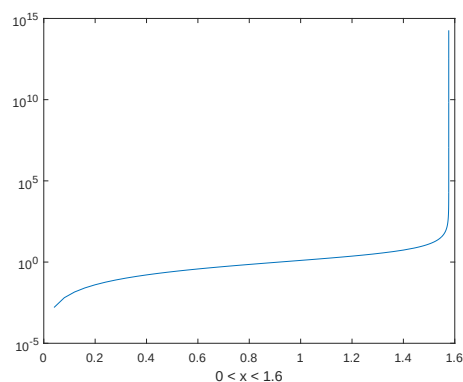
(a)



(b)



(c)



(d)

图 11.14: 解的爆炸

解

```

odefun = inline('2*x+u^2', 'x', 'y');
w = 1.57:0.01:1.60;
for i = 1:4
    [x,y] = ode45(odefun, [0, w(i)], 0);
    figure(i)

```

```

semilogy(x, y)      % 绘制y轴上对数图像
xlabel(['0<math>x</math>', num2str(w(i))]);
end                  % 结果如图 11.14

```

3. 求解方程组

$$\begin{cases} y_1' = -1000.25y_1 + 999.75y_2 + 0.5 \\ y_2' = 999.75y_1 - 1000.25y_2 + 0.5 \\ y_1(0) = 1, y_2(0) = -1, x \in [0, 10] \end{cases}$$

解 先构造微分方程

```

function f = eq3(x, y)
f(1) = -1000.25 * y(1) + 999.75 * y(2) + 0.5;
f(2) = 999.75 * y(1) - 1000.25 * y(2) + 0.5;
f = f(:);

```

求解方程组

```

[x, y] = ode23s(@eq3, [0, 10], [1, -1]);
semilogx(x, y)      % 绘制x轴上对数图像
str1 = '$y_1$'; str2 = '$y_2$';
h = legend(str1, str2, 'location', 'best');
set(h, 'interp', 'latex')
text(x(2), y(2,1) - 0.1, str1, 'interp', 'latex')
text(x(2), y(2,2) + 0.1, str2, 'interp', 'latex') % 结果如图 11.15

```

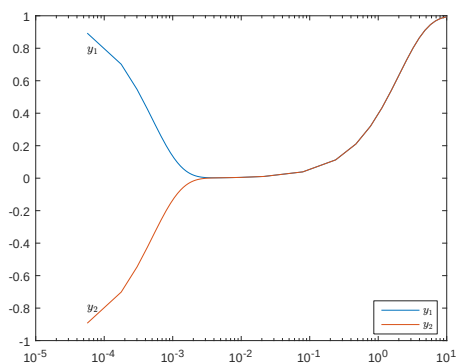


图 11.15: 刚性方程组的求解

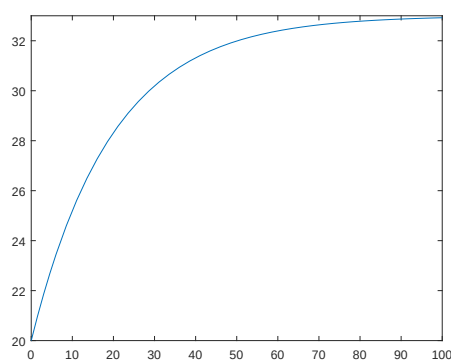


图 11.16: 温度变化过程

4. (温度过程)夏天把开有空调的室内一支读数为 20°C 的温度计放到户外, 10分钟后读 25.2°C , 再过10分钟后读数 28.32°C 。试建立合理的模型来推算户外温度。

解 建立模型

$$T' = k(c - T), T(0) = 20$$

用最小二乘法求常数 k, c

```
>> s = dsolve('DT_ = k*(c - T)', 'T(0) = 20', 't')
s =
c - exp(-k*t)*(c - 20)
>> fun = inline('c(1) + exp(-c(2)*t)*(-c(1) + 20)', 'c', 't');
>> c = lsqcurvefit(fun, [30, 1], [10, 20], [25.2, 28.32])
c =
    33.0000    0.0511
```

故模型为

$$T(t) = -13e^{-0.0511t} + 33.$$

绘图判断趋势

```
odefun = inline('0.0511*(33 - T)', 't', 'T');
[t, T] = ode45(odefun, [0, 100], 20);
plot(t, T)
axis([0, 100, 20, 33]) % 结果如图 11.16
```

由图 11.16 可知, 室外温度接近 33°C 。

5. (广告效应)某公司生产一种耐用消费品, 市场占有率为5%时开始做广告, 一段时间的市场跟踪调查后, 该公司发现: 单位时间内购买人口百分比的相对增长率与当时还没有买的百分比成正比, 且估得此比例系数为0.5。

- (1) 建立该问题的数学模型, 并求其数值解与模拟结果作以比较;
- (2) 厂家问: 要做多少时间广告, 可使市场购买率达到80%?

解 (1) 建立模型

$$\frac{x'}{x} = 0.5(1 - x), x(0) = 0.05.$$

编程求解

```
>> s = dsolve('Dx_ / x_ = 0.5*(1 - x)', 'x(0) = 0.05')
s =
1/(exp(log(19) - t/2) + 1)
```

故问题的解为

$$x(t) = \frac{1}{e^{\ln 19 - t/2} + 1} = \frac{1}{19e^{-t/2} + 1}.$$

绘图

```

odefun = inline('0.5*(1-x)*x', 't', 'x');
[t, x] = ode45(odefun, [0, 10], 0.05);
hold on
plot(t, x, 'ko-')
fplot(inline(s), [0, 10]) % s为求得精确解
axis([0, 10, 0, 1])
legend('数值解', '解析解', 'location', 'best');
hold off % 结果如图 11.17

```

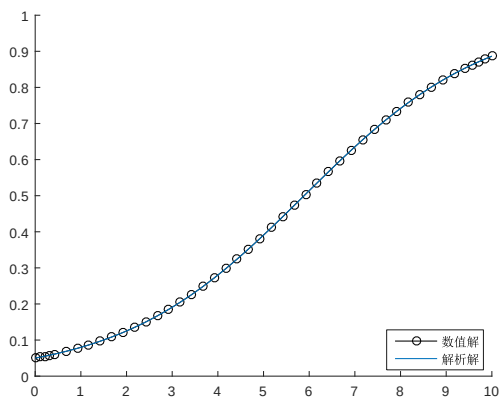


图 11.17: 广告效应

(2) 在第一问的基础上求值

```

>> tim = t(x > 0.8);
>> time = tim(1)
time =
    8.6731

```

或者

```

>> time = t(find(x > 0.8, 1))
time =
    8.6731

```

故做广告的时间至少为8.6731, 可使市场购买率达到80%。

6. (肿瘤生长) 肿瘤大小 V 生长的速率与 V 的 a 次方成正比, 其中 a 为形状参数, $0 \leq a \leq 1$; 而其比例系数 K 随时间减小, 减小速率又与当时的 K 值成正比, 比例系数为环境参数 b 。设某肿瘤参数 $a = 1$, $b = 0.1$, K 的初始值为 2, V 的初始值为 1。问

(1) 此肿瘤生长不会超过多大?

(2) 过多长时间肿瘤大小翻一倍?

(3) 何时肿瘤生长速率由递增转为递减?

解 (1)

```
>> s1 = dsolve('Dk=-0.1*k', 'k(0)=2', 't')
s1 =
2*exp(-t/10)
>> s2 = dsolve('Dv=2/(exp(t/10)*v^1)', 'v(0)=1', 't')
s2 =
exp(20)*exp(-20*exp(-t/10))
>> slim = limit(s2, 't', inf)
slim =
exp(20)
```

故肿瘤生长不会超过 e^{20} 。

(2)

```
>> t = 0:0.01:100;
>> y = subs(s2, 't', t);
>> plot(t, y) % 结果如图 11.18
>> time = t(find(y <= 2, 1, 'last'))
time =
    0.3500
```

故肿瘤大小翻一倍的时间为0.35。

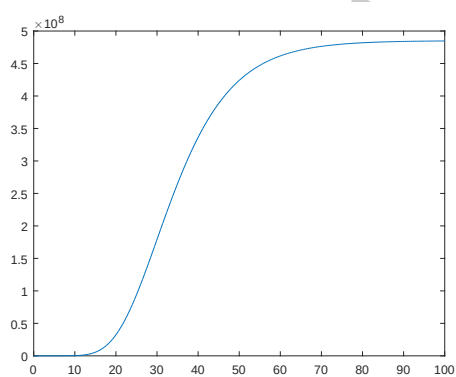


图 11.18: 肿瘤生长速率曲线

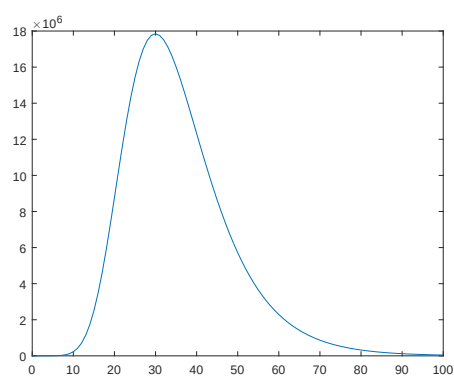


图 11.19: 肿瘤生长速率变化曲线

(3)

```

>> tzh = diff(s2, 't');
>> t = 0:50;
>> y = subs(tzh, 't', t);
>> plot(t, y) % 绘制生长速率变化图 11.19
>> tzh = diff(tzh);
>> fun = inline(tzh);
>> x = fzero(fun, [0, 50]) % 求根
x =
    29.9573

```

故肿瘤生长速率由递增转为递减的时间为29.96。

注 11.1 若参数 $a = 2/3$ ，以上题目的答案如何呢？请读者完成。

7. (生态系统的振荡现象)第一次世界大战中，因为战争很少捕鱼，按理战后应能捕到更多的鱼才是。可是大战后，在地中海却捕不到鲨鱼，因而渔民大惑不解。

令 x_1 为鱼饵的数量， x_2 为鲨鱼的数量， t 为时间。微分方程为

$$\begin{cases} \frac{dx_1}{dt} = x_1(a_1 - b_1x_2) \\ \frac{dx_2}{dt} = -x_2(a_2 - b_2x_1) \end{cases}$$

式中 a_1, a_2, b_1, b_2 都是正常数。第一式鱼饵 x_1 的增长速度大体上与 x_1 成正比，即按 a_1x_1 比率增加，而被鲨鱼吃掉的部分按 $b_2x_1x_2$ 的比率减少；第二式中鲨鱼的增长速度由于生存竞争的自然死亡或互相咬食按 a_2x_2 的比率减少，但又根据鱼饵的量的变化按 $b_2x_1x_2$ 的比率增加。对 $a_1 = 3, b_1 = 2, a_2 = 2.5, b_2 = 1, x_1(0) = x_2(0) = 1$ 求解。试画出解曲线图和相轨线图，观察周期振荡现象。

解 构造微分方程

```

function f = eq4(t, x)
a1 = 3; b1 = 2; a2 = 2.5; b2 = 1;
f(1) = x(1) * (a1 - b1 * x(2));
f(2) = -x(2) * (a2 - b2 * x(1));
f = f(:);

```

求解绘图

```

[t, x] = ode15s(@eq4, [0, 10], [1, 1]);
figure(1)
plot(t, x(:,1), 'r-', t, x(:,2), 'b--');
legend('鱼饵', '鲨鱼', 'location', 'best'); % 结果如图 11.20
figure(2)
plot(x(:,1), x(:,2)); % 结果如图 11.21

```

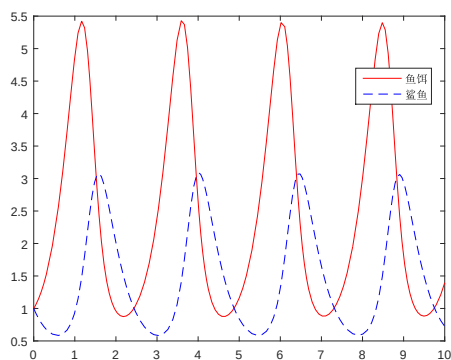


图 11.20: 生态系统的震荡现象

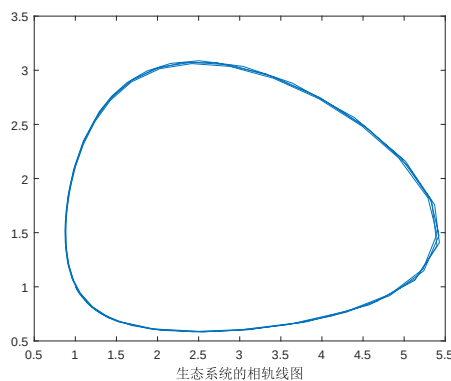


图 11.21: 生态系统的相轨线图

8. 编写四阶Runge-Kutta法程序并求解 $y' = x + y$, $y(0) = 1$, $1 < x < 3$, 其中Runge-Kutta法为

$$\begin{cases} y_{n+1} = y_n + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4) \\ K_1 = f(t_n, y_n) \\ K_2 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}K_1) \\ K_3 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}K_2) \\ K_4 = f(t_n + h, y_n + hK_3) \end{cases}$$

```
function [x, y] = Runge_Kutta(fun, xspan, y0, h)
% 四阶龙格库塔法求解微分方程
% Input -fun 微分方程 -xspan 求解区间 -y0 初值 -h 步长
% Output -x 自变量 -y 自变量对应的函数值
m = floor(abs(xspan(2) - xspan(1)) / h);
x = zeros(1, m);
y = x;
x(1) = min(xspan);
y(1) = y0;
for n = 1:m-1
    x(n+1) = x(n) + h;
    k1 = feval(fun, x(n), y(n));
    k2 = feval(fun, x(n) + h/2, y(n) + h/2 * k1);
    k3 = feval(fun, x(n) + h/2, y(n) + h/2 * k2);
    k4 = feval(fun, x(n+1), y(n) + h * k3);
    y(n+1) = y(n) + h/6 * (k1 + 2*k2 + 2*k3 + k4);
end
```

求解方程

```
fun = inline('x+y', 'x', 'y');
[x,y] = Runge_Kutta(fun, [0, 3], 1, 0.1);
plot(x, y, '-o')
```

```
s = dsolve('Dy=x+y', 'y(0)=1', 'x');  
hold on  
ezplot(s, [0, 3])  
legend('Runge-Kutta法', '解析解', 'Location','Best');  
hold off % 结果如图 11.22
```

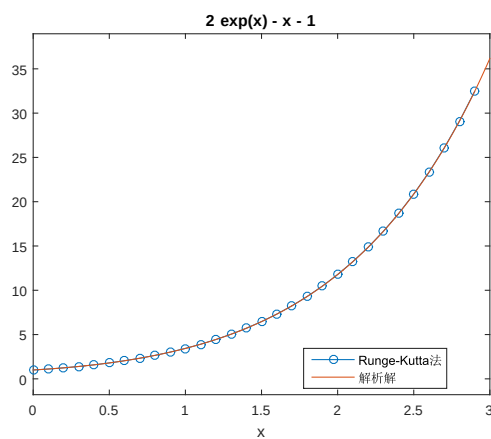


图 11.22: Runge-Kutta法求解方程

第十二章 偏微分方程数值解法

在这一章我们简单介绍偏微分方程(PDE)的数值解法。MATLAB中PDE工具箱可以帮助我们求解一系列的偏微分方程，有兴趣的读者可以使用pdetool命令按照其中的帮助文档进行操作，本章主要介绍求解方法，即有限差分法。

由于我们使用的计算机只能存储有限个数据且只能做有限次运算，因此任何一种计算机解题方法都必须把连续问题(如微分方程的边值问题、初值问题等)离散化，最终化成有限形式的线性方程组。用差分法和有限元方法将连续问题离散化的步骤是，首先对求解区域做网格剖分，用有限个节点代替连续区域，然后将微分算子离散化，从而将微分方程的定解问题化为线性代数方程组的求解问题。差分法和有限元法的主要区别在于第二步，前者从定解问题的微分形式或积分形式出发，用数值微商或数值积分公式导出相应的线性代数方程组。后者从定解问题的变分形式出发，导出相应的线性代数方程组(此时会用到泛函分析的内容)。

本章的每一节仅针对特定的方程进行数值求解，偏微分方程数值解法的详细介绍请参见李荣华、刘播编著的《微分方程数值解法(第四版)》。

12.1 导数近似与误差阶验证

在数值逼近和常微分方程数值解这两章中, 我们已经了解了一些导数近似的内容。本节以函数 $u = \sin x$ 为例, 介绍 $x = 1$ 处的一、二、三阶导数的近似并验证各自的误差阶数。

12.1.1 一阶导数的近似

x 处的一阶导数 $u'(x)$ 可以使用向前差分 $D_+u(x)$ 、中心差分 $D_0u(x)$ 和向后差分 $D_-u(x)$ 来近似, 其中

$$\begin{cases} D_+u(x) = \frac{u(x+h) - u(x)}{h} \\ D_0u(x) = \frac{u(x+h) - u(x-h)}{2h} \\ D_-u(x) = \frac{u(x) - u(x-h)}{h} \end{cases}$$

由导函数的定义知, 当 $h \rightarrow 0$ 时, $D_+u(x)$, $D_0u(x)$, $D_-u(x) \rightarrow u'(x)$ 。这三种差分的程序如下:

```
function ud = one_diff_appr(x, u, index)
% 计算(非)等距节点的u'(2)的一阶导数逼近
% Input -x 数据节点, 3维向量 -u x处的函数值, 与x的size一致
%       -index 差分格式 -1 向后差分 0 中心差分 1 向前差分
%       index = -1时, u'(2) = ( u(2) - u(1) ) / ( x(2) - x(1) )
%       index = 0时, u'(2) = ( u(3) - u(1) ) / ( x(3) - x(1) )
%       index = 1时, u'(2) = ( u(3) - u(2) ) / ( x(3) - x(2) )
% Output -ud 一阶导数逼近, 数值
index1 = 3; index2 = 1;
if (index == -1), index1 = 2; end
if (index == 1), index2 = 2; end
ud = ( u(index1) - u(index2) ) / ( x(index1) - x(index2) );
```

设步长为 h , x 处导数 $u'(x)$ 的差分近似为 $D_h(x)$, 此时误差记为 $R(h)$, 则有

$$R(h) = |D_h(x) - u'(x)| \approx Ch^r.$$

若步长变为 $\frac{h}{m}$, 误差 $R(\frac{h}{m}) = |D_{\frac{h}{m}}(x) - u'(x)| \approx C(\frac{h}{m})^r$ 。两项比较可知,

$$\left| \frac{R(h)}{R(\frac{h}{m})} \right| \approx m^r,$$

也就是误差阶数 $r \approx \log_m \left| \frac{R(h)}{R(\frac{h}{m})} \right|$ 。方便起见, m 常取2或10。

这里取 $m = 2$, 编写误差阶验证的程序如下:

```
function error_order_one(xo, index, n)
% 计算xo处的一阶导数的差分近似误差阶
% Input -xo 近似一阶导数的xo位置
%       -index 差分格式: -1 向后差分 0 中心差分 1 向前差分
```



```
%      -n 取n个步长
hmat   = 0.1 ./ (2 .^ (0:n-1)'); % h组成的矩阵, 第i次的步长为0.1/2^(i-1)
ud_ext  = cos(xo);           % 一阶导数的精确解
h_f     = -1:1;              % 步长h的系数
ud      = zeros(n, 1);       % ud一阶导数近似值
ud_error = zeros(n, 1);      % ud_error误差

for i = 1:n
    h = hmat(i);             % 步长h
    x = xo + h_f * h;        % 节点x
    u = sin(x);              % 节点x处的函数值u
    ud(i) = one_diff_appr(x, u, index); % 计算一阶导数近似值
    ud_error(i) = ud(i) - ud_ext;      % 计算误差
end

ud_err_degree = [NaN; log2(abs(ud_error(1:n-1) ./ ud_error(2:n)))]; % 阶数
% 以下命令用于输出结果
title_str = {'_步长 ', '近似解', '_误差 ', '误差阶'};
result = [title_str; num2cell(hmat), num2cell(ud), ...
    num2cell(ud_error), num2cell(ud_err_degree)];
result{2,4} = 'uuuu';
str = {'向后差分', '中心差分', '向前差分'}; % 差分格式的字符串
disp(['_uuuu=====_' , char(str(index + 2)), '_====='])
disp(result)
```

下面逐个对差分格式的误差阶进行验证:

```
>> error_order_one(1, -1, 6) % 向后差分
===== 向后差分 =====
'_步长 ', '近似解', '_误差 ', '误差阶',
[0.1000] [0.5814] [ 0.0411] 'uuuu',
[0.0500] [0.5611] [ 0.0208] [0.9834]
[0.0250] [0.5508] [ 0.0105] [0.9920]
[0.0125] [0.5455] [ 0.0052] [0.9961]
[0.0063] [0.5429] [ 0.0026] [0.9981]
[0.0031] [0.5416] [ 0.0013] [0.9990]
>> error_order_one(1, 0, 6) % 中心差分
===== 中心差分 =====
'_步长 ', '近似解', '_误差 ', '误差阶',
[0.1000] [0.5394] [-9.0005e-04] 'uuuu',
[0.0500] [0.5401] [-2.2510e-04] [1.9995]
[0.0250] [0.5402] [-5.6280e-05] [1.9999]
[0.0125] [0.5403] [-1.4070e-05] [2.0000]
[0.0063] [0.5403] [-3.5176e-06] [2.0000]
[0.0031] [0.5403] [-8.7940e-07] [2.0000]
>> error_order_one(1, 1, 6) % 向前差分
===== 向前差分 =====
'_步长 ', '近似解', '_误差 ', '误差阶',
[0.1000] [0.4974] [-0.0429] 'uuuu',
[0.0500] [0.5190] [-0.0213] [1.0143]
[0.0250] [0.5297] [-0.0106] [1.0074]
[0.0125] [0.5350] [-0.0053] [1.0038]
[0.0063] [0.5377] [-0.0026] [1.0019]
[0.0031] [0.5390] [-0.0013] [1.0010]
```

对一阶导数的近似, 从结果看, 向前差分的误差阶为1阶, 中心差分的误差阶为2阶, 向后差分的误差阶为1阶。

12.1.2 二阶导数的近似

x 处二阶导数 $u''(x)$ 可以使用 $D_+D_-u(x)$ 、 $D_-D_+u(x)$ 和中心差分 $\hat{D}_0\hat{D}_0u(x)$ 来近似, 其中 $\hat{D}_0u(x) = \frac{u(x + \frac{1}{2}h) - u(x - \frac{1}{2}h)}{h}$ 。二阶导数近似的程序如下:

```
function ud2 = two_diff_appr(x, u, index)
% u''(3)的二阶导数逼近
% Input -x 数据节点, 1×5的向量 -u x处的函数值,与x的size一致
%        -index 差分格式 -1 D-D+差分 0 DOD0差分 1 D+D-差分
% Output -ud2 二阶导数逼近, 数值
ud = zeros(1,3);
ud_index = [1, 3, 2];
in = 0:2;
for i = 1:3
    x_index = in + ud_index(i);
    xx = x(x_index); uu = u(x_index);
    ud(ud_index(i)) = one_diff_appr(xx, uu, -index); % 计算一阶导数
end
ud2 = one_diff_appr(xx, ud, index); % 计算二阶导数
```

接下来编写计算二阶导数并验证误差阶的程序。

```
function error_order_two(xo, index, n)
% 计算xo处的二阶导数的差分近似误差阶
% Input -xo 近似二阶导数的xo位置
%        -index 差分格式: -1 D-D+差分 0 DOD0差分 1 D+D-差分
%        -n 取n个步长
hmat = 0.1 ./ (2.^(0:n-1));
ud2_ext = -sin(xo);
h_f = -2:2;
if (index == 0), h_f = h_f/2; end
ud2 = zeros(n,1);
ud2_error = zeros(n,1);
for i = 1:n
    h = hmat(i); x = xo + h_f * h;
    u = sin(x);
    ud2(i) = two_diff_appr(x, u, index);
    ud2_error(i) = ud2(i) - ud2_ext;
end

ud2_err_degree = [nan; log2(abs(ud2_error(1:n-1) ./ ud2_error(2:n)))];
title_str = {'步长', '近似解', 'uuu误差', '误差阶'};
result = [title_str; num2cell(hmat), num2cell(ud2), ...
    num2cell(ud2_error), num2cell(ud2_err_degree)];
result{2,4} = 'uuuu';
str = {'D-D+差分', 'DOD0差分', 'D+D-差分'};
disp(['uuuu=====', char(str(index + 2)), '====='])
disp(result)
```

下面验证误差阶：

```
>> error_order_two(1, -1, 5)
===== D-D+差分 =====
'步长'      '近似解'      'uuu误差'      '误差阶'
[0.1000]    [-0.8408]    [7.0099e-04]    'uuuu'
[0.0500]    [-0.8413]    [1.7529e-04]    [1.9996]
[0.0250]    [-0.8414]    [4.3826e-05]    [1.9999]
[0.0125]    [-0.8415]    [1.0957e-05]    [2.0000]
[0.0063]    [-0.8415]    [2.7392e-06]    [2.0000]

>> error_order_two(1, 0, 5)
===== DODD差分 =====
'步长'      '近似解'      'uuu误差'      '误差阶'
[0.1000]    [-0.8408]    [7.0099e-04]    'uuuu'
[0.0500]    [-0.8413]    [1.7529e-04]    [1.9996]
[0.0250]    [-0.8414]    [4.3826e-05]    [1.9999]
[0.0125]    [-0.8415]    [1.0957e-05]    [2.0000]
[0.0063]    [-0.8415]    [2.7392e-06]    [2.0000]

>> error_order_two(1, 1, 5)
===== D+D-差分 =====
'步长'      '近似解'      'uuu误差'      '误差阶'
[0.1000]    [-0.8408]    [7.0099e-04]    'uuuu'
[0.0500]    [-0.8413]    [1.7529e-04]    [1.9996]
[0.0250]    [-0.8414]    [4.3826e-05]    [1.9999]
[0.0125]    [-0.8415]    [1.0957e-05]    [2.0000]
[0.0063]    [-0.8415]    [2.7392e-06]    [2.0000]
```

上述三种方式误差阶是一致的，均为2阶。

12.1.3 三阶导数的近似

类似地，三阶导数的近似可以使用 $D_-D_+D_-u(x)$ 、 $D_0D_+D_-u(x)$ 和 $D_+D_+D_-u(x)$ 。编写近似函数如下：

```
function ud3 = three_diff_appr(x, u, index)
% 计算u'''(3)的三阶导数逼近
% Input -x 数据节点, 1×5的向量 -u x处的函数值,与x的size一致
%       -index 差分格式 -1 D-D+D-差分 0 DOD+D-差分 1 D+D+D-差分
% Output -ud3 三阶导数逼近, 数值
ud2 = zeros(1,3);
ud2_index = [1, 3, 2];
in = 0:4;
for i = 1:3
    x_index = in + ud2_index(i);
    xx = x(x_index); uu = u(x_index);
    ud2(ud2_index(i)) = two_diff_appr(xx, uu, 1);
end
ud3 = one_diff_appr(xx(2:4), ud2, index);
```

接着编写计算三阶导数并验证误差阶的程序：

```

function error_order_three(xo, index, n)
% 计算xo处的三阶导数的差分近似误差阶
% Input -xo 近似三阶导数的xo位置
%        -index 差分格式: -1 向后差分 0 中心差分 1 向前差分
%        -n 取n个步长
hmat = 0.1 ./ (2 .^ (0:n-1));
ud3_ext = - cos(xo);
h_f = -3:3;
if (index == 0), h_f = h_f/2; end
ud3 = zeros(n,1);
ud3_error = zeros(n,1);
for i = 1:n
    h = hmat(i);
    x = xo + h_f * h;
    u = sin(x);
    ud3(i) = three_diff_appr(x, u, index);
    ud3_error(i) = ud3(i) - ud3_ext;
end

ud3_err_degree = [nan; log2(abs(ud3_error(1:n-1) ./ ud3_error(2:n)))];

title_str = {'步长 ', '近似解', 'uuu误差 ', '误差阶'};
result = [title_str; num2cell(hmat), num2cell(ud3), ...
    num2cell(ud3_error), num2cell(ud3_err_degree)];
result{2,4} = 'uuuu';
str = {'D-D+D-差分', 'DOD+D-差分', 'D+D+D-差分'};

disp(['uuuu=====u', char(str(index + 2)), 'u====='])
disp(result)

```

验证误差阶

```

>> error_order_three(1, -1, 4)
===== D-D+D-差分 =====
'步长 '      '近似解'      'uuu误差'      '误差阶'
[0.1000]      [-0.5810]      [-0.0407]      'uuuu'
[0.0500]      [-0.5610]      [-0.0207]      [0.9744]
[0.0250]      [-0.5507]      [-0.0104]      [0.9878]
[0.0125]      [-0.5455]      [-0.0052]      [0.9941]

>> error_order_three(1, 0, 4)
===== DOD+D-差分 =====
'步长 '      '近似解'      'uuu误差'      '误差阶'
[0.1000]      [-0.5400]      [3.3760e-04]      'uuuu'
[0.0500]      [-0.5402]      [8.4417e-05]      [1.9997]
[0.0250]      [-0.5403]      [2.1105e-05]      [1.9999]
[0.0125]      [-0.5403]      [5.2763e-06]      [2.0000]

>> error_order_three(1, 1, 4)
===== D+D+D-差分 =====
'步长 '      '近似解'      'uuu误差'      '误差阶'
[0.1000]      [-0.4969]      [0.0434]      'uuuu'
[0.0500]      [-0.5189]      [0.0214]      [1.0208]
[0.0250]      [-0.5297]      [0.0106]      [1.0110]
[0.0125]      [-0.5350]      [0.0053]      [1.0056]

```

$D_-D_+D_-u(x)$ 和 $D_+D_+D_-u(x)$ 的误差阶为1阶, $D_0D_+D_-u(x)$ 的误差阶为2阶。

12.2 差分方程求解与收敛阶验证

编写程序求解方程

$$\begin{cases} -u'' + u = f, & 0 \leq x \leq \pi, \\ u(0) = u(\pi) = 0, \end{cases}$$

已知方程的解为 $u(x) = e^x \sin x$, 先计算右端项函数 f , 再用差分方程近似求解并与精确解比较。试在不同范数下, 验证差分方程的收敛阶。

这里记步长为 h , 误差为 $e = \{e_i\}_{i=0}^{M+1}$, 其常用的范数有: (1) $\|e\| = \sqrt{(e, e)} = \sqrt{\sum_{i=1}^M e_i^2 h}$; (2) $\|e\| = \sqrt{(e, e)} = \sqrt{\sum_{i=1}^{M+1} e_i^2 h}$; (3) $\|e\|_\infty = \max_{1 \leq i \leq M} |e_i|$ 。针对这三种范数的计算, 编写函数如下:

```
function y = grid_norm(u, h, index)
% 计算网格范数
% Input -u 网格节点处的(误差)值, J维向量 -h 对应的步长, 数值
%       -index 范数类型,
%       1为||u|| = sqrt(u, u) = sqrt(sum_{i=2}^{J} (u_i^2 * h))
%       2为||u|| = sqrt(u, u) = sqrt(sum_{i=2}^{J+1} (u_i^2 * h))
%       3为||u||_inf = max_{i=2:J} |u_i|
% Output -y 网格范数
if index == 1, y = sqrt(sum(u(2:end-1) .^ 2 .* h)); end
if index == 2, y = sqrt(sum(u(2:end) .^ 2 .* h)); end
if index == 3, y = norm(u(2:end-1), inf); end
```

解 计算可知方程右端项为 $f = e^x \sin x - 2e^x \cos x$ 。

(1) 区域剖分

在区间 $[0, \pi]$ 上等距地取 $M+2$ 个点, 此时步长 $h = \frac{\pi}{M+1}$, 且 $x_i = ih, i = 0, 1, \dots, M+1$ 。

(2) 差分方程逼近

记 $f_i = f(x_i), i = 0, 1, \dots, M+1$ 。利用二阶中心差分逼近 u'' , 于是可得差分方程为

$$\begin{cases} -\frac{U_0 - 2U_1 + U_2}{h^2} + U_1 = f_1 \\ -\frac{U_1 - 2U_2 + U_3}{h^2} + U_2 = f_2 \\ \dots \\ -\frac{U_{M-1} - 2U_M + U_{M+1}}{h^2} + U_M = f_M \end{cases} \Rightarrow \begin{cases} -U_0 + (2 + h^2)U_1 - U_2 = h^2 f_1 \\ -U_1 + (2 + h^2)U_2 - U_3 = h^2 f_2 \\ \dots \\ -U_{M-1} + (2 + h^2)U_M - U_{M+1} = h^2 f_M \end{cases}$$

写成矩阵形式为 $AU = h^2 F$, 其中

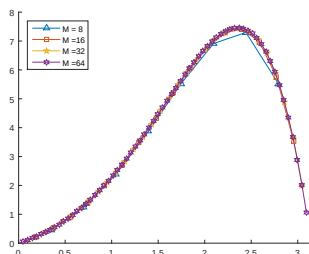
$$A = \begin{pmatrix} 2+h^2 & -1 & & & \\ -1 & 2+h^2 & -1 & & \\ & -1 & 2+h^2 & -1 & \\ & & \ddots & \ddots & \ddots \\ & & & -1 & 2+h^2 \end{pmatrix}_{M \times M}, U = \begin{pmatrix} U_1 \\ U_2 \\ U_3 \\ \vdots \\ U_M \end{pmatrix}, F = \begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ \vdots \\ f_M \end{pmatrix}.$$

求解该方程组的方法有很多, 本书第八章提及的方法都是可行的。考虑到系数矩阵的特殊性(系数矩阵为对称的三对角矩阵, 且同一对角线上元素均相同), 这里采用追赶法(详见第八章)来求解。

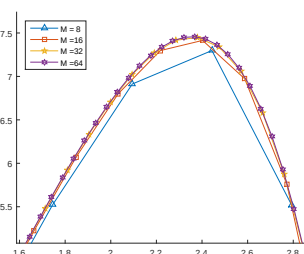
(3) 方程求解与收敛阶的验证

```
mk = {'-^', '-s', '-p', '-h'}; % 标记类型
index = 1:3; % 范数形式索引
x_0 = 0; x_end = pi; % x的范围[0, pi]
m_mat = 2 .^ (3:6); % 内点个数
n = length(m_mat); % n为m_mat的长度
e_norm = zeros(n, 3); % e_norm为误差的范数
order = zeros(n-1,3); % 收敛阶
h = zeros(1, n); % 对应于内点个数的步长
a = -1; c = -1; % 矩阵A的两条次对角线值
for i = 1:length(m_mat) % 取不同的内点个数计算
    M = m_mat(i); % 取M个内点
    x_all = linspace(x_0, x_end, M + 2); % [0, pi]上取M+2个点
    h(i) = (x_end - x_0) / (M + 1); % 步长
    b = (2 + h(i)^2); % 矩阵A的主对角线元素
    x_in = x_all(2:end-1); % 取内点
    u_acc = exp(x_in) .* sin(x_in); % 计算内点处精确解数值
    f_n = exp(x_in) .* (sin(x_in) - 2 * cos(x_in)); % 计算内点处右端项f数值
    u_c = thomas_algorithm(a, b, c, f_n * h(i)^2); % 追赶法求方程组
    u_error = u_acc - u_c; % 求误差
    hold on; plot(x_in, u_c, char(mk(i))) % 解的图像, 若绘制误差图可用plot(x_in, u_error)
    for j = 1:3
        e_norm(i,j) = grid_norm([0, u_error, 0], h(i), index(j));
    end
end
xlim([0, pi])
legend(strcat('M= ', num2str(m_mat')), 'location', 'NorthWest')
for j = 1:3
    order(:,j) = log(abs(e_norm(1:end-1,j) ./ e_norm(2:end,j))) ./ ...
        log(abs(h(1:end-1) ./ h(2:end))); % 计算收敛阶
end
order % 输出收敛阶
```

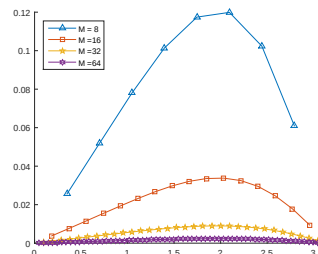
计算结果如下:



(a) 近似解图像



(b) 局部放大图



(c) 解的误差图

图 12.1: 解的图像

```
order =
    2.0018    2.0018    1.9914
    2.0005    2.0005    1.9978
    2.0001    2.0001    1.9995
```

因此, 这里采用的差分方程在三种不同的范数下均是2阶的。

12.3 一维扩散方程的求解

编写程序用Euler法、向后Euler法和C-N格式求解方程

$$\begin{cases} u_t - \kappa u_{xx} = f(x, t), & 0 < x < 1, \quad 0 < t < T \\ u(0, t) = g_1(t), \quad u(1, t) = g_2(t), \\ u(x, 0) = \eta(x), \end{cases}$$

其中 $\kappa = 1$, $T = 0.5$, $f(x, t) = 0$, $g_1(t) = g_2(t) = 0$, $\eta(x) = \sin(\pi x)$ 。已知精确解为 $u(x, t) = e^{-\pi^2 t} \sin(\pi x)$, 试比较三种数值格式的差异。

解 在 x 和 t 方向上均匀地取 $M_x + 2$ 和 $M_t + 1$ 个点, 相应的步长记为 h 和 k , 网比 $r = \frac{k\kappa}{h^2}$, 在 (x_i, t_n) 处的近似解记为 $U_i^n, i = 0, 1, \dots, M_x + 1, n = 0, 1, \dots, M_t$ 。

在 (x_i, t_n) 处三种格式可写为

$$\frac{U_i^{n+1} - U_i^n}{k} - \kappa \left[\theta \frac{U_{i+1}^{n+1} - 2U_i^{n+1} + U_{i-1}^{n+1}}{h^2} + (1 - \theta) \frac{U_{i+1}^n - 2U_i^n + U_{i-1}^n}{h^2} \right] = f_i^{n+\theta},$$

其中 $i = 1, 2, \dots, M_x$ 。当 $\theta = 0$ 时为Euler法, 当 $\theta = 1$ 时为向后Euler法, 当 $\theta = 0.5$ 时为C-N格式。该式写成矩阵形式为

$$AU^{n+1} = BU^n + F, \quad n = 0, 1, \dots, M_t,$$

其中

$$A = I - r\theta M, \quad B = I + r(1 - \theta)M, \quad F = kf^{n+\theta} + \theta G_1 + (1 - \theta)G_2,$$

$$M = \begin{pmatrix} -2 & 1 & & & \\ 1 & -2 & 1 & & \\ & 1 & -2 & 1 & \\ & & \ddots & \ddots & \ddots \\ & & & 1 & -2 \end{pmatrix}_{M_x \times M_x}, \quad G_1 = \begin{pmatrix} g_1^{n+1} \\ 0 \\ \vdots \\ 0 \\ g_2^{n+1} \end{pmatrix}_{M_x}, \quad G_2 = \begin{pmatrix} g_1^n \\ 0 \\ \vdots \\ 0 \\ g_2^n \end{pmatrix}_{M_x}.$$

求解程序编写如下:

```
function [U, x, t] = diffEqSolver1D(f, kappa, g1, g2, eta, ...
    xspan, tspan, Mx, Mt, ind, varargin)
% 一维扩散方程求解, 方程形式为
```

```

%      Ut - kappa * Uxx = f
%      U(x0, t) = g1(t), U(xend, t) = g2(t)
%      U(x, t0) = eta(x)
%
% Input  -f          源项函数, 例如, f = fun(x, t)
%         -xspan     x方向计算区间, 形式为xspan = [x0, xend];
%         -tspan     t方向计算区间, 形式为tspan = [t0, tend];
%         -[Mx, Mt]  x和t方向的内点个数, 形式为Mx = 100; Mt = 120;
%         -[g1, g2]  方程在左右边界处的函数形式, 如g1 = fun(t);
%         -eta       方程在t0处的函数形式, 如eta = fun(x);
%         -ind       时间离散方法 1.Euler法; 2.向后Euler法;
%                   3.C-N格式; 其他, 需输入theta值
% Output -U          一维扩散方程求解的数值解
%         -[x, y]    U对应的网格节点

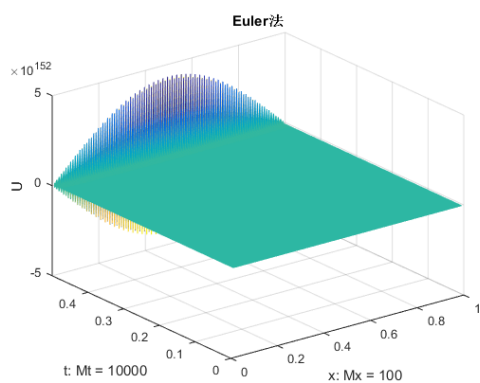
% 以情况给theta赋值
switch ind
case 1, theta = 0;
case 2, theta = 1;
case 3, theta = 1/2;
otherwise
    try
        theta = varargin{1};
    catch
        error('Not enough input arguments. ')
    end
    if theta < 0 || theta > 1, error('Error in THETA. '); end
end

U = zeros(Mt, Mx);
% 计算步长
h = abs(xspan(2) - xspan(1)) / (Mx + 1); % x方向的步长
k = abs(tspan(2) - tspan(1)) / Mt; % t方向的步长
r = kappa * k / (h ^ 2); % 网比
% 计算网格节点
x_all = linspace(xspan(1), xspan(2), Mx+2); % x方向所有节点
t_all = linspace(tspan(1), tspan(2), Mt+1); % t方向所有节点
x_in = x_all(2:end-1); % x方向内点
t_in = t_all(1:end-1); % t方向内点
[x, t] = meshgrid(x_in, t_in); % 网格节点
f = feval(f, x, t + theta * k); % 计算源项
% 计算边界数据
g1 = feval(g1, t_in); % 左边界
g2 = feval(g2, t_in); % 右边界
U(1,:) = feval(eta, x_in); % 初始值
% 计算系数矩阵
a = -r * theta; b = 1 + 2 * r * theta; c = a; % 左端矩阵元素
ar = r * (1 - theta); br = b - 2 * r; cr = ar; % 右端矩阵元素
% 依时间t循环求解
for i = 1:Mt - 1
    F = k * f(i,:);
    F(1) = F(1) + r * (theta * g1(i+1) + (1 - theta) * g1(i)) + ...
        br * U(i, 1) + cr * U(i, 2);
    F(Mx) = F(Mx) + r * (theta * g2(i+1) + (1 - theta) * g2(i)) + ...
        ar * U(i, Mx - 1) + br * U(i, Mx);
    F(2:Mx-1) = F(2:Mx-1) + ...
        ar * U(i, 1:Mx-2) + br * U(i, 2:Mx-1) + cr * U(i, 3:Mx);
    U(i+1, :) = thomas_algorithm(a, b, c, F); % 追赶法求方程组
end

```

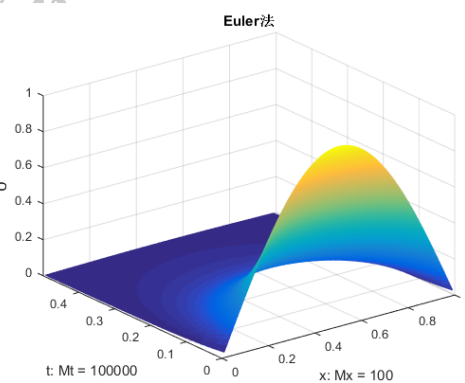

调用函数进行求解：

```
xspan = [0, 1]; tspan = [0, 0.5]; Mx = 100; Mt = 10000;
kappa = 1; ind = 2; theta = 0.9;
f = inline('zeros(size(x))', 'x', 't');
g1 = inline('zeros(size(t))', 't');
g2 = inline('zeros(size(t))', 't');
eta = inline('sin(pi*x)', 'x');
[U, x, t] = diffEqSolver1D(f, kappa, g1, g2, eta, ...
    xspan, tspan, Mx, Mt, ind, theta);
u = exp(-pi ^ 2 * t) .* sin(pi * x);
mesh(x, t, U)
xlabel(['x: Mx= ', num2str(Mx)]), ylabel(['t: Mt= ', num2str(Mt)])
zlabel('U'), xlim(xspan), ylim(tspan)
txt = {'Euler法', '向后Euler法', 'C-N格式', ['theta= ', num2str(theta)]};
title(txt(ind));
E = norm(u - U, inf)
```



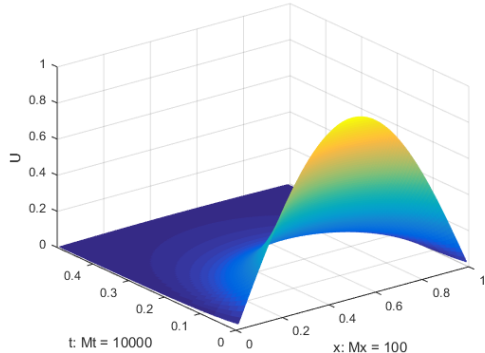
(a) $r > 0.5$

向后Euler法

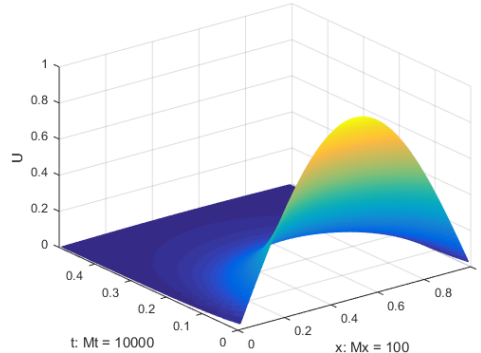


(b) $r < 0.5$

C-N格式



(c)



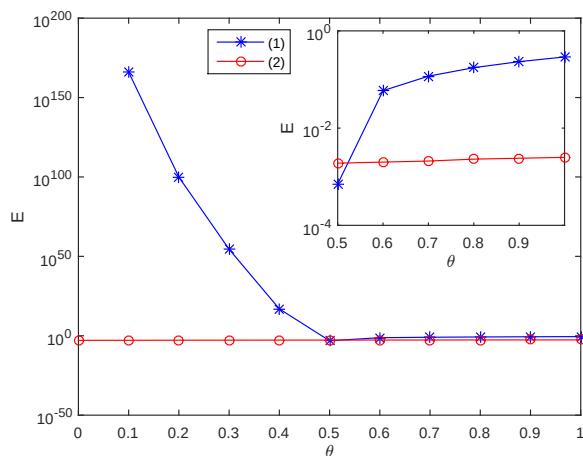
(d)

图 12.2: 三种格式的解图像

从图 12.2 可以看出, Euler 法在网比 $r > 0.5$ 时不稳定, 在 $r < 0.5$ 时稳定, 而向后 Euler 法和 C-N 格式则是稳定的。固定节点数 $M_x = 100$, 在两种情况 ((1) $M_t = 200$, 此时 $r > 0.5$; (2) $M_t = 10^5$, 此时 $r < 0.5$) 下, 讨论 θ 对 $E = \|u - U\|_\infty$ 的影响, 见表 12.1 和图 12.3 所示, 可以得到类似的结论。

表 12.1: 相对于 θ 的误差表

θ	0	0.1	0.2	0.3	0.4	0.5
(1)	NaN	1.12e+166	8.35e+99	3.01e+54	8.10e+16	7.07e-4
(2)	0.0013	0.0014	0.0016	0.0017	0.0018	0.0019
θ	0.6	0.7	0.8	0.9	1.0	
(1)	0.0590	0.1171	0.1751	0.2330	0.2907	
(2)	0.0020	0.0021	0.0023	0.0024	0.0025	

图 12.3: 相对于 θ 的误差图

12.4 二维热传导方程的求解

编写交替方向法(ADI)的程序, 求解二维热传导方程

$$\begin{cases} u_t - \kappa(u_{xx} + u_{yy}) = s(x, y, t), & 0 < x < 1, \quad 0 < y < 1, \quad 0 < t < T, \\ u(0, y, t) = g_1(y, t), \quad u(1, y, t) = g_2(y, t), \\ u(x, 0, t) = g_3(x, t), \quad u(x, 1, t) = g_4(x, t), \\ u(x, y, 0) = \eta(x, y), \end{cases}$$

其中 $\kappa = 1$, $T = 1$, $s(x, y, t) = e^{f(x, y)}(1 + 2t + 2f(x, y)t)$, $\eta(x, y) = 0$, $g_1(y, t) = g_2(y, t) = te^{-\frac{1}{2}[\frac{1}{4} + (y - \frac{1}{2})^2]}$, $g_3(x, t) = g_4(x, t) = te^{-\frac{1}{2}[\frac{1}{4} + (x - \frac{1}{2})^2]}$, $f(x, y) = -\frac{1}{2}[(x - \frac{1}{2})^2 + (y - \frac{1}{2})^2]$ (提示: 精确解为 $u(x, y, t) =$

$te^{-f(x,y)}$ 。

解 在 x, y, t 方向上分别均匀地取 $M_x + 2, M_y + 2, M_t + 1$ 个点, 相应的步长记为 h_x, h_y 和 2τ 。在 (x_j, y_k, t_n) 处的近似解记为 $U_{jk}^n, j = 0, 1, \dots, M_x + 1, k = 0, 1, \dots, M_y + 1, n = 0, 1, \dots, M_t$, 并记 $r_x = \frac{\tau\kappa}{h_x^2}, r_y = \frac{\tau\kappa}{h_y^2}$ 。

ADI格式可写为

$$\begin{cases} U_{jk}^* = U_{jk}^n + \tau\kappa(\delta_x^2 U_{jk}^* + \delta_y^2 U_{jk}^n) + \tau s_{jk}^*, \\ U_{jk}^{n+1} = U_{jk}^* + \tau\kappa(\delta_x^2 U_{jk}^* + \delta_y^2 U_{jk}^{n+1}) + \tau s_{jk}^*, \end{cases}$$

写成矩阵形式为

$$\begin{cases} (I_{M_x} - M_1 r_x) U_{:,k}^* = r_x G_1 + r_y U_{:,k+1}^n + (1 - 2r_y) U_{:,k}^n + r_y U_{:,k-1}^n + \tau s_{:,k}^*, \\ (I_{M_y} - M_2 r_x) [U_{j,:}^{n+1}]^T = [r_y G_2 + r_x U_{j+1,:}^* + (1 - 2r_x) U_{j,:}^* + r_x U_{j-1,:}^* + \tau s_{j,:}^*]^T, \end{cases}$$

其中

$$M_1 = \begin{pmatrix} -2 & 1 & & & \\ 1 & -2 & 1 & & \\ & 1 & -2 & 1 & \\ & & \ddots & \ddots & \ddots \\ & & & 1 & -2 \end{pmatrix}_{M_x \times M_x}, M_2 = \begin{pmatrix} -2 & 1 & & & \\ 1 & -2 & 1 & & \\ & 1 & -2 & 1 & \\ & & \ddots & \ddots & \ddots \\ & & & 1 & -2 \end{pmatrix}_{M_y \times M_y}, G_1 = \begin{pmatrix} (g_1)_k^* \\ 0 \\ \vdots \\ 0 \\ (g_2)_k^* \end{pmatrix}_{M_x}$$

$G_2 = [(g_3)_j^{n+1}, 0, \dots, 0, (g_4)_j^{n+1}]_{M_y}$, $\delta_x^2 U_{jk} = \frac{U_{j+1,k} - 2U_{jk} + U_{j-1,k}}{h_x^2}$, $\delta_y^2 U_{jk} = \frac{U_{j,k+1} - 2U_{jk} + U_{j,k-1}}{h_y^2}$, “ $\cdot$ ”表示取遍所有内点, “ $*$ ”表示在 $t_{n+1/2}$ 处的取值。

求解程序如下:

```
function [U, x, y] = diffEqSolver2D_ADI(s, kappa, g1, g2, g3, g4, eta, ...
    xspan, yspan, tspan, Mx, My, Mt)
% 二维扩散方程求解ADI方法, 方程形式为
%      Ut - kappa * (Uxx + Uyy) = s
%      U(x0, y, t) = g1(y, t)
%      U(xend, y, t) = g2(y, t)
%      U(x, y0, t) = g3(x, t)
%      U(x, yend, t) = g4(x, t)
%      U(x, y, t0) = eta(x, y)
%
% Input  -s          源项函数
%         -xspan      x方向计算区间, 形式为xspan = [x0, xend];
%         -yspan      y方向计算区间, 形式为yspan = [y0, yend];
%         -tspan      t方向计算区间, 形式为tspan = [t0, tend];
%         -[Mx, My, Mt] x, y和t方向的内点个数
%         -[g1, g2]    方程在左右边界处的函数形式
%         -[g3, g4]    方程在上下边界处的函数形式
%         -eta         方程在t0处的函数形式
% Output -U          二维扩散方程求解的数值解在tend处的矩阵
%         -[x, y]      数据结点
Ustar = zeros(Mx, My); % 预分配 Ustar = U(x, y, t(n+1/2))
% 计算步长
hx = abs(xspan(2) - xspan(1)) / (Mx + 1); % x方向的步长
hy = abs(yspan(2) - yspan(1)) / (My + 1); % y方向的步长
```

```

tao = abs(tspan(2) - tspan(1)) / Mt/2;           % t方向的1/2步长
rx = kappa * tao / (hx^2); ry = kappa * tao / (hy^2); % 网比
% 计算网格节点
x_all = linspace(xspan(1), xspan(2), Mx+2);      % x方向所有节点
y_all = linspace(yspan(1), yspan(2), My+2);      % y方向所有节点
t_all = linspace(tspan(1), tspan(2), Mt+1);      % t方向所有节点
x_in = x_all(2:end-1); y_in = y_all(2:end-1);    % 内点
[x, y] = meshgrid(x_in, y_in);                  % 网格节点
U = feval(eta, x, y)';                          % 初始值
% 计算系数矩阵
ax = -rx; bx = 1 + 2*rx; cx = ax;                % 左端矩阵元素
ay = -ry; by = 1 + 2*ry; cy = ay;                % 右端矩阵元素
% 依时间t循环求解
for i = 1:Mt
    t = t_all(i); ts = t + tao;
    % 第一步: 用U隐式求解Ustar
    for k = 1:My
        % 固定y方向, 对x方向进行计算
        if k ~= 1 && k ~= My
            % 当y在内部时, 计算vx
            vx = ry * U(:,k+1) + (1-2*ry) * U(:,k) + ry * U(:,k-1);
        elseif k == 1
            % 当y在下边界时
            vx = ry * U(:,2) + (1-2*ry) * U(:,1) + ry * feval(g3, x_in', t);
        else
            % 当y在上边界时
            vx = ry * feval(g4, x_in', t) + (1-2*ry) * U(:,end) + ...
                ry * U(:,end-1);
        end
        vx(1) = vx(1) + rx * feval(g1, y_in(k), ts); % 增加左端移项
        vx(end) = vx(end) + rx * feval(g2, y_in(k), ts); % 增加左端移项
        F = vx + tao * feval(s, x_in', y_in(k), ts); % 右端项之和
        Ustar(:,k) = thomas_algorithm(ax, bx, cx, F); % 追赶法求方程组
    end
    % 第二步: 用Ustar显式求解U, 过程与第一步类似, 注意时刻t的变化
    t = ts; ts = ts + tao;
    for j = 1:Mx
        if j ~= 1 && j ~= Mx
            vy = rx * Ustar(j+1,:) + (1-2*rx) * Ustar(j,:) + rx * Ustar(j-1,:);
        elseif j == 1
            vy = rx * Ustar(2,:) + (1-2*rx) * Ustar(1,:) + rx * feval(g1, y_in, t);
        else
            vy = rx * feval(g2, y_in, t) + (1-2*rx) * Ustar(end,:) + ...
                rx * Ustar(end-1,:);
        end
        vy(1) = vy(1) + ry * feval(g3, x_in(j), ts);
        vy(end) = vy(end) + ry * feval(g4, x_in(j), ts);
        F = vy + tao * feval(s, x_in(j), y_in, t);
        U(j,:) = thomas_algorithm(ay, by, cy, F);
    end
    pause(0.01), mesh(x', y', U) % 暂停0.01秒, 用于绘制结果
    title(['t=' num2str(ts)]) % 添加标题
    xlabel('X-轴'), ylabel('Y-轴'), zlabel('U(x,y,t)') % 添加坐标轴
end
x = x'; y = y';

```

调用求解

```

xspan = [0, 1]; yspan = [0, 1]; tspan = [0, 1]; kappa = 1;
Mx = 200; My = 100; Mt = 20;
fxy = '-1/2*(x-1/2)^2+(y-1/2)^2';
s = inline(['exp(', fxy, ')'.*(1+2*t+2*t^2)*('), fxy, ')'.*t)], 'x', 'y', 't');
fy = '-1/2*(1/4+(y-1/2)^2)';
g1 = inline(['t.*exp(', fy, ')'], 'y', 't');
g2 = g1;
fx = '-1/2*(1/4+(x-1/2)^2)';
g3 = inline(['t.*exp(', fx, ')'], 'x', 't');
g4 = g3;
eta = inline('zeros(size(x))', 'x', 'y');
[U, x, y] = diffEqSolver2D_ADI(s, kappa, g1, g2, g3, g4, eta, ...
    xspan, yspan, tspan, Mx, My, Mt);

```

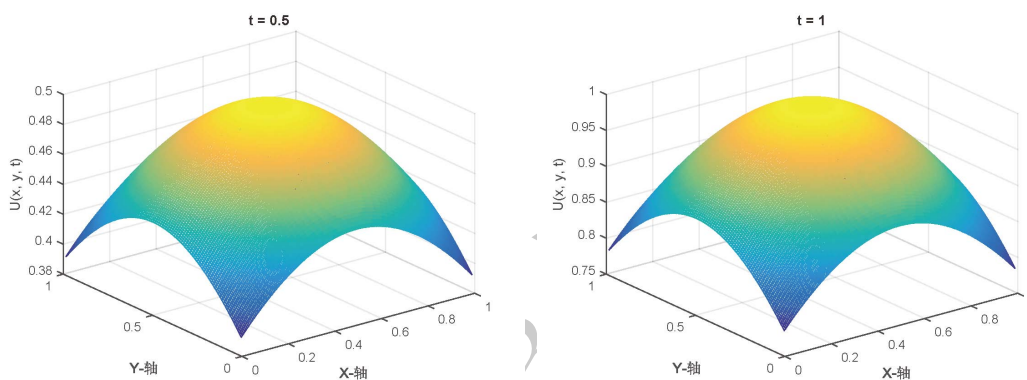


图 12.4: 不同时刻的解的图像

我的 MATLAB 报告

王保山

第三部分

统计分析篇

我的 MATLAB 报告

王保山

第十三章 统计分析

13.1 基础知识

说到概率统计，自然绕不开总体和样本。什么是总体呢？总体就是研究对象的某项数量指标的全体。比如，某次英语六级考试成绩的全体，某城全体市民的年龄。对总体的研究方法常用抽样调查法。在抽样过程中，被抽出的那部分个体称为总体的一个样本，个体的数量 n 称为样本容量。如果每次抽取一个个体时均是在相同条件下独立进行的，我们将这样得到的来自总体的一个样本称为简单随机样本(简称为样本)。统计量是指不含未知参数的样本的函数，均值、中位数、方差、极差、偏度和峰度是几种常用的统计量。

设总体为 X ，样本(观察值)为 $x_1, x_2, \dots, x_n$ 。

(1) 均值 $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ 表征了样本的平均程度，在MATLAB中可通过mean函数求取；

(2) 中位数 x_{med} 也表征了样本的平均程度，它需要将数据从小到大排列为 $x_{(1)}, x_{(2)}, \dots, x_n$ ，

$$x_{\text{med}} = \begin{cases} x_{(\frac{n+1}{2})}, & n \text{ 为奇数,} \\ \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2}, & n \text{ 为偶数,} \end{cases}$$

它可通过median函数求取；

表 13.1: 常见的概率分布及其命令

概率分布	命令字符串	函数命令	功能
正态分布	norm	pdf	概率密度
χ^2 分布	chi2	cdf	概率分布
t 分布	t	inv	逆概率分布
F 分布	f	stat	均值与方差
泊松分布	poiss	fit	参数估计
指数分布	exp	rnd	随机数生成

(3) 方差 $s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$ 表征了数据的离散程度，可通过var函数求取；标准差(均方

差) $s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$ 可通过std函数求取;

(4) 极差(数据的最大值和最小值之差) $x_{\text{range}} = \max\{x_i\} - \min\{x_i\}$, 可通过range函数求取;

(5) 偏度 $g_1 = \frac{1}{s^3} \sum_{i=1}^n (x_i - \bar{x})^3$ 用于衡量对称性, 可用skewness函数求取;

(6) 峰度 $g_2 = \frac{1}{s^4} \sum_{i=1}^n (x_i - \bar{x})^4$ 是衡量偏离正态分布的尺度之一, 可通过kurtosis函数求取。

在MATLAB的统计工具箱中约有20种概率分布, 每种分布(某些分布除外)大多都提供了6类函数, 如表 13.1 所示。

注 13.1 当求一种分布的某一类函数时, 可以将表 13.1 中的分布命令与函数命令拼接起来使用, 如, 求正态分布的概率密度函数为normpdf, 求正态分布的上 α 分位数的函数是norminv。

例 13.1 按要求计算:

- (1) 均值为2, 方差为0.25的正态分布在1, 1.5, 2, 2.5, 3, 3.5处的概率密度。
- (2) 自由度为20的 χ^2 分布在15, 20, 25处的分布函数的值。
- (3) 自由度为25的t分布的下0.9, 0.95, 0.975, 0.98, 0.99分位数。
- (4) 标准正态分布的上0.1, 0.05, 0.025, 0.02, 0.02分位数。
- (5) 自由度为10, 20的F分布的下0.9, 0.95, 0.975, 0.99分位数。

解

```
>> x = 1:0.5:3.5; y = normpdf(x, 2, sqrt(0.25))           % (1)
y =
    0.1080    0.4839    0.7979    0.4839    0.1080    0.0089
>> x = [15 20 25]; y = chi2cdf(x, 20)                   % (2)
y =
    0.2236    0.5421    0.7986
>> x = [0.9 0.95 0.975 0.98 0.99]; y = tinvt(x, 25)     % (3)
y =
    1.3163    1.7081    2.0595    2.1666    2.4851
>> x = [0.1 0.05 0.025 0.02 0.02]; y = norminv(1-x, 0, 1) % (4)
y =
    1.2816    1.6449    1.9600    2.0537    2.0537
>> x = [0.9 0.95 0.99]; y = finv(x, 10, 20)             % (5)
y =
    1.9367    2.3479    3.3682
```

例 13.2 画出以下图形:

- (1) 自由度分别为2, 10的t分布的概率密度曲线。
- (2) 自由度分别为5, 10的 χ^2 分布的概率密度曲线。

(3) 自由度分别为(10, 50), (10, 5)的 F 分布的概率密度曲线。

(4) 均值为0, 方差分别为1和2的正态分布的概率密度曲线。

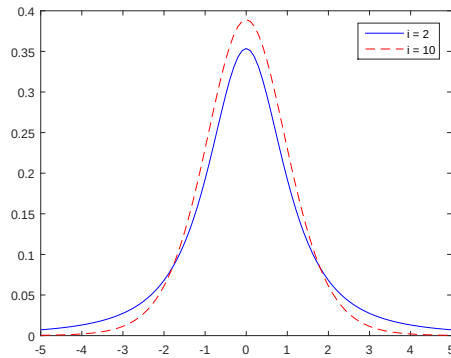


图 13.1: t 分布的概率密度曲线

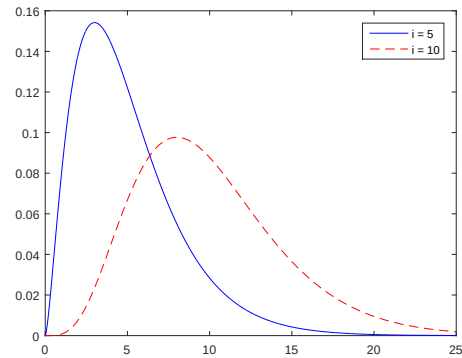


图 13.2: χ^2 分布的概率密度曲线

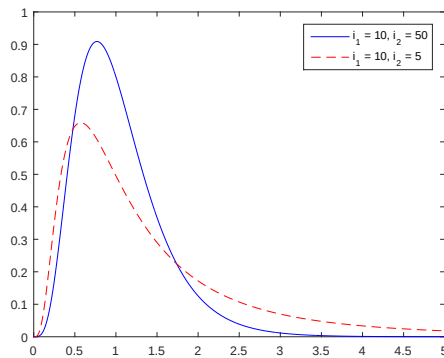


图 13.3: F 分布的概率密度曲线

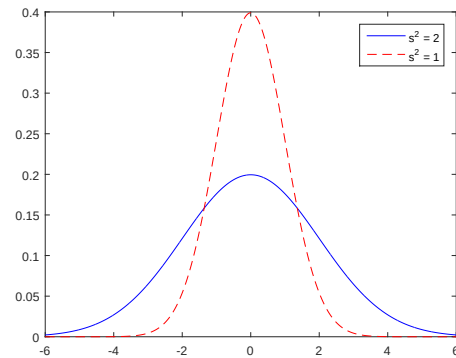


图 13.4: 正态分布的密度函数曲线

解

```
%% (1)
x = -5:0.1:5; y1 = tpdf(x, 2); y2 = tpdf(x, 10);
plot(x, y1, 'b-', x, y2, 'r--')
legend('i=2', 'i=10') % 结果如图 13.1

%% (2)
x = 0:0.01:25; y1 = chi2pdf(x, 5); y2 = chi2pdf(x, 10);
plot(x, y1, 'b-', x, y2, 'r--')
legend('i=5', 'i=10') % 结果如图 13.2

%% (3)
x = 0:0.01:5; y1 = fpdf(x, 10, 50); y2 = fpdf(x, 10, 5);
plot(x, y1, 'b-', x, y2, 'r--')
```

```

legend('i_1=10,i_2=50', 'i_1=10,i_2=5') % 结果如图 13.3
%% (4)
x = -6:0.01:6; y1 = normpdf(x, 0, 2); y2 = normpdf(x, 0, 1);
plot(x, y1, 'b-', x, y2, 'r--')
legend('s^2=2', 's^2=1') % 结果如图 13.4

```

13.2 参数估计与假设检验

在数理统计中, 对参数的估计主要有矩法和极大似然法等方法。区间估计是待估参数 θ , 在给定的显著性水平 α 下, 寻找 $\hat{\theta}$ 满足 $P\{\hat{\theta}_1 \leq \theta \leq \hat{\theta}_2\} = 1 - \alpha$ 的上下界 $\hat{\theta}_1, \hat{\theta}_2$ 的过程。我们将 $1 - \alpha$ 称作置信水平, 把区间 $[\hat{\theta}_1, \hat{\theta}_2]$ 称作置信区间。值得注意的是, 置信区间并不唯一。

正态总体的参数估计和区间估计可用normfit函数实现, 格式如下:

```

[mu, sigma, muc1, sigmac1] = normfit(x, alpha)
% Input -x 样本观察值 -alpha 显著性水平, 默认0.05
% Output -mu 均值 -sigma 标准差 -muc1 均值估计 -sigmac1 标准差估计

```

例 13.3 试对表 13.2 (数据文件: 实验报告数据.xls, 表名: 身高体重)中的身高与体重进行区间估计。

```

>> data = xlsread('实验报告数据.xls', '身高体重');
>> shenGao = data(:, 1:2:10);
>> tiZhong = data(:, 2:2:10);
>> shenGao = shenGao(:); % 将矩阵变成向量
>> tiZhong = tiZhong(:);
>> [mu, sigma, muc1, sigmac1] = normfit(shenGao)
mu =
    170.2500
sigma =
     5.4018
muc1 =
    169.1782
    171.3218
sigmac1 =
     4.7428
     6.2751
>> [mu, sigma, muc1, sigmac1] = normfit(tiZhong)
mu =
     61.2700
sigma =
     6.8929
muc1 =
     59.9023
     62.6377
sigmac1 =
     6.0520
     8.0073

```

表 13.2: 数据表

身高	体重	身高	体重	身高	体重	身高	体重	身高	体重
172	75	169	64	169	55	171	65	167	47
171	62	165	52	168	67	169	62	168	65
166	62	164	59	168	65	170	58	165	64
160	55	173	74	175	67	172	64	168	57
155	57	172	69	176	64	169	58	176	57
173	58	169	52	168	50	167	72	170	57
166	55	173	57	161	49	175	76	158	51
170	63	173	61	169	63	164	59	165	62
167	53	166	70	171	61	166	63	172	53
173	60	163	57	178	64	169	54	169	66
178	60	170	56	177	66	167	54	169	58
173	73	160	65	170	58	179	62	172	50
163	47	165	58	173	67	176	63	162	52
165	66	177	66	172	59	182	69	175	75
170	60	169	63	170	62	186	77	174	66
163	50	176	60	172	59	166	76	167	63
172	57	177	67	177	58	169	72	166	50
182	63	172	56	176	68	173	59	174	64
171	59	165	56	175	68	169	65	168	62
177	64	166	49	184	70	171	71	170	59

故身高均值的置信区间为[169.1782, 171.3218]，标准差的置信区间为[4.7428, 6.2751]；体重均值的置信区间为[59.9023, 62.6377]，标准差的置信区间为[6.0520, 8.0073]。

假设检验就是在给定置信水平 α 下，在零假设 H_0 成立的条件下，通过构造检验统计量，确定拒绝域，然后将现有信息代入进行检验的一种方法。MATLAB中提供了几种检验函数，详见表 13.3。

表 13.3: 常用的假设检验的函数

函数	说明
$[h, p, ci] = ztest(x, mu, sigma, alpha, tail)$	单个正态总体均值的检验，要求 σ^2 已知
$[h, p, ci] = ttest(x, mu, alpha, tail)$	单个正态总体均值的检验， σ^2 未知
$[h, p, ci] = ttest2(x, y, alpha, tail)$	两个正态总体均值的检验， $\sigma_1^2 = \sigma_2^2$ 但未知
qqplot、normplot	绘制总体分布的正态性检验的Q-Q图、P-P图
$[p, h] = ranksum(x, y, alpha)$	两总体均值的检验(Wilcoxon秩和检验)
$[p, h] = signrank(x, y, alpha)$	比较两组数据均值的假设检验(符号秩检验)

例 13.4 用`random`函数生成单个正态总体的100个随机数，并检验。

```
>> x = random('norm', 0, 1, 100, 1);
>> mu = 0; sigma = 1; alpha = 0.05; tail = 0;
>> [h, p, ci] = ztest(x, mu, sigma, alpha, tail) % sigma已知
h =
    0
p =
    0.9365
ci =
   -0.1880
    0.2040
>> [h, p, ci] = ttest(x, mu, alpha, tail) % sigma未知
h =
    0
p =
    0.9414
ci =
   -0.2066
    0.2225
>> normplot(x) % 结果如图 13.5
```

注意随机数的生成方法，在做模拟试验时经常会用到，MATLAB为此提供了很多函数，如表 13.4。

表 13.4: 随机数的生成

函数	功能
<code>R = rand(m, n)</code>	生成 $[0, 1]$ 上的均匀分布随机数
<code>R = randn(m, n)</code>	生成标准正态分布随机数
<code>R = randperm(N)</code>	生成 $1 : N$ 的随机排列
<code>R = random(str, p1, p2, ..., m, n)</code>	生成 <code>str</code> 类随机数
<code>R = unidrnd(N, m, n)</code>	生成 $1 : N$ 的等概率矩阵
<code>R = binornd(k, p, m, n)</code>	生成 $B(k, p)$ 二项分布随机数
<code>R = unifrnd(a, b, m, n)</code>	生成 $[a, b]$ 连续型均匀分布数
<code>R = normrnd(mu, sigma, m, n)</code>	正态分布随机数矩阵
<code>R = mvnrnd(mu, sigma, n)</code>	生成 n 维正态分布数

注 13.2 `str`表示分布类型有{`'binomial'` (二项分布), `'uniform'` (均匀分布), `'normal'` (正态分布), `'poisson'` (*Poisson*分布), `'t'`, `'f'`等}; `p1`, `p2`是对应的参数; `[m,n]`表示随机数矩阵的尺寸; `N`为正整数; `mu`为均值; `sigma`为方差或协方差。

例 13.5 生成单个正态总体的1000个随机数，并绘图。

```
>> R = randn(1000, 1);
>> plot(R, 'r'), ylim([-4, 4])    % 结果如图 13.6
```

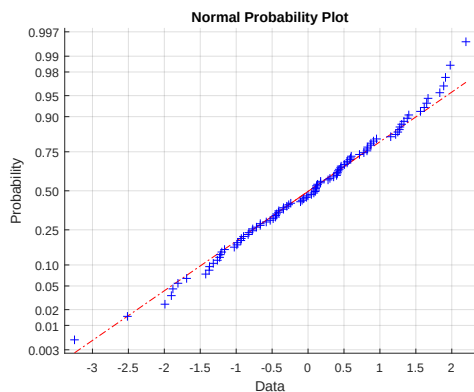


图 13.5: P-P图

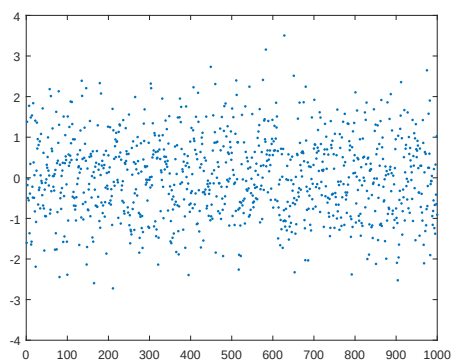


图 13.6: 正态总体的随机数

13.3 统计描述与分析报告

1. 设总体 $X \sim N(40, 5^2)$, 抽取容量为 n 的样本, 样本均值记作 $\bar{x}$.

(1) 设 $n = 36$, 求 $\bar{x}$ 在 38 和 43 之间的概率。

(2) 设 $n = 64$, 求 $\bar{x}$ 与总体均值之差不超过 1 的概率。

(3) 要使 $\bar{x}$ 与总体均值之差不超过 1 的概率达到 0.95, n 应多大?

解 (1) $\bar{x} \sim N(40, \frac{25}{36})$ 。

```
>> x = [38, 43]; s = sqrt(25 / 36);
>> y = normcdf(x(2), 40, s) - normcdf(x(1), 40, s)
y =
    0.9916
```

(2) $\bar{x} \sim N(40, \frac{25}{64})$ 。

```
>> x = [39, 41]; s = sqrt(25 / 64);
>> y = normcdf(x(2), 40, s) - normcdf(x(1), 40, s)
y =
    0.8904
```

(3) $\bar{x} \sim N(40, \frac{25}{64})$ 。

```
>> y = norminv(1-(1-0.95)/2, 0, 1);
>> n = ceil(y ^ 2 * 25)
n =
    97
```

2. 设总体 $X \sim N(\mu, \sigma^2)$, 现有容量 $n = 16$, 均值 $\bar{x} = 12.5$, 方差 $s^2 = 5$ 的样本。

(1) 已知 $\sigma = 2$, 求 μ 与 $\bar{x}$ 之差不超过 0.5 的概率。

(2) 若 σ 未知, 求 μ 与 $\bar{x}$ 之差不超过 0.5 的概率。

(3) 求 $\sigma > 3$ 的概率。

解 (1) $\bar{x} \sim N(\mu, \frac{\sigma^2}{n})$, 故 $\frac{\bar{x} - \mu}{\sqrt{\sigma^2/n}} \sim N(0, 1)$ 。

$$P\{|\bar{x} - \mu| \leq 0.5\} = P\left\{-\frac{0.5}{\sqrt{4/16}} \leq \frac{\bar{x} - \mu}{\sqrt{4/16}} \leq \frac{0.5}{\sqrt{4/16}}\right\} = P\left\{-1 \leq \frac{\bar{x} - \mu}{\sqrt{4/16}} \leq 1\right\}.$$

```
>> y = normcdf(1, 0, 1) - normcdf(-1, 0, 1)
y =
    0.6827
```

(2)

```
>> y = tcdf(0.5/(sqrt(5/15)), 15) - tcdf(-0.5/(sqrt(5/15)), 15)
y =
    0.5999
```

(3)

```
>> y = chi2cdf((16*5)/9, 15)
y =
    0.1167
```

3. 某厂从一台机床生产的滚珠中随机抽取 9 个, 测得直径 (mm) 如下:

14.6, 14.7, 15.1, 14.9, 14.8, 15.0, 15.1, 15.2, 14.8.

设滚珠直径服从正态分布, 试在置信水平分别为 0.85, 0.9, 0.95, 0.975, 0.99 的情况下, 对直径的均值和标准差作区间估计。

解


```
data = [14.6, 14.7, 15.1, 14.9, 14.8, 15.0, 15.1, 15.2, 14.8];
alpha = 0.85, 0.9, 0.95, 0.975, 0.99];
for i = 1:5
    [mu, sigma, muci, sigmaci] = normfit(data, alpha(i))
end % 结果如表 13.6
```

表 13.5: 直径的均值和标准差的区间估计表

置信水平	均值	均值区间估计	标准差	标准差区间估计
0.85	14.9111	[14.8979, 14.9243]	0.2028	[0.2017, 0.2224]
0.9	14.9111	[14.9023, 14.9199]	0.2028	[0.2049, 0.2187]
0.95	14.9111	[14.9067, 14.9155]	0.2028	[0.2082, 0.2151]
0.975	14.9111	[14.9089, 14.9133]	0.2028	[0.2099, 0.2133]
0.99	14.9111	[14.9102, 14.9120]	0.2028	[0.2109, 0.2123]

4. 某旅行社为调查当地旅游者的平均消费额, 随机访问了100名旅游者, 得知平均消费额 $\bar{x} = 80$ 元。据经验: 旅游者消费额服从正态分布, 且标准差为12元, 求该地旅游者平均消费额的置信水平为0.95的置信区间。

解 设平均消费额为 ξ , 期望为 a , 方差为 σ^2 , 则有旅游者消费额 $x \sim N(a, \sigma^2)$, $\bar{x} \sim N(a, \frac{\sigma^2}{n})$ 。故 $\frac{\bar{x} - a}{\sigma/\sqrt{n}} \sim N(0, 1)$, 有

$$P\{x_1 \leq \xi \leq x_2\} = P\left\{\frac{x_1 - a}{\sigma/\sqrt{n}} \leq \frac{\bar{x} - a}{\sigma/\sqrt{n}} \leq \frac{x_2 - a}{\sigma/\sqrt{n}}\right\} = 0.95.$$

```
>> p = 0.95; a = 80; sigma = 12; n = 100;
>> x1 = norminv( (1-p)/2, 0, 1) * sigma / sqrt(n) + a
x1 =
    77.6480
>> x2 = norminv(1-(1-p)/2, 0, 1) * sigma / sqrt(n) + a
x2 =
    82.3520
```

故旅游者平均消费额在置信水平0.95下的置信区间为[77.6480, 82.3520]。

5. 某元件的寿命服从正态分布, 现测得16只元件的寿命(小时)如下:

159, 280, 101, 212, 224, 379, 179, 264, 222, 362, 168, 250, 149, 260, 485, 170。

问是否有理由认为元件的平均寿命大于225小时?

解 设平均寿命少于225小时, 即 $H_0: u \leq u_0, H_1: u > u_0$, 默认显著性水平为0.05, 因方差未知, 采用 t 检验法。

```
>> x = [159, 280, 101, 212, 224, 379, 179, 264, 222, 362, 168, 250, 149, 260, 485, 170];
>> [h p ci] = ttest(x, 225, 0.05, 'right')
h =
    0
p =
    0.2570
ci =
    198.2321      Inf
```

故在5%的置信度下可以接受 H_0 ，拒绝 H_1 ，即平均寿命少于225小时。

6. 据说某地的汽油价格是每加仑115美分，为了验证这种说法，一位学者开车随机选择了一些加油站，得到某年1月和2月的数据如表 13.6 (文件名：实验报告数据.xls，表名：汽油价格) 所示。

表 13.6: 数据表

1月	119	117	115	116	112	121	115	122	116	118
	109	112	119	112	117	113	114	109	109	108
2月	118	119	115	122	118	121	120	122	128	116
	120	123	121	119	117	119	128	126	118	125

- (1) 分别用两个月的数据验证这种说法的可靠性;
- (2) 分别给出1月和2月汽油价格的置信区间($\alpha = 0.05$)。
- (3) 给出1月和2月汽油价格差价的置信区间($\alpha = 0.05$)。

解 (1) 设 $H_0 : u = u_0$, $H_1 : u \neq u_0$ ，假设汽油价格服从正态分布，由于方差未知，采用 t 检验法。

```
>> data = xlsread('实验报告数据.xls', '汽油价格');
>> x1 = [data(1,:), data(3,:)];
>> x2 = [data(2,:), data(4,:)];
>> [h1, sig1, ci1] = ttest(x1, 115, 0.05, 0)
h1 =
    1
sig1 =
    3.2439e-04
ci1 =
    116.8263    120.1737
>> [h2, sig2, ci2] = ttest(x2, 115, 0.05, 0)
h2 =
    0
sig2 =
    0.1740
ci2 =
    114.0841    119.7159
```

由一月份知应该拒绝原假设，由二月份知应该接受原假设。但一月份的 p 值较小，综合考虑应该

拒绝原假设。

(2) 由(1)可直接得出结果, 这里给出另一种方法。

```
>> [~, ~, chaci] = normfit(x1, 0.05)
chaci =
    116.8263
    120.1737
>> [~, ~, chaci] = normfit(x2, 0.05)
chaci =
    114.0841
    119.7159
```

故汽油价格的一月份的置信区间为[116.8263, 120.1737], 二月份的置信区间为[114.0841, 119.7514]。

(3) 将两个月份的作差, 再用 t 检验。

```
>> x3 = x2 - x1;
>> [~, ~, chaci] = normfit(x3, 0.05)
chaci =
   -4.7208
    1.5208
```

故差值的置信区间为[-4.7208, 1.5208]。

7. 第3题的数据是机床甲产生的, 另从机床乙生产的滚珠中抽取10个, 测得直径(mm)如下:

15.2, 15.1, 15.4, 14.9, 15.3, 15, 15.2, 14.8, 15.7, 15。

记两机床生产的滚珠直径分别为 μ_1, μ_2 , 试作 $\mu_1 = \mu_2, \mu_1 \leq \mu_2, \mu_1 \geq \mu_2$ 三种检验。

解 分别设 $H_0: \mu_1 = \mu_2, H_1: \mu_1 \neq \mu_2, H_0: \mu_1 \geq \mu_2, H_1: \mu_1 < \mu_2, H_0: \mu_1 \leq \mu_2, H_1: \mu_1 > \mu_2$, 因题中未说明服从正态分布, 故应先通过 $Q-Q$ 图检验分布。

```
>> x2 = [15.2, 15.1 15.4 14.9 15.3 15 15.2 14.8 15.7 15];
>> h = qqplot(x2); % 结果如图 13.7
```

从 $Q-Q$ 图上可以看, 可以认为机床乙生产的滚珠直径服从正态分布。

本题要求对两个总体的均值进行检验, $\sigma_1^2 = \sigma_2^2$ 未知但相等, 此时可用 t 检验法。

```
>> x1 = [14.6, 14.7, 15.1, 14.9, 14.8, 15.0, 15.1, 15.2, 14.8];
>> [h1, p1, ci1] = ttest2(x1, x2, 0.05, 0)
h1 =
     1
p1 =
    0.0352
```

```
ci1 =
    -0.4784    -0.0194
```

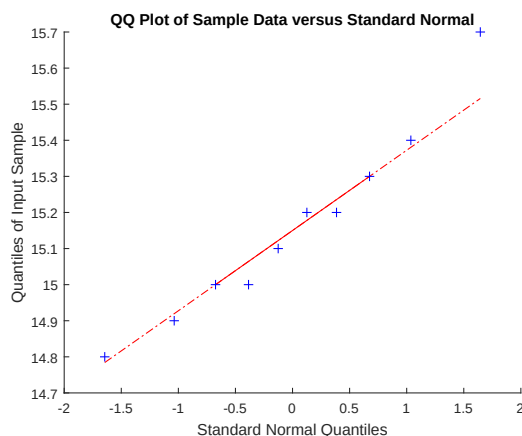


图 13.7: 机床乙生产的滚珠直径的Q-Q图

```
>> [h2, p2, ci2] = ttest2(x1, x2, 0.05, -1)
h2 =
     1
p2 =
    0.0176
ci2 =
    -Inf    -0.0597
>> [h3, p3, ci3] = ttest2(x1, x2, 0.05, 1)
h3 =
     0
p3 =
    0.9824
ci3 =
   -0.4381     Inf
```

从结果可以看出，双侧检验时 $h = 1$ ，说明拒绝原假设，接受 $\mu_1 \neq \mu_2$ ， p 值很小说明了显著性；左侧检验时 $h = 1$ ，说明拒绝原假设， p 值很小说明显著性，接受 $\mu_1 < \mu_2$ ；右侧检验时 $h = 0$ ，说明接受原假设，即接受 $\mu_1 \leq \mu_2$ ， p 值很大说明显著性。以上结果综合说明 $\mu_1 < \mu_2$ ，即机床甲的滚珠直径的均值小于乙是显著的。

8. 甲方向乙方成批供货，甲方承诺合格率为90%，双方商定置信概率为0.95。现从一批货中抽取50件，43件为合格品，问乙方应否接受这批货物？你能为乙方不接受它出谋划策吗？

解 设总体服从 $0-1$ 均匀分布，不合格率为 p ，当 n 较大时，可以看做服从正态分布。设 $H_0: p \leq p_0, H_1: p > p_0$ ，可得 $z = \frac{\bar{z} - p_0}{\sqrt{p_0(1-p_0)/n}}$ ，当 $P\{z \leq u_{1-\alpha}\}$ 时接受原假设即接受产品，否则拒绝。

```
>> a = 0.05; z1 = 7/50; p = 0.1; n = 50;
>> z = (z1 - p) / (p * (1 - p) / n) ^ 0.5;
>> u = norminv(1 - a, 0, 1);
>> h = z > u
h =
    0
```

由 $h = 0$ 应接受产品。若不接受此产品可提高 n 值:

```
>> n = ceil((u / (z1 - p)) ^ 2 * p * (1 - p))
n =
    153
```

当 $n \geq 153$ 时, 则应该拒绝此产品。

9. 为研究吸烟习惯与患慢性气管炎的关系, 调查了339名50岁的人, 得各类人数如表 13.7。

表 13.7: 数据表

	吸烟	不吸烟	小计
患气管炎	43	13	56
没患气管炎	162	121	283
小计	205	134	339
患病率	21.00%	9.70%	16.50%

若显著性水平为0.05, 试问吸烟对患气管炎有无影响?

解 这里每个对象考察两个指标, 记 ξ 为是否吸烟, η 为是否患病。

设 $H_0: F(x, y) = F_\xi(x)F_\eta(y)$, $H_1: F(x, y) \neq F_\xi(x)F_\eta(y)$, 得

$$\chi^2 = \left\{ \frac{n(n_{11}n_{22} - n_{12}n_{21})^2}{(n_{11} + n_{12})(n_{21}n_{22})(n_{11} + n_{21})(n_{12} + n_{22})} > \chi_{1-\alpha}^2(1) \right\}$$

```
>> n = 339; n11 = 43; n22 = 121; n12 = 13; n21 = 162;
>> chi2 = n * (n11 * n22 - n12 * n21) ^ 2 / ((n11 + n12) * (n21 + n22) * ...
    (n11 + n21) * (n12 + n22))
chi2 =
    7.4688
>> h = chi2 > chi2inv(0.95, 1)
h =
    1
```

从最后的结果看拒绝 H_0 , 即患病与吸烟有关。

13.4 方差分析

方差分析通常用于多个总体的均值检验。试验数据往往除了受随机因素的影响之外,还受某些其他因素的影响(称为效应),方差分析就是要检验哪几个因素所起的作用是显著的,并且估计这些效应。按因素个数的多少,方差分析可分为单因素方差分析、双因素方差分析和多因素方差分析。

我们下面以单因素方差分析为例讲述方差分析的过程。

要做方差分析,需要满足三个基本假设:

- (1) 对于检验的每一个总体,都服从正态分布;
- (2) 每一个总体均有相同的方差;
- (3) 从各总体抽取的是独立样本。

即 $x_{ij} \sim N(\mu_i, \sigma^2), i = 1, 2, \dots, r, j = 1, 2, \dots, n$ 。记实验数据 x 为

$$\begin{pmatrix} x_{11} & x_{21} & \cdots & x_{r1} \\ x_{12} & x_{22} & \cdots & x_{r2} \\ \vdots & \vdots & \ddots & \vdots \\ x_{1n} & x_{2n} & \cdots & x_{rn} \end{pmatrix}$$

x 的每一列记为 x 的一个水平。判断 r 个水平对 x 有无显著影响,相当于作假设检验: $H_0: \mu_1 = \mu_2 = \cdots = \mu_r, H_1: \mu_1, \mu_2, \dots, \mu_r$ 不全相等。

第一步:作分解

$x_{ij} = \mu_i + \epsilon_{ij}, i = 1, \dots, r, j = 1, \dots, n$, 其中 $\epsilon_{ij} \sim N(0, \sigma^2)$ 且相互独立。

记 $\mu = \frac{1}{r} \sum_{i=1}^r \mu_i$ 为总平均值, $\bar{x}_i = \frac{1}{n} \sum_{j=1}^n x_{ij}$ 为第 i 组数据的平均值, $\bar{x} = \frac{1}{r} \sum_{i=1}^r \bar{x}_i$ 为样本总均值。

第二步:偏差平方和分解

将全体数据对总均值 x 的偏差平方和分解:

$$\begin{aligned} S &= \sum_{i=1}^r \sum_{j=1}^n (x_{ij} - \bar{x})^2 = \sum_{i=1}^r \sum_{j=1}^n (x_{ij} - \bar{x}_i + \bar{x}_i - \bar{x})^2 \\ &= \sum_{i=1}^r \sum_{j=1}^n (x_{ij} - \bar{x}_i)^2 + \sum_{i=1}^r n(\bar{x}_i - \bar{x})^2 + 2 \sum_{i=1}^r \sum_{j=1}^n (x_{ij} - \bar{x}_i)(\bar{x}_i - \bar{x}) \\ &= \sum_{i=1}^r \sum_{j=1}^n (x_{ij} - \bar{x}_i)^2 + \sum_{i=1}^r n(\bar{x}_i - \bar{x})^2 = S_E + S_A \quad \left(\sum_{i=1}^r (\bar{x}_i - \bar{x}) = 0 \right) \end{aligned}$$

我们将 $S_A = \sum_{i=1}^r n(\bar{x}_i - \bar{x})^2$ 称作组间平方和(各组均值对总均值的偏差平方和),将 $S_E =$

$\sum_{i=1}^r \sum_{j=1}^n (x_{ij} - \bar{x}_i)^2$ 称作组内平方和(各组内的数据对改组均值的偏差平方和,也称误差平方和)。

第三步：进一步分析

$$ES_E = r(n-1)\sigma^2 = (r-1)\sigma^2 + \sum_{i=1}^r n(\mu_i - \mu)^2 = (r-1)\sigma^2 \quad (H_0 \text{成立时})$$

当 H_0 成立时，从数值上看，有 $\frac{S_A/(r-1)}{S_E/(n-1)} \approx \frac{\sigma^2}{\sigma^2} = 1$ 成立。从统计量上看，有

$$F = \frac{S_A/(r-1)}{S_E/r(n-1)} \sim F(r-1, r(n-1)).$$

第四步：列方差分析表并检验

由数据 x 列方差分析表如表 13.8 所示。

表 13.8: 方差分析表

方差来源	平方和	自由度	平方均值	F值	p值
因素A	S_A	$r-1$	$\bar{S}_A = \frac{S_A}{r-1}$	$F = \frac{\bar{S}_A}{\bar{S}_E}$	$p = P\{F(r-1, r(n-1)) > F\}$
误差	S_E	$r(n-1)$	$\bar{S}_E = \frac{S_E}{r(n-1)}$		
总和	S	$rn-1$			

第五步：判断

当 $F < F_{1-\alpha}(r-1, r(n-1))$ 时，接受原假设，否则，拒绝。

其他类型的方差分析将会涉及高等代数等课程的知识，不再介绍，有兴趣的读者可以查阅相关资料，深入研究。MATLAB中方差分析的函数如表 13.9。

表 13.9: 方差分析的相关函数

类别	函数	说明
单因素	$p = \text{anova1}(x, \text{group})$	x 为行向量， group 为 x 中的数据属于的水平，长度与 x 相同，若 x 为 $n \times r$ 的矩阵， group 可省略
双因素	无交互作用 $p = \text{anova2}(x)$	x 为 $s \times r$ 的矩阵， p 为两因素的p值
	有交互作用 $p = \text{anova2}(x, \text{rep})$	x 为数据表(A,B)按列排列的 $st \times r$ 的矩阵， rep 为试验重复次数

例 13.6 为比较5种合成木板的耐久性，对每个品牌取4个样品作摩擦试验，测量磨损量，数据如表 13.10 (文件名：实验报告数据.xls，表名：摩擦试验)。

- (1) 它们的耐久性有无显著差异；
- (2) 计算每种品牌的均值，有选择地作两品牌的比较，能得出什么结果。

表 13.10: 摩擦试验数据

品牌A	2.2	2.1	2.4	2.5
品牌B	2.2	2.3	2.4	2.6
品牌C	2.2	2.0	1.9	2.1
品牌D	2.4	2.7	2.6	2.7
品牌E	2.3	2.5	2.3	2.4

解 (1) 使用方差分析

```
>> x = xlsread('实验报告数据.xls', '摩擦试验');
>> p = anova1(x') % 结果如图 13.8
p =
    0.0019
```

Source	SS	df	MS	F	Prob>F
Columns	0.623	4	0.15575	7.19	0.0019
Error	0.325	15	0.02167		
Total	0.948	19			

由 $p = 0.0019 < 0.01$ 可知，各品牌的耐久性有非常显著的差异。

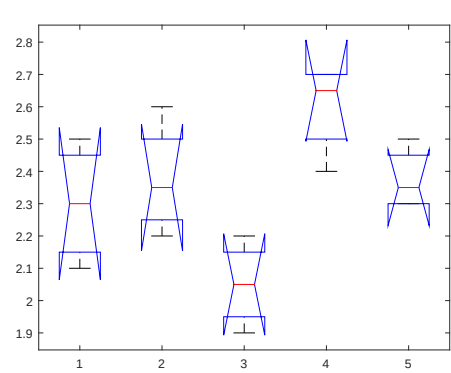


图 13.8: 方差分析图

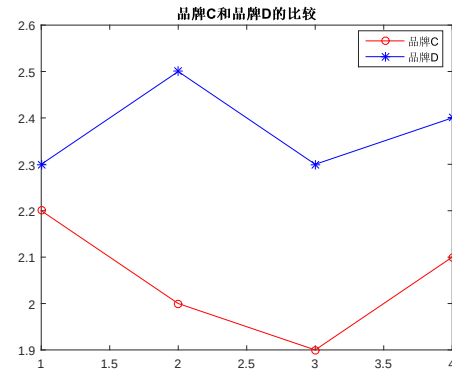


图 13.9: 品牌C和品牌D的比较

- (2) 求均值


```
>> aver = mean(x, 2)
aver =
    2.3000
    2.3750
    2.0500
    2.6000
    2.3750
>> plot(1:4, x(3,:), 'ro-', 1:4, x(5,:), 'b*-')
>> legend('品牌C', '品牌D', 'location', 'Best');
>> title('品牌C和品牌D的比较') % 结果如图 13.9
```

各品牌的均值为2.3、2.35、2.05、2.65、2.35，通过绘制品牌C和品牌D的图像，可以得出品牌C和品牌D的耐久度有比较大的不同。

例 13.7 6种农药在相同的条件下进行灭虫试验，结果如表 13.11 (文件名：实验报告数据.xls，表名：灭虫试验)所示(灭虫率以%计)，问它们的效果有无显著差异。需要作进一步的比较吗？

表 13.11: 灭虫率

农药A	87	85	80	
农药B	90	88	87	94
农药C	56	62		
农药D	55	48		
农药E	92	99	95	91
农药F	75	72	81	

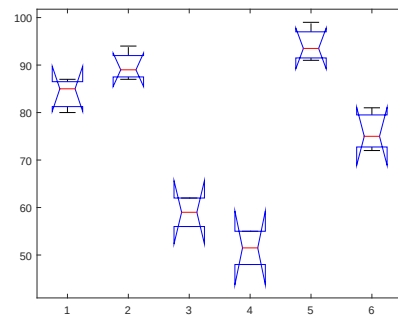


图 13.10: 方差分析图

```
>> x = xlsread('实验报告数据.xls', '灭虫试验');
>> x = x'; x = x(~isnan(x)); % 去除nan数据
>> group = [1*ones(1,3), 2*ones(1,4), 3*ones(1,2), 4*ones(1,2), 5*ones(1,4), 6*ones(1,3)];
>> anova1(x, group); % 注意group的格式，结果如图 13.10
```

Source	SS	df	MS	F	Prob>F
Groups	3794.5	5	758.9	51.16	1.11906e-07
Error	178	12	14.833		
Total	3972.5	17			

由于 $p \approx 1.12 \times 10^{-7} \ll 0.01$ ，因此在相同的条件下不同农药的灭虫效果有非常显著的差异。而药物C和药物D的效果相似，需要进一步的比较他们的灭虫效果的差异性。

例 13.8 将土质基本相同的一块耕地分成均等的5块，每块又分成均等的4个小区。在每一块地内，把4个品种的小麦分种在4个小区，每一小区小麦的播种量相同，测得收获量如表 13.12 (文件名：实验报告数据.xls，表名：小麦收获)所示。

表 13.12: 收获量

	A1	A2	A3	A4	A5
B1	32.3	34.0	34.7	36.0	35.5
B2	33.2	33.6	36.8	34.3	36.1
B3	30.8	24.4	32.3	35.8	32.8
B4	29.5	26.2	28.1	28.5	29.4

```
>> x = xlsread('实验报告数据.xls', '小麦收获');
>> anova2(x);
Source      SS      df      MS      F      Prob>F
-----
Columns    14.098    4      3.5245   1.61   0.2353
Rows       134.645   3      44.8818  20.49   0.0001
Error       26.282   12      2.1902
Total      175.026   19
```

由 $0.2353 > 0.05$ 得，土质对收获量并无显著影响，而 $0.0001 < 0.01$ 可知，小麦的种子对收获量的影响非常的显著。

例 13.9 为了研究合成纤维收缩率 B 和拉伸倍数 A 对纤维弹性的影响，进行了一些收缩试验。收缩率取4个水平：0，4，8，12；拉伸倍数也取4个水平：460，520，580，640，对两者的每个组合重复做两次试验，测得弹性数据如表 13.13 (文件名：实验报告数据.xls，表名：弹性试验)所示。

表 13.13: 弹性数据

	A1=460		A2=520		A3=580		A4=640	
B1=0	71	73	72	73	75	73	77	75
B2=4	73	75	76	74	78	77	74	74
B3=8	76	73	79	77	74	75	74	73
B4=12	75	73	73	72	70	71	69	69

- (1) 拉伸倍数、收缩率及其交互作用对弹性有无显著影响；
- (2) 使纤维弹性达到最大的生产条件是什么？

```
>> x = xlsread('实验报告数据.xls', '弹性试验');
>> y = [x(:, 1:2:end), x(:, 2:2:end)]';
>> y = reshape(y, size(x, 2)/2, [])'
```

```
y =
    71    72    75    77
    73    73    73    75
    73    76    78    74
    75    74    77    74
    76    79    74    74
    73    77    75    73
    75    73    70    69
    73    72    71    69

>> rep = 2;
>> anova2(y, rep);
Source          SS      df      MS      F      Prob>F
-----
Columns         8.594      3      2.8646    2.13    0.1363
Rows          70.594      3     23.5313   17.51     0
Interaction     79.531      9      8.8368    6.58    0.0006
Error           21.5     16      1.3438
Total         180.219     31
```

由 $P = 0.1363 > 0.05$ 得，拉伸倍数对弹性无显著影响，又因 $P = 0 < 0.01$ ，故收缩率对弹性的影响非常显著，而 $P = 0.0006 < 0.01$ ，因此其交互作用对弹性影响非常显著。使纤维弹性达到最大的产生条件是收缩率为8，拉伸倍数为520倍。

13.5 回归分析

回归分析在日常生活中应用最为广泛，尽管回归在前面的章节中已经讲过，这里再讲，除了说明回归分析的重要性外，还要对前面章节的内容进行扩充。实际上，MATLAB为回归分析提供了许多函数，而不仅限于表 13.15 中的函数。

例 13.10 用刀削机床加工时，为实地调整机床需测定刀具的磨损速度，现每隔1小时测量刀具的厚度得到以下数据(文件名：实验报告数据.xls，表名：磨损速度)，试建立刀具厚度关于刀削时间的回归模型，对模型和回归系数进行检验，预测12小时后刀具的厚度。

表 13.14: 数据表

时间(h)	0	1	2	3	4	5	6	7	8	9	10
刀具厚度(cm)	30.6	29.1	28.4	28.1	28.0	27.7	27.5	27.2	27.0	26.8	26.5

解 读取文件，绘制散点图

```
x = xlsread('实验报告数据.xls', '磨损速度');
plot(x(1,:), x(2,:), '*') % 结果如图 13.11
```

表 13.15: 常用的回归分析函数

类别	函数	说明
多元线性回归	<code>b = regress(Y, X)</code>	Y为因变量，列向量；X为第一列为1的数据矩阵，b为回归系数；r为残差， $r = Y - Xb$ ；stats为相关系数，F值，p值，误差估计值；alpha为置信度； bint为回归系数的置信区间；rint为残差置信区间
	<code>[b, bint, r, rint, stats] = regress(Y, X, alpha)</code>	
	<code>rcoplot(r, rint) % 画残差图</code>	
一元多项式回归	<code>[p, S] = polyfit(x, y, m)</code>	m为拟合多项式的次数，p为多项式系数；S为一种数据结构
	<code>[Y, delta] = polyconf(p, x, S, alpha)</code>	delta为置信区间半长
	<code>polytool(x, y, m)</code>	拟合值及置信区间的曲线绘图
多元二项式回归	<code>rstool(x, y, 'model', alpha)</code>	x为 $n \times m$ 的数据矩阵；y为n为列向量；model为回归模式{'linear', 'pure-quadratic', 'interaction', 'quadratic'}
非线性回归	<code>[betahat, f, J] = nlinfit(x, y, @fun, beta)</code>	betahat为beta的估计值；fun为非线性函数形式
	<code>betaci = nlparci(betahat, f, J)</code>	betaci给出参数betahat的置信区间
	<code>[yhat, delta] = nlpredci('funname', x, betahat, f, J)</code>	yhat为y的估计；'funname'为非线性函数的名字

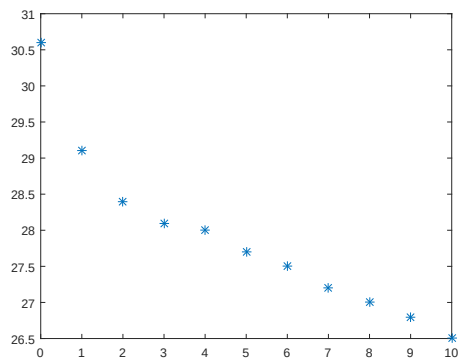


图 13.11: 磨损数据的散点图

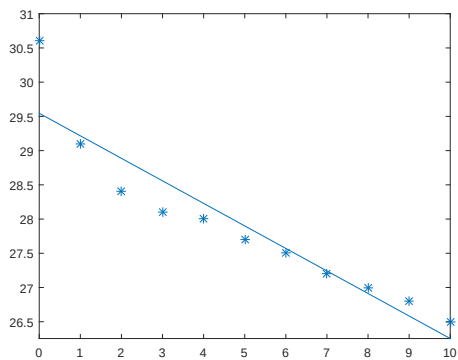


图 13.12: 磨损数据的回归曲线

由图 13.11 得，两者之间大致呈负相关关系。

```
>> lsline % 添加最小二乘拟合曲线, 如图 13.12
>> X = [ones(1,size(x,2)); x(1,:)]';
>> y = x(2,:);
>> [b, bint, r, rint, stats] = regress(y, X, 0.05);
>> b, bint, stats
b =
    29.5455
   -0.3291
bint =
    28.9769    30.1140
   -0.4252   -0.2330
stats =
    0.8696    60.0018    0.0000    0.1985
```

从结果看, $R^2 = 0.8696, p \ll 0.05$ 。这说明模型 $\hat{y} = 29.5455 - 0.3291x$ 回归较好。代入求解:

```
>> yNew = [1,12] * b
yNew =
    25.5964
```

故12小时后刀具厚度为25.5964 cm。

例 13.11 一种合金在某种添加剂的不同浓度下, 各做三次试验, 得到数据如下表(文件名: 实验报告数据.xls, 表名: 添加剂浓度)。

表 13.16: 添加剂浓度数据表

浓度 x	10	15	20	25	30
抗压强度 Y	25.2	29.8	31.2	31.7	29.4
抗压强度 Y	27.3	31.1	32.6	30.1	30.8
抗压强度 Y	28.7	27.8	29.7	32.3	32.8

(1) 作散点图;

(2) 以模型 $Y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon, \epsilon \sim N(0, \sigma^2)$ 拟合数据, 并检验回归模型, 诊断是否有异常点。

解 (1) 读取文件绘制散点图

```
data = xlsread('实验报告数据.xls', '添加剂浓度');
x = data(1,:);
xn = [x; x; x];
y = data(2:end, :);
plot(xn, y, '*'); xlim([5, 35]) % 如图 13.13
```

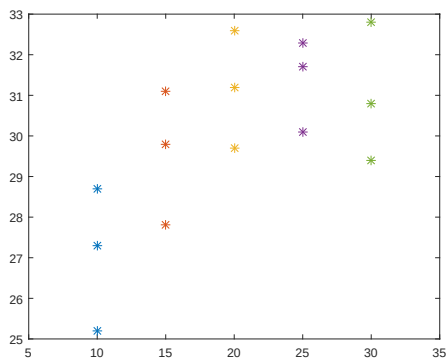


图 13.13: 添加剂数据的散点图



图 13.14: 添加剂数据的残差图

(2) 拟合并检验

```
>> xn = [x, x, x];
>> yn = y'; yn = yn(:);
>> X = [ones(1,length(xn)); xn; xn.^2]';
>> [b, bint, r, rint, stats] = regress(yn, X, 0.05);
>> b, stats
b =
    19.0333
     1.0086
    -0.0204
stats =
    0.6140    9.5449    0.0033    2.0397
>> rcoplot(r, rint)
>> set(gca, 'color', [1, 1, 1]) % 绘制残差图, 如图 13.14
```

从结果看, $R^2 = 0.6140$, $p = 0.033 < 0.05$, 残差图并无异常, 模型成立, 模型为 $y = 19.0333 + 1.0086x - 0.0204x^2$ 。

表 13.17: 耗电量数据表

序号	KWH	AC	DRYER	序号	KWH	AC	DRYER	序号	KWH	AC	DRYER
1	35	1.5	1	8	66	8	1	15	62	7.5	1
2	63	4.5	2	9	94	12.5	1	16	85	12	1
3	66	5	2	10	82	7.5	2	17	43	6	0
4	17	2	0	11	78	6.5	3	18	57	2.5	3
5	94	8.5	3	12	65	8	1	19	33	5	0
6	79	6	3	13	77	7.5	2	20	65	7.5	1
7	93	13.5	1	14	75	8	2	21	33	6	0

例 13.12 某人记录了21天中每天使用空调器的时间和使用烘干器的次数，并检测电表以计算每天的耗电量，数据见表 13.17 (文件名：实验报告数据.xls，表名：耗电量)，试研究耗电量(KWH)与空调器使用的小时数(AC)和烘干器使用次数(DRYER)之间的关系，建立并检验回归模型，诊断是否有异常点。

```
>> data = xlsread('实验报告数据.xls', '耗电量');
>> KWH = [data(:,2)', data(:,6)', data(:,10)'];
>> AC = [data(:,3)', data(:,7)', data(:,11)'];
>> DRYER = [data(:,4)', data(:,8)', data(:,12)'];
>> X = [ones(21,1), AC', DRYER'];
>> [b, bint, r, rint, stats] = regress(KWH', X, 0.05);
>> b, stats
b =
    8.1054
    5.4659
   13.2166
stats =
    0.9709    300.2412    0.0000    15.4872
>> rcoplot(r, rint)
>> set(gca, 'color', [1, 1, 1]) % 绘制残差图，如图 13.15
```

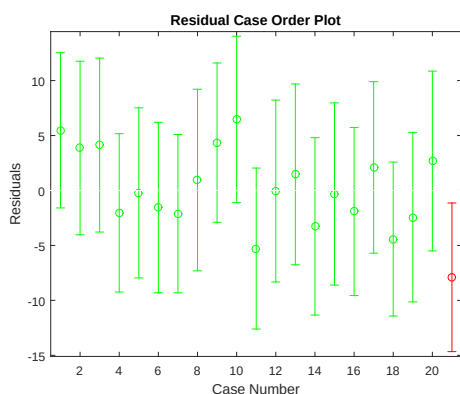


图 13.15: 耗电量数据的残差图

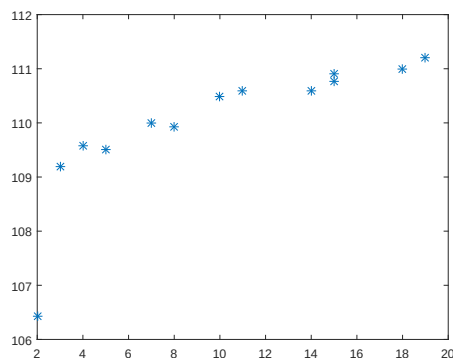


图 13.16: 金属含量数据的散点图

从结果看， $R^2 = 0.9709$, $p \ll 0.05$ ，残差图最后一点为异常点。模型为 $y = 8.1054 + 5.4659x_1 - 13.2166x_2$ ，其中 x_1, x_2 分别表示空调使用的小时数(AC)和烘干器使用次数(DRYER)。

例 13.13 一矿脉有13个相邻样本点，人为地设定一原点，现测得各样本点对原点的距离 x ，与该样本点处某种金属含量 y 的一组数据如表 13.18 (文件名：实验报告数据.xls，表名：金属含量)，画出散点图观察二者的关系，试建立合适的回归模型，如二次曲线、双曲线、对数曲线等。

解 先绘制散点图

```
data = xlsread('实验报告数据.xls', '金属含量');
x = [data(:,1)', data(:,3)', data(1:3,5)'];
y = [data(:,2)', data(:,4)', data(1:3,6)'];
plot(x, y, '*')      % 如图 13.16
```

表 13.18: 金属含量数据表

x	y	x	y	x	y
2	106.42	8	109.93	15	110.76
3	109.20	10	110.49	18	111.00
4	109.58	11	110.59	19	111.20
5	109.50	14	110.60		
7	110.00	15	110.90		

从散点图上看，用 $y = a + \frac{b}{x}$ 模型作回归较好。

```
>> X = [ones(size(x')), 1./x'];
>> [b, bint, r, rint, stats] = regress(y', X, 0.05);
>> b,stats
b =
    111.4405
    -9.0300
stats =
    0.9302    146.6733    0.0000    0.1184
```

从结果看， $R^2 = 0.9302, p \ll 0.05$ ，故模型拟合较好，模型为 $y = 111.4405 - \frac{9.03}{x}$ 。

例 13.14 *Logistic*增长曲线模型和*Gompertz*增长曲线模型是计量经济学等学科中的两个常用模型，可以用来拟合销售量的增长趋势。记*Logistic*增长曲线模型为 $y_t = \frac{L}{1 + ae^{-kt}}$ ；*Gompertz*增长曲线模型为 $y_t = Le^{-be^{-kt}}$ ，这两个模型中 L 的经济学意义都是销售量的上限。表 13.19 (文件名：实验报告数据.xls，表名：销售量)中给出的是某地高压锅的销售量(单位：万台)，为给出这两个模型的拟合结果，请考虑如下的问题：

- (1) *Logistic*增长曲线模型是一个可线性化模型吗？如果给定 $L = 3000$ ，是否是一个可线性化模型？如果是，试用线性化模型给出参数 a 和 k 的估计值；
- (2) 利用(1)所得到的 a 和 k 的估计值作为*Logistic*模型的拟合初值，对*Logistic*模型做非线性回归；
- (3) 取初值 $L^{(0)} = 3000, b^{(0)} = 30, k^{(0)} = 0.4$ ，拟合*Gompertz*模型，并与*Logistic*模型的结果进行比较。

表 13.19: 销售量数据表

年份	t	y	年份	t	y
1981	0	43.65	1988	7	1238.75
1982	1	109.86	1989	8	1560.00
1983	2	187.21	1990	9	1824.29
1984	3	312.67	1991	10	2199.00
1985	4	496.58	1992	11	2438.89
1986	5	707.65	1993	12	2199.00
1987	6	960.25			

解 (1) 可线性化, 令 $y = \ln \frac{1}{y_t} - \frac{1}{L}, \beta_0 = \ln \frac{a}{L}$, 有

$$y_t = \frac{L}{1 + ae^{-kt}} \Rightarrow \frac{1}{y_t} - \frac{1}{L} = \frac{a}{L} e^{-kt} \Rightarrow \ln \frac{1}{y_t} - \frac{1}{L} = \ln \frac{a}{L} - kt \Rightarrow y = \beta_0 - kt.$$

当 $L = 3000$ 时,

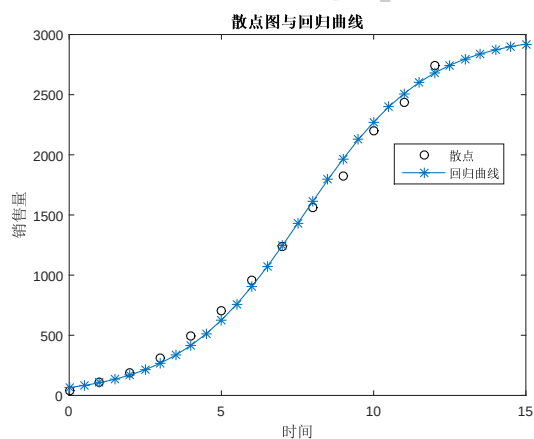


图 13.17: 销售量数据的散点图与回归曲线

```
>> data = xlsread('实验报告数据.xls', '销售量');
>> t = [data(:,2); data(1:6,5)];
>> y = [data(:,3); data(1:6,6)];
>> L = 3000;
>> tt = - t;
>> yy = log(1 ./ y - 1 / L);
>> aa = [ones(size(tt)), tt];
>> [b, bint, r, rint, stats] = regress(yy, aa);
>> b, stats
```

```

b =
    -4.2031
     0.4941
stats =
    1.0e+03 *
    0.0010    1.1508    0.0000    0.0000
>> a = exp(b(1)) * L
a =
    44.8463
>> k = b(2)
k =
    0.4941
>> plot(t, y, 'ko')
>> hold on
>> X = 0:0.5:15;
>> Y = L ./ (1 + a * exp(-k * X));
>> plot(X, Y, 'r-')
>> legend('散点', '回归曲线', 'location', 'best')
>> title('散点图与回归曲线'); xlabel('时间'); ylabel('销售量'); hold off % 结果如图13.17

```

从结果看, $R^2 \approx 1$, $p < 0.05$, 模型 $y = 3000/(1 + 44.8463e^{-0.4941t})$ 拟合较好。

(2) 非线性回归

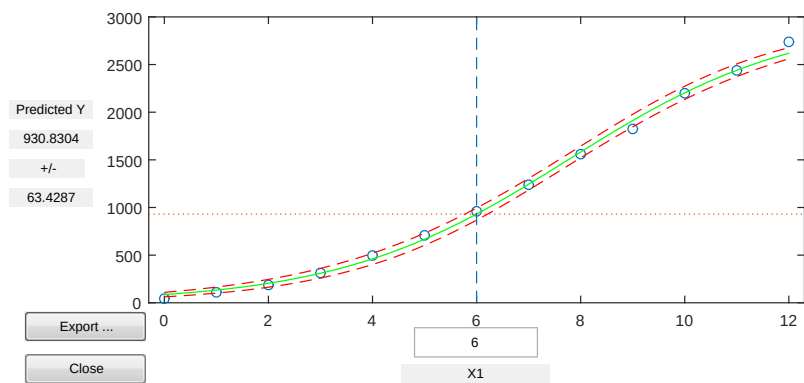


图 13.18: Logistic模型的非线性回归曲线

```

>> beta = [a, k];
>> fun = inline('3000./(1+exp(-beta(1)*exp(beta(2)*t)))', 'c', 't');
>> nlintool(t, y, fun, beta) % 结果如图 13.18
>> [betahat, f, J] = nlinfit(t, y, fun, beta);
>> betahat
betahat =
    33.9369    0.4543
>> betaci = nlparci(betahat, f, J)
betaci =
    26.4573    41.4165

```

0.4265 0.4821

由计算结果可知, 模型为 $y_t = \frac{3000}{1 + 33.9369e^{-0.4543t}}$ 。

(3)

```
>> fun = inline('3000*exp(-c(1)*exp(-c(2)*t))', 'c', 't');
>> nlintool(t, y, fun, beta) % 结果如图 13.19
>> [betahat, f, J] = nlinfit(t, y, fun, beta);
>> betahat
betahat =
    6.4116    0.2968
>> betaci = nlparci(betahat, f, J)
betaci =
    4.6012    8.2219
    0.2578    0.3357
```

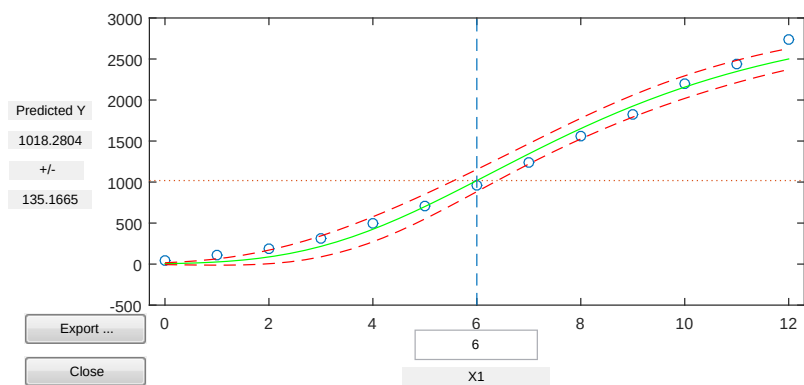


图 13.19: Gompertz模型的非线性回归曲线

从图 13.18 与图 13.19 的对比来看, 由 $62.4287 < 135.1665$ 可知, Logistic 增长曲线模型回归较好。

我的 MATLAB 报告

王保山

第十四章 气候数据的分析报告

注 14.1 本章气候数据的分析将用到多元统计的知识，如方差分析、主成分分析、聚类分析、判别分析等，理论性的推导可查阅相关资料，这里只做数据上的处理。

摘要 本文对全国31个主要城市的气候数据进行了分析，数据由年平均气温、年极端最高气温、年极端最低气温、年均相对湿度、全年日照时数、全年降水量等6个指标组成。本文将通过简单的图示对数据进行探索性分析，讨论带有异常点的几种一元线性回归、逐步回归和多元线性回归，对31个城市通过气候指标按多重聚类、系统聚类和动态聚类的方法进行分类，并对各市气候情况进行排名。

关键词： 气候数据；回归分析；聚类分析；主成分分析

14.1 数据与说明

2007年全国31个城市的气候观测数据如表 14.1 (文件名: 实验报告数据.xls, 表名: 气候数据)。

表 14.1: 气候观测数据

序号	城市	A	B	C	D	E	F
1	北京	14.0	37.3	-11.7	54	2351.1	483.9
2	天津	13.6	38.5	-10.6	61	2156.4	389.7
3	石家庄	14.9	39.7	-7.4	59	2167.7	430.4
4	太原	11.4	35.8	-13.2	55	2174.6	535.4
5	呼和浩特	9.0	35.6	-17.6	47	2647.8	261.2
6	沈阳	9.0	33.9	-23.1	68	2360.9	672.3
7	长春	7.7	35.8	-21.7	58	2533.6	534.2
8	哈尔滨	6.6	35.8	-22.6	58	2359.2	444.1
9	上海	18.5	39.6	-1.1	73	1522.2	1254.5
10	南京	17.4	38.2	-4.5	70	1680.3	1070.9
11	杭州	18.4	39.5	-1.9	71	1472.9	1378.5
12	合肥	17.4	37.2	-3.5	79	1814.6	929.7
13	福州	21.0	39.8	3.6	68	1543.8	1109.6
14	南昌	19.2	38.5	0.5	68	2102.0	1118.5
15	济南	15.0	38.5	-7.9	61	1819.8	797.1
16	郑州	16.0	39.7	-5.0	60	1747.2	596.4
17	武汉	18.6	37.2	-1.5	67	1934.2	1023.2
18	长沙	18.8	38.8	-0.5	70	1742.2	936.4
19	广州	23.2	37.4	5.7	71	1616.0	1370.3
20	南宁	21.7	37.7	0.7	76	1614.4	1008.1
21	海口	24.1	37.9	10.7	80	1669.1	1419.3
22	重庆	19.0	37.9	3.0	81	856.2	1439.2
23	成都	16.8	34.9	-1.6	77	935.6	624.5
24	贵阳	14.9	31.0	-1.7	75	1014.8	884.9
25	昆明	15.6	30.0	0.7	72	2038.6	932.7
26	拉萨	9.8	29.0	-9.8	34	3181.0	477.3
27	西安	15.6	39.8	-5.9	58	1893.6	698.5
28	兰州	11.1	34.3	-11.9	53	2214.1	407.9
29	西宁	6.1	30.7	-21.8	57	2364.7	523.1
30	银川	10.4	35.0	-15.4	52	2529.8	214.7
31	乌鲁木齐	8.5	37.6	-24.0	56	2853.4	419.5

注 14.2 表 14.1 各列分别为: A = 年平均气温($^{\circ}\text{C}$); B = 年极端最高气温($^{\circ}\text{C}$); C = 年极端最低气温($^{\circ}\text{C}$); D = 年均相对湿度(%); E = 全年日照时数(h); F = 全年降水量(mm)。该数据来源于中华人民共和国国家统计局网站。

14.2 统计图示

14.2.1 散点图

以全年日照时数与年平均气温为例。

```
>> n = 3; m = 7;
>> [climateData, txtInformation] = xlsread('实验报告数据.xls', '气候数据'); % 读取气候数据和文字信息
>> txtTitle = txtInformation(1,:); % 得到列名txtTitle
>> cityName = txtInformation(2:end,2); % 得到城市名称cityName
>> yearMeanTemperature = climateData(:, n); % 年平均气温yearMeanTemperature
>> sunDuration = climateData(:, m); % 日照时长sunDuration
>> plot(yearMeanTemperature', sunDuration', 'k.', 'Markersize', 15) % 散点图
>> xlabel(txtTitle(n)); ylabel(txtTitle(m))
>> titleTxt = [char(txtTitle(m)), '与', char(txtTitle(n)), '的散点图'];
>> title(titleTxt) % 结果如图 14.1
```

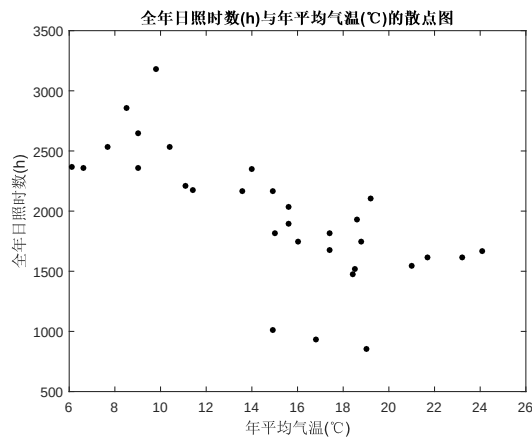


图 14.1: 全年日照时数与年平均气温的散点图

由散点图可知, 全年日照时数与年平均气温大致呈负相关关系, 但图中明显地出现了三个异常点, 现找出这三个点, 在研究回归问题时应对这些异常点进行处理。

```
>> [~, indexCityNum] = sort(sunDuration); % 对sunDuration排序
>> excepCity = cityName(indexCityNum(1:3)) % 取异常点的城市名excepCity
excepCity =
```

‘重庆’
‘成都’
‘贵阳’

14.2.2 散布图矩阵

```
textColumn = {'平均气温', '最高气温', '最低气温', '相对湿度', '日照时数', '年降水量'};  
gplotmatrix(climateData(:, 3:end), [], [], 'k', [], [], 'on', 'hist', textColumn);  
title('散布图矩阵') % 结果如图 14.2
```

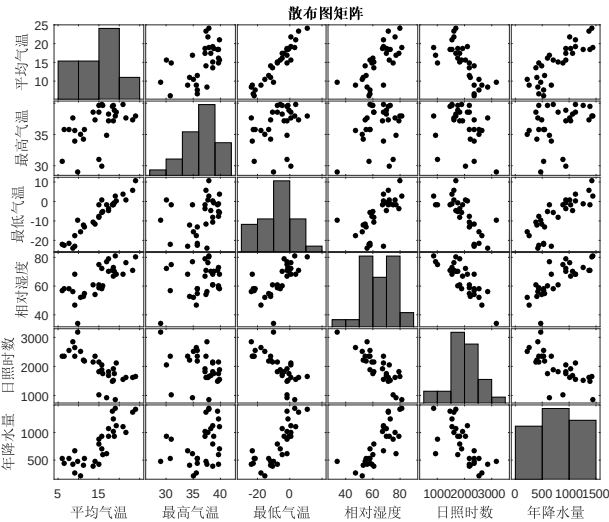


图 14.2: 散布图矩阵

从散布图矩阵来看，平均气温与最低气温、相对湿度、年降水量有着明显的正相关关系；最低气温与相对湿度、年降水量有着明显的正相关关系，而与日照时数则是负相关关系。

14.2.3 轮廓图

以北京、广州、南京、重庆、拉萨为例。

```
textColumn = {'平均气温', '最高气温', '最低气温', '相对湿度', '日照时数', '年降水量'};  
cityIndex = [1, 19, 10, 22, 26];  
x = 1:6;  
plot(climateData(cityIndex, 3:end))  
xx = 5*ones(5,1); yy = climateData(cityIndex, end-1)'+100;
```



```

yy([2,4]) = yy([2,4]) - 200;
text(xx, yy, cityName(cityIndex))
title('轮廓图')
set(gca, 'XTick', x, 'XTickLabel', char(textColumn')) % 设置x轴刻度为x
hold on
plot([1,4,4,1], [-200,-200,200,200], 'k')
grid on
annotation('arrow', [.4,.4], [.25,.32], 'Color', 'k');
h = axes('position', [0.2,0.32,0.35,0.3]);
axes(h)
plot(climateData(cityIndex, 3:end))
title('局部图')
axis([1, 4, -15, 80])
xx = 4*ones(5,1); yy = climateData(cityIndex, 6)';
yy(2) = yy(2) + 2; yy(3) = yy(3) - 2;
text(xx, yy, cityName(cityIndex))
set(h, 'XTick', x, 'XTickLabel', char(textColumn(1:4)))
grid on % 结果如图 14.3

```

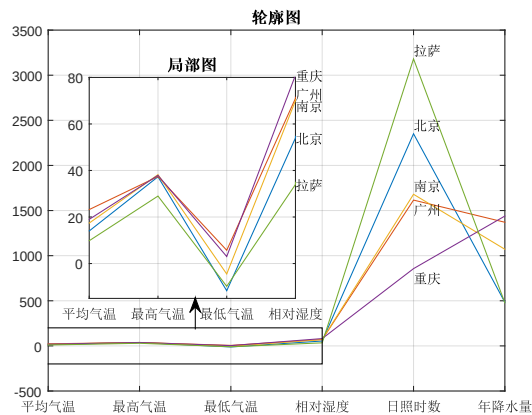


图 14.3: 轮廓图

观察上图，我们可以发现南京与重庆的轮廓大致相同，在聚类时应归为一类。

14.2.4 雷达图

MATLAB中并没有提供绘制雷达图的函数，然而幸运地是，我们可以借助绘制极坐标图 polar 函数实现雷达图的绘制。首先在命令行输入“open polar”，按下“回车”键打开 polar 函数，再在当前目录下新建 polar2 函数文件，将 polar 函数内容复制 polar2 函数文件中(第一行修改为 function hpol = polar2(varargin))，然后将程序中的以下内容注释掉并保存。

```

%           % plot spokes
%           th = (1 : 6) * 2 * pi / 12;
%           cst = cos(th);
%           snt = sin(th);
%           cs = [-cst; cst];
%           sn = [-snt; snt];
%           line(rmax * cs, rmax * sn, 'LineStyle', ls, 'Color', tc, 'LineWidth', 1, ...
%               'HandleVisibility', 'off', 'Parent', cax);

%           % annotate spokes in degrees
%           rt = 1.1 * rmax;
%           for i = 1 : length(th)
%               text(rt * cst(i), rt * snt(i), int2str(i * 30),...
%                   'HorizontalAlignment', 'center', ...
%                   'HandleVisibility', 'off', 'Parent', cax);
%               if i == length(th)
%                   loc = int2str(0);
%               else
%                   loc = int2str(180 + i * 30);
%               end
%               text(-rt * cst(i), -rt * snt(i), loc, 'HorizontalAlignment', 'center', ...
%                   'HandleVisibility', 'off', 'Parent', cax);
%           end

```

接下来，利用polar2函数，编写制雷达图的radar函数，程序如下：

```

function radar(num, nnum, txt, varargin)
% 绘制col个变量的多元数据的雷达图
% Input -num 多元数据，col列矩阵
%       -nnum row行的待处理数据矩阵，默认为num
%       -txt 标注变量名，col维的字符串数组或元胞数组，默认为X1,X2,...
%
% Using it by radar(num), radar(num, nnum) or radar(num, nnum, txt)
%
% Example:
%   num = [57, 73, 14, 58, 174, 176, 39;
%          54, 79, 11, 62, 156, 165, 46;
%          49, 73, 12, 76, 168, 168, 50;
%          44, 89, 13, 62, 178, 182, 44;
%          60, 90, 15, 76, 180, 185, 50;
%          30, 70, 10, 50, 150, 160, 30];
%   nnum = num(1:4,:); radar(num, nnum)
%
% See also PLOT, POLAR.

% Copyright Bao-Shan Wang
% $Revision: 8.5 $ $Date: 2016/10/22 11:56:28 $
col = size(num, 2);           % col列数，即变量个数
if nargin < 1, error('Not enough input arguments.'); end
if nargin < 2, nnum = num; end
if nargin < 3, c = num2str((1:col)'); txt = strcat('X_',c); end
if nargin < 4, tRow = length(txt); txt(tRow:col) = txt(tRow); end
mk = {'-o', '-^', '-s', '-p', '-h', '-d', '-x', '-+', '-*', '-v', '-<', '->'};
row = size(nnum, 1);           % nnum的行数row
ind = mod((1:row)-1, 12) + 1;

```

```

theta = linspace(0, 2*pi, col + 1); % 角度, 个数为col+1
y = theta(1:end-1); % 添加标记时的角度y
theta(1:row,:) = theta(ones(1,row),:); % 填充theta为row×col的矩阵, 也可用repmat
maxNum = max(num); minNum = min(num); % 取极值
maxNum(1:row,:) = maxNum(ones(1,row),:); % 填充maxNum, minNum
minNum(1:row,:) = minNum(ones(1,row),:); % 接下来计算极差归一化后的半径
r = ([nnum, nnum(:,1)] - [minNum, minNum(:,1)]) ./ ...
    ([maxNum, maxNum(:,1)] - [minNum, minNum(:,1)]);
r = 0.8 * r + 0.2; % 将极差归一化后的半径映射到[0.2,1]区间
for i = 1:row
    polar2(theta(i,:), r(i,:), char(mk(ind(i)))); hold on % 绘制变量的雷达线
end
for i = 1:col
    polar2([0,y(i)], [0,1], 'k')
    str = 'right'; if cos(y(i)) > 0, str = 'left'; end
    text(1.2*cos(y(i)), 1.1*sin(y(i)), txt(i,:), 'Color', 'k', ...
        'HorizontalAlignment', str)
end
end

```

最后以北京、广州、南京、重庆、拉萨为例绘制雷达图。

```

textColumn = {'平均气温', '最高气温', '最低气温', '相对湿度', '日照时数', '年降水量'};
cityIndex = [1, 19, 10, 22, 26];
climateDataTemp = climateData(cityIndex, 3:end);
radar(climateData(:,3:end), climateDataTemp, char(textColumn')) % 绘制雷达图
legend(cityName(cityIndex), 'location', 'BestOutside')
title('雷达图') % 结果如图 14.4

```

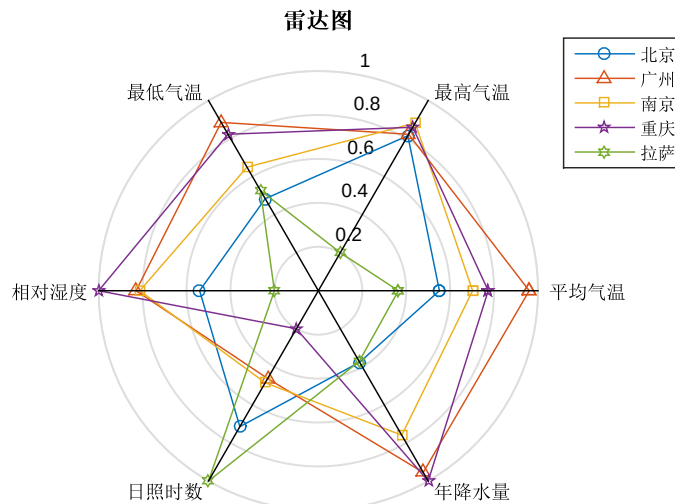


图 14.4: 雷达图

14.2.5 调和曲线图

以北京、南京、重庆、拉萨为例。

```
cityIndex = [1, 10, 22, 26];
climateDataTemp = climateData(cityIndex, 3:end);
andrewsplot(climateDataTemp)
title('安德鲁斯的多元数据图')
str=['$$f(t)=\frac{X(i,1)}{\sqrt{2}}+\dots'];
text(0.3, -3400, str, 'interpreter', 'latex')
text(ones(4,1), climateDataTemp(:,1) / sqrt(2) + climateDataTemp(:,3) + ...
      climateDataTemp(:,5), cityName(cityIndex')) % 结果如图 14.5
```

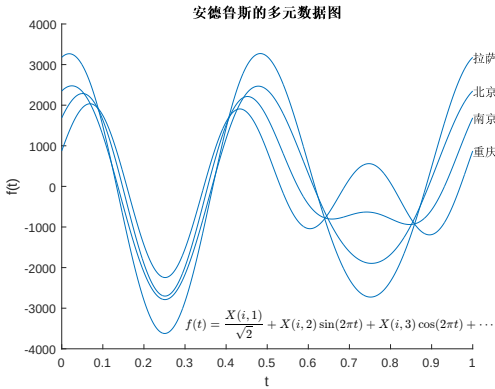


图 14.5: 调和曲线图



图 14.6: 脸谱图

从调和曲线图上看，北京与拉萨的图像比较相似；南京与重庆的图像变化类似。

14.2.6 脸谱图

```
glyphplot(climateData(:,3:8), 'Glyph', 'face', 'ObsLabels', char(cityName')) % 结果如图 14.6
```

从脸谱图上看，若单纯地依据脸的大小（即年平均温度）进行分类，可以将呼和浩特、沈阳、长春、哈尔滨、拉萨、兰州、西宁、银川、乌鲁木齐归为一类；将广州、南宁、海口归为一类。

14.3 一元线性回归分析

以全年日照时数与年平均气温为例。从散点图 14.1 可知，全年日照时数与年平均气温有着明显的线性相关关系。

14.3.1 计算相关系数

```
>> R = corrccoef(sunDuration, yearMeanTemperature) % 计算线性相关系数矩阵R
R =
    1.0000    -0.7095
   -0.7095     1.0000
```

全年日照时数与年平均气温的相关系数为-0.7095，表明全年日照时数与年平均气温负相关。

14.3.2 一元线性回归

```
>> Temperature = [ones(size(yearMeanTemperature,1),1), yearMeanTemperature];
>> [b, bint, r, rint, stats] = regress(sunDuration, Temperature);
>> rcoplot(r, rint)
>> set(gca, 'color', [1, 1, 1]) % 结果如图 14.7
>> sunDurationHat = Temperature * b;
>> head1 = {'系数的估计值', '95%置信下限', '95%置信上限'};
>> reg = [head1; num2cell([b, bint])]
```

系数的估计值	95%置信下限	95%置信上限
[3.1154e+03]	[2.6592e+03]	[3.5716e+03]
[-76.9616]	[-105.9970]	[-47.9261]



图 14.7: 残差图

从结果来看，常数项和回归系数估计值分别为3115.4和-76.962，从而可以得出线性回归方程为

$$\hat{y} = 3115.4 - 76.962x$$

其中，回归系数估计值的95%的置信区间为 $[-105.997, -47.926]$ 。

14.3.3 回归模型的检验

```
>> head2 = {'判定系数', 'F统计量的观测值', '检验的p值', '误差方差的估计值'};
>> tst = [head2; num2cell(stats)]
tst =
    '判定系数'      'F统计量的观测值'      '检验的p值'      '误差方差的估计值'
    [0.5033]        [ 29.3884]        [7.8739e-06]     [1.4689e+05]
```

通过对回归直线进行的显著性检验(原假设与对立假设分别为 $H_0: \beta_1 = 0$ 和 $H_1: \beta_1 \neq 0$), 我们可知检验的 p 值为 $7.8739 \times 10^{-6} < 0.05$, 故在显著水平 $\alpha = 0.05$ 下拒绝原假设 H_0 , 另外我们也可以根据 F 统计量的观测值与临界值 $F_\alpha(1, n-2)$ 的大小关系得出结论。

```
>> finv(0.95,1,29)
ans =
    4.1830
```

显然 $F = 29.3884 > 4.1830 = F_\alpha(1, 29)$, 故全年日照时数与年平均气温的线性关系是显著的。

14.3.4 回归模型的改进

尽管全年日照时数与年平均气温的线性关系是显著的, 但判定系数 $R^2 = 0.5033$, 再从残差图 14.7 上看, 仍然出现了异常点, 它们分别是重庆、成都、贵阳、拉萨, 有必要对这些异常点进行处理。

(1) 剔除法: 将异常点剔除后重新作散点图, 并计算线性相关系数。

```
yearMeanTemperatureNew = yearMeanTemperature(sunDuration < 3000 & sunDuration > 1250);
sunDurationNew = sunDuration(sunDuration < 3000 & sunDuration > 1250);
figure
plot(yearMeanTemperatureNew, sunDurationNew, '*')
xlabel(txtTitle(3)); ylabel(txtTitle(7));
titleTxt = [char(txtTitle(3)), '与', char(txtTitle(7)), '的散点图(剔除后)'];
title(titleTxt) % 结果如图 14.8
```

剔除数据后, 再次计算相关系数。

```
>> R = corrcoef(yearMeanTemperatureNew, sunDurationNew)
R =
    1.0000    -0.8554
   -0.8554     1.0000
```

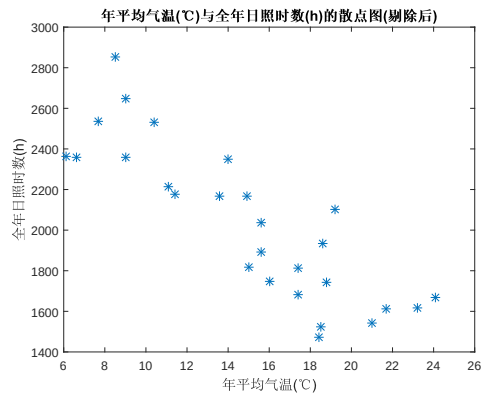


图 14.8: 剔除数据后的散点图



图 14.9: 剔除数据后的残差图

相关系数变为-0.8554, 相关性进一步加强, 在此基础上作线性回归:

```
TemperatureNew = [ones(size(yearMeanTemperatureNew,1),1), yearMeanTemperatureNew];
[b, bint, r, rint, stats] = regress(sunDurationNew, TemperatureNew);
rcoplot(r, rint)
set(gca, 'color', [1, 1, 1])    % 结果如图 14.9
```

结果仍然存在异常点, 再次剔除异常数据。



图 14.10: 再次剔除数据后的残差图

```
>> TemperatureNew = [ones(size(yearMeanTemperatureNew,1),1), yearMeanTemperatureNew];
>> [b, bint, r, rint, stats] = regress(sunDurationNew, TemperatureNew);
>> rcoplot(r, rint)
>> set(gca, 'color', [1, 1, 1])    % 结果如图 14.10
>> b, stats, p = stats(3)
b =
```

```

1.0e+03 *
2.9031
-0.0594
stats =
1.0e+04 *
0.0001    0.0065    0.0000    3.4419
p =
2.5812e-08

```

此时，异常点被剔除，回归方程为

$$\hat{y} = 2903.1 - 59.4x$$

且 p 值远小于0.001，判定系数 $R^2 \approx 1$ ，全年日照时数与年平均气温的线性相关关系更为显著。

(2) 用robustfit函数作稳健回归

robustfit函数采用的是加权最小二乘法模型中的参数，受异常值的影响小，常用于作稳健的线性或广义线性回归。

```

>> [b, stats] = robustfit(yearMeanTemperature, sunDuration);
>> b, p = stats.p
b =
1.0e+03 *
3.0348
-0.0683
p =
1.0e-05 *
0.0000
0.2119

```

最终得到的回归方程为

$$\hat{y} = 3034.8 - 68.3x$$

且 t 检验的 p 值小于0.001，线性回归是显著的。

14.3.5 回归模型的对比

```

figure
plot(yearMeanTemperature, sunDuration, 'bo')
hold on
plot(yearMeanTemperature, 3115.4 - 77.0*yearMeanTemperature, 'm-^')
plot(yearMeanTemperature, 2903.1 - 59.4*yearMeanTemperature, 'r-s')
plot(yearMeanTemperature, 3034.8 - 68.3*yearMeanTemperature, 'k-p')
xlabel(txtTitle(3)); ylabel(txtTitle(7))
titleTxt = [char(txtTitle(3)), '与', char(txtTitle(7)), '的回归曲线图'];
title(titleTxt)
legend('数据点', '原回归曲线', '剔除异常点', '稳健回归法') % 结果如图 14.11

```

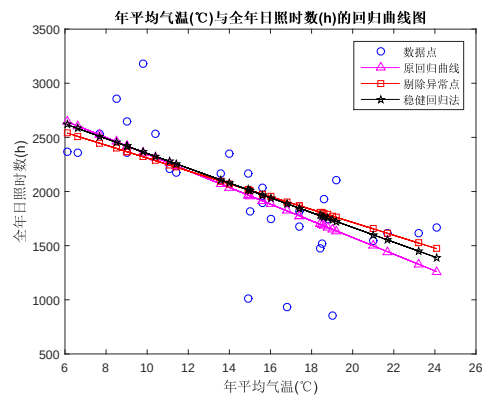



图 14.11: 回归曲线对比图

尽管三种回归方式得出结果并不相同，但它们都表明了“全年日照时数与年平均气温是显著的负相关关系”这一结果。于是我们就得出了一个非常有趣的结果：全年日照时数越高的地区其年平均气温越低。

当然，异常点的存在也表明了一元回归问题的不足之处，例如，海拔会对气温产生影响(一般说来，海拔每升高100m，气温将降低0.6°C)，地形地势也会对气温产生影响(成都位于四川盆地中，盆地的地形地势会阻碍盆地内热量的散失，尽管成都的日照时数不高，但它的年平均气温却很高)。

14.4 多元线性回归分析

14.4.1 逐步回归分析

用stepwisefit函数作逐步回归分析，筛选自变量。

```
>> B = stepwisefit(climateData(:,4:8), yearMeanTemperature) % 筛选自变量
Initial columns included: none
Step 1, added column 2, p=1.2582e-16
Step 2, added column 1, p=2.67108e-06
Step 3, added column 3, p=0.0248771
Step 4, added column 4, p=0.0163676
Final columns included: 1 2 3 4
      'Coeff'      'Std.Err.'      'Status'      'P'
      [ 0.3805]      [ 0.0563]      'In'         [3.6138e-07]
      [ 0.4539]      [ 0.0252]      'In'         [3.2744e-16]
      [ 0.0996]      [ 0.0272]      'In'         [ 0.0011]
      [ 0.0015]      [5.9728e-04]      'In'         [ 0.0164]
      [6.3328e-04]      [8.9727e-04]      'Out'        [ 0.4868]
B =
0.3805
0.4539
0.0996
0.0015
```

0.0006

合适的自变量为： x_1 (年极端最高气温)、 x_2 (年极端最低气温)、 x_3 (年均相对湿度)、 x_4 (全年日照时数)。

14.4.2 稳健回归

考虑到一元线性回归中出现异常点的问题，这里不再使用regress函数，而是使用robustfit函数作稳健回归。

```
>> [b, stats] = robustfit(climateData(:,4:7), yearMeanTemperature);
>> bb = b', pp = stats.p'
bb =
    -3.3647    0.3774    0.4434    0.0807    0.0012
pp =
    0.2969    0.0000    0.0000    0.0036    0.0405
```

稳健回归得出的回归方程为

$$\hat{y} = -3.3647 + 0.3774x_1 + 0.4434x_2 + 0.0807x_3 + 0.0012x_4$$

且回归系数的t检验的p值小于0.05，线性回归是显著的。

14.5 气候分类

为了更为形象地描述各地气候的异同，我们有必要进行聚类分析，将情况类似的城市归为一类，然后再研究它们的气候情况。

14.5.1 多重系统聚类

```
climateDataNewLength = size(climateData, 1);
climateDataNew = zeros(climateDataNewLength, 5);
climateDataNew(:, [1, 3:5]) = climateData(:, [3, 6:8]);
climateDataNew(:, 2) = climateData(:, 4) - climateData(:, 5);
climateDataNew = zscore(climateDataNew); % 标准化
y = pdist(climateDataNew, 'mahalanobis');
z = linkage(y);
julei = cluster(z, 5);
figure; plot(julei, '*'); ylim([0, 6]); % 结果如图 14.12
```

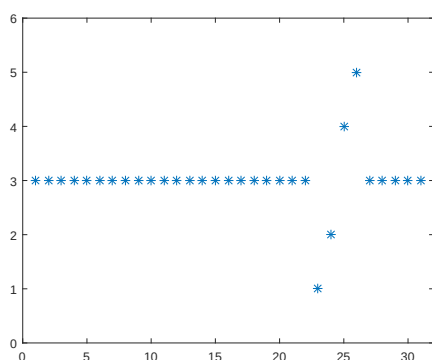


图 14.12: 聚类图

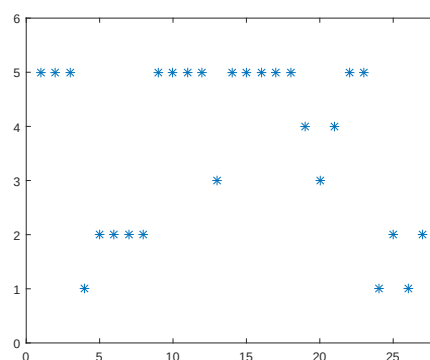


图 14.13: 第三类的聚类图

```
>> fl = cityName(julei ~= 3)
fl =
    '成都',    '贵阳',    '昆明',    '拉萨',
```

从图 14.12 可知，我们将城市分为5类时，第三类过于集中，而其他几类都只有一个点，显然这样的分类是不合理的，我们不妨再对第三类进行分类，而将其他四类(成都、贵阳、昆明、拉萨)看作一类(未处理类)，最后用谱系聚类图对分类进行修正。

```
cityNameNew = cityName(julei == 3)';
climateDataNew1 = climateDataNew(julei == 3);
y = pdist(climateDataNew1, 'mahalanobis');
z = linkage(y);
julei2 = cluster(z, 5);
figure; plot(julei2, '*'); ylim([0, 6]); % 结果如图 14.13
for i = 1:5 % 逐类输出
    str = ['city', num2str(i), '_='cityNameNew(julei2==', num2str(i), ')];
    eval(str)
end
```

输出结果如下：

```
city1 =
    '太原',    '兰州',    '银川',
city2 =
    '呼和浩特',    '沈阳',    '长春',    '哈尔滨',    '西宁',    '乌鲁木齐',
city3 =
    '福州',    '南宁',
city4 =
    '广州',    '海口',
city5 =
    Columns 1 through 7
```

```
      '北京',      '天津',      '石家庄',      '上海',      '南京',      '杭州',      '合肥',  
Columns 8 through 14  
      '南昌',      '济南',      '郑州',      '武汉',      '长沙',      '重庆',      '西安',
```

此时第五类还是太集中，继续对其分类。

```
cityNameNew = cityNameNew(julei2 == 5);  
climateDataNew2 = climateDataNew1(julei2 == 5);  
y = pdist(climateDataNew2, 'mahalanobis');  
z = linkage(y);  
julei3 = cluster(z, 4);  
figure; plot(julei3, '*'); ylim([0, 5]); % 结果如图 14.14  
for i = 1:4 % 逐类输出  
    str = ['city5', num2str(i), '_='cityNameNew(julei3_==', num2str(i), ')];  
    eval(str)  
end
```

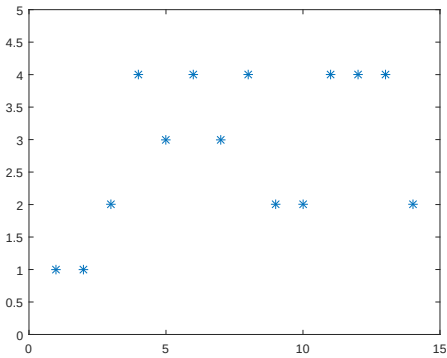


图 14.14: 第三次的聚类图

输出结果如下：

```
city51 =  
      '北京',      '天津',  
city52 =  
      '石家庄',      '济南',      '郑州',      '西安',  
city53 =  
      '南京',      '合肥',  
city54 =  
      '上海',      '杭州',      '南昌',      '武汉',      '长沙',      '重庆',
```

分类结果详见表 14.2。

表 14.2: 分类结果

类别	城市
1	成都、贵阳、昆明、拉萨
2	太原、兰州、银川
3	呼和浩特、沈阳、长春、哈尔滨、西宁、乌鲁木齐
4	福州、南宁
5	广州、海口
6	北京、天津
	石家庄、济南、郑州、西安
	南京、合肥
	上海、杭州、南昌、武汉、长沙、重庆

分类完成后，可用dendrogram函数绘制谱系聚类图(直观地反映分类情况)。

```
y = pdist(climateDataNew, 'mahalanobis');
z = linkage(y);
dendrogram(z, 'orientation', 'left');
textLabel = str2num(get(gca, 'YTickLabel'));
set(gca, 'YTickLabel', cityName(textLabel));    % 结果如图 14.15
```

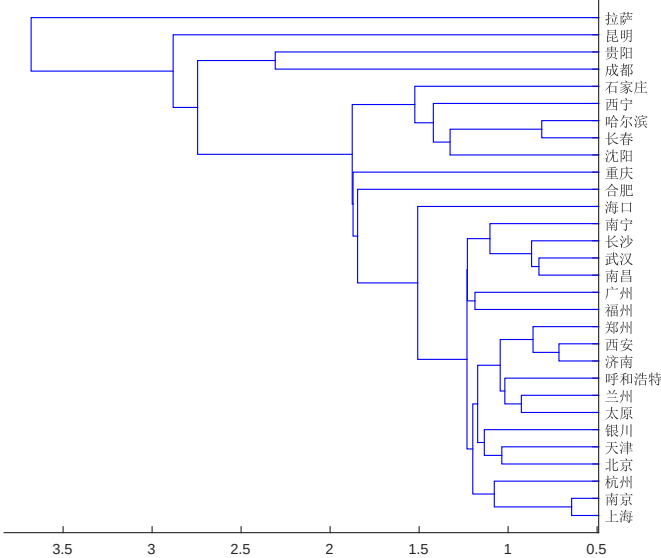


图 14.15: 谱系聚类图

通过图 14.15 可知，{成都，贵阳，昆明，拉萨}四市确实与其他类存在明显的差异。{石家庄，沈阳，长春，哈尔滨，西宁}五市为一类，{南宁，长沙，武汉，南昌}四市为一类，{广州，福州}两市为一类，{郑州，西安，济南，呼和浩特，兰州，太原}六市为一类，{银川，天津，北京}三市为一类，{杭州，南京，上海}为一类。

14.5.2 动态聚类

```
cityName = cityName';
IDX = kmeans(climateData(:, 3:8), 6);
figure; plot(IDX, '*'); axis([0, length(IDX)+1, 0, 7]) % 结果如图 14.16
for i = 1:6
    str = ['city', num2str(i), '_=cityName(IDX==', num2str(i), ')'];
    eval(str)
end
```

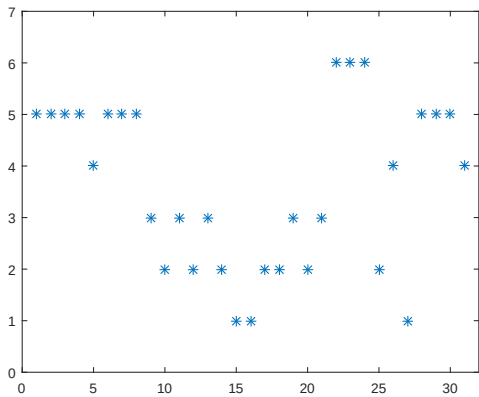


图 14.16: 动态聚类图

输出结果如下：

```
city1 =
    '济南',    '郑州',    '西安',
city2 =
    '南京',    '合肥',    '南昌',    '武汉',    '长沙',    '南宁',    '昆明',
city3 =
    '上海',    '杭州',    '福州',    '广州',    '海口',
city4 =
    '呼和浩特',    '拉萨',    '乌鲁木齐',
city5 =
    '北京',    '天津',    '石家庄',    '太原',    '沈阳',    '长春',    '哈尔滨',    '兰州',    '西宁',    '银川',
city6 =
    '重庆',    '成都',    '贵阳',
```

14.6 气候排名

sort函数是MATLAB内置的排序函数，比如按年降水量为例由多到少的顺序排序，所有单指标排序程序如下：

```
youXiaoClimateData = climateData(:, 3:8);
index = zeros(31, 6);
for i = 1:6
    [~, index(:,i)] = sort(youXiaoClimateData(:,i), 1, 'descend');
end
head3 = [{'城市'}, textColumn];
[head3; cityName', num2cell(index)]
```

通过sort函数，我们可以很方便地得出按某一指标条件下的排序，所有指标的排序结果如表14.3。

表 14.3: 各指标的排序结果

序号	城市	A	B	C	D	E	F	序号	城市	A	B	C	D	E	F
1	北京	21	13	21	22	26	22	17	武汉	15	1	27	2	27	6
2	天津	19	27	19	21	31	21	18	长沙	3	12	3	15	15	23
3	石家庄	20	3	13	12	5	11	19	广州	24	17	15	16	12	16
4	太原	13	16	22	23	7	19	20	南宁	1	4	26	3	16	4
5	呼和浩特	14	9	20	20	30	9	21	海口	2	7	2	7	18	7
6	沈阳	22	11	25	24	29	14	22	重庆	4	8	1	8	10	29
7	长春	18	18	14	9	6	13	23	成都	28	5	28	27	21	1
8	哈尔滨	17	2	18	25	8	10	24	贵阳	30	30	4	29	19	26
9	上海	9	14	9	11	1	17	25	昆明	26	23	30	31	20	8
10	南京	11	15	17	19	28	20	26	拉萨	5	28	5	4	13	3
11	杭州	10	10	23	10	4	18	27	西安	6	6	7	1	9	31
12	合肥	12	21	24	18	3	25	28	兰州	31	24	29	28	11	28
13	福州	23	22	11	6	2	12	29	西宁	7	29	8	30	24	2
14	南昌	16	20	12	13	14	24	30	银川	8	25	6	5	23	5
15	济南	25	31	10	14	25	15	31	乌鲁木齐	29	26	31	26	22	30
16	郑州	27	19	16	17	17	27								

仅用单个指标来排名往往不足以反映整体的排名情况。这里我们将对31个城市的气候情况进行排名，为使排名符合人们的生活习惯，需要对数据进行一些处理得出初步的得分后再进行排名，处理标准如表14.4。

表 14.4: 处理标准

数据列名	处理标准	处理缘由
年平均气温	城市的年平均气温与平均的年平均气温“距离”越近，等分越高	气温越高(或越低)，越不适于人们居住，故得分越低
年极端最高气温	年极端气温差(最高气温-最低气温)，气温差越低，得分越高	年极端气温差越高，气温变化越大，越不适于人们居住，故得分越低
年极端最低气温		
年均相对湿度	年均相对湿度与平均的年均相对湿度“距离”越近，等分越高	年均相对湿度在40% ~ 60%的地区适于人们居住。
全年日照时数	全年日照时数越大，得分越高	全年日照时数越大，表明天气状况越好，故得分越高
全年降水量	全年降水量越多，得分越高	并不是降水量越多越好，但考虑到我国城市普遍面临缺水问题，故降水量多的地区得分高

14.6.1 等权排序

```
climateDataNewLength = size(climateData, 1);
climateDataNew = zeros(climateDataNewLength, 5);
climateDataNew(:, [1,3:5]) = climateData(:, [3,6:8]);
climateDataNew(:, 2) = climateData(:, 4) - climateData(:, 5);
climateDataMin = repmat(min(climateDataNew), climateDataNewLength, 1);
climateDataNew = (climateDataNew - climateDataMin) ./ (...
    repmat(max(climateDataNew), climateDataNewLength, 1) - climateDataMin );
% 初次赋分
climateDataNew(:, [1,3]) = 1 - abs(climateDataNew(:, [1,3]) - ...
    repmat(mean(climateDataNew(:, [1,3])), climateDataNewLength, 1));
climateDataNew(:, 2) = 1 - climateDataNew(:, 2);
% 等权排序
weight = ones(1, 5);
climateScore = climateDataNew * weight';
[climateScoreSort, scoreIndex] = sort(climateScore, 1, 'descend');
% 输出
head4 = {'排名', '城市', '得分'};
score = [head4; num2cell((1:31)'), cityName(scoreIndex)', ...
    num2cell(climateScoreSort / max(climateScoreSort) * 100)]
```

输出结果如下:

```
score =
    '排名'    '城市'    '得分'
    [ 1]    '昆明'    [ 100]
    [ 2]    '南昌'    [95.0716]
    [ 3]    '广州'    [92.2960]
```



```

[ 4]  '武汉',      [92.0502]
[ 5]  '海口',      [91.0477]
[ 6]  '杭州',      [90.4643]
[ 7]  '上海',      [87.6497]
[ 8]  '福州',      [87.3695]
[ 9]  '南京',      [87.2547]
[10]  '长沙',      [85.6322]
[11]  '济南',      [85.1441]
[12]  '贵阳',      [84.1229]
[13]  '重庆',      [83.3418]
[14]  '合肥',      [82.2715]
[15]  '西安',      [81.8655]
[16]  '南宁',      [79.9284]
[17]  '石家庄',    [79.6050]
[18]  '郑州',      [79.3346]
[19]  '北京',      [77.2808]
[20]  '拉萨',      [76.9998]
[21]  '天津',      [76.4194]
[22]  '太原',      [73.1816]
[23]  '沈阳',      [71.5969]
[24]  '兰州',      [71.4859]
[25]  '成都',      [71.0633]
[26]  '长春',      [67.0372]
[27]  '银川',      [66.1543]
[28]  '西宁',      [65.8225]
[29]  '乌鲁木齐',  [65.1210]
[30]  '呼和浩特',  [61.5384]
[31]  '哈尔滨',    [60.8799]

```

14.6.2 主成分排序

```

climateDataCor = corrccoef(climateDataNew);
[v, lamda] = eig(climateDataCor);
lamda = diag(lamda);
figure; plot(flipud(lamda), '*'); xlim([0,6]); title('碎石图'); % 结果如图 14.17
climateScore = climateDataNew*v(:,5);
[climateScoreSort, scoreIndex] = sort(climateScore, 1, 'descend');
head4 = {'排名', '城市', '得分'};
newScore = [head4; num2cell((1:31)'), cityName(scoreIndex)', ...
            num2cell(climateScoreSort / max(climateScoreSort) * 100)]

```

输出结果如下：

```

newScore =
    '排名'    '城市'    '得分'
[ 1]    '昆明'    [ 100]
[ 2]    '拉萨'    [99.5312]
[ 3]    '南昌'    [94.6382]
[ 4]    '海口'    [91.5482]
[ 5]    '武汉'    [89.1732]

```

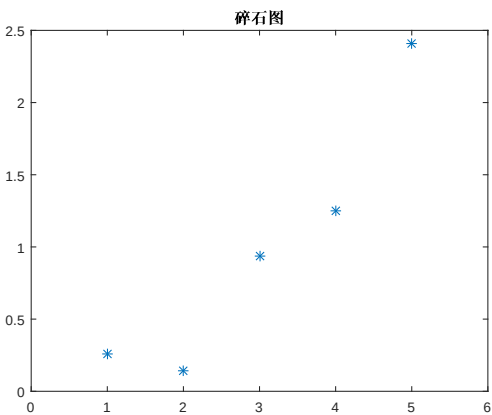


图 14.17: 碎石图

[6]	，广州，	[89.0854]
[7]	，福州，	[81.4538]
[8]	，长沙，	[81.2219]
[9]	，北京，	[79.6444]
[10]	，南京，	[79.2120]
[11]	，杭州，	[79.0383]
[12]	，石家庄，	[78.8702]
[13]	，合肥，	[78.8434]
[14]	，上海，	[78.2262]
[15]	，济南，	[77.3919]
[16]	，南宁，	[77.1810]
[17]	，西安，	[76.8307]
[18]	，天津，	[76.0728]
[19]	，兰州，	[75.6464]
[20]	，银川，	[75.3036]
[21]	，乌鲁木齐，	[75.1971]
[22]	，太原，	[74.7425]
[23]	，沈阳，	[73.6997]
[24]	，长春，	[73.5355]
[25]	，郑州，	[73.2444]
[26]	，呼和浩特，	[73.1140]
[27]	，西宁，	[72.8594]
[28]	，贵阳，	[69.9500]
[29]	，重庆，	[67.1182]
[30]	，哈尔滨，	[66.2409]
[31]	，成都，	[58.1817]

14.7 结论与展望

本文对全国31个主要城市的气候数据进行了分析，通过简单的图示对数据进行探索性分析，讨论带有异常点的几种一元线性回归、逐步回归和多元线性回归，对31个城市通过气候指标按多重聚类、系统聚类和动态聚类的方法进行分类，并对各市气候情况进行排名。

全年日照时数与年平均气温的回归方程为 $\hat{y} = 3034.8 - 68.3x$, 可知年平均气温与全年日照时数的方程为 $\hat{x} = 44.4334 - 0.0146y$ 。

年平均气温与年极端最高气温、年极端最低气温、年均相对湿度、全年日照时数之间的回归方程为 $\hat{y} = -3.3647 + 0.3774x_1 + 0.4434x_2 + 0.0807x_3 + 0.0012x_4$ 。

气候分类时动态聚类的效果(与地理上的区域划分相一致)比较好, 可分为以下8类:

- (1) 西南地区{重庆, 成都, 贵阳, 昆明};
- (2) 东南沿海地区{上海, 南京, 杭州, 福州, 广州, 南宁, 海口};
- (3) 中原地区{济南, 郑州, 西安};
- (4) 华中地区{合肥, 南昌, 武汉, 长沙};
- (5) 西部边疆地区{呼和浩特, 拉萨, 乌鲁木齐};
- (6) 华北地区{北京, 天津, 石家庄, 太原};
- (7) 东北地区{沈阳, 长春, 哈尔滨};
- (8) 西部地区{兰州, 西宁, 银川}。

在气候排名中, 中部地区和南方地区得分较高, 而北方东北地区和西北地区得分较低。其中, 昆明市得分最高, 哈尔滨与成都市得分最低。

本文还有一些未完成的工作和不足之处。

(1) 数据的缺乏。本文虽然涉及了气温、降雨量、日照等数据, 但仍缺乏一些关键性的数据, 例如, 各市的经纬度、海拔、年平均风速、常驻人口等数据。

(2) 一元线性回归中, 为操作方便, 作了全年日照时数与年平均气温的回归方程, 而没有考虑全年日照时数与年平均气温之间的因果关系, 负相关的结论也与常理相悖, 因而讨论它们之间的因果关系就显得很重要, 而本文并没有展开讨论。

(3) 气候分类应用的方法太少。聚类分析是比较主观的, 选用不同的距离、不同的方法, 得到的结果也会不同, 因而可以采用不同的方法寻找聚类的共同点, 而本文却只用了系统聚类和动态聚类。

(4) 在气候排名中, 利用主成分进行评价, 但评价时得到的最大特征值对应的特征向量中元素有正有负, 变量 $X_1, X_2, \dots, X_5$ 中有一部分正相关, 有一部分负相关或不相关, 因而用第一主成分作为排序评估指数是不合适的, 需用传统的专家评估和第一主成分评估法相结合修正评估结果, 而本文只是寻找第一主成分进行了排名。

(5) 本文没有使用判别分析对各市的气候情况进行归类, 没有使用方差分析去判断南方与北方的气温、降雨量、相对湿度等是否有显著性差异。

(6) 本文还有一些不清楚的问题, 如数据标准化能否用极差归一化、最大值归一化替代, 回归分析中剔除法是否有效, 若 x 轴出现异常点如何剔除, 剔除的标准如何给出, 判别分析如何应用等。

我的 MATLAB 报告

王保山

第四部分

数学实验篇

我的 MATLAB 报告

王保山

第十五章 古典密码

15.1 加法密码

密码学是古老神秘的学科之一，英文为Cryptography，它源于希腊文字，意为“秘密+书写”，旨在将信息传递给己方的接受者而又防止敌方知道信息的内容，主要包括密文(加密后或解密前的文字)、明文(加密前或解密后的文字)和密钥(加密或解密准则)等，密码学的相关概念如表 15.1 所示。

表 15.1: 密码学的相关概念

概念	说明
明文	需要采用某种方法对其进行变换来隐蔽它所载荷的信息或字符串
密文	明文经过加密过程的变换所得的消息或字符串
密钥	加密变换所使用的参数
加密过程	将明文变换成另一种不能被非授权者所理解的隐蔽信息的信息或字符串的过程
加密变换	将明文变为密文的变换
解密变换	将密文变为明文的变换

图 15.1 展示了加密解密过程。

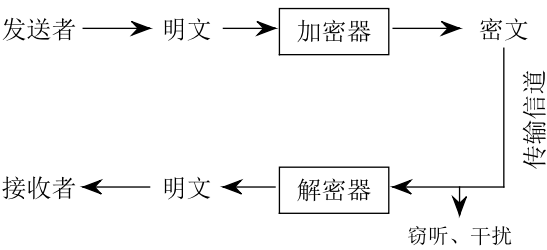


图 15.1: 加密解密过程

密码学在军事上有着非常重要的地位，凯撒大帝最先使用了加法密码。加法密码，就是将字母按

照字母表的顺序对应起来，以凯撒大帝的密码为例，如表将明文字母加密， $K = H + 3$ ， $h = e + 3$ ，便得到了密文，反过来，将密文字母减3即可得到明文。

表 15.2: 凯撒大帝的密码(密钥为3)

明文	He	is	the	ruler	of	ancient	Roma
密文	Kh	lv	wkh	uxohu	ri	dqflhqw	Urpd

MATLAB在进行字母与数字的四则运算时先将字母转化成ASCII码再与数字进行运算。

例 15.1 字母与数字四则运算

```
>> 'a' + 3 % double('a') = 97
ans =
    100
>> 'A' + 'z' * 2
ans =
    309
>> 'ab' + 2
ans =
    99    100
```

注 15.1 英文小写字母的ASCII码的范围是[97,122]，大写字母的ASCII码的范围是[65,90]，两者相差32。

例 15.2 编写加法密码的加密解密程序，并用'China'验证加密和解密过程。

```
function ctxt = cipher(txt, key, mode)
% 加法密码加密解密程序
% Input -txt 待加密或解密的文字，字符串 -key 密钥，数字
%       -mode 处理模式：1-加密；0-解密，默认为加密
% Output -ctxt 加密或解密的结果，字符串
% Example
%   ctxt = cipher('China', 3, 1)
%   txt = cipher(ctxt, 3, 0)
if nargin == 2, mode = 1; end
mode = sign(mode - 0.5);
n = length(txt);
for i = 1:n
    if(txt(i) >= 'A' && txt(i) <= 'Z')
        txt(i) = mod(txt(i) + mode * key - 'A', 26) + 'A';
    elseif(txt(i) >= 'a' && txt(i) <= 'z')
        txt(i) = mod(txt(i) + mode * key - 'a', 26) + 'a';
    end
end
ctxt = char(txt);
```


验证

```
>> ctxt = cipher('China', 3, 1)
ctxt =
Fklqd
>> txt = cipher(ctxt, 3, 0)
txt =
China
```

例 15.2 的函数使用了for循环的方式逐个字母进行加密或解密，这种方式可以同时应对大写字母、小写字母进行操作，但耗时长、速度慢。其实加法密码也可以通过向量化实现，尽管向量化程序只能实现对纯大写字母或纯小写字母的字符串的操作，但字母大小写并不影响我们对信息的理解。MATLAB 中lower、upper函数可以将字母转换成小写字母、大写字母，表 15.3 列出了与字符串有关的操作指令。

表 15.3: 与字符、字符串有关的操作指令

指令	说明
length/size	字符串长度/尺寸
char/double	转换成字符串/ASCII码
strcat/strrep	合并/替换字符串，区分大小写
strcmp/strcmpi	字符串是否完全相等，后者忽略大小写
strncmp/strncmpi	比较字符串前n个字符是否相等，后者忽略大小写
isspace/isletter	字符串中是否含有格式字符(空格等)/英文字母
isstrprop	字符串中每一个字符是否属于指定范围
strfind(str, pat)	查找str中是否有pat
strmatch(pat, str)	检查pat是否和str最左侧部分一致
findstr(s1, s2)	查找s1和s2中较短字符串在较长字符串中出现的位置
strtok(s, c)	返回s中由c指定的字符串前的部分和之后的部分
blanks(n)	创建有n个空格的字符串
deblank/strtrim	裁切字符串的尾部空格/首尾空格
lower/upper	将字符串中的字母转换成小写/大写

例 15.3 编写加法密码的向量化程序并验证。

```
function ctxt = ciphers(txt, key, mode)
% 加法密码加密解密的向量化程序
% Input -txt 待加密或解密的文字，字符串 -key 密钥，数字
%       -mode 处理模式：1-加密；0-解密，默认为加密
% Output -ctxt 加密或解密的结果，字符串
% Example
```

```

%   ctxt = ciphers('China', 3, 1)
%   txt = ciphers(ctxt, 3, 0)
if nargin == 2, mode = 1; end
mode = sign(mode - 0.5);
txt = lower(txt);
txt(~isstrprop(txt, 'lower')) = []; % 删除非小写字母的字符
ctxt = mod(txt + mode * key - 'a', 26) + 'a';
ctxt = char(ctxt);

```

验证

```

>> ctxt = ciphers('C_hin!a', 3, 1)
ctxt =
fklqd
>> txt = ciphers(ctxt, 3, 0)
txt =
china

```

15.2 维吉尼亚密码

维吉尼亚密码实际上是分组的加法密码，是加法密码的改进，它通过分组的形式避免传统加法密码不改变字母频率的缺点，同时也增加了破译的难度。就像我们军训的一个科目——一路纵队向二路纵队走时通过“1212...”报数一样，我们可以将一列字符串看作一路纵队，按照奇偶性分为两组，分别应用不同的密钥进行加法加密，再按原来的顺序排列成一列字符串，这样我们就完成了维吉尼亚密码的加密过程。

例 15.4 编写维吉尼亚密码的加密解密程序，并用凯撒大帝的密码进行验证。

```

function ctxt = vigenere(txt, key, mode)
% 维吉尼亚加密解密程序
% Input  -txt 待加密或解密的文字，字符串 -key 密钥，字母
%         -mode 处理模式：1-加密；0-解密，默认为加密
% Output -ctxt 加密或解密的结果，字符串
% Example
%   txt = 'He is the ruler of ancient Roma';
%   ctxt = vigenere(txt, 'red', 1)
%   txt = vigenere(ctxt, 'red', 0)
if nargin == 2, mode = 1; end
mode = sign(mode - 0.5);
n = length(txt); m = length(key);
for i = 1:n
    t = mod(i - 1, m) + 1;
    if(txt(i) >= 'A' && txt(i) <= 'Z')
        txt(i) = mod(txt(i) + mode * keyfun(key(t)) - 'A', 26) + 'A';
    elseif(txt(i) >= 'a' && txt(i) <= 'z')
        txt(i) = mod(txt(i) + mode * keyfun(key(t)) - 'a', 26) + 'a';
    end
end

```

```

    end
end
ctxt = char(txt);

% 子函数，用于求密钥的值
function y = keyfun(x)
if x >= 'A' && x <= 'Z'
    y = x - 'A';
elseif x >= 'a' && x <= 'z'
    y = x - 'a';
else
    error('密码错误!!!')
end
end

```

验证

```

>> txt = 'He is the ruler of ancient Roma';
>> ctxt = vigenere(txt, 'red', 1)
ctxt =
Yi zw klh vxciu si eqtmhex Ispr
>> txt = vigenere(ctxt, 'red', 0)
txt =
He is the ruler of ancient Roma

```

例 15.5 编写维吉尼亚密码的向量化程序，并验证。

```

function ctxt = vigeneres(txt, key, mode)
% 维吉尼亚加密解密向量化程序
% Input -txt 待加密或解密的文字，字符串 -key 密钥，小写字母
%        -mode 处理模式：1-加密；0-解密，默认为加密
% Output -ctxt 加密或解密的结果，字符串
% Example
% txt = 'He is the ruler of ancient Roma';
% ctxt = vigeneres(txt, 'red', 1)
% txt = vigeneres(ctxt, 'red', 0)
if nargin == 2, mode = 1; end
mode = sign(mode - 0.5);
txt = lower(txt);
txt(~isstrprop(txt, 'lower')) = []; % 删除非小写字母的字符
n = length(txt); m = length(key);
txt(floor(n/m)*m+1:ceil(n/m)*m) = txt(end); % 填充字母
txt = reshape(txt, m, []); % 将txt分组
key = mode * repmat(key - 'a', 1, ceil(n/m)); % 扩充key为m×ceil(n/m)的矩阵
txt = mod(txt + key - 'a', 26) + 'a'; % 逐行加密
ctxt = char(txt(1:n)); % 截取前n个字母

```

验证

```
>> txt = 'He is the ruler of ancient Roma';
>> ctxt = vigeneres(txt, 'red', 1)
ctxt =
yiljxkvvxciufjdegivrwispr
>> txt = vigeneres(ctxt, 'red', 0)
txt =
heistherulerofancientroma
```

注 15.2 `repmat(a, m, n)`将向量 a 当作元素拓展成 $m \times n$ 的数组, 例子如下

```
>> A = repmat(1:4, 3, 2)
A =
     1     2     3     4     1     2     3     4
     1     2     3     4     1     2     3     4
     1     2     3     4     1     2     3     4
```

15.3 Hill₂密码

Hill密码是一种简单的加密方法, 与前两者不同的是, Hill密码的密钥是一个矩阵(加密矩阵)。Hill₂密码的加密矩阵是一个二阶矩阵, 它要求先将26个英文字母进行编号(或将明文字母与0~25建立1-1的映射关系), 将明文或密文中的字母两两分组, 利用事先给出的字母计算出加密矩阵, 然后再通过矩阵乘法进行加密或解密。

Hill₂密码的关键就是求加密矩阵及其逆矩阵, 这样我们就不可避免地用到模运算(主要是mod 26运算)。设 a, b 是两个整数, 若 $a - b = km, k \in \mathbf{Z}$, 则称 a 模 m 等价于 b , 记作 $a \equiv b \pmod{m}$ 。模运算中有下列恒等式:

$$[a \odot b] \bmod m = [(a \bmod m) \odot (b \bmod m)] \bmod m,$$

其中 $\odot$ 是 $\{+, -, \times\}$ 之一。记 $\mathbf{Z}_m = \{0, 1, \dots, m-1\}$ 为模 m 的剩余集。设 $a \in \mathbf{Z}_m$, 若存在 $b \in \mathbf{Z}_m$ 使得 $ab \bmod m = 1$, 称 a 有模 m 倒数, 记作 $b = a^{-1} \bmod m$ 。

对应到矩阵上就是 $A = (a_{ij})_{n \times n}$ 且 $a_{ij} \in \mathbf{Z}_m$, 若存在 $B = (b_{ij})_{n \times n}, b_{ij} \in \mathbf{Z}_m$, 使得 $AB \bmod m = E$, 其中 E 为单位阵, 则称 B 为 A 的模 m 逆矩阵, 记作 $B = A^{-1} \bmod m$ 。

矩阵 A 模 m 可逆的等价条件为 $|A|$ 与 m 无公共因子。在此条件下, 其运算满足 $A^{-1} \bmod m = (|A|^{-1} \bmod m) \cdot (A^* \bmod m)$, 其中 A^* 为 A 的伴随矩阵。

例 15.6 编写一个函数, 判断是否存在 n 关于模 m 的倒数, 若存在, 请求出。

```
function r = modrecip(n, m)
% 求正整数n关于模m的倒数r
% Input  -[n,m] 均为正整数
% Output -r n的模m倒数
```

```

if(gcd(n, m) ~= 1), error('模倒数不存在! '); end
for i = 1:m-1
    if(mod(i * n, m) == 1), r = i; break; end
end

```

验证

```

>> r = modrecip( 3, 26)
r =
    9
>> r = modrecip(13, 26)
Error using modrecip (line 5)
模倒数不存在!

```

例 15.7 编写求模 m 意义下逆矩阵的函数 $B = \text{modinv}(A, m)$ ，并求解下列矩阵在模26意义下的逆矩阵。

$$(1) \begin{pmatrix} 5 & 8 \\ 3 & 7 \end{pmatrix} \quad (2) \begin{pmatrix} 5 & 3 \\ 7 & 11 \end{pmatrix} \quad (3) \begin{pmatrix} 2 & 3 \\ 7 & 11 \end{pmatrix} \quad (4) \begin{pmatrix} 7 & 3 \\ 1 & 10 \end{pmatrix}$$

```

function B = modinv(A, m)
% 计算方阵A在模m意义下的逆矩阵B
d = round(det(A)); % 四舍五入取整
r = modrecip(d, m); % 求A的行列式关于模m的倒数，见例 15.6
B = round(mod(d * r * inv(A), m));

```

求模逆矩阵

```

>> A = [5 8; 3 7]; B = modinv(A, 26)
B =
    3    4
   21   17
>> A = [5 3; 7 11]; B = modinv(A, 26)
Error using modrecip (line 5)
模倒数不存在!
Error in modinv (line 4)
r = modrecip(d, m); % 求A的行列式关于模m的倒数，见例 15.6
>> A = [2 3; 7 11]; B = modinv(A, 26)
B =
   11   23
   19    2
>> A = [7 3; 1 10]; B = modinv(A, 26)
B =
   18    5
   19   23

```

在求得加密解密矩阵后，便可以进行加密解密Hill₂密码，例 15.8 详细介绍了解密过程。

例 15.8 一段密文为 *goqbxcbuglosnfal*, 据分析是 $Hill_2$ 密码。明文字母表值为自然顺序 (*a* 对应 1, *b* 对应 2, $\dots$, *y* 对应 25, *z* 对应 0), 估计开始 4 个字母表示 *dear*, 试编写解密函数求明文。

```
function ctxt = hill2(txt, key)
% Hill2 密码的解密程序
% Input -txt 密文, 字符串 -key 密钥, 四个字母组成的字符串
% Output -ctxt 解密后的明文
n = length(txt); m = length(key);
if m ~= 4, error('密钥错误!'); end
%% 字符处理
txt = lower(txt); % 转化为小写字母
txt(~isstrprop(txt, 'lower')) = []; % 删除非小写字母的字符
txt(n + mod(n, 2)) = txt(n); % m 为奇时扩充为偶数
%% 转化为数值矩阵
txt = mod(txt - 'a' + 1, 26);
txt = reshape(txt, 2, []);
key = mod(key - 'a' + 1, 26);
key = reshape(key, 2, []);
%% 求模26逆矩阵
mod26 = modinv(txt(:, [1, 2]), 26); % 见例 15.7
%% 求解密矩阵
jiemi = mod(key * mod26, 26);
%% 解密
txt = mod(jiemi * txt, 26);
ctxt = char(txt(1:n) + 96);
```

解密

```
>> txt = 'goqbxcbuglosnfal'; key = 'dear';
ctxt = hill2(txt, key)
ctxt =
dearmacgodforbid
```

故明文为: “Dear mac god forbid”。

由于 $Hill_2$ 密码先将明文分组(两个一组), 再按组转换成密文。这种处理方式使得同一字母在不同组中所对应的密码不同, 增加了破译难度, 但由于要求事先给出几个明文字母, 它的破译可以通过穷举法实现(尽管穷举法计算量大)。

第十六章 图与网络

16.1 图论初步

图论自然是关于图的理论。在图论中，图是由一个三元数组(顶点集 V ，边集 E ，定点集与边集之间的关联函数 Φ)组成的，其中 $\Phi(e_i) = (v_m, v_n), e_i \in E, v_m, v_n \in V$ 。有时为了便于书写和表述，称上式中的 (v_m, v_n) 为相邻。如图 16.1 所示，定点 a 与 b 之间有边相连，则称 a 与 b 相邻，即 $\Phi(1) = (a, b)$ 。

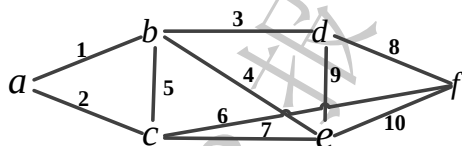


图 16.1: 图(边上的数字为边的序号)

图片和三元数组显然都无法有效地被计算机理解，因此还需引出两个重要的矩阵。

一个是关联矩阵 $M = [m_{ij}]$ ，若边 e_i 与定点 v_j 相关联，则 $m_{ij} = 1$ ，否则 $m_{ij} = 0$ ；另一个是邻接矩阵 $A = [a_{ij}]$ 。若定点 v_i 与 v_j 相邻，则 $a_{ij} = 1$ ，否则 $a_{ij} = 0$ 。图 16.1 的关联矩阵 M 和邻接矩阵 A 分别为

$$M = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}, A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}.$$

一般说来，邻接矩阵占用内存少，而且它更直观地表示了顶点之间的相邻情况而淡化了边的次序，其实际应用范围要比关联矩阵更广。

例 16.1 编写函数实现关联矩阵与邻接矩阵之间的相互转化并用图 16.1 中的两个矩阵进行验证。

```

function W = tchange(F, mode)
% 关联矩阵M与邻接矩阵A的转化函数
% Input -F 待转化矩阵 -mode 转化模式: 1-M2A; 0-A2M
% Output -W 转化结果
% Example
% F = zeros(6, 10); mode = 1;
% F(1,[1,2]) = 1; F(2,[1,3:5]) = 1;
% F(3,[2,5:7]) = 1; F(4,[3,8,9]) = 1;
% F(5,[4,7,9,10]) = 1; F(6,[6,8,10]) = 1;
% W = tchange(F, mode)
if mode
    [n, m] = size(F); % 顶点数n和边数m
    W = zeros(n);
    for i = 1:m
        index = find(F(:,i) ~= 0); % 寻找与第i条边相关联的顶点序号
        W(index([1,2]), index([2,1])) = 1;
    end
    W(eye(n) == 1) = 0; % 对角元赋值为0
else
    n = size(F, 1); % 顶点数
    m = sum(sum(F))/2; % 确定边数
    W = zeros(n, m);
    k = 0; % 初始为第0条边
    for i = 1:n
        for j = i:n
            if F(i,j) ~= 0
                k = k + 1; % 边数+1
                W([i,j], k) = 1; % 关联矩阵赋值
            end
        end
    end
end
end

```

验证

```

>> F = zeros(6, 10);
>> F(1, [1,2]) = 1; F(2, [1,3:5]) = 1; F(3, [2,5:7]) = 1;
>> F(4, [3,8,9]) = 1; F(5, [4,7,9,10]) = 1; F(6, [6,8,10]) = 1;
>> W = tchange(F, 1)
W =
     0     1     1     0     0     0
     1     0     1     1     1     0
     1     1     0     0     1     1
     0     1     0     0     1     1
     0     1     1     1     0     1
     0     0     1     1     1     0

>> F = tchange(W, 0)
F =
     1     1     0     0     0     0     0     0     0     0
     1     0     1     1     1     0     0     0     0     0
     0     1     1     0     0     1     1     0     0     0
     0     0     0     1     0     0     0     1     1     0
     0     0     0     0     1     1     0     1     0     1
     0     0     0     0     0     0     1     0     1     1

```


注 16.1 注意 F 矩阵的生成方法，它与逐个元素输入的方式相比，有着准确、易修改等优点。

将邻接矩阵和关联矩阵应用到实际生活中是没有多大意义的，因为人们除了考虑两点之间是否有路相连外，还会考虑距离、费用、流量、时间等因素。赋权图就是将上述因素当作边上的权值(距离大小、费用多少等)的图，相应的邻接矩阵中值为1的位置替换成权值，值为0的位置替换为Inf(无穷大，表示不可达)，还有为了表示可达的方向性的有向图。

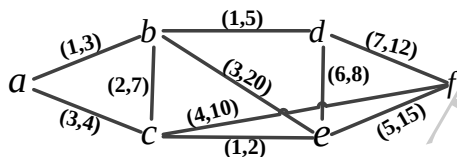


图 16.2: 赋权的有向图

图 16.2 是距离与时间的赋权图，相应的邻接矩阵为

$$A_{\text{距离}} = \begin{bmatrix} 0 & 1 & 3 & \text{Inf} & \text{Inf} & \text{Inf} \\ \text{Inf} & 0 & 2 & 1 & 3 & \text{Inf} \\ \text{Inf} & \text{Inf} & 0 & \text{Inf} & 1 & 4 \\ \text{Inf} & \text{Inf} & \text{Inf} & 0 & 6 & 7 \\ \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & 0 & 5 \\ \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & 0 \end{bmatrix}, \quad A_{\text{时间}} = \begin{bmatrix} 0 & 3 & 4 & \text{Inf} & \text{Inf} & \text{Inf} \\ \text{Inf} & 0 & 7 & 5 & 20 & \text{Inf} \\ \text{Inf} & \text{Inf} & 0 & \text{Inf} & 2 & 10 \\ \text{Inf} & \text{Inf} & \text{Inf} & 0 & 8 & 12 \\ \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & 0 & 15 \\ \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & 0 \end{bmatrix}.$$

例 16.2 编写绘制邻接矩阵的网络图程序。

```
function adjmatplot(W)
% 绘制邻接矩阵W(赋权或不赋权皆可)的图像
if nargin < 2, r = 0.08; end
if nargin < 3, rd = pi/18; end
if nargin < 4, s = 'b'; end
n = size(W, 1);
t = linspace(0, 2*pi, n+1);
x = sin(t); y = cos(t);
txt = strcat('v_{', num2str((1:n)'), '}');
text(1.1*x(1:n)-0.05, 1.1*y(1:n), txt, 'FontSize', 15, 'interpreter', 'latex')
hold on
for i = 1:n
    plot(x(i), y(i), 'ko'); % 绘制每个顶点
    nadj = W(i,:) ~= inf & W(i,:) ~= 0;
    x0 = x(nadj); y0 = y(nadj);
    for j = 1:length(x0)
        plotlinewitharrow(x(i), x0(j), y(i), y0(j), r, rd, s) % 绘制带箭头的直线
    end
    text((x(i)+x0)/2, (y(i)+y0)/2, num2str(W(i,nadj)))
end
axis equal off % 等刻度显示并隐藏坐标轴
```

其中绘制带箭头直线的函数plotlinewitharrow编写如下：

```
function plotlinewitharrow(x0, x, y0, y, r, rd, s)
% 绘制带箭头的直线
% Input -[x0,y0] 直线的起始点 -[x,y] 直线的终点
%      -r 箭头的长度 -rd 箭头的开角 -s 颜色线条标识设置
if nargin < 5, r = 0.08; end
if nargin < 6, rd = pi/18; end
if nargin < 7, s = 'b'; end
plot([x, x0], [y, y0], s); % 绘制直线
hold on
AB = sqrt((x - x0) ^ 2 + (y - y0) ^ 2);
AD = abs(y - y0); DB = abs(x - x0);
sinD = AD / AB; cosD = DB / AB;
sinC = sin(rd); cosC = cos(rd);
sinC1 = sinD * cosC + sinC * cosD;
cosC1 = cosD * cosC - sinC * sinD;
sinC2 = sinD * cosC - sinC * cosD;
cosC2 = cosD * cosC + sinC * sinD;
plot([x + r * cosC2 * locsign(x0, x), x, x + r * cosC1 * locsign(x0, x)], ...
     [y + r * sinC2 * locsign(y0, y), y, y + r * sinC1 * locsign(y0, y)], ...
     s); % 绘制箭头
end

function flag = locsign(x0, x)
flag = 1;
if x0 < x, flag = -1; end
end
```

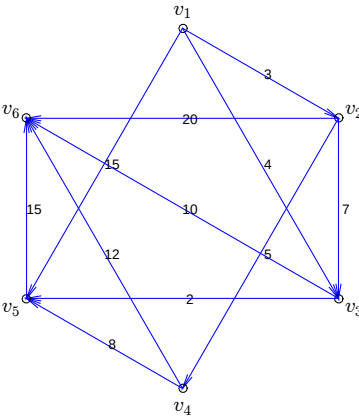


图 16.3: 邻接矩阵的网络图

绘制邻接矩阵的网络图

```
W = inf(6); % 生成6×6的Inf矩阵
W(1,[2,3,5]) = [3,4,15];
```

```

W(2,[3,4,6]) = [7,5,20];
W(3,[5,6])   = [2,10];
W(4,[5,6])   = [8,12];   W(5,6) = 15;
adjmatplot(W) % 结果如图 16.3

```

16.2 最小生成树算法

在图 G 中, 如果一个非空序列 W 是由顶点和边组成的交替项, 且每条边的前后是其关联的顶点, 则称 W 为一个途径。若 W 中每个顶点都互不相同, 则称 W 是图 G 的一条路。我们把任意两点都有路相连的图称为连通图, 连通的无圈图 G 称为树。树 G 中任意两点都有唯一的路相连。

生活中, 我们经常会遇到这样的问题: 修建一条输油管道, 使得管道通过所有站点且总距离最短。这类问题的解决要用到最小生成树算法—Kruskal算法。

算法 16.1 Kruskal算法

Algorithm 15: KRUSKAL(D)

Input: 赋权邻接矩阵 D

Output: 最小生成树顶点序号矩阵 T 和最小生成树的总权值 c

- 1 把赋权图 G 中的所有边按照权的非减次序排列
 - 2 按第1步排列的次序检查 G 中的每一条边, 如果这条边与已得到的边不产生圈, 则取这一条为树的一部分
 - 3 若已取到 $n - 1$ 条边, 算法终止
-

具体的MATLAB程序为

```

function [T, c] = kruskal(d, mode)
% Kruskal 最小生成树算法
% Input -d 赋权邻接矩阵 -mode mode非0时输出最小生成树图
% Output -T 顶点序号矩阵, 每一列的边的两个端点
%        -c 最小生成树的总权值
% Example
% d = inf(5);
% d(1,2:4) = [1,7,3]; d(2,[1,3,5]) = [10,6,4];
% d(3,[4,5]) = [8,5]; d(4,5) = 2;
% [T, c] = kruskal(d)
b = change(d); % 转化格式
n = max(max(b(1:2, :))); % 顶点数n
m = size(b, 2); % 边数m
B = sortrows(b', 3); % 排序
B = B'; t = 1:n; c = 0; k = 0; T = ones(2, n-1);
for i = 1:m
    if t(B(1, i)) ~= t(B(2, i)) % 不成圈
        k = k + 1; % 边 + 1

```

```

T(1:2, k) = B(1:2, i); % 取边
c = c + B(3, i); % 增加权值
tmin = min(t(B(1, i)), t(B(2, i)));
tmax = max(t(B(1, i)), t(B(2, i)));
t(t == tmax) = tmin;
end
if k == n - 1, break; end % 终止条件
end
if nargin || mode, treekplot(d, T); end % 绘图

function b = change(d)
%% 子程序, 转化为链表形式
n = size(d, 2); m = sum(sum(d ~= inf && d ~= 0))/2; b = zeros(3, m); k = 0;
for i = 1:n
    for j = i+1:n
        if d(i,j) ~= inf && d(i,j) ~= 0
            k = k + 1; b(1:3, k) = [i; j; d(i,j)];
        end
    end
end
end

function treekplot(d, T)
%% 子程序, 绘制最小生成树图
% Input -d 赋权邻接矩阵 -T 顶点序号矩阵
adjmatplot(d) % 绘制邻接矩阵图
n = size(d, 2); t = linspace(0, 2*pi, n+1); x = sin(t); y = cos(t);
for i = 1:n-1
    plot(x(T(:,i)), y(T(:,i)), 'r', 'LineWidth', 2)
end
end

```

例 16.3 试用 *Kruskal* 算法计算并绘出图 16.4 中的最小生成树。

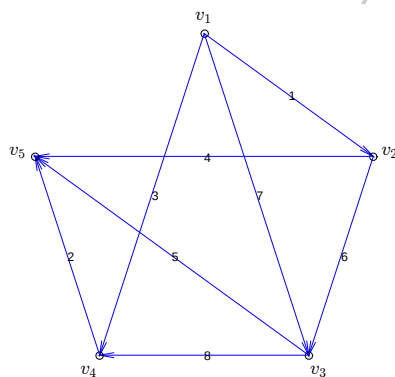


图 16.4: 有向图

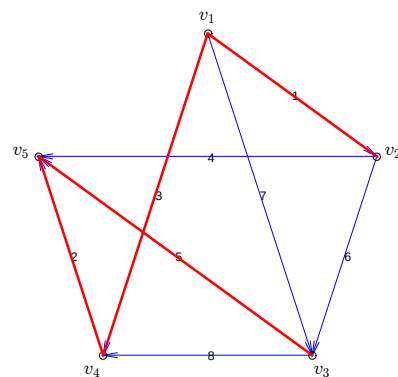


图 16.5: 最小生成树

```

>> d = inf(5);
>> d(1,2:4) = [1,7,3]; d(2,[3,5]) = [6,4];
>> d(3,[4,5]) = [8,5]; d(4,5) = 2;

```

```
>> [T, c] = kruskal(d) % 结果如图 16.5
T =
     1     4     1     3
     2     5     4     5
c =
    11
```

16.3 最短路径算法

最短路径算法(在赋权图 G 中, 求一条总权最小的 v_i 到 v_j 的路径问题)是图论中的一个基本问题, 它在数据通信、石油管线、公路铁路网络等实际问题中有着广泛的应用。Dijkstra算法是最短路径算法中的经典算法, 它可以求出从某一顶点到其他所有顶点的最短路径。

算法 16.2 Dijkstra算法

Algorithm 16: DIJKSTRA(W)

Input: 赋权邻接矩阵 W

Output: 第一个点到各点的最短距离 l 和各点的前一标号 z

- 1 图 G 有 n 个顶点, 赋权邻接矩阵为 W , $W(i, j)$ 表示顶点 v_i 到 v_j 的边的权值, 若边不存在则用 Inf 表示。 S 表示已求出的从始点 v_i 出发的最短路径的终点集合, 起始状态为空集。从 v_i 到 v_j 可能达到的最短路径长度的初值为 $D(k) = \min\{W(i, j) | v_k \in V - \{i\}\}$
 - 2 选择 v_j , 使得 $D(j) = \min\{D(k) | v_k \in V - S\}$, 则 v_j 就是当前求得的最短路的终点, 令 $S = S \cup \{j\}$
 - 3 修改从 v_i 出发到集合 $V - S$ 上任意顶点 v_k 可达的最短路径长度。若 $D(j) + W(j, k) < D(k)$, 则修改 $D(k) = D(j) + W(j, k)$
 - 4 重复2、3步 $n - 1$ 次, 并记录各最短路径经过的所有顶点
-

具体的MATLAB程序为

```
function [L, Z] = dijkstra(W)
% Dijkstra 算法: 求最短路径和最短距离
% Input -W 带权邻接矩阵
% Output -L 第一个点到各点的最短距离 -Z 各点的前一标号
% Example
% W = inf(6); W(1,[2,3]) = [3,4]; W(2,[3,4,5]) = [7,5,20];
% W(3,[5,6]) = [2,10]; W(4,[5,6]) = [8,12]; W(5,6) = 15;
% [L, Z] = dijkstra(W)
n = size(W, 1); L = W(1, :); Z = ones(1, n); i = 1;
while i <= n
    for j = 1:n
        if L(i) > L(j) + W(j, i)
            L(i) = L(j) + W(j, i); Z(i) = j;
            if j < i, i = j - 1; else i = j; end
        end
    end
```

```

end
i = i + 1;
end

```

注意该函数返回的第二个参数的意义，要得出每一条最短路径，还需如下转化：

```

function M = dijkstra_ind2mat(Z)
% Dijkstra 算法：将标号向量转化为路径矩阵
% Example
% Z = [1, 1, 1, 2, 3, 3]; P = dijkstra_ind2mat(Z)
n = length(Z); M = zeros(n); M(1, :) = 1:n; maxSize = zeros(1, n);
for i = 2:n
    j = i; k = 1;
    while j ~= 1 && k < n
        j = Z(j); k = k + 1; M(k, i) = j;
    end
    M(1:k, i) = flipud(M(1:k, i)); maxSize(i) = k;
end
K = max(maxSize); M = M(1:K, :);

```

在此基础上，编写Dijkstra算法计算结果的绘图程序。

```

function dijkstra_plot(W, M)
% Dijkstra 算法的绘图程序
% Input -W 赋权邻接矩阵 -M 最短路径的路径矩阵
adjmatplot(W) % 绘制邻接矩阵图
n = size(W, 1); m = size(M, 2);
t = linspace(0, 2*pi, n+1); x = sin(t); y = cos(t);
for i = 1:m
    aT = M(:, i); T = aT(aT ~= 0); plot(x(T), y(T), 'r', 'LineWidth', 2)
end

```

例 16.4 试求图 16.6 中顶点 a 到各顶点的最短距离和最短路径。

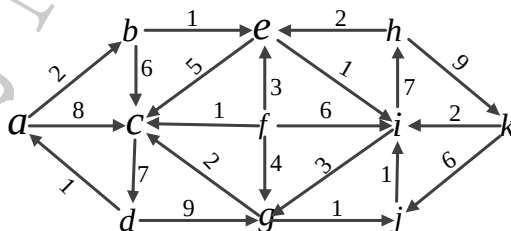


图 16.6: 有向图

```

>> W = inf(11);
>> W(1,[2,3]) = [2,8]; W(2,[3,5]) = [6,1]; W(3,4) = 7; W(4,[1,7]) = [1,9];
>> W(5,9) = 1; W(6,[3,5,7,9]) = [1,3,4,6]; W(7,[3,10]) = [2,1];
>> W(8,[5,11]) = [2,9]; W(9,8) = 7; W(10,9) = 1; W(11,[9,10]) = [2,6];
>> [L, Z] = dijkstra(W)
L =
    16     2     8    15     3   Inf    24    11     4    25    20
Z =
     4     1     1     3     2     1     4     9     5     7     8
>> M = dijkstra_ind2mat(Z)
M =
     1     1     1     1     1     1     1     1     1     1     1
     0     2     3     3     2     6     3     2     2     3     2
     0     0     0     4     5     0     4     5     5     4     5
     0     0     0     0     0     0     7     9     9     7     9
     0     0     0     0     0     0     0     8     0    10     8
     0     0     0     0     0     0     0     0     0     0    11
>> dijkstra_plot(W, M) % 结果如图 16.7

```

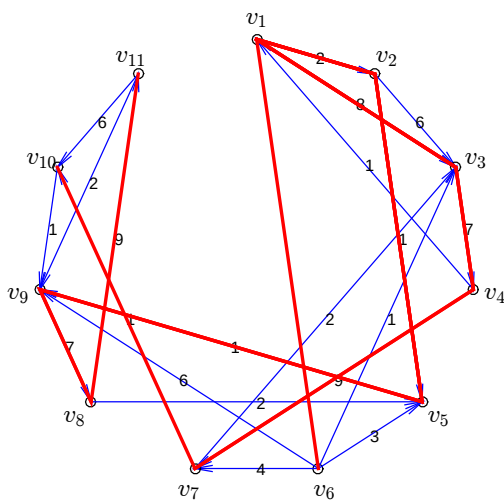


图 16.7: 最短路径图

例 16.5 某企业使用一种设备，每年年初，决定是购置新的，还是继续用旧的。若购置新设备，则需付一定的购置费用；若继续使用旧设备，则需付一定的维修费用。假设该企业在第一年开始时必须购置一台这种设备，并假设计划使用这种设备的时间为五年。下面给出一台新设备的价格以及一台设备的使用维修费用。这家企业希望确定应在哪一年购买一台新设备，使得维修费和新设备的购置费的总和最小。

表 16.1: 从第1年到第5年的设备价格(单位: 千元)

年号	1	2	3	4	5
价格	11	11	12	12	13

表 16.2: 不同使用年限的设备的维修费(单位: 千元)

使用年限	0~1	1~2	2~3	3~4	4~5
维修费	5	6	8	11	18

解 将问题转化为最短路径问题，根据设备价格和维修费计算权值矩阵 W ，

$$W = \begin{bmatrix} \text{Inf} & 16 & 22 & 30 & 41 & 59 \\ \text{Inf} & \text{Inf} & 16 & 22 & 30 & 41 \\ \text{Inf} & \text{Inf} & \text{Inf} & 17 & 23 & 31 \\ \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & 17 & 23 \\ \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & 18 \\ \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} & \text{Inf} \end{bmatrix}.$$

利用Dijkstra算法计算最短路径。

```
>> W = inf(6);
>> W(1,2:6) = [16,22,30,41,59];
>> W(2,3:6) = [16,22,30,41];
>> W(3,4:6) = [17,23,31];
>> W(4,5:6) = [17,23];
>> W(5, 6) = 18;
>> [L, Z] = dijkstra(W)
L =
    Inf     16     22     30     41     53
Z =
     1     1     1     1     1     3
```

故应在第3年购买新设备，花费总和最小为53千元。

16.4 图论算法汇总

图论中有一些非常经典且实用的算法，本章详细讲述了Kruskal算法和Dijkstra算法，对于其他算法，见表 16.3，相应的程序可在随书附盘中找到，或者向作者发送邮件索取，这里不再详细介绍，有兴趣的读者可以查看相关资料进一步深入研究这些算法。

表 16.3: 图论算法汇总

函数名称	算法名称	功能说明
dgraf	矩阵可达算法	判断邻接矩阵是否可达
floyd	Floyd算法	计算任意两点间的最短距离
prim	Prim算法	计算最小生成树
fleury	Fleury算法	在一个Euler图中找出Euler环游
fc, fc2	匈牙利算法	寻找二分图的最大匹配
matgraf	较大基数匹配算法	一般图中, 寻找包含边数最多的匹配
kuhmunkres	Kuhn-Munkres算法	求赋权完备二元图的最佳匹配
fulkersonf	2F算法	最大流的Ford-Fulkerson算法
bgowanf	Busacker-Gowan算法	求最大流最小费用

我的 MATLAB 报告

王保山

第十七章 动态规划

17.1 动态规划的概念与逆序算法

动态规划¹是求解决策过程最优化的有效数学方法，是根据“最优决策的任何截断仍是最优的”这一原理，通过将多阶段决策过程转化为一系列单阶段问题，逐个求解的优化求解方法。它在经济管理、生产调度、工程技术、最优控制等众多领域有着广泛的应用。

表 17.1: 基本概念

阶段	整个问题的解决可以分成若干个互不联系的阶段进行，描述阶段的变量称为阶段变量，记为 k
状态	每个阶段开始的条件或状况，描述各阶段状态的变量称为状态变量，用 x_k 表示第 k 阶段的状态变量。动态规划解决多阶段问题时，要求整个过程具有无后效性，即未来的状态只依赖于当前状态，而与之前的状态无关
决策	某一阶段的状态确定后，可以做出各种选择从而演变到下一阶段某一状态，这种选择手段称为决策，描述决策的变量称为决策变量， $u_k(x_k)$ 表示第 k 阶段处于状态 x_k 的决策变量， $D_k(x_k)$ 表示 $u_k(x_k)$ 的取值范围
策略	一个由每个阶段的决策按顺序排列组成的集合称为策略，记为 p ，由 x_k 状态到终止状态的策略记为 $p_k(x_k) = \{u_k(x_k), u_{k+1}(x_{k+1}), \dots, u_n(x_n)\}$
状态转移方程	第 k 阶段的状态变量为 x_k ，做出的决策为 u_k ，则第 $k+1$ 阶段的状态变量为 x_{k+1} 也完全被确定，这种演变规律可以用状态转移方程来表示，即 $x_{k+1} = T_k(x_k, u_k)$
指标函数	执行某一策略产生结果的数量表示，用来衡量策略优劣的数量指标，全过程指标函数记为 $G(u_1, u_2, \dots, u_n)$ ，后续过程的指标函数记为 $G_k(u_k, u_{k+1}, \dots, u_n)$ ，第 j 阶段的阶段指标函数记为 $v_j(u_j, x_j)$
最优	(策略、指标等)函数达到最优值称为最优(策略、指标等)。最优轨线是指按照最优策略和状态转移方程演变的状态序列 $x_1^*, x_2^*, \dots, x_{n+1}^*$

¹本章内容和算法程序来自于胡良剑、孙晓君编著的《MATLAB 数学实验》第十三章，有改动。

逆序算法就是从终端逐段向始端方向寻找最优解的一种方法。我们把方程

$$\begin{cases} f_k(x_k) = \min\{G_k(v_k(x_k, u_k(x_k)), f_{k+1}(x_{k+1})) | u_k \in D_k(x_k)\} \\ f_{k+1}(x_{k+1}) = 0; x_{k+1} = T_k(x_k, u_k), k = n, n-1, \dots, 1 \end{cases}$$

称为基本方程，它在动态规划逆序求解中起着本质的作用。逆序算法的MATLAB程序如下：

```
function [p_opt, fval] = dynprog(x, decisfun, subobjfun, transfun, objfun)
% 自由始段和终端的动态规划，求指数函数最小值的逆序算法递归计算程序
% Input -x 状态变量，一列代表一个阶段状态；
%        -decisfun(k,x) 由阶段k的状态变量x求出相应的允许决策变量；
%        -subobjfun(k,x,u) 阶段指标函数；
%        -transfun(k,x,u) 状态转移函数；
%        其中x是阶段k的某状态变量，u是相应的决策变量；
%        -objfun(v,f) 第k阶段直至最后阶段指标函数，
%        当objfun(v,f)=v+f时，objfun(v,f)可以省略
% Output -p_opt 由四列构成，[序号组；最优策略组；最优轨线组；指标函数值组]；
%         -fval 列向量，各元素分别表示p_opt各最优策略组对应始端状态x的最优函数值
k = length(x(1,:)); x_isnan = ~isnan(x); t_vub = inf;
t_vubm = inf(size(x)); f_opt = nan(size(x));
d_opt = f_opt;
% 计算终端(最后一段)相关值
tmp1 = find(x_isnan(:,k)); tmp2 = length(tmp1);
for i = 1:tmp2
    u = feval(decisfun, k, x(i,k)); tmp3 = length(u);
    for j = 1:tmp3
        tmp = feval(subobjfun, k, x(tmp1(i),k), u(j));
        if tmp <= t_vub
            f_opt(i,k) = tmp; d_opt(i,k) = u(j); t_vub = tmp;
        end
    end
end
for ii = k-1:-1:1
    tmp10 = find(x_isnan(:,ii)); tmp20 = length(tmp10);
    for i = 1:tmp20
        u = feval(decisfun, ii, x(i,ii)); tmp30 = length(u);
        for j = 1:tmp30
            tmp00 = feval(subobjfun, ii, x(tmp10(i),ii), u(j));
            tmp40 = feval(transfun, ii, x(tmp10(i),ii), u(j));
            tmp50 = x(:,ii+1) - tmp40;
            tmp60 = find(tmp50 == 0);
            if ~isempty(tmp60)
                if nargin < 5, tmp00 = tmp00 + f_opt(tmp60(1), ii + 1);
                else tmp00 = feval(objfun, tmp00, f_opt(tmp60(1), ii + 1));
            end
            if tmp00 <= t_vubm(i,ii)
                f_opt(i,ii) = tmp00; d_opt(i,ii) = u(j);
                t_vubm(i,ii) = tmp00;
            end
        end
    end
end
end
fval = f_opt(tmp1,1); fval = fval(~isnan(fval),1);
% 记录最优决策、最优轨线和相应指标函数值
p_opt = []; tmpx = []; tmpd = []; tmpf = [];
tmp0 = find(x_isnan(:,1)); tmp01 = length(tmp0);
```

```

for i = 1:tmp01
    tmpd(i) = d_opt(tmp0(i),1); tmpx(i) = x(tmp0(i),1);
    tmpf(i) = feval(subobjfun, 1, tmpx(i), tmpd(i));
    p_opt(k*(i - 1) + 1,[1,2,3,4]) = [1,tmpx(i),tmpd(i),tmpf(i)];
    for ii = 2:k
        tmpx(ii) = feval(transfun, ii - 1, tmpx(i), tmpd(i));
        tmp1 = x(:,ii) - tmpx(ii); tmp2 = find(tmp1 == 0);
        if ~isempty(tmp2), tmpd(ii) = d_opt(tmp2(1), ii); end
        tmpf(ii) = feval(subobjfun, ii, tmpx(ii), tmpd(ii));
        p_opt(k*(i - 1) + ii, [1,2,3,4]) = [ii, tmpx(ii), tmpd(ii), tmpf(ii)];
    end
end
end

```

17.2 实验报告

1. 某公司打算向它的四个营业区增设4个销售点，从各区赚取的利润与增设的销售店个数有关，其数据如表 17.2 所示(数据文件：实验报告数据.xls 表名：动态规划)。试求各区应分配几个增设的销售店，才能使总利润最大？其值是多少？

表 17.2: 数据表

销售点增加数	第一区利润(万元)	第二区利润(万元)	第三区利润(万元)	第四区利润(万元)
0	160	190	200	250
1	310	225	298	308
2	541	445	399	487
3	600	517	601	655
4	705	632	721	674

解 (1) 这是一个4阶段动态规划问题。

(2) 选择状态变量、决策变量及其取值范围。

```

function u = decisfun_ex1(k, x)
% k阶段由状态变量x求出的决策变量u的所有取值
if k == 4, u = x; else u = 0:x; end

```

(3) 编写状态转移方程。

```

function y = transfun_ex1(k, x, u)
% 状态转移方程
y = x - u;

```

(4) 编写各阶段指标函数。

```
function v = subobjfun_ex1(k, x, u)
% k阶段的指标函数
global w % 声明全局变量
v = (0:4 == u) * w(:,k);
```

(5) 用逆序算法求解。

```
>> global w % 声明全局变量
>> data = xlsread('实验报告数据.xls', '动态规划');
>> w = -data(:, 2:end); % 取负值, 转化为求最小值
>> x = repmat((0:4)', 1, 4);
>> [p, f] = dynprog(x, @decisfun_ex1, @subobjfun_ex1, @transfun_ex1)
p =
     1     0     0    -160
     2     0     0    -190
     3     0     0    -200
     4     0     0    -250
     1     1     1    -310
     2     0     0    -190
     3     0     0    -200
     4     0     0    -250
     1     2     2    -541
     2     0     0    -190
     3     0     0    -200
     4     0     0    -250
     1     3     2    -541
     2     1     0    -190
     3     1     1    -298
     4     0     0    -250
     1     4     2    -541
     2     2     2    -445
     3     0     0    -200
     4     0     0    -250
f =
    -800
   -950
  -1181
  -1279
 -1436
```

观察p与f的值可知, 在第一区和第二区各设2个销售点可使总利润最大, 最大为1436万元。2. 图17.1是一线路网络, 要铺设从 v_1 点到 v_{10} 点的电话线, 中间需经过3个点。第1点可以是 v_2, v_3, v_4 中的一个点, 第2点可以是 v_5, v_6, v_7 中的一个点, 第3点可以是 v_8, v_9 中的一个点。各点之间若能铺设电话线, 则在图中以连线表示, 连线旁的数字表示两点间的距离。用动态规划方法求一条从 v_1 至 v_{10} 的最短路线。

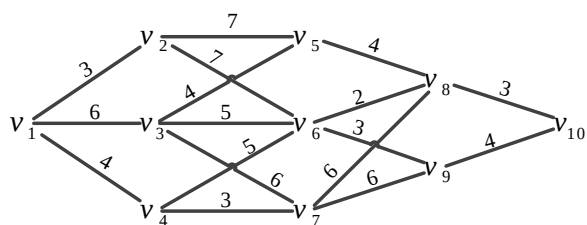


图 17.1: 线路网络

解 (1) 选择状态变量、决策变量及其取值范围。

```
function u = decisfun_ex2(k, x)
% k阶段由状态变量x求出的决策变量u的所有取值
switch x
case 1, u = [2;3;4];
case 2, u = [5;6];
case 3, u = [5;6;7];
case 4, u = [6;7];
case 5, u = 8;
case {6,7}, u = [8;9];
otherwise, u = 10;
end
```

(2) 编写状态转移方程。

```
function y = transfun_ex2(k, x, u)
% 状态转移方程
y = u;
```

(3) 编写各阶段指标函数。

```
function v = subobjfun_ex2(k, x, u)
% k阶段的指标函数
t = [3; 6; 4; 7; 7; 4; 5; 6; 5; 3; 4; 2; 3; 6; 6; 3; 4];
tmp = [x == 1 && u == 2, x == 1 && u == 3, x == 1 && u == 4, ...
        x == 2 && u == 5, x == 2 && u == 6, ...
        x == 3 && u == 5, x == 3 && u == 6, x == 3 && u == 7, ...
        x == 4 && u == 6, x == 4 && u == 7, ...
        x == 5 && u == 8, ...
        x == 6 && u == 8, x == 6 && u == 9, ...
        x == 7 && u == 8, x == 7 && u == 9, ...
        x == 8 && u == 10, ...
        x == 9 && u == 10];
v = tmp * t;
```

(4) 求解

```

>> x = nan(3, 5);
>> x(1, [1,5]) = [1,10];
>> x(1:3,[2,3]) = [(2:4)',(5:7)'];
>> x(1:2,4)      = [8,9];
>> [p, f] = dynprog(x, @decisfun_ex2, @subobjfun_ex2, @transfun_ex2)
p =
     1     1     4     4
     2     4     6     5
     3     6     8     2
     4     8    10     3
     5    10    10     0
f =
    14

```

故最短路径为 $v_1 \rightarrow v_4 \rightarrow v_6 \rightarrow w_8 \rightarrow v_{10}$ ，最短距离为14。

3. 某船厂接受七艘船的订货，要求第一、二、三、四季度末分别交货1、1、2、3艘，该厂每季度的生产能力为2艘，预计各季度的单位生产成本为4、6、7、9，各季度多生产的船可存在仓库中，其每季度每条船的保管费为1.2，试制定一个完成订货合同且总费用最省的生产计划。

解 (1) 选择状态变量、决策变量及其取值范围。

```

function u = decisfun_ex3(k, x)
% k阶段由状态变量x求出的决策变量u的所有取值
q = [1, 1, 2, 3];
if q(k) - x < 0, u = 0:2; else u = q(k) - x:2; end
u = u(:);

```

(2) 编写状态转移方程。

```

function y = transfun_ex3(k, x, u)
% 状态转移方程
q = [1, 1, 2, 3];
y = x + u - q(k);

```

(3) 编写各阶段指标函数。

```

function v = subobjfun_ex3(k, x, u)
% k阶段的指标函数
c = [4, 6, 7, 9];
v = c(k) * u + 1.2 * x;

```

(4) 求解


```
>> x = nan(3, 4);
>> x(1, 1) = 0; x(1:2, 2) = [0; 1];
>> x(1:3, [3,4]) = repmat((0:2)', 1, 2);
>> [p, f] = dynprog(x, @decisfun_ex3, @subobjfun_ex3, @transfun_ex3)
p =
    1.0000         0    2.0000    8.0000
    2.0000    1.0000    2.0000   13.2000
    3.0000    2.0000    2.0000   16.4000
    4.0000    2.0000    1.0000   11.4000
f =
    49
```

故前三个季度各生产2艘，四季度生产1艘，生产成本最少，总费用为49。

4. 某电子设备由5种元件1、2、3、4、5组成，这5种元件的可靠性分别为0.9、0.8、0.5、0.7、0.6。为保证电子设备系统的可靠性，同种元件可并联多个。现有允许设备使用元件的总数为15个，试设计使设备可靠性最大的元件安排方案。

解 将该问题看成一个5阶段动态规划问题，每个元件的配置看成一个阶段。记： x_k 为配置第 k 个元件时可用的元件总数(状态变量)； u_k 为第 k 个元件并联的数目(决策变量)； c_k 为第 k 个元件的可靠性，则 $v_k(x_k, u_k) = 1 - (1 - c_k)^{u_k}$ 为阶段指标函数， $x_{k+1} = x_k - u_k$ 为状态转移方程，其基本方程为

$$\begin{cases} f_k(x_k, u_k) = \min\{G_k(v_k(x_k, u_k), f_{k+1}(x_{k+1})) | u_k \in D_k(x_k)\}, k = 4, 3, 2, 1, \\ f_4(x_4, u_4) = v_4(x_4, u_4); G_k(a, b) = a \cdot b. \end{cases}$$

(1) 选择状态变量、决策变量及其取值范围。

```
function u = decisfun_ex4(k, x)
% k阶段由状态变量x求出的决策变量u的所有取值
if k == 5, u = x; else u = 1:x - 1; end
u = u(:);
```

(2) 编写状态转移方程。

```
function y = transfun_ex4(k, x, u)
% 状态转移方程
y = x - u;
```

(3) 编写各阶段指标函数。

```
function v = subobjfun_ex4(k, x, u)
% k阶段的指标函数
c = [0.9, 0.8, 0.5, 0.7, 0.6];
v = 1 - (1 - c(k)) .^ u;
v = - v;
```

(4) 编写后续指标函数。

```
function y = objfun_ex4(v, f)
% 第k阶段直至最后阶段的指标函数
y = - v * f;
```

(5) 求解

```
>> x1 = [15; nan(14, 1)];
>> x2 = repmat((1:15)', 1, 4);
>> x = [x1, x2];
>> [p, f] = dynprog(x, @decisfun_ex4, @subobjfun_ex4, @transfun_ex4, @objfun_ex4)
p =
    1.0000    15.0000     2.0000    -0.9900
    2.0000    13.0000     2.0000    -0.9600
    3.0000    11.0000     4.0000    -0.9375
    4.0000     7.0000     3.0000    -0.9730
    5.0000     4.0000     4.0000    -0.9744
f =
   -0.8447
```

故5种元件分别并联2、2、4、3、4个，系统可靠性最大，最大为0.8447。

第十八章 网站排名

18.1 问题背景与基础知识

几乎每个人都有使用Google搜索引擎进行网上搜索的体验。我们在Google搜索引擎中输入一些关键词后，Google会很快地找到所有与搜索关键词匹配的网页，并给出所有的网站排名情况(一般认为排在第一个的最重要，以下类推)。到目前为止，世界上有近千万个网站，十多亿个网页，难道Google搜索引擎真的如此神奇，能够在几秒、几十秒的时间内扫遍世界上所有的网站(网页)吗？答案是否定的。事实上，Google网站是基于自己的大型数据库系统的网站，它定期地(比如每个月一次)对世界上的所有网站进行大搜索，并将结果保存在自己的数据库中。我们通过Google搜索引擎进行网上搜索，实际上是在Google网站的数据库里进行搜索，因此，所用时间一般不会太长。

要验证这一点并不难。假如你是一个“网管”，你可以控制一个网站，比如你可以很快地向网站发布信息(内含某些特殊的关键词)。此后，你迅速利用Google搜索引擎搜索你刚才的关键词，一般情况下是找不到的。

我们关心的重点是：与某个关键词相关的网站可能有几个、几十、...，最多可能有几百万个，Google是如何给出网站排名情况的呢？本实验将要介绍的PageRank(网页级别)算法就是Google用于评测一个网页“重要性”的一种方法。虽然现在不断地有改善的排名算法，但其本质上与PageRank(网页级别)算法十分接近。相信如能彻底理解PageRank算法，对于理解、设计其它算法将是十分有益的。本章将主要介绍该算法。

PageRank算法用到的数学知识看上去并不十分高深(但十分实用)，主要用到线性代数的一些知识，包括：正矩阵性质、特征向量与特征值关系、幂迭代法(Power Iteration)和Gauss-Seidel迭代方法等。

实验中的一些程序看起来是小题大做，比如对已知的6阶方阵 A ，用MATLAB求解代数方程 $x = Ax$ 不是十分困难的事。但如若方阵 A 的阶数是60、600、6000、60000，你的方法将可能彻底崩溃。因此，认真学习本章的一些计算方法是有意义的。通过实践对比，你会发现这些算法的效率是相当高的，特别适合大规模计算。事实上，Google采用的是大规模的并行计算技术，以求解高达上亿阶的代数方程 $x = Ax$ 。

PageRank(网页级别)是Google用于评测一个网页“重要性”的一种方法。在揉合了诸如Title标识和Keywords标识等所有其它因素之后，Google通过PageRank来调整结果，使那些更具“重要性”的网页在搜索结果中令网站排名获得提升，从而提高搜索结果的相关性和质量。

简单说来, Google通过下述几个步骤来实现网页在其搜索结果页(SERPS)中的排名:

- (1) 找到所有与搜索关键词匹配的网页;
- (2) 根据页面因素如标题、关键词密度等排列等级;
- (3) 计算导入链接的锚文本中的关键词;
- (4) 通过PageRank得分调整网站排名结果。

事实上, 真正的网站排名过程并不是这么简单, 读者可参见有关网站, 获得更详细、深入的阐述。

Google的PageRank是基于这样一个理论: 若 B 网页设置有连接 A 网页的链接(B 为 A 的导入链接时), 说明 B 认为 A 有链接价值, 是一个“重要”的网页。当 B 网页级别(重要性)比较高时, 则 A 网页可从 B 网页这个导入链接分得一定的级别(重要性), 并平均分配给 A 网页上的导出链接。

导入链接(也叫逆向链接)指链接到你网站的站点, 也就是我们一般所说的“外部链接”。而当你链接到另外一个站点, 那么这个站点就是你的“导出链接”, 即你向其它网站提供的本站链接。

PageRank反映了一个网页的导入链接的级别(重要性)。一般说来, PageRank是由一个网站的导入链接的数量和这些链接的级别(重要性)所决定的。

搜索引擎网站排名算法中的各排名因子的重要性均取决于它们所提供信息的质量。但如果排名因子具有易操纵性, 则往往会被一些网站管理员利用来实现不良竞争。例如初引入的排名因子之一——关键词元标识(Meta Keywords), 是由于理论上它可以很好地概括反映一个页面的内容, 但后来却由于一些网站管理员的恶意操纵而不得不黯然退出。因此“加权值”(我们对该因子提供信息的信任程度)是由排名因子的易操纵程度和操纵程度共同决定的。

为了便于理解PageRank算法, 我们要引入有向图的概念。生活中的关系许多是对称的, 比如互相握手、交谈, 但也有不少关系不是对称的, 比如认识关系, 甲认识乙, 并不意味着乙认识甲, 还有投票、比赛等关系也是不对称的。有向图就是这些不对称关系的抽象。所谓有向图, 是指由有限的非空元素集和它的不同元素的有序数对集(可能为空集)组成的结构。图 18.1 就是一个有向图 D , 共有3个顶点 $V(D) = \{u, v, w\}$ (有限的非空元素集称为顶点集)和3条弧 $A(D) = \{(u, w), (w, u), (u, v)\}$ (不同元素的有序数对集称为弧集)。当有向图用画图的方式来表示时, 每条弧的方向就由箭头来指定。

有向图中有如下的术语:

- (1) D 的顶点集的基数称为 D 的阶, 记作 $p(D)$; D 的弧集的基数称为 D 的大小, 记作 $q(D)$;
- (2) $a = (u, v)$ 是有向图的一条弧, 并称之为从 u 邻接到 v , 而 v 是从 u 邻接的; 一般设从 u 到 v 的长度为1;
- (3) 有向图 D 的定点 u 的出度(out-degree)是 D 中从 u 邻接的顶点个数, 记作 $od(u)$; u 的入度(in-degree)是 D 中邻接到 u 的顶点个数, 记为 $id(u)$ 。

例 18.1 在图 18.1 中, $p(D) = 3$, $q(D) = 3$, $od(u) = 2$, $id(u) = 1$, $od(v) = 0$, $id(v) = 1$, $od(w) = 1$, $id(w) = 1$ 。

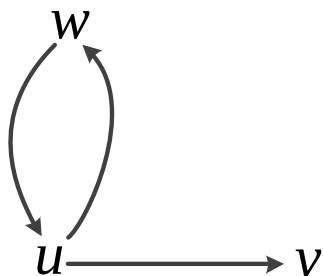


图 18.1: 示意图

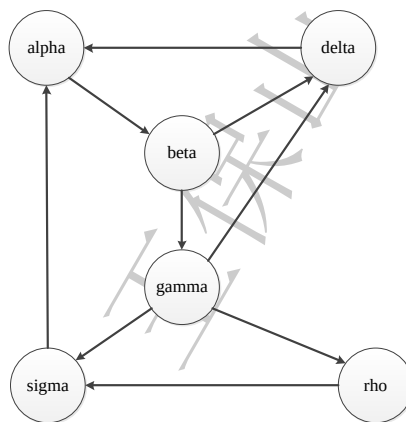


图 18.2: 示意图

例 18.2 在图 18.2 中, $p(D) = 6$, $q(D) = 9$, 出入度如表 18.1 所示。

表 18.1: 出入度表

序号(Index)	顶点(Node)	入度(In-degree)	出度(Out-degree)
1	alpha	2	1
2	beta	1	2
3	gamma	1	3
4	delta	2	1
5	rho	1	1
6	sigma	2	1

在第十六章中我们引入了邻接矩阵的概念, 这里有,

$$g_{ij} = \begin{cases} 1, & \text{若存在从 } j \text{ 到 } i \text{ 的弧} \\ 0, & \text{否则} \end{cases}$$

下面介绍邻接矩阵的性质:

(1) 若定义行和 $r_i = \sum_j g_{ij}$, 列和 $n_j = \sum_i g_{ij}$, 可以看出 r_i 就是第 i 个顶点的入度, n_j 就是第 j 个顶点的出度;

(2) $\sum_i r_i = \sum_j n_j = q(D)$, 即他们都等于有向图 D 的大小(弧的个数)。

了解这些基础知识之后, 理解PageRank算法就很容易了。

18.2 PageRank算法及其改进

如果我们把图 18.2 的每个顶点看成是一个网页，并把每个弧看成是网页之间的“超级链接”，则图 ?? 此时可代表一个小型的网络，其中有6个网页和9个“超级链接”。

例 18.3 一个自然的问题是：在图 18.2 所示的这6个网页中，哪个最重要？

解 一个简单的回答可以这样考虑：看谁的入度最多！此时，alpha、delta、sigma各有2票，其余各1票。因此，alpha、delta、sigma最重要，并列第1，而其它3者并列第2。

表 18.2: 按入度排名

序号(Index)	顶点(Node)	入度(In-degree)	排名(Rank)
1	alpha	2	1
2	beta	1	2
3	gamma	1	2
4	delta	2	1
5	rho	1	2
6	sigma	2	1

但这样的回答不能令人满意。按照现在的方法，我们无法说出alpha、delta、sigma中哪一个最重要。

算法 18.1 简化的 *PageRank* 算法

一个网页的重要性可以从以下2个方面来考虑：

- (1) 看谁的入度多；
- (2) 本网页在网络中的排名靠前(排名向量的分量数值大)。

用数学的语言可表达为：设 u 是某个网页，其排名为 $r(u)$ ，记 F_u 是 u 邻接到的那些网页的结合， B_u 是邻接到 u 的那些网页的集合， $n_v = |F_v|$ 是 v 邻接到的那些网页的总数，则有

$$r(u) = \sum_{v \in B_u} \frac{r(v)}{n_v} \quad (18.1)$$

根据上述的有向图理论，我们有 $|B_u| = id(u)$ ， $n_v = |F_v| = od(v)$ 。若我们用 $r = (r_i)$ 来表示各个网页的名次， $G = \{g_{ij}\}$ 表示邻接矩阵，则(18.1)可以写成

$$r_i = \sum_j \frac{g_{ij}}{n_j} r_j \quad (18.2)$$

或

$$r = G_n r, \quad (18.3)$$

其中 $G_n = \{g_{ij}/n_j\}$ 。

从(18.3)可以看出, 网页的名次 $r = (r_i)$ 实为矩阵 G_n 的对应于特征根为1的特征向量(问题是: 矩阵 G_n 一定有特征值1吗?)。

算法公式(18.1)存在明显的问题, 比如 $G = [0, 1; 1, 0]$ 时, 将得到 $r_1 = r_2$, 无法排名。为此, 要对算法公式(18.1)进行改进。

算法 18.2 改进的 PageRank 算法

设 $\eta(u)$ 是网页 u 开始时的名次, $x(u)$ 为某时刻的 PageRank 得分(名次), 采用下面的加权算法:

$$x(u) = p \sum_{v \in B_u} \frac{x(v)}{n_v} + \eta(u) \quad (18.4)$$

类似地, 记 $x = x(i), i = 1, 2, \dots, n, \eta = \eta(u)$, 一般取 $p = 0.85$, 而 $\eta = \delta e$, 其中 $e = \{1, 1, \dots, 1\}'$, $\delta = (1 - p)/n$, 于是(18.4)可以写成

$$(I - pGD)x = \delta e,$$

其中 G 为邻接矩阵, 而 $D = \text{diag}\{1/n_1, 1/n_2, \dots, 1/n_n\}$ 。

若规定

$$\sum_{i=1}^n x_i = e^T x = 1, \quad x_i > 0, \quad (18.5)$$

则 $\eta = \delta e = \delta e e^T x$ 。由 $x = pGDx + \delta e$ 得到

$$x = pGDx + \delta e = pGDx + \delta e \cdot 1 = pGDx + \delta e \cdot e^T x = (pGD + \delta e \cdot e^T)x = Ax,$$

其中

$$A = pGD + \delta e \cdot e^T, \quad (18.6)$$

即

$$A = \begin{bmatrix} p \frac{g_{11}}{n_1} + \delta & p \frac{g_{12}}{n_2} + \delta & \cdots & p \frac{g_{1n}}{n_n} + \delta \\ p \frac{g_{21}}{n_1} + \delta & p \frac{g_{22}}{n_2} + \delta & \cdots & p \frac{g_{2n}}{n_n} + \delta \\ \vdots & \vdots & \ddots & \vdots \\ p \frac{g_{n1}}{n_1} + \delta & p \frac{g_{n2}}{n_2} + \delta & \cdots & p \frac{g_{nn}}{n_n} + \delta \end{bmatrix}. \quad (18.7)$$

注 18.1 (1) 若 $\exists j$, 对 $\forall i$, 有 $g_{ij} = 0$, 即 $n_j = 0$, 此时规定 $\frac{g_{ij}}{n_j} = \frac{1}{n}$;

(2) 在约束条件(18.5)下, $x = Ax$ 具有唯一解 x^* , 其依据是 Perron Frobenius 定理。

定理 18.1 Perron Frobenius 定理

如矩阵 A 是正的方阵, 则:

- (1) A 的谱半径 $\rho(A) > 0$, 其中 $\rho(A) = \max_i |\lambda_i|$, λ_i 是 A 的特征值;
- (2) $\rho(A)$ 是 A 的特征值;
- (3) 存在唯一的 $x > 0$, 使得 $Ax = \rho(A)x$, $\sum x_i = 1$;
- (4) $\rho(A)$ 是 A 的单特征值;
- (5) 若 $\lambda \neq \rho(A)$, 则 $|\lambda| < \rho(A)$, $\rho(A)$ 是 A 的按模最大的唯一的特征值。

例 18.4 对图 18.2 所示的小型网络, 按照改进的PageRank算法, 计算6个网页的排名, 其中的 $p = 0.85$ 。

解 改进PageRank算法的程序如下:

```
function newpagerank(G, p, s, eps, snt, mode)
% 改进的PageRank算法
% Input -G 邻接矩阵 -p p的值 -s 顶点名称, 元胞或字符串, 长与G的长相等
%       -eps 计算精度 -snt 最大迭代次数
%       -mode 输出模式: 1(非0)-元胞型结果; 0-字符型结果
% Example
%   G = [0 1 0; 1 0 0; 0 1 0];
%   s = {'w', 'u', 'v'}; p = 0.85;
%   newpagerank(G, p, s)
n = size(G, 1);
if nargin < 6, mode = 1; end
if nargin < 5, snt = 500; end
if nargin < 4, eps = 1e-6; end
if nargin < 3, s = num2cell((1:n)); mode = 1; end
if nargin < 2, p = 0.85; end
sn = sum(G, 1);
delta = (1 - p) / n; D = zeros(n, 1);
for j = 1:n
    if sn(j) == 0
        D(j) = 1 / n; G(:,j) = 1;
    else
        D(j) = 1 / sn(j);
    end
end
D = diag(D); A = p * G * D + delta;
x = ones(n, 1) / n; z = zeros(n, 1); cnt = 0;
while max(abs(x - z)) > eps && cnt <= snt
    z = x; x = A * x; cnt = cnt + 1;
end
[x1, index] = sort(x); x1 = flipud(x1); index = flipud(index);
txt = {'排名', '得分', '顶点', '原序号'};
if mode
    disp([txt; num2cell((1:n)'), num2cell(x1), s(index)', num2cell(index)]);
else
    fprintf('按PageRank得分排名\n');
    fprintf('-----\n');
    fprintf('%-10s%-12s%-11s%-11s\n', txt{1}, txt{2}, txt{3}, txt{4});
    fprintf('-----\n');
    for i = 1:n
```



```

fprintf(' ');
fprintf('%-7d%-14f%-12s%-2d\n', i, x1(i), s{index(i)}, index(i));
end
fprintf('-----\n');
end

```

计算排名

```

>> G = zeros(6);
>> G(1,[4,6]) = 1; G(2,1) = 1; G(3,2) = 1;
>> G(4,[2,3]) = 1; G(5,3) = 1; G(6,[3,5]) = 1;
>> p = 0.85;
>> s = {'alpha', 'beta', 'gamma', 'delta', 'rho', 'sigma'};
>> newpagerank(G, p, s)

```

'排名'	'得分'	'顶点'	'原序号'
[1]	[0.2675]	'alpha'	[1]
[2]	[0.2524]	'beta'	[2]
[3]	[0.1697]	'delta'	[4]
[4]	[0.1323]	'gamma'	[3]
[5]	[0.1156]	'sigma'	[6]
[6]	[0.0625]	'rho'	[5]

依据例 18.3 按入度的排名, alpha、delta、sigma 并列第1, 现在按PageRank得分排名, 变成了第1、3、5; 而原来beta、gamma、rho 并列第2, 现在变成了第2、4、6。由此可见, 简单直观的想法往往是不准确的。事实上, 由于alpha的重要性(排名第1), 从而提升了beta的名次。上述的 $p = 0.85$ 不是最要紧的, 读者可以换为与之接近的别的数值, 看看将发生怎样的变化。

到此为止, 问题好像已经解决, 但实际情况远没有结束。前面的实验中用的是6阶方阵 A , 用MATLAB直接求解代数方程 $x = Ax$ 或求 A 的特征根与特征向量, 都不是十分困难的事。但如若方阵 A 的阶数是6000、60000, 简单地使用MATLAB的求解命令是不可能的, 也是不允许的。本章仅介绍幂迭代方法(Power Iteration), 一些经典的方法, 如Jacobi迭代方法、Gauss-Seidel迭代方法等都可以在此一用, 限于篇幅, 从略。

设满足 $x = Ax$ 的方阵 A 有 n 个线性无关的特征向量 $x, y_2, \dots, y_n$, 相应的特征值为 $\lambda_1 = 1, \lambda_2, \dots, \lambda_n, |\lambda_i| < 1, \forall i \geq 2$ 。注意 $x = \{x_i\}$ 为PageRank名次向量, 且满足 $\sum x_i = 1$ 。设 v 是任意一个向量, 若将 $x, y_2, \dots, y_n$ 看成一组基向量, 则 v 可由这组基向量线性表示, 即

$$v = a_1x + a_2y_1 + \dots + a_ny_n,$$

两边同时乘上方阵 A , 有

$$Av = a_1Ax + a_2Ay_2 + \dots + a_nAy_n = a_1x + a_2\lambda_2y_2 + \dots + a_n\lambda_ny_n,$$

重复 k 次, 有

$$A^k v = a_1x + a_2\lambda_2^k y_2 + \dots + a_n\lambda_n^k y_n.$$

由于 $|\lambda_i| < \lambda_1 = 1, \forall i \geq 2$, 故当 k 充分大后, $A^k v \approx a_1x$, 从而有

$$\sum A^k v \approx \sum a_1x = a_1 \sum x = a_1,$$

即

$$x \approx A^k v / a_1 \approx A^k v / \sum A^k v.$$

故PageRank的名次向量 $x = \{x_i\}$ 可利用下式得到,

$$x = A^k v / \sum A^k v, \quad k \rightarrow \infty.$$

算法 18.3 幂迭代方法

Algorithm 17: $PPAGERANK(A, v_0, \epsilon)$

Input: 方阵 A , 初始向量 v_0 , 精度 $\epsilon > 0$

Output: PageRank名次

- 1 设 $k = 0$
 - 2 计算 $v_{k+1} = Av_k$
 - 3 若 $|v_{k+1} - v_k| < \epsilon$, 则计算 $x = A^k v / \sum A^k v$, 算法终止; 否则 $k = k + 1$, 转第2步
-

利用幂迭代法的PageRank算法的程序如下:

```
function ppagerank(G, p, eps, snt)
% 利用幂迭代法的PageRank算法
% Input -G 邻接矩阵 -p p的值 -eps 计算精度 -snt 最大迭代次数
% Example
% G = [0 1 0; 1 0 0; 0 1 0];
% p = 0.85; ppagerank(G, p)
n = size(G, 1);
if nargin < 4, snt = 500; end
if nargin < 3, eps = 1e-6; end
if nargin < 2, p = 0.85; end
sn = sum(G, 1);
delta = (1 - p)/n; D = zeros(n, 1);
for j = 1:n
    if sn(j) == 0
        D(j) = 1 / n; G(:,j) = 1;
    else
        D(j) = 1 / sn(j);
    end
end
D = diag(D); A = p * G * D + delta;
x = ones(n,1) / n; z = zeros(n, 1); cnt = 0;
while max(abs(x - z)) > eps && cnt <= snt
    z = x; x = A * x; cnt = cnt + 1;
end
[x1, index] = sort(x); x1 = flipud(x1); index = flipud(index);
% 输出结果
out = [1:n; x1'; index'];
fprintf('迭代步数=%d时, 按PageRank得分排名\n', cnt);
str1 = '排名'; str2 = 'PageRank得分'; str3 = '序号';
```

```
fprintf('-----\n');
fprintf('  %4s%-13s%-5s\n', str1, str2, str3);
fprintf('-----\n');
fprintf('  %4d%-16s%-5d\n', out);
fprintf('-----\n');
```

例 18.5 对图 18.2 所示的小型网络, 按照幂迭代方法, 计算 6 个网页的排名, 其中的 $p = 0.85$ 。

```
>> G = zeros(6);
>> G(1,[4,6]) = 1; G(2,1) = 1; G(3,2) = 1;
>> G(4,[2,3]) = 1; G(5,3) = 1; G(6,[3,5]) = 1;
>> p = 0.85; ppagerank(G, p)
迭代步数=28时, 按PageRank得分排名
```

排名	PageRank得分	序号
1	2.675283e-01	1
2	2.523993e-01	2
3	1.697455e-01	4
4	1.322693e-01	3
5	1.155814e-01	6
6	6.247627e-02	5

18.3 问题与修正

仔细分析newpagerank和ppagerank的程序, 我们会发现一个致命的问题, 即要求有向图 D 的邻接矩阵 $G = \{g_{ij}\}$ 的列向量和不为0, 且上述两程序没有体现上述“若 $\exists j, g_{ij} = 0, \forall i$, 此时会导致 $n_j = 0$, 此时规定 $\frac{g_{ij}}{n_j} = \frac{1}{n}$ ”的思想。因此, 对于像下列图 18.4 所示的小型网络, 程序将无法运行(修改相应的 D 就会出问题)。注意图 18.4 与图 18.3 的区别。

那么, 应该如何修正这些程序呢?

有人给出了一个不错的程序, 程序如下:

```
function ppagerank2(G, p, eps, snt, mode)
% 利用幂迭代法的PageRank算法
% Input -G 邻接矩阵 -p p的值 -eps 计算精度 -snt 最大迭代次数 -mode 输出模式
% Example
% G = [0 1 0; 1 0 0; 0 1 0];
% p = 0.85; ppagerank2(G, p)
n = size(G, 1);
if nargin < 5, mode = 0; end
if nargin < 4, snt = 500; end
if nargin < 3, eps = 1e-6; end
if nargin < 2, p = 0.85; end
```

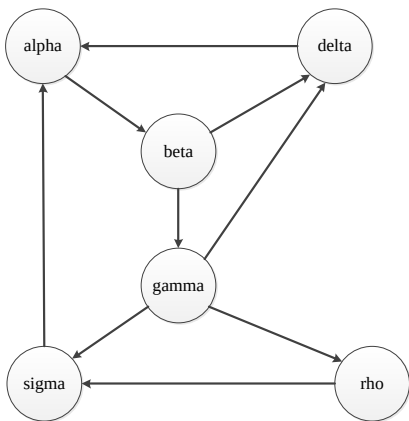


图 18.3: 示意图

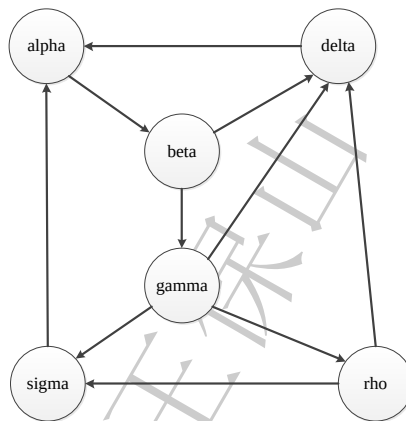


图 18.4: 示意图

```

sn = sum(G, 1); delta = (1 - p)/n; D = zeros(n, 1);
for j = 1:n
    if sn(j) == 0
        D(j) = 1 / n;
    else
        D(j) = 1 / sn(j);
    end
end
D = diag(D); A = p * G * D + delta;
x = ones(n, 1) / n; z = zeros(n, 1); cnt = 0;
while max(abs(x - z)) > eps && cnt <= snt
    z = x; x = A * x; cnt = cnt + 1;
end
[x1, index] = sort(x); x1 = flipud(x1); index = flipud(index);
% 输出结果
out = [1:n; x1'; index'];
fprintf('迭代步数 = %d\n', cnt);
str={'排名', 'PageRank得分', '序号'};
if mode
    fprintf('-----\n');
    fprintf('按PageRank得分排名 \n');
    fprintf('-----\n');
    fprintf('%-6s%-15s%-5s\n', str{1}, str{2}, str{3});
    fprintf('%-8d%-17f%-5d\n', out);
    fprintf('-----\n');
else
    x2 = x;
    [~, rank] = sort(index);
    out = [1:n; x2'; rank'];
    fprintf('-----\n');
    fprintf('按序号排列 \n');
    fprintf('-----\n');
    fprintf('%-6s%-15s%-5s\n', str{3}, str{2}, str{1});
    fprintf('%-8d%-17f%-5d\n', out);
    fprintf('-----\n');
end

```

虽然这是一个相当不错的程序，看起来很正确，其实仍然有问题(你能发现其中的问题并加以改正吗?)。

下面给出了一个标准的程序，来自非常权威的人物C. B. Moler的Numerical Computing With MATLAB的工具箱—ncm.zip中的函数pagerankpow.m(略有改动，命名为powpagerank.m)。

```
function powpagerank(G, p, s, eps, snt, mode)
% 算法PageRank
% Input -G 邻接矩阵 -p p的值 -s 顶点名称，元胞或字符串，长与G的长相等
%       -eps 计算精度 -snt 最大迭代次数 -mode 输出模式
% Example
%   G = [0 1 0; 1 0 0; 0 1 0];
%   s = {'w', 'u', 'v'}; p = 0.85;
%   powpagerank(G, p, s)
n = size(G, 1);
if nargin < 6, mode = 1; end
if nargin < 5, snt = 500; end
if nargin < 4, eps = 1e-6; end
if nargin < 3, s = num2cell((1:n)); mode = 0; end
if nargin < 2, p = 0.85; end
tic;
G = double(full(G));
for j = 1:n
    L{j} = find(G(:,j));
    c(j) = length(L{j});
end
delta = (1 - p) / n; x = ones(n,1) / n; z = zeros(n, 1); cnt = 0;
while max(abs(x-z)) > eps && cnt < snt
    z = x; x = ones(n, 1);
    for j = 1:n
        if c(j) == 0
            x = x + z(j) / n;
        else
            x(L{j}) = x(L{j}) + z(j) / c(j);
        end
    end
    x = p * x + delta; cnt = cnt + 1;
end
[x1, index] = sort(x); x1 = flipud(x1); index = flipud(index);
t = toc;
% 输出结果
txt = {'排名', '得分', '顶点', '原序号'};
if mode
    disp([txt; num2cell((1:n)'), num2cell(x1), s(index)', num2cell(index)])
else
    fprintf('按PageRank得分排名\n');
    fprintf('-----\n');
    fprintf('%-10s%-12s%-11s%\n', txt{1}, txt{2}, txt{3}, txt{4});
    fprintf('-----\n');
    for i = 1:n
        fprintf('%-7d%-14f%-12s%\n', i, x1(i), s(index(i)), index(i));
    end
    fprintf('-----\n');
end
fprintf('程序运行时间为: %12.4g 秒\n', t);
```

这个程序可以说是最高水平的MATLAB编程,几乎没有用到大规模的乘除法,而是以加减法为主。这大大节约了计算时间,更重要的是,该程序巧妙地使用了元胞数组,对于像邻接矩阵 G 这样的稀疏矩阵,其效率是惊人的,其优势是对于阶数庞大的稀疏矩阵中的大量0元素不作计算,而我们的上述程序都没有体现这一点。我们程序的缺点是明明看到像邻接矩阵这样的稀疏矩阵,但仍然把它看成一个普通矩阵,于是,大量的计算时间白白浪费。据报道, pagerankpow.m 程序的思想就是运行在Google网站的程序精髓。

例 18.6 下载 *C. B. Moler* 的 *Numerical Computing With MATLAB* 的工具箱—*ncm.zip*, 并安装在我们的 *MATLAB* (要求是 6.5 或以上版本) 的 *Toolbox* 文件夹中, 里面将出现 78 个文件。其中有一个文件 *harvard500.mat* 记录了 2003 年 8 月与哈佛大学的主页 <http://www.harvard.edu> 相关的 500 个网站的邻接矩阵, 我们可以用它来体会一下可以体现稀疏矩阵技术的 *pagerankpow.m* 程序的魅力。

解 首先介绍 *ncm* 工具箱¹ 的安装, 将下载好的 *ncm.zip* 解压到 *MATLAB* 安装目录的 *toolbox* 文件夹中(若不知道 *MATLAB* 安装在哪里可以使用 *winopen(matlabroot)* 命令进入安装目录下, 双击打开 *toolbox* 文件夹, 将解压好的 *ncm* 文件夹复制进去即可)。注意, 此时并没有完成 *ncm* 工具箱的安装, 因为我们还没有将 *ncm* 文件夹中的函数文件添加到搜索目录中, 不在搜索目录中的函数, 我们是无法调用的。

```
>> addpath([matlabroot, '\toolbox\ncm\'], '-frozen') % 将ncm文件夹中的函数文件添加到搜索目录中
>> savepath % 保存搜索目录, 否则再次启动MATLAB时ncm不再搜索路径中
>> help ncm % 验证是否安装完成
ncm M-files and data files for use with
Numerical Computing with MATLAB, (c) 2004, 2008, Cleve Moler.
See also ncmgui, blackjack, censusgui, eigsvdgui, fern, fftgui, floatgui,
fzerogui, interpgui, lorenzgui, lugui, pdegui, pennymelt, QUADGUI, RANDGUI,
SWINGER, TOUCHTONE, WALKER, WAVES, bslashtx, ffttx, fmintx, fzerotx, lutx,
membranetx, ode23tx, RANDTX, RANDNTX, QUADTX, pchiptx, SPLINETX
..... 结果从略 .....
```

一般来说, 工具箱的安装都是按照上述方式进行的, 添加搜索目录时也可以使用图形界面的形式(通过“home”菜单中的“Set Path”按钮进行安装)。

```
>> load harvard500 % 导入500个网站的邻接矩阵G,同时得到500个网站的地址名称矩阵U
>> spy(G); % 用spy函数看看稀疏矩阵G的稀疏形状, 结果如图 18.5
>> G = double(full(G)); % 将稀疏矩阵G转化方阵再转化为数值方阵
>> [x, cnt] = pagerankpow(G); % 调用pagerankpow求排名向量x, 以及收敛时的迭代次数cnt
>> pagerank(U, G); xlim([1, 500]) % 按PageRank大小给出前几十名的列表与图示等, 设置x轴范围, 结果如图 18.6
```

page-rank	in	out	url
1	0.0843	195	26 http://www.harvard.edu
10	0.0167	21	18 http://www.hbs.edu
42	0.0166	42	0 http://search.harvard.edu:8765/custom/query.html
130	0.0163	24	12 http://www.med.harvard.edu

¹ 相关介绍详见网页(http://cn.mathworks.com/moler/index_ncm.html);

其下载网址为<http://cn.mathworks.com/matlabcentral/fileexchange/37976-numerical-computing-with-matlab>.

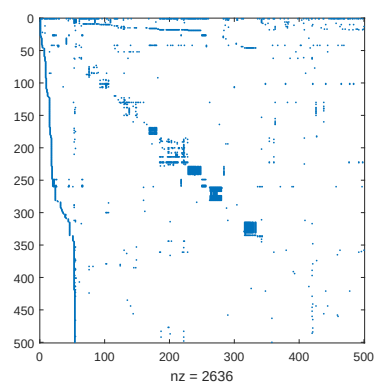


图 18.5: 稀疏矩阵的形状

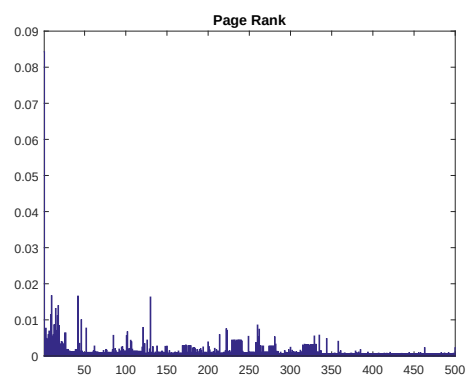


图 18.6: PageRank图示

```

18  0.0139  45  46  http://www.gse.harvard.edu
15  0.0131  16  49  http://www.hms.harvard.edu
 9  0.0114  21  27  http://www.ksg.harvard.edu
17  0.0111  13   6  http://www.hsph.harvard.edu
46  0.0100  18  21  http://www.gocrimson.com
13  0.0086   9   1  http://www.hsdm.med.harvard.edu
260 0.0086  26   1  http://search.harvard.edu:8765/query.html
19  0.0084  23  21  http://www.radcliffe.edu
121 0.0079   1   3  http://www.hsdm.harvard.edu
52  0.0077  11   2  http://whitepages.med.harvard.edu
 3  0.0077  21  12  http://lib.harvard.edu
222 0.0075  36  34  http://www.gse.harvard.edu/sitemap.html
262 0.0074  20   3  http://www.dana-farber.org
223 0.0071  36  10  http://www.gse.harvard.edu/search.html
16  0.0071  13  13  http://www.hds.harvard.edu
..... 结果从略 .....

```

18.4 实验报告

1. 将例 18.4 和例 18.5 中的 $p = 0.85$ 修改为 $p = 0.75, 0.80, 0.90$, 观察PageRank得分和排名结果的变化情况。

解 (1) 对例 18.4

```

G = zeros(6);
G(1,[4,6]) = 1; G(2,1) = 1; G(3,2) = 1;
G(4,[2,3]) = 1; G(5,3) = 1; G(6,[3,5]) = 1;
s = {'alpha', 'beta', 'gamma', 'delta', 'rho', 'sigma'};
for p = [0.75, 0.8, 0.9]
    disp(['p= ', num2str(p)])
    newpagerank(G, p, s)
end

```

结果为

```
p = 0.75
    '排名',      '得分',      '顶点',      '原序号',
    [ 1]      [0.2622]      'alpha',      [ 1]
    [ 2]      [0.2383]      'beta',      [ 2]
    [ 3]      [0.1638]      'delta',      [ 4]
    [ 4]      [0.1310]      'gamma',      [ 3]
    [ 5]      [0.1302]      'sigma',      [ 6]
    [ 6]      [0.0744]      'rho',      [ 5]

p = 0.8
    '排名',      '得分',      '顶点',      '原序号',
    [ 1]      [0.2651]      'alpha',      [ 1]
    [ 2]      [0.2454]      'beta',      [ 2]
    [ 3]      [0.1666]      'delta',      [ 4]
    [ 4]      [0.1315]      'gamma',      [ 3]
    [ 5]      [0.1231]      'sigma',      [ 6]
    [ 6]      [0.0684]      'rho',      [ 5]

p = 0.9
    '排名',      '得分',      '顶点',      '原序号',
    [ 1]      [0.2696]      'alpha',      [ 1]
    [ 2]      [0.2593]      'beta',      [ 2]
    [ 3]      [0.1734]      'delta',      [ 4]
    [ 4]      [0.1334]      'gamma',      [ 3]
    [ 5]      [0.1077]      'sigma',      [ 6]
    [ 6]      [0.0567]      'rho',      [ 5]
```

对例 18.5

```
G = zeros(6);
G(1,[4,6]) = 1; G(2,1) = 1; G(3,2) = 1;
G(4,[2,3]) = 1; G(5,3) = 1; G(6,[3,5]) = 1;
for p = [0.75, 0.8, 0.9]
    disp(['p= ', num2str(p)])
    ppagerank(G, p)
end
```

结果为

```
p = 0.75
迭代步数=22时, 按PageRank得分排名
-----
排名    PageRank得分    序号
-----
1       2.621932e-01     1
2       2.383118e-01     2
3       1.637923e-01     4
4       1.310339e-01     3
5       1.302437e-01     6
6       7.442505e-02     5
-----

p = 0.8
```


迭代步数=25时, 按PageRank得分排名

排名	PageRank得分	序号
1	2.650651e-01	1
2	2.453859e-01	2
3	1.665509e-01	4
4	1.314877e-01	3
5	1.231139e-01	6
6	6.839655e-02	5

$p = 0.9$

迭代步数=32时, 按PageRank得分排名

排名	PageRank得分	序号
1	2.696081e-01	1
2	2.593144e-01	2
3	1.733651e-01	4
4	1.333578e-01	3
5	1.076807e-01	6
6	5.667391e-02	5

从结果看未发生变化, 排名前四名的PageRank得分随 p 的增大而增大, 后二名随 p 的增大而减小。

2. 计算图 18.1 所示的小型网络的排名。

```
>> G = [0 1 0; 1 0 0; 0 1 0];
>> pagerank(G)
迭代步数=22时, 按PageRank得分排名
```

排名	PageRank得分	序号
1	3.936168e-01	2
2	3.031916e-01	3
3	3.031916e-01	1

```
>> s = {'u', 'v', 'w'}; p = 0.85;
>> newpagerank(G, p, s)
    '排名'      '得分'      '顶点'      '原序号'
    [ 1]      [0.3936]      'v'        [ 2]
    [ 2]      [0.3032]      'w'        [ 3]
    [ 3]      [0.3032]      'u'        [ 1]
>> pagerank2(G, p, 1e-3)
迭代步数 = 17
```

序号	PageRank得分	排名
1	0.002044	3
2	0.002804	1
3	0.002044	2

```
>> powpagerank(G, p, s)
```

```

'排名'      '得分'      '顶点'      '原序号'
[ 1]      [7.0851]      'v'      [ 2]
[ 2]      [5.4574]      'w'      [ 3]
[ 3]      [5.4574]      'u'      [ 1]
程序运行时间为:      0.008755 秒

```

从结果看, v 在排名上高于 w , w 高于 u 。

3. 计算图 18.4 所示的小型网络的排名, 分析其排名与图 18.4 所示的小型网络的排名发生变化的原因。

```

>> G = zeros(6);
>> G(1,[4,6]) = 1; G(2,1) = 1; G(3,[1,2]) = 1;
>> G(4,[2,3,5]) = 1; G(5,3) = 1; G(6,[3,5]) = 1;
>> p = 0.85;
>> s = {'alpha', 'beta', 'gamma', 'delta', 'rho', 'sigma'};
>> powpagerank(G, p, s)
'排名'      '得分'      '顶点'      '原序号'
[ 1]      [9.6722]      'alpha'      [ 1]
[ 2]      [7.1046]      'gamma'      [ 3]
[ 3]      [6.2343]      'delta'      [ 4]
[ 4]      [4.9857]      'beta'      [ 2]
[ 5]      [4.1153]      'sigma'      [ 6]
[ 6]      [2.8880]      'rho'      [ 5]
程序运行时间为:      0.005709 秒

```

排名发生变化的原因是 γ 增加了 α 的入度, δ 增加了 ρ 的入度。

4. 查看 ncm 工具箱中的 surfer.m 文件的作用。

解 surfer 函数可搜索从根网站出发的 n 个相关联的网站, 并建立相应的地址列表 U 与邻接矩阵 G 。

5. 用命令

```
[U, G] = surfer('http://www.ouc.edu.cn', 100);
```

建立与中国海洋大学主页有关的 100 个网页的网址矩阵和邻接矩阵, 请问中国海洋大学图书馆主页 (<http://library.ouc.edu.cn>) 排在第几名? PageRank 是多少? 它有多少个入度和出度? 中国海洋大学新闻网主页 (<http://xinwen.ouc.edu.cn>) 呢?

```

>> [U, G] = surfer('http://www.ouc.edu.cn', 100); % 结果如图 18.7
>> G = double(full(G));
>> figure
>> pagerank(U, G); xlim([1, 100]) % 结果如图 18.8
    page-rank    in    out    url
1    0.0999    28    71    http://www.ouc.edu.cn
4    0.0611    24    70    http://www.ouc.edu.cn/main.htm

```

```

50  0.0371  4  0  http://www.oceanol.com
54  0.0365  9  1  http://www.soa.gov.cn
37  0.0320 12  2  http://newsvideo.ouc.edu.cn
26  0.0282  9  9  http://xinwen.ouc.edu.cn
55  0.0228  8  1  http://www.moe.edu.cn
74  0.0226  4 71  http://www.ouc.edu.cn/index.htm
76  0.0197  8  0  http://web1.ouc.edu.cn/_s112/main.psp
 9  0.0167 11  0  http://iec.ouc.edu.cn/article
36  0.0164 12  1  http://xinwen.ouc.edu.cn/ghttxsz/hdyxzy/index.html
16  0.0157 13  2  http://www2.ouc.edu.cn/zsb/zsb
 3  0.0146 12 31  http://eweb.ouc.edu.cn
11  0.0142 11  3  http://web.ouc.edu.cn/grad
19  0.0136 12  1  http://library.ouc.edu.cn
15  0.0114 10  3  http://web.ouc.edu.cn/yzb
17  0.0105 10  1  http://iec.ouc.edu.cn
58  0.0104  7  3  http://hqs.ouc.edu.cn
..... 结果从略 .....

```

故中国海洋大学图书馆主页排在第16名，PageRank是0.0136，它有12个入度和1个出度。中国海洋大学新闻网主页则排在第6名，PageRank是0.0282，它有9个入度和9个出度。

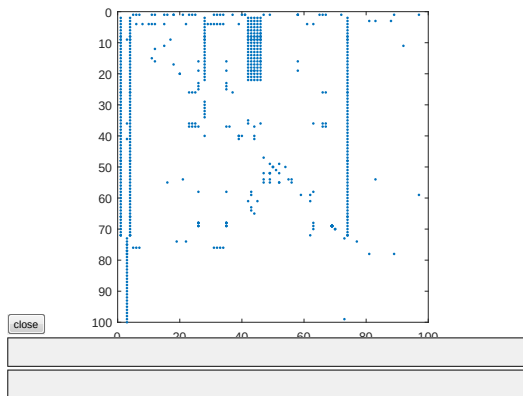


图 18.7: 稀疏矩阵的形状

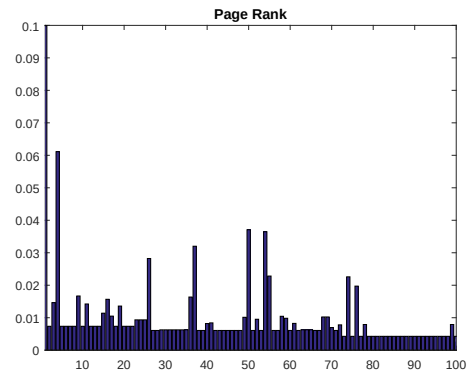


图 18.8: PageRank图示

6. 用命令

```
[U, G] = surfer('http://www.ecnu.edu.cn', 500)
```

建立与华东师范大学主页有关的500个网页的网址矩阵和邻接矩阵。华东师范大学数学系主页处于什么位置？

```

>> [U, G] = surfer('http://www.ecnu.edu.cn', 500); % 结果如图 18.9
>> G = double(full(G));

```

```
>> figure
>> x = pagerank(U, G); xlim([1, 500])           % 结果如图 18.10
    page-rank  in  out  url
218    0.0415   66    0  http://www.macromedia.com/go/getflashplayer
1     0.0413  162  143  http://www.ecnu.edu.cn
143    0.0285   65    0  http://www.eoffice.ecnu.edu.cn
433    0.0252   12   12  http://wmzx.ecnu.edu.cn/main.htm
173    0.0153   17    0  http://
434    0.0146   12    1  http://wmzx.ecnu.edu.cn/1416/list.htm
302    0.0119   24    0  http://www
212    0.0111   19    5  http://www.moe.gov.cn
2     0.0105   48   21  http://english.ecnu.edu.cn
3     0.0103   48   16  http://french.ecnu.edu.cn
331    0.0094   19    2  http://www.most.gov.cn
322    0.0090   26    6  http://www.moe.edu.cn
62     0.0084   38    3  http://www.lib.ecnu.edu.cn
23     0.0080   41   32  http://greatchina.ecnu.edu.cn
57     0.0079   41    8  http://news.ecnu.edu.cn
..... 结果从略 .....
>> n = strcmp(U, 'http://math.ecnu.edu.cn');
>> xn = x(n), r = sum(x > xn)
xn =
    0.0016
r =
    123
```

查询结果可知，华师大数学系主页排在第123名，得分为0.0016。

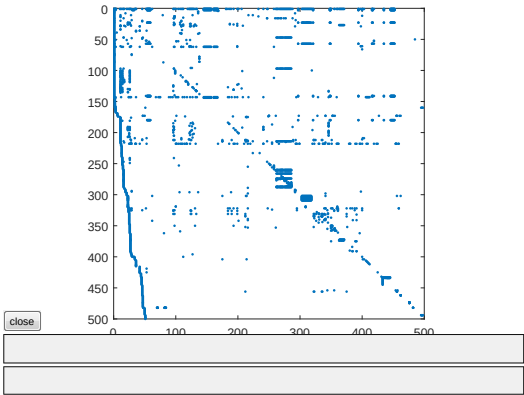


图 18.9: 稀疏矩阵的形状

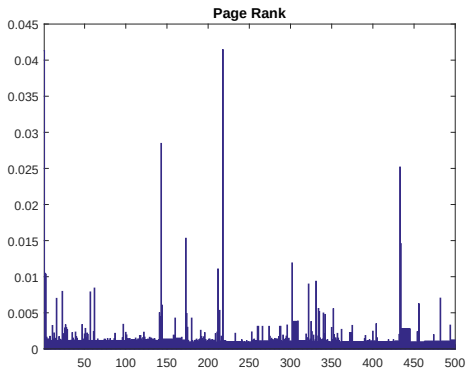


图 18.10: PageRank图示

第十九章 遗传算法

19.1 预备知识

遗传算法(Genetic Algorithm—GA)，是模拟达尔文的遗传选择和自然淘汰的生物进化过程的计算模型，它是由美国Michigan大学的J. Holland教授于1975年首先提出的。遗传算法作为一种新的全局优化搜索算法，以其简单通用、鲁棒性强、适于并行处理及应用范围广等显著特点，奠定了它作为21世纪关键智能计算之一的地位。本章将首先介绍一下遗传算法的基本理论，然后用其解决几个简单的函数最值问题，使读者能够学会利用遗传算法进行初步的优化计算。

19.1.1 遗传算法的基本原理

遗传算法的基本思想正是基于模仿生物界遗传学的遗传过程。它把问题的参数用基因代表，把问题的解用染色体代表(在计算机里用二进制码表示)，从而得到一个由具有不同染色体的个体组成的群体。这个群体在问题特定的环境里生存竞争，适者有最好的机会生存和产生后代，后代随机化地继承了父代的最好特征，并也在生存环境的控制支配下继续这一过程。群体的染色体都将逐渐适应环境，不断进化，最后收敛到一族最适应环境的类似个体，即得到问题最优的解。值得注意的一点是，现在的遗传算法是受生物进化论学说的启发提出的，这种学说对我们用计算机解决复杂问题很有用，而它本身是否完全正确并不重要(目前生物界对此学说尚有争议)。

由于遗传算法是由进化论和遗传学机理而产生的直接搜索优化方法；故而在该算法中要用到各种进化和遗传学的概念。表 19.1 给出了遗传学概念、遗传算法概念和相应的数学概念三者之间的对应关系。

遗传算法计算优化的操作过程就如同生物学上生物遗传进化的过程，主要有三个基本操作(或称为算子)：选择(Selection)、交叉(Crossover)、变异(Mutation)。

遗传算法基本步骤主要是：先把问题的解表示成“染色体”，在算法中也就是以二进制编码的串，在执行遗传算法之前，给出一群“染色体”，也就是假设的可行解。然后，把这些假设的可行解置于问题的“环境”中，并按适者生存的原则，从中选择出较适应环境的“染色体”进行复制，再通过交叉、变异过程产生更适应环境的新一代“染色体”群。经过这样的一代又一代地进化，最后就会收敛到最适应环境的一个“染色体”上，它就是问题的最优解。

表 19.1: 基本概念

遗传学概念	遗传算法概念	数学概念
个体	要处理的基本对象、结构	可行解
群体	个体的集合	被选定的一组可行解
染色体	个体的表现形式	可行解的编码
基因	染色体中的元素	编码中的元素
基因位	某一基因在染色体中的位置	元素在编码中的位置
适应值	个体对于环境的适应程度，或在环境压力下的生存能力	可行解所对应的适应函数值
种群	被选定的一组染色体或个体	根据入选概率定出的一组可行解
选择	从群体中选择优胜的个体，淘汰劣质个体的操作	保留或复制适应值大的可行解，去掉小的可行解
交叉	一组染色体上对应基因段的交换	根据交叉原则产生的一组新解
交叉概率	染色体对应基因段交换的概率	闭区间[0,1]上的值，常取0.65 ~ 0.90
变异	染色体水平上基因的变化	编码的某些元素被改变
变异概率	染色体上基因变化的概率	(0,1)内的值，一般为0.001 ~ 0.01
进化、适者生存	个体进行优胜劣汰的进化，逐代地优化	目标函数取到最大值，最优的可行解

算法 19.1 遗传算法

Algorithm 18: GA
Input: 参数集合
Output: 最优解
1 选择编码策略，把参数集合(可行解集合)转换染色体结构空间
2 定义适应函数，便于计算适应值
3 确定遗传策略，包括选择群体大小，选择、交叉、变异方法以及确定交叉概率、变异概率等遗传参数
4 随机产生初始化群体
5 计算群体中的个体或染色体解码后的适应值
6 按照遗传策略，运用选择、交叉和变异算子作用于群体，形成下一代群体
7 判断群体性能是否满足某一指标、或者是否已完成预定的迭代次数，不满足则返回第5步或者修改遗传策略再返回第6步

其流程如图 19.1 所示。

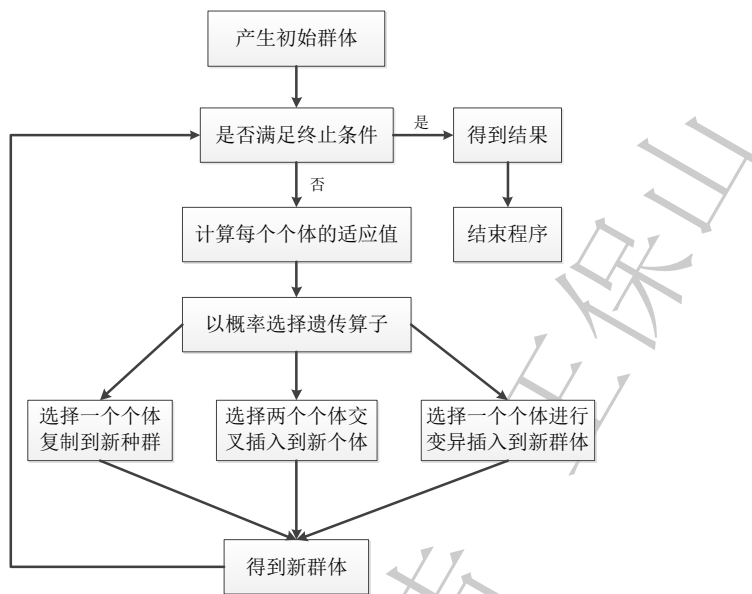


图 19.1: 遗传算法流程图

遗传算法有很多种具体的不同实现过程，以上介绍的是标准遗传算法的主要步骤，此算法会一直运行直到找到满足条件的最优解为止。

19.1.2 遗传算法的实际应用

例 19.1 设 $f(x) = -x^2 + 2x + 0.5$ ，用遗传算法求 $\max f(x)$, $x \in [-1, 2]$ 。

解 遗传算法的整个过程如下：

(1) 编码和产生初始群体

首先第一步要确定编码的策略，也就是说如何把 -1 到 2 这个区间内的数用计算机语言表示出来。

编码就是表现型到基因型的映射，编码时要注意以下三个原则：

- (a) 完备性：问题空间中所有点(潜在解)都能成为GA编码空间中的点(染色体位串)的表现型；
- (b) 健全性：GA编码空间中的染色体位串必须对应问题空间中的某一潜在解；
- (c) 非冗余性：染色体和潜在解必须一一对应。

这里我们通过采用二进制的形式来解决编码问题，将某个变量值代表的个体表示为一个 $\{0, 1\}$ 二进制串。当然，串长取决于求解的精度。如果要设定求解精度到六位小数，由于区间长度为 $2 - (-1) = 3$ ，则必须将闭区间 $[-1, 2]$ 分为 3×10^6 等分。因为 $2097152 = 2^{21} < 3 \times 10^6 < 2^{22} = 4194304$ ，所以编码的二进制串至少需要22位。

将一个二进制串 $(b_{21}b_{20}b_{19} \cdots b_1b_0)$ 转化为区间 $[-1, 2]$ 内对应的实数值很简单，只需采取以下两步：

(a) 将一个二进制串($b_{21}b_{20}b_{19}\cdots b_1b_0$)代表的二进制数化为10进制数:

$$(b_{21}b_{20}b_{19}\cdots b_1b_0)_2 = \left(\sum_{i=0}^{21} b_i \cdot 2^i\right)_1 0 = x',$$

(b) x' 对应的区间 $[-1, 2]$ 内的实数:

$$x = -1 + x' \cdot \frac{2 - (-1)}{2^{22} - 1}.$$

例如, 一个二进制串 $a = < 1000101110110101000111 >$ 表示实数0.637197。

计算得 $x' = (1000101110110101000111)_2 = 2288967$, $x = -1 + 2288967 \cdot \frac{3}{2^{22}-1} = 0.637197$ 。

二进制串 $< 0000000000000000000000 >$, $< 1111111111111111111111 >$, 则分别表示区间的两个端点值-1和2。

利用这种方法我们就完成了遗传算法的第一步——编码, 这种二进制编码的方法完全符合上述的编码的三个原则。

首先我们来随机地产生一个个体数为4个的初始群体:

$$\begin{aligned} pop(1) = \{ & \\ & < 1101011101001100011110 >, & a_1 \\ & < 1000011001010001000010 >, & a_2 \\ & < 0001100111010110000000 >, & a_3 \\ & < 0110101001101110010101 >, & a_4 \\ & \} \end{aligned}$$

化成十进制的数分别为: $pop(1) = \{1.523032, 0.574022, -0.697235, 0.247238\}$ 。接下来我们就要解决每个染色体个体的适应值问题了。

(2) 定义适应函数和适应值

由于给定的目标函数 $f(x) = -x^2 + 2x + 0.5$ 在区间 $[-1, 2]$ 内的值有正有负, 所以必须通过建立适应函数与目标函数的映射关系, 保证映射后的适应值非负, 而且目标函数的优化方向应对应于适应值增大的方向, 也为以后计算各个体的入选概率打下基础。

对于本题中的最大化问题, 定义适应函数, 采用下述方法:

$$g(x) = \begin{cases} f(x) - F_{\min}, & \text{若 } f(x) - F_{\min} > 0, \\ 0, & \text{其他,} \end{cases}$$

式中 $F_{\min}$ 既可以是特定的输入值, 也可以是当前所有代或最近 K 代中 $f(x)$ 的最小值, 这里为了便于计算, 将采用了一个特定的输入值。

若取 $F_{\min} = -1$, 则当 $f(x) = 1$ 时适应函数 $g(x) = 2$; 当 $f(x) = -1.1$ 时适应函数 $g(x) = 0$ 。

由上述所随机产生的初始群体, 我们可以先计算出目标函数值,

$$f[pop(1)] = \{1.226437, 1.318543, -1.380607, 0.933350\},$$

然后取 $F_{\min} = -1$ ，通过适应函数计算出适应值，

$$g[\text{pop}(1)] = \{2.226437, 2.318543, 0, 1.933350\}.$$

(3) 确定选择标准

这里我们用到了适应值的比例来作为选择的标准，得到的每个个体的适应值比例叫作入选概率。其计算公式如下：

对于给定的规模为 n 的群体 $\text{pop} = \{a_1, a_2, \dots, a_n\}$ ，个体 a_i 的适应值为 $g(a_i)$ ，则其入选概率为

$$P_s(a_i) = \frac{g(a_i)}{\sum_{i=1}^n g(a_i)}, \quad i = 1, 2, \dots, n.$$

由上述给出的群体，我们可以计算出各个个体的入选概率。

首先得 $\sum_{i=1}^4 g(a_i) = 6.478330$ ，然后分别计算出4个个体的入选概率，得

$$P(a_1, a_2, a_3, a_4) = \{0.343675, 0.357892, 0, 0.298433\}.$$

(4) 产生种群

计算完了入选概率后，就将入选概率大的个体选入种群，淘汰概率小的个体，并用入选概率最大的个体补入种群，得到与原群体大小同样的种群。

由初始群体的入选概率我们淘汰掉 a_3 ，再加入 a_2 补足成与群体同样大小的种群得到 $\text{newpop}(1)$ ，

$$\begin{aligned} \text{newpop}(1) = \{ & \\ & \langle 1101011101001100011110 \rangle, \quad a_1 \\ & \langle 1000011001010001000010 \rangle, \quad a_2 \\ & \langle 1000011001010001000010 \rangle, \quad a_3 \\ & \langle 0110101001101110010101 \rangle, \quad a_4 \\ & \} \end{aligned}$$

(5) 交叉

交叉也就是将一组染色体上对应基因段的交换得到新的染色体，然后得到新的染色体组，组成新的群体。

如图 19.2，我们把之前得到的 $\text{newpop}(1)$ 的四个个体两两组成一对，重复的不配对，进行交叉(可以在任一位进行交叉)。

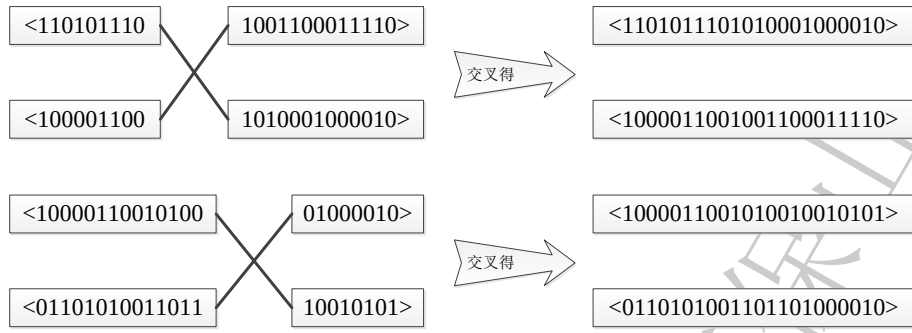


图 19.2: 交叉互换

通过交叉得到了四个新个体，得到新的群体jchpop(1)，

$$jchpop(1) = \{$$

$$\begin{aligned} &< 1101011101010001000010 >, & a_1 \\ &< 1000011001001100011110 >, & a_2 \\ &< 1000011001010010010101 >, & a_3 \\ &< 0110101001101101000010 >, & a_4 \end{aligned}$$

$$\}$$

这里采用的是单点交叉的方法，当然还有多点交叉的方法(不过有些烦琐，这里就不介绍了)。

(6) 变异

变异也就是通过一个小概率改变染色体位串上的某个基因。

现把刚得到的jchpop(1)中第3个个体中的第9位改变，就产生了变异，得到了新的群体pop(2)，

$$jchpop(1) = \{$$

$$\begin{aligned} &< 1101011101010001000010 >, & a_1 \\ &< 1000011001001100011110 >, & a_2 \\ &< 1000011011010010010101 >, & a_3 \\ &< 0110101001101101000010 >, & a_4 \end{aligned}$$

$$\}$$

然后重复上述的选择、交叉、变异直到满足终止条件为止。

(7) 终止条件

遗传算法的终止条件有两类常见条件：

(a) 采用设定最大(遗传)代数数的方法，一般可设定为50代，此时就可能得出最优解。此种方法简单易行，但可能不是很精确；

(b) 根据个体的差异来判断, 通过计算种群中基因多样性测度, 即所有基因位相似程度来进行控制。

19.1.3 遗传算法的收敛性

前面我们已经就遗传算法中的编码、适应度函数、选择、交叉和变异等主要操作的基本内容及设计进行了详细的介绍。作为一种搜索算法, 遗传算法通过对这些操作的适当设计和运行, 可以实现兼顾全局搜索和局部搜索的所谓均衡搜索, 具体实现见图 19.3 所示。

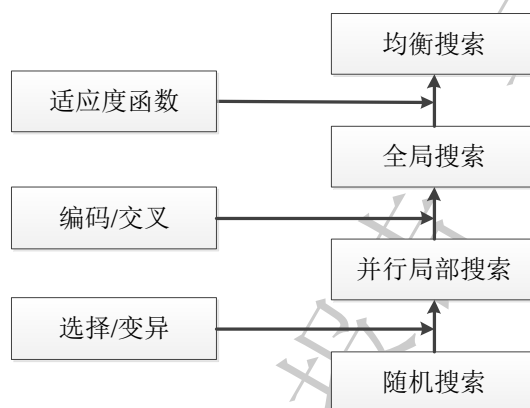


图 19.3: 均衡搜索的具体实现图示

应该指出的是, 遗传算法虽然可以实现均衡的搜索, 并且在许多复杂问题的求解中往往能得到满意的结果, 但是该算法的全局优化收敛性的理论分析尚待解决。目前普遍认为, 标准遗传算法并不保证全局最优收敛。但是, 在一定的约束条件下, 遗传算法可以实现这一点。

下面我们不加证明地罗列几个定理或定义, 供读者参考(在这些定理的证明中, 要用到许多概率论和随机过程的知识, 特别是有关马尔可夫链的理论, 读者可参阅有关文献)。

定理 19.1 如果变异概率为 $P_m \in (0, 1)$, 交叉概率为 $P_c \in [0, 1]$, 同时采用比例选择法(按个体适应度占群体适应度的比例进行复制), 则标准遗传算法的变换矩阵 P 是基本的。

定理 19.2 标准遗传算法(参数如定理 19.1)不能收敛至全局最优解。

由定理 19.2 可知, 具有变异概率 $P_m \in (0, 1)$, 交叉概率为 $P_c \in [0, 1]$ 以及按比例选择的标准遗传算法是不能收敛至全局最最优解。我们在前面求解例 19.1 时所用的方法就是满足定理 19.1 的条件的方法。这无疑是一个令人沮丧的结论。

然而庆幸的是, 只要对标准遗传算法作一些改进, 就能够保证其收敛性。我们对标准遗传算法作一定改进, 即不按比例进行选择, 而是保留当前所得的最优解(称作超个体), 该超个体不参与遗传。

最佳个体保存方法(Elitist model)的思想是把群体中适应度最高的个体不进行配对交叉而直接复制到下一代中。此种选择操作又称复制(Copy)。De Jong 对此方法作了如下定义:

设到时刻 t (第 t 代)时, 群体中 $a^*(t)$ 为最佳个体。又设 $A(t+1)$ 为新一代群体, 若 $A(t+1)$ 中不存在 $a^*(t)$, 则把 $a^*(t)$ 作为 $A(t+1)$ 中的第 $n+1$ 个个体(其中 n 为群体大小)。

采用此选择方法的优点是, 进化过程中某一代的最优解可不被交叉和变异操作所破坏。但是, 这也隐含了一种危机, 即局部最优个体的遗传基因会急速增加而使进化有可能限于局部解。也就是说, 该方法的全局搜索能力差, 它更适合单峰性质的搜索空间搜索, 而不是多峰性质的空间搜索。所以此方法一般都与其他选择方法结合使用。

定理 19.3 具有定理 19.1 所示参数, 选择后保留当前最优值的遗传算法最终能收敛到全局最优解。

当然, 在选择算子作用后保留当前最优解是一项比较复杂的工作, 因为该解在选择算子作用后可能丢失。但是定理 19.3 至少表明了这种改进的遗传算法能够收敛至全局最优解。有意思的是, 实际上只要在选前保留当前最优解, 就可以保证收敛, 定理 19.4 描述了这种情况。

定理 19.4 具有定理 19.1 所示参数, 且在选择前保留当前最优解的遗传算法可收敛于全局最优解。

19.2 算法程序

下面是遗传算法的程序, 要说明的是: 为保证收敛性, 程序采用了最佳个体保存方法(Elitist model), 终止条件为第一类终止条件。

```
function [Cnt, Result, BestMember] = Genetic(MumberLength, MemberNumber, ...
    FunctionFitness, MinX, MaxX, Fmin, MutationProbability, Gen)
% 遗传算法的主程序
% Input    -MumberLength 表示一个染色体位串的二进制长度
%           -MemberNumber 表示群体中染色体的个数
%           -FunctionFitness 表示目标函数, 可以是字符串表达式
%           -MinX 变量区间的下限 -MaxX 变量区间的上限
%           -Fmin 定义适应函数过程中给出的一个目标函数的可能的最小值
%           -MutationProbability 表示变异的概率, 一般都很小
%           -Gen 表示遗传的代数, 也就是终止程序的代数
% Output   -Cnt 表示遗传的代数
%           -Result 表示计算结果, 也就是最优解
%           -BestMember 表示最优个体及其适应值
Population = PopulationInitialize(MumberLength, MemberNumber);
global Count CurrentBest
Count = 1;
PopulationCode = Population;
PopulationFitness = Fitness(PopulationCode, FunctionFitness, MinX, MaxX, MumberLength);
PopulationFitnessF = FitnessF(PopulationFitness, Fmin);
PopulationProbability = Probability(PopulationFitnessF);
[Population, EachGenMaxFitness] = Elitist(PopulationCode, PopulationFitness, MumberLength);
EachMaxFitness(Count) = EachGenMaxFitness;
MaxFitness(Count) = CurrentBest(length(CurrentBest));
while Count < Gen
    NewPopulation = Select(Population, PopulationProbability, MemberNumber);
```

```

    Population = NewPopulation;
    NewPopulation = Crossing(Population);
    Population = NewPopulation;
    NewPopulation = Mutation(Population, MutationProbability);
    Population = NewPopulation;
    PopulationFitness = Fitness(Population, FunctionFitness, MinX, MaxX, MumberLength);
    PopulationFitnessF = FitnessF(PopulationFitness, Fmin);
    PopulationProbability = Probability(PopulationFitnessF);
    Count = Count + 1;
    [NewPopulation, EachGenMaxFitness] = Elitist(Population, ...
        PopulationFitness, MumberLength);
    EachMaxFitness(Count) = EachGenMaxFitness;
    MaxFitness(Count) = CurrentBest(length(CurrentBest));
    Population = NewPopulation;
end
Dim = size(Population, 1);
Result = ones(2, Dim);
for i = 1:Dim
    Result(1,i) = Translate(Population(i,:), MinX, MaxX, MumberLength);
end
Result(2,:) = PopulationFitness;
BestMember(1,1) = Translate(CurrentBest(1:MumberLength), MinX, MaxX, MumberLength);
BestMember(2,1) = CurrentBest(MumberLength+1);
Cnt = Count;
figure
subplot(211), plot(EachMaxFitness), ylabel('每一代的最大适应值')
subplot(212), plot(MaxFitness), ylabel('保留最佳个体的最大适应值')
xlabel('第n代群体')
end

function Population = PopulationInitialize(MumberLength, MumberNumber)
% 子程序1: 用于产生一个初始群体, 这个初始群体含有MumberLength个基因(二进制码)
Temporary = rand(MumberNumber, MumberLength);
Population = (Temporary >= 0.5);
end

function PopulationFitness = Fitness(PopulationCode, FunctionFitness, ...
    MinX, MaxX, MumberLength)
% 子程序2: 用于计算群体中每一个染色体的目标函数值
% Input -PopulationCode 表示用0-1代码表示的群体
%         -FunctionFitness 表示目标函数, 可以是一个字符串表达式
%         -MumberLength 表示染色体位串的二进制长度
%         -[MinX, MaxX] 分别指变量区间的上下限
Dim = size(PopulationCode, 1);
PopulationFitness = zeros(1, Dim);
for i = 1:Dim
    PopulationFitness(i) = Transfer(PopulationCode(i,:), ...
        FunctionFitness, MinX, MaxX, MumberLength);
end
end

function PopulationData = Translate(PopulationCode, MinX, MaxX, MumberLength)
% 子程序3: 把变成码的群体翻译成变量的数值
% Input -PopulationCode 群体编码 -[MinX, MaxX] 变量区间的上下限
%         -MumberLength 染色体位串的二进制长度
PopulationData = 0;
Dim = size(PopulationCode, 2);
for i = 1:Dim

```

```

    PopulationData = PopulationData + PopulationCode(i) * (2^(MumberLength-i));
end
PopulationData = MinX + PopulationData * (MaxX - MinX) / (2 ^ Dim-1);
end

function PopulationFitness = Transfer(PopulationCode, FunctionFitness, ...
    MinX, MaxX, MumberLength)
% 子程序4: 把群体中的染色体的目标函数值用数值表示出来, 是Fitness的重要子程序
PopulationData = Translate(PopulationCode, MinX, MaxX, MumberLength);
PopulationFitness = double(subs(FunctionFitness, 'x', sym(PopulationData)));
end

function PopulationFitnessF = FitnessF(PopulationFitness, Fmin)
% 子程序5: 用于计算每个染色体的适应函数值
% Input -PopulationFitness 群体中染色体的目标函数值
% -Fmin 定义函数参数过程中给出的一个目标函数的可能的最小值
Dim = size(PopulationFitness, 2);
PopulationFitnessF = zeros(1, Dim);
for i = 1:Dim
    if PopulationFitness(i) > Fmin
        PopulationFitnessF(i) = PopulationFitness(i) - Fmin;
    else
        PopulationFitnessF(i) = 0;
    end
end
end

function PopulationProbability = Probability(PopulationFitness)
% 子程序6: 用于计算群体中每个染色体的入选概率
% Input -PopulationFitness 群体中染色体的适应函数值
SumPopulationFitness = sum(PopulationFitness);
PopulationProbability = PopulationFitness / SumPopulationFitness;
end

function NewPopulation = Select(Population, PopulationProbability, MemberNumber)
% 子程序7: 根据入选概率(计算累计概率)在群体中按比例选择部分染色体组成种群
% Input -Population 群体 -PopulationProbability 入选概率
% -MemberNumber 群体中染色体的个数
NewPopulation = Population;
CProbability = cumsum(PopulationProbability); % 计算累计概率
for i = 1:MemberNumber
    r = rand(1);
    Index = 1;
    while r > CProbability(Index)
        Index = Index + 1;
    end
    NewPopulation(i,:) = Population(Index,:);
end
end

function NewPopulation = Crossing(Population)
% 子程序8: _____
Dim = size(Population);
if Dim(1) >= 3
    Temp = Population(Dim(1),:);
    Population(Dim(1),:) = Population(Dim(1)-1,:);
    Population(Dim(1)-1,:) = Temp;
end
end

```

```

for i = 1:2:Dim(1)-1
    SiteArray = randperm(Dim(2));
    Site = SiteArray(1);
    Temp = Population(i,1:Site);
    Population(i,1:Site) = Population(i+1,1:Site);
    Population(i+1,1:Site) = Temp;
end
NewPopulation = Population;
end

function NewPopulation = Mutation(Population, MutationProbability)
% 子程序9: 用于群体中少量个体变量并产生新的群体
% Output -Population 群体 -MutationProbability 变异概率
Dim = size(Population);
for i = 1:Dim(1)
    Probability = rand(1);
    Site = randperm(Dim(2));
    if Probability < MutationProbability
        if Population(i,Site(1)) == 1
            Population(i,Site(1)) = 0;
        end
        if Population(i,Site(1)) == 0
            Population(i,Site(1)) = 1;
        end
    end
end
NewPopulation = Population;
end

function [NewPopulationIncludeMax, EachGenMaxFitness] = ...
    Elitist(Population, PopulationFitness, MumberLength)
% 子程序10: 用到最佳个体保存方法(“优胜劣汰”思想)
% Input -Population 群体 -PopulationFitness 目标函数值
%         -MumberLength 染色体个数
global Count CurrentBest
[~, MinSite] = min(PopulationFitness);
[MaxFitness, MaxSite] = max(PopulationFitness);
EachGenMaxFitness = MaxFitness;
if Count == 1
    CurrentBest(1:MumberLength) = Population(MaxSite,:);
    CurrentBest(MumberLength+1) = PopulationFitness(MaxSite);
else
    if CurrentBest(MumberLength+1) < PopulationFitness(MaxSite);
        CurrentBest(1:MumberLength) = Population(MaxSite,:);
        CurrentBest(MumberLength+1) = PopulationFitness(MaxSite);
    end
end
Population(MinSite,:) = CurrentBest(1:MumberLength);
NewPopulationIncludeMax = Population;
end

```

下面是终止条件为第二类的算法主程序(所有子程序与上一程序相同, 略)。

```

function [Cnt,Result,BestMember] = Genetic2(MumberLength, MemberNumber, ...
    FunctionFitness, MinX, MaxX, Fmin, MutationProbability, Precision)

```

```

% Input -Precision 计算精度
Population = PopulationInitialize(MumberLength, MemberNumber);
Error = Precision+1;
global Count CurrentBest;
Count = 1;
PopulationCode = Population;
PopulationFitness = Fitness(PopulationCode, FunctionFitness, MinX, MaxX, MumberLength);
PopulationFitnessF = FitnessF(PopulationFitness, Fmin);
PopulationProbability = Probability(PopulationFitnessF);
[Population, EachGenMaxFitness] = Elitist(PopulationCode, PopulationFitness, MumberLength);
EachMaxFitness(Count) = EachGenMaxFitness;
MaxFitness(Count) = CurrentBest(length(CurrentBest));
while Error > Precision
    NewPopulation = Select(Population, PopulationProbability, MemberNumber);
    Population = NewPopulation;
    NewPopulation = Crossing(Population);
    Population = NewPopulation;
    NewPopulation = Mutation(Population, MutationProbability);
    Population = NewPopulation;
    PopulationFitness = Fitness(Population, FunctionFitness, MinX, MaxX, MumberLength);
    PopulationFitnessF = FitnessF(PopulationFitness, Fmin);
    PopulationProbability = Probability(PopulationFitnessF);
    Count = Count + 1;
    [NewPopulation, EachGenMaxFitness] = Elitist(Population, ...
        PopulationFitness, MumberLength);
    EachMaxFitness(Count) = EachGenMaxFitness;
    MaxFitness(Count) = CurrentBest(length(CurrentBest));
    Error = sum(abs(PopulationProbability - mean(PopulationProbability)));
    Population = NewPopulation;
end
Dim = size(Population, 1);
Result = ones(2, Dim);
for i = 1:Dim(1)
    Result(1,i) = Translate(Population(i,:), MinX, MaxX, MumberLength);
end
Result(2,:) = PopulationFitness;
BestMember(1,1) = Translate(CurrentBest(1:MumberLength), MinX, MaxX, MumberLength);
BestMember(2,1) = CurrentBest(MumberLength+1);
Cnt = Count;
figure
subplot(211), plot(EachMaxFitness), ylabel('每一代的最大适应值')
subplot(212), plot(MaxFitness), ylabel('保留最佳个体的最大适应值')
xlabel('第n代群体')
end

```

19.3 实验报告

1. 设 $f(x) = -x^2 + 2x + 0.5$, 用第二终止条件的遗传算法求 $\max f(x)$, $x \in [-1, 2]$.

```

>> [Cnt, Result, BestMember] = Genetic2(22, 8, '-x*x+2*x+0.5', -1, 2, -2, 0.01, 1e-6)
Cnt =
    17
Result =

```



```

0.9614    0.9614    0.9614    0.9614    0.9614    0.9614    0.9614    0.9614
1.4985    1.4985    1.4985    1.4985    1.4985    1.4985    1.4985    1.4985
BestMember =
0.9614
1.4985

```

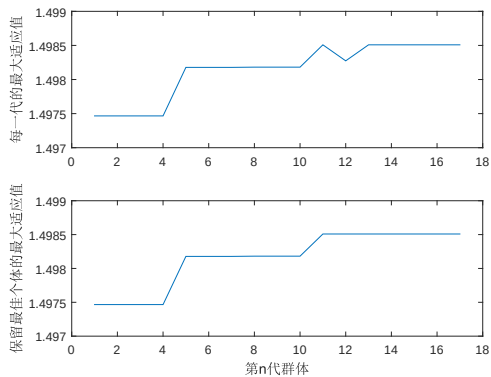


图 19.4: 第1题适应值变化图

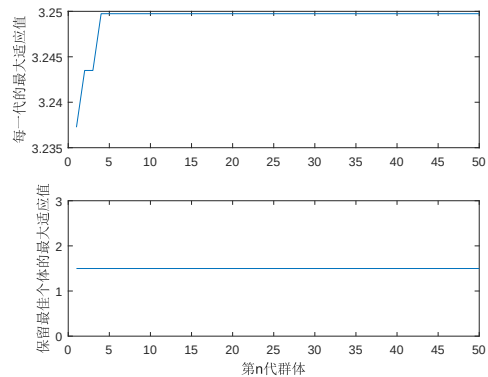


图 19.5: 第2题适应值变化图

2. 设 $f(x) = 3 + x - x^2$, 取编码长度为5, 用第一终止条件的遗传算法求 $\max f(x)$, $x \in [0, 2]$ 。

```

>> [Cnt, Result, BestMember] = Genetic(5, 6, '-x*x+x+3', 0, 2, 1, 0.01, 50)
Cnt =
50
Result =
0.5161    0.5161    0.5161    0.5161    0.5161    0.5161
3.2497    3.2497    3.2497    3.2497    3.2497    3.2497
BestMember =
0.5161
3.2497

```

3. 子程序Crossing.m中的第4行至第8行的程序表明, 当 $\text{Dim}(1) \geq 3$ 时, 将交换数组Population 的最后两行, 即交换最后面的两个个体。其目的是什么?

解 用于群体基因的交叉, 产生新染色体, 并产生新个体。

4. 设 $f(x) = -x^2 - 4x + 1$, 求解精度为15位小数, 用遗传算法求 $\max f(x)$, $x \in [-2, 2]$ 。

```

>> [Cnt, Result, BestMember] = Genetic2(50, 6, '-x*x-4*x+1', -2, 2, -3, 0.01, 1e-15)
Cnt =
7
Result =
-1.7041    -1.7041    -1.7041    -1.7041    -1.7041    -1.7041
4.9125     4.9125     4.9125     4.9125     4.9125     4.9125

```

```
BestMember =  
-1.7041  
4.9125
```

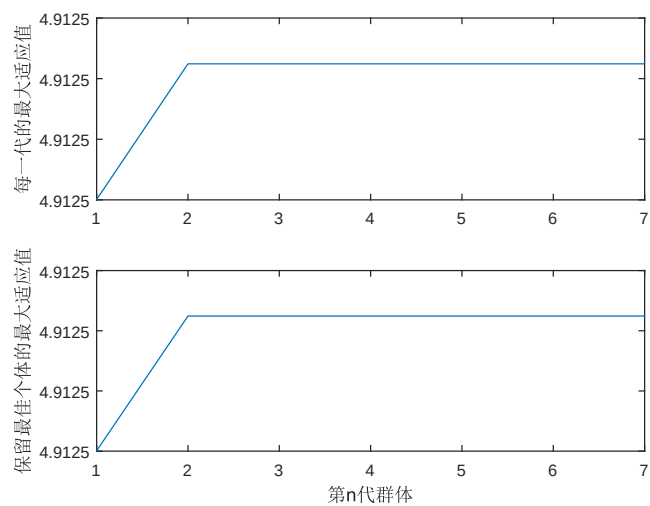


图 19.6: 第4题适应值变化图

第二十章 分形实验

20.1 分形初步

树木花草、山川河流、烟雾云彩等是不规则的。晶体的生长，分子的运动轨迹等也是不规则的。如何用几何来描述它？B. Mandelbrot观察到英国海岸线与Van Koch 曲线的关系，提出了一门描述大自然的几何形态的学科——分形(Fractal)。

分形的应用领域十分广泛，如数学中的动力系统、物理中的布朗运动、流体力学中的湍流、化学中酶的构造、生物中细胞的生长、地质构造、天文学中的行星的光环等，都会用到分形。

分形具有以下几种特性：

1. 具有无限精细的结构；
2. 局部与整体的相似性；
3. 具有非拓扑维数，并且它大于对应的拓扑维数；
4. 具有随机性；
5. 在大多数情况下，分形可以用非常简单的方法确定，可能由迭代产生。

分形可由图形迭代生成。给定初始图形 F_0 ，依照某一规则 R 对图形反复作用，

$$F_{k+1} = RF_k, \quad k = 0, 1, 2, \dots$$

得到图形序列 $\{F_1, F_2, \dots\}$ ，其极限图形是分形，作用规则 R 称为生成元。

例 20.1 分形的生成元



图 20.1: Cantor集的生成元



图 20.2: Van Koch雪花曲线的生成元

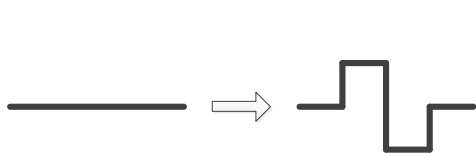


图 20.3: Minkowski香肠图的生成元

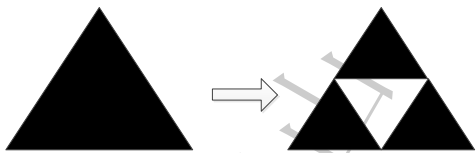


图 20.4: Sierpinski地毯图的生成元

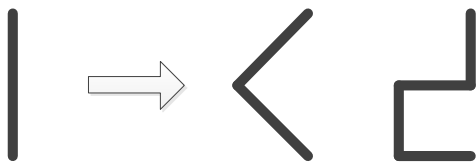


图 20.5: 龙曲线的生成元

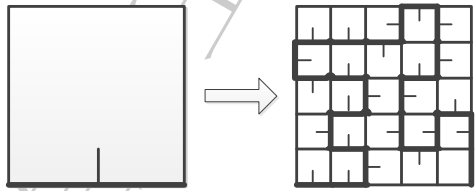


图 20.6: Hilbert曲线及其生成元

例 20.2 生成花草树木的L系统

生物学家Lindenmayer提出了L系统。一个L系统可表示为一个有序的元素集合： $G = \langle V, w, P \rangle$ ，其中 V 表示一些运动过程的集合， w 是初始形状， P 是生成式。

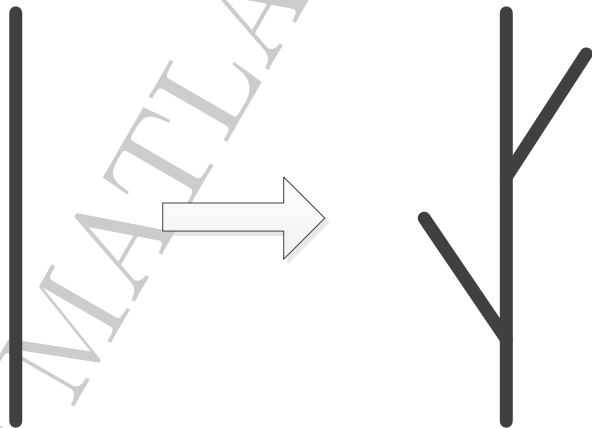


图 20.7: 树的生成元

分形除了可由图形迭代生成外，还可以由函数迭代产生。这里用 z 表示复数，定义在复平面上的函数 $f(z)$ 称为复变函数。

任意给定初值 z_0 ，定义复数序列 $z_{n+1} = f(z_n), n = 0, 1, 2, \dots$ 。我们经常考虑的是，对于什么样的初值 z_0 ，复数序列 $\{z_n\}$ 收敛或有界。

考虑复变函数迭代 $z_{n+1} = z_n^2 + c, n = 0, 1, 2, \dots$, 固定参数 c , 使得迭代序列 $\{z_n\}$ 有界的初值 z_0 在复平面上的分布图形称为Julia集, 记作 $J_c = \{z_0 | \text{迭代序列}\{z_n\} \text{有界}\}$ 。若固定初值 z_0 使得序列有界的参数 c 在复平面上的分布图形称为Mandelbrot集, 记作 $J_{z_0} = \{c | \text{迭代序列}\{z_n\} \text{有界}\}$ 。

记 $z = x + iy, c = p + iq$, 则 $z_{n+1} = z_n^2 + c$ 将变成

$$\begin{cases} x_{n+1} = x_n^2 - y_n^2 + p, \\ y_{n+1} = 2x_n y_n + q, \end{cases}$$

我们将该式称为Julia集的迭代公式。

算法 20.1 Julia集的绘制。

Algorithm 19: JULIA

Input: 初值 p, q , 图形的大小 a, b , 最大迭代次数 N , 使用的颜色数 K

Output: Julia集图像

- 1 设定区域的界值 $M \geq \max(2, \sqrt{p^2 + q^2})$
 - 2 将区域 $R = [-M, M] \times [-M, M]$ 分成 $a \times b$ 个网格, 分别以每个网格点为初值 (x_0, y_0) 利用迭代公式进行迭代
 - 3 如果对所有的 $n \leq N$ 都有 $x_n^2 + y_n^2 \leq M^2$, 则将像素 (i, j) 置为黑色。如果从某一步 n 开始, $x_n^2 + y_n^2 > M^2$, 将像素 (i, j) 置为颜色 $\text{mod}(n, K)$
-

程序如下:

```
function julia(p, q, a, b, N, K)
% 绘制Julia集
% Input -[p,q] 迭代初值 -[a,b] 图形大小, 默认[100,100]
%      -N 最大迭代次数, 默认100 -K 颜色数, 默认6
if nargin < 6, K = 6; end
if nargin < 5, N = 100; end
if nargin < 4, b = 100; end
if nargin < 3, a = 100; end
M = max(2, sqrt(p ^ 2 + q ^ 2));
if K <= 6
    color = ['b'; 'g'; 'r'; 'c'; 'm'; 'y'];
else
    color = rand(K, 3); % 生成颜色矩阵
end
X = linspace(-M, M, a); Y = linspace(-M, M, b);
for i = 1:a - 1
    for j = 1:b - 1
        C = 'k';
        tpoint = [X(i) + X(i+1), Y(j) + Y(j+1)] / 2;
        for k = 1:N
            tpoint1 = tpoint(1) ^ 2 - tpoint(2) ^ 2 + p;
            tpoint2 = 2 * tpoint(1) * tpoint(2) + q;
            if tpoint1 ^ 2 + tpoint2 ^ 2 > M ^ 2
                C = color(mod(k, K) + 1,:);
            end
        end
    end
end
```

```

        break;
    else
        tpoint = [tpoint1, tpoint2];
    end
end
end
xp = X([i, i + 1, i + 1, i]); yp = Y([j, j, j + 1, j + 1]);
patch(xp, yp, C);
end
end

```

例 20.3 以初值 $p = 0, q = 1$ 绘制Julia集。

```
julia(0, 1) % 结果如图 20.8
```

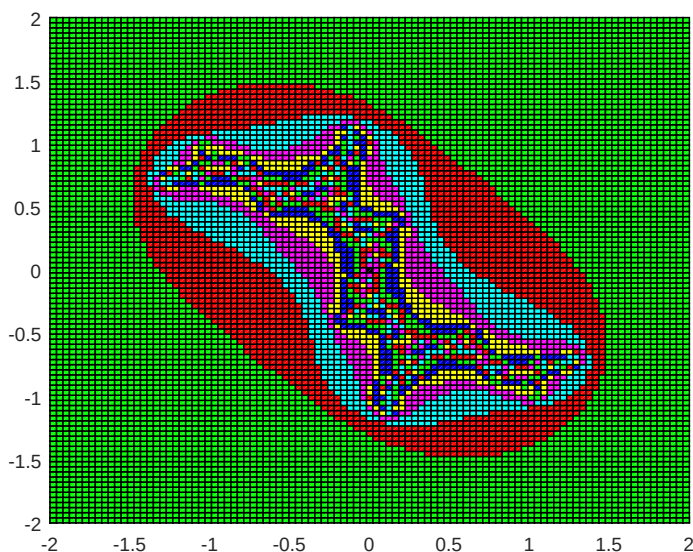


图 20.8: $p = 0, q = 1$ 时的Julia集

还有一种产生分形的方法，即IFS(Iterated Function System)迭代。

取定 n 个仿射变换 $w_i(z) = c_i z + d_i, i = 1, 2, \dots, n$ ，以及对应的 n 个概率 $p_1, p_2, \dots, p_n$ (要求 $\sum p_i = 1$)，任给初值 z_0 ，以概率 p_i 选取变化 w_i 进行迭代 $z_{k+1} = w_i(z_k), k = 0, 1, \dots$ ，则点集 $\{z_k\}$ 的导集(聚点的集合)称为一个IFS吸引子。

算法 20.2 用IFS绘制分形的方法

程序如下：

Algorithm 20: IFS

Input: 初值 z_0 , 图形区域 $V = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$, 总迭代次数 N , 颜色级 L

Output: IFS分形图像

- 1 将 V 分成 $a \times b$ 个网格, 格点为 (x_i, y_j) , 用 $V_{ij} = [x_i, x_{i+1}] \times [y_j, y_{j+1}]$ 表示矩形区域
- 2 计算在 N 次迭代中落入 V_{ij} 中点的个数 μ_{ij}
- 3 记 $\mu = \max\{\mu_{ij}\}$, 计算像素 (i, j) 的灰度 $G(i, j) = \mu_{ij}/(\mu \times L)$

```
function ifs(z, a, b, c, d, P, N, L, M)
% IFS迭代产生分形
% Input -z 迭代初值 -[a,b] 网格数
%       -[c,d] 仿射变换系数, n维复数向量, 默认随机生成
%       -P 仿射变换对应的概率向量, 要求和为1, 默认为 1 / n * ones(1,n)
%       -N 最大迭代次数, 默认1000 -L 颜色数, 默认16
%       -M 图像的可视范围[xmin,xmax,ymin,ymax]
% Example
%   c = [0.5 + 0.5*1i; 0.5 + 0.5*1i];
%   d = [1; -1]; P = [0.5, 0.5];
%   ifs(0, 100, 100, c, d, P, 100000, 5, [-3,3,-2,2])
if nargin == 0, z = 0; end
if nargin < 2, a = 100; end
if nargin < 3, b = 50; end
if nargin < 4
    cn = randn(100, 2);
    c = cn(:,1) + cn(:,2) * 1i;
end
if nargin < 5
    dn = randn(size(cn));
    d = dn(:,1) + dn(:,2) * 1i;
end
n = length(d);
if nargin < 6, P = 1 / n * ones(n, 1); end
if abs(sum(P)-1) >= 1e-4, error('sum(P)~=1'); end
if nargin < 7, N = 10000; end
if nargin < 8, L = 16; end
if nargin < 9
    M = max(5, abs(z)); M = [-M, M, -M, M];
end
X = linspace(M(1), M(2), a + 1);
Y = linspace(M(3), M(4), b + 1);
u = ones(a, b); cdfP = zeros(n, 1);
% 计算P的累积概率, 为确定仿射变换做准备
for k = 1:n - 1, cdfP(k+1) = cdfP(k) + P(k); end
tpoint = z;
for k = 1:N
    % 根据经验分布函数与均匀分布的关系, 可用均匀分布随机数, 确定迭代所使用的仿射变换
    pp = rand(1);
    ind = find(cdfP < pp, 1, 'last');
    tpoint = tpoint * c(ind) + d(ind);
    m = find(real(tpoint) >= X(1:a-1), 1, 'last');
    n = find(imag(tpoint) >= Y(1:b-1), 1, 'last');
    u(m,n) = u(m,n) + 1;
end
color = gray(L + 1); % 产生灰度矩阵
```

```

um = max(max(u));
for k = 1:a
    for j = 1:b
        xp = X([k, k + 1, k + 1, k]);
        yp = Y([j, j, j + 1, j + 1]);
        C = color(ceil(u(k,j) / um * L),:);
        patch(xp, yp, C);
    end
end
axis image off

```

例 20.4 给定仿射变换 $w_1(z) = sz + 1$, $w_2(z) = sz - 1$ 以及相应的概率 $P_1 = P_2 = 0.5$, 其中 s, z 均为复数。取 $s = 0.5 + 0.5i$, 绘制相应 IFS 迭代的吸引子的图形。

```

c = [0.5 + 0.5*1i; 0.5 + 0.5*1i];
d = [1; -1]; P = [0.5, 0.5];
ifs(0, 100, 100, c, d, P, 100000, 5, [-3,3,-2,2]) % 结果如图 20.9

```

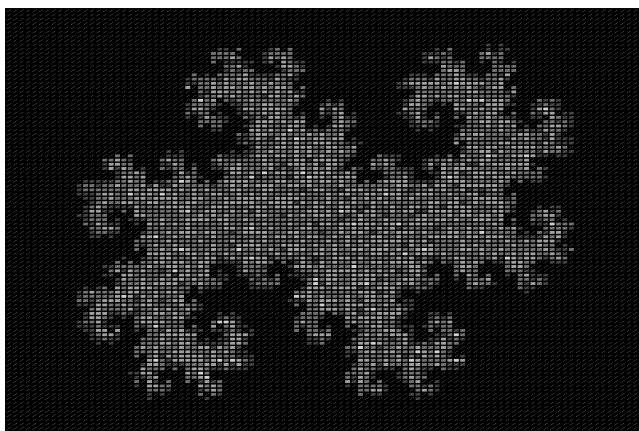


图 20.9: IFS 迭代的吸引子产生的龙曲线

IFS 迭代中有一个游戏比较有名。给定平面上三点 A, B, C , 任意给定初始点 z_0 , 做如下迭代

$$z_{n+1} = \begin{cases} (z_n + A)/2, & \text{当掷出的硬币呈正面,} \\ (z_n + B)/2, & \text{当掷出的硬币呈反面,} \\ (z_n + C)/2, & \text{当掷出的硬币呈侧面,} \end{cases}$$

按上述方式迭代数百次, 将呈现极不规则的图形, 故称为混沌游戏。混沌游戏的图形详见第二节, 混沌的相关内容详见下一章。

20.2 实验报告

1. 用计算机绘制出Koch曲线的图形。你能否自己构造一些生成元，并由此绘出相应的分形图形？任取分形图形的一个局部并将它放大，它同原来的分形图形有什么联系？

解 先编写绘制Koch曲线的koch函数

```
function koch(x, y, n)
% 绘制Koch雪花图
% Input -[x,y] 起点与终点的横纵坐标 -n 迭代次数
if n
    c = y - x;
    c = [-c(2), c(1)];
    c = (x + y) / 2 + c / sqrt(12);
    a = (2 * x + y) ./ 3;
    b = (x + 2 * y) ./ 3;
    koch(x, a, n-1);
    koch(a, c, n-1);
    koch(c, b, n-1);
    koch(b, y, n-1);
else
    plot([x(2), y(2)], -[x(1), y(1)]);
    hold on;
end
```

绘制Koch雪花图

```
n = 4; x = [0, 0]; y = [0, 1];
koch(x, y, n); axis image % 结果如图 20.10
```

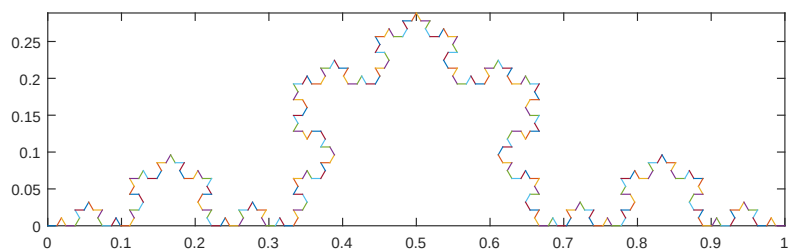


图 20.10: Koch雪花图

程序采用递归思想，程序中if后的语句用于构造生成元，else后的语句用于绘图。读者不妨试着修改if后的语句，构造一些有趣的生成元，绘制相应的分形图像。

下面给出绘制完整Koch雪花图的程序：

```

a = [0,0]; b = [1,0]; c = [0.5, 0.5 * sqrt(3)]; n = 4;
koch(a, c, n); koch(c, b, n); koch(b, a, n);
axis image off          % 结果如图 20.11

```

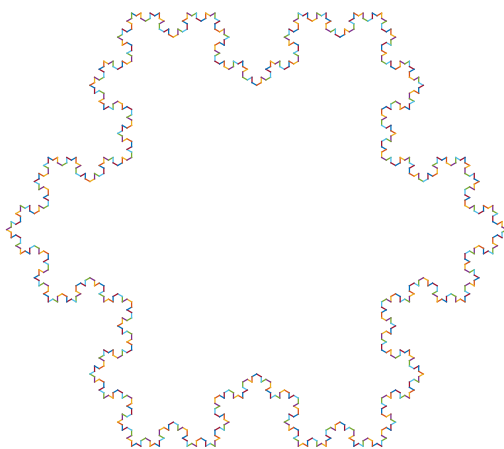


图 20.11: 完整的Koch雪花图

2. 定义Weierstrass函数如下:

$$W(x) = \sum_{k=1}^{\infty} \lambda^{(s-2)k} \sin(\lambda^k x), \quad \lambda > 1, \quad 1 < s < 2,$$

对不同的 s 值, 画出函数的图像, 观察图像的不规则性与 s 的关系, 由此猜测Weierstrass函数图像的维数与 s 的关系。

解 先编写Weierstrass函数

```

function W = weierstrass(x, s, la, n)
% Weierstrass函数
% Input -x 数据点 -s 区间(1,2)上的实数
%       -la lamda值, 默认为2 -n 求和次数
% Output -W W(x)的函数值
if nargin < 2, s = 1.6; end
if nargin < 3, la = 2; end
if nargin < 4, n = 1000; end
W = 0;
for i = 1:n
    W = W + la ^ ((s - 2) * i) * sin(la ^ i .* x);
end

```

对不同的 s 值, 绘制相应的函数图像

```

x = -5:0.2:5; s = 1.2:0.2:1.8;
mk = {'b-^', 'm-s', 'r-p', 'k-h'};
hold on
for i = 1:length(s)
    W = weierstrass(x,s(i));
    plot(x, W, char(mk(i)))
end
legend(strcat('s_=', num2str(s(:))), 'location', 'best', 'Orientation', 'horizontal')
ylim([-3,4]); xlabel('x'); ylabel('W(x)') % 结果如图 20.12

```

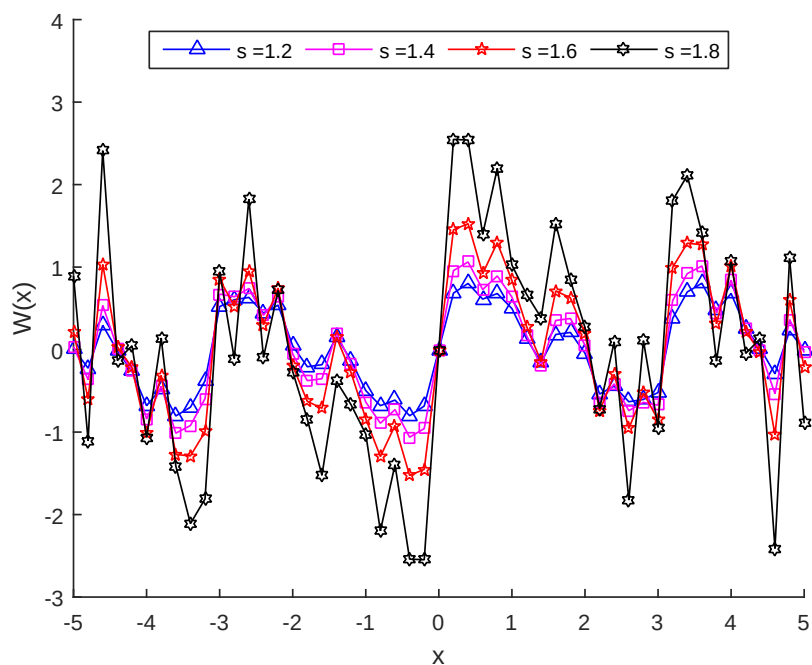


图 20.12: Weierstrass函数图像

故 s 越大, 维数越高。

3. 利用julia函数, 对不同的参数 $(p, q) = (0, 1), (0.1, 0), (0.11, 0.66), (-1.25, 0.01)$ 分别绘制Julia集, 取Julia集的不同局部放大, 观察自相似现象。

```

p = [0, -1, 0.11, -1.25]; q = [1, 0, 0.66, 0.01];
for i = 1:length(p)
    subplot(2, 2, i);
    julia(p(i), q(i))
    title(['p_=', num2str(p(i)), ', q_=', num2str(q(i))])
end
% 结果如图 20.13

```

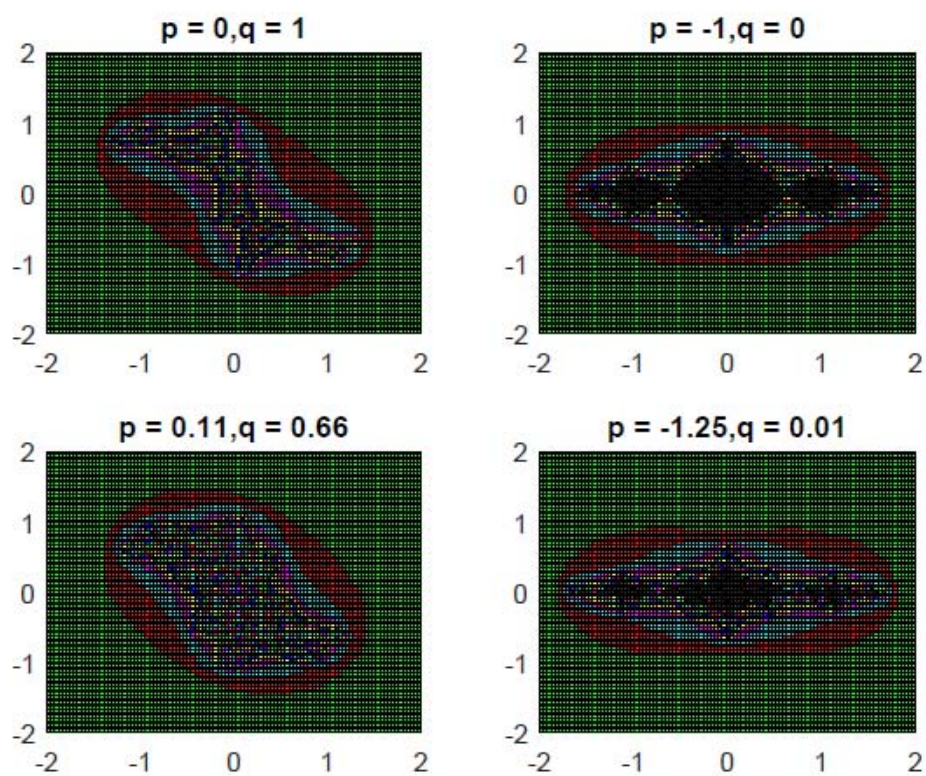


图 20.13: 不同初值的Julia集



图 20.14: Julia集的局部放大图

4. 回到前面的“混沌游戏”。如果把三个概率分别取为 $P_A = P_B = P_C = 1/3$ ，则迭代一万次的图形如何？再取一组其他的概率值绘制相应的图形，结果如何？据此，你认为概率的选取对最终的极限图形(IFS吸引子)有没有影响。

解 编写chaosplay函数

```
function chaosplay(z, p, n, x)
% 混沌游戏
% Input -z 迭代初值 -p 3维概率向量 -n 迭代次数
%       -x 3×2矩阵，第1列为三个点的横坐标，第2列为纵坐标
if nargin < 4, x = [1,0; -1,0; 0,sqrt(3)]; end
if nargin < 3, n = 10000; end
if nargin < 2, p = 1 / 3 * ones(1,3); end
for i = 1:n
    q = rand(1);
    if q < p(1)
        id = 1;
    elseif q < 1 - p(3)
        id = 2;
    else
        id = 3;
    end
    z = (z + x(id,:)) / 2;
    plot(z(1), z(2), 'r');
    hold on
end
```

```
z = [-1, 0];
chaosplay(z) % 结果如图 20.15
```

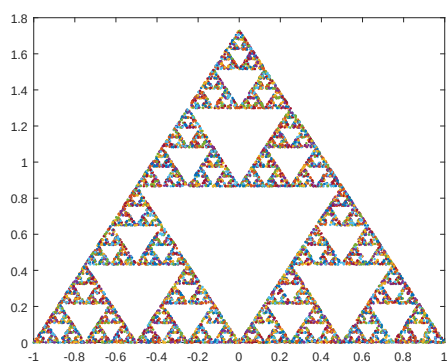


图 20.15: 混沌游戏的图形

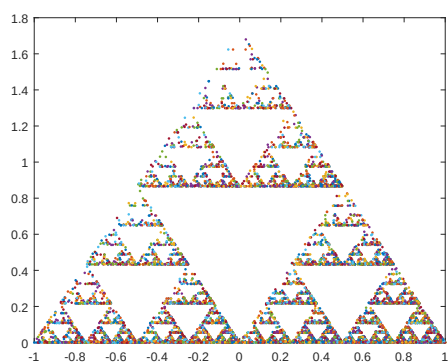


图 20.16: 改变概率后混沌游戏的图形

另取一组概率 $P_A = 1/2$, $P_B = 1/3$, $P_C = 1/6$ 。

```
z = [-1, 0]; p = [1/2, 1/3, 1/6];
chaosplay(z, p)      % 结果如图 20.16
```

通过对比可知，概率的选取对最终的极限图形(IFS吸引子)并没有影响。那么，请读者思考：概率的作用是什么？

5. 对于例 20.4 中的仿射变换，选取不同的 $s = 0.9 + 0.1i$, $0.5 + 0.1i$, $0.5 + 0.3i$, $0.5 + 0.5i$ ，观察图形的变化。

```
s = [0.9 + 0.1*1i, 0.5 + 0.1*1i, 0.5 + 0.3*1i, 0.5 + 0.5*1i];
d = [1; -1]; P = [0.5, 0.5];
for i = 1:4
    subplot(2, 2, i)
    ifs(0, 100, 80, [s(i); s(i)], d, P, 1000000, 5)
    title(['s_□='□', num2str(s(i))])
end
% 结果如图 20.17
```

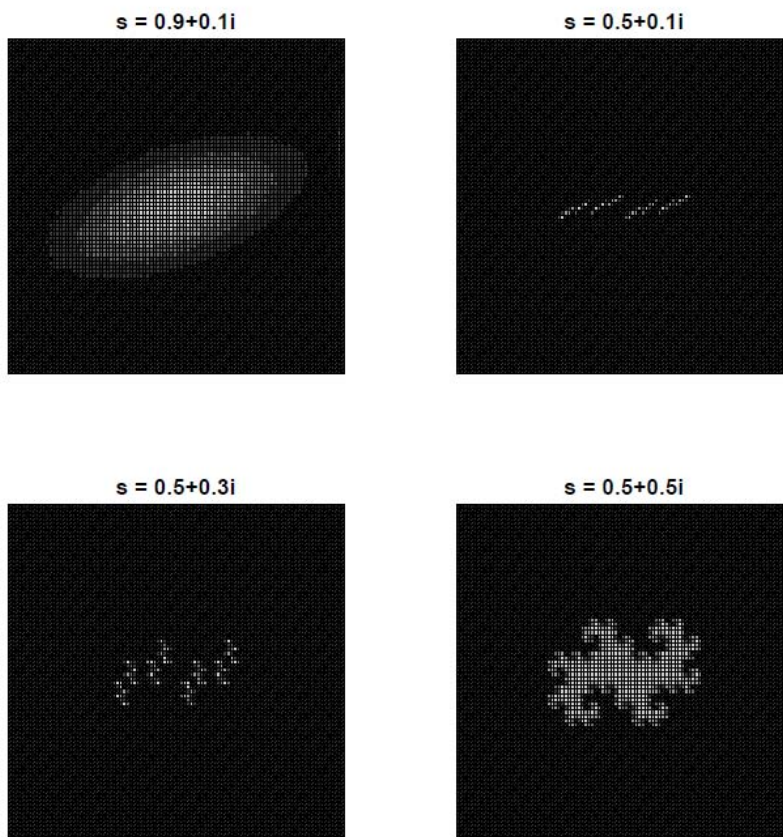


图 20.17: 不同参数的IFS曲线图

6. 考虑两个更一般的变换 $w_1(z) = \sqrt{z - C}$, $w_2(z) = \sqrt{z + C}$ 以及相应的概率 $P_1 = P_2 = 1/2$, 取不同的参数 C (如, $C = 1i, -1, 0.11 + 0.66i$)。观察图形的变化。你能发现什么现象?

```

C = [1i, -1, 0.11 + 0.66 * 1i]; N = 100000;
p = [1/2, 1/2]; L = 5; % 灰度等级
color = gray(L+1); % 灰度矩阵
n1 = 101; n2 = 81; % 网格数
for h = 1:length(C)
    r = rand(1,N); Z = zeros(1,N); Z(1) = 0;
    for k = 2:N
        if r(k) < p(1)
            Z(k) = sqrt(Z(k-1) - C(h));
        else
            Z(k) = sqrt(Z(k-1) + C(h));
        end
    end
    Zr = real(Z); Zi = imag(Z);
    % -----用灰度等级来显示图像-----
    x = linspace(min(Zr) - 0.1, max(Zr) + 0.1, n1);
    y = linspace(min(Zi) - 0.1, max(Zi) + 0.1, n2);
    Cd = zeros(n1-1, n2-1);
    for k = 1:N
        m = find(Zr(k) >= x(1:n1-1), 1, 'last');
        n = find(Zi(k) >= y(1:n2-1), 1, 'last');
        Cd(m,n) = Cd(m,n) + 1;
    end
    um = max(max(Cd));
    figure(h);
    for m = 1:n1-1
        X = [x(m), x(m+1), x(m+1), x(m)];
        for n = 1:n2-1
            Y = [y(n), y(n+1), y(n+1), y(n)];
            Col = color(ceil(Cd(m,n) / um * L) + 1, :);
            patch(X, Y, Col);
        end
    end
    axis image
end % 结果如图 20.18, 20.19, 20.20

```

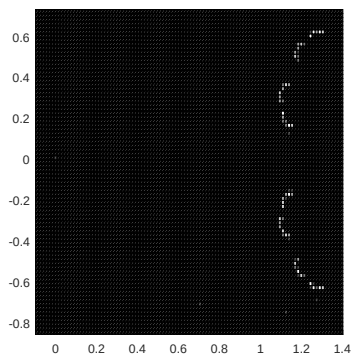


图 20.18: $C = 1i$ 的IFS曲线图

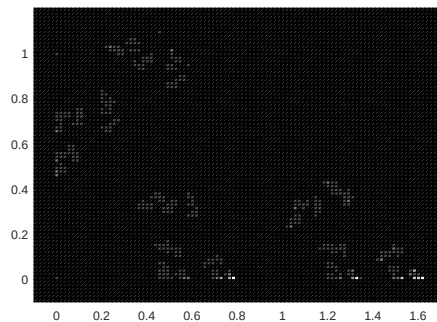


图 20.19: $C = -1$ 的IFS曲线图

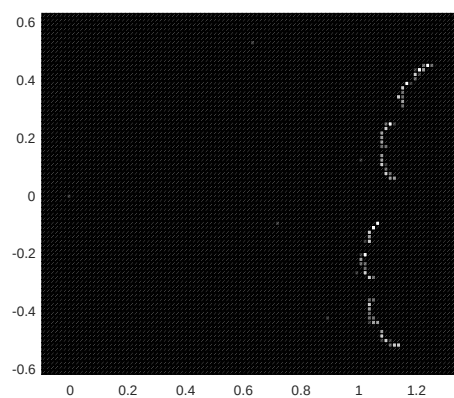


图 20.20: $C = 0.11 + 0.66i$ 的 IFS 曲线图

注 20.1 本题的程序可以写成更一般的形式，针对其他更一般的变换，试将上述程序编写为函数形式。

第二十一章 混沌实验

21.1 混沌初步

线性迭代要么收敛于它的不动点，要么趋于无穷大。不收敛的非线性迭代可能会趋于无穷大，也可能趋于一个周期解，但也可能在一个有限区域内杂乱无章地游荡，这类由确定性运动导致的貌似随机的现象称为混沌现象。

混沌现象一个非常简单的例子是昆虫数量Logistic迭代

$$x_{k+1} = ax_k(1 - x_k), \quad 0 \leq a \leq 4,$$

这里 x_k 表示第 k 代昆虫数量(1表示理想资源环境最大可能昆虫数量)， a 为资源系数， $0 \leq a \leq 4$ 保证了 x_k 在区间 $(0,1)$ 上封闭。Logistic迭代反映了下一代对上一代的既依赖又竞争的关系。当上一代很少时，繁殖能力不够，从而后代很少；当上一代很多，上一代会吃掉很多食物，后代难以存活，从而后代也很少。

若 $\alpha = g(\alpha)$ ，则称 α 为映射 $g(x)$ 的平衡点或不动点。如果对于不动点附近的初始值 x_0 ， $x_{k+1} = g(x_k)$ 收敛于此不动点，则称不动点是稳定的。若 $g^k(\alpha) = \alpha$ (g^k 表示 g 的 k 次复合)，而对于任意的 $j < k$ ， $g^j(\alpha) \neq \alpha$ ，则称 α 为 $g(x)$ 的周期 k 点，并称 $\alpha, g(\alpha), g^2(\alpha), \dots, g^{k-1}(\alpha)$ 为周期 k 轨道。

通常我们用蛛网图来显示Logistic迭代的过程。蛛网图就是将Logistic 迭代转化为平面迭代

$$\begin{cases} y_k = ax_k(1 - x_k) \\ x_{k+1} = y_k \end{cases}$$

此时，蛛网图恰好可以显示迭代计算 $x_0, y_0, x_1, x_2, \dots$ 的一系列变化过程。下面是绘制蛛网图的程序：

```
function logit(a, x0, len, m, n)
% 绘制Logistic迭代的蛛网图
% Input -a 资源系数, 0≤a≤4 -x0 初始值 -len [0,1]区间上的取点数
%       -[m,n] 绘图范围为第m步到第n步的迭代过程
if nargin < 3, len = 100;      end
if nargin < 4, m = 1;          end
if nargin < 5, n = m + 100; end
x = linspace(0, 1, len);
y = a * x .* (1 - x);
plot(x, x, 'r', x, y, 'r')
hold on
```

```

clear x y;
x(1) = x0;
y(1) = a * x(1) * (1 - x(1));
x(2) = y(1);
plot([x(1), x(1), x(2)], [0, y(1), y(1)], 'b')
for i = 2:n
    y(i) = a * x(i) * (1 - x(i));
    x(i + 1) = y(i);
    if i > m
        plot([x(i), x(i), x(i+1)], [y(i-1), y(i), y(i)], 'b')
    end
end
hold off

```

例 21.1 绘制蛛网图

```

x0 = 0.1;
a = [2.5, 3.4, 3.5, 4];
for i = 1:4
    subplot(2, 2, i)
    logit(a(i), x0, 100, 1, 200)
    title(['a_□', num2str(a(i))])
end
% 结果如图 21.1

```

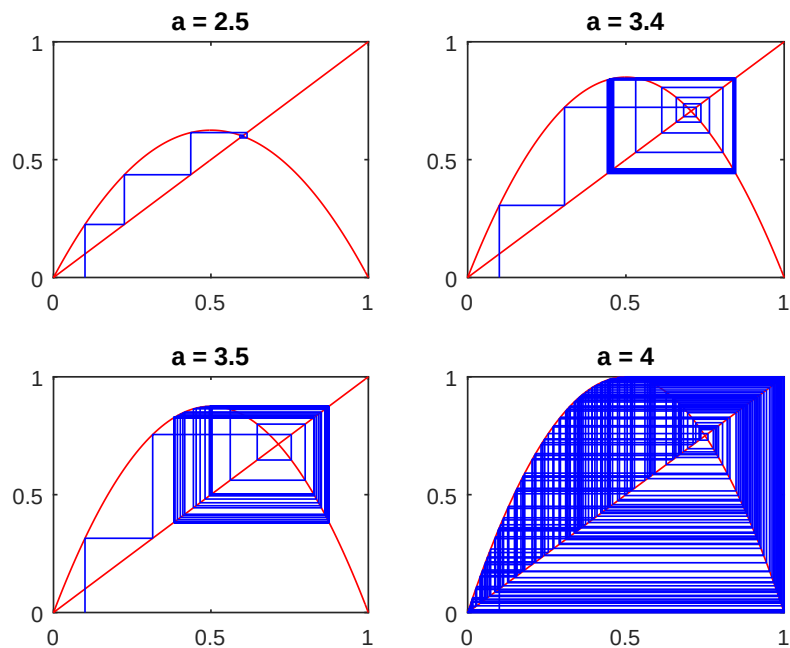


图 21.1: 不同参数的蛛网图

由 $x = ax(1 - x)$ 可以得到两个不动点 0 和 $(1 - a)/a$ 。

由 $x = f(f(x)) = a^2x(x - 1)(1 - ax(1 - x))$ 可得, 周期点为

$$\begin{cases} x_1 = 0, & x_2 = (a - 1)/a, \\ x_{3,4} = \frac{1+a \pm \sqrt{a^2 - 2a - 3}}{2a}, & a \geq 3. \end{cases}$$

当 $0 \leq a < 1$ 时, 在 $[0, 1]$ 内只有一个不动点 0, 而且由 $|g'(0)| = a < 1$ (满足 $|g'(x)| < 1$ 这一条件的不动点称为吸引不动点, 反之称为排斥不动点), 可知它是稳定的。这说明资源匮乏时, 昆虫趋于消亡。当 $a > 1$ 时, 不动点 0 不再稳定。由 $|g'(1 - 1/a)| = |2 - a| < 1$ 可知 $1 < a \leq 3$ 时不动点 $1 - 1/a$ 稳定。这说明资源适当时, 昆虫数量趋于稳定值; 当 $3 < a \leq 1 + \sqrt{6}$ 时, 迭代出现周期 2 解, 迭代开始出现倍周期分岔, 从周期 2, 周期 4, $\dots$, 周期 2^n , $\dots$, 直到周期 $a_\infty = 3.569945672 \dots$, 说明 a 在 $[3, a_\infty]$ 取值时, 昆虫数量呈现规律性振荡; 当 $a_\infty \leq a \leq 4$ 时, 迭代序列几乎杂乱无章, 产生混沌现象。

混沌是由确定性系统产生的貌似随机的现象。一般认为, 混沌具有以下主要性质:

- (1) 初值敏感性: 两个任意近的点出发的两条轨线迟早会分得很开;
- (2) 遍历性: 任意点出发的轨迹总会进入 $[0, 1]$ 内任意小的开区间内, 如图 21.1, $a = 4$ 时的情况。

为了更形象地研究迭代性质与 a 的关系, 我们通常将 a 的取值区间 $[0, 4]$ 以某个步长 Δa (如 $\Delta a = 0.04$) 离散化, 对每个离散点 a 值做迭代, 忽略前 50 个迭代值, 而把点 $(a, x_{51}), (a, x_{52}), \dots, (a, x_{100})$ 显示在坐标平面上, 最后就形成了 Feigenbaum 图。下面给出绘制 Feigenbaum 图的程序。

```
function feigenbaum(h, n, x0, a0)
% 绘制Feigenbaum图
% Input -n 迭代次数, 要求n>50 -x0 初始值 -a0 a的起始值, 要求小于4
if nargin < 4, a0 = 2.5; end
if nargin < 3, x0 = 0.5; end
if nargin < 2, n = 200; end
if nargin == 0, h = 0.001; end
x = zeros(1, n);
hold on
for a = a0:h:4
    x(1) = x0;
    for k = 1:n-1
        x(k+1) = a * x(k) * (1 - x(k));
    end
    plot(a * ones(n - 50, 1), x(51:end), 'b.', 'MarkerSize', 2)
end
axis([a0, 4, 0, 1]); xlabel('a值'); ylabel('x值');
hold off
```

例 21.2 绘制 Feigenbaum 图, 并回答下列问题:

- (1) 它的左部有一条曲线, 表示什么意义?
- (2) 从某一点 a_1 开始, 这条曲线分成两支, 这说明了迭代的什么性质? 迭代的点是如何运动的?
- (3) 再在下一个分支点 a_2 , 曲线分成几支? 这说明迭代的什么性质?
- (4) 上述分支过程是否一直进行下去? 是否存在极限分支点 a_∞ ?

(5) 在极限分支点之后, Feigenbaum图是否显得很混乱?

(6) 在Feigenbaum图的右部, 有一个有三条线穿过的空白地带, 它是一个周期为3的窗口。你能找到其它窗口吗? 它们的周期是什么? 窗口里有什么图案? 这些窗口与周期轨道有什么关系?

解 绘制Feigenbaum图

```
feigenbaum % 结果如图 21.2
```

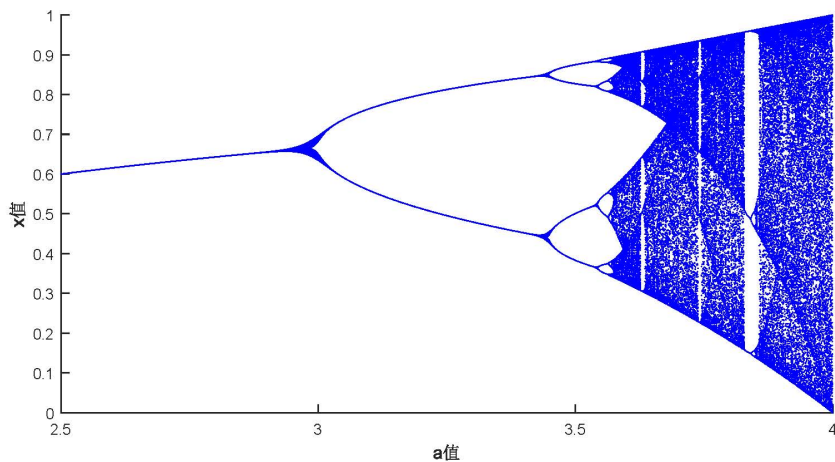


图 21.2: Feigenbaum图

(1) 左边有一条曲线, 说明这个点是收敛点。 $k = 0$ 到 1 时稳定到一个解 0; $k = 1$ 到 3 时, 稳定到一个非零解。这些点是对应 a 值的不动点, 并且是吸引不动点。

(2) 这条曲线分成两支, 这说明迭代对初值敏感, 同时这说明迭代可能具有周期点, 在迭代一定次数以后, 迭代点以周期 T 变化, 经过 T 次迭代回到本身。 $k = 3$ 到 3.45 时, 稳定到 2 个解 (2 周期解)。

(3) 四支。这说明迭代有周期点的性质, 不动点就是周期点。

(4) 会一直进行下去, 该极限分支是存在的。从图中可以看出之后又出现分支随着 k 的继续增加还会稳定到 8 个周期解, 16 个周期解, 直到 a_{∞} , 就成了 2^{∞} 个周期解, 这就是极限分支点 a_{∞} 。若 k 再增加就出现混沌。

(5) 在 $a = 4$ 后, 出现了混沌现象。图像分布不混乱, 所以在极限分支点之后图像不会显得很混乱。

(6) 将图 21.2 放大如图 21.3, 在图像中可以看到其他的窗口。如对应 $a = 3.63$ 处也有个空白地带, 其周期为 6。窗口对应的周期就是窗口内线穿过的条数。窗口中有暗线, 暗线就是 $x = 0.5$ 在各种 a 值下的 $f(x)$ 的迭代曲线。若暗线 P_{n-1} 过 $(a, 0.5)$ 点, 而 $i = 0, 1, 2, \dots, n-2$ 都不过 $(a, 0.5)$ 点, 则在 $k = a$ 处存在一个 n 周期窗口。

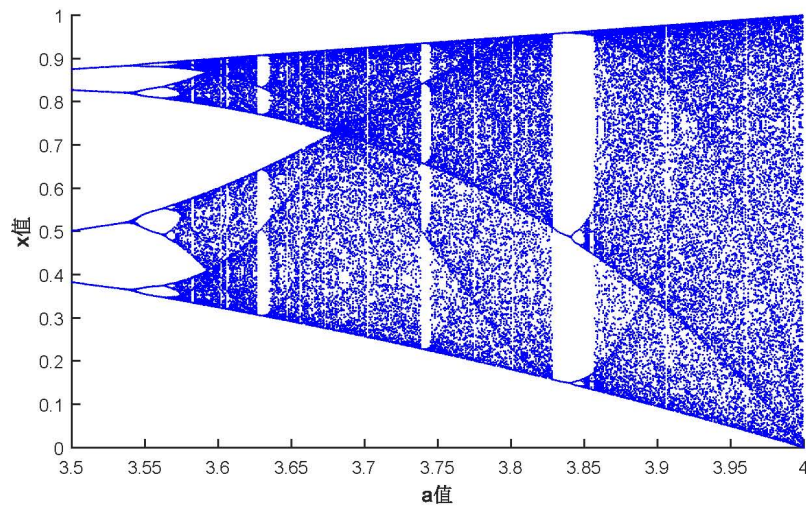


图 21.3: Feigenbaum图

以上只是Logistic迭代的相关内容，还有很多函数的迭代可以产生混沌现象，例如，锯齿函数 $S(x)$ 和帐篷函数 $T(x)$

$$S(x) = \begin{cases} 2x, & 0 \leq x \leq 1/2, \\ 2x - 1, & 1/2 < x \leq 1, \end{cases} \quad T(x) = \begin{cases} 2x, & 0 \leq x \leq 1/2, \\ 2(1 - x), & 1/2 < x \leq 1. \end{cases}$$

21.2 实验报告

设 $f(x) = ax(1 - x)$ 为二次迭代函数，则迭代格式为 $x_{k+1} = ax_k(1 - x_k)$ 。

1. 对几组不同的参数值 a (如 $a = 0.5, 1.4$)以及不同的初值 x_0 ，观察迭代是否收敛。

解 编写函数

```
function [id, y] = chaos1(a, x0, n)
% 判断迭代是否收敛
% Input -a 资源系数 -x0 迭代初值 -n 迭代次数，默认为100
% Output -id 迭代收敛的逻辑值，1-迭代收敛，0-不收敛
%        -y 迭代收敛值，若迭代发散，则y = NaN
if nargin < 3, n = 500; end
if nargin < 2, x0 = 0.5; end
x = zeros(n + 1, 1); x(1) = x0; k = 1;
% 收敛条件是|x(k+1)-x(k)|较小或者迭代次数太大
while abs(x(k+1)-x(k)) > 1e-6 && k < n
    x(k+1) = a * x(k) * (1 - x(k)); k = k + 1;
end
plot((1:k), x(1:k), 'b-', 'LineWidth', 1.5); grid on
if abs(x(k)) < 1e-5, x(k) = 0; end % 若收敛点的值很小，则认为其值为0
xlabel('迭代次数'); ylabel('迭代值');
```

```

if abs(x(k) - x(k-1)) < 1e-5
    title(['$$x(k) \rightarrow', num2str(x(k)), '(k \rightarrow \infty)$$'], ...
        'interpreter', 'latex');
    id = 1; y = x(k);
else
    title('迭代发散');
    id = 0; y = NaN;
end

```

选取参数值 a 以及不同的初值 x_0 进行判断

```

a = [0.5, 0.5, 1.4, 1.4];
x0 = [0.5, 1.2, 0.5, 1.2];
for i = 1:4
    subplot(2, 2, i); id(i) = chaos1(a(i), x0(i));
end
% 结果如图 21.4
id

```

输出结果

```

id =
     1     1     1     0

```

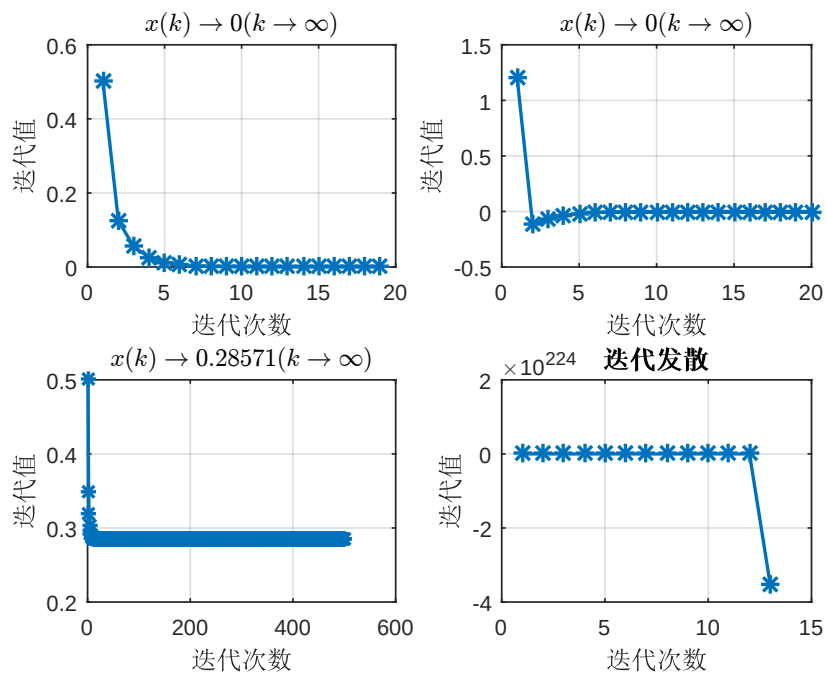


图 21.4: 不同参数下的收敛图

从id和图 21.4 可知, 前三者是收敛的, 最后一个是发散的。

2. 取参数 $a = 0.8$, 用不同的初值作迭代。你能找到一个吸引的不动点和一个排斥的不动点吗? 取不同的参数 $a = 1, 1.6, 2, 2.5$, 回答同样的问题。

解 找不动点

```
syms a x;
f = x - a * x * (1 - x);
s = abs(diff(x - f));
xs = solve(f)
as = [0.8, 1, 1.6, 2, 2.5];
for i = 1:length(as)
    c = subs(s, 'a', as(i));
    cs = subs(xs, 'a', as(i));
    id(1,i) = cs(1);           % 不动点1
    id(2,i) = (subs(c, 'x', cs(1)) < 1); % 判断不动点1是否为吸引型
    id(3,i) = cs(2);           % 不动点2
    id(4,i) = (subs(c, 'x', cs(2)) < 1); % 判断不动点2是否为吸引型
end
id
```

输出结果

```
xs =
    0
(a - 1)/a
id =
    0    0    0    0    0
    1.0000    0    0    0    0
   -0.2500    0    0.3750    0.5000    0.6000
    0    0    1.0000    1.0000    1.0000
```

故 $a = 0.8$ 时, -0.25 为排斥不动点, 0 为吸引不动点; $a = 1$ 时, 不动点 0 既不吸引也不排斥; $a = 1.6, 2, 2.5$ 时, 排斥不动点均为 0 , 吸引不动点分别为 $0.375, 0.5, 0.6$ 。

3. 找出一个参数 a , 使它对应的迭代具有2周期点。这种性质依赖于初值吗?

解 先确定 a 的范围

```
>> syms a x
>> f = x - a ^ 2 * x * (1 - x) * (1 - a * x * (1 - x));
>> solve(f)
ans =
    0
(a - 1)/a
(a + ((a + 1)*(a - 3))^(1/2) + 1)/(2*a)
(a - ((a + 1)*(a - 3))^(1/2) + 1)/(2*a)
```

从运行的结果看迭代具有2周期只与 a 有关。其中 $x_{3,4} = (1 + a \pm \sqrt{a^2 - 2a - 3})/(2a)$ 是2周期点。当 $a < 3$ 时, 结果出现复数, $a = 3$ 时, $x_2 = x_3 = x_4$, 不符合条件, 故当 $a > 3$ 时可以取到合适的点。

观察图 21.2, 使迭代具有周期2解的 a 是连续的, 故编写如下函数:

```
function b = fina2t(x0, n)
% 寻找混沌中2周期点的范围
% Input -x0 迭代初值 -n 最大迭代次数-50, 默认为500
% Output -b 迭代有二周期点是时a的范围, 格式为[min,max]
if nargin < 2, n = 500; end
a = 3:0.01:4; m = 1;
for j = 1:length(a)
    x = zeros(1, n+50); x(1) = x0;
    for i = 2:n
        x(i) = a(j) * x(i-1) * (1 - x(i-1));
    end
    for k = n+1:n+50
        x(k) = a(j) * x(k-1) * (1 - x(k-1));
        if abs(x(k) - x(k-2)) < 1e-5 && abs(x(k-1) - x(k-3)) < 1e-5 && ...
            abs(x(k) - x(k-1)) > eps
            b(m) = a(j); m = m + 1; break;
        end
    end
end
try
    b = [min(b), max(b)];
catch
    b = [NaN, NaN];
end
```

带入不同初值, 求解

```
>> b = fina2t(0.50)
b =
    3.0100    3.4400
>> b = fina2t(0.85)
b =
    3.0100    3.4400
>> b = fina2t(0.15)
b =
    3.0100    3.4400
```

从运行结果看, 当初值改变时, 出现周期2解的范围并没有改变, 我们可以采用绘图的方式来观察其他初值的情况。

```
x = 0.05:0.05:1-0.05; % 取间隔0.05
for i = 1:length(x)
    c = fina2t(x(i));
    b([1,2], i) = c';
end
plot(x, b(1,:), 'b-^', x, b(2,:), 'r-s')
legend('周期解1', '周期解2', 'location', 'South', 'Orientation', 'horizontal')
ylim([2.8, 3.5]); % 结果如图 21.5
```

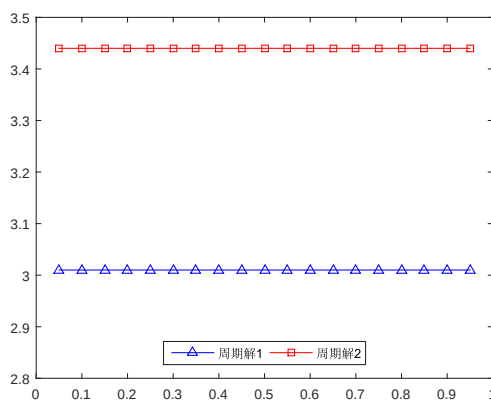



图 21.5: 不同参数下的周期解

从图上看, 随着横轴初值的变化, 纵轴上区间端点保持不变。任取 $[3.01, 3.44]$ 中一点作为 a 值即可。迭代的2周期点不依赖于初始值。

注 21.1 对任意的整数 k , 你能找到一个 a 值使得它对应的迭代具有 k 周期点吗? 对哪些 k 值能给出 k 周期点? 在每种情况下, 结果是否依赖于初值?

4. 根据前面的练习, 试着从理论上分析: 如何求不动点? 对哪些 a 值对应吸引的不动点? 哪些 a 值对应排斥的不动点? 初值对结果有什么影响? 对周期点做类似的分析。

解 设 x 是 $f(x)$ 的不动点, 如果在 x 附近有 $|f'(x)| < 1$, 则 x 是 $f(x)$ 的吸引的不动点; 否则, x 是 $f(x)$ 的排斥的不动点。由于 $f'(0) = a$, $f'((a-1)/a) = 2-a$, 故当 $0 < a < 1$ 时, 0 为吸引点, $(a-1)/a$ 为排斥点。当 $1 < a < 3$ 时, 0 为排斥点, $(a-1)/a$ 为吸引点。

具体方法: 选定参数和初始值进行迭代, 如果后一步迭代与前一步迭代的值相等或两者之间的差充分小, 则该点就是一个不动点。初始值不影响结果。

对于周期点: 选取初始值和参数, 迭代的极限经常会出现周期性, 观察周期点就可以得到不动点。初始值没有影响。

5. 对于迭代格式 $x_{k+1} = ax_k(1-x_k)$, 任取两个初值使它们之间的差的绝对值不超过0.1, 在迭代过程中, 两个迭代序列是否逐渐分开? 如果两个初值的差的绝对值不超过0.01, 0.001, 0.0001结果如何? 由此得出迭代对初值是否敏感?

解 编写函数

```
function chaos(a, x0, n)
% 混沌迭代
% Input -a 资源系数 -x0 初值, 可以是向量 -n 迭代次数
if nargin < 3, n = 50; end
m = length(x0);
```

```

x = zeros(1, n+1);
hold on
mk = {'-o', '-^', '-s', '-p', '-h', '-d', '-x', '-+', '-*', '-v', '-<', '->'};
ind = mod((1:m)-1, 12) + 1;
for i = 1:m
    x(1) = x0(i);
    for k = 1:n
        x(k+1) = a * x(k) * (1 - x(k));
    end
    plot(1:k+1, x, char(mk(ind(i))));
end
xlim([0, n]);
str = strcat('x0_□=□', num2str(x0(:)));
legend(str)
hold off

```

固定 a 值，绘图

```

a = 0.8; x0 = [0.2, 0.25, 0.255, 0.2555, 0.25555];
chaos(a, x0) % 结果如图 21.6

```

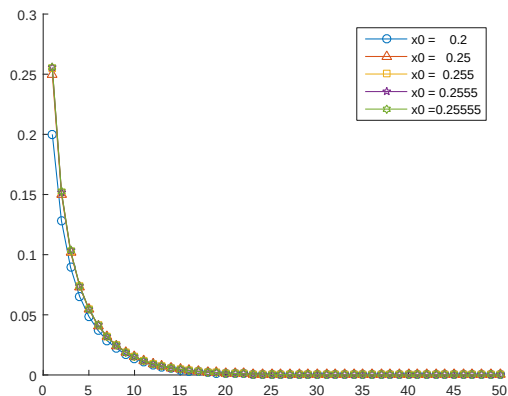


图 21.6: 不同参数下的迭代行为

改变 a 值和初值，多次运行程序，我们就可以得出：

当 $a < 3$ 时，随着迭代次数的增大，迭代序列均收敛到同一个值，没有逐渐分开，说明迭代不随初值的微小改变而改变。当具有稳定周期点时连续依赖于初始值。随着 a 趋于4，混沌现象越发明显，从绘制的图像中可以看到不同颜色的点重合的不多，这说明当混沌现象明显时，对初值敏感。

还有一点，需要考虑，因为当 a 较小时，1周期稳定，但是随着 a 的增大，2周期点比1周期点稳定，然后 a 再增大，4周期点又会更加稳定，这样出现了分岔的现象，所以在分支点上，初始值的微小改变量对迭代很敏感。

6. 从不同的初值 $x_0 \in (0,1)$ 出发, 统计迭代点列中分别落与区间 $(0, 1/2)$ 及 $(1/2, 1)$ 中的点的个数, 你得到的结果是随机的吗? 进一步, 将区间分成任意等份, 统计迭代点列落于每个子区间的点的个数? 结果如何?

解 编写统计函数

```
function s = chaossum(a, x0, n, m)
% 区间分成任意等份, 统计迭代点列落于每个子区间的点的个数
% Input -a 资源系数 -x0 迭代初值, 可以是向量 -n 迭代次数 -m 区间数
% Output -s length(x0)×m的矩阵, 第i行为x0(i)的统计结果
x = zeros(1, n); s = zeros(length(x0), m);
for i = 1:length(x0)
    x(1) = x0(i);
    for k = 1:n-1
        x(k+1) = a * x(k) * (1 - x(k));
    end
    for j = 1:m
        s(i,j) = sum(x >= (j-1) / m & x < j / m); % 统计落在各个区间的点数
    end
end
```

给定参数, 进行统计

```
>> x0 = 0.1:0.1:0.9; n = 5000; a = 3.5;
>> m = 2; s = chaossum(a, x0, n, m)
s =
    1256    3744
    1256    3744
    1252    3748
    1253    3747
    1250    3750
    1252    3748
    1251    3749
    1255    3745
    1255    3745

>> m = 4; s = chaossum(a, x0, n, m)
s =
     1    1255    1245    2499
     1    1255    1245    2499
     0    1252    1249    2499
     0    1253    1247    2500
     0    1250    1250    2500
     0    1252    1248    2500
     0    1251    1250    2499
     0    1255    1245    2500
     0    1255    1245    2500
```

从Feigenbaum图, 我们可以观察到随着 a 的增加, 出现了倍周期分岔现象, 所以若取不同的 a 值, 可以发现概率分布是和周期同步的, 例如, 当 $a = 3.2$ 时存在2周期点, 则有概率分布 $[0.5, 0.5]$ 。做实验发现, 若存在4周期点, 则将区间分成4份, 若存在 k 周期点, 则分成 k 份。当 $a = 3.57$ 时, 系统处于完全混沌状态, 遍历 $[0, 1]$ 区间。当初值不一样时, 概率依赖于参数 a 的取值。

我的 MATLAB 报告

王保山

第五部分

GUI设计篇

我的 MATLAB 报告

王保山

第二十二章 句柄图形对象

用户界面(或接口)是指:人与机器(或程序)之间交互作用的工具和方法。如键盘、鼠标、跟踪球、话筒都可成为与计算机交换信息的接口。图形用户界面(Graphical User Interfaces, GUI)则是由窗口、光标、按键、菜单、文字说明等对象(Objects)构成的一个用户界面。用户通过一定的方法(如鼠标或键盘)选择、激活这些图形对象,使计算机产生某种动作或变化,比如实现计算、绘图等。假如读者所从事的数据分析、解方程、计算结果可视工作比较单一,那么一般不会考虑GUI的制作。但是如果读者想向别人提供应用程序,想进行某种技术、方法的演示,想制作一个供反复使用且操作简单的专用工具,那么图形用户界面也许是最好的选择之一。

MATLAB为表现其基本功能而设计的演示程序demo是使用图形界面的最好范例。MATLAB的用户,在指令窗中运行demo打开图形界面后,只要用鼠标进行选择 and 点击,就可浏览那丰富多彩的内容。

本章只是大致介绍GUI的内容,不太系统,有兴趣的读者可以参考罗华飞老师编写的《MATLAB GUI 设计学习手记》(目前已出3版)。

在MATLAB中,图形用户界面包含了众多的GUI对象,但基本的图形界面对象可以分为3类:用户控件对象(uicontrol)、下拉式菜单对象(uimenu)和内容式菜单对象(uicontextmenu)。还有一些对象可以包含多个图形用户对象,这些对象被称为容器对象。容器对象可以包含多个图形用户对象,也可以包含多个容器对象。利用这些用户对象,进行组合和设计,最后就会设计出功能完善、易操作的GUI界面。

GUI图形对象之间的层次树如图 22.1 所示,其中Root对象是根对象,Figure是图形对象,Axes是轴对象,这些对象将在本章中重点介绍。

22.1 Root对象

图形对象的根对象相当于计算机屏幕,是Figure对象的父类,其句柄值为0。根对象在MATLAB启动时自动创建,在MATLAB退出时自动销毁,因此,用户不需创建也不能手动销毁。

查看根对象的属性可使用语句“get(0)”。根对象的主要属性见表 22.1(按属性名的字母顺序排列,有效属性的默认值用“{ }”标识)。

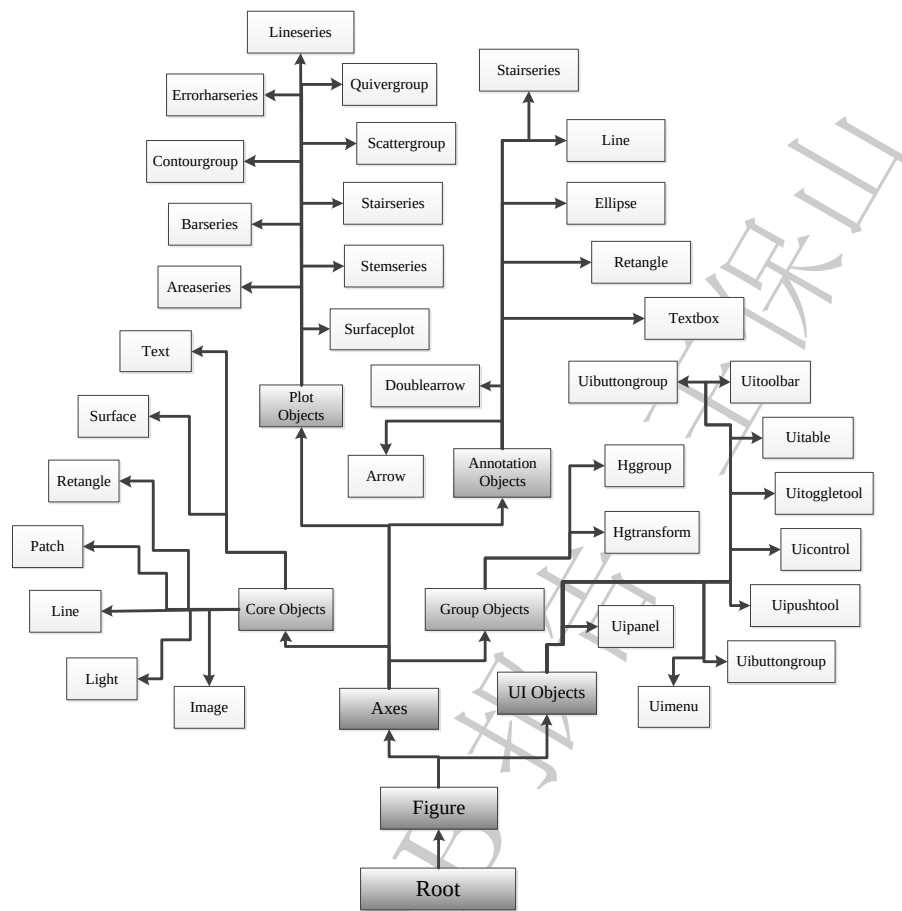


图 22.1: 图形对象之间的层次树

表 22.1: Root对象及描述

属性	描述	有效属性值
CallbackObject	当前正在执行的回调函数的对象句柄	图形对象的句柄
Children	可见的子对象的句柄	句柄向量
CurrentFigure	当前图形窗口的句柄	图形对象的句柄
FixedWidthFontName	GUI对象使用定宽字体时的字体	定宽字体名; {Courier New}
HandleVisibility	设置控件的句柄可访问性	{on}、callback、off
MonitorPositions	显示器的宽高; 格式为[1 1 宽高]	四维向量
Parent	父对象	根对象的父对象恒为空矩阵[]
PointerLocation	鼠标指针的当前位置	二维的位置向量, 左下角为[0,0]
ScreenDepth	屏幕的显示深度	正整数; {32}

续 表

属性	描述	有效属性值
ScreenPixelsPerInch	每像素的位数	正整数; {96}
ScreenSize	屏幕的显示尺寸	四维向量, [左, 底, 宽, 高]
ShowHiddenHandles	显示或隐藏标记为隐藏的句柄	on、{off}
Tag	用户定义的对象标识符	字符串
Type	对象类型	root
Units	计量单位	{pixels}、normalized、inches、points、character、centimeters
UserData	用户定义的数据	任意类型的数据, 默认为空

22.2 Figure对象

Figure对象, 即图形窗口对象, 是MATLAB中显示图形的窗口, 可被看做GUI对象的“容器”, 它是根对象的直接子对象, 其他所有句柄图形对象都直接或间接地继承于图形窗口对象。它可以包括表格、菜单、工具栏、控件对象、右键菜单、坐标轴及其子对象、ActiveX控件等。

创建图形窗口对象可使用“figure”函数, 常用格式如下:

```
figure           % 以默认属性值(见表 22.2)创建一个图形窗口, 并设为当前窗口
figure(h)        % 当h为图形对象句柄值时, 将对应图形窗口设为当前窗口, 并置于屏幕最前端
                 % 当h不是图形对象句柄但为整数时, 创建一个句柄为h的图形窗口, 并设为当前窗口
                 % 当h是一个当前目录或搜索目录下的GUI文件(包含.fig和.m文件)的文件名(不含扩展名)时, 运行GUI文件
h = figure(...) % 返回图形窗口的句柄值h
delete(h)        % 删除句柄值为h的对象, h可以是图形窗口对象, 也可以是其他对象, 例如后续介绍的对象
get(h, PropertyName) % 图形窗口h对应于属性PropertyName的属性值, h可用gcf替换, 表示当前图形窗口
set(h, PropertyName, PropertyValue) % 将图形窗口h对应于属性PropertyName的属性值设为PropertyValue
```

表 22.2: Figure对象及描述

属性	描述	有效属性值
AlphaMap	Alpha色图, 用于设定透明度	m 维向量, 分量在[0, 1]之间
BeingDeleted	调用DeleteFcn时属性值为on; 只读	on、{off}
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	在窗口下按下鼠标时执行的回调函数	字符串或函数句柄
Children	可见的子对象的句柄	句柄向量
Clipping	限幅模式, off为线对象可超出坐标框	{on}、off

续 表

属性	描述	有效属性值
CloseRequestFcn	关闭窗口时执行的回调函数	函数句柄字符串; {'closereq'}
Color	窗口的背景颜色	颜色数据; {[0.94,0.94,0.94]}
Colormap	色图	$m \times 3$ 的RGB颜色矩阵
CreateFcn	创建一个窗口时执行的回调函数	字符串或函数句柄
CurrentAxes	当前坐标轴的句柄	坐标轴句柄
CurrentCharacter	图形窗口中最后键入的字符	单个字符
CurrentObject	当前对象的句柄	图形对象的句柄
CurrentPoint	图形窗口中最后鼠标的位置	坐标 $[x, y]$, 单位取决于Units属性
DeleteFcn	销毁Figure对象时执行的回调函数	字符串或函数句柄
DockControls	图形嵌入控制	{on}、off
FileName	图形界面的文件名	字符串
GraphicsSmoothing	图形平滑	{on}、off
HandleVisibility	设置当前figure对象的句柄是否可见	{on}、callback、off
IntegerHandle	句柄是否为整数; off时句柄为浮点数	{on}、off
Interruptible	回调函数是否可中断	{on}、off
InvertHardcopy	打印时, 改变打印输出为白底黑图	{on}、off
KeyPressFcn	窗口内按下一个键时执行的回调函数	函数句柄、函数句柄和附加参数组成的单元数组、可执行字符串
KeyReleaseFcn	在窗口内释放按键时执行的回调函数	函数句柄、函数句柄和附加参数组成的单元数组、可执行字符串
MenuBar	使用或禁用菜单栏	{'figure'}、none
MinColormap	系统颜色表中能使用的最少颜色	任一标量; {64}
Name	图形窗口的标题	字符串
NextPlot	设定增加下一次绘图的方式	new、{add}、replacechildren、replace
NumberTitle	图形标题中是否显示图形编号	{on}、off
PaperOrientation	打印时的纸张方向	{portrait}、landscape
PaperPosition	打印页面上图形位置和大小	[左, 底, 宽, 高]
PaperPositionMode	打印页面图形位置模式	{manual}、auto
PaperSize	打印纸张大小	[宽, 高], 单位为PaperUnits
PaperType	打印纸张类型	{usletter}、uslegal1、tabloid、a3、a4letter、a5、b4
PaperUnits	打印页面的计量单位	{inches}、pixels、normalized、points、characters、centimeters
OuterPosition	图形窗口整个轮廓的大小和位置	[左, 底, 宽, 高], 单位为Units
Parent	父对象的句柄, 这里为根对象	恒为0

续 表

属性	描述	有效属性值
Pointer	选择鼠标指针形状	crosshair、{arrow}、topl、topr、botl、watch、botr、circle、cross、fleur、left、right、top、bottom、fullcrosshair、ibeam、custom
PointerShapeCData	Pointer为custom时定义的指针	16 × 16的矩阵
PointerShapeHotSpot	指针激活区域	二维向量, [行数, 列数], {[1,1]}
Position	图形窗口的位置与大小	[左, 底, 宽, 高]
RendererMode	着色模式是自动还是手选	{auto}、manual
Resize	窗口是否可以通过鼠标改变尺寸	{on}、off
SelectionType	最近一次鼠标操作的方式	{normal}、extend、alt、open
SizeChangedFcn	窗口尺寸改变时执行的回调函数	字符串或函数句柄
Tag	对象标识符	字符串
Toolbar	指定工具栏是否显示	none、{auto}、figure
Type	图形窗口对象的类型	figure
UIContextMenu	图形窗口的右键菜单	右键菜单句柄
Units	计量单位	{pixels}、inches、normalized、points、characters、centimeters
UserData	用户定义的数据	任一矩阵
Visible	设定图形窗口是否可见	{on}、off
WindowButtonDownFcn	窗口中按下鼠标时执行的回调函数	字符串或函数句柄
WindowButtonMotionFcn	窗口中移动鼠标时执行的回调函数	字符串或函数句柄
WindowButtonUpFcn	窗口中松开鼠标时执行的回调函数	字符串或函数句柄
WindowKeyPressFcn	窗口中按任意键时执行的回调函数	函数句柄、函数句柄和附加参数组成的单元数组、可执行字符串
WindowKeyReleaseFcn	窗口中释放按键时执行的回调函数	函数句柄、函数句柄和附加参数组成的单元数组、可执行字符串
WindowScrollWheelFcn	当前窗口中滚动鼠标滚轮时执行的回调函数	函数句柄、函数句柄和附加参数组成的单元数组、可执行字符串
WindowStyle	窗口为标准、模式或嵌入式窗口	{normal}、model、docked

22.3 Axes对象

Axes对象，即坐标轴对象，创建Axes对象可使用“axes”函数，常用格式如下：

```

axes                % 以默认属性值(见表 22.3)创建一个坐标轴对象, 并设为当前坐标轴对象
axes(ha)            % 当ha为坐标轴对象句柄值时, 将该对象设为坐标轴对象, 并置顶
                    % 当句柄为ha的坐标轴对象不存在时, 创建一个句柄为ha的坐标轴对象, 并置顶
ha = axes(...)      % 返回坐标轴对象的句柄值ha
ha = gca            % 返回当前坐标轴对象的句柄值ha
get(ha, PropertyName) % 坐标轴对象ha对应于属性PropertyName的属性值, ha可用gca替换, 表示当前坐标轴对象
set(ha, PropertyName, PropertyValue) % 将坐标轴对象ha对应于属性PropertyName的属性值设为PropertyValue

```

表 22.3: Axes对象及描述

属性	描述	有效属性值
ActivePositionProperty	坐标轴改变大小时使用的计算方式	{outerposition}、position
ALim	定义Alpha轴的范围	二维向量[amin, amax]
ALimMode	定义Alpha轴范围的模式	{auto}、manual
BeingDeleted	调用DeleteFcn时属性值为on; 只读	on、{off}
Box	指定是否显示坐标轴边框	on、{off}
BoxStyle	坐标轴边框的类型	{back}、full
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	窗口内按下鼠标时执行的回调函数	字符串或函数句柄
CameraPosition	相机位置	[左, 底, 宽, 高]
CameraPositionMode	相机位置模式	{auto}、manual
CameraTarget	相机的目标	[x,y,z]
CameraTargetMode	相机目标的选择模式	{auto}、manual
CameraUpVector	正位向量	三维向量[x,y,z], {[0,1,0]}
CameraUpVectorMode	正位向量模式	{auto}、manual
CameraViewAngle	视角	[0,180]之间的数值
CameraViewAngleMode	视角取值模式	{auto}、manual
Children	可见的子对象的句柄	句柄向量
CLim	定义色轴范围	二维向量[cmin, cmax]
CLimMode	指定CLim属性的操作方式	{auto}、manual
Clipping	限幅模式	{on}、off
ClippingStyle	限幅模式类型	{3dbox}、rectangle
Color	坐标轴的背景颜色	none(透明), 颜色数据; {[1,1,1]}
ColorOrder	定义多线绘图时线的颜色	$m \times 3$ 的RGB颜色矩阵
ColorOrderIndex	下个要使用的颜色编址	正整数, {1}
CreateFcn	创建Axes对象时执行的回调函数	字符串或函数句柄
CurrentPoint	图形窗口中最后单击鼠标的位置	2×3 的矩阵, 单位为Units
DataAspectRatio	x、y、z方向上数据单位的相对比例	格式为[dx, dy, dz]

续 表

属性	描述	有效属性值
DataAspectRatioMode	数据比例模式	{auto}、manual
DeleteFcn	销毁Axes对象时执行的回调函数	字符串或函数句柄
FontAngle	选择斜体或普通字体	{normal}、italic、oblique
FontName	坐标轴标签的字体名	系统支持的字体, FixedWidth
FontSize	定义坐标轴标签和标题的字体大小	整数, {12}
FontSmoothing	字体平滑	{on}、off
FontUnits	字体尺寸单位	{points}、normalized、pixels、inches、centimeters
FontWeight	选择粗体或正常字体	{normal}、bold、light、demi
GridAlpha	网格线的透明度	[0,1]之间的数值, {0.15}
GridAlphaMode	网格线的透明度选择模式	{auto}、manual
GridColor	网格线颜色	none(透明), 颜色数据; {[0.15,0.15,0.15]}
GridColorMode	网格线颜色设置模式	{auto}、manual
GridLineStyle	网格线类型	{-}、-、-、none
HandleVisibility	当前坐标轴对象的句柄是否可见	{on}、callback、off
HitTest	能否通过鼠标单击选择该对象	{on}、off
Interruptible	回调函数是否可中断	{on}、off
LabelFontSizeMultiplier	坐标轴标签字体大小比例	正数, {1.1}
Layer	轴线在刻度线的上方或下方	{bottom}、top
LineStyleOrder	绘图时线性和标记的顺序	线型(LineSpec), {-}
LineStyleOrderIndex	下次绘图时的线型编址	正整数, {1}
LineWidth	绘图时线宽	正数, {0.5}
MinorGridAlpha	次网格线的透明度	[0,1]之间的数值, {0.25}
MinorGridAlphaMode	次网格线的透明度设置模式	{auto}、manual
MinorGridColor	次网格线颜色	none(透明), 颜色数据; {[0.1,0.1,0.1]}
MinorGridColorMode	次网格线颜色设置模式	{auto}、manual
MinorGridLineStyle	次网格线类型	-、-、{:}、-、none
NextPlot	下一次绘图方式	{replace}、replacechildren、new、add
OuterPosition	坐标轴外边界的位置和大小	[左, 底, 宽, 高], {[0,0,1,1]}
Parent	父对象的句柄	Figure句柄
PickableParts	是否捕捉鼠标点击操作	{visible}、all、none
PlotBoxAspectRatio	轴绘图边框的相对比例	绘图边框的相对坐标, [pz,py,pz]

续 表

属性	描述	有效属性值
PlotBoxAspectRatioMode	轴绘图边框的相对比例设置模式	{auto}、manual
Position	绘图区域的位置与大小	[左, 底, 宽, 高]
Projection	二维屏幕上的投影类型	{orthographic}、perspective
Selected	指定对象能否被选择	{on}、off
SelectionHighlight	当图形窗口选中时是否突出显示	{on}、off
SortMethod	渲染对象的顺序	{depth}、childorder
Tag	Axes对象标识符	字符串
TickDir	指定刻度标记的方向	{in}、out
TickDirMode	刻度标记方向的设定模式	{auto}、manual
TickLabelInterpreter	刻度标签的字符注释	{tex}、latex、none
TickLength	刻度标记的长度	[2DLength, 3DLength]
TightInset	包含文本标签的最小区域	[左, 底, 宽, 高]
Title	定义坐标轴标题	标题文本对象的句柄
TitleFontSizeMultiplier	标题字体大小比例	正数, {1.1}
TitleFontWeight	标题文字的粗细	{bold}、normal
Type	坐标轴对象类型	axes
UIContextMenu	坐标轴对象的右键菜单	右键菜单句柄
Units	计量单位	pixels、inches、{normalized}、points、characters、centimeters
UserData	用户定义的数据	任一矩阵
Visible	设定坐标轴对象是否可见	{on}、off
XAxisLocation	设置X轴刻度标记和标签位置	top、{bottom}
YAxisLocation	设置Y轴刻度标记和标签位置	right、{left}
X(Y,Z)Color	坐标轴轴线的颜色	颜色数据
X(Y,Z)Dir	设置坐标值增加的方向	{normal}、reverse
X(Y,Z)Grid	切换坐标轴上网格线的开关状态	on、{off}
X(Y,Z)Label	设定坐标轴标签	文本对象的句柄
X(Y,Z)Lim	设定坐标轴范围	二维向量, [xmin,xmax]
X(Y,Z)LimMode	坐标轴范围的设定方式	{auto}、manual
X(Y,Z)MinorGrid	使用或禁用坐标轴次要网格线	on、{off}
X(Y,Z)MinorTick	使用或禁用坐标轴次要刻度标记	on、{off}
X(Y,Z)Scale	设定坐标轴坐标的刻度单位	{linear}、log
X(Y,Z)Tick	设定坐标轴刻度标记的位置	数值向量
X(Y,Z)TickLabel	设定坐标轴刻度的标签	字符串

续 表

属性	描述	有效属性值
X(Y,Z)TickMode	坐标轴刻度标记位置的设定模式	{auto}、manual
X(Y,Z)TickLabelMode	坐标轴刻度的标签的设定模式	{auto}、manual
X(Y,Z)TickLabelRotation	坐标轴刻度标记的旋转角度	数值

22.4 核心对象

核心图形对象除了坐标轴外还包括图像(Image)、线条(line)、文本(Text)、矩形(Rectangle)、块(Patch)和曲面(Surface)。每个核心对象都有自己的核心属性，见表 22.4。

表 22.4: 核心对象及描述

核心对象	核心属性	描述
Image对象	CData	定义Image对象的图像数据
Line对象	X(Y,Z)Data	定义Line对象的线条数据
Text对象	String	定义Text对象的文本数据
Light对象	Position, Style	分别定义Light对象的位置和类型
Patch对象	X(Y,Z)Data, Faces	分别定义Patch对象的顶点数据和连接方法
Rectangle对象	Position, Curvature	分别定义Rectangle对象的位置和曲率
Surface对象	X(Y,Z)Data, VertexNormals	分别定义Surface对象的顶点数据和法向向量

22.4.1 Image对象

创建Image对象采用image函数，调用格式为：

```
image(C) % 将矩阵C显示为图像
image(x, y, C) % 将矩阵C显示为图像并设定图像的坐标范围，x, y为二维向量
h = image(PropertyName, PropertyValue, ...) % 指定属性值创建Image对象并返回句柄
```

表 22.5: Image对象及描述

属性	描述	有效属性值
AlphaData	透明度	{1}、标量、与CData同维的数组
AlphaDataMapping	透明度值映射	{none}、scaled、direct
BeingDeleted	调用DeleteFcn时属性值为on；只读	on、{off}

续 表

属性	描述	有效属性值
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	Image对象上按下鼠标时的回调函数	字符串或函数句柄
CData	对象的图案	矩阵或 $m \times n \times 3$ 的数组
CDataMapping	数据到色图的映射	scaled、{direct}
Children	Image对象没有子对象	空矩阵
Clipping	限定Image对象是否在坐标轴范围内	{on}、off
CreateFcn	创建Image对象时的回调函数	字符串或函数句柄
DeleteFcn	销毁Image对象时的回调函数	字符串或函数句柄
HandleVisibility	当前uicontrol对象的句柄是否可见	{on}、callback、off
HitTest	能否通过鼠标单击选择该对象	{on}、off
Interruptible	回调函数是否可中断	{on}、off
Parent	父对象的句柄	Axes、hgggroup、hgtransform
PickableParts	是否捕捉鼠标点击操作	{visible}、all、none
Selected	Image对象是否能被选中	{on}、off
SelectionHighlight	选中Image对象时突出显示	{on}、off
Tag	Image对象的标识符	字符串
Type	Image对象的类型	image
UIContextMenu	Image对象的右键菜单	右键菜单句柄
UserData	用户定义的数据	任一矩阵
Visible	设定Image对象是否可见	{on}、off
X(Y)Data	定义图像沿 x/y 轴的范围	[min,max]

22.4.2 Line对象

创建Line对象采用line函数，调用格式为：

```
line(X, Y), line(X, Y, Z)           % 添加二维或三维Line对象到当前坐标轴
line(X, Y, Z, PropertyName, PropertyValue, ...) % 创建Line对象, X(Y,Z)Data = x(y,z)
h = line(PropertyName, PropertyValue, ...) % 按指定属性值创建Line对象
```

表 22.6: Line对象及描述

属性	描述	有效属性值
AlignVertexCenters	尖的垂直和水平线	on、{off}

续 表

属性	描述	有效属性值
BeingDeleted	调用DeleteFcn时属性值为on; 只读	on、{off}
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	Line对象上按下鼠标时的回调函数	字符串或函数句柄
Children	Line对象没有子对象	空矩阵
Clipping	限定Line对象是否在坐标轴范围内	{on}、off
Color	设置线条颜色	颜色字符串或颜色矩阵
CreateFcn	创建Line对象时的回调函数	字符串或函数句柄
DeleteFcn	销毁Line对象时的回调函数	字符串或函数句柄
DisplayName	插图中标明的注释字符串	字符串
HandleVisibility	当前Line对象的句柄是否可见	{on}、callback、off
HitTest	能否通过鼠标单击选择该对象	{on}、off
Interruptible	回调函数是否可中断	{on}、off
LineStyle	指定线型	{-}、-、-、none
LineWidth	指定线宽, 单位为points	标量
Marker	数据点的标记符号	标记定义符
MarkerEdgeColor	空心标记的颜色或封闭图形标记的边缘颜色	颜色字符串或颜色矩阵、none、{auto}
MarkerFaceColor	封闭图形标记的填充颜色	颜色字符串或颜色矩阵、{none}、auto
MarkerSize	标记的尺寸, 单位为points	正整数
Parent	父对象的句柄	Axes、hggroup、hgtransform句柄
PickableParts	是否捕捉鼠标点击操作	{visible}、all、none
Selected	Line对象是否能被选中	{on}、off
SelectionHighlight	选中Line对象时突出显示	{on}、off
Tag	Line对象的标识符	字符串
Type	Line对象的类型	line
UIContextMenu	Line对象的右键菜单	右键菜单句柄
UserData	用户定义的数据	任一矩阵
Visible	设定Line对象是否可见	{on}、off
X(Y,Z)Data	定义Line对象的坐标数据	同维的坐标向量

22.4.3 Text对象

使用text函数可以创建Text对象, 调用格式为:

```

text(x, y, str)          % 将字符串str添加到当前坐标系(x,y)位置上
text(x, y, z, str)       % 将字符串str添加到三维坐标系(x,y,z)位置上
h = text(PropertyName, PropertyValue, ...) % 创建Text对象并返回句柄值

```

表 22.7: Text对象及描述

属性	描述	有效属性值
BackgroundColor	对象背景颜色	颜色字符串或颜色矩阵、{none}
BeingDeleted	调用DeleteFcn时属性值为on; 只读	on、{off}
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	Text对象上按下鼠标时的回调函数	字符串或函数句柄
Children	Text对象没有子对象	空矩阵
Clipping	限定Text对象是否在坐标轴范围内	on、{off}
Color	设置文本颜色	颜色字符串或颜色矩阵
CreateFcn	创建Text对象时的回调函数	字符串或函数句柄
DeleteFcn	销毁Text对象时的回调函数	字符串或函数句柄
EdgeColor	文本区域矩形边框的颜色	颜色字符串或颜色矩阵
Editing	使用或禁用文本的编辑模式	on、{off}
Extent	显示文本对象的位置和尺寸; 只读	[左, 底, 宽, 高]
FontAngle	选择斜体或普通字体	{normal}、italic、oblique
FontName	字体名	系统支持的字体名称
FontSize	字体大小, 单位由FontUnits决定	整数, {12}
FontUnits	字体尺寸单位	{points}、normalized、pixels、inches、centimeters
FontWeight	选择粗体或正常字体	{normal}、bold、light、demi
HandleVisibility	当前Text对象的句柄是否可见	{on}、callback、off
HitTest	能否通过鼠标单击选择该对象	{on}、off
HorizontalAlignment	指定文本的水平对其方式	{left}、center、right
Interpreter	文本字符串转换方式	{tex}、latex、none
Interruptible	回调函数是否可中断	{on}、off
LineStyle	指定线型	{-}、-、-、none
LineWidth	指定线宽, 单位为points	标量
Margin	文本区域到矩形边框的距离	标量, 单位为pixels
Parent	父对象的句柄	Axes、hggroup、hgtransform句柄
PickableParts	是否捕捉鼠标点击操作	{visible}、all、none
Position	Text对象的位置	二维或三维向量, [x,y,(z)]

续 表

属性	描述	有效属性值
Rotation	文本倾斜角度	标量, {0}
Selected	Text对象是否能被选中	{on}、off
SelectionHighlight	选中Text对象时突出显示	{on}、off
String	文本字符串	字符串
Tag	Text对象的标识符	字符串
Type	Text对象的类型	text
Units	计量单位	pixels、{data}、inches、normalized、points、characters、centimeters
UIContextMenu	Text对象的右键菜单	右键菜单句柄
UserData	用户定义的数据	任一矩阵
VerticalAlignment	文本字符串垂直对其方式	top、cap、{middle}、baseline、bottom
Visible	设定Text对象是否可见	{on}、off

22.4.4 Light对象

创建Light对象的函数是light函数，调用格式为：

```
light(PropertyName, PropertyValue, ...) % 按指定属性值创建Light对象
h = light( ... ) % 创建Light对象并返回句柄值
```

表 22.8: Light对象及描述

属性	描述	有效属性值
BeingDeleted	调用DeleteFcn时属性值为on；只读	on、{off}
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	对Light对象无效	字符串
Children	Light对象没有子对象	空矩阵
Clipping	对Light对象无效	on、{off}
Color	Light对象发出的光线颜色	颜色字符串或颜色矩阵
CreateFcn	创建Light对象时的回调函数	字符串或函数句柄
DeleteFcn	销毁Light对象时的回调函数	字符串或函数句柄
HandleVisibility	当前Light对象的句柄是否可见	{on}、callback、off
HitTest	对Light对象无效	{on}、off
Interruptible	回调函数是否可中断	{on}、off

续 表

属性	描述	有效属性值
Parent	父对象的句柄	Axes对象句柄
PickableParts	是否捕捉鼠标点击操作	{visible}、all、none
Position	Axes中放置Light对象的位置	光源处坐标，[x,y,z]
Selected	对Light对象无效	{on}、off
SelectionHighlight	对Light对象无效	{on}、off
Style	光源为平行光还是发散光	{infinite}、local
Tag	Light对象的标识符	字符串
Type	Light对象的类型	light
UIContextMenu	对Light对象无效	右键菜单句柄
UserData	用户定义的数据	任一矩阵
Visible	设定Light对象是否可见	{on}、off

22.4.5 Patch对象

创建Patch对象的函数是patch函数，调用格式为：

```
patch(X, Y, C)           % 添加一个顶点由X和Y指定、填充颜色由C确定的块到当前坐标轴
patch(X, Y, Z, C)        % 添加一个三维块到当前坐标轴
patch(PropertyName, PropertyValue, ...) % 按指定属性值创建Patch对象
h = patch(...)           % 创建Patch对象并返回句柄值
```

表 22.9: Patch对象及描述

属性	描述	有效属性值
AlignVertexCenters	尖的垂直和水平线	on、{off}
AlphaDataMapping	透明度值映射	none、{scaled}、direct
AmbientStrength	环境光强度	[0,1]之间的标量，{0.3}
Annotation	Patch对象的插图方式	hg.Annotation句柄
BackFaceLighting	表面光照控制	unlit、lit、{reverselit}
BeingDeleted	调用DeleteFcn时属性值为on；只读	on、{off}
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	Patch对象上按下鼠标时的回调函数	字符串或函数句柄
CData	定义块的颜色	标量、向量或矩阵
CDataMapping	CData数据到色图的映射	{scaled}、direct
Children	Patch对象没有子对象	空矩阵

续 表

属性	描述	有效属性值
Clipping	Patch对象是否限定在坐标轴内	{on}、off
CreateFcn	创建Patch对象时的回调函数	字符串或函数句柄
DeleteFcn	销毁Patch对象时的回调函数	字符串或函数句柄
DiffuseStrength	发散光强度	[0,1]之间的标量, {0.6}
DisplayName	插图中标明的注释字符串	字符串
EdgeAlpha	块边缘的透明度	[0,1]之间的标量, flat, interp, {1}
EdgeColor	块边缘的颜色	颜色字符串、RGB向量、none、flat、interp、{[0,0,0]}
EdgeLighting	块边缘光照的方法	{none}、flat、 gouraud、 phong
FaceAlpha	块的透明度	[0,1]之间的标量, flat, interp, {1}
FaceColor	块的面颜色	颜色字符串、RGB向量、none、flat、interp、{[0,0,0]}
FaceLighting	块的面光照方法	{none}、flat、 gouraud、 phong
FaceNormals	面的法线向量	$m \times n \times 3$ 的数组, 法线数组
FaceNormalsMode	面的法线向量的设定方式	{auto}、manual
Faces	m 个块的 n 个顶点的连接方法	$m \times n$ 矩阵
FaceVertexAlphaData	面与顶点的透明度	$m \times 1$ 矩阵
FaceVertexCData	面与顶点的颜色	矩阵
HandleVisibility	当前Patch对象的句柄是否可见	{on}、callback、off
HitTest	Patch对象能否通过鼠标选中	{on}、off
Interruptible	回调函数是否可中断	{on}、off
LineStyle	块边缘线型	{-}、-、-., none
LineWidth	块边缘线宽, 单位为points	标量
Marker	顶点的标记符号	标记定义符
MarkerEdgeColor	顶点标记的边缘颜色	颜色字符串或颜色矩阵、none、{auto}、flat
MarkerFaceColor	顶点标记的填充颜色	颜色字符串或颜色矩阵、none、{auto}、flat
MarkerSize	标记的尺寸, 单位为points	正整数, {6}
Parent	父对象的句柄	Axes、hggroup、hgtransform句柄
PickableParts	是否捕捉鼠标点击操作	{visible}、all、none
Selected	Patch对象能否被选中	{on}、off
SelectionHighlight	Patch对象选中时是否高亮显示	{on}、off
SpecularColorReflectance	镜面反射光颜色	[0,1]之间的标量
SpecularExponent	镜面反射光的锐度	不小于1的标量
SpecularStrength	镜面反射的强度	[0,1]之间的标量, {0.9}

续 表

属性	描述	有效属性值
Tag	Patch对象的标识符	字符串
Type	Patch对象的类型	patch
UIContextMenu	Patch对象的右键菜单	右键菜单句柄
UserData	用户定义的数据	任一矩阵
VertexNormals	顶点的法向量	矩阵
VertexNormalsMode	顶点的法向量的设定方式	{auto}、 manual
Vertices	顶点的坐标值	矩阵
Visible	设定Patch对象是否可见	{on}、 off
X(Y,Z)Data	Patch对象的坐标数据	同维的坐标向量

22.4.6 Rectangle对象

创建Rectangle对象的函数是rectangle函数，调用格式为：

```
rectangle(PropertyName, PropertyValue, ...) % 按指定属性值创建Rectangle对象
h = rectangle( ... ) % 创建Rectangle对象并返回句柄值
```

表 22.10: Rectangle对象及描述

属性	描述	有效属性值
AlignVertexCenters	尖的垂直和水平线	on、 {off}
BeingDeleted	调用DeleteFcn时属性值为on； 只读	on、 {off}
BusyAction	指定如何处理中断调用函数	cancel、 {queue}
ButtonDownFcn	按下鼠标时的回调函数	字符串或函数句柄
Children	Rectangle对象没有子对象	空矩阵
Clipping	Rectangle对象是否限定在坐标轴内	{on}、 off
CreateFcn	创建Rectangle对象时的回调函数	字符串或函数句柄
Curvature	矩形沿水平和垂直方向的曲率	[x]或[x,y]
DeleteFcn	销毁Rectangle对象时的回调函数	字符串或函数句柄
EdgeColor	矩形边缘的颜色	颜色字符串、 RGB向量、 none、 flat、 interp、 {[0,0,0]}
FaceColor	矩形的填充颜色	颜色字符串、 RGB向量、 {none}、 flat、 interp
HandleVisibility	当前Rectangle对象的句柄是否可见	{on}、 callback、 off

续 表

属性	描述	有效属性值
HitTest	Rectangle对象能否通过鼠标选中	{on}、off
Interruptible	回调函数是否可中断	{on}、off
LineStyle	矩形边缘线型	{-}、-、-..、none
LineWidth	矩形边缘线宽，单位为points	标量，{0.5}
Parent	父对象的句柄	Axes、hggroup、hgtransform句柄
PickableParts	是否捕捉鼠标点击操作	{visible}、all、none
Position	矩形区域的位置与大小	[左，底，宽，高]
Selected	Rectangle对象能否被选中	{on}、off
SelectionHighlight	Rectangle对象选中时是否高亮显示	{on}、off
Tag	Rectangle对象的标识符	字符串
Type	Rectangle对象的类型	rectangle
UIContextMenu	Rectangle对象的右键菜单	右键菜单句柄
UserData	用户定义的数据	任一矩阵
VertexNormals	顶点的法向量	矩阵
VertexNormalsMode	顶点的法向量的设定方式	{auto}、manual
Vertices	顶点的坐标值	矩阵
Visible	设定Rectangle对象是否可见	{on}、off

22.4.7 Surface对象

曲面对象由surface函数创建，调用格式为：

```

surface(Z)                % 以Z及其索引值创建Surface对象
surface(Z, C)              % 以数据Z及其颜色矩阵C创建Surface对象
surface(X, Y, Z)           % 以数据点(X,Y,Z)创建Surface对象
surface(X, Y, Z, C)        % 以数据点(X,Y,Z)及颜色矩阵C创建Surface对象
surface(PropertyName, PropertyValue, ...) % 按指定属性值创建Surface对象
h = surface(...)           % 创建Surface对象并返回句柄值

```

表 22.11: Surface对象及描述

属性	描述	有效属性值
AlignVertexCenters	尖的垂直和水平线	on、{off}
AlphaData	透明度	{1}、标量、与CData同维的数组
AlphaDataMapping	透明度值映射	none、{scaled}、direct
AmbientStrength	环境光强度	[0,1]之间的标量，{0.3}

续 表

属性	描述	有效属性值
Annotation	Surface对象的插图方式	hg.Annotation句柄
BackFaceLighting	表面光照控制	unlit、lit、{reverselit}
BeingDeleted	调用DeleteFcn时属性值为on；只读	on、{off}
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	按下鼠标时的回调函数	字符串或函数句柄
CData	定义曲面的颜色	标量、向量或矩阵
CDataMapping	CData数据到色图的映射	{scaled}、direct
CDataMode	曲面颜色的设定方式	{auto}、manual
Children	Surface对象没有子对象	空矩阵
Clipping	Surface对象是否限定在坐标轴内	{on}、off
CreateFcn	创建Surface对象时的回调函数	字符串或函数句柄
DeleteFcn	销毁Surface对象时的回调函数	字符串或函数句柄
DiffuseStrength	发散光强度	[0,1]之间的标量, {0.6}
DisplayName	插图中标明的注释字符串	字符串
EdgeAlpha	曲面边缘的透明度	[0,1]之间的标量, flat, interp, {1}
EdgeColor	曲面边缘的颜色	颜色字符串、RGB向量、none、flat、interp、{[0,0,0]}
EdgeLighting	曲面边缘光照的方法	{none}、flat、gouraud、phong
FaceAlpha	曲面的透明度	[0,1]之间的标量, flat, interp, {1}
FaceColor	曲面的颜色	颜色字符串、RGB向量、none、flat、interp、{[0,0,0]}
FaceLighting	曲面光照方法	{none}、flat、gouraud、phong
FaceNormals	曲面的法线向量	$m \times n \times 3$ 的数组, 法线数组
FaceNormalsMode	曲面的法线向量的设定方式	{auto}、manual
HandleVisibility	当前Surface对象的句柄是否可见	{on}、callback、off
HitTest	Surface对象能否通过鼠标选中	{on}、off
Interruptible	回调函数是否可中断	{on}、off
LineStyle	曲面边缘线型	{-}、-、-、none
LineWidth	曲面边缘线宽, 单位为points	标量, {0.5}
Marker	标记符号	标记定义符
MarkerEdgeColor	标记的边缘颜色	颜色字符串或颜色矩阵、none、{auto}、flat
MarkerFaceColor	标记的填充颜色	颜色字符串或颜色矩阵、{none}、auto、flat
MarkerSize	标记的尺寸, 单位为points	正整数, {6}

续 表

属性	描述	有效属性值
MeshStyle	画行线、列线还是全部都画	{bith}、row、column
Parent	父对象的句柄	Axes、hggroup、hgtransform句柄
PickableParts	是否捕捉鼠标点击操作	{visible}、all、none
Selected	Surface对象能否被选中	{on}、off
SelectionHighlight	Surface对象选中时是否高亮显示	{on}、off
SpecularColorReflectance	镜面反射光颜色	[0,1]之间的标量
SpecularExponent	镜面反射光的锐度	不小于1的标量
SpecularStrength	镜面反射的强度	[0,1]之间的标量, {0.9}
Tag	Surface对象的标识符	字符串
Type	Surface对象的类型	surface
UIContextMenu	Surface对象的右键菜单	右键菜单句柄
UserData	用户定义的数据	任一矩阵
VertexNormals	顶点的法向向量	矩阵
VertexNormalsMode	顶点的法向向量的设定方式	{auto}、manual
Vertices	顶点的坐标值	矩阵
Visible	设定Surface对象是否可见	{on}、off
X(Y,Z)Data	Surface对象的坐标数据	同维的坐标向量
X(Y,Z)DataMode	坐标数据的设定方式	{auto}、manual

22.4.8 Hggroup对象

坐标轴的子对象除了核心图像外，还可以是由核心对象组合而成的对象(简称组对象)。组对象由函数hggroup创建，调用格式为：

```
h = hggroup(PropertyName, PropertyValue, ...) % 用指定属性和属性值创建Hggroup对象并返回句柄
```

Hggroup对象可以由以下核心对象自由组合而成：Image对象、Line对象、Patch对象、Rectangle对象、Surface对象和Text对象。除了Light对象均可作为Hggroup对象的子对象。它可以将多个核心对象“捆绑”起来，方便用户同时对多个核心对象的控制。常见的Hggroup对象有：图形注释(由annotation函数生成)、插图(由legend函数生成)、火柴杆图(由stem函数生成)、直方图(由bar函数生成)、图形截取框(由imrect函数生成)等。

Hggroup对象与其子对象的可见性(Visible)、可选择性(HitTest)保持一致。Hggroup对象设置为可见和可选择时，其子对象也会可见、可选择。

表 22.12: Hggroup对象及描述

属性	描述	有效属性值
Annotation	Hggroup对象的插图方式	hg.Annotation句柄
BeingDeleted	调用DeleteFcn时属性值为on; 只读	on、{off}
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	按下鼠标时的回调函数	字符串或函数句柄
Children	Hggroup对象没有子对象	空矩阵
CreateFcn	创建Hggroup对象时的回调函数	字符串或函数句柄
DeleteFcn	销毁Hggroup对象时的回调函数	字符串或函数句柄
DisplayName	插图中标明的注释字符串	字符串
HandleVisibility	当前Hggroup对象的句柄是否可见	{on}、callback、off
HitTest	Hggroup对象能否通过鼠标选中	{on}、off
Interruptible	回调函数是否可中断	{on}、off
Parent	父对象的句柄	Axes、hggroup、hgtransform句柄
PickableParts	是否捕捉鼠标点击操作	{visible}、all、none
Selected	Hggroup对象能否被选中	{on}、off
SelectionHighlight	Hggroup对象选中时是否高亮显示	{on}、off
Tag	Hggroup对象的标识符	字符串
Type	Hggroup对象的类型	hggroup
UIContextMenu	Hggroup对象的右键菜单	右键菜单句柄
UserData	用户定义的数据	任一矩阵
Visible	设定Hggroup对象是否可见	{on}、off

22.5 UI对象

22.5.1 Uicontrol对象

Uicontrol对象是用户接口控制(User interface control)图形对象的简称，由函数uicontrol创建，其格式为：

```
h = uicontrol(PropertyName, PropertyValue, ...) % 指定属性值创建uicontrol对象并返回句柄
h = uicontrol(Parent, PropertyName, PropertyValue, ...) % 在Parent内创建uicontrol对象并返回句柄
uicontrol(h) % 将uicontrol对象h设置为当前对象
```

表 22.13: Uicontrol对象及描述

属性	描述	有效属性值
BackgroundColor	对象背景颜色	颜色字符串或颜色矩阵
BeingDeleted	调用DeleteFcn时属性值为on; 只读	on、{off}
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	按下鼠标时的回调函数	字符串或函数句柄
Callback	控制uicontrol对象时的回调函数	字符串或函数句柄
CData	控制uicontrol对象的图案	矩阵
Children	uicontrol对象没有子对象	空矩阵
Clipping	对uicontrol对象无效	{on}、off
CreateFcn	创建uicontrol对象时的回调函数	字符串或函数句柄
DeleteFcn	销毁uicontrol对象的回调函数	字符串或函数句柄
Enable	使用或禁用uicontrol对象	{on}、inactive、off
Extent	uicontrol对象上字符串的位置和尺寸	[左, 底, 宽, 高]
FontAngle	选择斜体或普通字体	{normal}、italic、oblique
FontName	字体名	系统支持的字体名称
FontSize	字体大小, 单位由FontUnits决定	整数, {12}
FontUnits	字体尺寸单位	{points}、normalized、pixels、inches、centimeters
FontWeight	选择粗体或正常字体	{normal}、bold、light、demi
ForegroundColor	文本颜色	颜色字符串或颜色矩阵
HandleVisibility	当前uicontrol对象的句柄是否可见	{on}、callback、off
HorizontalAlignment	文本字符串的水平对齐方式	left、{center}、right
Interruptible	回调函数是否可中断	{on}、off
KeyPressFcn	该对象上按下一个键时的回调函数	字符串或函数句柄
KeyReleaseFcn	该对象上释放按键时的回调函数	字符串或函数句柄
ListboxTop	显示在listbox最顶端的字符串索引值	正整数, {1}
Max	指定Value属性的最大值	数值
Min	指定Value属性的最小值	数值
Parent	父对象的句柄	Figure句柄
Position	uicontrol对象的位置与大小	[左, 底, 宽, 高]
SliderStep	指定slider的步长	二维向量, [minstep, maxstep]
String	文本标签、选项或列表项	字符串或字符串单元数组
Style	uicontrol对象的类型(见表 22.14)	{pushbutton}、togglebutton、edit、text、radiobutton、checkbox、slider、frame、listbox、popupmenu

续 表

属性	描述	有效属性值
Tag	uicontrol对象的标识符	字符串
TooltipString	uicontrol对象的提示字符	字符串
Type	uicontrol对象的类型	uicontrol
UIContextMenu	uicontrol对象的右键菜单	右键菜单句柄
Units	计量单位	{pixels}、 inches、 normalized、 points、 characters、 centimeters
UserData	用户定义的数据	任一矩阵
Visible	设定uicontrol对象是否可见	{on}、 off

控件对象是事件响应的图形界面对象。当某一事件发生时，应用程序会做出响应并执行某些预定的功能子程序(Callback)。MATLAB中的控件大致可分为两种，一种为动作控件，鼠标点击这些控件时会产生相应的响应。一种为静态控件，是一种不产生响应的控件，如文本框等。每种控件都有一些可以设置的参数，用于表现控件的外形、功能及效果，即属性。属性由两部分组成：属性名和属性值，它们必须是成对出现的。常用的控件对象如表 22.14 所示。

表 22.14: 控件对象及描述

控件对象类型	描述
按钮(Push Buttons)	执行某种预定的功能或操作
开关按钮(Toggle Button)	产生一个动作并指示一个二进制状态(开或关)，当鼠点击它时按钮将下陷，并执行callback(回调函数)中指定的内容，再次点击，按钮复原，并再次执行callback中的内容
单选框(Radio Button)	单个的单选框用来在两种状态之间切换，多个单选框组成一个单选框组时，用户只能在一组状态中选择单一的状态
复选框(Check Boxes)	单个的复选框用来在两种状态之间切换，多个复选框组成一个复选框组时，可使用户在一组状态中作组合式的选择
文本编辑框(Editable Texts)	用来使用键盘输入字符串的值，可以对编辑框中的内容进行编辑、删除和替换等操作
静态文本框(Static Texts)	仅仅用于显示单行的说明文字
滚动条(Slider)	通过滑动滑块，调整显示内容或从指定范围中选择一个数值
列表框(List Boxes)	定义一系列可供选择的条目
弹出式菜单(Popup Menus)	让用户从一系列菜单项中选择一项作为参数输入
坐标轴(Axes)	用于显示图形和图象
ActiveX控件(ActiveX Control)	在GUI中显示控件，但只能在Windows操作系统中使用

22.5.2 按钮组与面板对象

按钮组(Uibuttongroup)与面板(Uipanel)都是其他GUI对象的容器，但按钮组的功能更强大。当按钮组包含多个单选框或开关按钮对象时，这些对象在同一时刻只能有一个的状态为on。按钮组与面板对象的属性见表 22.15。

```
h = uibuttongroup(PropertyName, PropertyValue, ...) % 创建uibuttongroup对象并返回句柄
h = uipanel(PropertyName, PropertyValue, ...) % 创建uipanel对象并返回句柄
```

表 22.15: 按钮组与面板对象及描述

属性	描述	有效属性值
BackgroundColor	对象背景颜色	颜色字符串或颜色矩阵
BorderType	对象的边界类型	none、{etchedin}、etchedout、line、beveledin、beveledout
BorderWidth	对象的边界宽度，单位为像素	正整数
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	在对象上按下鼠标时的回调函数	字符串或函数句柄
CData	控制uicontrol对象的图案	矩阵
Children	按钮组/面板对象的子对象	句柄向量
Clipping	子对象能否超出边框范围	{on}、off
CreateFcn	创建按钮组/面板对象的回调函数	字符串或函数句柄
DeleteFcn	销毁按钮组/面板对象的回调函数	字符串或函数句柄
FontAngle	选择斜体或普通字体	{normal}、italic、oblique
FontName	字体名	系统支持的字体名称
FontSize	字体大小，单位由FontUnits决定	整数
FontUnits	字体尺寸单位	{points}、normalized、pixels、inches、centimeters
FontWeight	选择粗体或正常字体	{normal}、bold、light、demi
ForegroundColor	文本颜色	颜色字符串或颜色矩阵
HandleVisibility	当前对象的句柄是否可见	{on}、callback、off
HighlightColor	高亮显示时的颜色	颜色字符串或颜色矩阵
HitTest	能否通过鼠标单击选择该对象	{on}、off
Interruptible	回调函数是否可中断	{on}、off
Parent	父对象的句柄	窗口、面板或按钮组对象句柄
Position	按钮组/面板对象的位置与大小	[左, 底, 宽, 高]

续 表

属性	描述	有效属性值
ResizeFcn	按钮组/面板或窗口大小改变时的回调函数	字符串或函数句柄
Selected	按钮组/面板对象是否能被选中	{on}、off
SelectedObject	当前选中的单选框或开关按钮对象	单个句柄(仅按钮组有该属性)
SelectionChangeFcn	选中的单选框或开关按钮对象改变的回调函数	字符串或函数句柄(仅按钮组有该属性)
SelectionHighlight	选中按钮组/面板对象时突出显示	{on}、off
ShadowColor	3D框架的阴影颜色	颜色字符串或颜色矩阵
Tag	按钮组/面板对象的标识符	字符串
Title	按钮组/面板对象的标题	字符串
TitlePosition	按钮组/面板对象的标题位置	{lefttop}、centertop、righttop、leftbottom、centerbottom、rightbottom
Type	按钮组/面板对象类型	uipanel
UIContextMenu	uicontrol对象的右键菜单	右键菜单句柄
Units	计量单位	pixels、 inches、 {normalized}、points、 characters、 centimeters
UserData	用户定义的数据	任一矩阵
Visible	设定按钮组/面板对象是否可见	{on}、off

22.5.3 菜单与右键菜单对象

MATLAB中uimenu函数可以创建自定义菜单(Uimenu)对象，也可以创建菜单选项，uicontextmenu函数仅可以创建右键菜单(Uicontextmenu)对象，并不能创建右键菜单选项，创建右键菜单选项只能使用uimenu函数。

```
h = uimenu(PropertyName, PropertyValue, ...) % 创建自定义菜单对象并返回句柄
h = uimenu(Parent, PropertyName, PropertyValue, ...) % 为自定义菜单对象或右键菜单对象创建菜单选项
h = uicontextmenu(PropertyName, PropertyValue, ...) % 创建右键菜单对象并返回句柄
```

表 22.16: 菜单与右键菜单对象及描述

属性	描述	有效属性值
Accelerator	设定快捷键，Ctrl+Accelerator	字符(仅菜单有该属性)
BusyAction	指定如何处理中断调用函数	cancel、 {queue}

续 表

属性	描述	有效属性值
Callback	选择菜单选项时的回调函数	字符串或函数句柄
Checked	设置菜单选项的检查标识符“√”	on、{off}
CreateFcn	创建菜单/右键菜单对象的回调函数	字符串或函数句柄
DeleteFcn	销毁菜单/右键菜单对象的回调函数	字符串或函数句柄
Enable	使用或禁用菜单/右键菜单对象	{on}、off
ForegroundColor	菜单标签字符串颜色	颜色字符串或颜色矩阵
HandleVisibility	当前对象的句柄是否可见	{on}、callback、off
Interruptible	回调函数是否可中断	{on}、off
Label	菜单标签	字符串
Parent	父对象的句柄	窗口、面板或按钮组对象句柄
Position	菜单/右键菜单对象的相对位置	标量
Separator	指定菜单选项上方是否设置分割线	on、{off}
Tag	菜单/右键菜单对象的标识符	字符串
Type	菜单/右键菜单对象类型	uimenu/uicontextmenu
UserData	用户定义的数据	任一矩阵
Visible	设定菜单/右键菜单对象是否可见	{on}、off

22.5.4 工具栏与工具栏按钮对象

工具栏对象由uitoolbar创建，因此也称Uitoolbar对象，工具栏按钮对象可分为Uipushtool对象和Uitoggletool对象两种，分别由uipushtool和uitoggletool创建。工具栏及其子对象仅能显示在WindowStyle属性值为normal或docked的窗口中。若窗口WindowStyle属性值为modal，则该工具栏仍然存在，但不会显示出来。

```

ht = uitoolbar(PropertyName, PropertyValue, ...) % 创建工具栏对象并返回句柄
ht = uitoolbar(h, PropertyName, PropertyValue, ...) % 为窗口h创建工具栏对象并返回句柄
htt = uipushtool(PropertyName, PropertyValue, ...) % 在当前窗口的当前工具栏创建工具栏按钮
htt = uipushtool(ht, PropertyName, PropertyValue, ...) % 在工具栏ht内创建工具栏按钮
htt = uitoggletool(PropertyName, PropertyValue, ...) % 当前窗口顶部工具栏内创建Uitoggletool对象
htt = uitoggletool(ht, PropertyName, PropertyValue, ...) % 在工具栏ht内创建Uitoggletool对象

```

表 22.17: 工具栏与工具栏按钮对象及描述

属性	描述	有效属性值
BeingDeleted	调用DeleteFcn时属性值为on；只读	on、{off}
BusyAction	指定如何处理中断调用函数	cancel、{queue}

续 表

属性	描述	有效属性值
ButtonDownFcn	工具栏/工具栏按钮对象上按下鼠标时的回调函数	字符串或函数句柄
CData	工具栏按钮对象的图案	矩阵(工具栏对象无该属性)
Children	工具栏有子对象, 值为工具栏按钮对象, 工具栏按钮无子对象, 值为空	工具栏按钮对象/空矩阵
ClickedCallback	该对象或周围5像素内单击鼠标的回调函数	字符串或函数句柄(工具栏对象无该属性)
Clipping	限幅模式	{on}、off
CreateFcn	创建对象的回调函数	字符串或函数句柄
DeleteFcn	销毁对象的回调函数	字符串或函数句柄
Enable	使用或禁用工具栏按钮对象	{on}、off(工具栏对象无该属性)
HandleVisibility	当前对象的句柄是否可见	{on}、callback、off
Interruptible	回调函数是否可中断	{on}、off
OffCallback	Uitoggletool对象弹起时的回调函数	字符串或函数句柄(仅Uitoggletool对象有该属性)
OnCallback	Uitoggletool对象按下时的回调函数	字符串或函数句柄(仅Uitoggletool对象有该属性)
Parent	父对象的句柄	figure/工具栏对象句柄
Separator	指定该对象左侧是否设置分割线	on、{off}(工具栏对象无该属性)
State	显示Uitoggletool对象的状态	on、{off}(仅Uitoggletool对象有该属性)
Tag	对象的标识符	字符串
TooltipString	工具栏按钮对象的提示字符	字符串(工具栏对象无该属性)
Type	对象类型	uitoolbar/uipushtool/uitoggletool
UserData	用户定义的数据	任一矩阵
Visible	设定对象是否可见	{on}、off

22.5.5 表格对象

表格(Uitable)对象为二维的可视化表格, 由函数uitable创建, 其格式为:

```
uitable(PropertyName, PropertyValue, ...) % 按指定属性值创建表格对象
uitable(Parent, PropertyName, PropertyValue, ...) % 为Parent(Figure或Uipanel对象)创建表格对象
ht = uitable( ... ) % 返回表格对象的句柄
```

表格对象及其属性见表 22.18。

表 22.18: 表格对象及描述

属性	描述	有效属性值(n 为表格列数)
BackgroundColor	表格单元的背景颜色或条纹色	[0,1]之间的 1×3 或 2×3 的RGB矩阵
BeingDeleted	调用DeleteFcn时属性值为on; 只读	on、{off}
BusyAction	指定如何处理中断调用函数	cancel、{queue}
ButtonDownFcn	表格上按下鼠标时的回调函数	字符串或函数句柄
CellEditCallback	修改表格单元格时的回调函数	函数句柄、函数句柄和附加参数组成的单元数组、可执行字符串
CellSelectionCallback	表格单元格被选中时的回调函数	函数句柄、函数句柄和附加参数组成的单元数组、可执行字符串
ColumnEditable	用户是否可以编辑列	n 维的逻辑向量、标量逻辑值、[]
ColumnFormat	表格单元的显示格式	字符串、字符串单元数组, {}
ColumnName	指定表格列名	n 维的单元数组, {numbered}, []
ColumnWidth	表格每列的宽度, 单位为像素	n 维的单元数组, {auto}
CreateFcn	创建对象的回调函数	字符串或函数句柄
Data	表格数据	数值矩阵, 逻辑矩阵, 数值单元数组, 逻辑单元数组, 字符串单元数组以及它们的混合单元数组
DeleteFcn	销毁对象的回调函数	字符串或函数句柄
Enable	使用或禁用表格对象	{on}、inactive、off
Extent	表格框架的尺寸	[左, 底, 宽, 高], 只读
FontAngle	选择斜体或普通字体	{normal}、italic、oblique
FontName	字体名	系统支持的字体名称
FontSize	字体大小, 单位由FontUnits决定	整数
FontUnits	字体尺寸单位	{points}、normalized、pixels、inches、centimeters
FontWeight	选择粗体或正常字体	{normal}、bold、light、demi
ForegroundColor	文本颜色	颜色字符串或颜色矩阵
HandleVisibility	当前对象的句柄是否可见	{on}、callback、off
Interruptible	回调函数是否可中断	{on}、off
KeyPressFcn	表格对象上按下一个键时的回调函数	字符串或函数句柄
KeyReleaseFcn	表格对象上释放按键时的回调函数	字符串或函数句柄
Parent	父对象的句柄	figure句柄
Position	表格对象的位置与大小	[左, 底, 宽, 高], 单位为Units
RearrangeableColumns	表格数据是否可以按列重新排列	on、{off}
RowName	表格的行头名称	n 维的单元数组, {numbered}, []
RowStripping	指定表格的行是否采用彩色条纹模式	{on}、off

续 表

属性	描述	有效属性值
Tag	表格对象的标识符	字符串
TooltipString	表格对象的提示字符	字符串
Type	表格对象类型	uitable
UIContextMenu	表格对象的右键菜单	右键菜单句柄
Units	计量单位	{pixels}、 inches、 normalized、 points、 characters、 centimeters
UserData	用户定义的数据	任一矩阵
Visible	设定表格对象是否可见	{on}、 off

22.6 2048游戏设计



图 22.2: 2048游戏界面

2048是一款比较流行的数字游戏，最早于2014年3月20日由Gabriele Cirulli在GitHub上发布，后被移植到各个平台。这款游戏的玩法很简单，每次可以选择上下左右滑动，每滑动一次，所有的数字方块都会往滑动的方向靠拢，系统也会在空白的地方乱数出现一个数字方块，相同数字的方块在靠拢、相撞时会相加。不断的叠加最终拼凑出2048这个数字就算成功，若系统无法向某一方向滑动且未

达到2048，则该局游戏失败。

下面给出2048游戏的主程序，这些程序参考了Jiro Doke编写的“2048 MATLAB Edition”¹，其中一部分是界面设计(如图 22.2)的内容，它使用了诸如图形界面、坐标轴、矩形、菜单栏、工具栏等对象。本文的程序在设计之初就添加一些有趣的功能以保证玩家的游戏体验，具体内容详见注 22.1。尽管程序中大多数代码已添加注释，但仍然晦涩难懂，读者可以参照前几节图形对象的介绍进行理解。

```
function play2048game
% -----
% MATLAB版 2048 游戏GUI界面(含自动求解最佳移动方向和自动演示功能)
% MADE BY Bao-Shan Wang
% -----
global handles iconsData iseq
% 主界面
if datenum(version('-date')) > datenum('2-29-2012') % 对比版本在R2012a的前后
    iseq = str2func('isequaln');
else
    iseq = str2func('isequalwithequalnans');
end
try
    setMail; % 设置邮箱服务器，为留言做准备，详见第二十四章
catch
    uiwait(warndlg('邮箱服务器设置错误，无法使用留言功能！', '提示...'));
end

set(0, 'Units', 'Pixels') % 设置根对象的单位为像素
sc = get(0, 'ScreenSize'); % 获取屏幕尺寸
val = sc(3:4); % 获取屏幕的宽高
close all force % 关闭所有界面
if ~exist('game2048.mat', 'file') % 判断game2048.mat文件是否存在
    handles.numb = 1; % 游戏次数置为1
    handles.Scores(1).HighScore = 0; % 当前游戏的最高分数置为0
    handles.Scores(1).Scores = 0; % 游戏的分数置为0
    handles.Scores(1).NumMoves = 0; % 移动次数置为0
    handles.Scores(1).HighestBlock = 0; % 最大块值置为0
    handles.Best = 0; % 最佳分数置为0
    handles.game = game2048; % 建立游戏数据
    load('splat.mat') % 加载声音文件
    handles.Sound.y = y(3000:4:5000); % 取音乐文件的一部分
    handles.Sound.b = y(5500:4:8000); % 爆炸声
    handles.Sound.Fs = Fs; % 音乐频率
else
    handles = load('game2048.mat'); % 获取上次游戏数据
    if isGameOver(handles.game) % 判断上次游戏是否结束
        handles.numb = handles.numb + 1; % 游戏次数加1
        handles.Scores(handles.numb).HighScore = 0; % 初始为0
        handles.Scores(handles.numb).Scores = 0; % 初始为0
        handles.Scores(handles.numb).NumMoves = 0; % 初始为0
        handles.Scores(handles.numb).HighestBlock = 0; % 初始为0
        handles.game = game2048; % 建立游戏数据
    end
end
handles.sflag = 1; % 打开声音，0为关闭
```

¹下载地址: <http://cn.mathworks.com/matlabcentral/fileexchange/46124-2048-matlab-edition>

```

handles.hflag = 1; % 打开动画, 0为关闭
handles.pflag = 0; % 关闭演示, 1为打开
handles.aifun = @Oliver_AI_v2; % 默认为高级AI算法
handles.xPts = repmat(0.5:3.5, 4, 1); % 小块的X轴坐标
handles.yPts = repmat((3.5:-1:0.5)', 1, 4); % 小块的Y轴坐标
% 创建游戏主界面
hFig = figure(...
    'NumberTitle', 'off', ... % 关闭数字标题
    'Name', '2048', ... % 界面名称2048
    'HandleVisibility', 'off', ...
    'Toolbar', 'none', ... % 无工具栏
    'Menu', 'none', ... % 无菜单栏
    'Color', [0.8,0.8,0.8], ... % 背景颜色
    'Units', 'Pixels', ... % 单位像素
    'Position', [val/2-[210, 240], 420, 480], ... % 位置居中
    'Visible', 'off', ... % 隐藏界面
    'BusyAction', 'cancel', ...
    'Interruptible', 'off', ...
    'Resize', 'off', ... % 不改变大小
    'DockControl', 'off', ...
    'DeleteFcn', @deleteHFig, ... % 删除函数
    'WindowKeyPressFcn', @keyPressFcn); % 按键响应函数
handles.hFig = hFig; % 保存数据到结构体handles
% 以下创建菜单栏按钮及响应函数
uigame1 = uimenu(handles.hFig, 'Label', '游戏');
uimenu(uigame1, 'Label', '新建游戏', 'callback', @newgame);
uimenu(uigame1, 'Label', '游戏提示', 'callback', @suggest);
uimenu(uigame1, 'Label', '当前统计', 'callback', @nowgame);
handles.ht = uimenu(uigame1, 'Label', '历史记录', 'Enable', 'off', 'callback', @histgame);
if handles.numb > 1, set(handles.ht, 'Enable', 'on'); end % 若次数大于1, 则历史记录可查
uimenu(uigame1, 'Label', '退出', 'callback', @deleteHFig);
uigame2 = uimenu(handles.hFig, 'Label', '设置');
uigame21 = uimenu(uigame2, 'Label', '动画');
uimenu(uigame21, 'Label', '开启', 'callback', @animOn);
uimenu(uigame21, 'Label', '关闭', 'callback', @animOff);
uigame22 = uimenu(uigame2, 'Label', '声音');
uimenu(uigame22, 'Label', '开启', 'callback', @soundOn);
uimenu(uigame22, 'Label', '关闭', 'callback', @soundOff);
uigame23 = uimenu(uigame2, 'Label', '颜色');
uimenu(uigame23, 'Label', '设置', 'callback', @cset);
uimenu(uigame23, 'Label', '默认', 'callback', @mcset);
uigame24 = uimenu(uigame2, 'Label', '演示');
uimenu(uigame24, 'Label', '开启', 'callback', @demoOn);
uimenu(uigame24, 'Label', '关闭', 'callback', @demoOff);
uigame25 = uimenu(uigame2, 'Label', 'AI算法');
uimenu(uigame25, 'Label', '高级', 'callback', @aiHigh);
uimenu(uigame25, 'Label', '中级', 'callback', @aiMid);
uimenu(uigame25, 'Label', '初级', 'callback', @aiLow);
uimenu(uigame25, 'Label', '自定义', 'callback', @aiCustom);
uigame26 = uimenu(uigame2, 'Label', '清空');
uimenu(uigame26, 'Label', '历史记录', 'callback', @clearScores);
uimenu(uigame26, 'Label', '最佳记录', 'callback', @clearBest);
uigame3 = uimenu(handles.hFig, 'Label', '帮助');
uimenu(uigame3, 'Label', '帮助', 'callback', @mhelp);
uimenu(uigame3, 'Label', '留言', 'callback', @sm);
uimenu(uigame3, 'Label', '关于', 'callback', @about);
% 以下创建工具栏按钮及响应函数
uitbar = uitoolbar('Parent', handles.hFig); % 创建工具栏

```

```

uipb1 = uipushtool(uitbar, ...
    'ClickedCallback', @newgame, ...
    'TooltipString', '新建游戏');
uipb2 = uipushtool(uitbar, ...
    'ClickedCallback', @SaveScreenshot, ...
    'TooltipString', '保存游戏界面');
uipb3 = uipushtool(uitbar, ...
    'ClickedCallback', @suggest, ...
    'TooltipString', '搜索最佳移动方向');
uipb4 = uipushtool(uitbar, ...
    'ClickedCallback', @bomb, ...
    'TooltipString', '用鼠标左键选中一个块并炸掉, 用右键取消');
uitb1 = uitoggletool(uitbar, ...
    'OffCallback', @animOff, ...
    'OnCallback', @animOn, ...
    'Separator', 'on', ...
    'TooltipString', '动画开关');
uitb2 = uitoggletool(uitbar, ...
    'OffCallback', @soundOff, ...
    'OnCallback', @soundOn, ...
    'TooltipString', '音乐开关');
uitb3 = uitoggletool(uitbar, ...
    'OffCallback', @demoOff, ...
    'OnCallback', @demoOn, ...
    'TooltipString', '演示开关');
uipb5 = uipushtool(uitbar, ...
    'ClickedCallback', @nowgame, ...
    'Separator', 'on', ...
    'TooltipString', '当前游戏记录');
uipb6 = uipushtool(uitbar, ...
    'ClickedCallback', @histgame, ...
    'TooltipString', '历史游戏记录');
if handles.hflag == 1
    set(uitb1, 'State', 'on' )
else
    set(uitb1, 'State', 'off')
end
if handles.sflag == 1
    set(uitb2, 'State', 'on' )
else
    set(uitb2, 'State', 'off')
end
if handles.pflag == 1
    set(uitb3, 'State', 'on' )
else
    set(uitb3, 'State', 'off')
end
if exist('iconsData.mat', 'file') % 判断iconsData文件是否存在mat
    iconsData = load('iconsData.mat');
    iconsData.AIOff = double(iconsData.AIOff ) / 255;
    iconsData.AIOn = double(iconsData.AIOn ) / 255;
    iconsData.AnimOff = double(iconsData.AnimOff ) / 255;
    iconsData.AnimOn = double(iconsData.AnimOn ) / 255;
    iconsData.Bomb = double(iconsData.Bomb ) / 255;
    iconsData.History = double(iconsData.History ) / 255;
    iconsData.New = double(iconsData.New ) / 255;
    iconsData.Now = double(iconsData.Now ) / 255;
    iconsData.Search = double(iconsData.Search ) / 255;

```

```

iconsData.SoundOff = double(iconsData.SoundOff) / 255;
iconsData.SoundOn  = double(iconsData.SoundOn ) / 255;
set(uijb1, 'CData', iconsData.New      )
set(uijb2, 'CData', iconsData.Save     )
set(uijb3, 'CData', iconsData.Search   )
set(uijb4, 'CData', iconsData.Bomb     )
set(uijb5, 'CData', iconsData.Now      )
set(uijb6, 'CData', iconsData.History)
if handles.hflag == 1
    set(uitb1, 'CData', iconsData.AnimOn )
else
    set(uitb1, 'CData', iconsData.AnimOff )
end
if handles.sflag == 1
    set(uitb2, 'CData', iconsData.SoundOn )
else
    set(uitb2, 'CData', iconsData.SoundOff)
end
if handles.pflag == 1
    set(uitb3, 'CData', iconsData.AIOn     )
else
    set(uitb3, 'CData', iconsData.AIOff    )
end
end
else
    iconsData = [];
end
handles.uitbar.uitb1 = uitb1;
handles.uitbar.uitb2 = uitb2;
handles.uitbar.uitb3 = uitb3;
% 构建主坐标轴
handles.hAx = axes(...
    'Parent', handles.hFig, ...           % 父对象为handles.hFig
    'Units', 'Pixels', ...                % 单位像素
    'Position', [10 10 400 400], ...       % 位置
    'Color', [187 173 160]/255, ...        % 背景颜色
    'XLim', [0 4], ...                    % X轴范围
    'YLim', [0 4], ...                    % Y轴范围
    'XTick', [], ...                      % X轴刻度为空
    'YTick', [], ...                      % Y轴刻度为空
    'XColor', [187 173 160]/255, ...       % X轴颜色
    'YColor', [187 173 160]/255, ...       % Y轴颜色
    'Box', 'on', ...                      % 显示边框
    'XTickLabel', [], ...                  % X轴刻度标签为空
    'YTickLabel', [], ...                  % Y轴刻度标签为空
    'PlotBoxAspectRatio', [1 1 1]);
handles.hScore = uicontrol(...
    'Parent', handles.hFig, ...           % 父对象为handles.hFig
    'Units', 'Pixels', ...                % 单位像素
    'Position', [10 415 200 50], ...       % 位置
    'Style', 'Text', ...                  % 类型为text
    'FontUnits', 'Pixels', ...            % 字体单位像素
    'FontSize', 14, ...                   % 字体大小14号
    'HorizontalAlignment', 'left', ...    % 对齐方式左对齐
    'BackgroundColor', get(handles.hFig, 'Color'), ... % 背景颜色
    'String', sprintf('Score: %0\nBest: %d', handles.Best)); % 显示文本
handles.hMoves = uicontrol(...
    'Parent', handles.hFig, ...           % 父对象为handles.hFig
    'Units', 'Pixels', ...                % 单位为像素

```

```

    'Position', [300 415 120 50], ...           % 位置
    'Style', 'Text', ...                       % 类型为text
    'FontUnits', 'Pixels', ...                 % 字体单位像素
    'FontSize', 14, ...                       % 字体大小14号
    'HorizontalAlignment', 'left', ...        % 对齐方式左对齐
    'BackgroundColor', get(handles.hFig, 'Color'), ... % 背景颜色
    'String', sprintf('Moves:%d', handles.game.NumMoves)); % 显示文本
% 创建16个空白小块
handles.blocks = gameBlock(handles.xPts(:), handles.yPts(:), ...
    repmat({''}, 16, 1), .9, .9, handles.hAx);
handles = updateblocks(handles); % 更新小块
if exist('2048.png','file') % 若2048.png存在, 设置图标
    warning off all % 屏蔽所有警告
    newIcon = javax.swing.ImageIcon('2048.png'); % 读取2048.png
    figFrame = get(handles.hFig, 'JavaFrame'); % 获得界面的JavaFrame
    figFrame.setFigureIcon(newIcon); % 更改图标
end
set(handles.hFig, 'Visible', 'on'); % 显示游戏主界面
end

% 颜色设置
function cset(varargin)
global handles;
c = uisetcolor; % 打开颜色设置对话框
if c ~= 0 % 若用户选择了颜色
    set(handles.hFig, 'Color', c); % 设置主界面背景
    set(handles.hMoves, 'BackgroundColor', c); % 设置文本框hMoves的背景
    set(handles.hScore, 'BackgroundColor', c); % 设置文本框hScore的背景
end
end

% 默认颜色
function mcset(varargin)
global handles;
set(handles.hFig, 'Color', [0.8,0.8,0.8]);
set(handles.hMoves, 'BackgroundColor', [0.8,0.8,0.8]);
set(handles.hScore, 'BackgroundColor', [0.8,0.8,0.8]);
end

% 新建游戏数据
function game = game2048
game.HighScore = 0; % 最高分数
game.Movement = reshape(1:16,4,4); % 移动矩阵
game.Board = nan(4,4); % 小块的值的矩阵
game.Scores = [0, nan(1,1000)]; % 每步的分数
game.NumMoves = 0; % 移动次数
game.StopNumber = 2048; % 停止数2048
game.HighestBlock = 0; % 最大块的值
game.NewBlockValue = [2 4]; % 新块的值
game.NewBlockValueProbability = [9 1]; % 出现新块的值对应的可能性
game.Board = addNewBlock(game); % 添加新块
end

% 添加新块
function board = addNewBlock(game)
nanID = find(isnan(game.Board)); % 找到空白块的位置
num = randi(numel(nanID), 1); % 选中第num个空白块
probList = cumsum(game.NewBlockValueProbability); % 计算行累计和

```

```

maxI      = sum(game.NewBlockValueProbability); % 求和
ID        = find(probList >= randi(maxI), 1);   % 求块值的ID (1或2)
game.Board(nanID(num)) = game.NewBlockValue(ID); % 对第num个空白块赋值
board = game.Board; % 更新块矩阵
end

% 创建块
function blocks = gameBlock(x, y, txt, wd, ht, hAx)
blocks = struct('Position', cell(16,1), 'Width', [], 'Height', [], ...
    'Txt', [], 'hBox', [], 'hText', []); % 新建空结构体,包括位置,宽高,文字,box框,文本
for id = 1:length(x)
    blocks(id).Position = [x(id), y(id)];
    blocks(id).Width     = wd;
    blocks(id).Height    = ht;
    blocks(id).Txt       = txt{id};
    val = [blocks(id).Position - [blocks(id).Width / 2, blocks(id).Height / 2], ...
        blocks(id).Width, blocks(id).Height];
    blocks(id).hBox = rectangle(... % 绘制矩形框
        'Parent', hAx, ...
        'Position', val, ...
        'Curvature', [.1 .1], ...
        'EdgeColor', 'none', ... % 边界颜色为空
        'FaceColor', [205 192 180]/255);
    blocks(id).hText = text(x(id), y(id), txt{id}, ...
        'Parent', hAx, ...
        'VerticalAlignment', 'middle', ... % 上下居中
        'HorizontalAlignment', 'center', ... % 文字居中
        'FontName', 'Calibri', ... % 文字字体名
        'FontWeight', 'bold', ... % 粗体
        'FontUnits', 'Pixels', ... % 单位为像素
        'FontSize', 48);
end
updateColors(blocks) % 更新小块颜色
end

% 更新颜色
function updateColors(blocks)
txt = get([blocks.hText], 'String'); % 获得小块的文字
bgcolors = repmat([237 194 46]/255, length(blocks), 1); % 表面颜色数据
fgcolors = repmat([249 246 242]/255, length(blocks), 1); % 文字颜色数据
fontsize = 28 * ones(length(blocks), 1); % 预定义文字大小为28
fontsizeTable = [48 48 36 32 28 24]; % 文字大小向量
ii = strcmp('', txt); % 匹配空白块
bgcolors(ii, :) = repmat([205 192 180]/255, nnz(ii), 1); % nnz为非零元素的个数
fontsize(ii) = fontsizeTable(1); % 空白块的字体为48
ii = strcmp('2', txt); % 匹配文字为2的块
bgcolors(ii, :) = repmat([238 228 218]/255, nnz(ii), 1);
fgcolors(ii, :) = repmat([119 110 101]/255, nnz(ii), 1);
fontsize(ii) = fontsizeTable(1); % 文字为2块的字体为48
ii = strcmp('4', txt);
bgcolors(ii, :) = repmat([237 224 200]/255, nnz(ii), 1);
fgcolors(ii, :) = repmat([119 110 101]/255, nnz(ii), 1);
fontsize(ii) = fontsizeTable(1);
ii = strcmp('8', txt);
bgcolors(ii, :) = repmat([242 178 121]/255, nnz(ii), 1);
fontsize(ii) = fontsizeTable(1);
ii = strcmp('16', txt);
bgcolors(ii, :) = repmat([245 151 100]/255, nnz(ii), 1);

```



```

    fontsize(ii) = fontsizeTable(2);
    ii = strcmp('32', txt);
    bgcolors(ii, :) = repmat([246 124 95]/255, nnz(ii), 1);
    fontsize(ii) = fontsizeTable(2);
    ii = strcmp('64', txt);
    bgcolors(ii, :) = repmat([246 94 59]/255, nnz(ii), 1);
    fontsize(ii) = fontsizeTable(2);
    ii = strcmp('128', txt);
    bgcolors(ii, :) = repmat([237 207 114]/255, nnz(ii), 1);
    fontsize(ii) = fontsizeTable(3);
    ii = strcmp('256', txt);
    bgcolors(ii, :) = repmat([237 204 97]/255, nnz(ii), 1);
    fontsize(ii) = fontsizeTable(3);
    ii = strcmp('512', txt);
    bgcolors(ii, :) = repmat([237 200 80]/255, nnz(ii), 1);
    fontsize(ii) = fontsizeTable(3);
    ii = strcmp('1024', txt);
    bgcolors(ii, :) = repmat([237 197 63]/255, nnz(ii), 1);
    fontsize(ii) = fontsizeTable(4);
    ii = ismember(txt, {'2048', '4096', '8192'}); % 匹配文字为2048,4096,8192的块
    bgcolors(ii, :) = repmat([237 194 46]/255, nnz(ii), 1);
    fontsize(ii) = fontsizeTable(4);
    ii = ismember(txt, {'16384', '32768', '65536'});
    bgcolors(ii, :) = repmat([237 194 46]/255, nnz(ii), 1);
    fontsize(ii) = fontsizeTable(5);
    ii = ismember(txt, {'131072', '262144', '524288'});
    bgcolors(ii, :) = repmat([237 194 46]/255, nnz(ii), 1);
    fontsize(ii) = fontsizeTable(6);
    set([blocks.hBox], {'FaceColor'}, num2cell(bgcolors, 2)); % 设置表面颜色
    set([blocks.hText], {'Color', 'FontSize'}, ...
        [num2cell(fgcolors, 2), num2cell(fontsize)]); % 设置颜色和文字大小
end

% 新建游戏
function newgame(varargin)
global handles
btn = questdlg('退出当前游戏?', '新游戏', '是', '否', '是'); % 提问对话框
if strcmp(btn, '是')
    handles = renew(handles); % 重设对象
end
end

% 重设对象
function handles = renew(handles)
if handles.game.NumMoves ~= 0 % 如果当前游戏移动次数不为0
    handles.Scores(handles.numb).HighScore = handles.game.HighScore; % 最高分数
    handles.Scores(handles.numb).Scores = handles.game.Scores; % 分数
    handles.Scores(handles.numb).NumMoves = handles.game.NumMoves; % 移动次数
    handles.Scores(handles.numb).HighestBlock = handles.game.HighestBlock; % 最大块值
    handles.Best = max(handles.Best, handles.game.HighScore); % 最佳得分
    % 保存数据
    save('game2048', '-struct', 'handles', 'Sound', 'numb', 'Scores', 'game', 'Best');
    handles.numb = handles.numb + 1; % 游戏次数加1
    set(handles.ht, 'Enable', 'on') % 显示菜单栏按钮“历史记录”
end
handles.game = game2048; % 新建游戏数据
handles = updateblocks(handles); % 更新小块
end

```

```

% 炸掉一个选中的小块，置其值为NaN
function bomb(varargin)
global handles
set(handles.hFig,'WindowButtonDownFcn', @buttonDownFcn, ...
    'WindowButtonMotionFcn', @buttonMotionFcn)
end

function buttonDownFcn(varargin)
global handles
if strcmp(get(handles.hFig, 'SelectionType'), 'normal')
    pos = get(handles.hFig, 'CurrentPoint');
    ridx = ceil((410 - pos(2)) / 100);
    cidx = ceil((-10 + pos(1)) / 100);
    if ~isnan(handles.game.Board(ridx,cidx)) && handles.game.NumMoves ~= 0
        hsc = handles.game.Board(ridx,cidx);
        handles.game.HighScore = handles.game.HighScore - hsc;
        handles.game.Board(ridx,cidx) = NaN;
        handles.game.Scores(handles.game.NumMoves) = handles.game.HighScore;
        handles.game.NumMoves = handles.game.NumMoves + 1;
        handles.game.Scores(handles.game.NumMoves) = handles.game.HighScore;
        if handles.sflag, sound(handles.Sound.b,handles.Sound.Fs); end % 播放音乐
        handles.Best = max(handles.game.HighScore,handles.Best); % 更新最佳分数
        txt = cellfun(@num2str, num2cell(handles.game.Board), ...
            'UniformOutput', false); % 将数字转化为文字
        txt(strcmp(txt, 'NaN')) = {' '}; % 将文字“NaN”，转化为“ ”
        setn(handles.blocks, [handles.xPts(:), handles.yPts(:)], txt);
        updateColors(handles.blocks); % 更新小块信息
        set(handles.hScore, 'String', sprintf('Score: %d\nBest: %d', ...
            handles.game.HighScore, handles.Best)); % 显示最高分数与最佳分数
        set(handles.hMoves, 'String', ...
            sprintf('Moves: %d', handles.game.NumMoves)); % 显示移动次数
    end
end

set(handles.hFig,'WindowButtonDownFcn', '', 'WindowButtonMotionFcn', '')
set(handles.hFig, 'Pointer', 'arrow')
end

function buttonMotionFcn(varargin)
global handles
set(handles.hFig, 'Pointer', 'cross')
end

% 更新小块位置和信息
function handles = updateblocks(handles, varargin)
if ishandle(handles.hFig) % 如果游戏界面未被关闭
    if handles.sflag, sound(handles.Sound.y,handles.Sound.Fs); end % 播放音乐
    blocksMoving = (handles.game.Movement ~= reshape(1:16,4,[]));
    if nnz(blocksMoving) > 0 % 若小块有移动
        xFrom = handles.xPts(:); % 起始位置X轴坐标
        yFrom = handles.yPts(:); % 起始位置Y轴坐标
        xTo = handles.xPts(handles.game.Movement(:)); % 终止位置X轴坐标
        yTo = handles.yPts(handles.game.Movement(:)); % 终止位置Y轴坐标
        bringToTop(handles.blocks(blocksMoving)); % 置顶显示
        frms = 10; % 移动步数
        for id = 1:frms
            setn(handles.blocks, [xFrom + id / frms * (xTo - xFrom), ...
                yFrom + id / frms * (yTo - yFrom)], []); % 移动
        end
    end
end

```

```

        if handles.hflag, drawnow expose; end % 若开启动画, 则显示动画效果
    end

    end

    handles.Best = max(handles.game.HighScore, handles.Best); % 更新最佳分数
    txt = cellfun(@num2str, num2cell(handles.game.Board), ...
        'UniformOutput', false); % 将数字转化为文字
    txt(strcmp(txt, 'NaN')) = {' '}; % 将文字“NaN”, 转化为“ ”
    setn(handles.blocks, [handles.xPts(:), handles.yPts(:)], txt);
    updateColors(handles.blocks); % 更新小块信息
    set(handles.hScore, 'String', sprintf('Score: %d\nBest: %d', ...
        handles.game.HighScore, handles.Best)); % 显示最高分数与最佳分数
    set(handles.hMoves, 'String', ...
        sprintf('Moves: %d', handles.game.NumMoves)); % 显示移动次数
end
end

% 设置
function setn(obj, pos, txt)
if ~isempty(pos) && ~isempty(txt)
    set([obj.hBox], {'Position'}, ...
        num2cell([pos - [obj.Width / 2; obj.Height / 2]; ...
            obj.Width; obj.Height]', 2))
    set([obj.hText], {'String', 'Position'}, ...
        [txt(:), num2cell([pos, zeros(size(pos,1),1)], 2)])
elseif isempty(pos) && ~isempty(txt)
    set([obj.hText], {'String'}, txt(:))
elseif ~isempty(pos) && isempty(txt)
    set([obj.hBox], {'Position'}, ...
        num2cell([pos - [obj.Width / 2; obj.Height / 2]; ...
            obj.Width; obj.Height]', 2))
    set([obj.hText], {'Position'}, num2cell([pos, zeros(size(pos,1),1)], 2))
end
end

% 置顶
function bringToTop(obj)
uistack([obj.hBox], 'top');
uistack([obj.hText], 'top');
end

% 按键响应
function keyPressFcn(varargin)
global handles
key = get(handles.hFig, 'CurrentKey'); % 获得按键字符
if any(strcmp(key, ...
    {'escape', 'uparrow', 'downarrow', 'rightarrow', 'leftarrow', ...
    'w', 's', 'a', 'd', 'space'})) % 若是其中之一
    switch key
    case {'uparrow', 'w'}
        handles = movech(handles, 'up'); % 检查游戏是否结束、赢, 是否可移动
    case {'downarrow', 's'}
        handles = movech(handles, 'down');
    case {'rightarrow', 'd'}
        handles = movech(handles, 'right');
    case {'leftarrow', 'a'}
        handles = movech(handles, 'left');
    case 'escape'
        deleteHFig % 关闭主界面
    end
end
end

```

```

        case 'space'
            if handles.pflag == 0, suggest; end % 若已关闭演示, 则提示
        otherwise
            return
        end
    end
end
end

% 移前检查
function handles = movech(handles, direction)
tf = isGameWon(handles.game); % 判断游戏是否胜利
if tf, handles = gameWon(handles); end
tff = isGameOver(handles.game); % 判断游戏是否结束
if tff, handles = gameOver(handles); end
if ~(tf || tff), handles = move(handles, direction); end % 否则, 移动小块
end

% 移动小块
function handles = move(handles, direction)
global iseq
game = handles.game;
movement = reshape(1:16,4,4); % 移动矩阵
orig_movement = movement; % 暂存移动矩阵
emptyID = isnan(game.Board); % 获得不为NaN的下标
% 将四个方向的块矩阵和移动矩阵转化为“up”方向
switch lower(direction)
    case 'up'
        m = game.Board;
    case 'down'
        m = flipud(game.Board);
        movement = flipud(movement);
    case 'right'
        m = flipud(game.Board');
        movement = flipud(movement');
    case 'left'
        m = game.Board';
        movement = movement';
    otherwise
end
thisScore = 0; % 当前分数置为0
for ii = 1:4 % 对四列数据分别处理
    x = nan(4,1); % 预定义块向量
    y = movement(:,ii); % 移动向量
    idx = ~isnan(m(:,ii)); % 获得非NaN的数据下标
    x(1:nnz(idx)) = m(idx,ii); % 移动前的块向量
    y(idx) = movement(1:nnz(idx),ii); % 移动前的移动向量
    sameID = diff(x) == 0; % 判断是否有可以合并的块
    if any(sameID)
        if isequal(sameID, [true; true; true]) || ...
            isequal(sameID, [true; false; true])
            % 判断是否存在形如[2,2,2,2]'或[4,4,2,2]'的结构
            x = [x(1) + x(2); x(3) + x(4); nan; nan]; % 合并
            [a,b] = ismember(y, movement([2 3 4],ii)); % 合并的下标
            c = movement([1 2 2], ii);
            y(a) = c(b(a)); % 合并后的移动向量
            thisScore = thisScore + x(1) + x(2); % 增加合并的得分
        elseif isequal(sameID, [true; true; false]) || ...
            isequal(sameID, [true; false; false])

```

```

        % 判断是否存在形如[2,2,2,4]或[4,4,2,8]的结构
        x = [x(1)+x(2); x(3); x(4); nan]; % 合并
        [a, b] = ismember(y, movement([2 3 4], ii)); % 合并的下标
        c = movement([1 2 3], ii);
        y(a) = c(b(a)); % 合并后的移动向量
        thisScore = thisScore + x(1); % 增加合并的得分
    elseif isequal(sameID, [false; true; true]) || ...
        isequal(sameID, [false; true; false])
        % 判断是否存在形如[2,4,4,4]或[4,2,2,8]的结构
        x = [x(1); x(2) + x(3); x(4); nan]; % 合并
        [a, b] = ismember(y, movement([3 4], ii)); % 合并的下标
        c = movement([2 3], ii);
        y(a) = c(b(a)); % 合并后的移动向量
        thisScore = thisScore + x(2); % 增加合并的得分
    elseif isequal(sameID, [false; false; true])
        % 判断是否存在形如[4,2,8,8]的结构
        x = [x(1); x(2); x(3) + x(4); nan]; % 合并
        [a, b] = ismember(y, movement(4, ii)); % 合并的下标
        c = movement(3, ii);
        y(a) = c(b(a)); % 合并后的移动向量
        thisScore = thisScore + x(3); % 增加合并的得分
    else
        error('Should not be here!')
    end
end
m(:,ii) = x; % 合并后的块矩阵
movement(:,ii) = y; % 合并后的移动矩阵
end
% 将四个方向的块矩阵和移动矩阵转化为原来的方向
switch direction
case 'up'
case 'down'
    m = flipud(m);
    movement = flipud(movement);
case 'right'
    m = flipud(m)';
    movement = flipud(movement)';
case 'left'
    m = m';
    movement = movement';
end
% 更新最高分数
if max(game.Scores) + thisScore > game.HighScore
    game.HighScore = max(game.Scores) + thisScore;
end
% 若合并后的块矩阵与原块矩阵不同, 即出现合并时
if ~iseq(game.Board, m)
    game.NumMoves = game.NumMoves + 1; % 移动次数加1
    if game.NumMoves > length(game.Scores)
        game.Scores = [game.Scores, nan(1,1000)]; % 扩展分数
    end
    game.Scores(game.NumMoves) = max(game.Scores) + thisScore; % 记录分数
    game.Board = m; % 记录块矩阵
    movement(emptyID) = orig_movement(emptyID); % 更新移动矩阵
    game.Movement = movement; % 记录移动矩阵
    game.Board = addNewBlock(game); % 添加一个新块
    game.HighestBlock = max(max(game.Board)); % 更新最大块值
    handles.game = game; % 更新游戏数据
end

```

```

        handles      = updateblocks(handles);          % 更新块
    else
        handles.game.Movement = reshape(1:16,4,4);
    end
end

% 判断游戏结束否
function tf = isGameOver(game)
% 没有空白块且无法合并
tf = nnz(isnan(game.Board)) == 0 && ...
    nnz(diff(game.Board,1,1) == 0) == 0 && ...
    nnz(diff(game.Board,1,2) == 0) == 0;
end

% 判断你赢否
function tf = isGameWon(game)
tf = nnz(game.Board == game.StopNumber) > 0;    % 达到停止数
end

% 游戏结束
function handles = gameOver(handles)
btn = questdlg({'你已经无法移动了! ', ' ', '是否再玩一次? '}, ...
    '游戏结束', '新游戏', '退出', '新游戏');
switch btn
    case '新游戏'
        handles = renew(handles);
    case '退出'
        deleteHFig
end
end

% 游戏胜利
function handles = gameWon(handles)
btn = questdlg({'恭喜你赢了! ', ' ', '是否继续游戏? '}, ...
    '胜利', '继续玩', '新游戏', '退出', '继续玩');
switch btn
    case '继续玩'
        handles.game.StopNumber = 2 * handles.game.StopNumber; % 停止数翻倍
    case '新游戏'
        handles = renew(handles);
    case '退出'
        save('game2048', '-struct', 'handles', 'Sound', 'numb', ...
            'Scores', 'game', 'Best');    % 保存游戏数据
        delete(handles.hFig)            % 删除主界面
end
end

% 清空分数
function clearScores(varargin)
global handles
handles.Scores      = {};
handles.numb        = 1; % 游戏次数置为1
handles.Scores(1).HighScore = 0; % 当前游戏的最高分数置为0
handles.Scores(1).Scores = 0; % 游戏的分数置为0
handles.Scores(1).NumMoves = 0; % 移动次数置为0
handles.Scores(1).HighestBlock = 0; % 最大块值置为0
set(handles.ht, 'Enable', 'off')    % 隐藏“历史记录”按钮
end

```

```

% 清空最佳分数
function clearBest(varargin)
global handles
handles.Best = 0; % 最佳分数重置为0
handles.Best = max(handles.Best, handles.game.HighScore); % 与当前最佳分数比较
set(handles.hScore, 'String', sprintf('Score: %d\nBest: %d', ...
    handles.game.HighScore, handles.Best)); % 显示最高分数与最佳分数
end

% 关闭游戏
function deleteHFig(varargin)
global handles
handles.pflag = 0;
% 保存游戏数据, 并删除主界面
save('game2048', '-struct', 'handles', 'Sound', 'numb', 'Scores', 'game', 'Best');
delete(handles.hFig)
end

% 历史记录
function histgame(varargin)
global handles h
if handles.numb == 1
    set(handles.ht, 'Enable', 'off'); % 若游戏次数为1, 则无历史记录可查
else
    hn = figure('Units', 'Pixels', ... % 单位为像素
        'Position', get(handles.hFig, 'Position'), ... % 位置与主界面相同
        'Resize', 'off', ... % 大小固定, 不可改变
        'ToolBar', 'none', ... % 无菜单栏工具栏
        'Menu', 'none');
    hna = axes('Parent', hn, ... % 在hn上新建坐标轴
        'Units', 'Pixels', ...
        'Position', get(handles.hAx, 'Position') - [-55, -200, 90, 150]);
    c = {'k', 'b', 'g', 'r', 'c', 'm', 'y'}; % 颜色字符串
    for i = 1:handles.numb - 1 % 逐次绘图
        if handles.Scores(i).NumMoves % 若移动次数不为0
            h{i} = plot(hna, 1:handles.Scores(i).NumMoves, ...
                handles.Scores(i).Scores(1:handles.Scores(i).NumMoves), ...
                'Color', char(c(mod(i,7) + 1)));
        else % 否则绘制[0,0]
            h{i} = plot(hna, 0, 0, 'Color', char(c(mod(i,7) + 1)));
        end
    end
    hold on
end
grid on
xlabel('Parent', hna, '步数')
ylabel('Parent', hna, '得分')
t1 = uitable('Parent', hn, 'tag', 't1', 'Data', ...
    [handles.Scores(1:handles.numb-1).HighScore; ...
    handles.Scores(1:handles.numb-1).NumMoves; ...
    handles.Scores(1:handles.numb-1).HighestBlock]', ...
    'ColumnName', {'分数', '步数', '最大格值'}, ...
    'Position', get(handles.hAx, 'Position') - [-65, -10, 120, 260], ...
    'ColumnEditable', [false false false], ...
    'FontSize', 10); % 新建数据表格
set(t1, 'CellSelectionCallback', @cellSelection); % 设置选中响应函数
end
end

```

```

function cellSelection(~, eventdata, varargin)
global h
ind = eventdata.Indices(1);          % 获取选中表格的行标
set(h{ind}, 'lineWidth', 2)          % 将第ind条线的线宽设为2
pause(2)                             % 暂停2s
set(h{ind}, 'lineWidth', 1)          % 将第ind条线的线宽设为1
end

% 当前统计, 与历史统计类似
function nowgame(varargin)
global handles
hn = figure('Units', 'Pixels', ...
    'Position', get(handles.hFig, 'Position'), ...
    'Resize', 'off', ...
    'Toolbar', 'none', ...
    'Menu', 'none');
hna = axes('Parent', hn, ...
    'Units', 'Pixels', ...
    'NextPlot', 'add', ...
    'Position', get(handles.hAx, 'Position') - [-60,-110,90,70]);
if handles.game.NumMoves
    plot(hna, handles.game.Scores(1:handles.game.NumMoves))
    xlim(hna, [-1, handles.game.NumMoves])
end
xlabel('Parent', hna, '步数')
ylabel('Parent', hna, '得分')
t1 = uitable('Parent', hn, 'Data', ...
    [handles.game.HighScore, handles.game.NumMoves, ...
    handles.game.HighestBlock], ...
    'ColumnName', {'分数', '步数', '最大格值'}, ...
    'Position', get(handles.hAx, 'Position') - [-70,-5,140,345]);
set(t1, 'FontSize', 10, 'ColumnEditable', [false false false])
end

% 保存截图
function SaveScreenshot(varargin)
global handles
[file, path] = uiputfile({'*.png'; '*.bmp'; '*.gif'; '*.jpg'; '*.jpeg'; '*.tiff'}, ...
    '保存游戏截图', ['Game_', datestr(now, 30), '.png']);
if ~isequal(file, 0) && ~isequal(path, 0)
    im = getframe(handles.hFig);
    imwrite(im.cdata, [path, file])
end
end

% 帮助信息
function mhelp(varargin)
h = msgbox({'规则: ', ...
    'UUU用←↑↓→或WASD键移动小块, 当两个      ', ...
    'UUU小块相等时可以合并, 某小块达到2048时, ', ...
    'UUU您就赢了, 当无法移动小块时您就输了。    ', ...
    '提示: ', ...
    'UUU1.按Esc键关闭界面, 重新打开继续游戏;  ', ...
    'UUU2.游戏菜单下可查看当前统计和历史记录; ', ...
    'UUU3.设置菜单下可设置动画、声音、颜色等; ', ...
    'UUU4.按''清空历史记录''可清空历史数据;    ', ...
    'UUU5.按''空格键''将会得到最佳提示。      ', 'U', 'U'}, ...

```



```

    '帮助...', 'modal');          % 新建信息对话框
    uiwait(h)                     % 等待响应
end

% 发送信息
function sm(varargin)
prompt = {'请输入您的联系方式: ', '请输入留言: '};
name    = '留言';
numlines = [1; 4];
answer = inputdlg(prompt, name, numlines); % 新建输入对话框
if isempty(answer)                  % 选择取消
    uiwait(warndlg('您的留言已取消发送! ', '提示...'));
elseif isempty(answer{2})          % 未输入留言
    uiwait(warndlg('留言为空, 已取消发送! ', '提示...'));
else
    try
        a = char(answer{2}');
        m = a(:)';                  % 将留言信息转化为一行文字
        message = [answer{1}, '留言:', m]; % 组成留言格式
        sendm(message)              % 发送信息函数sendm.p, 详见第二十四章
        if isempty(answer{1})       % 若联系方式为空
            uiwait(msgbox('联系方式为空, 已匿名发送! ', '提示...'));
        else
            uiwait(msgbox('您的留言已发送! ', '提示...'));
        end
    catch exception
        if strcmp(exception.identifier, 'MATLAB:sendmail:SmtpError') % 发送有误
            uiwait(msgbox('发送失败! 请手动发送邮件至mmatlab@sina.com!', '提示...'));
        end
    end
end

% 关于
function about(varargin)
uiwait(msgbox({'基于MATLAB的 2048 游戏', ...
    '版本: MATLAB R2015a', ...
    '编者: 王保山', ...
    '时间: 2015年5月24日'}, ...
    '关于...', 'Help', 'modal'))); % 版本信息
end

% 提示
function suggest(varargin)
global handles
board = handles.game.Board;
depth = 3 + sum(sum(isnan(board(:))) < [3 7]); % 搜索深度
try
    direction = feval(handles.aifun, board, depth); % 获得最佳的移动方向
    handles = movech(handles, char(direction)); % 按方向direction移动
catch exception
    if strcmp(exception.identifier, 'MATLAB:UndefinedFunction') % 文件未找到
        errorldg('AI演示&帮助程序未找到! ', '提示');
        demoOff % 关闭演示
    end
    if strcmp(exception.identifier, 'MATLAB:nomem') % 内存不足
        direction = feval(handles.aifun, board, depth-1); % 深度-1
        disp('MATLAB:nomem')
    end
end

```

```

        handles = movech(handles, char(direction)); % 按方向direction移动
    end
end
end

% 高级AI算法
function aiHigh(varargin)
global handles
handles.aifun = @Oliver_AI_v2;
end

% 中级AI算法
function aiMid(varargin)
global handles
handles.aifun = @Athi_AI_v1;
end

% 初级AI算法
function aiLow(varargin)
global handles
handles.aifun = @Random_AI;
end

% 自定义AI算法
function aiCustom(varargin)
global handles
[filename, pathname] = uigetfile({'*.m'}, '选择AI文件');
if filename == 0, return; end
id = strfind(filename, '.m');
if isempty(id), uiwait(errordlg('AI程序需为.m文件! ', '提示')); return; end
if ~strcmp([pwd, '\'], pathname)
    [status, msg] = copyfile([pathname, filename], [pwd, '\', filename]);
    if status == 0, uiwait(errordlg(msg, '提示')); return; end
end

funname = filename(1:id-1);
aifun = str2func(funname);
handles.aifun = aifun;
end

% 开启动画
function animOn(varargin)
global handles iconsData
handles.hflag = 1;
set(handles.uitbar.uitb1, 'State', 'on' )
if ~isempty(iconsData)
    set(handles.uitbar.uitb1, 'CData', iconsData.AnimOn)
end
end

% 关闭动画
function animOff(varargin)
global handles iconsData
handles.hflag = 0;
set(handles.uitbar.uitb1, 'State', 'off')
if ~isempty(iconsData)
    set(handles.uitbar.uitb1, 'CData', iconsData.AnimOff)
end
end

```

```

end

% 开启声音
function soundOn(varargin)
global handles iconsData
handles.sflag = 1;
set(handles.uitbar.uitb2, 'State', 'on' )
if ~isempty/iconsData)
    set(handles.uitbar.uitb2, 'CData', iconsData.SoundOn)
end
end

% 关闭声音
function soundOff(varargin)
global handles iconsData
handles.sflag = 0;
set(handles.uitbar.uitb2, 'State', 'off' )
if ~isempty/iconsData)
    set(handles.uitbar.uitb2, 'CData', iconsData.SoundOff)
end
end

% 开启演示
function demoOn(varargin)
global handles iconsData
handles.pflag = 1;
set(handles.uitbar.uitb3, 'State', 'on')
if ~isempty/iconsData)
    set(handles.uitbar.uitb3, 'CData', iconsData.AIOn)
end
Demo
end

% 关闭演示
function demoOff(varargin)
global handles iconsData
handles.pflag = 0;
set(handles.uitbar.uitb3, 'State', 'off')
if ~isempty/iconsData)
    set(handles.uitbar.uitb3, 'CData', iconsData.AIOff)
end
end

% 演示
function Demo
global handles
n = 1; % 演示次数
prompt = sprintf('输入演示次数: ');
name = '演示设置';
answer = inputdlg(prompt, name);
N = 0; % 将最大演示次数初设为0
if ~isempty(answer), N = str2double(answer); end
ind = 1; % 游戏未胜利且未结束
while n <= N && handles.pflag && ind
    ind = ~(isGameWon(handles.game) || isGameOver(handles.game));
    suggest % 提示并移动
    n = n + 1; % 演示次数加1
    pause(0.1) % 暂停0.1s
end

```

```

end
demoOff          % 演示结束，自动关闭演示
end

```

下面给出MathWorks的File Exchange中发布的三个2048游戏的AI程序，近期的研究表明，合理的AI策略可以达到4096、8192，甚至可能达到16384、32768，有兴趣的读者可以进一步深入研究：

1. 初级算法——随机算法²，
2. 中级算法——Athi算法³，
3. 高级算法——Oliver算法⁴。

```

function direction = Random_AI(varargin)
% 随机算法
direction = {'up', 'down', 'right', 'left'};
direction = direction(randi(4));
end

```

```

function direction = Athi_AI_v1(board, depth)
% Athi算法
persistent score
if isempty(score), score = 0; end
board(isnan(board)) = 0;          % 将矩阵中等于NaN的元素置为0，便于计算
d = {'up', 'right', 'down', 'left'}; % 方向的集合
hint = findBestMove(board, score, depth); % 搜索最佳的方向
[newBoard, newScore] = move(board, score, hint); % 试移动一次
score = newScore;                  % 更新score
if isequal(newBoard, board)        % 若为产生合并
    switch (hint)
        case 1
            newMat = [2, 3, 4];
        case 2
            newMat = [1, 3, 4];
        case 3
            newMat = [1, 2, 4];
        case 4
            newMat = [1, 2, 3];
    end
    hint = newMat(randi(3));        % 随机赋予hint一个值
end

direction = d(hint);               % 最终的方向
end

function direction = findBestMove(board, score, depth)

```

²下载网址: <http://www.mathworks.com/matlabcentral/fileexchange/46124-2048-matlab-edition>

³下载网址: <http://www.mathworks.com/matlabcentral/fileexchange/46483-2048-game-solver>

⁴下载网址: http://blogs.mathworks.com/pick/2014/04/04/submit-your-algorithms-to-solve-2048/#view_comments

```

minVal = -2^32; % 初始最小值
maxVal = 2^32; % 初始最大值
player = 1; % 表示用户, 0表示对手
[direction, ~] = alphaBeta(board, score, depth, minVal, maxVal, player);
end

function [direction, outScore] = alphaBeta(board, score, depth, alpha, beta, player)
bestDirection = 1; % 初始最佳方向
if isGameTerminated(board) % 若游戏结束
    bestScore = min([score, 1]);
elseif depth == 0 % 若深度为0, 计算得分
    bestScore = heuristicScore(score, getNumberOfEmptyCells(board), ...
        calculateClusteringScore(board));
else
    if player
        for dirIdx = 1 : 4
            newBoard = board;
            newScore = score;
            [newBoard, newScore] = move(newBoard, newScore, dirIdx);
            if isequal(newBoard, board), continue; end
            [~, currentScore] = alphaBeta(newBoard, ...
                newScore, depth-1, alpha, beta, 0);
            if currentScore > alpha
                alpha = currentScore;
                bestDirection = dirIdx;
            end
            if beta <= alpha, break; end
        end
        bestScore = alpha;
    else
        [row, col] = getEmptyCellIds(board);
        possibleValues = [2, 4];
        len = length(row);
        for idx = 1 : len
            i = row(idx);
            j = col(idx);
            len2 = length(possibleValues);
            for idx2 = 1 : len2
                value = possibleValues(idx2);
                newBoard = board;
                newScore = score;
                newBoard(i, j) = value;
                [~, currentScore] = alphaBeta(newBoard, ...
                    newScore, depth-1, alpha, beta, 1); % User
                if currentScore < beta, beta = currentScore; end
                if beta <= alpha, break; end
            end
            if beta <= alpha, break; end
        end
        bestScore = beta;
        if len == 0, bestScore = 0; end
    end
end
outScore = bestScore;
direction = bestDirection;
end

```

```

function hScore = heuristicScore(actualScore, numberOfEmptyCells, clusteringScore)
tempScore = actualScore + log(actualScore) * numberOfEmptyCells - clusteringScore;
tempScore = floor(tempScore);
hScore = max([tempScore, min([actualScore, 1])]);
end

function [board, score] = move(board, score, dirIdx)
BOARD_SIZE = 4;
points = 0;
if dirIdx == 1, board = rotateLeft(board);
elseif dirIdx == 2, board = board(:,end:-1:1);
elseif dirIdx == 3, board = rotateRight(board);
end
for i = 1 : BOARD_SIZE
    lastMergePosition = 1;
    for j = 2 : BOARD_SIZE
        if board(i,j)==0, continue; end
        previousPosition = j-1;
        while (previousPosition>lastMergePosition) && ...
            (board(i,previousPosition)==0)
            previousPosition = previousPosition - 1;
        end
        if previousPosition == j
            elseif board(i,previousPosition) == 0
                board(i,previousPosition) = board(i,j);
                board(i,j) = 0;
            elseif board(i,previousPosition) == board(i,j)
                board(i,previousPosition) = 2 * board(i,previousPosition);
                board(i,j) = 0;
                points = points + board(i,previousPosition);
                lastMergePosition = previousPosition+1;
            elseif board(i,previousPosition) ~= board(i,j) && ...
                (previousPosition+1) ~= j
                board(i,(previousPosition+1)) = board(i,j);
                board(i,j) = 0;
            end
        end
    end
    score = score + points;
    if dirIdx == 1, board = rotateRight(board);
    elseif dirIdx == 2, board = board(:,end:-1:1);
    elseif dirIdx == 3, board = rotateLeft(board);
    end
end

function [sortedRow, sortedCol] = getEmptyCellIds(board)
[row, col] = find((board == 0));
[sortedRow, idx] = sort(row, 'ascend');
sortedCol = col(idx);
end

function count = getNumberOfEmptyCells(board)
count = sum(sum(board == 0));
end

function clusteringScore = calculateClusteringScore(board)

```

```

sz = size(board);
clusteringScore = 0;
neighbors = [-1,0,1];
for i = 1:sz(1)
    for j = 1:sz(2)
        if board(i,j) == 0,continue;end
        numOfNeighbors = 0;
        s = 0;
        len = length(neighbors);
        for neiIdx1 = 1:len
            k = neighbors(neiIdx1);
            x = i + k;
            if (x<1) || (x>sz(1)),continue;end
            for neiIdx2 = 1:len
                l = neighbors(neiIdx2);
                y = j + l;
                if (y<1) || (y>sz(2)),continue;end
                if board(x,y)>0
                    numOfNeighbors = numOfNeighbors + 1;
                    s = s + abs(board(i,j)-board(x,y));
                end
            end
        end
        clusteringScore = clusteringScore + floor(s/numOfNeighbors);
    end
end

function terminated = isGameTerminated(board)
terminated = getNumberOfEmptyCells(board) == 0 && ...
    nnz(diff(board,1,1) == 0) == 0 && ...
    nnz(diff(board,1,2) == 0) == 0;
end

function newBoard = rotateRight(board)
BOARD_SIZE = 4;
newBoard = zeros(BOARD_SIZE, BOARD_SIZE);
for i = 1 : BOARD_SIZE
    for j = 1 : BOARD_SIZE
        newBoard(i,j) = board(BOARD_SIZE-j+1, i);
    end
end

function newBoard = rotateLeft(board)
BOARD_SIZE = 4;
newBoard = zeros(BOARD_SIZE, BOARD_SIZE);
for i = 1 : BOARD_SIZE
    for j = 1 : BOARD_SIZE
        newBoard(BOARD_SIZE-j+1, i) = board(i,j);
    end
end

```

```

function d = Oliver_AI_v2(B, depth)
% Oliver算法
% Convert to log board
B(isnan(B)) = 1;
B = uint16(log2(B));

% Slide in each direction
d = [];
s = [];
A = zeros(4, 4, 0, 'uint16');
for d_ = 1:4
    [C, s_] = slide(B, d_);
    if ~isequal(B, C)
        A = cat(3, A, C);
        d = [d d_];
        s = [s; s_];
    end
end
B = A;
p = ones(numel(d), 1);

for a = 2:depth
    % For each board in B, consider all possible positions and values for
    % the new tile
    B = reshape(B, 16, []);
    N = sum(B == 0, 1); % Number of free squares
    I = zeros(sum(N)*2, 1);
    i = 0;
    % Expand the boards array to hold all possible examples
    for b = find(N)
        I(i+1:i+N(b)*2) = b;
        i = i + N(b) * 2;
    end
    B = B(:,I);
    d = d(:,I);
    p = p(I);
    s = s(I);
    i = 0;
    % For each board
    for b = find(N)
        j = find(B(:,i+1) == 0);
        % Add in the new 2s
        for c = 1:N(b)
            p(c+i) = p(c+i) * (0.9 / N(b));
            B(j(c),c+i) = 1;
        end
        i = i + N(b);
        % Add in the new 4s
        for c = 1:N(b)
            p(c+i) = p(c+i) * (0.1 / N(b));
            B(j(c),c+i) = 2;
        end
        i = i + N(b);
    end
    B = reshape(B, 4, 4, []);
    % Slide all the boards in each direction
    A = zeros(4, 4, 0, 'uint16');
    d_ = zeros(a, 0);

```



```

p_ = [];
s_ = [];
for d_ = 1:4
    [C, s_] = slide(B, d_);
    M = any(reshape(B ~= C, 16, []));
    A = cat(3, A, C(:, :, M));
    p_ = [p_; p(M)];
    s_ = [s_; s(M)+s_(M)];
    d_ = [d_ [d(:, M); zeros(1, sum(M))+d_]];
end
B = A;
p = p_;
s = s_;
d = d_;
end

% Compute the expected gain in score for a each set of moves and maximize
% over this
[s_, d_] = max(accumarray(((4 .^ (0:depth-1)) * (d-1))'+1, double(s(:)) .* p(:), [4^depth 1]));
d_ = mod(d_ - 1, 4) + 1;
% Maximize the expected number of zeros after the moves
% [d, d] = max(accumarray(d(1,:)', sum(reshape(B, 16, []) == 0, 1)' .* p(:), [4 1]));

% Output the string for the direction
d = {'up', 'down', 'right', 'left'};
d = d{d_};
end

function [B, s] = slide(B, dir)
persistent lut lutf luts
if isempty(lut)
    % Precompute the fast lookup for sliding up
    [a, b, c, d] = ndgrid(0:13);
    a = [a(:)'; b(:)'; c(:)'; d(:)'];
    lut = zeros(4, size(a, 2), 'uint16'); % Compress for better use of cache
    luts = zeros(size(a, 2), 1, 'uint16'); % Compress for better use of cache
    s_ = 2 .^ (1:14);
    % Compute the slide manually
    for b = 1:size(a, 2)
        d = a(:, b);
        d = d(d ~= 0);
        s = 0;
        for c = 1:numel(d)-1
            if d(c) == d(c+1) && d(c)
                d(c) = d(c) + 1;
                s = s + s_(d(c));
                d(c+1:end) = [d(c+2:end); 0];
            end
        end
        lut(1:numel(d), b) = d;
        luts(b) = s;
    end
    lutf = flipud(lut); % Make a flipped table too
end

% Do the slide using fast table look up
switch dir
case 1
    I = sum(bsxfun(@times, uint16([1 14 196 2744]')), B), 1, 'native') + uint16(1);
    B = reshape(lut(:, I), 4, 4, []);

```

```

case 2
    I = sum(bsxfun(@times, uint16([2744 196 14 1]'), B), 1, 'native') + uint16(1);
    B = reshape(lutf(:,I), 4, 4, []);
case 3
    I = sum(bsxfun(@times, uint16([2744 196 14 1]), B), 2, 'native') + uint16(1);
    B = permute(reshape(lutf(:,I), 4, 4, []), [2 1 3]);
case 4
    I = sum(bsxfun(@times, uint16([1 14 196 2744]), B), 2, 'native') + uint16(1);
    B = permute(reshape(lut(:,I), 4, 4, []), [2 1 3]);
end
if nargin > 1
    s = sum(reshape(luts(I), 4, []), 1, 'native')';
end
end

```

注 22.1 2048游戏说明书

1. 通过按键(W, A, S, D或方向键)控制小块的移动方向, 当有两个小块相同时, 把两者合并, 产生一个新块(其值90%为2, 10%为4), 给予一定的分数, 并将分数记录在数据文件中, 显示在界面上。
2. 当最大块的值达到2048时, 您将获得胜利, 胜利后您可以选择继续游戏(当最大块的值为4096时, 您将获得连胜, 依次类推), 或者退出游戏(下次游戏时从退出时的界面开始)。
3. 如果小块不能再移动, 您将输掉该游戏, 此时, 您可以选择新游戏或者退出游戏(下次游戏时重新开始)。
4. 游戏过程中, 您都有权停止游戏(按Esc键或者直接关闭界面), 下次打开该游戏时将回到本次停止的状态, 以便您继续游戏, 当然你也可以随时新建游戏。
5. 该游戏为了增加游戏的观赏性, 在工具栏内特为您增加了几项设置, 如新建游戏、游戏界面截图、动画、声音、演示、游戏得分记录等, 您可以按照您的喜好进行设置。
6. 游戏还增加了数据分析的功能, 您可以查看当前统计, 也可以查看历史记录(只要您没有删除他们)。
7. 为了降低游戏难度, 本游戏还提供了提示功能, 按“空格”键即可提示, 但连续按下空格键的时间间隔不得少于1s, 否则会卡住, 影响娱乐效果。
8. 游戏还提供了AI演示功能, 您可以设置电脑演示步数(AI算法完成2048大概需要960步), 让电脑替您玩游戏。游戏提供了初级、中级和高級的AI算法, 您可以通过设置AI算法自由选择, 或者通过设置AI算法的“自定义”按钮将自行编写AI算法(自编算法请参照Athi算法的输入输出规则)导入游戏。
9. 由于AI提示程序在个别情况下无法得出最佳的移动策略, 在这些情况下, 您可以选择工具栏中“炸弹”按钮, 用鼠标左键选中一个块将其炸掉(您无法在游戏开始时炸掉界面内唯一的块), 然后继续您的游戏, 当您想取消炸弹时, 可以在游戏界面中点击鼠标右键。作为炸掉有数字的块

的惩罚，您将损失该块的分数。原则上您可以炸毁界面内所有的数字块，但若界面内的块均被您炸掉，则游戏终止(除非您再次新建游戏)。

10. 点击“炸弹”按钮后，您也可以继续您的游戏，移动几步后再进行炸毁小块，比如AI演示时进行炸毁操作。如果该小块没有数值，则炸毁无效，不扣分。若您出现无法移动的情况，在您使用“炸弹”按钮前，请不要在弹出的对话框中选择“新游戏”或“退出”，直接点击对话框红色“×”号关闭对话框后再使用炸毁功能，否则游戏将结束或关闭，而无法继续该局游戏。
11. 在您进行游戏之前，还请您注意版本之间的区别。
12. 如果您有不明白的地方可以查看帮助，或者给我们留言；若程序出现错误请直接发送邮件到编者邮箱，谢谢！
13. 祝您游戏愉快！

我的 MATLAB 报告

王保山

第二十三章 GUI设计

23.1 GUIDE

MATLAB提供了一套可视化地创建图形窗口的工具，使用用户界面开发环境可方便的创建GUI应用程序，它可以根据用户设计的GUI布局，自动生成M文件的框架，用户使用这一框架编制自己的应用程序。

在MATLAB中，GUI的设计是以M文件的编程形式实现的，GUI的布局代码存储在fig文件中，在M文件中并不包含GUI的布局代码，因此在开发应用程序时代码量大大减少。

在命令窗口输入guide命令或点击工具栏中的guide图标都可以打开空白的布局编辑器，在命令窗口输入guide filename可打开一个已存在的名为filename图形用户界面。

```
>> guide % 结果如图 23.1
```

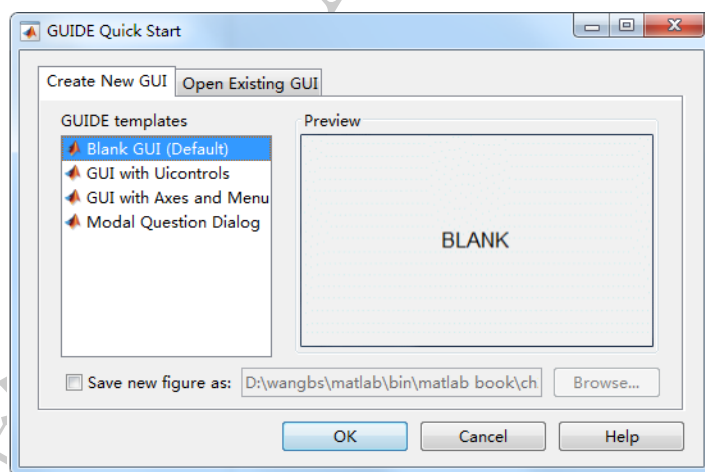


图 23.1: GUIDE的开始界面

点击“OK”按钮，我们就可以进入默认的GUI设计界面如图 23.2。如果新建的GUI界面要求带有坐标轴(菜单或者对话框)，则可以选择其他界面。

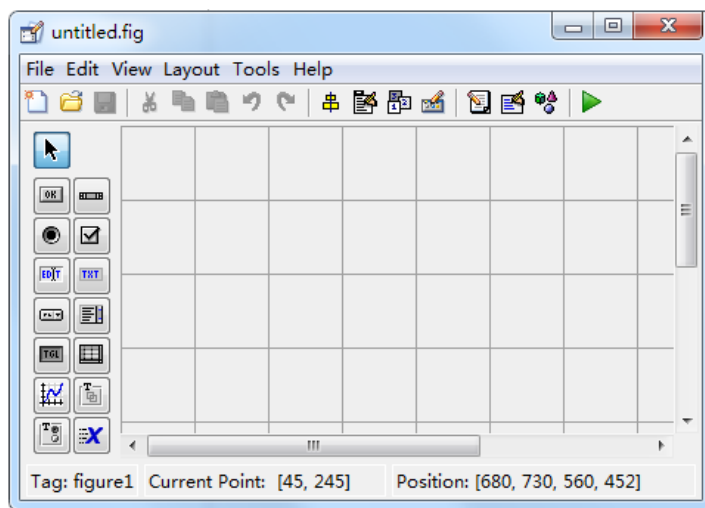


图 23.2: 默认的GUI设计界面

界面左侧为各种控件对象，鼠标在控件对象上驻留一段时间后，会出现控件对象的相关提示，选中所需控件并拖动到布局区(图中网格区域)，便可创建对应的控件，大小可由鼠标左键拖动其边框改变，也可双击控件，在弹出的属性查看器(Inspector)中修改Position参数值，其他属性的修改类似。

依照前面的描述，读者可以尝试实现QQ密码登录框布局(详细的界面设计会在第3节中给出)。

布局编辑器(Layout Editor)，是可以启动用户界面的控制面板，借助它可在图形窗口中加入及安排对象。

(1) 将控件对象放置到布局区。(a) 用鼠标选择并放置控件到布局区内；(b) 移动控件到适当的位置；(c) 改变控件的大小；(d) 选中多个对象等。

(2) 激活图形窗口。选“Tools”菜单中的“Run”项或点击工具栏上的“Run Figure”按钮，激活图形窗口。若所建立的布局还没有进行存储，用户界面开发环境将打开一个“Save As”对话框，按输入的文件名，存储一对同名的M文件和带有.fig扩展名的FIG文件，待文件保存后再激活图形窗口。

(3) 运行GUI程序。在命令窗口键入GUI文件名或用openfig，open或hgload命令运行GUI程序。

(4) 布局编辑器参数设置。选“File”菜单下的“Preferences”菜单项打开参数设置窗口，点击树状目录中的GUIDE，既可设置布局编辑器的参数。

(5) 布局编辑器的弹出菜单。在任一控件上按下鼠标右键，会弹出一个菜单，通过该菜单可以完成布局编辑器的大部分操作。

当界面相对复杂时，我们可以借助于工具栏上的按钮实现对各个控件的管理和编辑，如图23.3，由左及右，分别为几何位置排列工具(Align Objects)、菜单编辑器(Menu Editor)、顺序编辑器(Tab Order Editor)、工具栏编辑器(Toolbar Editor)、函数编辑器(Editor)、属性查看器(Property Inspector)、对象浏览器(Object Browser)和运行按钮(Run Figure)。



图 23.3: 工具栏上的控件管理按钮

表 23.1: 控件管理与编辑工具

工具	描述
几何位置排列工具	调节各控件对象之间的相对位置
菜单编辑器	菜单的设计和编辑, 可以利用它任意添加或删除菜单, 设置菜单项的属性, 包括名称(Label)、标识(Tag)、选择是否显示分隔线(Separator above this item)、是否在菜单前加上选中标记(Item is checked)、调用函数(Callback)
顺序编辑器	调节各控件对象之间的上下位置
工具栏编辑器	添加或删除工具栏按钮, 并设置其属性
函数编辑器	编写GUI界面的M文件, 如回调函数(Callback)等
属性查看器	查看和修改控件属性, 如布置控件, 定义文本框的属性, 定义坐标系, 定义按钮属性, 定义复选框等
对象浏览器	浏览当前程序所使用的全部对象信息, 可以在对象浏览器中选中一个或多个控件来打开属性编辑器

23.2 预定义对话框

在图形用户界面程序设计中, 对话框是重要的信息显示和获取输入数据的用户界面对象。MATLAB中预定义的对话框详见表 23.2。

表 23.2: 预定义对话框

函数	描述	函数	描述	函数	描述
uigetfile	文件打开对话框	uiputfile	文件保存对话框	uigetdir	目录选择对话框
uisetcolor	颜色设置对话框	uisetfont	字体设置对话框	listdlg	列表选择对话框
psgesetupdlg	打印设置对话框	printpreview	打印预览对话框	printdlg	打印对话框
menu	菜单选择对话框	dialog	普通对话框	waitbar	进度条
errordlg	错误对话框	warndlg	警告对话框	questdlg	提问对话框
helpdlg	帮助对话框	msgbox	信息对话框	inputdlg	输入对话框

23.2.1 文件打开对话框

文件打开对话框用于获取打开文件的路径和文件名，常用格式如下：

```
[FileName, PathName] = uigetfile
[FileName, PathName] = uigetfile(FilterSpec)
[FileName, PathName] = uigetfile(FilterSpec, Title)
[FileName, PathName] = uigetfile(FilterSpec, Title, File)
[... , FilterIndex] = uigetfile(...)
```

注 23.1 参数意义为：输入参数`FilterSpec`指定文件的后缀名，`Title`指定对话框的标题，`File`指定默认的文件名，输出参数`FlieName`表示选中的文件名，`PathName`表示选中的文件路径，`FilterIndex`表示选中文件的后缀名索引值。如果点击“取消”或“关闭”按钮，则所有返回值均为0。

例 23.1 文件打开对话框

```
[filename, pathname] = uigetfile({'*.m'; '*.fig'; '*.*'}, '选择文件'); % 结果如图 23.4
```

23.2.2 文件保存对话框

文件保存对话框同样用于获取打开文件的路径和文件名，常用格式与`uigetfile`函数类似，如下：

```
[FileName, PathName] = uiputfile
[FileName, PathName] = uiputfile(FilterSpec)
[FileName, PathName] = uiputfile(FilterSpec, Title)
[FileName, PathName] = uiputfile(FilterSpec, Title, File)
[... , FilterIndex] = uiputfile(...)
```

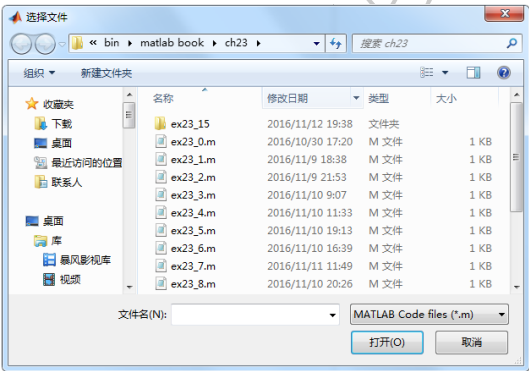


图 23.4: 文件打开对话框

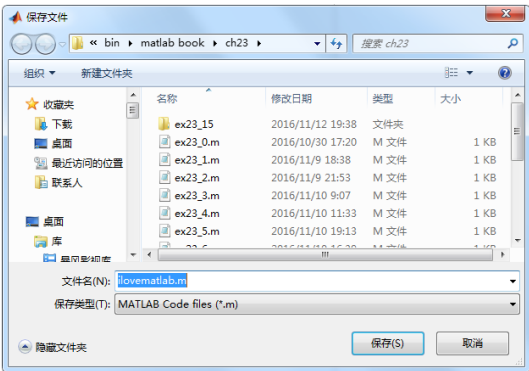


图 23.5: 文件保存对话框

注 23.2 `uiputfile`函数的参数意义与igetfile函数相同，但需要注意的是uiputfile与uigetfile并不具备打开和保存文件的功能，它们只是获取文件的路径和名称，具体的打开和保存详见本书第四章内容。

例 23.2 文件打开对话框

```
[filename, pathname] = uiputfile('ilovematlab.m', '保存文件'); % 结果如图 23.5
```

23.2.3 颜色设置对话框

颜色设置对话框用于图形对象颜色的交互设置。

```
C = uisetcolor
C = uisetcolor([R G B])
C = uisetcolor(H)
C = uisetcolor(..., DialogTitle)
```

注 23.3 参数意义为：输入参数R G B指定默认的颜色向量，每个分量在0和1之间，H指定对象句柄，DialogTitle指定对话框标题，输出参数C表示选中的颜色向量。如果点击“取消”或“关闭”按钮，则返回值均为0。

例 23.3 颜色设置对话框

```
C = uisetcolor([1,1,1], '颜色设置'); % 结果如图 23.6
```

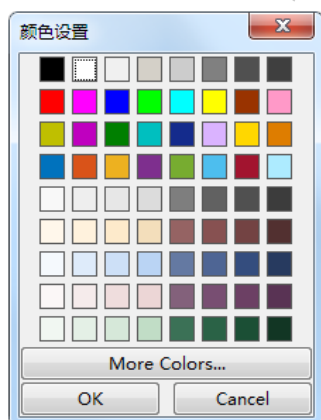


图 23.6: 颜色设置对话框

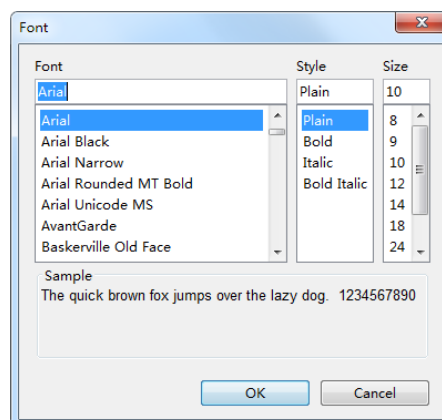


图 23.7: 字体设置对话框

23.2.4 字体设置对话框

字体设置对话框用于字体属性的交互式设置。

```
S = uisetfont
S = uisetfont(S)
S = uisetfont(H)
S = uisetfont(..., DialogTitle)
```

注 23.4 参数意义为：输入参数 S 指定默认的字体属性结构体，该结构体包含 $FontName$ 、 $FontUnits$ 、 $FontSize$ 、 $FontWeight$ 、 $FontAngle$ 域名，它完整地表示了字体的名称、单位、大小、粗细、形状等属性， H 指定对象句柄，即对对象 H 中的字体设置属性， $DialogTitle$ 指定对话框标题，输出参数 S 也是字体属性结构体。如果点击“取消”或“关闭”按钮，则返回值均为0。

例 23.4 字体设置对话框

```
S = uisetfont; % 结果如图 23.7
```

23.2.5 信息对话框

信息对话框用于显示提示信息。

```
msgbox(Message)
msgbox(Message, Title)
msgbox(Message, Title, Icon)
msgbox(Message, Title, 'custom', IconData, IconCMap)
msgbox(Message, ..., CreateMode)
h = msgbox(...)
```

注 23.5 输入参数 $Message$ 指定信息内容， $Title$ 指定对话框窗口标题， $Icon$ 指定对话框的图标，有效值为'none'(无图标，默认)、'error'(错误图标)、'help'(帮助图标)、'warn'(警告图标)或'custom'(自定义图标，图标由数据 $IconData$ 和色图 $IconCMap$ 指定)。 $CreateMode$ 指定对话框的窗口模式。输出参数 h 返回对话框句柄。

例 23.5 信息对话框

```
msgbox('%%%%%%%%%%%%%%海大欢迎您! ', '欢迎') % 结果如图 23.8
```

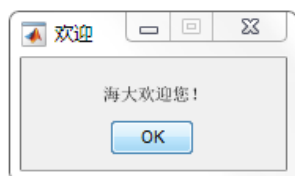


图 23.8: 信息对话框

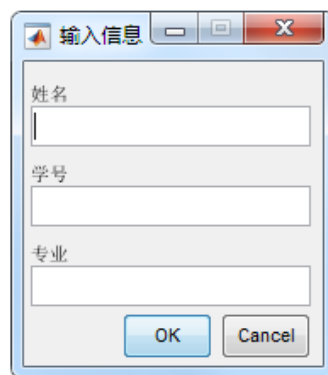


图 23.9: 输入对话框

23.2.6 输入对话框

输入对话框用于获取输入的信息。

```
Answer = inputdlg(Prompt)
Answer = inputdlg(Prompt, Name)
Answer = inputdlg(Prompt, Name, NumLines)
Answer = inputdlg(Prompt, Name, NumLines, DefaultAnswer)
Answer = inputdlg(Prompt, Name, NumLines, DefaultAnswer, Options)
```

注 23.6 参数意义为：输入参数 *Prompt* 为单元数组，用于定义输入数据窗口的个数和显示提示信息，*Name* 确定对话框的标题。*NumLines* 可以是标量、列矢量或 $m \times 2$ 阶矩阵，若为标量，表示每个文本输入框的行数均为 *NumLines*，若为列向量，则每个文本输入框的行数由列向量 *NumLines* 的每个元素确定，若为矩阵，每个元素对应一个文本输入框，每行的第一列为输入窗口的行数，第二列为输入窗口的宽度。*DefaultAnswer* 为一个单元数组，存储每个输入数据的默认值，元素个数必须与 *Prompt* 所定义的输入窗口数相同，所有元素必须是字符串。*Options* 是一个结构体，它的 *Resize* 域决定输入对话框的大小能否被调整，可选值为 'on' 或 'off'，*WindowState* 域决定窗口模式，可选值为 'normal' 和 'modal'，*Interpreter* 域决定文字编译器，可选值为 'none' 和 'tex'。输出参数 *Answer* 是存储输入数据的单元数组，若点击“取消”或“关闭”按钮，*Answer* 返回值为空。

例 23.6 输入对话框

```
prompt = {'姓名'; '学号'; '专业'};
title = '输入信息';
answer = inputdlg(prompt, title); % 结果如图 23.9
```

23.2.7 帮助对话框

帮助对话框用于帮助提示信息。

```
H = helpdlg(HelpString, DlgName)
```

注 23.7 参数意义为：输入参数`HelpString`指定错误信息，`DlgName`指定对话框标题，输出参数`H`是对话框图形对象句柄。

例 23.7 帮助对话框

```
h = helpdlg('出门左拐，垃圾桶两步就到。', '帮助...') % 结果如图 23.10
```

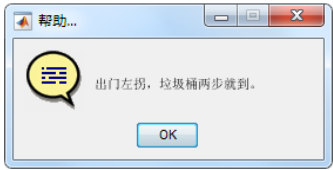


图 23.10: 帮助对话框



图 23.11: 提问对话框

23.2.8 提问对话框

提问对话框用于选择同一问题的不同回答。

```
ButtonName = questdlg(Question)
ButtonName = questdlg(Question, Title)
ButtonName = questdlg(Question, Title, Default)
ButtonName = questdlg(Question, Title, Btn1, Btn2, Default);
ButtonName = questdlg(Question, Title, Btn1, Btn2, Options);
```

注 23.8 输入参数`Question`指定问题内容，`Title`指定对话框窗口标题，`Btn1`和`Btn2`指定对话框中的按钮名称，若未指定，则使用‘Yes’、‘No’和‘Cancel’，`Default`指定默认选中的按钮，`Options`是结构体，但必须包含`Interpreter`和`Default`域。输出参数`ButtonName`返回选中的按钮名称，若点击“关闭”按钮，则返回空字符串。

例 23.8 提问对话框

```
ButtonName = questdlg('您喜欢MATLAB吗?', '请问', ...  
    '喜欢', '很喜欢', '很喜欢') % 结果如图 23.11
```

23.2.9 警告对话框

警告对话框用于提示警告信息。

```
H = warndlg(WarnString, DlgName)  
H = warndlg(WarnString, DlgName, CreatMode)
```

注 23.9 输入参数 *WarnString* 指定警告内容, *DlgName* 指定对话框窗口标题, *CreatMode* 指定窗口模式。 *H* 返回对话框句柄。

例 23.9 警告对话框

```
warndlg('uuuuuuu自习室禁止喧哗!', '警告') % 结果如图 23.12
```

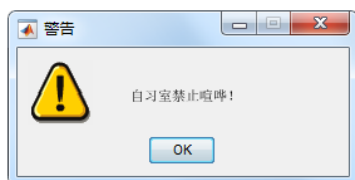


图 23.12: 警告对话框



图 23.13: 错误对话框

23.2.10 错误对话框

错误对话框用于提示错误信息。

```
H = errordlg(ErrorString, DlgName)  
H = errordlg(ErrorString, DlgName, CreatMode)
```

注 23.10 参数意义为: 输入参数 *ErrorString* 指定错误信息, *DlgName* 指定对话框标题, *CreatMode* 指定对话框窗口模式, 其有效值为 '*modal*' (模式窗口), '*non-modal*' (非模式窗口) 和 '*replace*' (替换窗口), 输出参数 *H* 是对话框图形对象句柄。

例 23.10 错误对话框

```
H = errorDlg('禁止吸烟!', '提示...') % 结果如图 23.13
```

23.2.11 列表选择对话框

列表选择对话框用于在多个选项中选择需要的值。

```
[Selection, OK] = listdlg(PropertyName, PropertyValue)
```

注 23.11 输入参数及意义详见表 23.14。点击“OK”按钮时，*Selection*返回选项的索引值，*OK*返回值为1，点击“Cancel”按钮或“关闭”按钮时，*OK*值为0，*Selection*为空。若当前操作模式为多选模式时，即*SelectionMode*值为*multiple*，则显示“Select all”按钮。

表 23.3: 列表选择对话框的输入参数

参数	描述
ListString	定义列表选项的字符串或字符串数组
SelectionMode	设置选取方式为单选(single)还是多选(multiple)
ListSize	定义列表框的大小，格式为[宽，高]，单位为像素
InitialValue	设置对话框创建时选择的项，默认值为1，表示默认选取第1个选项
Name	列表对话框的标题
PromptString	列表提示语，为字符串或字符串单元数组
OKString	定义“OK”按钮的文字
CancelString	定义“Cancel”按钮的文字
uh	定义按钮的高，单位为像素，默认值为18
fus	定义框架与内部UI对象间的空间大小，单位为像素，默认值为8
ffs	定义Figure窗口与内部框架间的空间大小，单位为像素，默认值为8

例 23.11 列表选择对话框

```
[sel, ok] = listdlg(...
    'ListString', {'2015', '2016', '2017', '2018'}, ...
    'Name', '选择年级', ...
    'OKString', '确定', ...
    'CancelString', '取消', ...
    'SelectionMode', 'single', ...
    'ListSize', [150,80]); % 结果如图 23.14
```

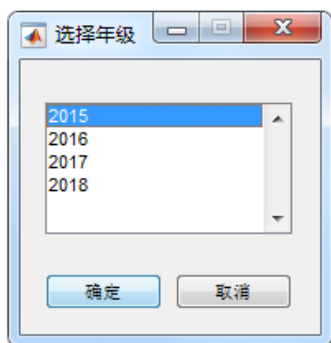


图 23.14: 列表选择对话框

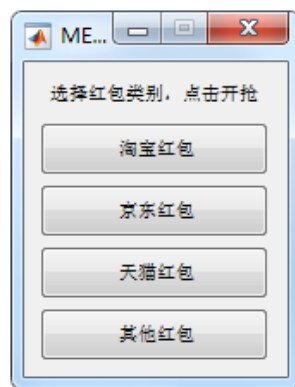


图 23.15: 菜单选择对话框

23.2.12 菜单选择对话框

创建菜单选择对话框可以使用menu函数，调用格式如下：

```
Choice = menu(Header, Item1, Item2, ... )
Choice = menu(Header, ItemList)
```

注 23.12 输入参数Header指定对话框标题，Item1、Item2等确定菜单选项，或者使用字符串数组ItemList确定菜单选项列表。Choice返回选中菜单选项对应的索引值。

例 23.12 菜单选择对话框

```
choice = menu('选择红包类别, 点击开抢', ...
    '淘宝红包', ...
    '京东红包', ...
    '天猫红包', ...
    '其他红包') % 结果如图 23.15
```

23.2.13 目录选择对话框

创建目录选择对话框可以使用uigetdir函数，调用格式如下：

```
DirName = uigetdir
DirName = uigetdir(StartPath, Title)
DirName = uigetdir(StartPath, Title, X, Y)
```

注 23.13 输入参数 $StartPath$ 指定起始目录， $Title$ 确定对话框标题， X ， Y 指定对话框的坐标为 $[x, y]$ (单位为像素，屏幕左下角坐标为 $[0, 0]$)。 $DirName$ 返回选中的目录，若单击“取消”或“关闭”按钮， $DirName$ 返回值为0。

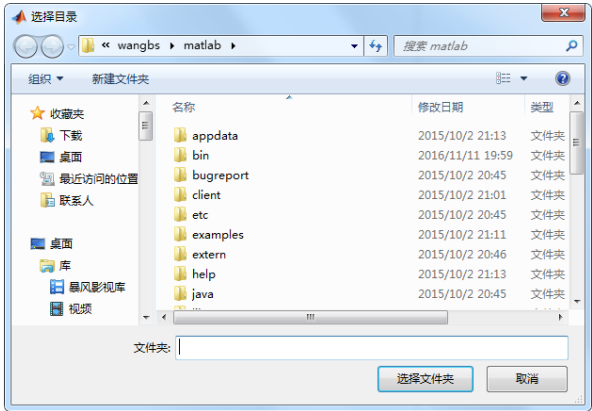


图 23.16: 目录选择对话框

例 23.13 目录选择对话框

```
dirname = uigetdir(matlabroot, '选择目录') % 开始地址为MATLAB安装目录，结果如图 23.16
```

23.2.14 进度条

进度条以图形方式显示运算或处理的进程，它可使用uigetdir函数创建，调用格式如下：

```
waitbar(x, H)
waitbar(x, Title)
waitbar(x, Title, 'CreatCancelBtn', button_callback)
H = waitbar(..., PropertyName, PropertyValue)
```

注 23.14 输入参数 x 指定进程条的比例长度，其值必须在0和1之间， H 为进度条的图形句柄，表示更新进度条 h 的进度， $Title$ 确定进度条标题，'CreatCancelBtn'创建一个“撤销”按钮，回调函数由 $button_callback$ 指定，同时还可以设置其他属性。 H 返回进度条的图形句柄。

例 23.14 进度条


```

h = waitbar(0, '请耐心等待...', 'Name', '加载中'); % 结果如图 23.17
for i = 1:100
    if i < 90
        waitbar(i/100, h, ['载入完成', num2str(i), '%']);
    else
        waitbar(i/100, h, '载入即将完成! ');
    end
    pause(0.1);
end
delete(h);

```

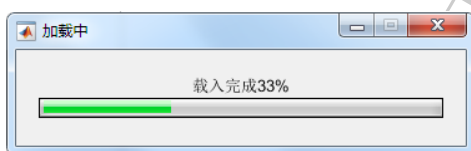


图 23.17: 进度条

23.3 QQ登录框的设计

MATLAB在产生对象时都为所建立的每个对象返回一个句柄(或句柄的列向量), 比如, 我们在使用plot、mesh绘制图形时也会返回一个句柄, 该句柄是一个结构体, 它包含当前绘制的图形界面内的所有对象及其属性或数据, 可以用get、set函数进行获取、修改, 也可以通过专门的函数进行修改。在GUI设计中经常会遇到hObject、eventdata、handles等参数, 其中hObject是指当前操作的对象, eventdata是指鼠标、键盘等键入的键值信息, handles是一个结构体, 它包含当前GUI界面的所有对象及其属性或数据, 也是GUI设计经常使用的参数, 可以用get、set函数进行获取、修改。

例 23.15 利用图 23.18, 设计如图 23.19 所示的QQ密码登录框。

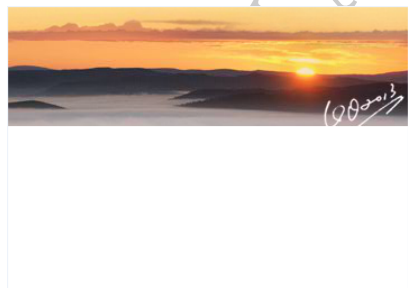


图 23.18: 背景图片



图 23.19: 界面预览

打开GUIDE, 界面设计具体如下(未列出的属性均采用默认值)。

1. 双击布局区打开Figure的属性查看器并设置如下：

属性	属性值
Name	QQLoginer
Position	[100.0, 50.0, 80.0, 20.0]

2. 在界面上添加坐标轴(Axes)，用于加载背景界面

属性	属性值	属性	属性值
Position	[0, 0, 1, 1]	Units	normalized
Tag	axes	X(Y)Tick	[]

3. 在Figure上添加两个静态文本框(Text)，用于显示“用户名”和“密码”

属性	用户名静态文本属性值	密码静态文本属性值
BackgroundColor	[1.0,1.0,1.0]或白色	[1.0,1.0,1.0]或白色
FontSize	12.0	12.0
Position	[20.0,8.25,12.0,1.5]	[14.0,5.25,12.0,1.5]
String	空字符串	空字符串
Tag	text_Name	text_Pwd

4. 在Figure上添加两个文本编辑框(Edit)，用于输入用户名和密码

属性	用户名文本编辑框属性值	密码文本编辑框属性值
FontSize	10.0	10.0
HorizontalAlignment	left	left
Position	[32.4,8.2,26.0,2.0]	[32.4,5.2,26.0,2.0]
String	空字符串	空字符串
Tag	edit_Name	edit_Pwd

5. 在Figure上添加两个按钮(Pushbutton)，用于登录和退出

属性	“登录”按钮属性值	“退出”按钮属性值
FontSize	10.0	10.0
Position	[24.0,2.0,15.0,2.3]	[40.0,2.0,15.0,2.3]
String	空字符串	空字符串
Tag	Login	exit

将设计好的文件保存为QQLoginer.fig，这时系统会产生一个同名的M文件并打开，之后的任务就是将预想的功能通过编写QQLoginer.m的子函数逐一实现，这里我们要实现主要功能有：(1) 在启动界面时，利用创建的Axes对象，设置界面背景；

(2) 为保证密码输入安全，用户在密码输入框输入一个有效字符(字母、数字、下划线)后，系统将在密码输入框显示“*”号，类似于QQ密码的输入。

(3) 用户点击“登录”按钮后，检查输入信息是否完整，检查输入信息是否在数据库中(数据文件为userinfo.xlsx，在界面启动时读取)，当用户名和密码匹配时，登录成功。

(4) 静态文本的背景与坐标轴对象的背景一致。

(5) 该程序可以作为其他大型程序的登录界面使用，然而还有不少地方需要改进，比如用户的注册、密码修改和找回等功能，密码信息的安全问题需要讨论。

我们逐一来实现上述功能。

1. 主函数

```
function varargout = QQLoginer(varargin)
% 注释部分从略
gui_Singleton = 1;
gui_State = struct('gui_Name',       mfilename, ...
    'gui_Singleton',   gui_Singleton, ...
    'gui_OpeningFcn', @QQLoginer_OpeningFcn, ...
    'gui_OutputFcn',   @QQLoginer_OutputFcn, ...
    'gui_LayoutFcn',   [] , ...
    'gui_Callback',    []);
if nargin && ischar(varargin{1})
    gui_State.gui_Callback = str2func(varargin{1});
end

if nargout
    [varargout{1:nargout}] = gui_mainfcn(gui_State, varargin{:});
else
    gui_mainfcn(gui_State, varargin{:});
end
```

这是GUIDE自动生成的主函数，其中包含的结构体是程序运行的关键。一般来说，没有特殊要求时，不要修改该程序。

注 23.15 GUIDE在自动生成代码时会生成3-5行的注释以提示函数的输入输出参数。考虑到注释会占用太多篇幅，我们将省略下面函数的注释。

2. 界面打开函数

```
function QQLoginer_OpeningFcn(hObject, eventdata, handles, varargin)
handles.output = hObject;
if exist('userinfo.xlsx', 'file') % 读取用户名和密码信息
    [~, txt] = xlsread('userinfo.xlsx', 'Sheet1');
```

```

        handles.data = txt;
    else
        handles.data = []; % 若数据文件不存在, 数据置为空
    end
    axes(handles.axes) % 读取背景图片, 并处理
    try
        a = imread('pic.bmp');
        index = find(a(:,1) == 255, 1, 'first');
        b = zeros(1,3);
        for i = 1:3
            b(i) = ceil(mean(a(index-1, :, i)));
            a(index:end,:,i) = repmat(b(i), size(a,1)-index+1, size(a,2));
        end
        imshow(a); % 展示背景图片
        c = b/255; % 颜色数据归一化
    catch
        c = [0.94, 0.94, 0.94]; % 若图片不存在, 控件的背景颜色设为界面默认值
    end
    axis off
    set(handles.text_Name, 'BackgroundColor', c); % 设置用户名静态文本框背景
    set(handles.text_Pwd, 'BackgroundColor', c); % 设置密码静态文本框背景
    guidata(hObject, handles); % 更新界面数据

```

3. 界面退出函数

```

function varargout = QQLoginer_OutputFcn(hObject, eventdata, handles)
varargout{1} = handles.output; % 界面的输出参数, 这里不需要输出

```

4. 文本编辑框创建时执行的函数

```

function edit_Name_CreateFcn(hObject, eventdata, handles)
if ispc && isequal(get(hObject,'BackgroundColor'), ...
    get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white'); % 设置用户名输入框背景
end

function edit_Pwd_CreateFcn(hObject, eventdata, handles)
if ispc && isequal(get(hObject,'BackgroundColor'), ...
    get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white'); % 设置密码输入框背景
end

```

5. 按键响应函数

```

function edit_Pwd_KeyPressFcn(hObject, eventdata, handles)
c = eventdata.Character; % 获取按下的字符
if isstrprop(c, 'graphic') % 若是图形字符
    set(hObject,'UserData', [get(hObject, 'UserData') c]) % 将c加入到UserData中, 表示密码
    set(hObject,'string', [get(hObject, 'string'), c]); % 将c显示在密码输入框

```

```

end

function edit_Pwd_KeyReleaseFcn(hObject, eventdata, handles)
c = eventdata.Character;
ch = get(hObject, 'UserData');
if isstrprop(c, 'graphic')
    pause(0.1)
    set(hObject, 'string', char(42*ones(size(ch))));
end
if double(c) == 8 % 若是删除键
    set(hObject, 'UserData', ch(1:end-1), ... % 删除UserData最后一个字符
        'string', char(42*ones(size(ch(1:end-1))))); % 删除显示的最后字符
end
end

```

6. 按钮回调函数

```

function Login_Callback(hObject, eventdata, handles)
Pwd = get(handles.edit_Pwd, 'UserData'); % 获取密码
User = get(handles.edit_Name, 'String'); % 获取用户名
if isempty(Pwd) || isempty(User) % 若两者之一为空
    uiwait(warndlg('用户名或密码为空!', '警告')); % 弹出警告对话框
else
    uq = strcmpi(User, handles.data(1:end,1)); % 匹配用户名
    if all(uq) == 0 % 若用户名不匹配
        uiwait(errordlg('用户名不存在!', '错误')); % 弹出错误对话框
    else
        uqPwd = handles.data(uq,2); % 正确密码
        if strcmp(Pwd, uqPwd) % 匹配密码是否一致
            uiwait(msgbox('登录成功!', '提示')); % 登陆成功
            % =====
            % 这里添加登陆后的主程序
            % =====
        else
            set(handles.edit_Pwd, 'string', '', 'UserData', ''); % 密码输入框和UserData置为空
            uiwait(errordlg('密码错误!', '错误')); % 弹出错误对话框
        end
    end
end
end

function exit_Callback(hObject, eventdata, handles)
select = questdlg('您点击了退出按钮,您是想:', ...
    '提示', '返回程序', '退出程序', '返回程序'); % 弹出提问对话框
switch select
case '返回程序'
    return % 返回程序
case '退出程序'
    clear; close all; close(gcf); % 清除变量、关闭界面并退出
end
end

```

这样整个GUI设计基本上完成,最后,还要进行程序的调试,这可是个辛苦活。我们可以按功能逐一进行调试,也可以按函数进行。为了避免复杂的程序调试,编写程序时应力图程序简洁、高效,避免程序出错退出或者出现不想要的结果。

我的 MATLAB 报告

王保山

第二十四章 有趣的程序

例 24.1 用MATLAB模拟游戏中的倒数、准备和开始。

```
sp = actxserver('SAPI.SpVoice');
poc = get(0,'ScreenSize'); % 获取屏幕尺寸
figure('Position', [poc(3)/2-120, poc(4)/2-120, 240, 240]) % 界面居中
for i = 5:-1:1
    cla reset % 清除当前轴
    axis off
    text(0.5, 0.5, ['$$', num2str(i), '$$'], 'FontSize', 90, ...
        'interpreter', 'latex', 'HorizontalAlignment', 'center')
    pause(0.1)
    sp.Speak(num2str(i)); % 倒数54321
end % 结果如图 24.1
pause(0.1)
sp.Speak('预备');
close gcf
sp.Speak('开始!');
```

2

图 24.1: 倒数五个数

例 24.2 用MATLAB播放国歌。

方案(1)

```

clear all
fs = 48000; % 频率
dt = 1 / fs; % 周期
f0 = 320;
T16 = 0.125;
t16 = 0:dt:T16;
m = size(t16, 2);
t2 = linspace(0, 8*T16, 8*m);
t4 = linspace(0, 4*T16, 4*m);
t8 = linspace(0, 2*T16, 2*m);
t12 = linspace(0, 4/3*T16, 4/3*m);
index = {'2', '4', '8', '12', '16'};
for n = 1:5
    str = ['mod',char(index(n)), '_sin(pi*t',char(index(n)),...
           '/t',char(index(n)),'(end));'];
    eval(str);
end
ScaleTable = [1/2, 9/16, 5/8, 2/3, 3/4, 5/6, 15/16, ...
              1, 9/8, 5/4, 4/3, 3/2, 5/3, 9/5, 15/8, ...
              2, 9/4, 5/2, 8/3, 3, 10/3, 15/4, 4];
txt = {'do', 're', 'mi', 'fa', 'so', 'la', 'si'};
nstr = {'0', '1', '2'};
mstr = {'g', 'f', 'e', 'd', 's'};
for nn = 1:5
    for jj = 1:3
        for ii = 1:7
            mm = 7 * (jj - 1) + ii;
            str = [char(txt(ii)), char(nstr(jj)), char(mstr{nn}), ...
                  '_mod', char(index(nn)), ...
                  '.*cos(2*pi*ScaleTable(', num2str(mm), ')*f0*t', ...
                  char(index(nn)), ');'];
            eval(str);
        end
    end
end
g = zeros(1, size(t2, 2));
f = zeros(1, size(t4, 2));
e = zeros(1, size(t8, 2));
d = zeros(1, size(t12, 2));
s = zeros(1, size(t16, 2));
save matlabsongs1.mat * % 存储音节数据到matlabsongs1.mat, 以后使用时直接load即可
%%
yuepu = [dole, s, m1s, so1e, so1e, ...
         la1f, so1f, ...
         m1e, s, do1s, so1d, so1d, so1d, ...
         m1f, do1f, ...
         so0d, so0d, so0d, so0d, so0d, so0d, ...
         do1f, e, so0e, ...
         do1f, e, do1e, ...
         do1e, s, do1s, so0e, la0s, si0s, ...
         do1f, do1f, ...
         e, m1e, do1e, re1s, m1s, ...
         so1f, so1f, ...
         m1e, s, m1s, do1e, s, m1s, ...
         so1e, s, m1s, re1f, ...
         re1g, ...
         la1f, so1f, ...
         re1f, m1f, ...

```



```

so1e, mi1e, e, so1e, ...
mi1e, re1s, mi1s, do1f, ...
mi1f, f, ...
so0e, s, la0s, do1e, do1e, ...
mi1e, s, mi1s, so1e, so1e, ...
re1e, re1s, re1s, la0f, ...
re1f, e, so0e, ...
do1f, e, do1e, ...
mi1f, e, mi1e, ...
so1g, ...
do1e, s, mi1s, so1e, so1e, ...
la1f, so1f, ...
mi1e, s, do1s, so1d, so1d, so1d, ...
mi1e, e, do1e, e, ...
so0f, do1f, ...
mi1e, s, do1s, so1d, so1d, so1d, ...
mi1e, e, do1e, e, ...
so0f, do1f, s, ...
so0f, do1f, s, ...
so0f, do1f, s, ...
do1f, e, e];

```

```
s = yuepu;
```

```
s = s/max(s);
```

```
sound(s, fs);
```

```
%f
```

yuepu数组内变量的命名规则：“音阶+强度+音符”形式

音阶字符串如下：

音阶	1	2	3	4	5	6	7
字符串	do	re	mi	fa	so	la	si

强度字符串如下：

强度	低音	正常	高音
字符串	0	1	2

音符字符串如下：

音符	二分音符	四分音符	八分音符	十二分音符	十六分音符
字符串	g	f	e	d	s

例如：2/4拍子中

	5	6	1	2	3		
	-----		-----				可以表示为
	.		----				

yuepu = [so0e la1e do1e re1s mi1s]

【另请注意】

1. 关于休止符0

这里提供了用于表示休止符(各分音符)的数组，其中g, f, e, d, s分别是全零的数组

音符	二分音符	四分音符	八分音符	十二分音符	十六分音符
数组	g	f	e	d	s

例如：2/4拍子中

```
| 0 6 1 0 3 |
| ----- | 可以表示为
| ----- |
|         |
yuepu = [e lale dole s miis]
```

2. 关于附点音符

目前用零向量来表示，与休止符类似，附点音符分类如下

音符	二分音符	四分音符	八分音符
数组	f	e	s

例如：2/4拍子中

```
| 1. 6 1 2 3 |
| ----- | 可以表示为
| ----- |
|         |
yuepu = [so0e s lale dole re1s miis]
```

```
%}
```

另一种播放方案(方案(2))

```
clear all
fs = 48000; % 频率
dt = 1/fs; % 周期
f0 = 320;
T32 = 0.0625;
t32 = 0:dt:T32;
m = length(t32);
t1 = linspace(0, 32*T32, 32*m);
t2 = linspace(0, 16*T32, 16*m);
t4 = linspace(0, 8*T32, 8*m);
t8 = linspace(0, 4*T32, 4*m);
t12 = linspace(0, 8/3*T32, 8/3*m);
t16 = linspace(0, 2*T32, 2*m);
t2b = linspace(0, 16*T32, 24*m);
t4b = linspace(0, 8*T32, 12*m);
t8b = linspace(0, 4*T32, 6*m);
t16b = linspace(0, 2*T32, 3*m);
index = {'1'; '2'; '4'; '8'; '12'; '16'; '32'; '2b'; '4b'; '8b'; '16b'};
for n = 1:11
    str = ['mod', char(index(n)), ' = sin(pi*t', char(index(n)), ...
        '/t', char(index(n)), '(end));'];
    eval(str);
end
ScaleTable = [1, 9/8, 5/4, 4/3, 3/2, 5/3, 9/5, 0];
txt = {'1', '2', '3', '4', '5', '6', '7', '0'};
nstr = {'0', '1', '2', '3', '4'};
mstr = {'a1', 'a2', 'a4', 'a8', 'a12', 'a16', 'a32', 'b2', 'b4', 'b8', 'b16'};
for n = 1:11
    for j = 1:5
        for i = 1:8
            str = [char(mstr{n}), char(nstr{j}), char(txt{i}), ...
                ' = mod', char(index(n)), ...
                '.*cos(2*pi*2^(j-3)*ScaleTable(', num2str(i), ...
```

```

        ')*f0*t', char(index(n)), ');'];
    eval(str);
end
end
end
save matlabsongs2.mat a* b* % 存储音节数据到matlabsongs.mat, 以后使用时直接load即可
%% 产生乐谱数据并播放
yuepu = [b821, a1623, a825, a825, ...
    a426, a425, ...
    b823, a1621, a1225, a1225, a1225, ...
    a423, a421, ...
    a1215, a1215, a1215, a1215, a1215, a1215, ...
    a421, a820, a815, ...
    b421, a821, ...
    b821, a1621, a815, a1616, a1617, ...
    a421, a421, ...
    a820, a823, a821, a1622, a1623, ...
    a425, a425, ...
    b823, a1623, b821, a1623, ...
    b825, a1623, a422, ...
    a222, ...
    a426, a425, ...
    a422, a423, ...
    a825, a823, a820, a825, ...
    a823, a1622, a1623, a421, ...
    a423, a420, ...
    a815, a1616, a821, a821, ...
    b823, a1623, a825, a825, ...
    a822, a1622, a1622, a416, ...
    b422, a815, ...
    b421, a821, ...
    b423, a823, ...
    a225, ...
    b821, a1623, a825, a825, ...
    a426, a425, ...
    b823, a1621, a1225, a1225, a1225, ...
    a823, a820, a821, a820, ...
    a415, a421, ...
    b823, a1621, a1225, a1225, a1225, ...
    a823, a820, a821, a820, ...
    a415, a421, ...
    a415, a421, ...
    a415, a421, ...
    a421];
s = yuepu;
s = s/max(s);
sound(s, fs);
%{
yuepu数组内变量的命名规则: “音符+强度+音阶”形式

```

音阶字符串如下:

音阶	1	2	3	4	5	6	7	0(休止符)
字符串	1	2	3	4	5	6	7	0

强度字符串如下:

强度	倍低音	低音	正常	高音	倍高音
字符串	0	1	2	3	4

音符字符串(不含附点音符)如下:

音符	全音符	二分音符	四分音符	八分音符	十二分音符	十六分音符	三十二分音符
字符串	a1	a2	a4	a8	a12	a16	a32

音符字符串(含附点音符)如下:

音符	二分音符	四分音符	八分音符	十六分音符
字符串	b2	b4	b8	b16

例如: 2/4拍子中

	5	6	1	2	3	
	-----		-----			
	.			-----		

可以表示为

yuepu = [a815 a826 a821 a1622 a1623]

【另请注意】

1. 关于休止符0

例如: 2/4拍子中

	0	6	1	0	3	
	-----		-----			

可以表示为

yuepu = [a820 a826 a821 a1620 a1623]

2. 关于附点音符

例如: 2/4拍子中

	1.	6	1	2	3	
	-----		-----			
	-		-----			

可以表示为

yuepu = [b821 a1626 a821 a1622 a1623]

%}

注 24.1 这里仅提供了国歌的 *yuepu* 数组, 有兴趣的读者可以参照上述说明, 将某一歌曲(2/4拍子或4/4拍子)的简谱转化为 *yuepu*, 就可享受 MATLAB 给您带来的音乐魅力了。

例 24.3 用 MATLAB 上网搜索信息。

```
function websearch(str)
% 用MATLAB自带的网页浏览器进行百度搜索
% Input -str 待搜索信息
str = ['www.baidu.com/s?wd=', str]; % 百度搜索的URL
web(str) % 网页浏览
```

搜索“中国海洋大学在哪儿”。

```
websearch('中国海洋大学在哪儿') % 结果如图 24.2
```

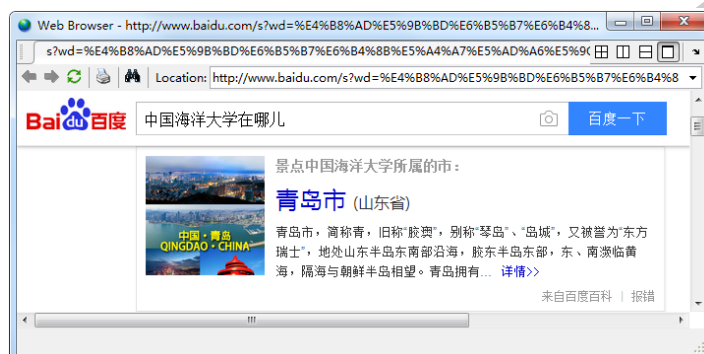


图 24.2: 网页搜索

例 24.4 用 MATLAB 发送电子邮件

(1) 设置邮箱

```
function mflag = setMail
address = 'mmatlab@sina.com'; % 输入邮箱地址
password = '*****'; % 输入邮箱密码，为信息安全已隐藏密码
mailsmtp = 'smtp.sina.com'; % 输入邮箱SMTP服务器地址
setpref('Internet','E_mail', address);
setpref('Internet','SMTP_Server', mailsmtp);
setpref('Internet','SMTP_Username', address);
setpref('Internet','SMTP_Password', password);
props = java.lang.System.getProperties();
props.setProperty('mail.smtp.auth', 'true');
props.setProperty('mail.smtp.socketFactory.class', 'javax.net.SSLSocketFactory');
props.setProperty('mail.smtp.socketFactory.port', '25');
mflag = 1; % mflag = 1提示邮箱设置成功
```

(2) 利用sendmail函数发送邮件

```
sendmail(To, Subject, Message, Attachments) % To为发送邮件目标邮箱，Subject表示邮件的主题，
% Message为邮件信息，它可以是一个字符串或元胞数组，如果是字符串，文字将在第75个字符处分行，
% 如果是元胞数组，每个元胞单独显示在一行中，其他情况可以使用char(10)分行，Attachments表示邮件
% 附件的本地路径列表。该函数只要求To和Subject，其他参数可缺省
```

注 24.2 这时读者可能会有疑问：邮箱密码容易被人看到，岂不是很不安全？

不用担心，MATLAB提供了一个加密函数pcode，它可以将M文件编译成p文件(加密文件)。由于经过一次编译，p文件比M文件的优先级要高，也就是说，当p文件与M文件同时存在时，MATLAB将

优先执行p文件。p文件保存了M文件的信息，提高了文件的执行效率，同时又不易被破解。但p文件也有缺点，它的保密性使得它不能被修改，在只有p文件的情况下，程序一旦出错将会成为很棘手的问题。建议待M文件调试完成后，再编译成p文件。编译如下：

```
pcode setMail
```

这是就会产生一个名为“setMail.p”的文件。如果有人想使用你的代码而你不想他知道你的代码，你就可以给他p文件。

例 24.5 用MATLAB发送QQ消息

```
function QQsentMsg(QQNum, Message, TimePerLine)
% 本程序可以实现用MATLAB自动发送QQ信息
% Input -QQNum 接收信息的QQ号码 -Message 信息，可以是元胞数组 -TimePerLine 发送的时间间隔
% Example
%   QQNum = 2947836879; TimePerLine = 5;
%   Message = {'在吗?', '吃饭去吗?', '老地方见!'};
%   QQsentMsg(QQNum, Message, TimePerLine)
vb = actxserver('wscript.shell');
loopnum = length(Message);
eval(['!start Tencent://message/?uin=', num2str(QQNum)]); % 打开QQ
pause(1); % 等待窗口打开

for msgline = 1:loopnum
    sentm(vb, char(Message(msgline)), TimePerLine);
end
disp('发送完毕!');
end

function sentm(vb, Message, TimePerLine)
pause(TimePerLine)
clipboard('copy', Message); % 复制信息
pause(0.1); % 暂停0.1秒
vb.SendKeys('^v'); % 粘贴
vb.SendKeys('%s'); % 发送
end
```

注 24.3 在使用MATLAB发送QQ消息时，请确保QQ版本不是太旧；确保输入的对方QQ号是自己的好友，否则会提示添加好友；该程序不能后台运行，使用过程中最好不要随意的切换窗口。

参考文献

- [1] <http://baike.baidu.com/item/MATLAB>
- [2] <http://cn.mathworks.com/>
- [3] <http://cn.mathworks.com/help/matlab/index.html>
- [4] <http://cn.mathworks.com/matlabcentral/fileexchange/37976-numerical-computing-with-matlab>
- [5] <http://cn.mathworks.com/matlabcentral/fileexchange/46124-2048-matlab-edition>
- [6] <http://cn.mathworks.com/matlabcentral/fileexchange/46483-2048-game-solver>
- [7] <http://www.ilovematlab.cn>
- [8] Randall J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*, University of Washington, Seattle, Washington, 2007.
- [9] John C. Strikwerda, *Finite Difference Schemes and Partial Differential Equations (Second Edition)*, SIAM, 2004.
- [10] Cleve B. Moler, *MATLAB之父：编程实践(英文版)*, 北京：北京航空航天大学出版社, 2013.12.
- [11] Cleve B. Moler, *MATLAB数值计算：2013修订版*●英文版, 北京：北京航空航天大学出版社, 2014.10.
- [12] Cleve B. Moler著, 薛定宇译, *MATLAB之父：编程实践*, 北京：北京航空航天大学出版社, 2014.3.
- [13] Cleve B. Moler著, 张志涌等译, *MATLAB数值计算(2013修订版)●中译本*, 北京：北京航空航天大学出版社, 2015.1.
- [14] Thomas H. Cormen, Charles E. Leiserson等著, 殷建平等译, *算法导论(原书第3版)*, 北京：机械工业出版社, 2013.1.
- [15] Thomas H. Cormen著, 王宏志译, *算法基础：打开算法之门*, 北京：机械工业出版社, 2015.12.
- [16] 何晓群, *多元统计分析(第四版)*, 北京：中国人民大学出版社, 2015.4.
- [17] 胡良剑, 孙晓君, *MATLAB数学实验*, 北京：高等教育出版社, 2006.6.

- [18] 胡晓冬, 董辰辉, *MATLAB从入门到精通*, 北京: 人民邮电出版社, 2010.6.
- [19] 李刚, *MATLAB函数速查手册*, 北京: 清华大学出版社, 2011.3.
- [20] 李荣华, 刘播, *微分方程数值解法(第四版)*, 北京: 高等教育出版社, 2009.1.
- [21] 刘浩, 韩晶, *MATLAB R2012a 完全自学一本通*, 北京: 电子工业出版社, 2013.1.
- [22] 罗华飞, *MATLAB GUI设计学习手记*, 北京: 北京航空航天大学出版社, 2009.2.
- [23] 罗华飞, *MATLAB GUI设计学习手记(第2版)*, 北京: 北京航空航天大学出版社, 2011.2.
- [24] 罗华飞, *MATLAB GUI设计学习手记(第3版)*, 北京: 北京航空航天大学出版社, 2014.
- [25] 马莉, *MATLAB数学实验与建模*, 北京: 清华大学出版社, 2010.1.
- [26] 谢中华, 李国栋, 刘焕进等, *MATLAB从零到进阶*, 北京: 北京航空航天大学出版社, 2012.12.
- [27] 徐萃薇, 孙绳武, *计算方法引论(第三版)*, 北京: 高等教育出版社, 2007.
- [28] 《运筹学》教材编写组, *运筹学(本科版)*, 北京: 清华大学出版社, 2005.9.

常用函数表

表 1: 基本数学函数

函数	功能	函数	功能
sin, sinh	正弦函数与双曲正弦函数	asin, asinh	反正弦函数与反双曲正弦函数
cos, cosh	余弦函数与双曲余弦函数	acos, acosh	反余弦函数与反双曲余弦函数
tan, tanh	正切函数与双曲正切函数	atan, atanh	反正切函数与反双曲正切函数
cot, coth	余切函数与双曲余切函数	acot, acoth	反余切函数与反双曲余切函数
sec, sech	正割函数与双曲正割函数	asec, asech	反正割函数与反双曲正割函数
csc, csch	余割函数与双曲余割函数	acsc, acsch	反余割函数与反双曲余割函数
atan2	四象限的反正切函数	abs	数值的绝对值与复数的幅值
exp	求以e为底的指数函数	sort	排序函数
log	求自然对数	log10	求常用对数
ceil	朝正无穷大方向取整	fix	向零方向取整
round	朝最近的方向取整	floor	朝负无穷大方向取整
rem	求余数	mod	求模数
nchoosek	二项式系数或所有的组合数	complex	创建复数
real	复数的实数部分	image	复数的虚数部分
angle	求复数的相角	conj	复数的共轭

表 2: 矩阵生成

函数	功能	函数	功能
zeros	生成零矩阵	ones	生成全1阵
eye	生成单位矩阵	blkdiag	产生输入元素为对角线的矩阵
rand	生成[0,1]上均匀分布随机矩阵	randn	生成标准正态分布随机矩阵
randperm	产生随机序列	cat	创建多维数组
linspace	生成线性等分向量	logspace	产生对数等分向量
compan	生成友矩阵	pascal	生成Pascal矩阵

续 表

函数	功能	函数	功能
hankel	生成Hankel方阵	toeplitz	生成托普利兹矩阵
hilb	生成Hilbert矩阵	invhilb	生成逆Hilbert矩阵
wilkinson	生成Wilkinson特征值测试阵		

表 3: 矩阵运算

函数	功能	函数	功能
dot	向量的点积	cross	向量×乘
conv	矩阵的卷积和多项式乘法	deconv	反褶积和多项式除法运算
kron	张量积	ismember	检测元素是否在集合中
setdiff	求两集合的差	intersect	求两个集合的交集
setxor	求两个集合交集的非(异或)	union	求两集合的并集
unique	取集合的单值元素	funm	方阵的函数运算
expm	方阵指数函数	logm	求矩阵的对数
sqrtn	矩阵的方根	polyvalm	求矩阵的多项式
det	求方阵的行列式	trace	矩阵的迹
inv	求矩阵的逆	pinv	求矩阵的伪逆矩阵
norm	求矩阵和向量的范数	cond	求矩阵的条件数
condest	1-范数的条件数估计	rcond	矩阵可逆的条件数估值
condeig	特征值的条件数	rank	矩阵的秩
diag	矩阵对角线元素的抽取	tril	下三角阵的抽取
triu	上三角阵的抽取	reshape	矩阵变维
rot90	矩阵旋转	flipdim	按指定维数翻转矩阵
fliplr	矩阵的左右翻转	flipud	矩阵的上下翻转
repmat	复制和平铺矩阵	rat	用有理数形式表示矩阵
rem	矩阵元素的余数	numel	确定矩阵元素个数

表 4: 矩阵分解

函数	功能	函数	功能
chol	Cholesky分解	orth	将矩阵正交规范化
lu	LU分解	qr	QR分解
qrdelete	从QR分解中删除列	qrinsert	从QR分解中添加列
schur	Schur分解	hess	海森伯格形式的分解
rsf2csf	实Schur分解转为复Schur分解	cdf2rdf	复对角矩阵转化为实对角矩阵
eig	特征值分解	svd	奇异值分解
gsvd	广义奇异值分解	qz	特征值问题的QZ分解

表 5: 线性方程组求解

函数	功能	函数	功能
null	求线性齐次方程组的通解	symmlq	线性方程组的LQ解法
bicg	双共轭梯度法解方程组	bicgstab	稳定双共轭梯度方法解方程组
cgs	复共轭梯度平方法解方程组	lsqr	共轭梯度的LSQR方法
qmr	广义最小残差法	minres	最小残差法解方程组
pcg	预处理共轭梯度方法	qmr	准最小残差法解方程组

表 6: 稀疏矩阵

函数	功能	函数	功能
sparse	创建稀疏矩阵	speye	单位稀疏矩阵
full	将稀疏矩阵转化为满矩阵	find	稀疏矩阵非零元素的索引
spconvert	外部数据转化为稀疏矩阵	spdiags	生成带状(对角)稀疏矩阵
sprand	稀疏均匀分布随机矩阵	sprandn	生成稀疏正态分布随机矩阵
sprandsym	稀疏对称随机矩阵	nnz	返回稀疏矩阵非零元素的个数
nonzeros	找到稀疏矩阵的非零元素	nzmax	稀疏矩阵非零元素的内存分配
spfun	稀疏矩阵的非零元素应用	spy	稀疏矩阵非零元素的分布图形
colmmd	稀疏矩阵的排序	colperm	非零元素的列变换
condest	稀疏矩阵的1-范数	normest	稀疏矩阵的2-范数估计值
dmperm	Dulmage-Mendelsohn分解	luinc	稀疏矩阵的分解
eigs	稀疏矩阵的特征值分解		

表 7: 插值与拟合

函数	功能	函数	功能
interp1	一维数据插值函数	interp2	二维数据内插值
interp3	三维数据插值	interp3	n维数据插值
spline	三次样条插值	interpft	用快速Fourier算法作一维插值
table1	一维查表函数	table2	二维查表

表 8: 数据分析

函数	功能	函数	功能
max	最大值函数	min	求最小值函数
mean	平均值计算	median	中位数计算
sum	求和	prod	连乘计算
cumsum	累积总和值	cumprod	累积连乘

表 9: 数值微积分

函数	功能	函数	功能
quad	一元函数的数值积分	trapz	用梯形法进行数值积分
rat, rats	有理数近似求取	fzero	求一元函数的零点
dblquad	计算矩形区域二元函数重积分	quad2dggen	任意区域二元函数的数值积分
diff	微分函数	int	积分函数
roots	求多项式的根	dsolve	求解常微分方程式
ode*	龙格-库塔法解微分方程		

表 10: 符号运算

函数	功能	函数	功能
size	符号矩阵的维数	compose	复合函数运算
colspace	返回列空间的基	symsum	符号表达式求和
real	求符号复数的实数部分	image	求符号复数的虚数部分
collect	合并同类项	expand	符号表达式展开
factor	符号因式分解	simplify	符号表达式的化简
numden	符号表达式的分子与分母	double	将符号矩阵转化为浮点型数值
solve	代数方程的符号解析解	finverse	函数的反函数
poly	求特征多项式	int	符号函数的积分
findsym	找出符号变量	limit	求极限
diff	符号函数导数求解	poly2sym	将多项式系数向量转化为符号多项式
dsolve	常微分方程的符号解		

表 11: 积分变换

函数	功能	函数	功能
fourier	Fourier积分变换	ifourier	逆Fourier积分变换
laplace	Laplace变换	ilaplace	逆Laplace变换
ztrans	z-变换	iztrans	逆z-变换
vpa	可变精度算法计算	subs	符号替换
taylor	符号函数的Taylor级数展开式	jacobian	求Jacobian矩阵
jordan	Jordan标准形	rsums	交互式计算Riemann和
latex	符号表达式的LaTex的表示式	syms	创建多个符号对象的快捷函数
maple	调用Maple内核	mfund	Maple数学函数的数值计算
mhelp	Maple函数帮助	sym2poly	符号多项式转化为数值多项式
ccode	符号表达式的C语言代码	fortran	符号表达式的Fortran语言代码

表 12: 概率统计

函数	功能	函数	功能
binornd	二项分布的随机数据的产生	normrnd	正态分布的随机数据的产生
random	通用函数求各分布的随机数据	pdf	通用函数计算概率密度函数值
binopdf	二项分布的密度函数	chi2pdf	求卡方分布的概率密度函数
ncx2pdf	求非中心卡方分布的密度函数	lognpdf	对数正态分布
fpdf	F分布的概率密度函数	ncfpdf	非中心F分布函数
tpdf	求T分布	gampdf	求 Γ 分布函数
nbinpdl	求负二项分布函数	exppdf	指数分布函数
raylpdf	瑞利分布	weibpdf	求韦伯分布
normpdf	正态分布的概率值	poisspdf	泊松分布的概率值
cdf	通用函数计算累积概率	binocdf	二项分布的累积概率值
normcdf	正态分布的累积概率值	icdf	计算逆累积分布函数
norminv	正态分布逆累积分布函数	sortrows	按行方式排序
mean	计算样本均值	var	求样本方差
std	求标准差	nanstd	忽略NaN计算的标准差
geomean	计算几何平均数	nanmean	忽略NaN元素计算算术平均值
median	计算中位数	nanmedian	忽略NaN计算中位数
harmmean	求调和平均数	range	求最大值与最小值之差
skewness	样本的偏斜度	unifstat	均匀分布的期望和方差
normstat	正态分布的期望和方差	binostat	二项分布的均值和方差
cov	协方差	corrcoef	相关系数

表 13: 参数估计

函数	功能	函数	功能
unifit	均匀分布的参数估计	normfit	正态分布的参数估计
binofit	二项分布的参数估计	betafit	计算 β 分布的参数估计
mle	指定分布的参数估计	expfit	指数分布的参数估计
gamfit	γ 分布参数的参数估计	weibfit	韦伯分布的参数估计
poissfit	泊松分布的估计值	normfit	正态分布的估计值
nlparci	非线性模型参数的置信区间	nlpredci	非线性模型置信区间预测
lsnonneg	非负最小二乘法	lsqnonneg	有非负限制的最小二乘法
nlinfit	非线性最小二乘拟合	nlintool	非线性拟合
betalike	负 β 分布的对数似然函数	gamlike	负 γ 分布的对数似然估计
normlike	负正态分布的对数似然函数	weiblike	威布尔分布的对数似然函数

表 14: 假设检验

函数	功能	函数	功能
ttest	t检验法	ztest	u检验法
ttest2	两个正态总体均值差的检验	ranksum	秩和检验
signrank	符号秩检验	signtest	符号检验
kstest	单个样本分布的Kolmogorov-Smirnov测试	jbtest	正态分布的拟合优度测试
		kstest2	两样本相同连续分布的检验

表 15: 图形绘制

函数	功能	函数	功能
lsline	最小二乘拟合直线	normplot	绘制正态分布概率图形
tabulate	正整数的频率表显示	capaplot	样本的概率图形
cdfplot	经验累积分布函数图形	weibplot	绘制威布尔(Weibull)概率图形
histfit	带有正态密度曲线的直方图	boxplot	样本数据的盒图
refline	给当前图形加一条参考线	refcurve	当前图形中加入多项式曲线
normspec	在界线之间画正态密度曲线	qqplot	绘制Q-Q图

表 16: 二维图形绘制

函数	功能	函数	功能
plot	基本平面图形函数	legend	加图例
text	添加字符串	subplot	分区绘图
grid, box	给坐标加网格和边框	figure	多图形窗口绘制
hold	图形保持	fplot	函数f(x)曲线

表 17: 三维图形绘制

函数	功能	函数	功能
plot3	绘制三维曲线	mesh	绘制三维网格图
surf	三维曲面图	contour3	三维等高线绘制
contour	曲面的等高线	clabel	等高线填标签
contourc	等高线图形计算	fill3	填充三维图
sphere	绘制球体	contourf	填充二维等高线
pie3	三维饼图	comet3	三维彗星图绘制
waterfall	瀑布图	cylinder	生成圆柱图形
surfc	绘制阴影图及等高线	surfl	带光照模式的曲面图

表 18: 特殊图形绘制

函数	功能	函数	功能
loglog	绘制双对数坐标图形	semilogx	单对数坐标
polar	绘制极坐标图	pie	画饼图
bar	二维垂直条形图	barh	二维水平条形图
stairs	阶梯图形	fill	填充图形
zoom	对图形缩放	meshgrid	生成数据点矩阵
compass	从原点画箭头图	comet	绘制二维彗星图
errorbar	绘制误差图	feather	画速度向量图
hist	二维条形直方图	rose	角度直方图
stem	画二维离散数据图	stem3	画三维离散数据图

表 19: 便捷函数绘图

函数	功能	函数	功能
ezplot	画符号函数的图形	ezplot3	三维曲线图
ezcontour	画符号函数的等高线图	ezcontourf	用不同颜色填充的等高线图
ezpolar	画极坐标图形	ezmesh	符号函数的三维网格图
ezmeshc	同时画曲面网格图与等高线图	ezsurf	三维带颜色的曲面图
ezsurf	同时画出曲面图与等高线图		

表 20: 图形展示

函数	功能	函数	功能
view	视点处理	colormap	获取当前色图
brighten	色图控制函数	colorbar	显示颜色条
contrast	提高灰色对比度	rgbplot	画出色图
shading	设置颜色色调	hidden	隐含线条的显示
line	创建线条	surface	生成面
light	光照处理		

表 21: 控制语句

函数	功能	函数	功能
input	数据的输入	disp	数据的输出
pause	程序的暂停	continue	继续运行
break	结束循环	return	正常退出
keyboard	停止文件执行并转交控制	error, warning	出错与警告命令

表 22: 界面设计

函数	功能	函数	功能
get	获得对象属性	set	设置对象属性
gcf	返回当前图形句柄	gca	返回当前坐标轴句柄
figure	图形窗口的建立	axes	创建坐标轴
uimenu	自制用户菜单的创建	helpdlg	帮助窗口对话框
errordlg	错误窗口对话框	warndlg	警告对话框
uisetcolor	颜色设置对话框	questdlg	提问对话框设计
msgbox	消息框设计	uicontrol	控件编写